

算法设计与分析

董洪伟

2026-05-18

Table of contents

前言	1
本书内容编排	1
本书特色	1
读者对象	2
资源与联系方式	2
关于作者	3
1 算法设计与分析基础	5
1.1 什么是算法?	5
1.1.1 算法的定义	5
1.1.2 算法与程序的区别	7
1.1.3 算法解决问题的过程	7
1.1.4 算法有什么用?	7
1.2 算法的设计	8
1.2.1 算法设计策略	8
1.2.2 算法的表示与实现	16
1.3 算法的分析	26
1.3.1 正确性分析	26
1.3.2 效率分析	29
1.4 渐近分析	33
1.4.1 渐近分析的基本思想	34
1.4.2 渐近记号	35
1.4.3 渐近记号的性质	39
1.4.4 常见时间复杂度类别	40
1.4.5 常见时间复杂度函数的典型例子	41
1.4.6 渐近分析的局限性	43
1.4.7 问题复杂性类简介(拓展)	44
1.4.8 小结	44
1.5 递归方程的求解	45
1.5.1 迭代展开法	45
1.5.2 递归树法	45
1.5.3 换元迭代法	49
1.5.4 假设归纳法	49
1.5.5 高阶方程的化简	52
1.5.6 主定理(简化版)	53
1.5.7 一般形式的主定理及其证明	55
1.5.8 小结	57
1.6 习题	57

1.6.1	选择题	57
1.6.2	判断题	57
1.6.3	填空题	58
1.6.4	简答题	58
1.6.5	算法表示练习	58
1.6.6	客观编程题	59
1.7	附录：本书伪代码风格规范	59
2	线性穷举	63
2.1	线性穷举的基本思想	63
2.2	简单迭代	63
2.2.1	数组的简单迭代	63
2.2.2	链表的简单迭代	65
2.2.3	线性查找	66
2.2.4	删除有序链表中的重复元素	68
2.3	嵌套迭代	69
2.3.1	二维数组遍历	69
2.3.2	两数之和（暴力解法）	70
2.3.3	百钱买百鸡	71
2.3.4	字符串匹配（朴素字符串匹配算法）	73
2.3.5	子集枚举与背包问题	75
2.3.6	简单排序算法：冒泡排序	81
2.3.7	简单排序算法：冒泡排序（面向初学者的通俗描述）	81
2.4	多表迭代	82
2.4.1	合并两个有序数组	82
2.4.2	合并两个有序链表	84
2.5	双指针迭代	86
2.5.1	左右指针（相向而行）	87
2.5.2	左右指针（背向而行）	92
2.5.3	快慢指针	97
2.6	滑动窗口	106
2.7	小结	113
2.8	习题	113
2.8.1	选择题	113
2.8.2	判断题	114
2.8.3	填空题	114
2.8.4	简答题	114
2.8.5	客观编程题（共 10 题）	115
3	贪心法	121
3.1	贪心法的基本思想	121
3.1.1	贪心法的基本框架	121
3.1.2	贪心法的适用条件	121

3.2	找零钱问题 (Coin Change)	122
3.2.1	问题描述	122
3.2.2	贪心策略	122
3.2.3	算法实现	122
3.2.4	贪心法的最优性分析	125
3.2.5	贪心法失效的反例	126
3.2.6	思考题	126
3.3	区间调度问题 (Interval Scheduling)	126
3.3.1	问题描述	126
3.3.2	可能的贪心策略	127
3.3.3	贪心算法: 最早结束时间优先	128
3.3.4	算法实现	128
3.3.5	正确性证明	130
3.3.6	思考题	130
3.4	分数背包问题 (Fractional Knapsack)	131
3.4.1	问题描述	131
3.4.2	贪心策略	131
3.4.3	算法实现	131
3.4.4	正确性证明	133
3.4.5	与 0-1 背包问题的对比	133
3.5	最大物品数装载问题	134
3.5.1	问题描述	134
3.5.2	贪心策略	134
3.5.3	算法描述	134
3.5.4	正确性证明	137
3.5.5	复杂度分析	138
3.5.6	示例	138
3.5.7	小结	139
3.6	哈夫曼编码 (Huffman Coding)	139
3.6.1	问题引入	139
3.6.2	前缀码与二叉树表示	139
3.6.3	哈夫曼算法: 贪心合并	140
3.6.4	手算演示: 逐步合并	140
3.6.5	算法实现	142
3.6.6	正确性证明	144
3.6.7	思考题	147
3.7	最小生成树问题	147
3.7.1	问题描述	147
3.7.2	Prim 算法	147
3.7.3	正确性证明	151
3.7.4	Kruskal 算法	151
3.7.5	3.7.5 Kruskal 算法的正确性证明	156
3.7.6	思考题	157

3.8	单源最短路径 Dijkstra 算法	157
3.8.1	问题描述	157
3.8.2	Dijkstra 算法	157
3.8.3	正确性证明	162
3.8.4	时间复杂度	163
3.8.5	思考题	165
3.9	贪心策略总结	165
3.9.1	本章实例的贪心策略	165
3.9.2	贪心法的特点与局限性	165
3.10	习题	165
3.10.1	选择题 (共 10 题)	165
3.10.2	判断题 (共 10 题)	167
3.10.3	填空题 (共 10 题)	167
3.10.4	简答题 (共 5 题)	168
3.10.5	证明题 (共 6 题)	168
3.10.6	客观编程题	171
4	分治法	177
4.1	分治法的基本思想	177
4.2	二分查找	177
4.2.1	问题描述	177
4.2.2	分治思路	177
4.2.3	例子	178
4.2.4	伪代码	178
4.2.5	完整代码	178
4.2.6	时间复杂度分析	179
4.3	汉诺塔问题	180
4.3.1	问题描述	180
4.3.2	分治思路	180
4.3.3	例子: $n = 3$ 的递归树	180
4.3.4	伪代码	181
4.3.5	完整代码	182
4.3.6	时间复杂度分析	183
4.4	归并排序	183
4.4.1	问题描述	183
4.4.2	分治思路	183
4.4.3	例子	183
4.4.4	伪代码	184
4.4.5	完整代码	185
4.4.6	时间复杂度分析	186
4.5	逆序数	187
4.5.1	从音乐评分到逆序对	187
4.5.2	逆序数的应用	187

4.5.3	用分治法计算逆序数	187
4.5.4	如何高效计算逆序数?	188
4.5.5	伪代码	189
4.5.6	代码实现	190
4.5.7	时间复杂度分析	192
4.6	快速排序	192
4.6.1	问题描述	192
4.6.2	分治思路	192
4.6.3	划分过程详解 (Hoare 划分)	193
4.6.4	递归与完整算法	194
4.6.5	关键观察	195
4.6.6	关键点说明	196
4.6.7	完整代码	197
4.6.8	4.6.8 时间复杂度分析	198
4.7	大整数乘法 (Karatsuba 算法)	198
4.7.1	问题的引出	198
4.7.2	分治法的初步想法	198
4.7.3	一个简单的例子	199
4.7.4	Karatsuba 的巧妙观察	199
4.7.5	递归过程与复杂度分析	200
4.7.6	算法伪代码	200
4.7.7	代码实现	200
4.7.8	思考与拓展	202
4.8	矩阵乘法 (Strassen 算法)	202
4.8.1	问题的引出	202
4.8.2	分治法的初步想法	202
4.8.3	一个简单的例子	203
4.8.4	Strassen 的巧妙观察	203
4.8.5	递归过程与复杂度分析	204
4.8.6	代码实现	206
4.8.7	思考与拓展	209
4.9	选择最小与第二小元素: 从特例到一般	210
4.9.1	问题引入	210
4.9.2	最小值: 线性扫描	210
4.9.3	第二小: 朴素两遍扫描	210
4.9.4	锦标赛算法: 最优解法	211
4.9.5	分治法求解第二小	214
4.9.6	小结	214
4.9.7	习题	214
4.10	选择问题 (第 k 小元素) 和快速选择算法	215
4.10.1	问题提出	215
4.10.2	直观例子: 分治递归思想的体现	215
4.10.3	核心思想 (图示理解)	215

4.10.4	算法描述	216
4.10.5	伪代码	216
4.10.6	Quickselect 正确性证明	217
4.10.7	复杂度分析	218
4.10.8	改进版本：随机化快速选择	219
4.10.9	实际应用	222
4.10.10	练习题	222
4.11	Median-of-Medians 算法	222
4.11.1	核心思想：如何保证“好”的 pivot?	223
4.11.2	算法步骤	223
4.11.3	代码实现	223
4.11.4	时间复杂度分析：为什么能保证线性时间?	227
4.11.5	小结	230
4.11.6	练习题	230
4.12	最大子数组问题	231
4.12.1	问题提出	231
4.12.2	直观示例	231
4.12.3	算法思想	231
4.12.4	算法描述	232
4.12.5	伪代码	232
4.12.6	代码实现	234
4.12.7	正确性证明	238
4.12.8	复杂度分析	239
4.13	最近点对问题	239
4.13.1	问题描述	239
4.13.2	分治思路	239
4.13.3	合并步骤的关键	239
4.13.4	为什么最多检查 6 个点?	240
4.13.5	分治算法框架	242
4.13.6	一个具体的例子	242
4.13.7	算法伪代码（改进版）	243
4.13.8	代码实现	244
4.13.9	时间复杂度分析	247
4.13.10	小结	248
4.14	凸包问题	248
4.14.1	问题提出	248
4.14.2	直观示例	248
4.14.3	算法思想	249
4.14.4	算法描述	249
4.14.5	伪代码	250
4.14.6	代码实现	252
4.14.7	正确性证明	258
4.14.8	复杂度分析	258

4.15	棋盘覆盖问题	259
4.15.1	问题提出	259
4.15.2	直观示例	259
4.15.3	算法思想	260
4.15.4	算法描述	261
4.15.5	伪代码	262
4.15.6	代码实现	263
4.15.7	正确性证明	267
4.15.8	复杂度分析	268
4.16	求最大最小值问题	268
4.16.1	问题描述	268
4.16.2	朴素算法	268
4.16.3	分治法	269
4.16.4	成对比较法（最优迭代算法）	272
4.16.5	锦标赛算法	273
4.16.6	代码实现	274
4.16.7	比较次数的理论下界	279
4.16.8	算法对比与总结	280
4.16.9	习题	280
4.17	总结	280
4.17.1	常见递推关系及解	280
4.17.2	分治法的适用条件	281
4.17.3	思考题	281
4.18	习题	281
4.18.1	选择题	281
4.18.2	判断题	282
4.18.3	填空题	283
4.18.4	简答题	284
4.18.5	客观编程题	285
4.19	实验	289
4.19.1	基于分治与暴力法的逆序数计算比较	289
4.20	最大子段和（输出子数组下标）	291
4.20.1	题目描述	291
4.20.2	输入格式	291
4.20.3	输出格式	291
4.20.4	样例	291
5	动态规划	293
5.1	动态规划的基本思想	293
5.1.1	从一个简单例子开始：斐波那契数列	293
5.1.2	动态规划的一般步骤	296
5.2	找零钱问题	296
5.2.1	问题描述	296

5.2.2	定义目标函数与递归方程	296
5.2.3	自顶向下实现	297
5.2.4	自底向上实现	298
5.2.5	输出最优方案	299
5.2.6	小结	301
5.3	机器人路径问题	301
5.3.1	问题描述	301
5.3.2	定义目标函数	301
5.3.3	自底向上实现（二维表）	302
5.3.4	空间优化	303
5.3.5	理解优化过程	303
5.4	动态规划的两个重要特征	305
5.4.1	重叠子问题	305
5.4.2	最优子结构	305
5.4.3	两个特征的关系	307
5.4.4	小结	307
5.5	0-1 背包问题	307
5.5.1	问题描述	307
5.5.2	定义目标函数	307
5.5.3	自底向上实现（二维表）	308
5.5.4	举例填表	309
5.5.5	输出最优方案	309
5.5.6	空间优化	311
5.6	最长公共子序列（LCS）	312
5.6.1	问题描述	312
5.6.2	定义目标函数与递归方程	312
5.6.3	自底向上实现（二维表）	312
5.6.4	举例填表	313
5.6.5	输出一个最长公共子序列	314
5.6.6	空间优化	315
5.6.7	小结	316
5.7	编辑距离	316
5.7.1	问题描述	316
5.7.2	定义目标函数与递归方程	317
5.7.3	自底向上实现（二维表）	317
5.7.4	举例填表	318
5.7.5	输出编辑方案（可选）	319
5.7.6	空间优化	320
5.7.7	小结	321
5.8	最大子段和（Kadane 算法）	322
5.8.1	问题描述	322
5.8.2	定义目标函数与递推公式	322
5.8.3	自底向上实现	322

5.8.4	空间优化 (Kadane 算法)	323
5.8.5	举例演示	324
5.8.6	小结	324
5.9	最长递增子序列 (LIS)	324
5.9.1	问题描述	324
5.9.2	定义目标函数与递推公式	324
5.9.3	自底向上实现 (基础版, $O(n^2)$)	325
5.9.4	举例演示	326
5.9.5	小结	326
5.10	矩阵链乘法	326
5.10.1	问题描述	326
5.10.2	定义目标函数与递归方程	327
5.10.3	自底向上实现 (按反对角线填表)	327
5.10.4	复杂度分析	330
5.10.5	小结	330
5.11	小结	330
5.12	习题	331
5.12.1	选择题 (单选)	331
5.12.2	判断题 (正确打 $\checkmark$, 错误打 $\times$)	332
5.12.3	填空题	332
5.12.4	客观编程题 (带参考解法)	333
6	状态空间搜索与高级剪枝技术	337
6.1	引言: 从八数码看状态探索	337
6.2	状态空间搜索基础	340
6.2.1	状态、操作与状态空间	341
6.2.2	后继函数: 从状态出发能走到哪里	341
6.2.3	三个典型例子	342
6.2.4	状态空间图与搜索树	343
6.2.5	解与解路径	344
6.2.6	搜索算法的四个关键问题	345
6.2.7	本节小结	345
6.3	基本搜索策略: DFS 与 BFS	345
6.3.1	引例: 数字变换问题	346
6.3.2	统一搜索框架: 一个模板, 两种走法	346
6.3.3	深度优先搜索 (DFS) —— 一条道走到黑	347
6.3.4	广度优先搜索 (BFS) —— 水波式扩散	349
6.3.5	6.3.5 DFS vs BFS: 本质对比	350
6.3.6	6.3.4 广度优先搜索 (BFS) —— 一层一层往外走	350
6.3.7	在状态空间搜索中的应用	352
6.3.8	小结	363
6.4	回溯法	365
6.4.1	回溯法的本质: 聪明的暴力搜索	365

6.4.2	多步决策与回溯：以 4 皇后问题为例	366
6.4.3	回溯法的通用递归框架	369
6.4.4	回溯法与 DFS 的关系	369
6.4.5	全排列	370
6.4.6	N 皇后问题（约束剪枝）	376
6.4.7	0-1 背包问题：在指数空间中寻找最优解	381
6.5	分支限界法	386
6.5.1	从一个例子出发：0-1 背包问题	386
6.5.2	基本概念	387
6.5.3	两种分支限界方式	387
6.5.4	0-1 背包问题的 LC 分支限界求解	388
6.5.5	分支限界法与回溯法的对比	397
6.5.6	本节小结	398
6.5.7	本章小结	398
6.6	习题	398
6.6.1	概念理解题	398
6.6.2	BFS 与 DFS 算法应用题	399
6.6.3	回溯法应用题	399
6.6.4	分支限界法应用题	399
6.6.5	综合设计题	400
6.6.6	编程实践题	400

Appendices 401

A	附录 A 图的基本概念	401
A.0.1	A.1 图的定义与分类	401
A.0.2	A.2 完全图与子图	402
A.0.3	A.3 顶点的度	403
A.0.4	A.4 路径与回路	403
A.0.5	A.5 树	404

前言

在计算机科学的世界里，算法既是核心的灵魂，也是解决问题的艺术。无论你是初入编程殿堂的学子，还是希望系统提升算法功底的科研人员，抑或是备战面试的求职者，掌握算法设计与分析的能力，都是通往更高技术层次的关键阶梯。

本书的写作初衷，源于我在教学与科研实践中的一个深刻体会：**算法的本质并不复杂，复杂的往往是那些绕弯子的表述**。许多教材为了追求理论的完备性，堆砌了大量抽象的数学推导，却让初学者望而却步；另一些资料则过于注重代码实现，忽略了算法设计背后的思想脉络，使读者陷入“只见代码，不见森林”的困境。

因此，本书尝试走一条中间路线——**以问题为驱动，以策略为主线，以实践为归宿**。我们希望用最直白的语言，讲清楚每一种算法设计策略“是什么、为什么、怎么用”，并通过大量精选的经典例子，让读者在具体的应用场景中领悟算法的精髓。

本书内容编排

全书共分七章，按照算法设计策略的自然逻辑展开：

- **第一章算法基础**：从算法的定义出发，厘清算法与程序的关系，介绍算法设计的基本策略（穷举、贪心、分治、动态规划等），并系统讲解算法分析的核心工具——渐近记号与递归方程求解。这一章为后续内容打下坚实的理论基础。
- **第二章线性穷举**：从最简单的穷举法入手，通过“两数之和”“N 皇后”等经典问题，展示如何系统性地搜索解空间，并分析其效率瓶颈，为后续更高效的策略埋下伏笔。
- **第三章贪心法**：介绍“每步取最优”的贪心思想，通过活动安排、哈夫曼编码、最小生成树等例子，剖析贪心策略适用的场景与证明方法。
- **第四章分治法**：讲解“分而治之”的经典范式，从归并排序、快速排序到 Karatsuba 乘法、Strassen 矩阵乘法，展现分治法如何将复杂问题化整为零，并利用递归树与主定理进行复杂度分析。
- **第五章动态规划**：深入探讨动态规划的“最优子结构”与“重叠子问题”两大核心特征，通过背包问题、最长公共子序列、矩阵链乘等经典案例，引导读者掌握状态定义与递推关系的构建技巧。
- **第六章状态空间搜索**：介绍回溯法与分支限界法，通过 N 皇后、旅行商问题、图着色等组合优化问题，展示如何通过剪枝策略高效搜索大规模解空间。

每一章都配有丰富的例题与习题，所有算法均先用清晰易懂的**伪代码**描述逻辑，再给出 **C++** 与 **Java** 两种语言的完整实现，方便读者对照学习与实践验证。

本书特色

- **深入浅出，直击本质**：摒弃冗长的理论推导，用最通俗的语言讲清每种策略的核心思想。
- **例子驱动，即学即用**：每个知识点都配有精心挑选的经典例题，边学边练，加深理解。
- **双语言实现，兼顾不同背景**：同时提供 C++ 和 Java 代码，满足不同编程习惯的读者需求。
- **无废话，全是干货**：每章内容都经过反复提炼，确保每一页都有价值，绝不凑字数。

读者对象

本书适合作为高等院校计算机相关专业“算法设计与分析”课程的教材或参考书，也适合自学算法的编程爱好者、准备求职面试的开发者，以及希望系统梳理算法知识体系的科研人员。

资源与联系方式

- 电子版购买: <https://leanpub.com/c01>
- 作者博客: <https://hwdong-net.github.io>
- 配套代码: <https://github.com/hwdong-net/DAA>
- Youtube 频道: <https://www.youtube.com/@hwdong>
- B 站: hw-dong
- 公众号: hwdong 编程

关于作者

我是一名大学计算机教师，长期讲授数据结构、算法设计与分析、C++ 编程、计算机图形学、计算机网络，以及高等数学、线性代数、概率论等课程。早年研究方向为计算机图形学。

1985 年高考，我的总分超过清华大学在江苏省录取线 20 多分，数学和物理接近满分。因当时招生规则及外部因素，未能报考清华大学，最终进入哈尔滨工业大学学习。这段经历让我深刻认识到：外部环境无法控制，唯有自学能力是终身可靠的财富。

大学期间因成绩优异，经校长特批，可以免听课，只参加考试。从此我开始了完全自学的学习方式。硕士和博士阶段延续这一习惯，博士数学考试取得 99 分。2008 年在美国德州农工大学计算机系担任访问学者，2016 年在休斯顿大学开展学术访问，这些经历拓宽了我的视野。

教学之余，我编写了多本技术书籍，致力于为读者提供清晰易懂的学习资料。我的主要著作包括：

电子版

- 《解剖深度学习原理：从 0 编写深度学习库》
https://leanpub.com/dl_0
- 《现代 C++ 编程实战：零基础 C++ 项目实战》
<https://leanpub.com/c01>
- 《算法设计与分析》
<https://leanpub.com/daa>

纸质版

- 《解剖深度学习原理：从 0 编写深度学习库》
电子工业出版社，2021 年 7 月
- 《Python 3 从入门到实战》
电子工业出版社 / 亚马逊平台，2019 年 10 月
- 《C++17 从入门到精通》
清华大学出版社，2019 年 8 月

关于本书

算法是计算机科学的灵魂。本书面向已掌握一门编程语言（如 C++、Python 或 Java）的读者，系统讲解分治、动态规划、贪心、回溯、图论等经典算法策略，并注重时间/空间复杂度的严谨分析。与许多重代码轻推导的算法书不同，本书坚持“思路先行”的原则：每个算法都会先用手工推演或数学归纳的方式解释其正确性，再给出可运行的 C++ 实现。本书适合准备面试、参加竞赛、或希望夯实内功的读者。如果你愿意静下心来跟随推导过程，相信你一定能建立起扎实的算法思维。

书中难免有疏漏，欢迎读者提出宝贵意见，与我一同完善这部作品。

作者博客：<https://hwong-net.github.io>

本书电子版：<https://leanpub.com/daa>

1 算法设计与分析基础

1.1 什么是算法？

1.1.1 算法的定义

算法就是**解决特定问题的一组明确、有限、可实际执行的步骤**。无论多么复杂的计算任务，最终都要分解成计算机（或人）能够一步一步完成的简单动作。好的算法应该具备以下几个基本要求：

- 有明确的输入（可以是 0 个或多个）和至少一个输出；
- 每一步都写得清楚明白，没有歧义；
- 执行过程必须在有限步内结束，不能无限循环下去；
- 每一步都能实际完成（用加减乘除、比较、读写等基本操作就能实现）。

下面通过两个制作珍珠奶茶的例子来直观感受什么是算法，以及同一问题可以有不同的算法。

例 1.1. 制作珍珠奶茶

方法 1：传统煮制法（算法 A）

1. 准备材料：红茶包 1 个、奶精粉 30g、白糖 40g、珍珠 100g、热水 500ml、冰块适量。
2. 煮珍珠：将 100g 珍珠放入沸水中，大火煮 30 分钟，关火后加盖焖 25 分钟，捞出过冰水沥干备用。
3. 泡红茶：将红茶包放入 500ml 沸水中浸泡 5 分钟，取出茶包。
4. 调奶茶：在茶水中加入白糖搅拌至溶解，再加入奶精粉搅拌均匀。
5. 装杯：将煮好的珍珠放入杯中，倒入调好的奶茶，加入适量冰块。

方法 2：微波炉速成法（算法 B）

1. 准备材料：红茶包 1 个、奶精粉 30g、白糖 40g、即食珍珠（无需煮制）100g、热水 200ml、冷水 300ml、冰块适量。
2. 制茶浓缩液：将红茶包放入 200ml 沸水中浸泡 3 分钟，取出茶包，得到浓茶汁。
3. 调制奶精糖浆：在另一容器中用少量热水将白糖和奶精粉化开，搅拌成均匀糖浆。
4. 混合：将浓茶汁、奶精糖浆、冷水 300ml 和冰块一起倒入杯中，搅拌均匀。
5. 加入珍珠：将即食珍珠直接放入杯中。

对比分析

算法 A 采用传统的煮珍珠和泡茶方法，耗时较长（煮珍珠 55 分钟，泡茶 5 分钟），但每一步都是经典做法。算法 B 则通过即食珍珠和浓茶汁大幅缩短时间，步骤顺序也与算法 A 完全不同。两个算法都能成功制作一杯珍珠奶茶，但步骤数量、操作方式、耗时均有明显差异。这表明**同一个问题可以有多种算法**，而不同的算法往往在效率、资源消耗等方面有所区别，这正是算法分析要研究的核心内容。

例 1.2. 两个整数相乘（小学竖式乘法

问题：计算 23×14

算法步骤（按小学竖式展开）：

$$\begin{array}{r}
 23 \\
 \times 14 \\
 \hline
 92 \quad \leftarrow \text{先用个位 } 4 \text{ 乘 } 23: 4 \times 3 = 12 \text{ (写2, 进1), } 4 \times 2 = 8 \text{ 加进1得9} \rightarrow 92 \\
 230 \quad \leftarrow \text{再用十位 } 1 \text{ 乘 } 23 \text{ (右移一位): } 1 \times 3 = 3, 1 \times 2 = 2 \rightarrow 23 \text{ (右移一位变成230)} \\
 \hline
 322 \quad \leftarrow \text{把上面两行相加: } 92 + 230 = 322
 \end{array}$$

输出：322

例 1.3. 矩形周长

问题：长 a ，宽 b 的矩形的周长是多少？

算法步骤：

1. 计算长 + 宽 $= a + b$
2. 周长 $= 2 \times (a + b)$
3. 输出结果

例如： $a = 8 \text{ cm}$ ， $b = 5 \text{ cm}$ 周长 $= 2 \times (8 + 5) = 26 \text{ cm}$

例 1.4. 一组数中的最大值

问题：求一组数 $(a_1, a_2, \dots, a_n)$ 中的最大值。

算法步骤：

1. 把第一个数当作当前最大值： $\max \leftarrow a_1$
2. 从第二个数开始，依次比较每个数 a_i ($i=2$ 到 n)：如果 $a_i > \max$ ，则更新 $\max \leftarrow a_i$
3. 全部比较完后， $\max$ 就是这组数中的最大值

例如：一组数 $[7, 3, 9, 2, 8, 1]$ 过程：

- 开始 $\max = 7$
- 7 与 3 比较 $\rightarrow \max$ 仍为 7
- 7 与 9 比较 $\rightarrow \max$ 更新为 9
- 9 与 2 比较 $\rightarrow \max$ 仍为 9
- 9 与 8 比较 $\rightarrow \max$ 仍为 9
- 9 与 1 比较 $\rightarrow \max$ 仍为 9

输出：9

这些例子虽然简单，但都体现了算法的核心：把一个问题分解成清晰、可执行的有限步骤，并得到确定的结果。

1.1.2 算法与程序的区别

算法和程序虽然密切相关，但并不是一回事。

- **算法** 是对解决问题方法的**抽象描述**，不依赖于任何具体的计算机或编程语言。它通常用自然语言、伪代码、流程图等方式表达，关注“怎么做”的本质逻辑。
- **程序** 是算法在计算机上的**具体实现**，必须用某种编程语言（如 Python、C++、Java）编写成可执行代码，才能在机器上运行。

简单说：**算法是“菜谱”，程序是“用菜谱做出来的菜”**。同一个算法可以用多种编程语言实现成不同的程序；反过来，一个程序背后通常对应着一个或多个算法。有些程序（如简单的输入输出或界面交互）并不一定是严格意义上的算法。

1.1.3 算法解决问题的过程

用算法解决问题通常遵循一个典型的迭代过程。以“求一组数的最大值”为例：

1. **理解和定义问题**：明确输入是一组数 $a_1, a_2, \dots, a_n$ ，输出是其中的最大值。最大值 $\max$ 必须满足： $\max \geq a_i$ 对所有 $1 \leq i \leq n$ ，且至少有一个 a_i 等于 $\max$ 。
2. **数学建模**：用数组或序列表示这组数，定义“最大值”的数学性质。
3. **设计算法**：选择合适的策略（如从头到尾线性比较），描述清晰的步骤（如初始化 $\max$ 为第一个数，然后逐个比较更新）。
4. **证明正确性**：用数学方法验证算法在所有输入下都输出正确结果（例如用循环不变量：每一步结束后， $\max$ 至少是当前已扫描部分的最大值；最后扫描完所有元素， $\max$ 就是全局最大）。
5. **分析算法性能**：评估时间复杂度 ($O(n)$)、空间复杂度 ($O(1)$)，以及正确性、稳定性等。

这个过程是通用的：从实际问题出发，经过抽象建模、设计、验证、分析和优化，最终得到高效可靠的解决方案。如果性能不理想，可以返回上一步改进。

1.1.4 算法有什么用？

算法是计算机科学的核心，也是现代科技和各行各业的“灵魂”。它的重要性体现在以下几个方面：

1. **是计算机学科的主干** 几乎每一个计算机科学分支都以算法为核心：
 - **操作系统和编译器**：进程调度、内存管理、词法分析、语法分析等算法。
 - **计算机网络**：路由算法、拥塞控制。
 - **搜索引擎**：页面排名、索引构建。
 - **机器学习和人工智能**：神经网络训练、随机森林、支持向量机、聚类等算法。
 - **密码学**：加密/解密、数论算法。
 - **计算生物学**：基因序列比对、同源性分析。
 - **计算机图形学**：计算几何、光照渲染、流体仿真、动画生成等。
2. **在实际应用中的广泛作用**
 - **大数据处理**：计算速度取决于硬件和算法。即使硬件再快，没有高效算法也处理不了海量数据。
 - **深度学习/现代人工智能**：人脸识别、下棋程序（如 AlphaGo）、自动驾驶、语音识别等，都依赖精心设计的算法。

- **电子商务平台**：推荐算法决定用户看到什么商品，直接影响销售额。
 - **自媒体与信息流**：算法决定内容推送顺序，影响用户思维、舆论导向和社会热点。
3. **算法有趣且富有挑战性** 算法设计是一种创造性的数学活动，既有趣又有挑战。面对一个问题，如何设计出高效、优雅的解决方案，往往令人激动。例如：
- **大整数乘法**：（从小学竖式到 Karatsuba 算法，再到 FFT 加速）。
 - **旅行商问题（TSP）** 的近似算法。
 - **排序算法的极限** ($O(n \log n)$ 下界)。

学习算法不仅能让你写出高效程序，还能训练逻辑思维、问题分解能力和创造力。这些能力在编程、科研、工程甚至日常决策中都非常有用。

一个真正好的算法，不仅要正确，还要尽可能快（时间效率高）和省资源（空间效率高）。这正是我们学习“算法设计”（怎么构造）和“算法分析”（怎么评价）的根本目的。

1.2 算法的设计

算法设计的核心问题是：**面对一个具体问题，如何系统地构造出正确且高效的解决方案？**

算法设计既是一门**技术**，也是一门**艺术**。作为技术，它遵循一些经典的设计模式（如分治、贪心、动态规划等），这些模式为我们解决各类问题提供了可复用的框架；作为艺术，它又需要创造性的思维，面对新问题时能够跳出常规，构想出巧妙的解法。

以两个简单问题为例。

第一个是求 1 到 100 的和。普通的做法是循环累加： $1+2+3+\dots+100$ ，需要执行 99 次加法。而高斯在童年时就发现可以用公式 $100 \times 101/2 = 5050$ ，一次乘法加一次除法即可得到结果。这不仅是计算技巧，更是一种算法思维的体现——通过数学归纳将重复操作转化为常数时间计算。

第二个是更复杂的大整数乘法。我们从小学习的**竖式乘法**，逐位相乘再相加，时间复杂度为 $O(n^2)$ 。而 20 世纪 60 年代，Karatsuba 发现了一种分治策略，将乘法次数从 4 次减少到 3 次，从而将复杂度降至 $O(n^{\log_2 3}) \approx O(n^{1.585})$ 。这一突破展示了创造性思维如何颠覆传统方法的效率瓶颈。

这两个例子告诉我们：**同一个问题往往有多种解法，但它们的效率可能相差巨大**。设计算法的过程，正是不断在技术规范与创意灵感之间权衡，在正确性与效率之间寻找最佳平衡。

正如这个例子所揭示的，同一个问题往往有多种解法，但它们的效率可能相差巨大。设计算法的过程，正是不断在技术规范与创意灵感之间权衡，在正确性与效率之间寻找最佳平衡。

本书将系统介绍算法设计的经典策略（也称设计范式或方法），它们如同一个工具箱，帮助我们针对不同性质的问题选择合适的“路径”。同时，我们也会学习如何分析算法的效率以及如何证明算法的正确性。

下一小节将首先介绍算法的常用描述方式，为后续设计策略的学习做好准备。

1.2.1 算法设计策略

下面通过一些简单经典的例子，介绍几种最常用的算法设计策略。这些策略将在后续章节详细展开，这里只作初步认识。

1.2.1.1 暴力法 / 穷举法 (Brute Force / Exhaustive Search)

思路：把所有可能的解都列出来，一个一个检查，直到找到满足条件的答案。

例 1.5. 两数之和

问题：给定一个整数数组 `nums` 和一个目标值 `target`，请找出数组中两个不同元素的下标 `i` 和 `j`，使得 $nums[i] + nums[j] = target$ 。假设每个输入有且仅有一个解，且同一个元素不能使用两次，返回下标对 `[i, j]`（顺序任意）。

通用做法：用两重循环枚举数组中每一对不同的元素 ($i < j$)，检查它们的和是否等于 `target`，如果等于则返回它们的下标。

例如：数组 `nums = [3, 5, 2, 8, 11]`，`target = 10`。

穷举所有数对：

- $i=0 (3) + j=1 (5) = 8 \neq 10$
- $i=0 (3) + j=2 (2) = 5 \neq 10$
- $i=0 (3) + j=3 (8) = 11 \neq 10$
- $i=0 (3) + j=4 (11) = 14 \neq 10$
- $i=1 (5) + j=2 (2) = 7 \neq 10$
- $i=1 (5) + j=3 (8) = 13 \neq 10$
- $i=1 (5) + j=4 (11) = 16 \neq 10$
- $i=2 (2) + j=3 (8) = 10 \rightarrow$ 找到，返回 `[2, 3]`

例 1.6. 在一组数中找出最大值

问题：给定一组数，找出其中最大的那个数。

通用做法：把第一个数作为当前最大值，然后从第二个数开始遍历所有元素，与当前最大值逐个比较，如果当前元素更大，就更新当前最大值。

具体例子：数组 `[28, 3, 21, 69, 7]`。过程：

- 先取第一个数 28 作为当前最大值
- 比较第二个数 3: $3 < 28$ ，最大值仍为 28
- 比较第三个 21: $21 < 28$ ，最大值仍为 28
- 比较第四个 69: $69 > 28$ ，更新最大值为 69
- 比较第五个 7: $7 < 69$ ，最大值仍为 69 最终最大值为 69。

例 1.7. N 皇后问题（暴力穷举思路）

问题：在 $N \times N$ 棋盘上放 N 个皇后，任意两个皇后不能互相攻击（不同行、不同列、不同对角线）。

穷举思路：逐行放置皇后。每行尝试所有可能的列位置，检查是否与前面皇后冲突；若冲突则换位置，若某行无合法位置则回溯到上一行换其他列。遍历所有可能的放置路径，找出合法解。

简化例子 (4 皇后)：为了便于图示，我们以 4 皇后为例展示搜索过程。下图用“Q”表示皇后，“.”表示空格。逐行放置：

1. 第 1 行放在列 1:

```
Q . . .  
. . . .
```

.

2. 第 2 行尝试列 3 (避开同一列和对角线):

Q . . .
 . . Q .

3. 第 3 行尝试列 2 (检查与前面皇后的冲突):

Q . . .
 . . Q .
 . Q . .

→ 冲突! (第 2 行 Q 与第 0 行 Q 在副对角线上: 行差 = 2, 列差 = 1, 不等, 但需检查所有对角线规则)

实际穷举会继续尝试其他列; 若某行放不下, 则回溯换前一行的位置。

结论: 这种方式本质上是穷举所有可能的放置序列。只要棋盘不大 (如 $N=4$ 有 2 解, $N=8$ 有 92 解), 全部试一遍就能找到答案。但当 N 增大, 路径数量呈指数级爆炸, 纯穷举效率极低。

(后续章节将介绍如何通过“提前剪枝 + 回溯”大幅加速。)

暴力法是最原始、最容易想到的方法, 正确性高, 但效率通常很低。适合问题规模小、或作为其他方法的对比基准。

1.2.1.2 贪心法 (Greedy Algorithm)

思路: 每一步都做出当前看起来“最好”的选择, 希望这些局部最优最终能得到全局最优。

例 1.8. 找零钱

问题: 给定一组硬币面值和—个目标金额 $target$, 要求用最少的硬币数量凑出正好 $target$ 的金额 (假设硬币面值合理, 总能凑出)。

通用做法: 每次选面值最大的硬币, 直到凑够金额。

具体例子: 硬币面值 $[1, 5, 10, 25]$, $target = 36$ 。步骤: 先选 1 个 25 元 (剩 11 元) → 选 1 个 10 元 (剩 1 元) → 选 1 个 1 元 → 总共 3 枚硬币。

例 1.9. 分数背包问题

问题: 给定 n 个物品, 每个物品有重量和价值, 可以任意切分物品, 背包容量为 W , 要求在不超重的情况下得到最大总价值。

通用做法: 计算每个物品的价值密度 (价值 / 重量), 按密度从高到低排序, 然后依次装入背包, 直到装满。

具体例子: 3 个物品:

- A: 重量 2, 价值 10 (密度 5.0)
- B: 重量 3, 价值 12 (密度 4.0)
- C: 重量 5, 价值 15 (密度 3.0) 背包容量 $W = 8$ 。

按密度排序: $A \rightarrow B \rightarrow C$ 。

先全装 A (价值 10, 剩容量 6), 再装 B (价值 12, 剩容量 3), 最后装 C 的 3/5 (价值 9), 总价值 $10+12+9=31$ 。

贪心法非常高效 (通常线性或接近线性时间), 但不是所有问题都适用——必须证明局部最优能推出全局最优。

1.2.1.3 分治法 (Divide and Conquer)

思路: 把大问题分解成规模更小的相同或相似子问题, 递归求解子问题, 再合并结果。分治法的典型步骤是:

1. 分解 (Divide): 将原问题拆分成若干个规模较小的子问题;
2. 递归求解 (Conquer): 递归地解决这些子问题 (如果子问题足够小, 则直接求解);
3. 合并 (Combine): 将子问题的解组合起来得到原问题的解。

例 1.10. n 的阶乘

问题: 给定非负整数 n , 计算 n 的阶乘 ($n! = n \times (n-1) \times \dots \times 1$, $0! = 1$)。

分治做法:

- 分解: $n! = n \times (n-1)!$
- 递归求解: 计算 $(n-1)!$
- 合并: 把 n 乘以 $(n-1)!$ 的结果

具体例子:

$n = 5$ 。 $5! = 5 \times 4!$ $4! = 4 \times 3!$ $3! = 3 \times 2!$ $2! = 2 \times 1!$ $1! = 1$ 最终 $5! = 120$ 。

例 1.11. 斐波那契数列

问题: 给定非负整数 n , 计算第 n 个斐波那契数, 其中 $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$ ($n \geq 2$)。

分治做法:

- 分解: 将 $F(n)$ 分解成两个子问题 $F(n-1)$ 和 $F(n-2)$
- 递归求解: 分别递归计算 $F(n-1)$ 和 $F(n-2)$
- 合并: 把两个子问题的结果相加得到 $F(n)$

具体例子: $n = 6$ 。 $F(6) = F(5) + F(4)$ $F(5) = F(4) + F(3)$ $F(4) = F(3) + F(2)$ $F(3) = F(2) + F(1)$ $F(2) = F(1) + F(0) = 1 + 0 = 1$ 最终 $F(6) = 8$ 。(这是一个非常典型的二叉递归分治例子, 清晰地体现了“分解成两个更小规模的同类子问题 $\rightarrow$ 递归求解 $\rightarrow$ 合并 (简单相加)”的过程。但由于 $F(n-1)$ 和 $F(n-2)$ 的子问题会大量重叠, 导致重复计算非常多, 时间复杂度呈指数级增长, 后续动态规划章节会给出高效的优化方法。)

例 1.12. 汉诺塔问题

有三根柱子 A、B、C。A 柱上有 n 个大小不同的盘子（从上到下由小到大），要求把所有盘子从 A 柱移动到 C 柱。规则：

- 每次只能移动一个盘子
- 大盘子不能放在小盘子上

分治做法：

- 把 $n-1$ 个盘子从 A 移到 B（借助 C）
- 把第 n 个盘子从 A 移到 C
- 把 $n-1$ 个盘子从 B 移到 C（借助 A）

具体例子： $n = 3$ （三个盘子：小盘 1、中盘 2、大盘 3，初始都在 A 柱）

初始状态：A 柱：3 → 2 → 1（从下到上） B 柱：空 C 柱：空

递归过程示意（树形结构，从上到下为递归深入，从左到右为执行顺序）：

```

move(3, A→C, 借助B)
├─ move(2, A→B, 借助C)      ← 先把上面2个盘子从A移到B（借助C）
│   ├─ move(1, A→C, 借助B)   ← 递归：移动1个盘子从A到C
│   │   └─ 直接移动：盘子1 A → C
│   └─ 直接移动：盘子2 A → B
│       └─ move(1, C→B, 借助A) ← 递归：把1个盘子从C移到B
│           └─ 直接移动：盘子1 C → B
└─ 直接移动：盘子3 A → C
    └─ move(2, B→C, 借助A)    ← 再把上面2个盘子从B移到C（借助A）
        ├─ move(1, B→A, 借助C) ← 递归：移动1个盘子从B到A
        │   └─ 直接移动：盘子1 B → A
        └─ 直接移动：盘子2 B → C
            └─ move(1, A→C, 借助B) ← 递归：把1个盘子从A移到C
                └─ 直接移动：盘子1 A → C
  
```

实际 7 步移动序列（对应递归树从左到右、从上到下执行）：

1. 盘子 1: A → C
2. 盘子 2: A → B
3. 盘子 1: C → B
4. 盘子 3: A → C
5. 盘子 1: B → A
6. 盘子 2: B → C
7. 盘子 1: A → C

最终：所有盘子从 A 移到 C。

最终状态：所有盘子从 A 移到 C（总共 $2^3 - 1 = 7$ 步）。

例 1.13. 归并排序

问题：给定一个无序整数数组，将其从小到大排序。

分治做法：

- 分解：将数组分成两个大致相等的子数组
- 递归求解：递归地对左右两个子数组进行排序
- 合并：将两个已经有序的子数组合并成一个有序数组

具体例子：数组 [38, 27, 43, 3, 9, 82, 10]。

递归过程（树形展开）：

```
[38, 27, 43, 3, 9, 82, 10]
├─ 左：[38, 27, 43]
│   ├── [38]
│   └── [27, 43]
│       ├── [27]
│       ├── [43]
│       └── 合并：[27, 43]
└─ 合并：[27, 38, 43]
├─ 右：[3, 9, 82, 10]
│   ├── [3, 9]
│   │   ├── [3]
│   │   └── [9]
│   │   └── 合并：[3, 9]
│   └── [82, 10]
│       ├── [82]
│       └── [10]
│       └── 合并：[10, 82]
└─ 合并：[3, 9, 10, 82]
```

最终合并：[27, 38, 43] + [3, 9, 10, 82] → [3, 9, 10, 27, 38, 43, 82]

分治法常带来高效的算法（如 $O(n \log n)$ 排序），但需要子问题相对独立且合并代价不高。如果子问题之间有大量重叠（如斐波那契数列），则效率会急剧下降。

1.2.1.4 动态规划 (Dynamic Programming)

思路：动态规划是一种特殊的**分治递归**。当我们用分治法将大问题递归分解成更小的子问题来求解时，如果子问题之间大量重叠（同一个子问题会被多次独立求解），就会造成严重的重复计算。动态规划通过**存储已经计算过的子问题结果**，避免同一个子问题被重复计算，从而大幅提高效率。

一句话原理：“记住已经算过的答案，避免重复计算”——用一点空间换取大量的时间。

对于例 1-11 的斐波那契数列问题，采用朴素的分治递归计算 $F(5)$ 的递归过程会形成下面这样的树形结构（如 @fig-F5_recursion.jpg 所示）：

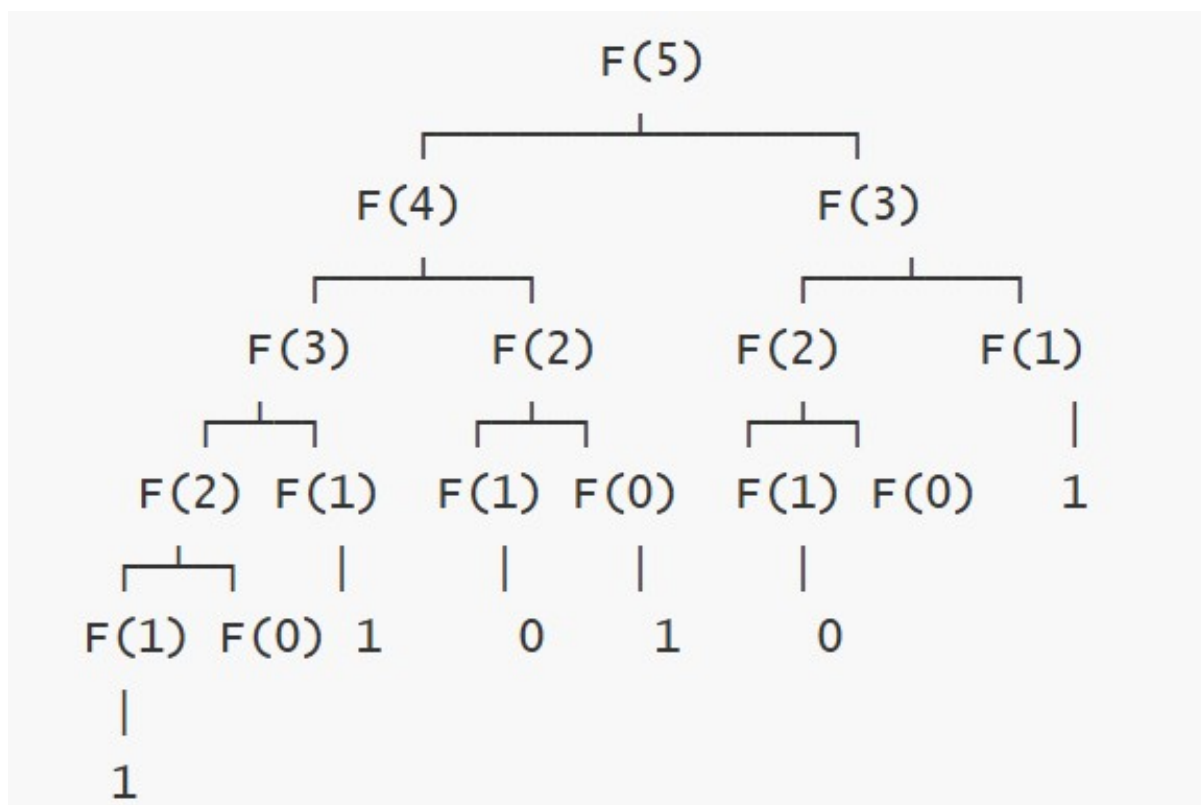
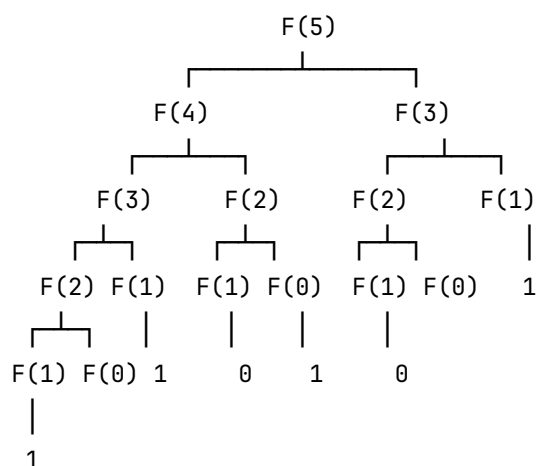


Figure 1.1: 斐波那契数列数 $F(5)$ 的朴素递归计算过程



可见： $F(3)$ 被算了 2 次， $F(2)$ 被算了 3 次， $F(1)$ 被算了 5 次。这种大量重复正是因为子问题高度重叠，导致时间复杂度约为 $O(1.618^n)$ ， n 稍大就极慢。

斐波那契数列的动态规划思路

既然子问题会反复出现，我们就把每个子问题的结果**保存下来**，下次遇到时直接查表使用，不再重新计算。

方式一：从大问题开始，边算边记（自顶向下 + 记忆化）

忆化技术通过存储已计算的中间结果，避免了递归树中的重复计算。Figure 1.2 展示了采用记忆化方式计算 $F(5)$ 的过程：每个斐波那契数首次出现时被递归求解并存入“备忘录”，后续再次需要时（如 $F(3)$ 和 $F(2)$ ）直接查表返回，不再展开子问题。这一机制使得每个子问题仅计算一次，将指数级的时间复杂度优化至线性，大幅提升效率。

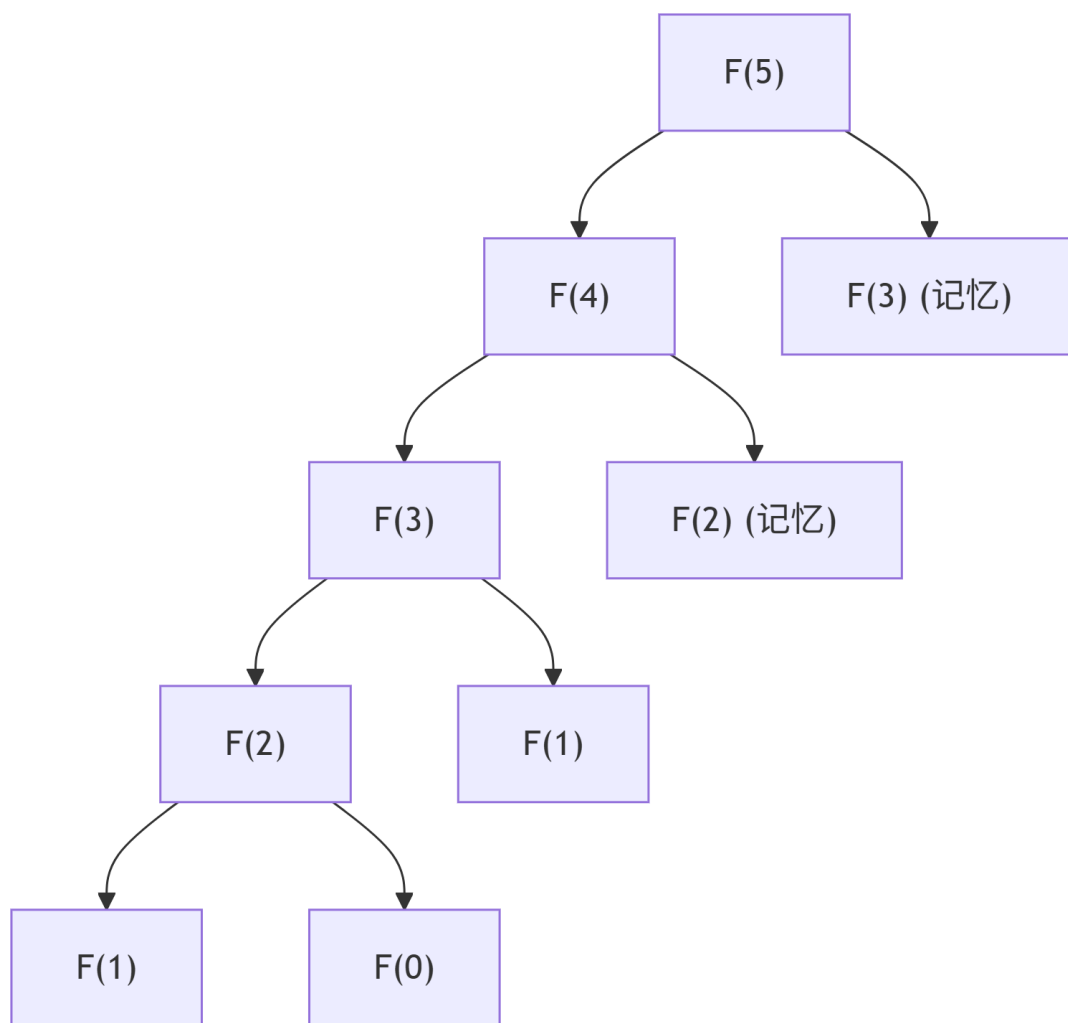


Figure 1.2: 斐波那契数列数 $F(5)$ 的记忆化递归计算过程

方式二：从小到大一步步填表（自底向上递推）

直接从最小的子问题 $F(0)$ 和 $F(1)$ 开始，依次计算更大的值，直到求出 $F(n)$ 。

填表过程示意（ $n=5$ 示例）：

text

i	0	1	2	3	4	5
值	0	1	1	2	3	5
	↑	↑	↑	↑	↑	↑
	起点	起点	1+0	1+1	2+1	3+2

每一步都只依赖前面已经算好的结果，没有任何重复计算。

小结：

动态规划特别适合那些“子问题重叠严重”的问题。通过“保存中间结果”，它把指数级的重复计算变成了每个子问题只算一次，大大提高了效率。斐波那契数列是最简单、最直观的例子，帮助我们初步理解动态规划“记住答案”的核心思想。

(后续章节会介绍更复杂的动态规划问题，如背包问题、最长公共子序列等。)

1.2.1.5 小结：如何选择设计策略？

面对一个新问题，可以按以下顺序思考：

1. 规模很小？→ 先试暴力法
2. 每步有明显的“最好选择”？→ 尝试贪心（需证明）
3. 能把问题分成独立的小问题？→ 分治
4. 有重叠子问题 + 最优子结构？→ 动态规划

后续章节将围绕这些策略逐一深入讲解，还会介绍更多高级方法和搜索类策略。

通过这些简单例子，你可以看到：同一个问题往往有多种解法，关键在于选对策略。这正是算法设计的魅力所在。

1.2.2 算法的表示与实现

设计出算法后，需要将其清晰地表达出来，以便交流、分析和最终在计算机上运行。算法的表示（描述）方式有多种，从非形式化的自然语言到形式化的程序语言，各有适用场景。实现则是将算法描述转换为可执行程序的过程。

1.2.2.1 自然语言

直接用日常语言描述算法的步骤。这种方式通俗易懂，无需专业知识即可理解，适合描述算法的高层思想或用于非技术交流。

示例（求一组数中的最大值）：

给定一组数，先假设第一个数是最大值。然后依次与后面的每个数比较，如果遇到更大的数，则更新最大值。最后输出这个最大值。

优点：容易理解，无需学习专门符号。

缺点：容易产生歧义，对于复杂算法描述冗长，难以精确表达分支、循环等控制结构。

1.2.2.2 流程图

用图形符号直观展示算法的执行流程。常用符号包括：起止框（椭圆形）、处理框（矩形）、判断框（菱形）、流程线（箭头）等。

示例（求一组数的最大值）：Figure 1.3

优点：直观、形象，能清晰展现分支和循环。

缺点：绘制和修改较麻烦，对于大规模算法流程图过于庞大，且缺乏精确的语义定义。

1.2.2.3 伪代码

一种介于自然语言和编程语言之间的描述方式，通常混合使用数学符号、英语关键词和结构化编程元素（如 if-else、for、while）。伪代码简洁、清晰，且与具体编程语言无关，是算法教材中最常用的表示方法。

示例（求一组数的最大值）：

```
Algorithm ArrayMax(A[0..n-1])
    // 返回数组 A 中的最大值
    max ← A[0]
    for i ← 1 to n-1 do
        if A[i] > max then
            max ← A[i]
        end if
    end for
    return max
end Algorithm
```

优点：

简洁，突出算法逻辑，避免语言细节；

易于阅读和修改；

可直接作为编写程序的基础。

缺点：没有统一的标准，不同作者风格可能略有差异。

1.2.2.4 程序语言

直接用某种高级语言（如 C、C++、Java、Python）编写算法。这种方式下的算法可以直接在计算机上运行验证。

示例（C 语言实现）：

```
#include <stdio.h>

int findMaximum(int arr[], int n) {
```

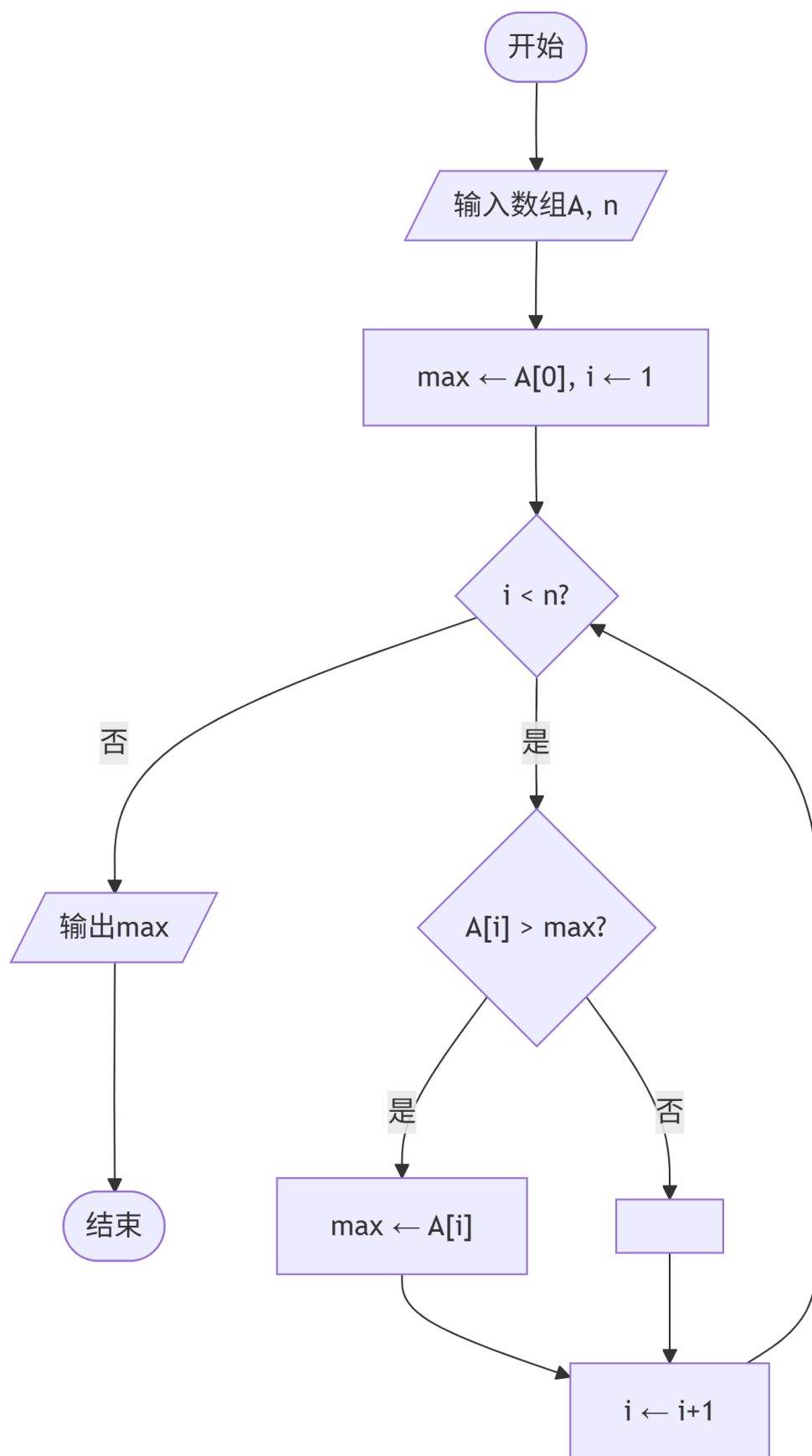


Figure 1.3: 求一组数的最大值的流程图

```

    if (n <= 0) {
        return -1; // 表示无效 (数组为空)
    }

    int max_val = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max_val) {
            max_val = arr[i];
        }
    }
    return max_val;
}

int main() {
    int arr1[] = {7, 3, 9, 2, 8, 1};
    int n1 = sizeof(arr1) / sizeof(arr1[0]);
    int res1 = findMaximum(arr1, n1);
    if (res1 != -1) printf(" 数组 1 最大值: %d\n", res1);
    else printf(" 数组 1 为空, 返回 -1\n");

    int arr2[] = {5};
    int n2 = sizeof(arr2) / sizeof(arr2[0]);
    int res2 = findMaximum(arr2, n2);
    if (res2 != -1) printf(" 数组 2 最大值: %d\n", res2);

    int arr3[] = {};
    int n3 = sizeof(arr3) / sizeof(arr3[0]);
    int res3 = findMaximum(arr3, n3);
    if (res3 != -1) printf(" 数组 3 最大值: %d\n", res3);
    else printf(" 数组 3 为空, 返回 -1\n");

    return 0;
}

```

示例 (C++ 实现):

```

#include <iostream>
#include <vector>
#include <stdexcept>

using namespace std;

int findMaximum(const vector<int>& arr) {

```

```

    if (arr.empty()) {
        throw invalid_argument(" 数组不能为空");
        // 或者 return INT_MIN;  // 如果不想抛异常
    }

    int max_val = arr[0];
    for (size_t i = 1; i < arr.size(); ++i) {
        if (arr[i] > max_val) {
            max_val = arr[i];
        }
    }
    return max_val;
}

int main() {
    try {
        vector<int> arr1 = {7, 3, 9, 2, 8, 1};
        cout << " 数组 1 最大值: " << findMaximum(arr1) << endl;

        vector<int> arr2 = {5};
        cout << " 数组 2 最大值: " << findMaximum(arr2) << endl;

        vector<int> arr3 = {};
        cout << findMaximum(arr3) << endl;  // 会抛异常
    } catch (const invalid_argument& e) {
        cerr << " 错误: " << e.what() << endl;
    }

    return 0;
}

```

示例 (Java 实现):

```

public class FindMaxExample {

    public static int findMaximum(int[] arr) {
        if (arr == null || arr.length == 0) {
            throw new IllegalArgumentException(" 数组不能为空");
        }

        int maxVal = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > maxVal) {

```

```

        maxVal = arr[i];
    }
}

return maxVal;
}

public static void main(String[] args) {
    try {
        int[] arr1 = {7, 3, 9, 2, 8, 1};
        System.out.println(" 数组 1 最大值: " + findMaximum(arr1));

        int[] arr2 = {5};
        System.out.println(" 数组 2 最大值: " + findMaximum(arr2));

        int[] arr3 = {};
        System.out.println(findMaximum(arr3)); // 抛异常
    } catch (IllegalArgumentException e) {
        System.err.println(" 错误: " + e.getMessage());
    }
}
}

```

示例 (Python 实现):

```

def find_maximum(arr):
    if not arr:
        raise ValueError(" 列表不能为空")
        # 或者 return None # 如果不想抛异常

    max_val = arr[0]
    for i in range(1, len(arr)):
        if arr[i] > max_val:
            max_val = arr[i]
    return max_val

if __name__ == "__main__":
    try:
        arr1 = [7, 3, 9, 2, 8, 1]
        print(f" 数组 1 最大值: {find_maximum(arr1)}")

        arr2 = [5]
        print(f" 数组 2 最大值: {find_maximum(arr2)}")
    
```



```

arr3 = []
print(find_maximum(arr3)) # 抛异常
except ValueError as e:
    print(f" 错误: {e}")

```

优点:

- 精确、可执行，能直接在计算机上运行并通过大量测试用例验证正确性
- 可以测量实际运行时间、内存占用，评估真实性能
- 便于调试、集成到实际项目中

缺点:

- 受具体语言语法约束，容易混杂语言特有的细节（如指针、内存管理、异常处理、垃圾回收等），可能掩盖算法的核心本质
- 不同语言的实现细节差异较大（如数组越界处理、类型系统、性能优化方式），不利于跨语言交流和算法本质的抽象描述
- 对于初学者，语言本身的语法负担可能干扰对算法逻辑的理解

1.2.2.5 递归算法的表示示例

上述例子展示了迭代算法的表示。对于递归算法，伪代码和程序语言同样适用。下面以计算阶乘为例，展示递归算法的描述。

例 1.14. 递归计算 “n 的阶乘

给定非负整数 n ，计算 $n! = n \times (n-1) \times \dots \times 1$ ，其中 $0! = 1$

伪代码表示:

```

factorial(n)
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)

```

Python 实现:

```

def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)

```

C++ 实现:

```

#include <iostream>

long long factorial(int n) {

```

```

if (n == 0) {
    return 1;
} else {
    return n * factorial(n - 1);
}
}

```

说明:

- 使用 long long 类型以支持较大 n 的阶乘（避免 int 溢出）。
- main 函数提供交互式输入，便于运行测试。
- 递归算法的表示清晰地体现了“问题分解”的思想，是后续学习递归算法的基础。

例 1.15. 汉诺塔的递归求解

汉诺塔（Tower of Hanoi）是一个著名的递归问题：有三根柱子 A、B、C。A 柱上有 n 个大小不同的圆盘（从上到下由小到大），要求将所有圆盘从 A 移动到 C，规则是每次只能移动一个盘子，且大盘子不能放在小盘子上（如 Figure 1.4 所示）。

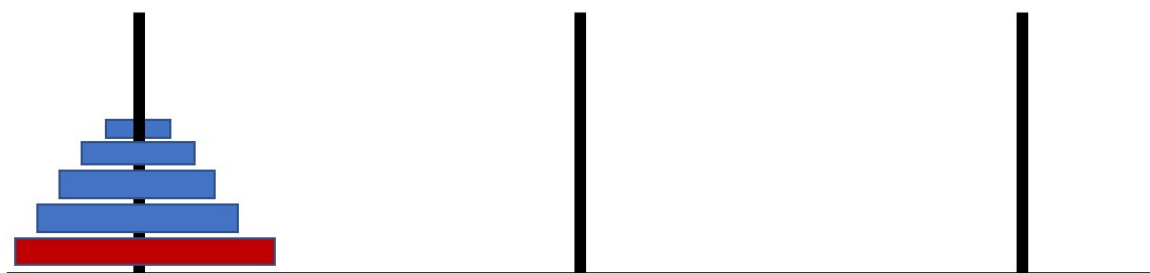


Figure 1.4: 汉诺塔问题

分治递归思想：汉诺塔问题无法直接一步解决，但可以利用**分治**策略将大问题分解为更小的同类子问题。具体来说，要把 n 个盘子从 A 移到 C，只需解决三个规模更小的子问题：

- 把 n-1 个盘子从 A 移到 B（借助 C）
- 把第 n 个盘子从 A 移到 C
- 把 n-1 个盘子从 B 移到 C（借助 A）

这三个子问题中，前后两个是完全相同的子任务（移动 n-1 个盘子），只是起始柱和目标柱不同。通过递归调用自身来解决这些子问题，最终当 n=1 时（只有一个盘子），直接移动即可结束递归。

这种“把大问题分解成更小的相同问题 → 递归求解 → 合并结果”的模式，正是分治递归的核心，也是汉诺塔被视为递归算法经典例子的原因。

自然语言描述：要将 n 个盘子从 A 移到 C，可分为三步：

1. 将上面的 n-1 个盘子从 A 移到 B（借助 C）
2. 将第 n 个盘子从 A 直接移到 C
3. 将 B 上的 n-1 个盘子从 B 移到 C（借助 A）

当 $n = 1$ 时，直接移动即可。

递归算法的递归求解过程，如图 1-5 所示

流程图（文本描述）：

可用如 Figure 1.5 的递归算法求解汉诺塔问题。

伪代码表示：

```
hanoi(n, src, aux, dst)
    if n == 1:
        print("Move disk 1 from " + src + " to " + dst)
    else:
        hanoi(n-1, src, dst, aux)
        print("Move disk " + n + " from " + src + " to " + dst)
        hanoi(n-1, aux, src, dst)
```

Python 实现（带移动打印）：

```
def hanoi(n, src="A", aux="B", dst="C"):
    if n == 1:
        print(f"Move disk 1 from {src} to {dst}")
    else:
        hanoi(n-1, src, dst, aux)
        print(f"Move disk {n} from {src} to {dst}")
        hanoi(n-1, aux, src, dst)

# 测试 n=3
hanoi(3)
```

C++ 实现：

```
#include <iostream>

void hanoi(int n, char src = 'A', char aux = 'B', char dst = 'C') {
    if (n == 1) {
        std::cout << "Move disk 1 from " << src << " to " << dst << std::endl;
    } else {
        hanoi(n-1, src, dst, aux);
        std::cout << "Move disk " << n << " from " << src << " to " << dst << std::endl;
        hanoi(n-1, aux, src, dst);
    }
}

int main() {
    hanoi(3);
    return 0;
}
```

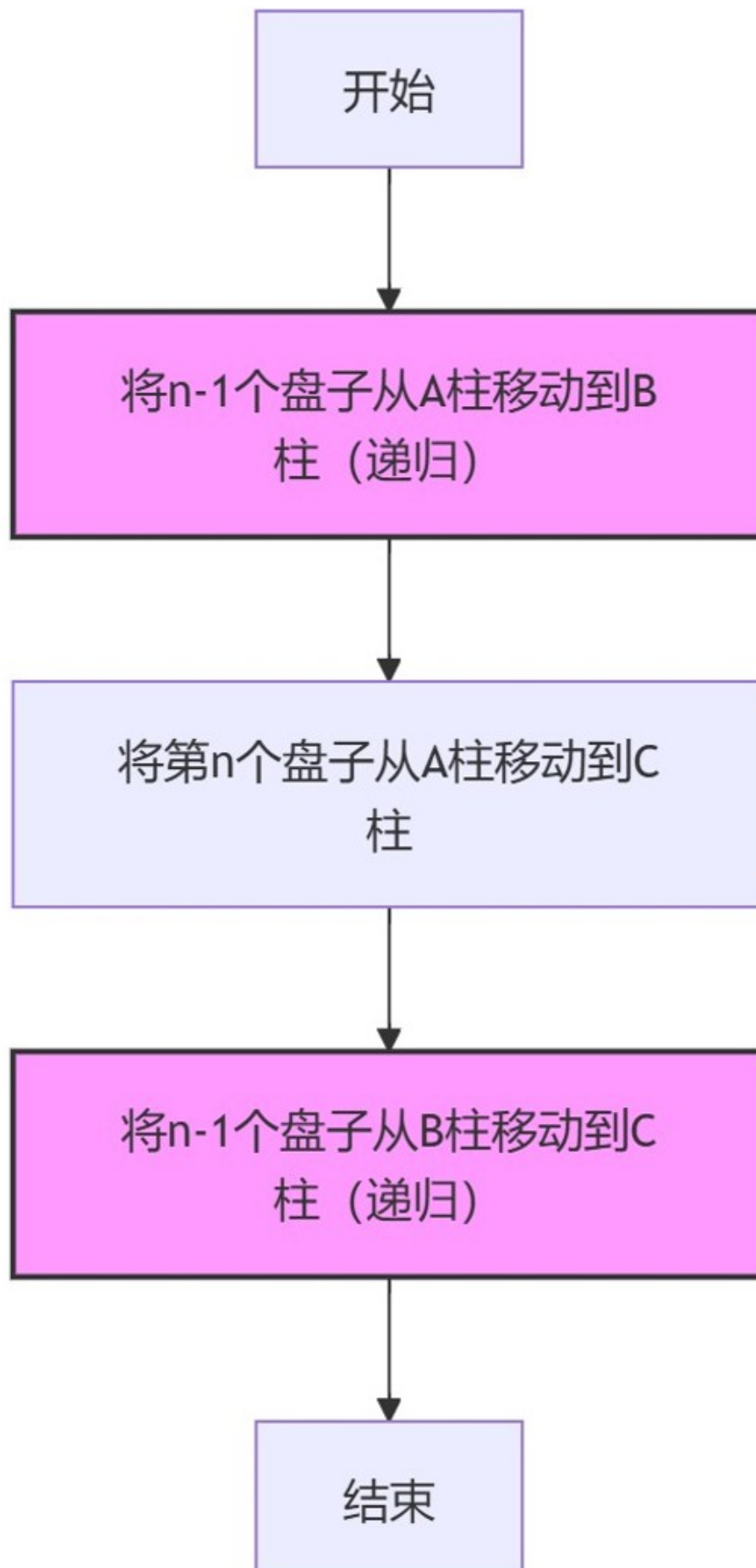


Figure 1.5: Flowchart of the recursive solution for the Tower of Hanoi problem

```
}
```

通过汉诺塔，我们可以看到递归不仅能处理线性递减问题，还能优雅地解决“分而治之”的复杂分解问题。

1.3 算法的分析

算法分析的主要目的是评估一个算法是否“好”。所谓“好”，至少包括两个方面：

1. **算法是否正确** —— 是否对所有合法输入都能输出正确结果；
2. **算法是否高效** —— 在时间和空间资源上是否可接受。

一个不正确的算法毫无意义；一个正确但效率极低的算法，在大规模输入下同样无法使用。因此，算法分析包含两个核心内容：

- 正确性分析
- 效率分析（时间复杂度与空间复杂度）

1.3.1 正确性分析

算法正确性是算法设计的基础。正确性证明的目标是说明：

对任意合法输入，算法都能在有限步内终止，并产生符合问题要求的输出。

常见证明方法包括：

- 循环不变量法（用于迭代算法）
- 数学归纳法（用于递归算法）
- 反证法（用于下界证明或最优性证明）
- 直接证明法

本节重点介绍前三种。

1.3.1.1 循环不变量法

循环不变量法是证明迭代算法正确性的标准方法。

核心思想

找到一个在循环执行过程中始终成立的性质（称为**不变量**），并通过三个步骤证明：

1. **初始化**：循环开始前，不变量成立；
2. **保持**：若某次迭代开始时不变量成立，则本次迭代结束后仍成立；
3. **终止**：循环结束时，不变量 + 终止条件 $\square$ 得到正确结果。

例 1.16. 插入排序的正确性

插入排序思想：

从第二个元素开始，将当前元素插入到前面已排序部分的合适位置。

循环不变量：

每次外层循环开始时，子数组 $A[0..i-1]$ 已按非降序排列。

三步证明：

初始化

当 $i=1$ 时， $A[0]$ 只有一个元素，自然有序。

保持

假设 $i=k$ 时 $A[0..k-1]$ 有序。现在插入 $A[k]$ ，通过从后向前比较并后移元素，把 $A[k]$ 插入到正确位置后， $A[0..k]$ 仍然有序（插入位置前后的元素保持相对顺序）。

终止

当 $i=n$ 时， $A[0..n-1]$ 有序，排序完成。

因此插入排序正确。

1.3.1.2 数学归纳法

最常用于证明递归算法或递推关系的正确性。

证明步骤：

- **基情况：**证明 $n=$ 最小值时正确。
- **归纳假设：**假设 $n=k$ 时正确。
- **归纳步骤：**证明 $n=k+1$ 时正确（利用 k 的假设）。

例 1.17. 阶乘递归计算的正确性

算法： $\text{fact}(n) = \text{if } n=0 \text{ then } 1 \text{ else } n \times \text{fact}(n-1)$

证明： $\text{fact}(n)$ 总是返回 $n!$

证明其正确计算 $n!$ ：

- **基情况：** $n=0$, $\text{fact}(0)=1 = 0!$ ，正确。
- **归纳假设：**假设 $n=k$ 时 $\text{fact}(k) = k!$ 。
- **归纳步骤：** $n=k+1$ 时， $\text{fact}(k+1) = (k+1) \times \text{fact}(k) = (k+1) \times k! = (k+1)!$ ，正确。

因此算法对所有 $n \geq 0$ 正确。

1.3.1.3 反证法

假设算法不正确，推导出矛盾，从而证明假设错误。常用于证明贪心算法的最优性或某些算法的正确性。

反证法常用于：

- 证明算法的最优性
- 证明复杂度下界

例 1.18. 线性查找的最坏复杂度

求证：线性查找在最坏情况下必须比较 n 次。

问题：在一组 n 个元素中查找目标值。线性查找做法：从头到尾逐个比较。

问题：给定一个包含 n 个元素的数组 $A[0..n-1]$ ，要查找目标值 x 是否存在（或首次出现的位置）。线性查找的做法是从头到尾逐个比较：

```
for i = 0 to n-1:
    if A[i] == x:
        return i
return -1 (或 “不存在”)
```

定理：在最坏情况下，任何正确的线性查找算法都必须进行至少 n 次元素比较。

证明（反证法）：

假设存在一个正确的算法 Alg，在最坏情况下最多只进行 $n-1$ 次比较，就能正确返回 x 是否存在以及（若存在）其位置。

我们考虑两种典型的“最坏输入”：

情形 1：x 位于数组最后一个位置 ($A[n-1] = x$)

- 如果 Alg 只比较了前 $n-1$ 个元素 ($A[0]$ 到 $A[n-2]$)，没有比较 $A[n-1]$ ，
- 那么 Alg 看到前 $n-1$ 个元素都不等于 x ，就会得出“ x 不存在”或“位置在前 $n-1$ 个中”的结论，
- 但实际上 x 就在 $A[n-1]$ ，所以输出必然错误。→ 与“Alg 是正确的”矛盾。

情形 2：x 在整个数组中都不存在

- 如果 Alg 只比较了前 $n-1$ 个元素，没有比较 $A[n-1]$ ，
- 那么 Alg 无法知道 $A[n-1]$ 是否等于 x ，
- 如果它输出“ x 不存在”，但万一 $A[n-1] == x$ （虽然这次输入没有，但算法必须对所有可能输入正确），就会错；
- 如果它输出“ x 存在于某位置”，但实际上不存在，也会错。→ 无论输出什么，都无法保证正确 → 与“Alg 是正确的”矛盾。

因此：无论 x 存在还是不存在，只要算法在某些输入下只比较了 $n-1$ 次，就必然存在某种输入让它出错。故任何正确的线性查找算法，在最坏情况下都必须至少进行 n 次比较。

证毕。

1.3.1.4 其他方法

- **直接证明**：一步步推导算法输出与预期一致。
- **归纳 + 不变量结合**：递归算法常用。
- **模型检查 / 测试**：实际运行大量测试用例（但不是严格证明）。

正确性证明是理论计算机科学的重要内容，本课程会逐步引入，不要求一开始就掌握所有方法。

1.3.2 效率分析

效率分析关注算法在实际运行中的资源消耗，主要包括：

- **时间复杂度**：算法执行基本操作（如比较、赋值、加法）的次数随输入规模 n 的增长趋势。
- **空间复杂度**：算法运行时需要的额外内存空间（不包括输入本身）。

其中时间复杂度最重要。

1.3.2.1 为什么要分析效率？

解决同一个问题往往有多种算法，它们的效率可能天差地别。下面通过几个简单例子直观感受这一点。

问题 1：计算矩形周长

已知长 a 、宽 b ，周长公式为 $2(a+b)$ 。

算法 A：直接计算 $2 \times (a+b)$ ；

算法 B：计算 $2a+2b$ 。

算法 A 只需一次加法和一次乘法，算法 B 需两次乘法和一次加法。在计算机中乘法通常比加法慢，因此算法 A 略优。

问题 2：求 1 到 n 的和

问题：计算 $1+2+3+\dots+n$

方法一：循环累加

```
sum = 0
for i = 1 to n:
    sum += i
```

该算法需要执行 n 次加法，操作次数随 n 线性增长。为了定量描述这种增长，我们引入**时间复杂度**的概念。时间复杂度用函数 $T(n)$ 表示算法执行基本操作的总次数，其中 n 为输入规模。本例中， $T(n) = n$ ，用渐近记号可表示为 $O(n)$ （读作“大 O n”），称为**线性阶**，意指算法执行时间与 n 成正比或者说不超过 n 的常数倍。实际上， $T(n)$ 的增长速度与 n 完全相同，因此更精确地可以用 $\Theta(n)$ （读作“大 Theta n”）表示，这将在 1.4 节详细讨论。

方法二：利用高斯公式

直接计算 $\frac{n(n+1)}{2}$ ，只需一次乘法、一次加法和一次除法。无论 n 多大，操作次数固定，因此时间复杂度为 $O(1)$ ，称为**常数阶**（表示基本操作执行次数是常数，与问题规模无关）。当 $n = 10^6$ 时，方法一需百万次加法，方法二仅需常数次运算，**效率天壤之别**。

问题 3：有序数组中的查找

假设有一个已排序的数组，要查找某个数是否存在。线性查找（从头到尾比较）在最坏情况下需比较 n 次，时间复杂度 $O(n)$ 。而二分查找每次将搜索区间减半，最坏只需比较 $\log_2 n$ 次，时间复杂度 $O(\log n)$ 。当 $n = 10^6$ 时，线性查找最多百万次，二分查找最多 20 次，**效率差异巨大**。

问题 4：斐波那契数计算

计算第 n 个斐波那契数。朴素递归版本： $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ ，其递归树呈指数膨胀，时间复杂度约为 $O(1.618^n)$ ，当 $n=40$ 时已需数亿次调用；而迭代版本从 $F(0), F(1)$ 开始逐步递推，仅需 $O(n)$ 次加法， $n = 40$ 时只需约 40 次加法。这个例子揭示了**算法设计（避免重复计算）对效率的决定性作用**。

以上例子说明：**算法的选择直接影响程序运行速度，甚至决定问题能否在可接受时间内解决**。因此，我们必须学会分析算法的效率，以便在实际应用中选择或设计最合适的算法。

1.3.2.2 时间复杂度分析

时间复杂度的定义：设算法处理输入规模为 n 的问题时，执行**基本操作**的总次数记为 $T(n)$ ，称为该算法的**时间复杂度**。 $T(n)$ 是输入规模 n 的函数，它刻画了算法运行时间随 n 增长的变化趋势。

- 输入规模 n 根据问题定义（例如数组元素个数、矩阵阶数、数的位数等）。
- 基本操作的选择应能反映算法的主要耗时（如比较、赋值、加法等），且次数易于统计。
- $T(n)$ 通常指**最坏情况**下的操作次数，因为最坏情况给出了算法性能的保证；但在分析时也可明确区分最好、最坏和平均情况。

基本操作的选择：不同算法关注的基本操作不同。例如，排序算法常将“比较”作为基本操作；矩阵乘法常将“乘法”作为基本操作；遍历算法常将“访问元素”作为基本操作。选择基本操作的原则是：该操作占算法总运行时间的主导地位，且次数易于统计。

分析要点：

- **最坏情况**：最不利的输入下的复杂度（最常用，确保算法在任何情况下都不会超过程度）。
- **平均情况**：所有可能输入的期望复杂度（需要知道输入的概率分布）。
- **最好情况**：最有利输入下的复杂度（参考意义较小，但有时可用于优化）。

分析步骤（通用流程）：

1. 确定输入规模 n ，并选择合适的基本操作。
2. 分析算法结构：循环、递归、条件分支等，统计基本操作的执行次数与 n 的函数关系 $T(n)$ 。
3. 简化表达：忽略低阶项和常数因子，用渐近记号（如 O, Ω, Θ ）表示 $T(n)$ 的增长趋势（详见 1.4 节）。

4. 比较不同算法：根据渐近复杂度评估算法的优劣。

例 1-19：数组求和

例 1.19. 数组求和

问题：计算数组 $A[0..n-1]$ 所有元素的和。

算法：

```
sum = 0
for i = 0 to n-1
    sum = sum + A[i]
return sum
```

基本操作：加法（或赋值加）。

- **操作次数：**循环执行 n 次，每次做一次加法，共 n 次加法， $n+1$ 次赋值（包括初始化）。忽略常数， $T(n)=n$ 。
- **复杂度：**无论数组内容如何，循环次数固定为 n ，所以最好、最坏、平均情况均为 $\Theta(n)$ 。
- **空间复杂度：**只需变量 sum 和 i ， $O(1)$ 。

例 1.20. 冒泡排序

问题：将数组 $A[0..n-1]$ 按升序排列。

算法（未优化版本）：

```
for i = 0 to n-2
    for j = 0 to n-2-i
        if A[j] > A[j+1]
            swap(A[j], A[j+1])
```

基本操作的选择：主要关注比较 ($A[j] > A[j+1]$) 和交换 (swap)。

- 比较次数固定为 $\frac{n(n-1)}{2}$ ，与输入顺序无关。
- 交换次数取决于初始顺序：在最坏情况（逆序）下，每次内层循环都发生交换，总交换次数也接近 $\frac{n(n-1)}{2}$ ；在最好情况（已有序）下，交换次数为 0。

操作次数分析：总比较次数 $\sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}$ ，最坏情况下交换次数也接近 $\frac{n(n-1)}{2}$ 。在实际运行中，交换操作的代价通常高于比较（涉及多次内存读写和赋值），因此最坏情况下的总时间开销仍由 $\frac{n(n-1)}{2}$ 阶主导。

时间复杂度：

- 最坏情况（逆序）：比较 + 交换均 $\Theta(n^2)$ ，时间复杂度 $\Theta(n^2)$ 。
- 最好情况（已有序）：比较仍 $\Theta(n^2)$ ，但交换为 0；若加入“提前终止”优化（用标志位判断本轮是否有交换），最好情况可降至 $\Theta(n)$ 。
- 平均情况：比较 $\Theta(n^2)$ ，交换平均约 $\frac{n(n-1)}{4}$ ，总体仍 $\Theta(n^2)$ 。

空间复杂度：仅需常数个临时变量用于交换， $O(1)$ 。

例 1.21. 二分查找

问题：在有序数组 $A[0..n-1]$ 中查找目标值 x ，返回其下标或 -1。

算法：

```
left = 0, right = n-1
while left <= right
    mid = (left + right) / 2
    if A[mid] == x
        return mid
    else if A[mid] < x
        left = mid + 1
    else
        right = mid - 1
return -1
```

- **基本操作：**比较 ($A[mid]$ 与 x)。
- **操作次数分析：**每次循环将搜索区间减半。最坏情况下，循环持续到区间为空，比较次数等于 $\lfloor \log_2 n \rfloor + 1$ 。
- **时间复杂度：**
 - 最坏情况： $\Theta(\log n)$ 。
 - 最好情况：第一次比较就找到目标， $\Theta(1)$ 。
 - 平均情况： $\Theta(\log n)$ 。
- **空间复杂度：**只用了几个变量， $O(1)$ 。

例 1.22. 递归斐波那契数

问题：计算第 n 个斐波那契数 $F(n)$ ，其中 $F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$ 。

算法（朴素递归）：

```
fib(n)
    if n <= 1
        return n
    else
        return fib(n-1) + fib(n-2)
```

- **基本操作：**加法（也可统计递归调用次数）。
- **操作次数分析：**设 $T(n)$ 为计算 $F(n)$ 所需的加法次数。
递推关系： $T(0) = 0, T(1) = 0, T(n) = T(n-1) + T(n-2) + 1$ 。
解此递推可得 $T(n) = \Theta(\phi^n)$ ，其中 $\phi \approx 1.618$ 。
- **时间复杂度：** $T(n) = \Theta(\phi^n)$ ，指数级。
- **空间复杂度：**递归调用栈深度为 n ，因此额外空间 $T(n) = \Theta(\phi^n)$ 。

1.3.2.3 空间复杂度分析

空间复杂度衡量算法在运行过程中临时占用的存储空间，通常不包括输入数据本身占用的空间。主要考虑：

- **固定空间：**与输入规模无关的变量、常量等，如循环计数器、临时变量。
- **可变空间：**随输入规模变化的空间，如动态分配的数组、递归调用栈。

前面例子的空间复杂度：

- 数组求和： $O(1)$
- 冒泡排序： $O(1)$
- 二分查找： $O(1)$
- 递归斐波那契： $\Theta(n)$ （递归栈）

注意：时间与空间有时可以相互权衡，例如使用更多空间缓存中间结果（动态规划）可以大幅降低时间复杂度。

1.3.2.4 小结

效率分析是算法设计的核心环节之一。通过分析时间复杂度和空间复杂度，我们可以：

- 预测算法在不同规模输入下的表现；
- 比较不同算法的优劣，选择最适合的算法；
- 发现算法的瓶颈，为优化提供方向。

在接下来的章节中，我们将系统学习渐近记号（1.4 节）、常见复杂度的计算（1.5 节），并结合具体算法深入分析其效率。

1.4 渐近分析

在 1.3 节中，我们通过统计基本操作次数，得到了算法的精确运行时间函数 $T(n)$ 。例如，冒泡排序（未优化版本）的比较次数为：

$$T(n) = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

这个精确表达式包含了二次项、一次项和常数系数。然而，当输入规模 n 变得非常大时（如 $n = 10^6$ 或更大），**二次项 $\frac{1}{2}n^2$ 的增长速度远超其他项**，一次项和常数的影响变得微不足道。

渐近分析正是关注这种“当 $n \rightarrow \infty$ 时算法运行时间的增长趋势”，它忽略机器性能、编程语言、常数因子、低阶项等次要细节，只保留主导增长的部分，从而在宏观上比较不同算法的优劣。

因此，对于上面的冒泡排序，我们可以说其时间复杂度为 $O(n^2)$ ，意思是：算法的运行时间最多以 n^2 的速度增长（忽略常数和低阶项后）。这种描述方式让我们能够轻松比较冒泡排序（ $O(n^2)$ ）、归并排序（ $O(n \log n)$ ）、插入排序（最坏 $O(n^2)$ ，最好 $O(n)$ ）等算法的效率差异。

渐近分析来源于数学中的“渐近线”概念，它描述函数在趋于无穷时的极限行为。例如：

- $f(n) = n^2 + 3n + 1000$ ：当 n 很大时，主导项是 n^2
- $g(n) = 500n + 20000$ ：线性增长，远慢于 n^2

通过这种方式，我们可以用简洁的符号（如 O 、 Ω 、 Θ ）刻画算法的增长率，而不必纠结精确的系数和低阶项。

1.4.1 渐近分析的基本思想

渐近分析（asymptotic analysis）研究当输入规模

$$n \rightarrow \infty$$

时，算法时间或空间开销的极限行为。它来源于数学中的“渐近线”概念，即描述函数在趋于无穷时的变化趋势。

例如，考虑函数

$$f(n) = n^2 + 3n$$

当 n 变得非常大时，与 n^2 相比， $3n$ 项可以忽略不计，因此 $f(n)$ 的增长趋势与 n^2 相同。

又如， $1000n + 3000$ 和 $0.6n^2$

相比，当 n 足够大时，前者远小于后者。如 Figure 1.6 所示：

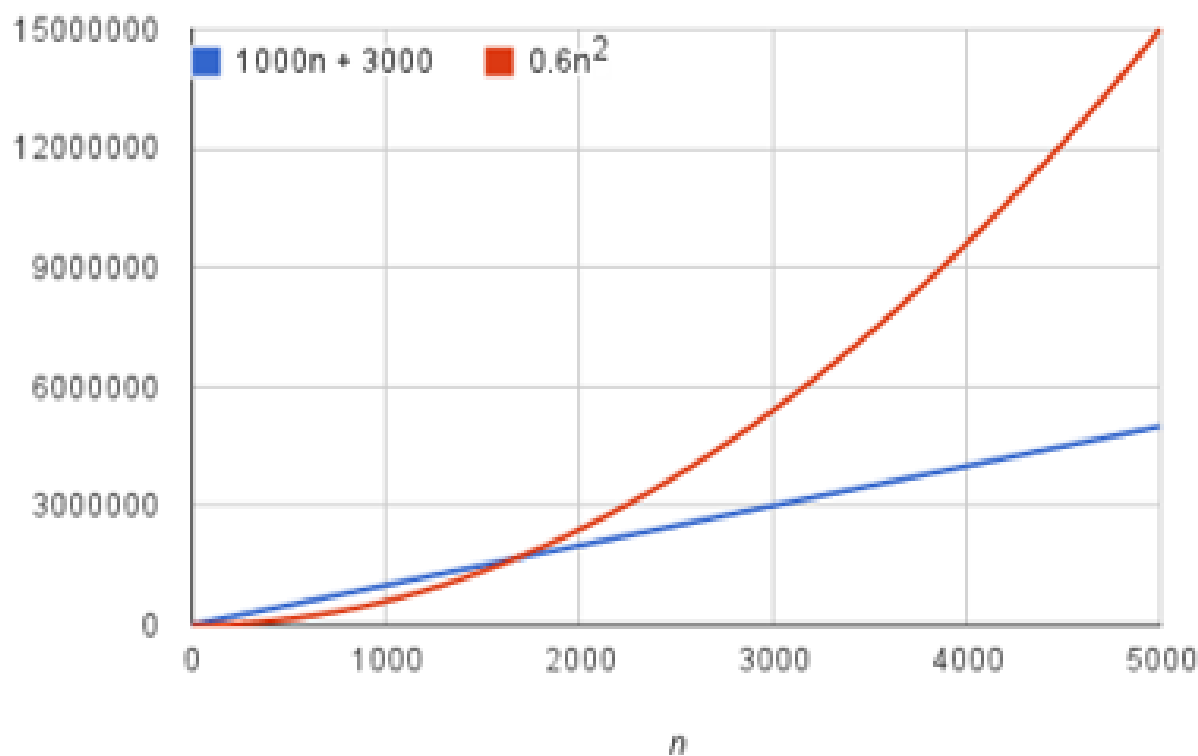


Figure 1.6: $1000n + 3000$ 和 $0.6n^2$ 的增长曲线

渐近分析通过去掉常数系数和低阶项，专注于算法的增长率，使我们能够用简洁的符号刻画不同算法的效率。

1.4.2 渐近记号

设算法的时间复杂度函数为 $T(n)$ ，其中 n 为输入规模。我们引入五个常用的渐近记号：大 O 、大 Ω 、大 Θ 、小 o 、小 ω 。其中前三个最为常用。

1.4.2.1 大 O 记号（上界）

定义：

如果存在正常数 c 和 n_0 ，使得对于所有 $n \geq n_0$ ，都有

$$0 \leq T(n) \leq c \cdot f(n),$$

则记作 $T(n) = O(f(n))$ ，读作“ $T(n)$ 是 $f(n)$ 的大 O ”。

大 O 给出算法时间复杂度的**上界**，即算法在最坏情况下不会比 $f(n)$ 增长更快。

例：

- 冒泡排序：

$$T(n) = \frac{n(n-1)}{2} = O(n^2)$$

因为 $\frac{n(n-1)}{2} \leq n^2$ 对所有 $n \geq 1$ 成立。

- 二分查找：

$$T(n) = \lfloor \log_2 n \rfloor + 1 = O(\log n)$$

- 线性查找：

$$T(n) = n = O(n)$$

注意：

上界可以放宽，例如 $n = O(n^2)$ 也成立，但通常取**最紧上界**。

例 1-23：多项式的上界

例 1.23. 多项式的上界

设

$$T(n) = a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0,$$

其中 $a_k \neq 0$ 。

取 $n_0 = 1$, 令

$$a^* = \max_i |a_i|$$

则

$$T(n) \leq (k+1)a^*n^k$$

取 $c = (k+1)a^*$, 得

$$T(n) = O(n^k)$$

即: 所有 k 次多项式的上界是 $O(n^k)$ 。

1.4.2.2 大 Ω 记号 (下界)

定义:

如果存在正常数 c 和 n_0 , 使得对所有 $n \geq n_0$,

$$0 \leq c \cdot f(n) \leq T(n),$$

则记作

$$T(n) = \Omega(f(n)).$$

大 Ω 给出时间复杂度的**下界**。

例:

- 冒泡排序:

$$T(n) = \frac{n(n-1)}{2} \geq \frac{n^2}{4} \quad (n \geq 2)$$

因此

$$T(n) = \Omega(n^2)$$

- 二分查找:

$$T(n) = \lfloor \log_2 n \rfloor + 1 \geq \log_2 n$$

所以

$$T(n) = \Omega(\log n)$$

- 线性查找（最坏情况）：

$$T(n) = \Omega(n)$$

1.4.2.3 大 Θ 记号（紧确界）

定义：

如果存在正常数 c_1, c_2 和 n_0 ，使得对所有 $n \geq n_0$ ，

$$0 \leq c_1 f(n) \leq T(n) \leq c_2 f(n),$$

则记作 $T(n) = \Theta(f(n))$. 表示 $T(n)$ 与 $f(n)$ 同阶增长。

例：

- 数组求和：

$$T(n) = n = \Theta(n)$$

- 冒泡排序：

$$T(n) = \frac{n(n-1)}{2} = \Theta(n^2)$$

- 二分查找：

$$T(n) = \lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$$

- 递归斐波那契：

$$T(n) = \Theta(\varphi^n)$$

$$\varphi \approx 1.618$$

其中

$$\varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

为黄金比例。

由于 $\varphi < 2$ ，因此也可给出一个较为粗略的上界

$$T(n) = O(2^n)$$

其时间复杂度的推导过程将在下一节“1.5 递归方程”的求解中详细讲解。

1.4.2.4 极限判定定理

若

$$\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = c \quad (c > 0),$$

则

$$T(n) = \Theta(f(n)).$$

例如：

$$\lim_{n \rightarrow \infty} \frac{n^2/3 - 4n}{n^2} = 1/3$$

因此

$$n^2/3 - 4n = \Theta(n^2)$$

该极限判定定理的证明并不复杂，读者可作为练习自行完成。

1.4.2.5 小 o 记号（非紧上界）

若对任意 $c > 0$ ，存在 n_0 使得

$$0 \leq T(n) < cf(n)$$

则

$$T(n) = o(f(n))$$

等价于

$$\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = 0$$

例如：

$$5n^2 + 9n = o(n^3)$$

1.4.2.6 小 ω 记号（非紧下界）

若对任意 $c > 0$ ，存在 n_0 使得

$$0 \leq cf(n) < T(n)$$

则

$$T(n) = \omega(f(n))$$

等价于

$$\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = \infty$$

例如：

$$n^2 = \omega(n)$$

1.4.2.7 记号之间的关系

- $T(n) = \Theta(f(n))$ 当且仅当 $T(n) = O(f(n))$ 且 $T(n) = \Omega(f(n))$
- $T(n) = O(f(n))$ 等价于 $f(n) = \Omega(T(n))$
- $T(n) = o(f(n))$ 等价于 $f(n) = \omega(T(n))$

通常我们更关注最坏情况下的上界，因此大 O 最常用。

1.4.3 渐近记号的性质

1.4.3.1 传递性

若 $f = O(g)$ 且 $g = O(h)$ ，则 $f = O(h)$ 。

同理适用于 Ω 、 Θ 、 o 、 ω 。

1.4.3.2 自反性

$$f = O(f), \quad f = \Omega(f), \quad f = \Theta(f)$$

1.4.3.3 对称性

$$f = \Theta(g) \iff g = \Theta(f)$$

1.4.3.4 加法规则

若

$$T_1(n) = O(f(n)), \quad T_2(n) = O(g(n))$$

则

$$T_1(n) + T_2(n) = O(\max(f(n), g(n)))$$

总复杂度由最高阶项决定。

1.4.3.5 乘法规则

若

$$T_1(n) = O(f(n)), \quad T_2(n) = O(g(n))$$

则

$$T_1(n) \cdot T_2(n) = O(f(n)g(n))$$

1.4.4 常见时间复杂度类别

以下表格列出了算法分析中最常见的几种时间复杂度类别，按增长速度从慢到快排列：

记号	名称	增长趋势简述	典型场景示例
$\Theta(1)$	常数阶	时间固定，与 n 无关	数组下标访问、哈希表平均查找

记号	名称	增长趋势简述	典型场景示例
$\Theta(\log n)$	对数阶	每次操作规模减半，增长极慢	二分查找、有序集合查找
$\Theta(n)$	线性阶	时间与 n 成正比	遍历数组、求最大值、线性查找
$\Theta(n \log n)$	线性对数阶	高效的分治算法常见	归并排序、快速排序平均情况、堆排序
$\Theta(n^2)$	平方阶	双重循环常见， n 较大时已明显变慢	冒泡排序、插入排序最坏情况、简单选择排序
$\Theta(2^n)$	指数阶	规模稍大即爆炸，通常只适用于小 n	汉诺塔、子集枚举、暴力解背包问题
$\Theta(n!)$	阶乘阶	增长最极端，仅限极小规模	全排列生成、旅行商问题（TSP）暴力解

Figure 1.7 直观地给出了几类常见时间复杂度函数的增长趋势。

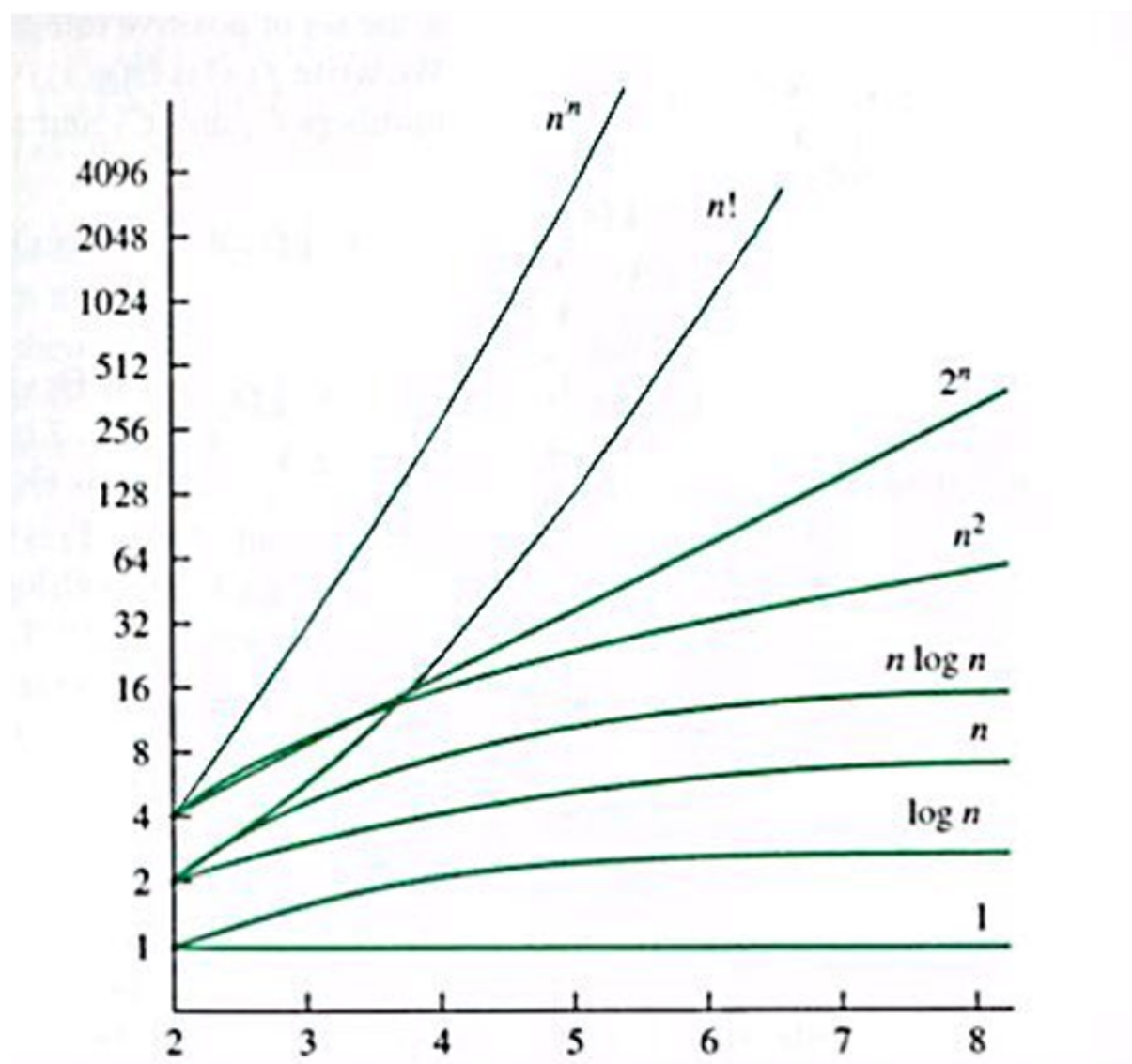


Figure 1.7: 常见时间复杂度函数

1.4.5 常见时间复杂度函数的典型例子

1.4.5.1 $\Theta(1)$ 常数阶

无论数组有多大，访问第 k 个元素总是 1 次操作。

```
get_element(arr, k)
    return arr[k] // 永远 1 次访问
```

1.4.5.2 $\Theta(\log n)$ 对数阶

二分查找每次把搜索范围减半，最坏只需 $\log_2 n$ 次比较。

```
binary_search(arr, target)
    left = 0, right = n-1
    while left ≤ right
        mid = (left + right) / 2
        if arr[mid] == target
            return mid
        if arr[mid] < target
            left = mid + 1
        else
            right = mid - 1
    return -1
```

1.4.5.3 $\Theta(n)$ 线性阶

求一组数中的最大值，需要遍历所有元素一次。

```
find_max(arr, n)
    max_val = arr[0]
    for i = 1 to n-1
        if arr[i] > max_val
            max_val = arr[i]
    return max_val
```

1.4.5.4 $\Theta(n \log n)$ 线性对数阶

归并排序将数组不断二分，然后合并有序子数组，总时间 $n \log n$ 。（伪代码略，后续章节详述）

1.4.5.5 $\Theta(n^2)$ 平方阶

前面的冒泡排序的在最坏情况下的交换操作的时间复杂度是 ($O(n^2)$)

再如，插入排序最坏情况下，每插入一个元素都要与前面元素比较，最多约 n^2 次。

```
insertion_sort(arr, n)
    for i = 1 to n-1
        key = arr[i]
        j = i - 1
        while j ≥ 0 and arr[j] > key
            arr[j+1] = arr[j]
```

```

    j = j - 1
    arr[j+1] = key

```

1.4.5.6 $\Theta(2^n)$ 指数阶

汉诺塔移动 n 个盘子需要 $2^n - 1$ 次移动, $n = 20$ 时已超百万。

```

hanoi(n, src, aux, dst)
    if n == 1
        move 1 from src to dst
    else
        hanoi(n-1, src, dst, aux)
        move n from src to dst
        hanoi(n-1, aux, src, dst)

```

1.4.5.7 $\Theta(n!)$ 阶乘阶

生成 n 个元素的全排列需要 $n!$ 次操作, $n = 10$ 已有 362 万种, $n = 15$ 超万亿。(暴力枚举所有排列)

1.4.5.8 小结

这些复杂度级别形成了从“极快”到“不可行”的清晰梯度。理解它们能帮助你快速判断一个算法是否适合实际问题规模：

- $n \leq 10^6$ 时, $O(n \log n)$ 通常很安全；
- $O(n^2)$ 在 $n \approx 10^4$ 时已开始变慢；
- 指数级和阶乘级只适用于 $n \leq 20 \sim 30$ 的极小规模。

通过这些例子, 你可以更直观地感受不同复杂度在实际运行中的差异。

1.4.6 渐近分析的限制性

尽管渐近分析是算法效率评价的核心工具, 但在实际应用中需注意以下几点：

- **常数系数可能影响实际性能** 两个算法可能具有相同的渐近复杂度 (如 $O(n)$), 但它们的常数系数可能相差悬殊。例如, $100n$ 和 $2n$ 在 n 很大时都呈线性增长, 但后者在同等条件下快 50 倍。因此, 当渐近阶数相同时, 必须考虑常数系数的影响。
- **低阶项在小规模输入时可能占主导**

渐近分析关注的是 $n \rightarrow \infty$ 时的极限行为, 但实际问题的输入规模未必总是很大。

示例：

- 算法 A (低阶但常数大)： $f(n) = n + 100$ —— 复杂度 $O(n)$
- 算法 B (高阶但常数小)： $f(n) = n^2$ —— 复杂度 $O(n^2)$

对比分析：

- 当 $n = 5$ 时: A 耗时 105, B 耗时 25。此时 $O(n^2)$ 的算法反而比 $O(n)$ 快。
- 只有当 $n > 10.5$ (跨越平衡点) 后, $O(n)$ 的优势才会显现。

这说明在处理极小规模数据 (如短字符串处理、小集合搜索) 时, 盲目追求低阶渐近复杂度的算法可能并不明智。

• 最坏情况与平均情况的差异

渐近分析通常默认讨论**最坏情况 (Worst-case)**, 但某些算法在实际运行中的**平均表现 (Average-case)** 远优于其最坏情况 (如快速排序)。反之, 有些算法 (如归并排序) 的最坏与平均情况高度一致。在进行算法选型时, 应明确当前分析的是哪种场景。

• 隐藏的硬件与实现细节

理论分析中的“基本操作”在物理设备上的执行时间并不均等。现代计算机的**内存访问速度**、**缓存命中率 (Cache Hit)**、**分支预测**等底层机制, 可能导致理论复杂度相同的算法在实际运行中表现迥异。

结论: 渐近分析提供的是算法效率的**宏观增长趋势**。在实际的算法选择与工程优化中, 还需结合**问题规模**、**常数因子**、**硬件环境**以及**数据的分布特征**进行综合权衡。

1.4.7 问题复杂性类简介 (拓展)

本节之前讨论的是**算法**的时间复杂度, 即给定算法后分析其随输入规模增长的效率。而计算复杂性理论从更宏观的视角, 根据问题本身是否能在多项式时间内求解, 将问题划分为不同的**复杂性类**。以下是几个基本概念:

- **P 类问题:** 存在多项式时间算法 (如 $O(n^k)$) 的问题。例如排序、最短路径问题。
- **NP 类问题:** 解可以在多项式时间内被验证的问题。例如, 给定一个数是否合数, 若有人给出因子, 可快速验证; 旅行商问题中给定一条路径, 可快速验证其长度是否不超过某值。
- **NP 完全问题:** NP 中最难的问题, 且所有 NP 问题都可以在多项式时间内归约到它。如果找到任一 NP 完全问题的多项式时间算法, 则 $P = NP$ 。例如 SAT 问题、哈密顿回路问题。
- **NP 难问题:** 至少与 NP 完全问题一样难, 但不一定属于 NP (即解可能无法在多项式时间内验证)。例如旅行商问题的优化版本。

著名的 P 与 NP 是否相等问题是计算机科学领域的重大开放问题, 也是克雷数学研究所“千禧年大奖难题”之一。本书将在后续章节 (如第 X 章) 进一步探讨这些概念。

1.4.8 小结

渐近分析通过 O 、 Ω 、 Θ 等记号刻画算法增长趋势, 使我们忽略次要因素, 从宏观上比较算法优劣。

对于递归算法, 其时间复杂度的分析通常要求解递归方程。例如, 前面提到的斐波那契数列朴素递归算法, 其时间复杂度满足递归式 $T(n) = T(n-1) + T(n-2) + 1$, 解此方程才能得到确切的指数阶。下一节将系统介绍递归方程的求解方法, 包括迭代展开、递归树、换元、假设归纳以及主定理等。

1.5 递归方程的求解

在分析递归算法的时间复杂度时，我们经常会遇到递归方程（recurrence equation）。递归方程通过将原问题规模为 n 的时间函数 $T(n)$ 表示为更小规模问题的时间函数加上一些额外的代价来描述算法的运行时间。例如，归并排序的时间复杂度可以表示为

$$T(n) = 2T(n/2) + O(n), \quad T(1) = O(1).$$

求解递归方程就是要找出 $T(n)$ 的显式表达式或其渐近阶。本节将介绍几种常用的递归方程求解方法：迭代展开、递归树、换元迭代、假设归纳、高阶方程化简以及主定理。

1.5.1 迭代展开法

迭代展开法（iteration method）是最直接的方法，它通过反复将递归式代入自身，直到到达初始条件，然后将得到的表达式求和。

例 1.24. 求解递归方程

$$T(n) = 2T(n-1) + 1, \quad T(1) = 1.$$

解：逐次展开：

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2(2T(n-2) + 1) + 1 = 2^2T(n-2) + 2 + 1 \\ &= 2^2(2T(n-3) + 1) + 2 + 1 = 2^3T(n-3) + 2^2 + 2 + 1 \\ &\vdots \\ &= 2^{n-1}T(1) + (2^{n-2} + 2^{n-3} + \dots + 2 + 1) \\ &= 2^{n-1} \cdot 1 + (2^{n-1} - 1) = 2^n - 1. \end{aligned}$$

因此 $T(n) = 2^n - 1 = \Theta(2^n)$ 。

注意，在展开过程中我们使用了等比数列求和公式

$$1 + 2 + 2^2 + \dots + 2^{n-2} = 2^{n-1} - 1.$$

1.5.2 递归树法

递归树法（recursion tree method）是迭代展开的图形化表示。它将递归式的每一层展开为树形结构，每层的代价之和即为总代价。这种方法直观且易于理解。

例 1.25. 求解递归方程

$$T(n) = 2T(n/2) + n - 1, \quad T(1) = 0.$$

根据递归公式，问题规模为 n 的时间复杂度 $T(n)$ 由 2 个问题规模为 $n/2$ 的子问题的时间复杂度 $T(n/2)$ 和分解与合并子问题需要的代价 $n - 1$ 构成，可表示为如 Figure 1.8 的递归树：

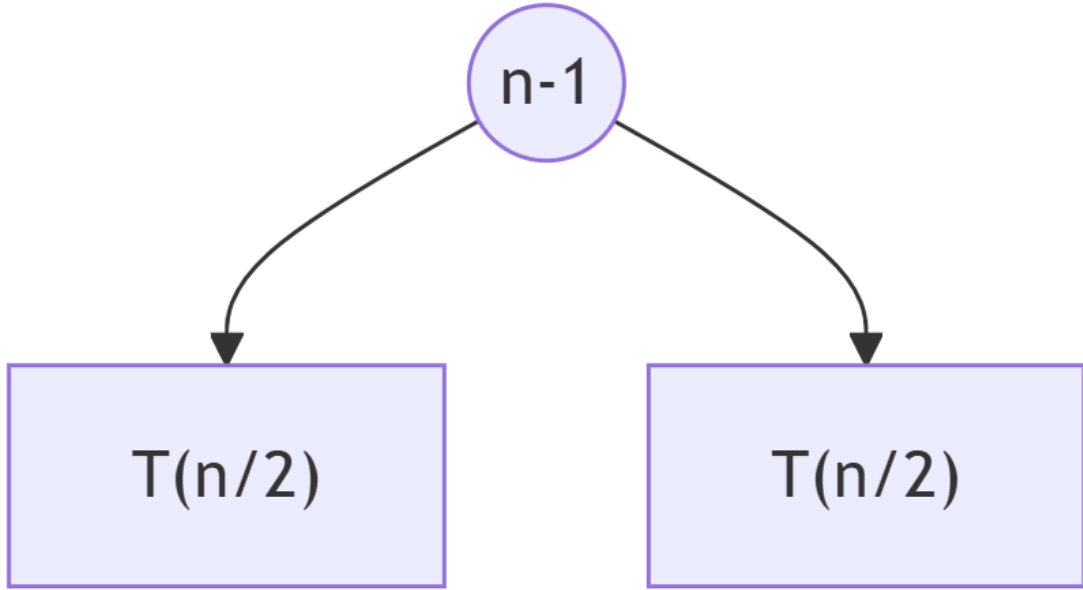


Figure 1.8

图 1-8 $T(n)$ 时间复杂度表示为代价为 $n - 1$ 的根节点和 2 个问题规模为 $n/2$ 子节点的时间复杂度 $T(n/2)$

同样，对于问题规模为 $T(n/2)$ （对应上图的 2 个子节点），每个 $T(n/2)$ 又可以表示为代价为 $n/2 - 1$ 和 2 个问题规模为 $n/4$ 子节点的时间复杂度 $T(n/4)$ 。如 @fig-recursive_tree-2 所示。

对于问题规模为 $T(n/4)$ （对应上图的 4 个子节点），每个 $T(n/4)$ 又可以表示为代价为 $n/4 - 1$ 和 2 个问题规模为 $n/8$ 子节点的时间复杂度 $T(n/8)$ 。以此类推，直到规模为 1 的结点代价为 0。下图展示了继续展开的部分结构（深层用省略号表示，实际叶子节点数量为 n ，代价为 0）。如 @fig-recursive_tree-3

树的高度为 $\log_2 n$ ，第 i 层 ($i = 0, 1, \dots, \log_2 n - 1$) 有 2^i 个结点，每个结点代价为 $n/2^i - 1$ 。因此总代价为

$$\begin{aligned}
 T(n) &= \sum_{i=0}^{\log_2 n - 1} 2^i \left(\frac{n}{2^i} - 1 \right) + 0 \\
 &= \sum_{i=0}^{\log_2 n - 1} (n - 2^i) = n \log_2 n - (2^{\log_2 n} - 1) \\
 &= n \log_2 n - n + 1 = \Theta(n \log n).
 \end{aligned}$$

例 1.26. 求解递归方程： $T(n) = 3T(n/2) + n$

根据递归公式，问题规模为 n 的时间复杂度 $T(n)$ 由 3 个问题规模为 $n/2$ 的子问题的代价 $T(n/2)$ 和分解与合并子问题需要的代价 n 构成，可表示为如下的递归树 (Figure 1.11)：

同样，对于问题规模为 $T(n/2)$ （对应上图的 3 个子节点），每个 $T(n/2)$ 又可以表示为代价为 $n/2$ 和 3 个问题规模为 $n/4$ 子节点的时间复杂度 $T(n/4)$ 。

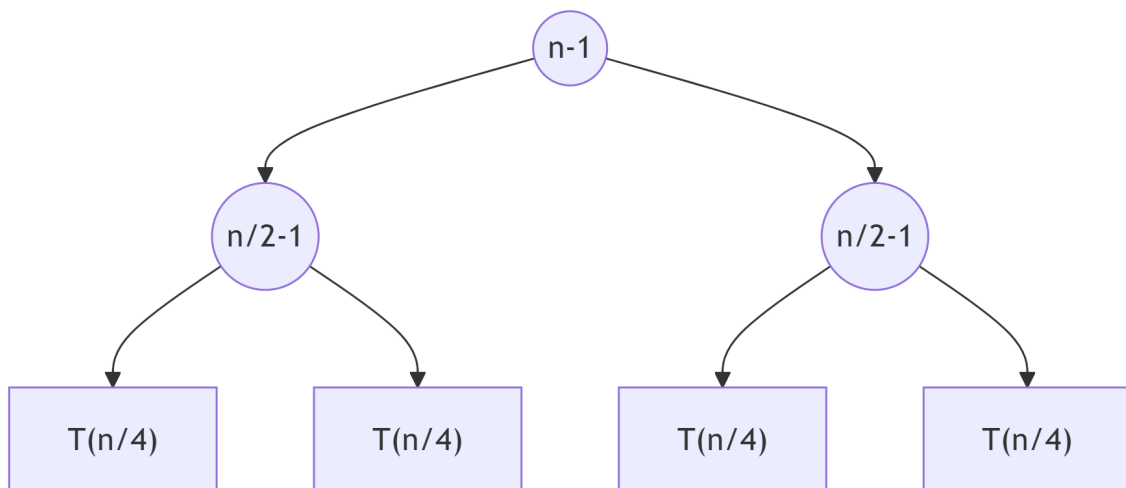


Figure 1.9: 将 $T(n/2)$ 进一步展开, 得到第 2 层代价节点和 $T(n/4)$ 子节点

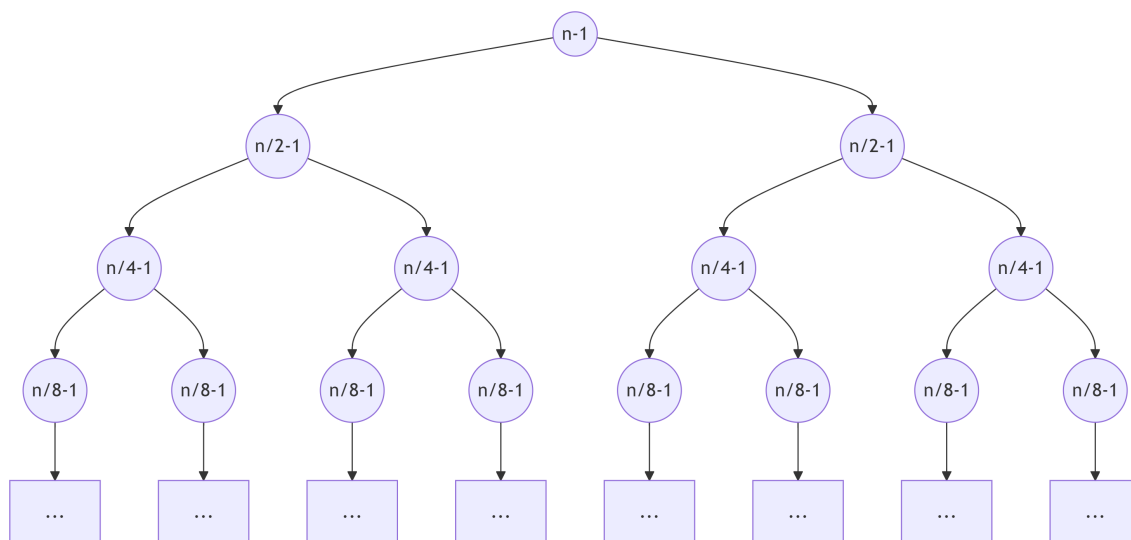


Figure 1.10: 递归树的完整结构 (部分展开), 圆圈内为合并代价, 矩形为子问题的 $T(\cdot)$, 最底层叶子代价为 0

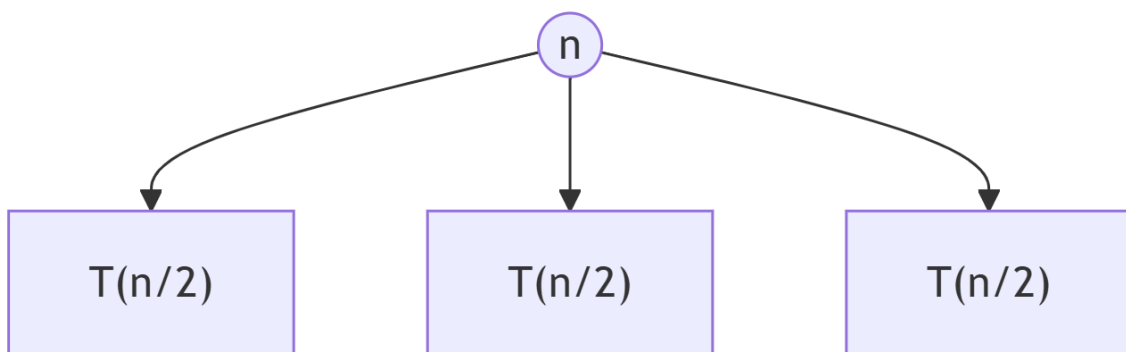


Figure 1.11: $T(n)$ 时间复杂度表示为代价为 n 的根节点和 3 个问题规模为 $n/2$ 子节点的时间复杂度 $T(n/2)$

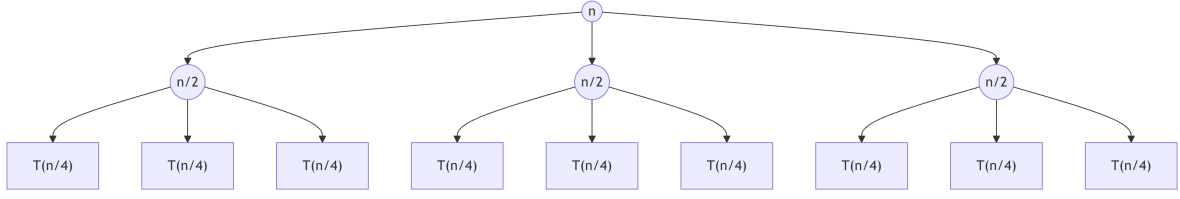


Figure 1.12: 将 $T(n/2)$ 进一步展开, 得到第 2 层代价节点和 $T(n/4)$ 子节点

对于问题规模为 $T(n/4)$ (对应上图的 9 个子节点), 每个 $T(n/4)$ 又可以表示为代价为 $n/4$ 和 3 个问题规模为 $n/8$ 子节点的时间复杂度 $T(n/8)$ 。以此类推, 直到规模为 1 的结点代价为 0。下图展示了继续展开的部分结构 (深层用省略号表示, 实际叶子节点数量为 $3^{\log_2 n} = n^{\log_2 3}$, 每个叶子代价为 0)。

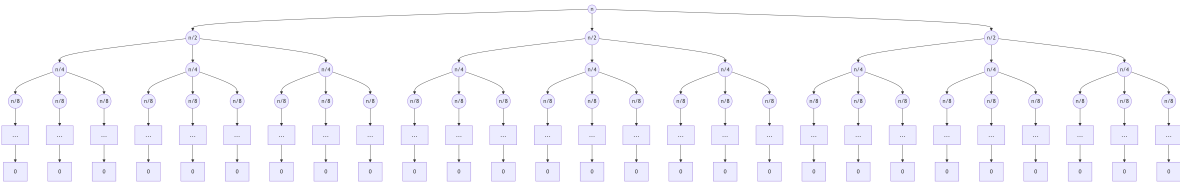


Figure 1.13: 递归树的完整结构 (部分展开), 圆圈内为合并代价, 矩形为子问题的 $T(\cdot)$, 最底层叶子代价为 0

树的高度为 $\log_2 n$, 第 i 层 ($i = 0, 1, \dots, \log_2 n - 1$) 有 3^i 个结点, 每个结点代价为 $n/2^i$ 。因此总代价为

$$\begin{aligned}
 T(n) &= \sum_{i=0}^{\log_2 n - 1} 3^i \cdot \frac{n}{2^i} + 0 \\
 &= n \sum_{i=0}^{\log_2 n - 1} \left(\frac{3}{2}\right)^i \\
 &= n \cdot \frac{(3/2)^{\log_2 n} - 1}{(3/2) - 1} \\
 &= \Theta\left(n \left(\frac{3}{2}\right)^{\log_2 n}\right) \\
 &= \Theta(3^{\log_2 n}) \\
 &= \Theta(n^{\log_2 3})
 \end{aligned}$$

故 $T(n) = \Theta(n^{\log_2 3})$ 。

例 1.27. 求解递归方程: $T(n) = 3T(n/2) + n^2$

类似地, 第 i 层有 3^i 个结点, 每个结点代价 $(n/2^i)^2 = n^2/4^i$, 该层总代价为 $(3/4)^i n^2$ 。总代价为

$$T(n) = \sum_{i=0}^{\log_2 n} \left(\frac{3}{4}\right)^i n^2.$$

由于 $3/4 < 1$, 该等比级数收敛到一个常数, 因此

$$T(n) = \Theta(n^2).$$

1.5.3 换元迭代法

当递归式中出现 n/b 或 $n/2$ 等形式时, 可以通过变量代换将方程转化为更易处理的形式。通常令 $n = b^k$ 或 $n = 2^k$ 将规模化为整数指数。

例 1.28. 求解递归方程: $T(n) = 2T(n/2) + n - 1, T(1) = 0$

解答: 令 $n = 2^k$, 则 $T(2^k) = 2T(2^{k-1}) + 2^k - 1$ 。再令 $S(k) = T(2^k)$, 得到

$$S(k) = 2S(k-1) + 2^k - 1.$$

迭代展开 $S(k)$:

$$\begin{aligned} S(k) &= 2S(k-1) + 2^k - 1 \\ &= 2(2S(k-2) + 2^{k-1} - 1) + 2^k - 1 = 2^2S(k-2) + 2 \cdot 2^{k-1} + 2^k - (2 + 1) \\ &= 2^2S(k-2) + 2^k + 2^k - (2 + 1) \\ &= \dots \\ &= 2^k S(0) + k \cdot 2^k - (2^{k-1} + 2^{k-2} + \dots + 2 + 1) \\ &= 2^k \cdot 0 + k2^k - (2^k - 1) = k2^k - 2^k + 1. \end{aligned}$$

将 $k = \log_2 n$ 代回, 得

$$T(n) = n \log_2 n - n + 1 = \Theta(n \log n).$$

1.5.4 假设归纳法

假设归纳法 (guess-and-prove) 先根据直觉或少量展开猜测解的形式, 然后用数学归纳法严格证明。

例 1.29. 证明 $T(n) = 2T(n/2) + n - 1, T(1) = 0$ **的解为** $\Theta(n \log n)$

猜测:

观察递归式的形式, 它与归并排序的递归式 $T(n) = 2T(n/2) + n$ 非常相似, 而我们知道归并排序的时间复杂度为 $\Theta(n \log n)$ 。此外, 通过递归树法分析 (见例 1-25) 可以发现, 每层的代价之和约为 n , 树高为 $\log_2 n$, 因此总代价应约为 $n \log n$ 。基于这些直观认识, 我们猜测 $T(n) = \Theta(n \log n)$ 。

证明: 先证上界 $T(n) \leq cn \log_2 n$ 。取 $c = 1$, 假设对所有小于 n 的正整数 k 有 $T(k) \leq ck \log_2 k$ 。则

$$\begin{aligned} T(n) &= 2T(n/2) + n - 1 \\ &\leq 2 \left(\frac{n}{2} \log_2 \frac{n}{2} \right) + n - 1 \\ &= n(\log_2 n - 1) + n - 1 \\ &= n \log_2 n - 1 \leq n \log_2 n. \end{aligned}$$

归纳基础 $n = 1$ 时 $T(1) = 0 \leq 1 \cdot 1 \log_2 1 = 0$ 成立。因此 $T(n) = O(n \log n)$ 。

下界证明类似（取 $c = 1/2$ 等），可得 $T(n) = \Omega(n \log n)$ ，故 $T(n) = \Theta(n \log n)$ 。

例 1.30. 求解递归方程

对于形如

$$T(n) \leq T(n/5) + T(7n/10) + an$$

的递归式，猜测 $T(n) = O(n)$ 。

常数 c 的选取分析：为了用归纳法证明 $T(n) \leq cn$ ，我们需要找到适当的常数 c 使得归纳步骤成立。假设对所有小于 n 的规模有 $T(m) \leq cm$ ，代入递归式得：

$$T(n) \leq c \frac{n}{5} + c \frac{7n}{10} + an = \left(\frac{c}{5} + \frac{7c}{10} \right) n + an = \frac{9c}{10} n + an.$$

我们希望上式不超过 cn ，即

$$\frac{9c}{10} n + an \leq cn \quad \Rightarrow \quad an \leq \left(c - \frac{9c}{10} \right) n = \frac{c}{10} n \quad \Rightarrow \quad a \leq \frac{c}{10} \quad \Rightarrow \quad c \geq 10a.$$

此外，归纳基础要求 $T(1) = 1 \leq c \cdot 1$ ，因此还需 $c \geq 1$ 。综合两者，取 $c = \max\{10a, 1\}$ 即可满足条件。这样，我们就通过归纳法证明了 $T(n) = O(n)$ 。

严格的数学归纳法证明：

取常数 $c \geq \max\{10a, 1\}$ ，假设对所有小于 n 的规模有 $T(m) \leq cm$ 。则

$$\begin{aligned} T(n) &\leq c \frac{n}{5} + c \frac{7n}{10} + an = \frac{9c}{10} n + an \\ &\leq \frac{9c}{10} n + \frac{c}{10} n = cn. \end{aligned}$$

归纳基础 $T(1) = 1 \leq c \cdot 1$ 成立，故 $T(n) = O(n)$ 。

1.5.4.1 吸收常数技巧 (Absorbing Constants)

在利用代入法 (substitution method) 证明递归式的渐近上界时，递归式中常常会出现 **向上取整、向下取整或常数项**。这些常数在归纳推导中可能产生额外的加法项，从而使得直接证明

$$T(n) \leq Cf(n)$$

变得困难。

为了解决这一问题，通常在归纳假设中引入一个常数项，证明

$$T(n) \leq Cf(n) - d$$

其中 $d > 0$ 为常数。由于

$$Cf(n) - d \leq Cf(n)$$

因此一旦证明

$$T(n) \leq Cf(n) - d$$

便可立即得到

$$T(n) = O(f(n)).$$

这种通过 负常数抵消递归推导中产生的正的常数项的方法称为 吸收常数技巧。

例 1.31. 线性递归式的证明

考虑递归式

$$T(n) \leq T(n/5) + T(7n/10) + an$$

其中 $a > 0$ 为常数。下面证明

$$T(n) = O(n).$$

证明

我们用强归纳法证明存在常数 $C > 0, d > 0$, 使

$$T(n) \leq Cn - d$$

对充分大的 n 成立。

归纳假设

假设对所有 $k < n$ 有

$$T(k) \leq Ck - d.$$

归纳步骤

由递归关系

$$T(n) \leq T(n/5) + T(7n/10) + an$$

根据归纳假设

$$T(n/5) \leq C(n/5) - d$$

于是

$$T(n) \leq C(n/5) + C(7n/10) + an - 2d.$$

整理得

$$T(n) \leq \left(\frac{9}{10}C + a\right)n - 2d.$$

常数选择

取

$$C > 10a$$

则

$$\frac{9}{10}C + a < C.$$

因此

$$T(n) \leq Cn - 2d.$$

只要再取

$$d > 0$$

即可得到

$$T(n) \leq Cn - d$$

对充分大的 n 成立。

结论

因此

$$T(n) = O(n).$$

证毕。

1.5.5 高阶方程的化简

某些递归方程可以转化为一阶差分方程求解。例如形如 $T(n) = 2T(n-1) + 1$ 的方程可通过引入新变量化为等比数列。

例 1.32. 快速排序的时间复杂度

快速排序的平均情况分析得到如下方程：

$$T(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + n - 1, \quad T(0) = 0.$$

为求解此方程，两边乘以 n ：

$$nT(n) = 2 \sum_{k=0}^{n-1} T(k) + n(n-1).$$

对 $n-1$ 写出类似等式：

$$(n-1)T(n-1) = 2 \sum_{k=0}^{n-2} T(k) + (n-1)(n-2).$$

两式相减消去求和项：

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2n - 2,$$

整理得

$$nT(n) = (n+1)T(n-1) + 2n - 2.$$

两边除以 $n(n+1)$:

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2}{n+1} - \frac{2}{n(n+1)}.$$

注意到 $\frac{2}{n(n+1)} = 2\left(\frac{1}{n} - \frac{1}{n+1}\right)$, 代入得

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2}{n+1} - 2\left(\frac{1}{n} - \frac{1}{n+1}\right).$$

令 $W(n) = T(n)/(n+1)$, 则

$$W(n) = W(n-1) + \frac{2}{n+1} - \frac{2}{n} + \frac{2}{n+1} = W(n-1) + \frac{4}{n+1} - \frac{2}{n}.$$

迭代展开:

$$W(n) = W(1) + \sum_{i=2}^n \left(\frac{4}{i+1} - \frac{2}{i} \right) = \frac{T(1)}{2} + 4 \sum_{i=3}^{n+1} \frac{1}{i} - 2 \sum_{i=2}^n \frac{1}{i}.$$

利用调和级数 $H_n = \sum_{i=1}^n 1/i = \Theta(\ln n)$, 可得 $W(n) = \Theta(\ln n)$, 因此

$$T(n) = (n+1)W(n) = \Theta(n \ln n) = \Theta(n \log n).$$

1.5.6 主定理 (简化版)

对于许多分治算法, 递归方程具有形式

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^d),$$

其中 $a \geq 1$, $b > 1$, $d \geq 0$ 为常数。这里 $O(n^d)$ 表示合并子问题解所需的工作量。这类方程的解可直接由下面的主定理给出。

定理 1.5.1 (主定理简化版) 设 $a \geq 1$, $b > 1$, $d \geq 0$ 为常数, 递归式 $T(n) = aT(n/b) + O(n^d)$ 的解为: 1. 若 $a = b^d$, 则 $T(n) = O(n^d \log n)$; 2. 若 $a < b^d$, 则 $T(n) = O(n^d)$; 3. 若 $a > b^d$, 则 $T(n) = O(n^{\log_b a})$ 。

证明: 为简化分析, 假设 n 是 b 的整数幂, 即 $n = b^k$ (对于一般情况, 通过取整处理不影响渐近结果)。

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^d)$$

的递归树如 Figure 1.14 所示。

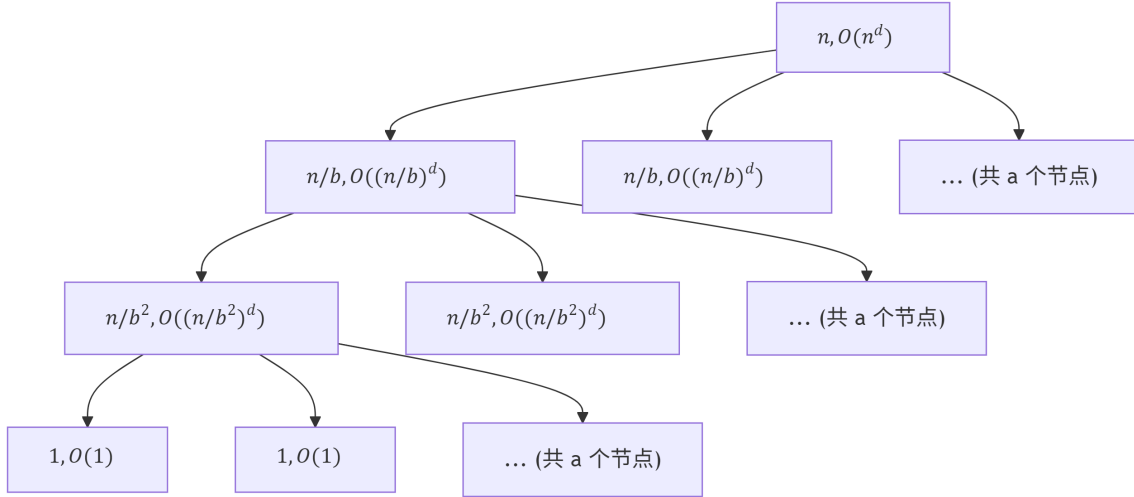


Figure 1.14: $T(n) = aT(\frac{n}{b}) + O(n^d)$ 的递归树展开示意图

根结点对应原问题，（其子问题分解与合并的）代价为 $O(n^d)$ ，它有 a 个子结点，每个子结点对应规模为 n/b 的子问题，代价为 $O((n/b)^d)$ ；第二层有 a^2 个结点，代价为 $O((n/b^2)^d)$ ，依此类推。树高为 $k = \log_b n$ ，第 j 层 ($j = 0, 1, \dots, k-1$) 有 a^j 个结点，每个结点的代价为 $O((n/b^j)^d)$ ，故该层总代价为 $a^j \cdot O((n/b^j)^d) = O(n^d (\frac{a}{b^d})^j)$ 。叶结点（第 k 层）有 $a^k = n^{\log_b a}$ 个，每个代价为常数 $O(1)$ ，故叶结点总代价为 $O(n^{\log_b a})$ 。因此总代价为

$$T(n) = O(n^{\log_b a}) + \sum_{j=0}^{k-1} O\left(n^d \left(\frac{a}{b^d}\right)^j\right).$$

下面分三种情况讨论。

1. 情况 1: $a = b^d$

此时 $\frac{a}{b^d} = 1$ ，求和项中的每一项均为 $O(n^d)$ ，共有 $k = \log_b n$ 项，故

$$\sum_{j=0}^{k-1} O(n^d \cdot 1^j) = O(n^d \log_b n).$$

叶结点代价 $O(n^{\log_b a}) = O(n^d)$ （因为 $a = b^d$ 意味着 $\log_b a = d$ ），与求和项中的 $O(n^d \log n)$ 相比是低阶项，因此 $T(n) = O(n^d \log n)$ 。

2. 情况 2: $a < b^d$

此时 $\frac{a}{b^d} < 1$ ，求和项为收敛的几何级数：

$$\sum_{j=0}^{k-1} O\left(n^d \left(\frac{a}{b^d}\right)^j\right) \leq O(n^d) \sum_{j=0}^{\infty} \left(\frac{a}{b^d}\right)^j = O(n^d) \cdot \frac{1}{1 - \frac{a}{b^d}} = O(n^d).$$

叶结点代价 $O(n^{\log_b a})$ ，由于 $a < b^d$ 意味着 $\log_b a < d$ ，故 $n^{\log_b a} = o(n^d)$ ，因此 $T(n) = O(n^d)$ 。

3. 情况 3: $a > b^d$

此时 $\frac{a}{b^d} > 1$, 求和项为递增的几何级数, 其主导项为最后一项:

$$\sum_{j=0}^{k-1} O\left(n^d \left(\frac{a}{b^d}\right)^j\right) = O\left(n^d \cdot \frac{(a/b^d)^k - 1}{(a/b^d) - 1}\right) = O\left(n^d \left(\frac{a}{b^d}\right)^k\right).$$

由于 $k = \log_b n$, 有

$$\left(\frac{a}{b^d}\right)^k = \frac{a^k}{b^{dk}} = \frac{n^{\log_b a}}{n^d} = n^{\log_b a - d}.$$

因此求和项为 $O(n^d \cdot n^{\log_b a - d}) = O(n^{\log_b a})$ 。叶结点代价也为 $O(n^{\log_b a})$, 故 $T(n) = O(n^{\log_b a})$ 。

综上所述, 定理得证。注意, 上述证明中我们仅给出了上界 (O 表示), 实际可类似证明下界, 从而得到更精确的 Θ 表示 (若 $f(n) = \Theta(n^d)$ 而非 $O(n^d)$)。

1.5.7 一般形式的主定理及其证明

定理 1.5.2(一般形式的主定理) 设 $a \geq 1, b > 1$ 为常数, $f(n)$ 为渐近正函数, 递归式 $T(n) = aT(n/b) + f(n)$ (其中 n/b 指 $\lfloor n/b \rfloor$ 或 $\lceil n/b \rceil$) 的解为: 1. 若存在常数 $\varepsilon > 0$ 使得 $f(n) = O(n^{\log_b a - \varepsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$ 。2. 若 $f(n) = \Theta(n^{\log_b a})$, 则 $T(n) = \Theta(n^{\log_b a} \log n)$ 。3. 若存在常数 $\varepsilon > 0$ 使得 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, 且存在常数 $c < 1$ 与足够大的 n 满足 $af(n/b) \leq cf(n)$ (正则性条件), 则 $T(n) = \Theta(f(n))$ 。

证明: 为简化证明, 假设 n 是 b 的正整数幂, 即 $n = b^k$ 。对于一般情况, 通过取整处理不影响渐近结果。递归树有 $k+1$ 层 ($k = \log_b n$), 第 j 层 ($j = 0, 1, \dots, k$) 有 a^j 个结点, 每个结点对应规模为 n/b^j 的子问题, 该层总代价为 $a^j f(n/b^j)$ 。此外, 叶结点 (第 k 层) 的每个子问题规模为 1, 其代价为常数 $\Theta(1)$, 叶结点总数为 $a^k = n^{\log_b a}$, 故叶结点总代价为 $\Theta(n^{\log_b a})$ 。因此总代价为

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right).$$

我们只需分析求和项 $\sum_{j=0}^{k-1} a^j f(n/b^j)$ 的渐近阶, 并比较其与 $\Theta(n^{\log_b a})$ 的大小。

情况 1: $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$ 。则存在常数 $C > 0$ 使得 $f(n) \leq Cn^{\log_b a - \varepsilon}$ 。于是

$$a^j f\left(\frac{n}{b^j}\right) \leq a^j C \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon} = Cn^{\log_b a - \varepsilon} \cdot \frac{a^j}{(b^j)^{\log_b a - \varepsilon}}.$$

由于 $a = b^{\log_b a}$, 有

$$\frac{a^j}{(b^j)^{\log_b a - \varepsilon}} = \frac{b^{j \log_b a}}{b^{j(\log_b a - \varepsilon)}} = b^{j\varepsilon}.$$

因此

$$\sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \leq C n^{\log_b a - \varepsilon} \sum_{j=0}^{k-1} b^{j\varepsilon} = C n^{\log_b a - \varepsilon} \cdot \frac{b^{\varepsilon k} - 1}{b^{\varepsilon} - 1}.$$

而 $b^{\varepsilon k} = (b^k)^{\varepsilon} = n^{\varepsilon}$, 故上式等于 $C n^{\log_b a - \varepsilon} \cdot O(n^{\varepsilon}) = O(n^{\log_b a})$ 。结合叶结点代价 $\Theta(n^{\log_b a})$, 得 $T(n) = O(n^{\log_b a})$ 。同时, 叶结点代价本身给出下界 $\Omega(n^{\log_b a})$, 因此 $T(n) = \Theta(n^{\log_b a})$ 。

情况 2: $f(n) = \Theta(n^{\log_b a})$ 。则存在正常数 c_1, c_2 使得 $c_1 n^{\log_b a} \leq f(n) \leq c_2 n^{\log_b a}$ 。于是

$$a^j f\left(\frac{n}{b^j}\right) \geq a^j c_1 \left(\frac{n}{b^j}\right)^{\log_b a} = c_1 n^{\log_b a}$$

同理可得上界 $c_2 n^{\log_b a}$ 。因此求和项中的每一项均为 $\Theta(n^{\log_b a})$, 共有 $k = \log_b n$ 项, 故

$$\sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) = \Theta(n^{\log_b a} \log n).$$

叶结点代价 $\Theta(n^{\log_b a})$ 相对于 $\Theta(n^{\log_b a} \log n)$ 是低阶项, 所以 $T(n) = \Theta(n^{\log_b a} \log n)$ 。

情况 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$, 且正则性条件 $a f(n/b) \leq c f(n)$ 对某个 $c < 1$ 和所有足够大的 n 成立。由正则性条件递推可得

$$a^j f\left(\frac{n}{b^j}\right) \leq c^j f(n)$$

于是

$$\sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \leq f(n) \sum_{j=0}^{k-1} c^j \leq f(n) \sum_{j=0}^{\infty} c^j = \frac{f(n)}{1-c} = O(f(n))$$

另一方面, 显然 $T(n) \geq f(n)$ (第一层代价), 故 $T(n) = \Omega(f(n))$ 。同时需注意叶结点代价 $\Theta(n^{\log_b a})$ 相对于 $f(n) = \Omega(n^{\log_b a + \varepsilon})$ 是低阶项, 可忽略。因此 $T(n) = \Theta(f(n))$ 。

证明完毕。

1.5.7.1 一般形式主定理的应用实例:

- 归并排序: $T(n) = 2T(n/2) + n$, $\log_b a = \log_2 2 = 1$, $f(n) = n = \Theta(n^1)$, 属于情况 2, 故 $T(n) = \Theta(n \log n)$ 。
- 二分查找: $T(n) = T(n/2) + 1$, $\log_2 1 = 0$, $f(n) = 1 = \Theta(n^0)$, 情况 2, 得 $T(n) = \Theta(\log n)$ 。
- Karatsuba: $T(n) = 3T(n/2) + n$, $\log_2 3 \approx 1.585$, $f(n) = n = O(n^{1.585-\varepsilon})$, 取 $\varepsilon = 0.5$, 属于情况 1, 得 $T(n) = \Theta(n^{\log_2 3})$ 。
- Strassen: $T(n) = 7T(n/2) + n^2$, $\log_2 7 \approx 2.807$, $f(n) = n^2 = O(n^{2.807-\varepsilon})$, 情况 1, 得 $T(n) = \Theta(n^{\log_2 7})$ 。
- 例 1.4.4: $T(n) = 3T(n/2) + n^2$, $\log_2 3 \approx 1.585$, $f(n) = n^2 = \Omega(n^{1.585+\varepsilon})$, 且易验证正则性条件 $3(n/2)^2 = \frac{3}{4}n^2 \leq cn^2$ 对 $c = 3/4 < 1$ 成立, 属于情况 3, 得 $T(n) = \Theta(n^2)$ 。

注意，简化版主定理是当 $f(n) = O(n^d)$ 时的一般形式特例（此时 $n^{\log_b a}$ 与 n^d 比较）。但简化版要求 $f(n)$ 恰为 $O(n^d)$ ，而一般形式允许更复杂的 $f(n)$ 。

1.5.8 小结

本节介绍了求解递归方程的多种方法：- 迭代展开和递归树直观且通用，适用于简单递归式。- 换元迭代能将非齐次项转化为可求和形式。- 假设归纳法需要一定猜测能力，但证明严谨。- 高阶方程化简常用于带求和形式的递归式。- 主定理是处理形如 $T(n) = aT(n/b) + f(n)$ 问题的标准工具，应熟练掌握其两个版本的使用条件与结论。

掌握这些方法，能够有效分析递归算法的时间复杂度，为后续学习分治、动态规划等算法打下坚实基础。

1.6 习题

1.6.1 选择题

1. 下列哪个选项最准确地描述了算法的核心特征？
A. 算法必须用计算机程序实现
B. 算法可以没有输入，但至少有一个输出
C. 算法的每一步可以有不同的解释，以方便不同读者理解
D. 算法可以无限执行下去，只要每次都能得到正确结果
2. 在算法设计策略中，以下哪种方法通过将大问题分解为若干个规模较小的相同子问题，递归求解后再合并结果？
A. 贪心法
B. 动态规划
C. 分治法
D. 暴力法
3. 假设一个算法的时间复杂度函数为 $T(n) = 2n^2 + 100n + 1000$ ，则以下哪个渐近表达式最准确？
A. $T(n) = O(n)$
B. $T(n) = \Theta(n^2)$
C. $T(n) = O(2^n)$
D. $T(n) = \Theta(1000)$
4. 对于递归方程 $T(n) = 2T(n/2) + n$ ， $T(1) = 1$ ，其时间复杂度为：
A. $\Theta(1)$
B. $\Theta(n)$
C. $\Theta(n \log n)$
D. $\Theta(n^2)$
5. 以下关于算法与程序的说法，正确的是：
A. 算法必须用特定的编程语言实现才能称为算法
B. 同一个算法用不同编程语言实现，其渐近时间复杂度可能不同
C. 程序是算法在计算机上的具体实现，而算法是解决问题方法的抽象描述
D. 任何一段程序都对应一个严格意义上的算法

1.6.2 判断题

1. O 一个算法的空间复杂度 $O(1)$ 意味着该算法不使用任何额外的存储空间。
2. Θ 对于同一个问题，设计出的不同算法其时间复杂度可能相差很大，例如线性时间和指数时间。
3. Θ 用主定理求解 $T(n) = 2T(n/2) + n \log n$ 时，可以直接得到 $T(n) = \Theta(n \log n)$ 。
4. Θ 在渐近分析中， $100n$ 和 $0.5n$ 都属于 $\Theta(n)$ ，但前者的常数系数更大，实际运行时间可能更慢。
5. Θ 算法的正确性只需要通过大量测试用例验证即可严格证明。

1.6.3 填空题

1. 算法的五个重要特性是：有穷性、确定性、_____、输入和输出。
2. 算法分析主要关注两个方面：_____和效率分析。
3. 若 $T(n) = 3T(n/4) + \Theta(n^2)$ ，则根据主定理， $T(n) = \Theta(\text{_____})$ 。
4. 递归算法的时间复杂度通常用 _____ 方法求解。
5. 快速排序在最坏情况下的时间复杂度为 _____，在平均情况下的时间复杂度为 _____。

1.6.4 简答题

1. 简述算法的定义，并说明算法与程序的主要区别。
2. 分别解释 O 、 Ω 、 Θ 三种渐近记号的含义，并举例说明。
3. 简述使用循环不变量法证明迭代算法正确性的三个步骤，并以“选择排序”为例说明。
4. 比较贪心法和动态规划两种算法设计策略的异同点，各举一个典型问题说明。
5. 某算法的时间复杂度为 $T(n) = 3T(n/2) + n^2$ ，请用主定理求解其渐近紧确界，并说明理由。
6. 分析以下递归算法的时间复杂度，写出推导过程。

```
def mystery(n):
    if n <= 1:
        return 1
    else:
        return mystery(n-1) + mystery(n-1)
```

1.6.5 算法表示练习

1. **自然语言描述**：用自然语言描述一个算法，找出三个整数中的最大值。
2. **流程图绘制**：画出求解一元二次方程 $ax^2 + bx + c = 0$ 实根的流程图中（不考虑复数根，需考虑 $\Delta > 0$ ， $\Delta = 0$ ， $\Delta < 0$ 以及 $a = 0$ 的情况）。
3. **伪代码编写**：用伪代码编写一个算法，计算一个整数数组中所有正数的和。数组下标从 0 开始，长度记为 n 。
4. **伪代码改错**：以下伪代码试图计算 1 到 n 的平方和，但存在错误。请找出错误并给出修正后的伪代码。

```
Algorithm SumSquare(n)
    sum = 0
    for i = 1 to n-1
        sum = sum + i * i
    end for
    return sum
```

5. **程序语言实现**：将第 3 题的伪代码分别用 C++ 和 Python 语言实现。

1.6.6 客观编程题

1.6.6.1 题目描述

给定一个包含 n 个整数的数组 A ，请设计一个算法找出数组中的最大值和最小值。要求使用**分治法**实现，即递归地将数组分成两半，分别求出左半部分和右半部分的最大值和最小值，然后合并得到整个数组的最大值和最小值。**禁止**使用简单的遍历方法。

1.6.6.2 输入格式

第一行一个整数 n ，表示数组元素个数。第二行 n 个整数，表示数组 A 中的元素，元素之间用空格分隔。

约束： $-1 \leq n \leq 10^5$ - $-10^9 \leq A[i] \leq 10^9$

1.6.6.3 输出格式

输出一行两个整数，分别表示数组中的最大值和最小值，中间用一个空格分隔。

1.6.6.4 样例输入

```
6
3 5 1 8 2 4
```

1.6.6.5 样例输出

```
8 1
```

1.7 附录：本书伪代码风格规范

本书伪代码采用**简洁的、类似 Python 的缩进风格**，目的是让读者把注意力集中在算法逻辑上，而非语法细节。数组下标默认从 0 开始。

1.7.0.1 核心规则（全书统一）

- 使用**缩进**（4 个空格）表示代码块层次，无显式结束标记（如 end if、end for）。
- 控制结构头部以冒号：结尾。
- 关键字全部小写英文（if、else、for、while、return）。
- 赋值使用 =。
- 注释使用 //（行尾或单独行）。
- 变量名使用下划线风格（snake_case），语义清晰。
- 逻辑运算使用 and、or、not。
- 比较运算使用 ==、!=、>、<、>=、<=。
- 循环范围示例：for i = 0 to n-1:（包含 0 到 n-1）或 for i = 1 to n:（视语境）。
- 返回使用 return 值或 return（无值时）。
- 特殊值：无穷大用 infinity，空值用 None 或 -1（视语境）。

1.7.0.2 常用格式简单示例

1. 函数定义

```
find_maximum(arr, n)
    max_val = arr[0]
    return max_val
```

2. 条件语句 (if / else / elif)

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "D"
```

3. 单分支 if

```
if n <= 0:
    return -1 // 表示无效输入
```

4. for 循环 (两种常见写法)

```
// 从 0 到 n-1
for i = 0 to n-1:
    print(arr[i])

// 从 1 到 n
for i = 1 to n:
    sum += i
```

5. while 循环

```
while b ≠ 0:
    temp = b
    b = a % b
    a = temp
```

6. 递归函数

```
factorial(n)
    if n <= 1:
        return 1
    return n * factorial(n-1)
```

7. 带注释的完整小示例 (求数组和)

```
sum_array(arr, n)
    total = 0
```

```
for i = 0 to n-1:  
    total += arr[i] // 累加当前元素  
return total
```

1.7.0.3 使用建议

- 优先使用**语义清晰的变量名**，避免过多单字母变量（如 i、j 仅用于循环索引）。
- 复杂逻辑建议多加 // 注释说明意图。
- 实际编程时，可直接将伪代码转换为 Python、C++、Java 等语言，只需调整少量语法细节。

通过这种风格，本书伪代码既简洁易读，又接近现代编程习惯，便于你快速理解算法本质并转化为真实代码。

2 线性穷举

穷举法 (Exhaustive Search / Brute Force) 是最直接、最朴素的算法设计方法，其核心思想是**穷举所有可能的候选情况**，从中找出满足条件的解。这种方法虽然往往效率不高，但它是算法设计的基石，许多高效算法都是在穷举法的基础上通过剪枝、优化、利用数学性质等方式改进而来。

根据穷举方式的不同，可分为**线性穷举**和**树形穷举**两类：

- **线性穷举**：通过线性迭代的方式遍历所有元素或情况（例如遍历数组、链表）。
- **树形穷举**：通过树形搜索的方式探索所有可能的解空间（例如 N 皇后问题、子集生成），将在后续章节介绍。

本章聚焦于线性穷举，它是算法设计中最基本、最直观的方法。

2.1 线性穷举的基本思想

线性穷举针对的是**线性结构**（如数组、链表、字符串等）或**线性可枚举的问题**，通过一次或多次迭代，逐一处理每个元素或候选情况。其基本框架如下：

```
for 每个情况（元素）：
    处理该情况（元素）
```

这种方法的正确性保证来自于“穷举所有可能性”，因此只要算法正确实现了遍历，就不会遗漏解。其缺点在于当候选情况数量巨大时，运行时间会急剧增长。

根据迭代的结构，线性穷举可分为以下几种常见形式：

- **简单迭代**：单层循环，遍历一个线性序列。
- **嵌套迭代**：多层循环嵌套，用于处理多维结构或多重组合。
- **多表迭代**：同时遍历多个线性表（如合并有序序列）。
- **双指针迭代**：使用两个指针协同遍历，可提高效率，避免不必要的穷举。

2.2 简单迭代

简单迭代是最基本的线性穷举形式，它通过单层循环遍历一个线性序列（如数组、链表）或逐一枚举所有可能的情况（例如从 0 到 $n-1$ 的整数）。其设计思路非常直接：既然需要处理所有候选对象，那就从头到尾依次访问每个对象。

2.2.1 数组的简单迭代

对于数组，元素在内存中连续存储，我们可以使用下标直接访问。

设计思路：通过一个循环变量 i 从 0 递增到 $n-1$ ，在每次迭代中处理 $a[i]$ 。这样就能保证每个元素都被访问一次。如 `@#fig-traverse_array` 所示：

伪代码：

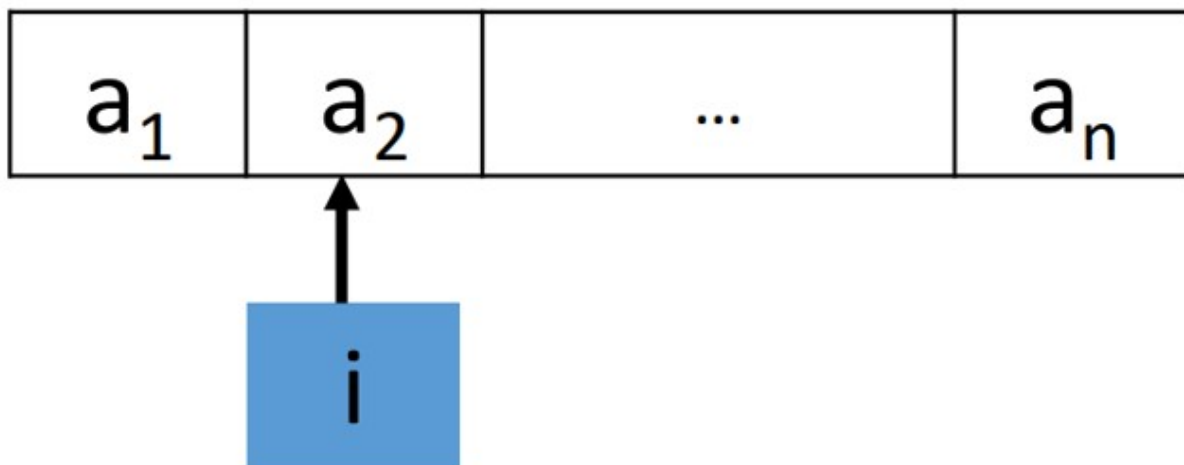


Figure 2.1: 对每个 i (从 0 到 $n - 1$)，处理 $a[i]$

```
traverse_array(a[0..n-1]):  
    for i = 0 to n-1:  
        // 处理元素 A[i]
```

C++ 实现:

```
// 使用 for 循环  
void traverse(vector<int>& a) {  
    for (int i = 0; i < a.size(); i++) {  
        // 处理元素 a[i]  
    }  
}  
  
// 使用 while 循环  
void traverse(vector<int>& a) {  
    int i = 0;  
    while (i < a.size()) {  
        // 处理元素 a[i]  
        i++;  
    }  
}
```

Python 实现:

```
def traverse_array(A):  
    for i in range(len(A)):  
        # 处理元素 A[i]  
        pass
```

2.2.2 链表的简单迭代

链表由结点通过指针链接而成，结点在内存中不一定连续，因此必须通过指针依次访问。

设计思路：设置一个指针 p 指向头结点，然后循环判断 p 是否为空，若非空则处理当前结点，并将 p 移动到下一个结点。这样就能遍历整个链表。如 Figure 2.2 所示。

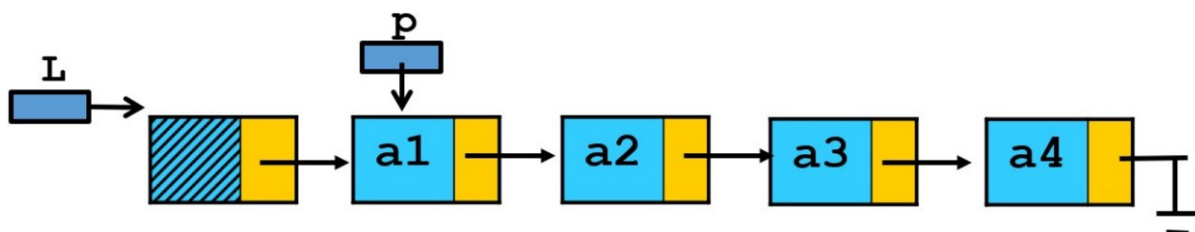


Figure 2.2: 遍历指针 p 从第 1 个结点到最后一个结点，处理 p 指向的结点

首先定义链表结点（后续所有链表算法均沿用此定义）：

C++ 结点定义：

```

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

```

Python 结点定义：

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

```

遍历算法伪代码：

```

traverse_list(head):
    p = head
    while p ≠ null:
        // 处理结点 p.data
        p = p.next

```

C++ 实现：

```

void traverseList(ListNode* head) {
    ListNode* p = head;
    while (p ≠ nullptr) {
        // 处理结点 p->val
        p = p->next;
    }
}

```

Python 实现:

```
def traverse_list(head):
    p = head
    while p:
        # 处理结点 p.val
        p = p.next
```

2.2.3 线性查找

问题: 在给定序列中查找是否存在等于目标值 x 的元素, 若存在则返回其位置 (下标或结点指针), 否则返回 -1 或 nullptr。

数组的顺序查找

设计思路: 顺序查找是最直接的查找方法。从数组的第一个元素开始, 逐个与目标值 x 比较, 如果相等则返回当前下标; 如果遍历完整个数组都没有找到, 则返回 -1。这种方法不需要数组有序, 适用于任何数组。如 Figure 2.3 所示:

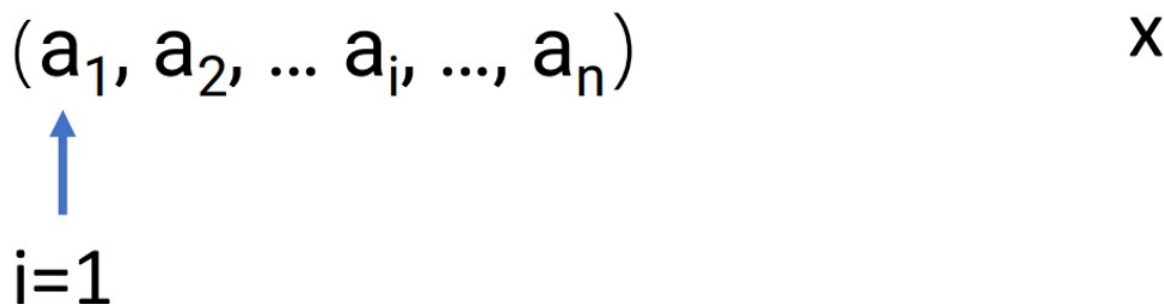


Figure 2.3: 对每个 i (从 0 到 $n-1$), 比较 $a[i]$ 和 x 是否相等

算法分析:

- 最好情况: 目标正好在第一个位置, 只需比较 1 次, 时间复杂度 $O(1)$ 。
- 最坏情况: 目标在最后一个位置或不存在, 需要比较 n 次, 时间复杂度 $O(n)$ 。
- 平均情况: 假设目标等概率出现在每个位置, 平均比较次数为 $(1 + 2 + \dots + n)/n = (n + 1)/2$, 时间复杂度 $O(n)$ 。

伪代码:

```
sequential_search(A[0..n-1], x):
    for i = 0 to n-1:
        if A[i] == x:
            return i
    return -1
```

C++ 实现:

```
template <typename T>
int sequentialSearch(vector<T>& A, T x) {
    for (int i = 0; i < A.size(); i++) {
```

```

    if (A[i] == x) return i;
}
return -1;
}

```

Python 实现:

```

def sequential_search(A, x):
    for i in range(len(A)):
        if A[i] == x:
            return i
    return -1

```

2.2.3.1 链表的链式查找

设计思路: 与数组顺序查找类似，但需要通过指针遍历链表。从链表头开始，每次判断当前结点值是否等于 x ，若相等返回该结点指针；若遍历完整个链表仍未找到，返回 `null`。如 Figure 2.4 所示：

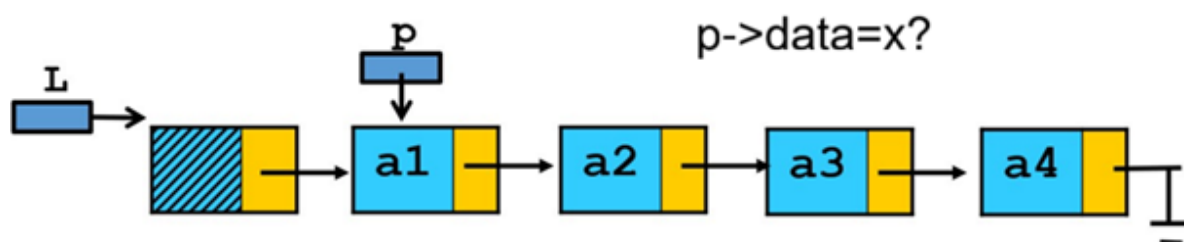


Figure 2.4: 遍历指针 p 从第 1 个结点到最后一个结点，比较 p 指向结点值是否等于 x : $p \rightarrow data == x?$

伪代码:

```

search_list(head, x):
    p = head
    while p != null:
        if p.data == x:
            return p
        p = p.next
    return null

```

C++ 实现:

```

ListNode* searchList(ListNode* head, int x) {
    ListNode* p = head;
    while (p != nullptr) {
        if (p->val == x) return p;
        p = p->next;
    }
    return nullptr;
}

```

Python 实现:

```
def search_list(head, x):
    p = head
    while p:
        if p.val == x:
            return p
        p = p.next
    return None
```

2.2.4 删除有序链表中的重复元素

问题: 给定一个按升序排列的链表, 删除所有重复的元素, 使每个元素只出现一次。

设计思路: 由于链表是有序的, 重复元素必定相邻。因此我们可以遍历链表, 检查当前结点与下一个结点的值是否相同。如果相同, 则删除下一个结点, 并保持当前结点不变 (继续与新后继比较); 如果不同, 则移动当前结点到下一个位置。这样只需一趟遍历即可完成去重。如 Figure 2.5 所示:

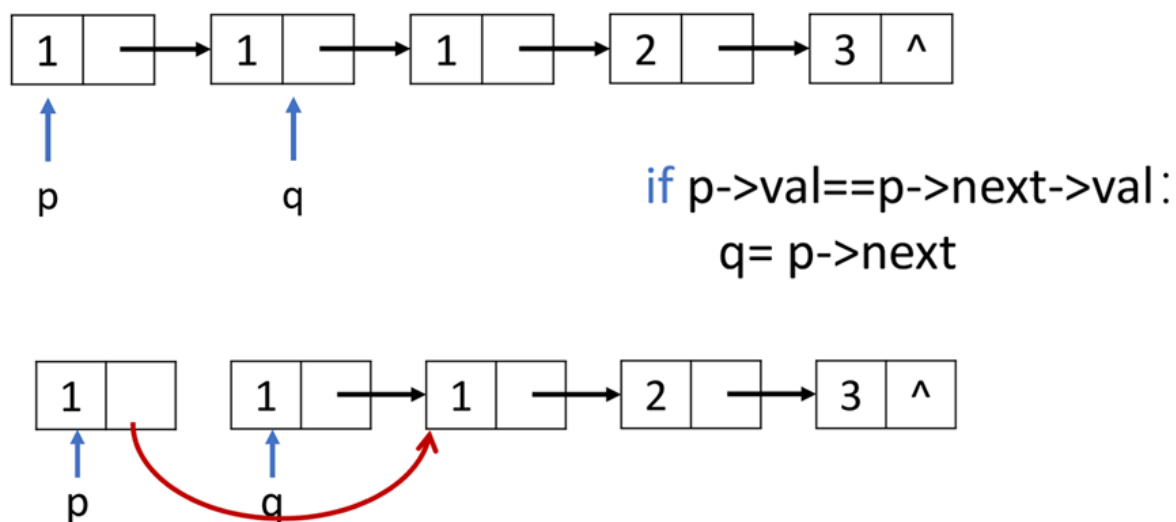


Figure 2.5: 检查当前结点与下一个结点的值是否相同。如果相同, 则删除下一个结点, 并保持当前结点不变

算法分析: 每个结点最多被访问一次, 时间复杂度 $O(n)$ 。空间复杂度 $O(1)$ (只使用常数个指针)。

注意: 在 C++ 中需要手动释放被删除结点的内存; 在 Python 中由垃圾回收自动处理, 但代码中同样体现删除逻辑。

伪代码:

```
delete_duplicates(head):
    if head == null or head.next == null:
        return head
    p = head
    while p.next != null:
        if p.data == p.next.data:
            q = p.next
            p.next = q.next
```

```

        // 释放结点 q (在伪代码中可省略释放操作)
    else:
        p = p.next
    return head

```

C++ 实现:

```

ListNode* deleteDuplicates(ListNode* head) {
    if (head == nullptr || head->next == nullptr) return head;
    ListNode* p = head;
    while (p->next != nullptr) {
        if (p->val == p->next->val) {
            ListNode* q = p->next;
            p->next = q->next;
            delete q; // 释放内存
        } else {
            p = p->next;
        }
    }
    return head;
}

```

Python 实现:

```

def delete_duplicates(head):
    if not head or not head.next:
        return head
    p = head
    while p.next:
        if p.val == p.next.val:
            p.next = p.next.next # Python 无需手动释放
        else:
            p = p.next
    return head

```

2.3 嵌套迭代

嵌套迭代是指循环内部再嵌套循环，用于处理二维结构、需要穷举多重组合的问题，以及元素间的比较排序等。典型的应用包括：二维数组遍历（矩阵）、两数之和的暴力解法、百钱买百鸡问题、字符串匹配等。此外，许多简单的排序算法（如冒泡排序、选择排序）也依赖于嵌套循环来反复比较和交换元素，它们同样是嵌套迭代的经典例子。

2.3.1 二维数组遍历

设计思路：二维数组可看作由行和列组成的矩阵。外层循环遍历行，内层循环遍历列，这样就能访问每个元素。

伪代码:

```
for i = 0 to m-1:
    for j = 0 to n-1:
        // 处理 A[i][j]
```

C++ 实现:

```
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        // 处理 A[i][j]
    }
}
```

Python 实现:

```
for i in range(m):
    for j in range(n):
        # 处理 A[i][j]
    pass
```

2.3.2 两数之和（暴力解法）

问题 (LeetCode 1): 给定一个整数数组 `nums` 和一个目标值 `target`, 请找出和为目标值的两个整数, 并返回它们的下标。假设每种输入只对应一个答案, 且不能重复使用同一个元素。

设计思路: 最直接的想法是穷举所有可能的数对 (i, j) (其中 $i < j$), 检查它们的和是否等于 `target`。如果找到, 则返回下标。这种方法虽然简单, 但需要 $O(n^2)$ 的时间。如 Figure 2.6 所示:

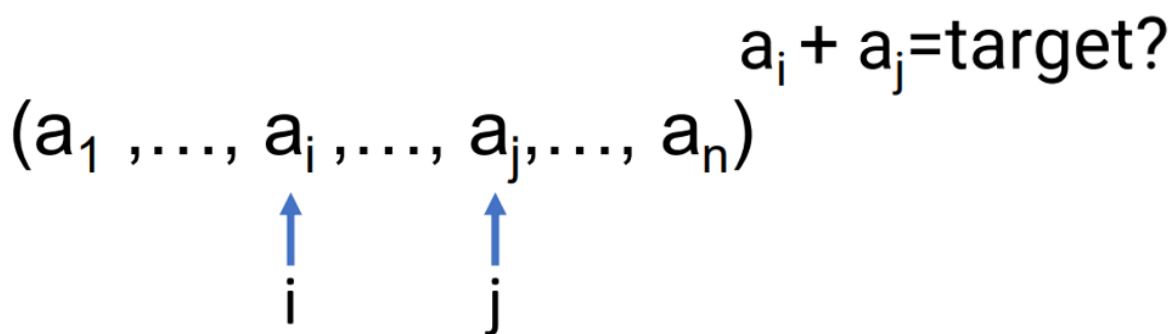


Figure 2.6: 穷举所有可能的数对 (i, j) (其中 $i < j$), 检查它们的和 $a_i + a_j$ 是否等于 `target`

算法分析: 外层循环 n 次, 内层循环平均 $n/2$ 次, 总比较次数约为 $n(n-1)/2$, 时间复杂度 $O(n^2)$ 。空间复杂度 $O(1)$ 。

优化思路: 后续可学习哈希表方法, 将时间复杂度降至 $O(n)$, 但需要额外空间。

伪代码:

```
two_sum(nums[0..n-1], target):
    for i = 0 to n-2:
```

```

    for j = i+1 to n-1:
        if nums[i] + nums[j] == target:
            return (i, j)
    return (-1, -1) // 无解

```

C++ 实现:

```

vector<int> twoSum(vector<int>& nums, int target) {
    int n = nums.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (nums[i] + nums[j] == target) {
                return {i, j};
            }
        }
    }
    return {-1, -1};
}

```

Python 实现:

```

def two_sum(nums, target):
    n = len(nums)
    for i in range(n - 1):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return [i, j]
    return [-1, -1]

```

2.3.3 百钱买百鸡

问题: 公鸡 5 文钱一只, 母鸡 3 文钱一只, 小鸡 3 只一文钱。用 100 文钱买 100 只鸡, 问公鸡、母鸡、小鸡各多少只?

数学建模: 设公鸡 x 只, 母鸡 y 只, 小鸡 z 只, 则

$$\begin{cases} x + y + z = 100 & (1) \\ 5x + 3y + \frac{z}{3} = 100 & (2) \end{cases}$$

其中 x, y, z 为非负整数, 且 z 必须是 3 的倍数。直接穷举所有可能的 (x, y, z) 组合, 检查是否满足方程。

优化思路: 注意 x 的范围为 $0 \leq x \leq 20$ (因为 $5x \leq 100$), y 的范围为 $0 \leq y \leq 33$ 。对于每个 x, y , 可以计算出 $z = 100 - x - y$, 然后检查是否满足价格方程。这样可以减少一层循环, 提高效率。

算法分析: 优化后的算法需要两层循环, 总迭代次数约为 $21 \times 34 = 714$, 时间复杂度 $O(1)$ (因为规模固定)。但若将钱数 n 作为变量, 则时间复杂度为 $O(n^2)$ 。

伪代码 (优化版):

```

buy_chicken(money, total):
    for x = 0 to floor(money / 5):
        for y = 0 to floor(money / 3):
            z = total - x - y
            if z < 0:
                break
            if z % 3 == 0 and 5*x + 3*y + z/3 == money:
                output (x, y, z)

```

C++ 实现:

```

#include <iostream>
using namespace std;

void buyChicken(int money = 100, int total = 100) {
    cout << " 公鸡\t母鸡\t小鸡" << endl;
    for (int x = 0; x <= money / 5; x++) {           // 公鸡最多 money/5 只
        for (int y = 0; y <= money / 3; y++) {       // 母鸡最多 money/3 只
            int z = total - x - y;                  // 小鸡数量
            if (z < 0) break;                        // 小鸡数量不能为负
            if (z % 3 == 0 && 5*x + 3*y + z/3 == money) {
                cout << x << "\t" << y << "\t" << z << endl;
            }
        }
    }
}

// 测试主函数
int main() {
    cout << "==== 使用默认参数 =====> << endl;
    buyChicken(); // 默认 money=100, total=100

    cout << "\n==== 自定义参数示例 =====> << endl;
    int money, total;
    cout << " 请输入总钱数: ";
    cin >> money;
    cout << " 请输入鸡的总数: ";
    cin >> total;
    buyChicken(money, total);

    return 0;
}

```

Python 实现:

```

def buy_chicken(money=100, total=100):
    print(" 公鸡\t母鸡\t小鸡")
    for x in range(money // 5 + 1):          # 公鸡最多 money//5 只
        for y in range(money // 3 + 1):      # 母鸡最多 money//3 只
            z = total - x - y                # 小鸡数量
            if z < 0:
                break
            if z % 3 == 0 and 5*x + 3*y + z//3 == money:
                print(f"{x}\t{y}\t{z}")

# -----
# 测试主函数
# -----
if __name__ == "__main__":
    print("===== 使用默认参数 =====")
    buy_chicken() # 默认 money=100, total=100

    print("\n===== 自定义参数示例 =====")
    try:
        money = int(input(" 请输入总钱数: "))
        total = int(input(" 请输入鸡的总数: "))
        buy_chicken(money, total)
    except ValueError:
        print(" 请输入整数数字! ")

```

2.3.4 字符串匹配（朴素字符串匹配算法）

问题与背景：给定主串 $T[0..n-1]$ 和模式串 $P[0..m-1]$ ，找出 P 在 T 中第一次出现的位置。若不存在，返回 -1。

字符串匹配是计算机科学中最常见的问题之一，广泛用于文本编辑器的查找功能、搜索引擎的网页匹配、入侵检测系统中的特征码匹配、生物信息学中的 DNA 序列分析等。朴素字符串匹配算法是最直观的方法，虽然效率不高，但为理解更高效的匹配算法（如 KMP、BM 等）奠定了基础。

设计思路：从主串的每个可能起始位置 i ($0 \leq i \leq n - m$)，逐一比较 P 与 $T[i..i + m - 1]$ 是否相等。如果某个起始位置处所有字符都匹配，则返回 i ；如果所有起始位置都尝试后仍不匹配，则返回 -1。如 Figure 2.7 所示：

伪代码：

```

brute_force_string_match(T[0..n-1], P[0..m-1]):
    for i = 0 to n-m:
        j = 0
        while j < m and T[i+j] == P[j]:
            j = j+1

```

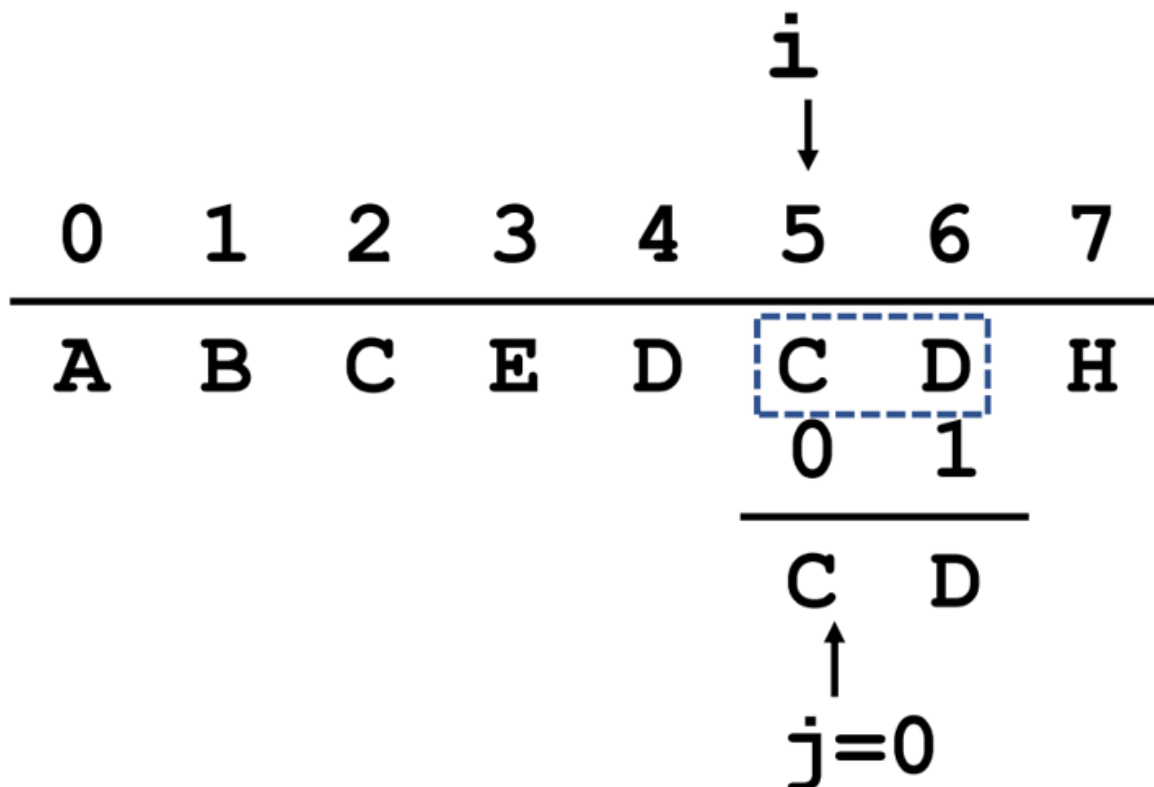


Figure 2.7: 从主串的每个可能起始位置 i ($0 \leq i \leq n - m$), 逐一比较 P 与 $T[i..i + m - 1]$ 是否相等

```

if j == m:
    return i
return -1

```

算法分析: - **最坏情况:** 每次内层循环都要比较 m 次, 外层循环有 $n - m + 1$ 次, 总比较次数为 $O(n \times m)$ 。例如主串为 "aaaaaaaaab", 模式串为 "aaab", 每次都在最后一个字符失败, 复杂度接近 $O(nm)$ 。 - **最好情况:** 第一次比较第一个字符就不匹配, 复杂度 $O(n)$ 。 - **平均情况:** 通常认为平均复杂度也为 $O(n + m)$, 但最坏情况是 $O(nm)$ 。

C++ 实现:

```

#include <string>
using namespace std;

int bruteForceStringMatch(const string& T, const string& P) {
    int n = T.size(), m = P.size();
    for (int i = 0; i <= n - m; i++) {
        int j = 0;
        while (j < m && T[i + j] == P[j]) {
            j++;
        }
        if (j == m) return i;
    }
    return -1;
}

```

```
}
```

Python 实现:

```
def brute_force_string_match(T, P):
    n, m = len(T), len(P)
    for i in range(n - m + 1):
        j = 0
        while j < m and T[i + j] == P[j]:
            j += 1
        if j == m:
            return i
    return -1
```

2.3.5 子集枚举与背包问题

2.3.5.1 子集枚举

子集枚举问题: 给定 n 个物品, 每个物品可以选或不选, 需要枚举所有可能的组合。这等价于生成集合 $\{0, 1, \dots, n-1\}$ 的所有子集。每个子集可以用一个长度为 n 的二进制数组 $x[0..n-1]$ 表示, 其中 $x[i] = 1$ 表示第 i 个元素在子集中, $x[i] = 0$ 表示不在。

枚举方法: 共有 2^n 个子集, 可以用一个整数 $mask$ 从 0 到 $2^n - 1$ 遍历, 每个 $mask$ 的二进制表示恰好对应一个子集 (通常约定最低位表示第 0 个元素)。通过位运算可以提取每一位的状态。

另一种方法是模拟二进制加法, 用一个数组 A 表示当前二进制串, 通过进位操作生成下一个串 (见下方拓展)。但位运算方法更简洁高效, 是实际编程中的首选。

位运算枚举伪代码:

```
for mask = 0 to 2^n - 1:
    for i = 0 to n-1:
        if (mask >> i) & 1 == 1:
            // 第 i 个元素在子集中
        else:
            // 第 i 个元素不在子集中
```

该伪代码完全适用于 Python 和 C++, 因为两者都支持整数按位操作 (如左移 $\ll$ 、右移 $\gg$ 、按位与 $\&$)。以下是用代码直观展示位运算枚举子集的过程:

C++ 示例: 枚举所有子集并打印

```
#include <iostream>
#include <vector>
using namespace std;

void enumerateSubsets(const vector<int>& items) {
```

```

int n = items.size();
int total = 1 << n; // 2^n
for (int mask = 0; mask < total; mask++) {
    cout << " 子集 { ";
    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) { // 检查第 i 位是否为 1
            cout << items[i] << " ";
        }
    }
    cout << "}" << endl;
}

int main() {
    vector<int> items = {2, 5, 7}; // 示例物品
    enumerateSubsets(items);
    return 0;
}

```

输出 (展示了所有 $2^3 = 8$ 个子集):

```

子集 { }
子集 { 2 }
子集 { 5 }
子集 { 2 5 }
子集 { 7 }
子集 { 2 7 }
子集 { 5 7 }
子集 { 2 5 7 }

```

Python 示例: 枚举所有子集并打印

```

def enumerate_subsets(items):
    n = len(items)
    total = 1 << n # 2^n
    for mask in range(total):
        subset = []
        for i in range(n):
            if (mask >> i) & 1: # 检查第 i 位是否为 1
                subset.append(items[i])
        print(f" 子集 {subset}")

# 示例
items = [2, 5, 7]
enumerate_subsets(items)

```

输出与 C++ 类似。

位运算过程说明

- $1 \ll n$: 计算 2^n , 即所有子集数量。
- $\text{mask} \& (1 \ll i)$ (C++ 风格) 或 $(\text{mask} \gg i) \& 1$ (Python 风格): 检查整数 mask 的第 i 位是否为 1, 从而判断第 i 个元素是否属于当前子集。
- 外层循环遍历所有 mask 从 0 到 $2^n - 1$, 每个 mask 的二进制形式唯一对应一个子集。

2.3.5.2 应用实例: 0-1 背包问题

问题: 给定 n 个物品, 第 i 个物品重量为 $w[i]$, 价值为 $v[i]$, 背包容量为 C 。选择若干物品装入背包, 使得总重量不超过 C , 且总价值最大。每个物品最多选一次。

设计思路: 枚举所有子集 (即所有可能的物品组合), 对每个子集计算总重量和总价值, 若重量不超过容量, 则更新最大价值。

伪代码:

```
knapsack_bruteforce(w[0..n-1], v[0..n-1], C):
```

```
    n = length(w)
    best = 0
    for mask = 0 to 2^n - 1:
        totalW = 0
        totalV = 0
        for i = 0 to n-1:
            if (mask >> i) & 1:
                totalW += w[i]
                totalV += v[i]
        if totalW ≤ C:
            best = max(best, totalV)
    return best
```

C++ 实现:

```
int knapsackBruteforce(const vector<int>& w, const vector<int>& v, int C) {
    int n = w.size();
    int best = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        int totalW = 0, totalV = 0;
        for (int i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                totalW += w[i];
                totalV += v[i];
            }
        }
    }
}
```



```

        if (totalW <= C) {
            best = max(best, totalV);
        }
    }
    return best;
}

```

Python 实现:

```

def knapsack_bruteforce(w, v, C):
    n = len(w)
    best = 0
    for mask in range(1 << n):
        total_w = 0
        total_v = 0
        for i in range(n):
            if (mask >> i) & 1:
                total_w += w[i]
                total_v += v[i]
        if total_w <= C:
            best = max(best, total_v)
    return best

```

复杂度分析: - 时间复杂度: 外层循环 2^n 次, 内层循环 n 次, 总比较次数 $O(n \cdot 2^n)$ 。该算法仅适用于 n 较小的情况 (如 $n \leq 20$)。 - 空间复杂度: $O(1)$ (仅需几个变量)。

2.3.5.3 拓展: 进位枚举法

除了位运算, 还可以用数组模拟二进制加法生成所有子集。初始数组 $A[0..n-1]$ 全为 0。每次从最低位开始, 将遇到的第一个 0 改为 1, 并将比它低的位全部清零。重复 $2^n - 1$ 次即可。

伪代码:

```

A[0..n-1] = 0
for step = 0 to 2^n - 1:
    // 输出当前子集 A
    j = 0
    while j < n and A[j] == 1:
        A[j] = 0
        j = j + 1
    if j < n:
        A[j] = 1

```

该方法与二进制加法原理相同, 可用于理解“进位”思想, 但实际编程中位运算更为常见。

以下是进位枚举法的 C++ 和 Python 实现, 通过数组模拟二进制加法来生成所有子集。

C++ 实现 (进位枚举法)

```

#include <iostream>
#include <vector>
using namespace std;

void enumerateSubsetsByCarry(const vector<int>& items) {
    int n = items.size();
    vector<int> A(n, 0);          // 二进制数组, 初始全 0
    int total = 1 << n;           // 总共 2^n 个子集
    for (int step = 0; step < total; step++) {
        // 输出当前子集
        cout << " 子集 { ";
        for (int i = 0; i < n; i++) {
            if (A[i] == 1) {
                cout << items[i] << " ";
            }
        }
        cout << "}" << endl;

        // 模拟二进制加法: 从最低位开始找第一个 0, 将其变为 1, 并将比它低的位清零
        int j = 0;
        while (j < n && A[j] == 1) {
            A[j] = 0;    // 清零低位
            j++;
        }
        if (j < n) {
            A[j] = 1;    // 将找到的第一个 0 变为 1
        }
        // 如果 j == n, 说明已经达到全 1 状态, 下一次循环将退出 (但 step 不会超过 total-1)
    }
}

int main() {
    vector<int> items = {2, 5, 7};
    enumerateSubsetsByCarry(items);
    return 0;
}

```

Python 实现 (进位枚举法)

```

def enumerate_subsets_by_carry(items):
    n = len(items)
    A = [0] * n          # 二进制数组, 初始全 0

```

```

total = 1 << n          # 总共 2^n 个子集
for step in range(total):
    # 输出当前子集
    subset = [items[i] for i in range(n) if A[i] == 1]
    print(f" 子集 {subset}")

    # 模拟二进制加法：从最低位开始找第一个 0，将其变为 1，并将比它低的位清零
    j = 0
    while j < n and A[j] == 1:
        A[j] = 0          # 清零低位
        j += 1
    if j < n:
        A[j] = 1          # 将找到的第一个 0 变为 1

# 示例
items = [2, 5, 7]
enumerate_subsets_by_carry(items)

```

代码说明

- **数组 A**：长度为 n ，下标 0 表示最低位（对应第 0 个物品），下标越大位数越高。
- **外层循环**：执行 2^n 次，每次对应一个子集。循环次数虽然用 `total` 控制，但实际生成子集的逻辑独立于循环计数器，完全由 A 的进位操作决定。
- **进位过程**：
 - 从最低位 ($j=0$) 开始扫描，遇到 $A[j]=1$ 就将其清零（相当于加法中的进位），并继续向高位移动。
 - 当遇到第一个 $A[j]=0$ 时，将其设为 1，停止。
 - 如果所有位都是 1（即 $j=n$ ），说明已经生成了最后一个子集（全 1），此时不再做任何操作（因为循环即将结束）。
- **输出**：每次循环开始时输出当前 A 对应的子集。初始 A 全 0，对应空集；经过 $2^n - 1$ 次进位后，A 变为全 1，对应全集。

运行结果（以 $n=3$ 为例）

```

子集 []
子集 [2]
子集 [5]
子集 [2, 5]
子集 [7]
子集 [2, 7]
子集 [5, 7]
子集 [2, 5, 7]

```

这种方法的本质与二进制加法一致，但通过数组直观展示了“进位”过程，有助于理解二进制计数的原理。虽然实际编程中更常用位运算，但进位枚举法在教学中可作为辅助理解。

小结：子集枚举是组合问题的基础，背包问题的穷举解是其典型应用。通过这个例子可以看到，穷举法虽然简单直观，

但时间复杂度 $O(n \cdot 2^n)$ 随 n 增长迅速不可行，这为后续学习动态规划等高效算法埋下伏笔。该枚举方法仅适用于 n 较小（如 $n \leq 20$ ）的情况。

2.3.6 简单排序算法：冒泡排序

问题：给定一个无序数组 $A[0..n-1]$ ，将其按非递减顺序排列。

2.3.7 简单排序算法：冒泡排序（面向初学者的通俗描述）

问题：给定一个无序数组 $A[0..n-1]$ ，我们希望把它按从小到大的顺序排好。

设计思路（生活中的比喻）

想象一下，你面前站着一排高矮不同的人，你想让他们按身高从矮到高排列。冒泡排序的做法是这样的：

- 从左边开始，依次比较相邻的两个人。如果左边的人比右边的人高，就让他们交换位置。这样，一趟比较下来，最高的人就会像水里的气泡一样，“冒泡”到队伍的**最右边**（因为每次比较都把高的往后移）。
- 第一趟结束后，最右边的人已经是队伍中最高的了，接下来我们就不需要再动他了。
- 然后对剩下的（从左边到倒数第二个）人，重复同样的过程：再次从左到右比较相邻的两个人，把当前最高的人“冒泡”到剩下部分的末尾。
- 每一趟都会确定一个当前范围内的最高者，并把它放到正确的位置。经过若干趟，整个队伍就排好序了。

算法描述（对应计算机步骤）

- 用数组 A 存储这些“人”，共有 n 个元素。
- 总共需要 $n-1$ 趟排序（因为最后一个人自然就位）。
- 我们用变量 i 来记录已经完成的趟数（ i 从 0 开始）。
- 第 0 趟：我们把整个数组（ $A[0]$ 到 $A[n-1]$ ）中的最大元素“冒泡”到末尾 $A[n-1]$ 。
- 第 1 趟：忽略已经排好的最后一个，对 $A[0]$ 到 $A[n-2]$ 做同样操作，把最大元素放到 $A[n-2]$ 。
- 依此类推，第 i 趟时，我们只需要处理 $A[0]$ 到 $A[n-1-i]$ 这部分元素，把其中的最大元素送到该部分的末尾 $A[n-1-i]$ 。
- 在每一趟中，我们用一个内层循环 j 从 0 遍历到 $n-2-i$ （因为最后一个元素 $A[n-1-i]$ 不需要再和自己比较），比较相邻的 $A[j]$ 和 $A[j+1]$ ，如果 $A[j] > A[j+1]$ ，就交换它们。

这样，通过两层嵌套循环（外层控制趟数，内层控制比较），整个数组就排好序了。

伪代码：

```
bubble_sort(A[0..n-1]):
    // 外层循环：i 表示已经排好的趟数，也是已经放到末尾的大元素个数
    for i = 0 to n-2:
        // 内层循环：j 从 0 到 n-2-i，比较相邻元素
        for j = 0 to n-2-i:
            if A[j] > A[j+1]:           // 如果左边比右边大，则交换
                swap(A[j], A[j+1])
```

优化提示：如果在某一趟中没有发生任何交换，说明数组已经有序，可以提前结束排序。这可以在代码中加入一个标志变量来实现。

复杂度分析：- 最坏情况：数组逆序，比较次数为 $\frac{n(n-1)}{2}$ ，时间复杂度 $O(n^2)$ 。- 最好情况：数组已有序，若不加优化仍需 $O(n^2)$ 次比较；但可引入标志位提前退出，使最好情况达到 $O(n)$ 。- 空间复杂度： $O(1)$ ，原地排序。

C++ 实现：

```
void bubbleSort(vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n-1; i++) {
        bool swapped = false; // 优化：记录本趟是否有交换
        for (int j = 0; j < n-1-i; j++) {
            if (A[j] > A[j+1]) {
                swap(A[j], A[j+1]);
                swapped = true;
            }
        }
        if (!swapped) break; // 若无交换，说明已有序，提前结束
    }
}
```

Python 实现：

```
def bubble_sort(A):
    n = len(A)
    for i in range(n-1):
        swapped = False
        for j in range(n-1-i):
            if A[j] > A[j+1]:
                A[j], A[j+1] = A[j+1], A[j]
                swapped = True
        if not swapped:
            break
```

小结：冒泡排序是嵌套迭代的典型代表，它通过双重循环实现了元素间的比较与交换，虽然效率不高，但直观易懂，适合初学者理解排序的基本思想。后续将学习更高效的排序算法（如快速排序、归并排序），它们往往采用分治策略，时间复杂度可降至 $O(n \log n)$ 。

2.4 多表迭代

多表迭代是指同时遍历多个线性表，常用于合并、比较等操作。它的核心思想是同时维护多个指针，分别指向各个表，根据条件决定前进哪个指针。

2.4.1 合并两个有序数组

问题：给定两个非递减有序数组 a 和 b ，长度分别为 m 和 n ，将它们合并为一个新的非递减有序数组 c 。

设计思路：使用两个指针 i 和 j 分别指向两个数组的起始位置，再使用一个指针 k 指向结果数组的当前位置。比较 $a[i]$ 和 $b[j]$ ，将较小的放入 $c[k]$ ，并移动相应指针（如 Figure 2.8 所示）。当某个数组遍历完后，将另一个数组的剩余元素直接复制到结果数组。

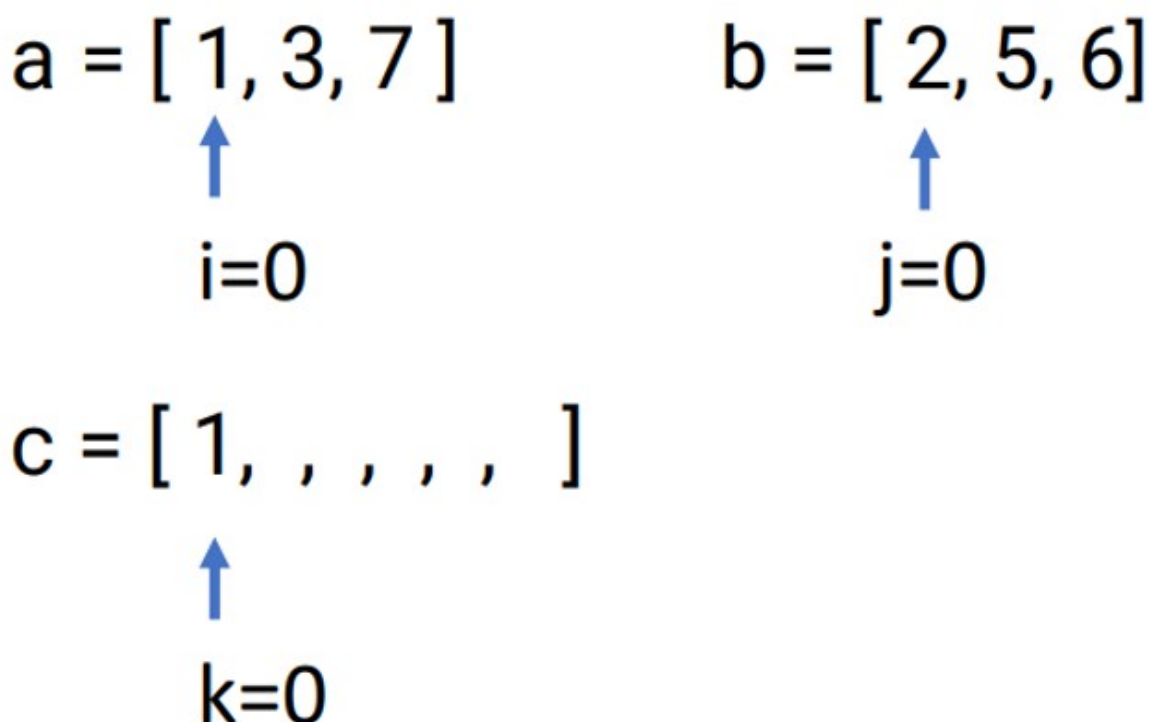


Figure 2.8: 有序表的合并：使用两个指针 i 和 j 分别指向两个数组的起始位置，再使用一个指针 k 指向结果数组的当前位置。比较 $a[i]$ 和 $b[j]$ ，将较小的放入 $c[k]$ ，并移动相应 i 或 j 、 k

算法分析：每个元素恰好被访问一次，时间复杂度 $O(m + n)$ 。需要额外 $O(m + n)$ 空间存放结果数组。

伪代码：

```
merge_sorted_arrays(a[0..m-1], b[0..n-1]):
    c = new array of size m+n
    i = 0; j = 0; k = 0
    while i < m and j < n:
        if a[i] ≤ b[j]:
            c[k] = a[i]; i = i+1
        else:
            c[k] = b[j]; j = j+1
        k = k+1
    while i < m:
        c[k] = a[i]; i = i+1; k = k+1
    while j < n:
        c[k] = b[j]; j = j+1; k = k+1
    return c
```

C++ 实现：

```
vector<int> mergeSortedArrays(const vector<int>& a, const vector<int>& b) {
    int m = a.size(), n = b.size();
    vector<int> c(m + n);
    int i = 0, j = 0, k = 0;
    while (i < m && j < n) {
        if (a[i] <= b[j]) {
            c[k++] = a[i++];
        } else {
            c[k++] = b[j++];
        }
    }
    while (i < m) c[k++] = a[i++];
    while (j < n) c[k++] = b[j++];
    return c;
}
```

Python 实现:

```
def merge_sorted_arrays(a, b):
    i, j = 0, 0
    c = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            c.append(a[i])
            i += 1
        else:
            c.append(b[j])
            j += 1
    c.extend(a[i:])
    c.extend(b[j:])
    return c
```

2.4.2 合并两个有序链表

问题: 将两个递增链表合并为一个新的递增链表并返回。新链表通过拼接原链表的结点组成。

设计思路: 与数组合并类似，但链表操作涉及指针修改。为了简化边界条件，可以引入一个虚拟头结点 dummy，其 next 指向合并后的链表头。然后用 tail 指向当前合并链表的尾结点。比较两个链表当前结点的值，将较小的结点挂到 tail 后面，并移动相应指针。当一个链表遍历完后，将另一个链表剩余部分直接链接到 tail 后面。Figure 2.9

算法分析: 每个结点访问一次，时间复杂度 $O(m + n)$ 。空间复杂度 $O(1)$ （仅使用几个指针，未创建新结点）。

伪代码:

```
merge_two_lists(list1, list2):
    dummy = new Node() // 虚拟头结点
```

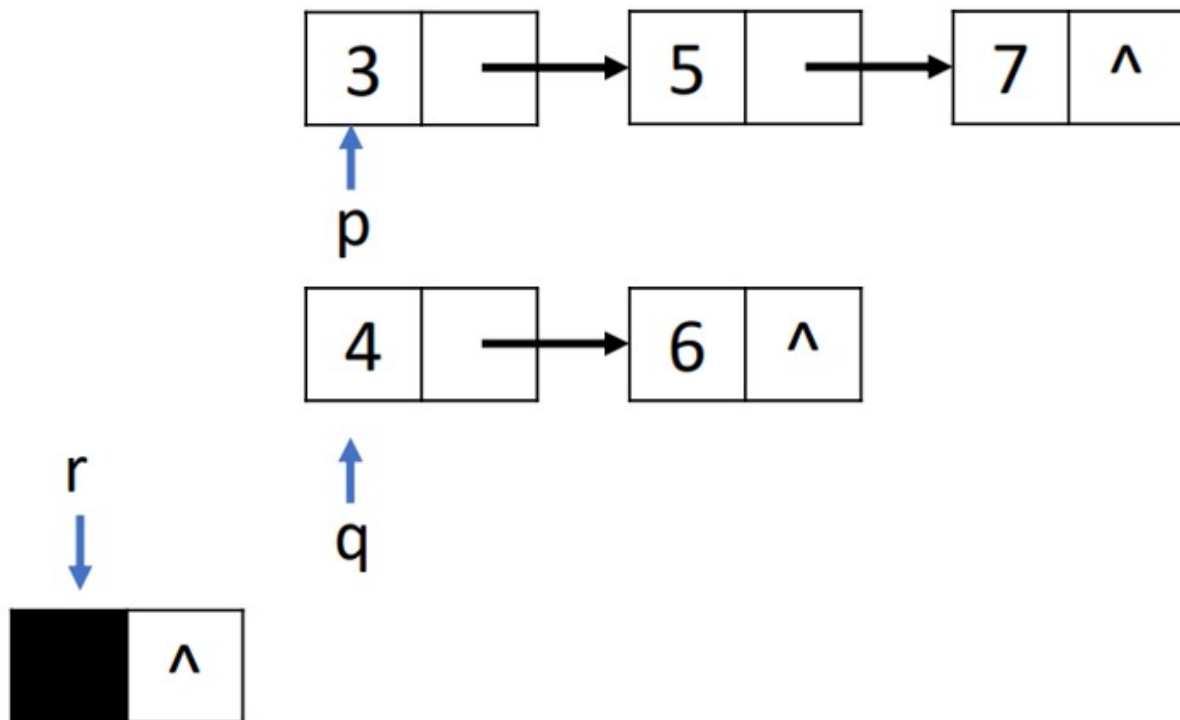


Figure 2.9: 合并两个有序链表

```

tail = dummy
p = list1
q = list2
while p != null and q != null:
    if p.data <= q.data:
        tail.next = p
        p = p.next
    else:
        tail.next = q
        q = q.next
    tail = tail.next
if p != null:
    tail.next = p
else:
    tail.next = q
result = dummy.next
// 释放 dummy (可选)
return result

```

C++ 实现:

```

ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
    ListNode dummy(0); // 虚拟头结点
    ListNode* tail = &dummy;

```



```
ListNode* p = list1;
ListNode* q = list2;
while (p && q) {
    if (p->val <= q->val) {
        tail->next = p;
        p = p->next;
    } else {
        tail->next = q;
        q = q->next;
    }
    tail = tail->next;
}
tail->next = p ? p : q;
return dummy->next;
}
```

Python 实现:

```
def merge_two_lists(list1, list2):
    dummy = ListNode() # 虚拟头结点
    tail = dummy
    p, q = list1, list2
    while p and q:
        if p.val <= q.val:
            tail.next = p
            p = p.next
        else:
            tail.next = q
            q = q.next
        tail = tail.next
    tail.next = p if p else q
    return dummy.next
```

2.5 双指针迭代

双指针法是对线性穷举的一种优化技巧，通过两个指针协同遍历，避免不必要的穷举，从而提高效率。根据指针移动方向，可分为三类：

- **左右指针（夹逼指针）**：两个指针从两端向中间移动（相向而行），或从中间向两端移动（背向而行）。
- **快慢指针（前后指针）**：两个指针从同一端出发，以不同速度同向移动。
- **滑动窗口**：两个指针同向移动，但窗口大小动态变化，用于处理子数组/子串问题。

2.5.1 左右指针（相向而行）

左右指针常用于有序数组或字符串问题，通过控制两个指针从两端向中间移动，逐步缩小搜索范围。

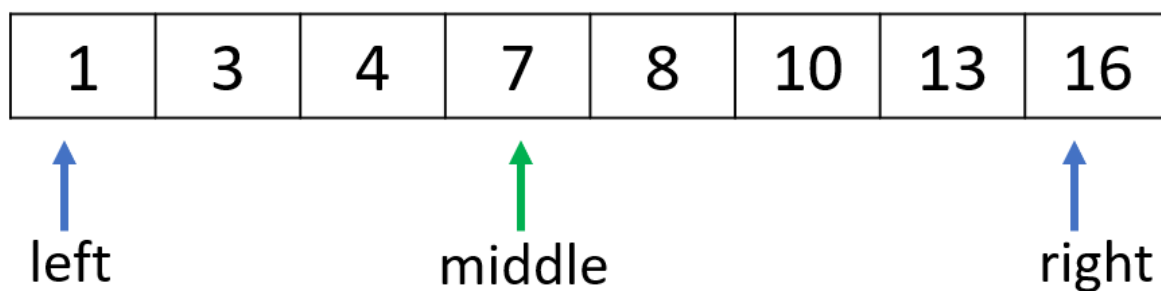
2.5.1.1 二分查找

问题：在有序数组中查找目标值 x 。

设计思路：二分查找利用了数组有序的性质。初始时 $left$ 指向数组开头， $right$ 指向数组末尾。每次取中间位置 mid ，比较 $A[mid]$ 与 x (如 Figure 2.10 所示)：

- 如果相等，则找到；
- 如果 $A[mid] < x$ ，说明目标在右半部分，将 $left$ 更新为 $mid+1$ ；
- 否则目标在左半部分，将 $right$ 更新为 $mid-1$ 。重复直到 $left > right$ ，表示未找到。

查找4



查找4

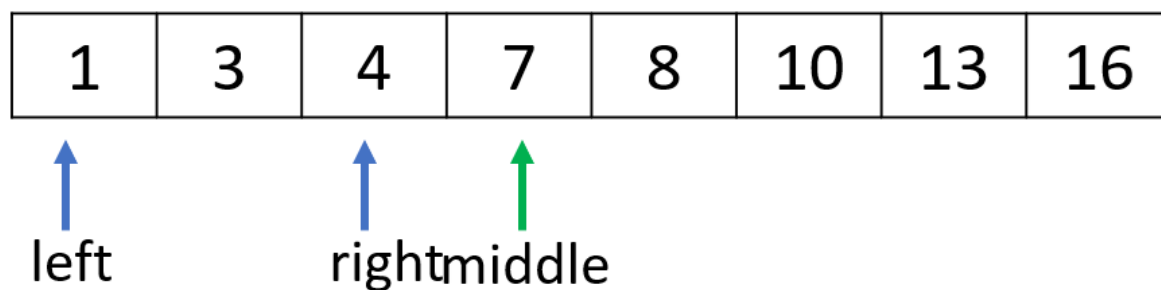


Figure 2.10: 二分查找

算法分析：每次将搜索区间减半，时间复杂度 $O(\log n)$ 。空间复杂度 $O(1)$ 。

伪代码：

```

binary_search(A[0..n-1], x):
    left = 0
    right = n-1
    while left ≤ right:
        mid = left + floor((right-left)/2)
        if A[mid] == x:
            return mid
        else if A[mid] < x:
            left = mid + 1
        else:
            right = mid - 1
    return -1

```

C++ 实现:

```

int binarySearch(vector<int>& A, int x) {
    int left = 0, right = A.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (A[mid] == x) return mid;
        else if (A[mid] < x) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

```

Python 实现:

```

def binary_search(A, x):
    left, right = 0, len(A) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if A[mid] == x:
            return mid
        elif A[mid] < x:
            left = mid + 1
        else:
            right = mid - 1
    return -1

```

2.5.1.2 回文串判断

问题: 给定一个字符串, 判断它是否为回文串 (正读反读相同)。

设计思路: 回文串具有对称性, 因此可以用左右指针从两端向中间移动, 比较对应字符是否相等。如果某对字符不等, 则不是回文; 如果所有对应字符都相等, 则是回文。如 Figure 2.11 所示:

a b c d c b a

↑ ↑

left right

a b c d c b a

↑ ↑

left right

Figure 2.11: 回文串的判断：用左右指针从两端向中间移动，比较对应字符是否相等。如果某对字符不等，则不是回文；如果所有对应字符都相等，则是回文

算法分析：最多比较 $n/2$ 次，时间复杂度 $O(n)$ 。空间复杂度 $O(1)$ 。

伪代码：

```
is_palindrome(s):
    left = 0
    right = len(s) - 1
    while left < right:
        if s[left] ≠ s[right]:
            return false
        left = left + 1
        right = right - 1
    return true
```

C++ 实现：

```
bool isPalindrome(string s) {
    int left = 0, right = s.size() - 1;
    while (left < right) {
        if (s[left] ≠ s[right]) return false;
        left++;
        right--;
    }
    return true;
}
```

Python 实现：

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] ≠ s[right]:
            return False
        left += 1
        right -= 1
    return True
```

2.5.1.3 两数之和 II（输入有序数组）

问题（LeetCode 167）：给定一个已按升序排列的整数数组 `numbers` 和一个目标值 `target`，请找出两个数使得它们的和等于目标值，返回它们的下标（从 1 开始计数）。

设计思路：利用数组有序的特性，使用左右指针。初始时 `left` 指向最小元素，`right` 指向最大元素。计算当前和 `sum = numbers[left] + numbers[right]`（如 Figure 2.12 所示）：

- 若 `sum == target`，则找到答案；
- 若 `sum < target`，说明和太小，需要增大，因此将 `left` 右移（指向更大的数）；

- 若 $\text{sum} > \text{target}$ ，说明和太大，需要减小，因此将 right 左移（指向更小的数）。这样每次移动一个指针，直至找到目标或指针相遇。

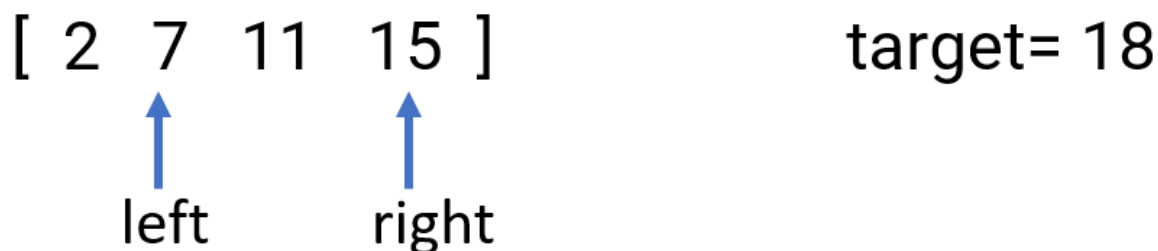
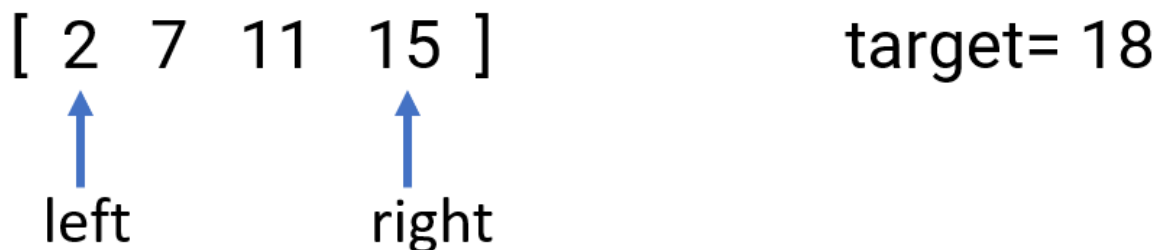


Figure 2.12: 夹逼指针解决有序数组两数之和问题

算法分析： 每个元素最多被访问一次，时间复杂度 $O(n)$ 。空间复杂度 $O(1)$ 。

伪代码：

```
two_sum_ii(numbers[0..n-1], target):
    left = 0
    right = n-1
    while left < right:
        sum = numbers[left] + numbers[right]
        if sum == target:
            return (left+1, right+1)
        else if sum < target:
            left = left + 1
        else:
            right = right - 1
    return (-1, -1)
```

C++ 实现：

```
vector<int> twoSumII(vector<int>& numbers, int target) {
    int left = 0, right = numbers.size() - 1;
    while (left < right) {
```

```

    int sum = numbers[left] + numbers[right];
    if (sum == target) return {left + 1, right + 1};
    else if (sum < target) left++;
    else right--;
}
return {-1, -1};
}

```

Python 实现:

```

def two_sum_ii(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        s = numbers[left] + numbers[right]
        if s == target:
            return [left + 1, right + 1]
        elif s < target:
            left += 1
        else:
            right -= 1
    return [-1, -1]

```

2.5.2 左右指针（背向而行）

背向而行是指两个指针从中间向两端移动，常用于需要从中心向两边扩展的问题，如最长回文子串。

2.5.2.1 最长回文子串（LeetCode 5）

1) 问题: 给定一个字符串 s ，需要找出其中最长的回文子串。回文串是指正读和反读都相同的字符串，例如“aba”、“abba”都是回文串。

形式化定义: 对于字符串 $s[1..n]$ ，求最大的长度 L ，使得存在下标 i 和 j ($1 \leq i \leq j \leq n$)，满足 $s[i..j]$ 是回文串，且 $j - i + 1 = L$ 。输出该子串（或长度）。

应用背景: 最长回文子串问题在多个领域有重要应用：

- **生物信息学:** 在 DNA 序列中寻找回文结构，这些结构可能与基因调控有关。
- **数据压缩:** 某些压缩算法利用重复模式，回文是一种特殊的对称重复。
- **密码学:** 回文串在密码分析中可能作为特征出现。
- **文本处理:** 用于查找对称模式，如诗歌中的回文句。

2) 示例

考虑字符串 $s = \text{"babad"}$ 。我们需要找出其中的最长回文子串。可能的回文子串有：

- “b”（单个字符，长度为 1）
- “a”（长度为 1）
- “bab”（从下标 1 到 3，长度为 3）

- “aba” (从下标 2 到 4, 长度为 3)
- “bad” 不是回文
- 等等

其中“bab”和“aba”长度均为 3, 都是最长回文子串(答案可能不唯一)。通常我们只需要输出任意一个, 例如“bab”。

另一个例子: $s = \text{”cbbd”}$, 最长回文子串是“bb”, 长度为 2。

这个示例说明, 回文子串可以是奇数长度(如“bab”)或偶数长度(如“bb”), 寻找时需要同时考虑这两种情况。

3) 蛮力法

最直接的思路是枚举所有可能的子串, 检查每个子串是否为回文, 并记录最长的。

算法描述:

1. 枚举子串的起始位置 i 从 1 到 n 。
2. 对于每个 i , 枚举结束位置 j 从 i 到 n 。
3. 对于子串 $s[i..j]$, 检查它是否是回文: 可以双指针从两端向中间比较。
4. 如果是回文且长度大于当前最长, 则更新最长回文子串。

复杂度分析: - 子串个数为 $O(n^2)$ 。 - 每个子串检查回文需要 $O(n)$ 时间(最坏情况)。 - 总时间复杂度 $O(n^3)$ 。

尽管可以通过提前终止等方式优化常数, 但 $O(n^3)$ 的上限使得该算法对于稍长的字符串(如 $n = 1000$) 就变得不可行。例如, $n = 1000$ 时, 子串数约 5×10^5 , 每个子串平均比较长度约 500, 总操作数达 2.5×10^8 , 实际运行时间可能难以接受。

4) 背向指针(中心扩展)思想

关键观察: 回文串是关于其中心对称的。例如: - 奇数长度回文“aba”的中心是中间的‘b’(下标 2), 向左和向右扩展相同字符。 - 偶数长度回文“abba”的中心是两个字符“bb”之间的空隙(可以想象中心在‘b’和‘b’之间), 向左和向右扩展也相同。

因此, 我们可以枚举每一个可能的回文中心, 然后从中心向两边扩展(如 Figure 2.13 所示), 直到不能形成回文为止。这样, 每个中心对应一个最长的回文子串, 我们取所有中心中的最大值即可。



Figure 2.13: 背向指针计算从中心向两边扩展的最长回文子串

为什么比蛮力法好?

1. 蛮力法枚举了所有子串, 而许多子串并不是回文, 我们却花费了时间去检查。中心扩展法只考虑那些以某个中心向两边扩展得到的子串, 这些子串天然就是回文(直到扩展失败), 从而避免了无效检查。

2. 每个中心扩展时, 只需 $O(n)$ 时间 (最坏情况扩展整个字符串), 而中心个数为 $2n - 1$ (因为每个字符和每两个相邻字符之间都可以作为中心), 因此总时间复杂度 $O(n^2)$, 且空间复杂度 $O(1)$ 。

这种“背向而行”的双指针方法, 正是利用了回文的对称性质, 从中心向两边扩散, 类似于双指针从中间向两端移动。

5) 算法描述

输入: 字符串 s , 长度为 n

输出: 最长回文子串

1. 初始化最长回文子串的起始位置 $start = 0$, 长度 $maxLen = 1$ (单个字符总是回文)。
2. 遍历所有可能的中心位置:
 - 对于每个下标 i 从 0 到 $n - 1$, 考虑两种中心:
 - a. 奇数长度中心: 以 $s[i]$ 为中心, 向两边扩展, 即左右指针 $l = i - 1$, $r = i + 1$ 。
 - b. 偶数长度中心: 以 $s[i]$ 和 $s[i + 1]$ 之间的空隙为中心, 即左右指针 $l = i$, $r = i + 1$ 。
3. 对于每个中心, 执行扩展:
 - 当 $l \geq 0$ 且 $r < n$ 且 $s[l] == s[r]$ 时, 继续扩展: $l --$, $r ++$ 。
 - 循环结束后, 当前中心对应的最长回文子串是从 $l + 1$ 到 $r - 1$, 长度为 $(r - 1) - (l + 1) + 1 = r - l - 1$ 。
 - 如果该长度大于 $maxLen$, 则更新 $start = l + 1$, $maxLen = r - l - 1$ 。
4. 返回子串 $s[start..start + maxLen - 1]$ 。

注意: 需要小心处理边界条件, 确保指针不越界。

6) 伪代码

Algorithm LongestPalindrome(s):

Input: 字符串 s , 长度为 n

Output: 最长回文子串

```

if n == 0: return ""
start = 0
maxLen = 1

for i = 0 to n-1:
    // 奇数长度中心
    l = i - 1
    r = i + 1
    while l ≥ 0 and r < n and s[l] == s[r]:
        l = l - 1
        r = r + 1
    // 当前回文区间为 [l+1, r-1]
    curLen = r - l - 1
    if curLen > maxLen:
        maxLen = curLen
        start = l + 1

    // 偶数长度中心

```

```

    l = i
    r = i + 1
    while l ≥ 0 and r < n and s[l] == s[r]:
        l = l - 1
        r = r + 1
    curLen = r - l - 1
    if curLen > maxLen:
        maxLen = curLen
        start = l + 1

return s[start .. start+maxLen-1]

```

7) 代码实现

C++ 实现

```

#include <iostream>
#include <string>
using namespace std;

string longestPalindrome(string s) {
    int n = s.length();
    if (n == 0) return "";
    int start = 0, maxLen = 1;

    for (int i = 0; i < n; i++) {
        // 奇数长度
        int l = i - 1, r = i + 1;
        while (l >= 0 && r < n && s[l] == s[r]) {
            l--; r++;
        }
        int curLen = r - l - 1;
        if (curLen > maxLen) {
            maxLen = curLen;
            start = l + 1;
        }

        // 偶数长度
        l = i, r = i + 1;
        while (l >= 0 && r < n && s[l] == s[r]) {
            l--; r++;
        }
        curLen = r - l - 1;
        if (curLen > maxLen) {

```

```

        maxLen = curLen;
        start = l + 1;
    }
}
return s.substr(start, maxLen);
}

int main() {
    string s = "babad";
    cout << " 最长回文子串: " << longestPalindrome(s) << endl;
    return 0;
}

```

Python 实现

```

def longest_palindrome(s):
    n = len(s)
    if n == 0:
        return ""
    start, max_len = 0, 1

    for i in range(n):
        # 奇数长度中心
        l, r = i - 1, i + 1
        while l >= 0 and r < n and s[l] == s[r]:
            l -= 1
            r += 1
        cur_len = r - l - 1
        if cur_len > max_len:
            max_len = cur_len
            start = l + 1

        # 偶数长度中心
        l, r = i, i + 1
        while l >= 0 and r < n and s[l] == s[r]:
            l -= 1
            r += 1
        cur_len = r - l - 1
        if cur_len > max_len:
            max_len = cur_len
            start = l + 1

    return s[start:start+max_len]

```

```
# 测试
s = "babad"
print(" 最长回文子串:", longest_palindrome(s))
```

8) 复杂度分析

时间复杂度：外层循环有 n 个位置，每个位置处理两种中心，每个中心最多扩展 n 次（最坏情况如全相同字符），因此总时间 $O(n^2)$ 。虽然最坏是 $O(n^2)$ ，但平均情况通常接近 $O(n^2)$ ，且实现简单，空间开销小。

空间复杂度：只使用了常数个额外变量，因此空间复杂度为 $O(1)$ 。

9) 算法扩展

中心扩展法虽然简单，但仍有更优算法：- **动态规划**：用 $dp[i][j]$ 表示子串 $s[i..j]$ 是否为回文，可在 $O(n^2)$ 时间和 $O(n^2)$ 空间内求解。- **Manacher 算法**：能在 $O(n)$ 时间内找到最长回文子串，但算法较复杂，需要巧妙利用对称性。

中心扩展法作为理解回文对称性的入门算法，极具启发性，为学习更高级算法奠定了基础。

10) 思考题

1. 如果要求输出所有最长回文子串（而不是任意一个），如何修改算法？
2. 中心扩展法能否处理长度为 1 的字符串？为什么？
3. 尝试用动态规划实现最长回文子串，并比较两种方法的优劣。
4. 给定一个字符串，如何用中心扩展法统计所有回文子串的个数？

2.5.3 快慢指针

快慢指针是指两个指针从同一端出发，以不同速度移动，常用于链表问题，如检测环、找中点、找倒数第 k 个结点等。

2.5.3.1 奇偶调序

问题：将整数数组中的所有奇数放在所有偶数之前。

设计思路：使用快慢指针。如 Figure 2.14 所示，慢指针 $slow$ 指向下一个奇数应该存放的位置，快指针 $fast$ 遍历数组寻找奇数。当 $fast$ 找到奇数时，将其与 $slow$ 位置的元素交换，然后 $slow$ 前进一位。这样遍历结束后，所有奇数都被移到了数组前部，且相对顺序保持不变（因为是从前向后扫描）。

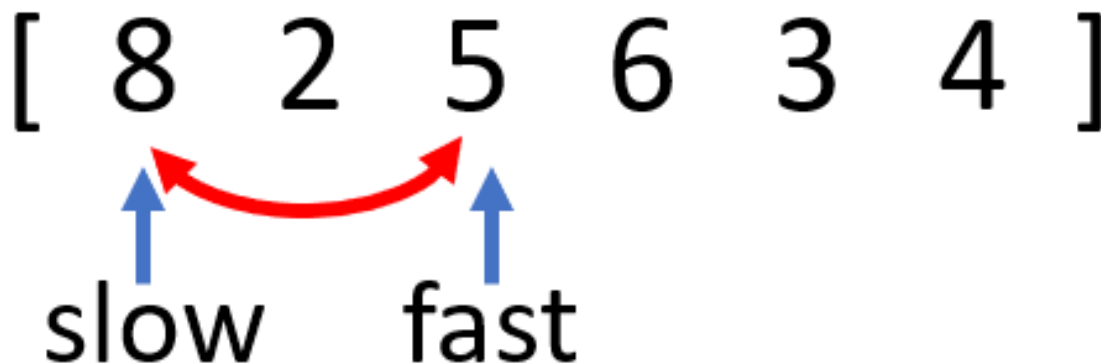


Figure 2.14: “奇偶调序”的快慢指针算法

算法分析：只需遍历一次数组，时间复杂度 $O(n)$ 。空间复杂度 $O(1)$ （原地交换）。

伪代码：

```
reorder_odd_even(A[0..n-1]):
    slow = 0
    for fast = 0 to n-1:
        if A[fast] % 2 == 1:
            swap(A[slow], A[fast])
            slow = slow + 1
```

C++ 实现：

```
void reorderOddEven(vector<int>& A) {
    int slow = 0;
    for (int fast = 0; fast < A.size(); fast++) {
        if (A[fast] % 2 == 1) {
            swap(A[slow], A[fast]);
            slow++;
        }
    }
}
```

Python 实现：

```
def reorder_odd_even(A):
    slow = 0
    for fast in range(len(A)):
        if A[fast] % 2 == 1:
            A[slow], A[fast] = A[fast], A[slow]
            slow += 1
```

2.5.3.2 删除有序数组中的重复项 (LeetCode 26)

1) 问题提出

给定一个 **有序数组** `nums`，要求原地删除重复出现的元素，使得每个元素只出现一次，并返回删除后数组的新长度。不能使用额外的数组空间，必须通过原地修改输入数组来完成。

形式化定义：

对于有序数组 `nums[0..n-1]`，删除重复项后，前 k 个元素应包含原数组中的所有不重复元素，且保持相对顺序，函数返回 k 。之后数组的前 k 个元素即为结果（超出部分可忽略）。

应用背景：数组去重是数据处理中的常见操作，在数据清洗、集合运算等场景中经常用到。利用有序性质可以高效地实现去重。

2) 直观示例

输入: `nums = [1, 1, 2]`

输出: 新长度 2, 修改后的数组前两个元素为 `[1, 2]` (后面元素无所谓)。

输入: `nums = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]`

输出: 新长度 5, 修改后的数组前五个元素为 `[0, 1, 2, 3, 4]`。

3) 蛮力法

最直接的想法是: 遍历数组, 每当发现一个重复元素, 就将它后面的所有元素向前移动一位, 覆盖掉这个重复元素。这样需要反复移动元素。

算法描述

- 用变量 `i` 遍历数组, 同时用变量 `len` 记录当前有效长度 (初始为 `n`)。
- 当 `i < len-1` 时, 检查 `nums[i]` 和 `nums[i+1]` 是否相等:
 - 若相等, 说明存在重复, 将下标 `i+1` 到 `len-1` 的元素整体前移一位, 然后 `len--` (注意此时 `i` 不要增加, 因为新的 `nums[i+1]` 可能是原来 `nums[i+2]`, 需要继续比较)。
 - 若不相等, 则 `i++` 继续检查下一个位置。
- 循环结束后, 返回 `len`。

复杂度分析

- **时间复杂度:** 最坏情况下, 每次删除都需要移动 $O(n)$ 个元素, 总共可能删除 $O(n)$ 次, 因此最坏时间复杂度为 $O(n^2)$ 。
- **空间复杂度:** 只使用常数个辅助变量, $O(1)$ 。

虽然空间符合要求, 但时间效率太低, 不适合处理大规模数据。

4) 快慢指针法

利用数组有序的特点, 我们可以使用 **快慢指针** 技巧, 在一次遍历中完成去重。

核心思想

- 定义两个指针 `slow` 和 `fast`, 初始都指向数组开头。
- `fast` 指针负责遍历整个数组, 寻找新的不重复元素。
- `slow` 指针指向当前已处理好的无重复数组的末尾 (即下一个不重复元素应该存放的位置)。
- 因为数组有序, 所以当 `nums[fast] != nums[slow]` 时, 说明找到了一个新的不重复元素, 将其放到 `slow+1` 位置, 然后 `slow` 前进一位。(如 Figure 2.15 所示)

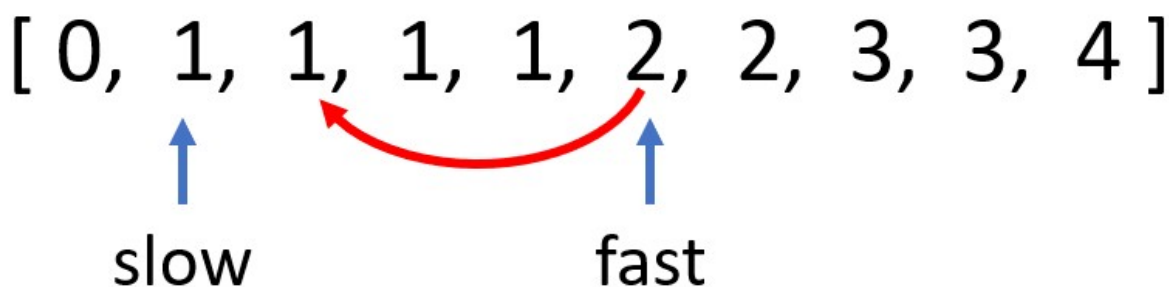


Figure 2.15: 快慢指针将快指针 (`fast`) 遇到的奇数移到慢指针 (`slow`) 的位置

算法描述

1. 如果数组长度 $n = 0$ ，直接返回 0。
2. 初始化 $slow = 0$ 。
3. 用 $fast$ 从 1 遍历到 $n-1$:
 - 如果 $nums[fast] \neq nums[slow]$ ，则说明 $nums[fast]$ 是新的不重复元素，将 $slow++$ ，然后将 $nums[fast]$ 赋值给 $nums[slow]$ 。
4. 遍历结束后， $slow+1$ 就是新数组的长度，返回 $slow+1$ 。

代码实现

C++ 版本

```
int removeDuplicates(vector<int>& nums) {
    if (nums.empty()) return 0;
    int slow = 0;
    for (int fast = 1; fast < nums.size(); fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1;
}
```

Python 版本

```
def removeDuplicates(nums):
    if not nums:
        return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1
```

复杂度分析

- **时间复杂度：** $O(n)$ ，只需遍历数组一次。
- **空间复杂度：** $O(1)$ ，只使用了常数个指针变量。

5) 对比总结

方法	时间复杂度	空间复杂度	实现复杂度
蛮力法	$O(n^2)$	$O(1)$	简单
快慢指针法	$O(n)$	$O(1)$	简洁

- **蛮力法** 虽然直观，但需要频繁移动大量元素，效率极低，仅适用于理解问题本质。

- **快慢指针法** 充分利用了数组有序的性质，通过两个指针一次遍历完成去重，是解决此类问题的经典高效方法。

快慢指针法不仅适用于删除有序数组中的重复项，还可推广到其他需要原地修改的数组问题，如移除指定元素、移动零等。掌握这一技巧对算法学习大有裨益。

2.5.3.3 判断链表是否有环 (LeetCode 141)

1) 问题:

给定一个单链表的头节点 head，判断该链表中是否存在环（循环链表）。如果链表中有某个节点，可以通过连续跟踪 next 指针再次到达该节点，则链表中存在环 (Figure 2.16)。

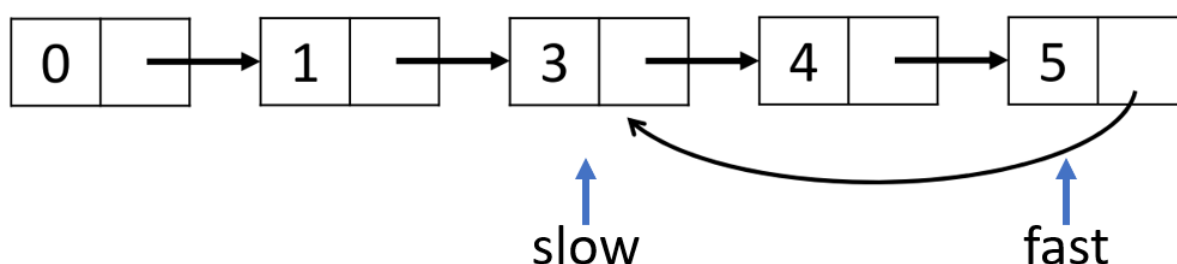


Figure 2.16: 链表中是否存在环?

形式化定义:

链表由一系列节点组成，每个节点包含一个值和一个指向下一个节点的指针 next。如果链表中存在某个节点，从它出发沿着 next 指针可以回到自身或之前访问过的节点，则称该链表存在环。

应用背景: 检测链表中的环在计算机科学中具有重要意义，例如：- 内存管理：检测内存分配中的循环引用。- 数据结构：验证链表操作的正确性，避免无限循环。- 算法设计：作为许多复杂算法的基础，如寻找环入口、计算环长度等。

2) 示例

考虑如下链表：1 → 2 → 3 → 4 → 5 → 3（即 5 的 next 指向 3），这样就形成了一个环。

从节点 1 开始遍历：1 → 2 → 3 → 4 → 5 → 3 → 4 → 5 → 3 ... 永远无法到达 null。如何检测这种结构？

3) 蛮力法 (哈希表)

最直接的方法是记录已经访问过的节点。遍历链表，每遇到一个新节点，就检查它是否已经出现在记录中；如果是，则存在环；否则将其加入记录。当遍历到 null 时，说明无环。

算法步骤: 1. 创建一个哈希表（或集合）visited。2. 从 head 开始遍历，对于每个节点 cur：- 如果 cur 已经在 visited 中，说明存在环，返回 true。- 否则将 cur 加入 visited，并继续 cur = cur→next。3. 如果遍历到 cur = null，则返回 false。

复杂度分析: - 时间复杂度： $O(n)$ ，最坏情况下每个节点访问一次。- 空间复杂度： $O(n)$ ，需要存储所有访问过的节点。

缺点: 需要使用额外的存储空间，当链表很大时可能占用较多内存。

4) 快慢指针法 (Floyd 判圈算法)

利用两个指针，一个快一个慢，同时遍历链表。如果链表中没有环，快指针会先到达末尾（null）；如果存在环，快指针最终会追上慢指针，两者相遇。

核心理想： - 慢指针 slow 每次移动一步，快指针 fast 每次移动两步。 - 若无环，fast 会先到达 null，算法结束。
- 如果有环，fast 和 slow 最终都会进入环内，由于相对速度差为 1 步，fast 一定会在某时刻追上 slow（即相遇）。

为什么相遇一定能发生？

当两个指针都进入环后，可以看作在环形跑道上，fast 以 1 步的相对速度追赶 slow，因此必定会在有限步内相遇。

5) 算法描述

1. 如果 head == null 或 head→next == null，直接返回 false（至少两个节点才能形成环）。
2. 初始化两个指针：slow = head, fast = head。
3. 当 fast ≠ null 且 fast→next ≠ null 时，循环执行：
 - slow = slow→next（移动一步）
 - fast = fast→next→next（移动两步）
 - 如果 slow == fast，则说明相遇，存在环，返回 true。
4. 如果循环结束（fast 到达末尾），则返回 false。

6) 伪代码

```
function hasCycle(head):
    if head == null or head.next == null:
        return false
    slow = head
    fast = head
    while fast ≠ null and fast.next ≠ null:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return true
    return false
```

7) 代码实现

C++ 实现

```
struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

bool hasCycle(ListNode *head) {
    if (head == nullptr || head->next == nullptr)
        return false;

    ListNode *slow = head;
```

```

ListNode *fast = head;

while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast) {
        return true;
    }
}
return false;
}

```

Python 实现

```

class ListNode:
    def __init__(self, x):
        self.val = x
        self.next = None

def hasCycle(head):
    if not head or not head.next:
        return False

    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False

```

8) 复杂度分析

- **时间复杂度：** $O(n)$ 。
无环时，快指针遍历 n 个节点；有环时，慢指针进入环前最多走 n 步，相遇时走的步数不超过环的长度，总步数仍是 $O(n)$ 。
- **空间复杂度：** $O(1)$ ，只使用了两个指针，无需额外存储。

与蛮力法相比，快慢指针法在保持线性时间的同时，将空间复杂度降为常数，是更优的解法。

9) 思考题

1. 如何找到环的入口节点（即环的起始位置）？
2. 如何计算环的长度？

3. 快慢指针的步长可以设置为其他值吗？比如 fast 每次走 3 步是否可行？为什么通常选择 2 步？

2.5.3.4 寻找环的入口

问题 (LeetCode 142)：给定一个链表，返回链表开始入环的第一个节点。如果链表无环，则返回 null。

设计思路：首先用快慢指针判断是否有环，并找到相遇点。然后根据数学推导（详见注释），将其中一个指针放回链表头，两个指针都以一步的速度移动，再次相遇的位置即为环入口。

数学推导：如 Figure 2.17 设链表头到环入口距离为 a ，环入口到相遇点距离为 b ，相遇点再到环入口距离为 c （环周长 $L = b + c$ ）。快慢指针相遇时，慢指针走了 $a + b$ ，快指针走了 $a + b + kL$ （ k 为快指针在环内绕的圈数）。由于快指针速度是慢指针的 2 倍，有 $2(a + b) = a + b + kL$ ，得 $a + b = kL$ 。因此 $a = kL - b = (k - 1)L + c$ 。所以当慢指针从相遇点继续走，另一指针从头开始走，两者会在环入口相遇。



Figure 2.17: 寻找环的入口

算法分析：时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

伪代码：

```
detect_cycle(head):
    // 寻找相遇点
    slow = head
    fast = head
    has_cycle = false
    while fast != null and fast.next != null:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            has_cycle = true
            break
    if not has_cycle:
        return null

    // 寻找环入口
```

```

slow = head
while slow != fast:
    slow = slow.next
    fast = fast.next
return slow

```

C++ 实现:

```

ListNode* detectCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    bool hasCycle = false;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            hasCycle = true;
            break;
        }
    }
    if (!hasCycle) return nullptr;

    slow = head;
    while (slow != fast) {
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}

```

Python 实现:

```

def detect_cycle(head):
    slow = fast = head
    has_cycle = False
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            has_cycle = True
            break
    if not has_cycle:
        return None

    slow = head

```

```
while slow is not fast:
    slow = slow.next
    fast = fast.next
return slow
```

2.6 滑动窗口

滑动窗口是双指针技巧的一种重要形式，它通过维护一个窗口（即两个指针之间的区间）来解决问题。窗口的左右边界随着遍历而动态调整，常用于处理子数组或子串相关的问题，例如寻找满足特定条件的最长/最短连续子序列。

2.6.0.1 基本思想

滑动窗口的核心是使用两个指针 `left` 和 `right` 表示窗口的左右边界，初始时通常都指向序列的起始位置。然后不断移动 `right` 指针扩大窗口，当窗口内的元素满足（或不满足）某个条件时，再移动 `left` 指针缩小窗口。在这个过程中，我们记录下符合条件的窗口，并从中找出最优解。

滑动窗口之所以高效，是因为它避免了重复计算：当窗口滑动时，只需考虑新加入或移出的元素，而不是重新计算整个子数组。这样可以将许多原本需要 $O(n^2)$ 时间的问题优化到 $O(n)$ 。

常见类型

根据问题的不同，滑动窗口可分为两类：

1. **固定窗口大小**：窗口大小固定，只需滑动窗口并维护窗口内的信息（如最大值、最小值、和等）。
2. **可变窗口大小**：窗口大小可变，需要根据条件动态调整左右边界，通常用于寻找满足约束的最长/最短子数组。

下面通过几个经典例子来深入理解滑动窗口的应用。

2.6.0.2 长度最小的子数组 (LeetCode 209)

问题：给定一个含有 n 个正整数的数组和一个正整数 `target`，找出该数组中满足其和 $\geq target$ 的长度最小的连续子数组，并返回其长度。如果不存在符合条件的子数组，返回 0。

例如：`nums = [2,3,1,2,4,3]`，`target = 7`，最短子数组是 `[4,3]`，和为 7，长度为 2。

核心思想

滑动窗口是一种双指针技巧，维护一个动态的窗口（连续子数组），通过调整左右指针来探索所有可能的子数组，同时利用窗口内元素和的单调性（所有元素为正）来避免重复计算。

步骤分解：

1. **初始化**：左指针 `left = 0`，右指针 `right = 0`，窗口和 `sum = 0`，最小长度 `minLen = 无穷大`。
2. **扩展右指针**：不断右移 `right`，将 `nums[right]` 加入窗口，更新 `sum`。这一步是为了找到一个满足 `sum  $\geq$  target` 的窗口。
3. **收缩左指针**：一旦 `sum  $\geq$  target`，说明当前窗口已满足条件，但可能不是最短的。此时尝试右移 `left` 来缩小窗口：
 - 记录当前窗口长度 `right - left + 1`，并更新 `minLen` 为较小值。

- 将 `nums[left]` 移出窗口, `sum -= nums[left]`, `left++`。
 - 重复上述过程, 直到 `sum < target` (窗口不再满足条件)。
4. 重复步骤 2 和 3: 继续右移 `right`, 重复“扩展 → 收缩”过程, 直到 `right` 遍历完整个数组。
 5. 结果: 若 `minLen` 仍为无穷大, 返回 0; 否则返回 `minLen`。

为什么高效?

- 每个元素最多被加入窗口一次、移出窗口一次, 因此时间复杂度为 $O(n)$ 。
- 利用窗口和单调性 (增加元素和增加, 减少元素和减少), 避免了暴力枚举所有子数组的 $O(n^2)$ 开销。
- 空间复杂度 $O(1)$ 。

以示例数组演示过程

数组: [2, 3, 1, 2, 4, 3], target = 7。

步骤	left	right	窗口范围	窗口元素	窗口和	是否满足	操作	minLen
1	0	0	[0,0]	[2]	2	否	右移 right	∞
2	0	1	[0,1]	[2,3]	5	否	右移 right	∞
3	0	2	[0,2]	[2,3,1]	6	否	右移 right	∞
4	0	3	[0,3]	[2,3,1,2]	8	是	记录长度 4, 收缩 left	4
5	1	3	[1,3]	[3,1,2]	6	否	停止收缩, 右移 right	4
6	1	4	[1,4]	[3,1,2,4]	10	是	记录长度 4, 收缩 left	4
7	2	4	[2,4]	[1,2,4]	7	是	记录长度 3, 收缩 left	3
8	3	4	[3,4]	[2,4]	6	否	停止收缩, 右移 right	3
9	3	5	[3,5]	[2,4,3]	9	是	记录长度 3, 收缩 left	3
10	4	5	[4,5]	[4,3]	7	是	记录长度 2, 收缩 left	2
11	5	5	[5,5]	[3]	3	否	停止收缩, right 到头, 结束	2

最终最小长度为 2, 对应子数组 [4,3]。

伪代码:

```

min_sub_array_len(target, nums[0..n-1]):
    left = 0
    sum = 0
    min_len = infinity
    for right = 0 to n-1:
        sum = sum + nums[right]
        while sum ≥ target:
            min_len = min(min_len, right - left + 1)
            sum = sum - nums[left]
            left = left + 1
    if min_len == infinity:
        return 0
    else:
        return min_len

```

C++ 实现:

```
int minSubArrayLen(int target, vector<int>& nums) {
    int left = 0, sum = 0, minLen = INT_MAX;
    for (int right = 0; right < nums.size(); right++) {
        sum += nums[right];
        while (sum >= target) {
            minLen = min(minLen, right - left + 1);
            sum -= nums[left];
            left++;
        }
    }
    return minLen == INT_MAX ? 0 : minLen;
}
```

Python 实现:

```
def min_sub_array_len(target, nums):
    left = 0
    total = 0
    min_len = float('inf')
    for right in range(len(nums)):
        total += nums[right]
        while total >= target:
            min_len = min(min_len, right - left + 1)
            total -= nums[left]
            left += 1
    return min_len if min_len != float('inf') else 0
```

2.6.0.3 无重复字符的最长子串 (LeetCode 3)

问题: 给定一个字符串 s , 请你找出其中不含有重复字符的 **最长子串** 的长度。

设计思路:

- 使用滑动窗口, 维护窗口内字符是否重复的信息。可以用哈希集合 (或数组) 记录当前窗口内出现的字符。
- 初始时 $left = 0$, $right = 0$, 窗口为空。
- 移动 $right$ 指针, 将 $s[right]$ 加入窗口。如果该字符已经存在于窗口中 (即与窗口内某字符重复), 则需要移动 $left$ 指针, 逐步将窗口左侧的字符移出, 直到窗口中不再包含该重复字符。
- 每步都更新最长长度 $maxLen = \max(maxLen, right - left + 1)$ 。
- 继续移动 $right$, 直到遍历完整个字符串。

算法分析:

- 每个字符最多被加入和移出窗口各一次, 时间复杂度 $O(n)$ 。
- 空间复杂度取决于字符集大小, 通常为 $O(|\Sigma|)$ 。

伪代码:

```
length_of_longest_substring(s):
    left = 0
    max_len = 0
    char_set = empty set
    for right = 0 to len(s)-1:
        while s[right] in char_set:
            remove s[left] from char_set
            left = left + 1
        add s[right] to char_set
        max_len = max(max_len, right - left + 1)
    return max_len
```

C++ 实现:

```
int lengthOfLongestSubstring(string s) {
    int left = 0, maxLen = 0;
    unordered_set<char> charSet;
    for (int right = 0; right < s.size(); right++) {
        while (charSet.count(s[right])) {
            charSet.erase(s[left]);
            left++;
        }
        charSet.insert(s[right]);
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Python 实现:

```
def length_of_longest_substring(s):
    left = 0
    max_len = 0
    char_set = set()
    for right in range(len(s)):
        while s[right] in char_set:
            char_set.remove(s[left])
            left += 1
        char_set.add(s[right])
        max_len = max(max_len, right - left + 1)
    return max_len
```


2.6.0.4 最小覆盖子串 (LeetCode 76)

问题：给你一个字符串 s 、一个字符串 t 。返回 s 中涵盖 t 所有字符的最小子串。如果 s 中不存在涵盖 t 所有字符的子串，则返回空字符串 $""$ 。

设计思路：

- 这是一个更复杂的滑动窗口问题，需要统计窗口内字符是否满足包含 t 中所有字符（包括重复次数）。
- 使用两个哈希表（或数组）分别记录 t 中字符的需求量 $need$ 和当前窗口内字符的拥有量 $have$ 。
- 用一个变量 $count$ 记录窗口中已经满足需求的字符种类数（当某个字符的拥有量达到需求量时， $count$ 增加）。
- 初始时 $left = 0$, $right = 0$ ，窗口为空。
- 移动 $right$ 扩大窗口，更新 $have$ 和 $count$ 。当 $count$ 等于 $need$ 中不同字符的种类数时，说明当前窗口已覆盖 t 。
- 此时尝试移动 $left$ 缩小窗口，以找到更短的覆盖子串：
 - 更新最小长度和起始位置。
 - 将 $s[left]$ 移出窗口，更新 $have$ 和 $count$ （如果移出后该字符的拥有量小于需求量，则 $count$ 减少）。
 - 重复直到窗口不再满足覆盖条件。
- 继续移动 $right$ ，直到遍历完 s 。

算法分析：

- 每个字符最多被加入和移出窗口各一次，时间复杂度 $O(n)$ （假设字符集大小固定，哈希操作 $O(1)$ ）。
- 空间复杂度 $O(|\Sigma|)$ 。

伪代码：

```
min_window(s, t):
    need = empty dictionary // 记录 t 中每个字符的出现次数
    have = empty dictionary // 记录当前窗口内每个字符的出现次数
    for each char in t:
        need[char] = need[char] + 1

    left = 0
    min_len = infinity
    start = 0
    count = 0
    required = len(need)

    for right = 0 to len(s)-1:
        c = s[right]
        have[c] = have[c] + 1
        if c in need and have[c] == need[c]:
            count = count + 1

        while count == required:
            if right - left + 1 < min_len:
                min_len = right - left + 1
```

```

        start = left
        c2 = s[left]
        have[c2] = have[c2] - 1
        if c2 in need and have[c2] < need[c2]:
            count = count - 1
        left = left + 1

    if min_len == infinity:
        return ""
    else:
        return s[start .. start+min_len-1]

```

C++ 实现:

```

string minWindow(string s, string t) {
    unordered_map<char, int> need, have;
    for (char c : t) need[c]++;

    int left = 0, minLen = INT_MAX, start = 0;
    int count = 0, required = need.size();

    for (int right = 0; right < s.size(); right++) {
        char c = s[right];
        have[c]++;
        if (need.count(c) && have[c] == need[c]) {
            count++;
        }

        while (count == required) {
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                start = left;
            }
            char c2 = s[left];
            have[c2]--;
            if (need.count(c2) && have[c2] < need[c2]) {
                count--;
            }
            left++;
        }
    }

    return minLen == INT_MAX ? "" : s.substr(start, minLen);
}

```

Python 实现:

```
def min_window(s, t):
    from collections import defaultdict
    need = defaultdict(int)
    have = defaultdict(int)
    for ch in t:
        need[ch] += 1

    left = 0
    min_len = float('inf')
    start = 0
    count = 0
    required = len(need)

    for right in range(len(s)):
        c = s[right]
        have[c] += 1
        if c in need and have[c] == need[c]:
            count += 1

        while count == required:
            if right - left + 1 < min_len:
                min_len = right - left + 1
                start = left
            c2 = s[left]
            have[c2] -= 1
            if c2 in need and have[c2] < need[c2]:
                count -= 1
            left += 1

    return "" if min_len == float('inf') else s[start:start+min_len]
```

2.6.0.5 滑动窗口的通用框架

从以上例子可以看出，滑动窗口问题通常遵循以下模板：

初始化左指针 `left = 0`，以及所需的数据结构（如哈希表、变量等）

for `right` **in** `0..n-1`:

将 `nums[right]` 加入窗口，更新相关状态

while 窗口不满足条件（或满足条件但需要缩小）：

更新答案（如果需要）

将 `nums[left]` 移出窗口，更新相关状态

left++

更新答案（有时也在 **while** 之前或之后）

根据问题不同，答案的更新位置和条件会有所变化，但核心都是通过动态调整窗口边界来避免重复计算。

2.6.0.6 滑动窗口的适用特征

- 问题涉及 **连续子数组/子串**。
- 问题具有 **单调性**：当窗口扩大时，某些指标（如和、长度、包含字符数）单调变化，从而可以通过移动左边界来缩小窗口。
- 通常需要维护窗口内的统计信息（如和、计数、是否存在重复等）。

2.6.0.7 小结

滑动窗口是双指针技巧中极其重要的一种，它将嵌套循环的暴力解法优化为线性时间，是解决子串/子数组问题的利器。掌握滑动窗口的关键在于理解窗口扩缩的条件以及如何高效维护窗口内的状态。通过上述例子，读者应能体会到滑动窗口的设计思路和分析方法，并在实际问题中灵活运用。

2.7 小结

线性穷举是最基础、最直观的算法设计方法。通过本章学习，已经掌握了：

- **简单迭代**：遍历数组或链表的基本方法。
- **嵌套迭代**：处理多维数据或多重组合问题。
- **多表迭代**：同时遍历多个线性表，常用于合并或比较。
- **双指针迭代**：通过协同指针提高效率，如左右指针、快慢指针、滑动窗口。

优点：简单直观、通用性强、正确性有保证，可作为优化的起点。**缺点**：效率低，不适合大规模问题。

适用场景：问题规模较小，或用于验证更高效算法的正确性；也可以作为算法设计的起点，为后续优化提供思路。

通过理解这些方法的设计思想、实现方式及适用条件，读者可以为后续学习更高效的算法奠定坚实基础。

2.8 习题

2.8.1 选择题

1. 以下哪种算法不属于线性穷举法的范畴？A. 顺序查找 B. 二分查找 C. 两数之和的暴力解法 D. 冒泡排序
2. 在长度为 n 的无序数组中进行顺序查找，目标元素等概率出现在每个位置，其平均比较次数是：A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

3. 使用快慢指针判断链表是否有环，快指针每次移动两步，慢指针每次移动一步。若链表有环，两指针相遇时，慢指针在环内走过的圈数：A. 一定小于 1 圈 B. 一定等于 1 圈 C. 一定大于 1 圈 D. 取决于环的大小
 4. 对于有序数组的两数之和问题（LeetCode 167），使用左右指针法的时间复杂度是：A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$
 5. 关于滑动窗口算法，下列说法错误的是：A. 滑动窗口通常用于解决连续子数组或子串问题 B. 滑动窗口的时间复杂度通常为 $O(n)$ C. 滑动窗口必须使用哈希表来维护窗口信息 D. 滑动窗口的核心是动态调整左右边界
-

2.8.2 判断题

1. O 线性穷举法的正确性保证来自于“穷举所有可能性”，因此不会遗漏解。
 2. O 冒泡排序算法在最好情况下的时间复杂度可以达到 $O(n)$ 。
 3. O 对于有序链表的去重操作，必须使用双指针才能完成，简单迭代无法实现。
 4. O 使用位运算枚举子集时， 2^n 个子集对应的 mask 取值范围是 0 到 $2^n - 1$ 。
 5. O 滑动窗口算法中，窗口的大小必须是固定的，不能变化。
-

2.8.3 填空题

1. 二分查找算法每次将搜索区间缩小为原来的 _____，因此时间复杂度为 _____。
 2. 使用快慢指针判断链表是否有环，当快指针速度为 2、慢指针速度为 1 时，两指针相遇时慢指针走过的路程为 _____，快指针走过的路程为 _____。
 3. 在长度为 n 的数组中，使用暴力法枚举所有子集并检查每个子集，时间复杂度为 _____。
 4. 合并两个长度分别为 m 和 n 的有序数组，使用双指针法的时间复杂度为 _____，空间复杂度为 _____。
 5. 回文串判断使用左右指针从两端向中间移动，最多需要比较 _____ 次。
-

2.8.4 简答题

1. 简述线性穷举法的基本思想及其优缺点。
请简要说明线性穷举法的核心思想，并分析其在什么情况下适用，什么情况下不适用。
 2. 比较顺序查找和二分查找的适用条件和时间复杂度。
为什么二分查找比顺序查找更快？二分查找需要满足什么前提条件？
 3. 解释快慢指针法判断链表是否有环的原理。
请说明为什么快指针每次走两步、慢指针每次走一步时，有环的情况下两指针一定会相遇。
 4. 滑动窗口算法的核心思想是什么？它相比暴力枚举的优势在哪里？
请以“长度最小的子数组”问题为例，说明滑动窗口如何避免重复计算。
 5. 子集枚举的位运算方法如何实现？
给定 n 个元素，如何用一个整数 $mask$ 表示一个子集？如何判断第 i 个元素是否在子集中？
-

2.8.5 客观编程题（共 10 题）

编程题 1：顺序查找

题目描述

给定一个包含 n 个整数的数组和一个目标值 x ，请实现顺序查找算法，返回目标值在数组中第一次出现的下标（下标从 0 开始）。如果不存在，返回 -1。

输入格式

- 第一行：两个整数 n 和 x
- 第二行： n 个整数，表示数组元素

输出格式

一个整数，表示目标值的下标或 -1

样例输入

```
5 3
1 4 2 3 5
```

样例输出

```
3
```

编程题 2：删除有序链表中的重复元素

题目描述

给定一个按升序排列的单链表，删除所有重复的元素，使每个元素只出现一次。返回删除重复元素后的链表头节点。

输入格式

- 第一行：整数 n ，表示链表长度
- 第二行： n 个整数，表示链表中各节点的值（升序排列）

输出格式

一行整数，表示去重后的链表元素，元素之间用空格分隔

样例输入

```
6
1 1 2 3 3 4
```

样例输出

```
1 2 3 4
```

编程题 3：两数之和（暴力解法）

题目描述

给定一个整数数组 `nums` 和一个目标值 `target`，请找出和为目标值的两个整数，并返回它们的下标（下标从 0 开始）。假设每种输入只对应一个答案，且不能重复使用同一个元素。使用嵌套循环的暴力解法实现。

输入格式

- 第一行：两个整数 n 和 $target$
- 第二行： n 个整数，表示数组元素

输出格式

两个整数，表示两个元素的下标，按升序输出（先小后大），中间用空格分隔。如果无解，输出 -1 -1。

样例输入

```
4 9
2 7 11 15
```

样例输出

```
0 1
```

编程题 4：冒泡排序**题目描述**

使用冒泡排序算法对给定的整数数组进行升序排序。要求实现优化版本（当某一趟没有发生交换时提前结束）。

输入格式

- 第一行：整数 n
- 第二行： n 个整数，表示待排序数组

输出格式

一行整数，表示排序后的数组，元素之间用空格分隔

样例输入

```
5
5 2 8 1 9
```

样例输出

```
1 2 5 8 9
```

编程题 5：判断链表是否有环**题目描述**

给定一个单链表的头节点，判断链表中是否有环。使用快慢指针法实现。

注意：输入中，除了给出节点值序列外，还会给出环的起始位置（从 0 开始计数）。如果环的起始位置为 -1，表示无环；否则，最后一个节点的 next 指向该位置的节点。

输入格式

- 第一行：两个整数 n 和 pos ， n 表示链表节点个数， pos 表示环的起始位置（-1 表示无环）
- 第二行： n 个整数，表示链表中各节点的值

输出格式

一个字符串 "true" 或 "false"，表示链表中是否存在环

样例输入

```
5 2
3 2 0 -4 1
```

样例输出

```
true
```

编程题 6：合并两个有序数组**题目描述**

给定两个按非递减顺序排列的整数数组 `nums1` 和 `nums2`，将它们合并为一个新的非递减顺序数组并返回。

输入格式

- 第一行：两个整数 m 和 n ，分别表示两个数组的长度
- 第二行： m 个整数，表示数组 `nums1`
- 第三行： n 个整数，表示数组 `nums2`

输出格式

一行整数，表示合并后的有序数组，元素之间用空格分隔

样例输入

```
4 3
1 3 5 7
2 4 6
```

样例输出

```
1 2 3 4 5 6 7
```

编程题 7：回文串判断**题目描述**

给定一个字符串，判断它是否为回文串（正读和反读相同）。使用左右指针法实现，只考虑字母和数字字符，忽略大小写，忽略非字母数字字符。

输入格式

一行字符串，可能包含空格、标点符号等

输出格式

一个字符串 `"true"` 或 `"false"`

样例输入

```
A man, a plan, a canal: Panama
```

样例输出

```
true
```

编程题 8：奇偶调序**题目描述**

给定一个整数数组，将所有奇数放在所有偶数之前。要求保持奇数和偶数各自原有的相对顺序，使用快慢指针法实现，空间复杂度为 $O(1)$ 。

输入格式

- 第一行：整数 n
- 第二行： n 个整数，表示数组元素

输出格式

一行整数，表示调整后的数组，元素之间用空格分隔

样例输入

```
8
1 2 3 4 5 6 7 8
```

样例输出

```
1 3 5 7 2 4 6 8
```

编程题 9：删除有序数组中的重复项**题目描述**

给定一个有序数组 `nums`，请原地删除重复出现的元素，使每个元素只出现一次，返回删除后数组的新长度。使用快慢指针法实现。

输入格式

- 第一行：整数 n
- 第二行： n 个整数，表示有序数组元素

输出格式

- 第一行：一个整数，表示去重后数组的长度 k
- 第二行： k 个整数，表示去重后的数组前 k 个元素（按原顺序）

样例输入

```
10
0 0 1 1 1 2 2 3 3 4
```

样例输出

```
5
0 1 2 3 4
```

编程题 10：百钱买百鸡

题目描述

公鸡 5 文钱一只，母鸡 3 文钱一只，小鸡 3 只一文钱。现在用 $money$ 文钱买 $total$ 只鸡，问公鸡、母鸡、小鸡各多少只？输出所有可能的整数解，按公鸡数从小到大输出。若无解，输出 "No solution"。

输入格式

一行两个整数 $money$ 和 $total$ ，分别表示总钱数和总鸡数

输出格式

每行三个整数，分别表示公鸡数、母鸡数、小鸡数，用空格分隔。按公鸡数升序输出。若无解，输出 "No solution"。

样例输入

100 100

样例输出

0 25 75

4 18 78

8 11 81

12 4 84

3 贪心法

3.1 贪心法的基本思想

贪心法 (Greedy Method) 是一种求解最优化问题的常用策略。它将问题的求解看作**多步决策过程**，在每一步决策时，都选取当前状态下最优的选择（即**局部最优解**），希望通过一系列局部最优选择最终得到全局最优解。

贪心法的思想源于生活中的“贪婪”行为：例如爬山时，在每个岔路口总是选择最陡的路径，期望能最快到达顶峰；又如找零钱时，总是优先使用面值最大的硬币，希望用最少的硬币数。然而，贪心并不总能保证全局最优，它只适用于具有特定性质的问题。

3.1.1 贪心法的基本框架

```
Algorithm Greedy(输入)
    初始化一个空解 S
    对于每一步决策：
        根据某种策略选择当前最优的候选对象 x
        如果 x 加入 S 后满足可行性条件，则将 x 并入 S
        否则丢弃 x
    返回 S
```

3.1.2 贪心法的适用条件

证明贪心算法的正确性通常采用数学归纳法或交换论证法，通过调整最优解使其逐步与贪心解一致。

一个问题能否用贪心法得到最优解，通常需要满足以下两个性质：

1. **贪心选择性质**：局部最优解可以导致全局最优解。即每一步的贪心选择最终会构成问题的最优解。换句话说，我们可以通过每一步的局部最优选择来构造全局最优解，而不需要考虑其他可能性。
2. **最优子结构性质**：问题的最优解包含其子问题的最优解。即原问题的最优解可以由子问题的最优解组合得到。

下面通过两个简单例子来理解这两个性质。

例 3-1：找零钱问题（贪心选择性质）

假设有 1 分、5 分、10 分、25 分四种硬币（美元硬币体系），需要找零 41 分。贪心策略是：每次选择不超过剩余金额的最大面值硬币。过程如下：

- 剩余 41 分，最大面值 25 分，选 1 枚 25 分，剩余 16 分；
- 剩余 16 分，最大面值 10 分，选 1 枚 10 分，剩余 6 分；
- 剩余 6 分，最大面值 5 分，选 1 枚 5 分，剩余 1 分；
- 剩余 1 分，选 1 枚 1 分。得到解：25+10+5+1，共 4 枚硬币。事实上，这也是最优解（可以用其他组合验证）。这个例子说明，每次的贪心选择（取最大面值）最终导致了全局最优解，这就是**贪心选择性质**的体现。

例 3-2：最短路径问题（最优子结构）

考虑一个简单的路径问题：从城市 A 到城市 C 有一条最短路径，假设它经过城市 B（如图 3-1 所示）。那么从 A 到 B 的这段路径，也一定是 A 到 B 的所有路径中最短的。为什么？因为如果存在一条比它更短的 A 到 B 的路径，那么我们可以用这条更短的路径替换原路径中的 A→B 段，从而得到一条更短的 A→C 路径，这与原路径是最短路径矛盾。

例如在 Figure 3.1 中，有四个城市（A, B, C, D）以及连接它们的道路。道路上的数字表示路程。从 A 到 C 的最短路径是 A → D → B → C，总路程为 5。

这个例子很好地体现了**最优子结构性质**。我们来看这条最短路径中的一个片段：从 A 到 B 的子路径 A → D → B。它是不是从 A 到 B 的所有路径中最短的呢？

假设存在另一条从 A 到 B 的更短路径，例如 A → [某个其他节点] → B，长度为 [比当前子路径更小的值]。那么，我们可以用这条更短的路径来替换原最短路径中的 A → [D] → [B] 这一段。替换后，我们就会得到一条从 A 到 C 的新路径：A → [那个其他节点] → B → C，其总路程将会小于原来的 5。

这就产生了一个矛盾：我们一开始假设的 A → [D] → [B] → C 是最短的，但通过替换却得到了一个更短的路径。因此，假设不成立，原最短路径中的子路径 A →] → B 必然是从 A 到 B 的所有路径中最短的。

这个例子清晰地说明了**最优子结构**的含义：一个全局最优解（A 到 C 的最短路径）中，包含的子问题的解（A 到 B 的最短路径）也一定是最优的。

这两个性质是贪心法能否成功的关键。在后续章节中，我们会反复遇到满足这两个性质的问题，并证明其贪心算法的正确性。

3.2 找零钱问题 (Coin Change)

3.2.1 问题描述

给定一些面值的硬币，例如美元硬币有 1 分 (penny)、5 分 (nickel)、10 分 (dime)、25 分 (quarter)、50 分 (half dollar) 和 1 美元 (dollar)。需要给顾客找零 x 分，要求用最少数量的硬币。假设每种硬币的数量无限。

3.2.2 贪心策略

贪心策略：每次选择不超过剩余金额的最大面值硬币。

例如，找零 34 分：先选 25 分，剩余 9 分；再选 5 分，剩余 4 分；最后选 4 个 1 分。共需 $1+1+4=6$ 枚硬币。

3.2.3 算法实现

伪代码：

```
Algorithm coinChange(x, coins[0..n-1])
    // 假设硬币面值已按降序排列: coins[0] > coins[1] > ... > coins[n-1]
    S ← empty list
    for i ← 0 to n-1
        while x ≥ coins[i]
            S.append(coins[i])
```

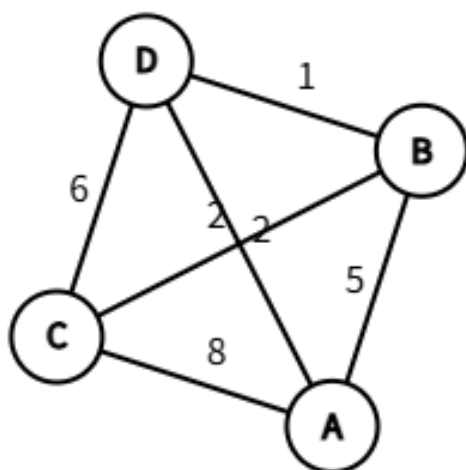


Figure 3.1: A 到 C 最短路径 ADBC 的子段 ADB 必定是 A 到 B 的最短路径

```

        x ← x - coins[i]
    return S

```

C++ 实现:

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<int> coinChange(int x, vector<int>& coins) {
    sort(coins.rbegin(), coins.rend()); // 降序排列
    vector<int> result;
    for (int coin : coins) {
        while (x >= coin) {
            result.push_back(coin);
            x -= coin;
        }
    }
    return result;
}

// 主函数示例
int main() {
    vector<int> coins = {1, 5, 10, 25, 50, 100};
    int amount = 34;
    vector<int> change = coinChange(amount, coins);
    cout << " 找零 " << amount << " 分: ";
    for (int c : change) cout << c << " ";
    cout << endl;
    return 0;
}

```

Python 实现:

```

def coin_change(x, coins):
    coins.sort(reverse=True)
    result = []
    for coin in coins:
        while x >= coin:
            result.append(coin)
            x -= coin
    return result

# 主函数示例

```

```
if __name__ == "__main__":
    coins = [1, 5, 10, 25, 50, 100]
    amount = 34
    change = coin_change(amount, coins)
    print(f"找零 {amount} 分: {change}")
```

3.2.4 贪心法的最优性分析

定理

对于硬币系统

$$\{1, 5, 10, 25, 100\}$$

贪心算法得到的解一定是最优解。

证明:

设贪心解为

$$G(n) = (g_{100}, g_{25}, g_{10}, g_5, g_1)$$

最优解为

$$O(n) = (o_{100}, o_{25}, o_{10}, o_5, o_1)$$

假设贪心解不是最优解，则存在最大不同面额 d 使得

$$g_d \neq o_d$$

由于贪心算法总是优先选择大面额硬币，必有

$$g_d > o_d$$

即最优解中 d 面额硬币比贪心少。

在最优解中去掉所有 $\geq d$ 的硬币，剩余小于 d 的硬币总金额设为 S 。

由于贪心至少比最优解多选了一枚 d ，必须有

$$S \geq d$$

小硬币数量上界（保证最少硬币数）

1. 面值 1 的硬币 $P \leq 4$
 - 否则 5 个 1 可用 1 个 5 替代，更优。
2. 面值 5 的硬币 $N \leq 1$
 - 否则 2 个 5 可用 1 个 10 替代，更优。
3. 面值 5 和 10 的组合 $N + D \leq 2$
 - 否则：

- $N=0, D=3 \rightarrow$ 金额 30, 可用 $25 + 5$ 替代
 - $N=1, D=2 \rightarrow$ 金额 25, 可用 25 替代
 - 都违反最优解性质。
4. 面值 25 的硬币 $Q \leq 3$
- 否则 4 个 25 可用 1 个 100 替代, 更优。

**** 小硬币无法凑成 d ****

d	小于 d 的硬币最多金额	小于 d 的硬币最大组合
5	1 的最多 4 个 = 4	< 5
10	1 的最多 4 个 + 5 的最多 1 个 = 9	< 10
25	1 的最多 4 个 + (N+D) 最多 2 个 $\leq 4+2 \times 10 = 24$	< 25
100	1 的最多 4 个 + (N+D) 最多 2 个 + Q 最多 3 $\leq 4+2 \times 10+3 \times 25 = 99$	< 100

由此, 最优解中小于 d 的硬币**无法凑成 d** 。

因此, 为了凑够金额, 最优解中必须至少使用 **1 枚 d** 。于是

$$g_d = o_d$$

对所有面额递归应用上述逻辑, 得到

$$G(n) = O(n)$$

结论

贪心算法得到的解与最优解相同, 因此贪心法总能得到最优解。

3.2.5 贪心法失效的反例

假设硬币面值为 25 分、20 分、10 分、5 分、1 分, 需要找零 41 分。贪心法会选 $25+10+5+1=41$, 用 4 枚硬币。但最优解是 $20+20+1=41$, 只用 3 枚硬币。因此, 贪心法并非普遍适用, 需要具体问题具体分析。

3.2.6 思考题

1. 设计一套硬币面值 (至少 4 种), 使得贪心法不能得到最优解, 并给出一个反例。
2. 如何判断一套硬币体系是否满足贪心性质?

3.3 区间调度问题 (Interval Scheduling)

3.3.1 问题描述

假设有一间教室, 多个课程希望使用它。每个课程有一个开始时间和结束时间, 例如课程 A 从 1 点到 4 点, 课程 B 从 3 点到 5 点, 等等。如果两个课程的时间不重叠, 它们可以共用这间教室; 否则只能安排其中一个。问题是如何选择尽

可能多的课程，使得它们的时间互不重叠。

形式化地，给定 n 个区间 (s_i, e_i) ，每个区间表示一个任务从开始时间 s_i 到结束时间 e_i 。任务是选择尽可能多的互不重叠的区间（即任意两个区间不能重叠）。通常认为如果两个区间端点相接（如一个结束等于另一个开始），可以视为不重叠。

例如，考虑以下区间集合：

(1,4), (3,5), (0,6), (5,7), (3,8), (5,9), (6,10), (8,11), (8,12), (2,13), (12,14), (1,4), (3,5), (0,6), (5,7), (3,8), (5,9), (6,10), (8,11), (8,12), (2,13), (12,14)

如 Figure 3.2 所示，这些区间有很多重叠的，我们需要从中选出尽可能多的互不重叠的区间。

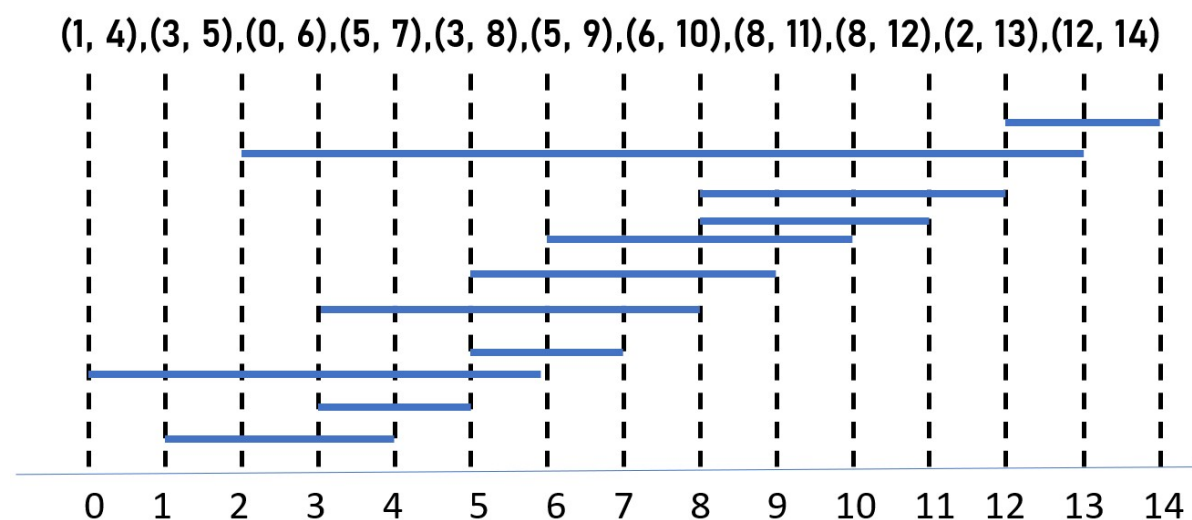


Figure 3.2: 重叠的区间

3.3.2 可能的贪心策略

面对这个问题，直觉上我们可以考虑以下几种贪心策略：

- **最早开始时间优先**：按开始时间 s_i 从小到大选择区间，优先选择开始最早的。
- **最短区间优先**：按区间长度 $e_i - s_i$ 从小到大选择，优先选择长度最短的。
- **冲突数最少的优先**：优先选择与其它区间冲突数量最少的区间，即与它重叠的区间个数最少。
- **最早结束时间优先**：按结束时间 e_i 从小到大选择，优先选择结束最早的。

这些策略听起来都有道理，但哪一种能保证得到最优解呢？

如 @fig-Interval_Scheduling_2 所示，前 3 个优先策略并不能保证贪心法产生最优解。下面是前 3 个优先策略的具体反例。

最早开始时间优先策略的反例考虑以下三个区间：[0, 5], [1, 2], [3, 4]。

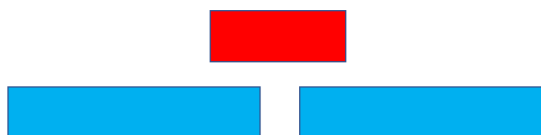
- 按最早开始时间排序，首先选择 [0, 5]。由于它与 [1, 2] 和 [3, 4] 都重叠，后续无法再选其他区间，最终只能选 1 个区间。
- 但最优解显然是选择 [1, 2] 和 [3, 4]，共 2 个区间。因此，最早开始时间优先策略不能保证最优。

最短区间优先策略的反例考虑以下三个区间：[0, 4], [5, 9], [3, 5]。

最早开始时间



最短区间



最少冲突

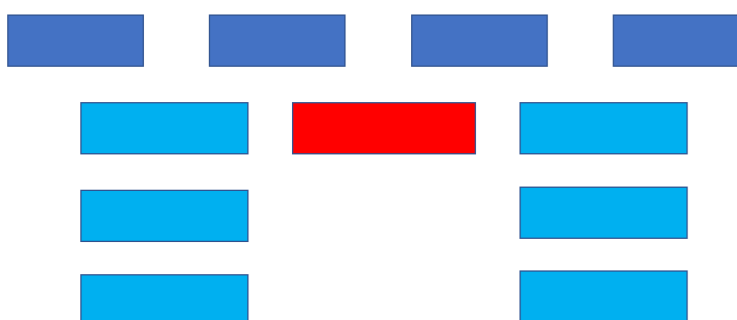


Figure 3.3: 不同优先策略的反例

- 区间长度分别为 4、4、2，最短的是 $[3, 5]$ 。若选择它，则它与 $[0, 4]$ 重叠（公共部分 $[3, 4]$ ），与 $[5, 9]$ 在点 5 处重合（视为重叠），因此不能再选其他区间，最终只能选 1 个区间。
- 而最优解是选择 $[0, 4]$ 和 $[5, 9]$ ，它们互不重叠，共 2 个区间。因此，最短区间优先策略也不能保证最优。

通过以上反例，我们发现并非所有贪心策略都有效。接下来介绍**最早结束时间优先策略**，它被证明是正确的。

3.3.3 贪心算法：最早结束时间优先

贪心算法：

1. 将所有区间按结束时间 e_i 从小到大排序。
2. 初始化结果集 S 为空，当前时间 $t = -\infty$ 。
3. 遍历排序后的区间，如果当前区间的开始时间 $s_i \geq t$ ，则将该区间加入 S ，并更新 $t = e_i$ 。

3.3.4 算法实现

```
Algorithm intervalScheduling(intervals[0..n-1])
    sort intervals by end time ascending
    S ← empty list
    t ← -∞
    for each interval (s, e) in sorted order
        if s ≥ t
            S.append(interval)
            t ← e
    return S
```

C++ 实现:

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Interval {
    int start, end;
};

bool cmp(const Interval& a, const Interval& b) {
    return a.end < b.end;
}

vector<Interval> intervalScheduling(vector<Interval>& intervals) {
    sort(intervals.begin(), intervals.end(), cmp);
    vector<Interval> result;
    int last_end = -1e9;
    for (const auto& inv : intervals) {
        if (inv.start >= last_end) {
            result.push_back(inv);
            last_end = inv.end;
        }
    }
    return result;
}

// 主函数示例
int main() {
    vector<Interval> intervals = {{1,4}, {3,5}, {0,6}, {5,7}, {3,8}, {5,9}, {6,10}, {8,11}, {8,12}};
    auto res = intervalScheduling(intervals);
    cout << " 最大不重叠区间数量: " << res.size() << endl;
    cout << " 选中的区间: ";
    for (auto& inv : res) cout << "[" << inv.start << "," << inv.end << "] ";
    cout << endl;
    return 0;
}

```

Python 实现:

```

def interval_scheduling(intervals):
    intervals.sort(key=lambda x: x[1]) # 按结束时间排序
    result = []

```

```

last_end = -float('inf')
for s, e in intervals:
    if s >= last_end:
        result.append((s, e))
        last_end = e
return result

# 主函数示例
if __name__ == "__main__":
    intervals = [(1,4), (3,5), (0,6), (5,7), (3,8), (5,9), (6,10), (8,11), (8,12), (2,13), (12,14)]
    res = interval_scheduling(intervals)
    print(" 最大不重叠区间数量: ", len(res))
    print(" 选中的区间: ", res)

```

3.3.5 正确性证明

我们证明：按结束时间最早选择的贪心算法得到的是最优解（即包含区间数最多）。

证明思路（交换论证）：

设贪心算法选择的区间依次为 $i_1, i_2, \dots, i_k$ ，其结束时间分别为 $e_1 \leq e_2 \leq \dots \leq e_k$ 。设任意最优解为 $j_1, j_2, \dots, j_m$ ，也按结束时间排序。

引理：对于任意 $r \leq k$ ，有 $e_r \leq e_{j_r}$ ，即贪心选择的第 r 个区间的结束时间不大于最优解中第 r 个区间的结束时间。

证明（归纳法）：

- 当 $r = 1$ 时，贪心选择的是所有区间中结束时间最早的，所以 $e_1 \leq e_{j_1}$ 。
- 假设对 $r - 1$ 成立，即 $e_{r-1} \leq e_{j_{r-1}}$ 。由于最优解中区间不重叠，有 $e_{j_{r-1}} \leq s_{j_r}$ 。而贪心算法在选择第 r 个区间时，是在所有开始时间不小于 e_{r-1} 的区间中选结束时间最早的，因此 e_r 不大于任何这样的区间的结束时间，特别地， $e_r \leq e_{j_r}$ 。

结论：如果 $m > k$ ，则存在第 $k + 1$ 个最优区间 j_{k+1} ，其开始时间 $s_{j_{k+1}} \geq e_{j_k} \geq e_k$ ，因此该区间在贪心算法第 k 步之后仍是候选区间，贪心算法会继续选择，与 k 是贪心算法选择的个数矛盾。所以 $k \geq m$ ，又因为最优解最多，所以 $k = m$ ，贪心算法最优。

3.3.6 思考题

1. 如果每个区间有一个权重，目标是选择权重和最大的不重叠区间集合，贪心法（按结束时间最早）还正确吗？为什么？
2. 上述证明中，如果区间是左闭右开或闭区间，对结论有何影响？

3.4 分数背包问题 (Fractional Knapsack)

3.4.1 问题描述

假设你是一个探险家，发现了一堆金粉、银粉和铜粉，每种粉末都可以任意分割（例如按克计量）。你有容量为 W 的背包，每种粉末有一定重量和总价值。目标是装入背包的粉末总价值最大。

更一般地，有 n 个物品，每个物品有重量 w_i 和价值 v_i ，背包容量为 W 。物品可以被分割（如液体、粉末），即可以取物品的一部分。目标是使装入背包的总价值最大。

例子：有三种金粉，重量分别为 10g、20g、30g，总价值分别为 60 元、100 元、120 元。背包容量为 50g。由于金粉可以任意分割，我们可以取一部分。如何装使得总价值最大？

直观想法是优先取“性价比”高的粉末，即单位重量价值高的。计算各物品的性价比：金粉 1：60/10=6，金粉 2：100/20=5，金粉 3：120/30=4。因此应先装金粉 1（全取 10g，得 60 元），剩余 40g，再装金粉 2（全取 20g，得 100 元），剩余 20g，最后装金粉 3，但只能取 20g（即 20/30=2/3），得 120×(20/30)=80 元，总价值 60+100+80=240 元。而如果全取金粉 2 和 3，总重 50g，价值 100+120=220 元，小于 240 元。显然按性价比贪心得到最优解。

3.4.2 贪心策略

直观上，应该优先选择“性价比”高的物品，即单位重量价值 v_i/w_i 大的物品。因此，贪心策略为：

1. 将物品按性价比降序排序。
2. 依次考虑每个物品，如果当前物品能全部装入，则全装；否则装一部分，装满背包。

3.4.3 算法实现

伪代码：

```
Algorithm fractionalKnapsack(items[0..n-1], W)
    sort items by v_i/w_i descending
    totalValue ← 0
    for each item in sorted order
        if W ≥ item.weight
            totalValue ← totalValue + item.value
            W ← W - item.weight
        else
            totalValue ← totalValue + item.value * (W / item.weight)
            break
    return totalValue
```

C++ 实现：

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
```

```

struct Item {
    int value, weight;
};

bool cmp(Item a, Item b) {
    double r1 = (double)a.value / a.weight;
    double r2 = (double)b.value / b.weight;
    return r1 > r2;
}

double fractionalKnapsack(int W, vector<Item>& items) {
    sort(items.begin(), items.end(), cmp);
    double totalValue = 0.0;
    for (const auto& item : items) {
        if (W >= item.weight) {
            totalValue += item.value;
            W -= item.weight;
        } else {
            totalValue += item.value * ((double)W / item.weight);
            break;
        }
    }
    return totalValue;
}

// 主函数示例
int main() {
    vector<Item> items = {{60, 10}, {100, 20}, {120, 30}};
    int capacity = 50;
    double maxValue = fractionalKnapsack(capacity, items);
    cout << " 最大价值: " << maxValue << endl;
    return 0;
}

```

Python 实现:

```

def fractional_knapsack(W, items):
    # items: list of (value, weight)
    items.sort(key=lambda x: x[0]/x[1], reverse=True)
    total_value = 0
    for value, weight in items:
        if W >= weight:
            total_value += value

```

```

        W -= weight
    else:
        total_value += value * (W / weight)
        break
    return total_value

# 主函数示例
if __name__ == "__main__":
    items = [(60, 10), (100, 20), (120, 30)]
    capacity = 50
    max_value = fractional_knapsack(capacity, items)
    print(" 最大值: ", max_value)

```

3.4.4 正确性证明

证明：贪心算法得到的是最优解。

设物品按性价比降序排列为 $1, 2, \dots, n$ 。记贪心解为 $G = \{y_1, y_2, \dots, y_n\}$ ，其中 y_i 是第 i 个物品被选取的比例 ($0 \leq y_i \leq 1$)。设某个最优解为 $OPT = \{x_1, x_2, \dots, x_n\}$ ，同样 x_i 表示选取比例。物品 i 的重量和价值分别为 w_i 和 v_i (物品的固有属性)。我们需要证明 OPT 的总价值不可能大于 G 的总价值。

假设 G 不是最优，则存在某个最优解 OPT 使得 $\sum v_i x_i > \sum v_i y_i$ 。找出第一个使得 $y_i \neq x_i$ 的下标 i 。由于贪心策略优先取性价比高的物品，且前 $i-1$ 个物品在贪心解中都已取满 (即 $y_j = 1$ 对 $j < i$ 成立)，而 OPT 中前 $i-1$ 个物品可能未取满，因此必然有 $y_i > x_i$ (因为如果 $y_i < x_i$ ，则前 i 个物品的总重量 OPT 会超过贪心解，但 OPT 是可行的，所以不可能)。现在，由于总重量守恒， OPT 中必然存在某个 $j > i$ 使得 $x_j > y_j$ (否则 OPT 的总重量会小于等于贪心解的总重量，而贪心解的总重量已达到容量上限，矛盾)。

令 $\epsilon = \min(w_i(y_i - x_i), w_j x_j)$ ，其中 w_i 和 w_j 分别是物品 i 和 j 的重量。 ϵ 表示我们能够从物品 j 中取出的最大重量 (不超过当前 x_j 部分)，同时能够加到物品 i 上 (不超过当前 $y_i - x_i$ 的剩余容量)。然后对 OPT 进行如下调整：从物品 j 中取出重量 ϵ (即减少比例 ϵ/w_j)，加到物品 i 中 (增加比例 ϵ/w_i)。调整后总重量不变，总价值变化为：

$$\Delta = \epsilon \cdot \frac{v_i}{w_i} - \epsilon \cdot \frac{v_j}{w_j} = \epsilon \left(\frac{v_i}{w_i} - \frac{v_j}{w_j} \right) > 0,$$

因为物品 i 的性价比高于物品 j (排序保证)。于是得到一个新的可行解，其总价值大于 OPT ，与 OPT 是最优解矛盾。因此贪心解 G 必然是最优的。

3.4.5 与 0-1 背包问题的对比

0-1 背包问题不允许分割物品，每个物品要么整件拿，要么不拿。对于 0-1 背包，按性价比贪心不一定得到最优解。例如，考虑以下三个物品：

- 物品 1：重量 10，价值 60 (性价比 6)
- 物品 2：重量 20，价值 100 (性价比 5)
- 物品 3：重量 30，价值 120 (性价比 4) 背包容量 50。贪心法先选物品 1 和 2，总价值 160，剩余容量 20，无法再装物品 3。而最优解是选物品 2 和 3，总价值 220。因此 0-1 背包不能用贪心，通常用动态规划求解。

分数背包与 0-1 背包的本质区别在于物品的可分割性。分数背包中，背包总能被装满（除非总重量小于容量），且贪心策略可以通过调整最后一件物品的比例来充分利用容量，从而保证最优性。而 0-1 背包的整数约束使得贪心选择可能导致剩余容量浪费，从而错失更优的组合。

3.5 最大物品数装载问题

3.5.1 问题描述

在物流运输中，我们经常需要将一批集装箱装上轮船。每个集装箱有各自的重量，轮船有最大载重量限制。若希望在不超载重量的前提下，尽可能多地装载集装箱（即最大化装载数量），应如何选择？

形式化定义：

给定 n 个集装箱，第 i 个集装箱的重量为 $w_i > 0$ ，轮船载重量为 $C > 0$ 。要求选择一个子集 $S \subseteq \{1, 2, \dots, n\}$ ，使得 $\sum_{i \in S} w_i \leq C$ ，且 $|S|$ 最大。

与经典的“0-1 背包问题”不同，这里每个物品的价值相同（均为 1），因此目标不再是总价值最大，而是**数量最大**。这一特性使得问题可以用贪心法高效求解。

3.5.2 贪心策略

直观上，为了装下更多集装箱，我们应当优先选择重量最轻的集装箱，因为轻的集装箱占用载重量小，留出更多空间给其他集装箱。因此贪心策略为：

按重量从小到大排序，依次选取集装箱，直到下一个集装箱无法装入为止。

该策略的本质是：每次局部最优（选当前最轻的）导致全局最优（总数量最大）。

3.5.3 算法描述

算法 5.3 最大物品数装载（贪心法）

输入：集装箱重量数组 $w[1..n]$ ，载重量 C

输出：可装载的最大数量 maxNum ，以及选中的集装箱索引集合 S

```

1. 将集装箱按重量从小到大排序，得到序列  $w[1] \leq w[2] \leq \dots \leq w[n]$ 
2. 初始化  $\text{totalWeight} = 0$ ,  $S = \emptyset$ 
3. for  $i = 1$  to  $n$ :
4.     if  $\text{totalWeight} + w[i] \leq C$ :
5.          $\text{totalWeight} = \text{totalWeight} + w[i]$ 
6.          $S = S \cup \{i\}$     // 实际索引需映射回原编号
7.     else:
8.         break
9. return  $|S|$ ,  $S$ 
```

注意：若需要输出原集装箱编号，排序时需同时记录原始下标。

3.5.3.1 C++ 实现

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// 贪心求解最大物品数装载问题
// 参数: weights - 集装箱重量数组, C - 轮船载重量
// 返回: 最大装载数量, 同时通过 selectedIndices 返回选中集装箱的原下标 (0-based)
int maxLoadGreedy(const vector<double>& weights, double C, vector<int>& selectedIndices) {
    int n = weights.size();
    // 存储 (重量, 原始下标)
    vector<pair<double, int>> items;
    for (int i = 0; i < n; ++i) {
        items.emplace_back(weights[i], i);
    }
    // 按重量升序排序
    sort(items.begin(), items.end());

    double totalWeight = 0.0;
    selectedIndices.clear();
    for (const auto& [w, idx] : items) {
        if (totalWeight + w <= C) {
            totalWeight += w;
            selectedIndices.push_back(idx);
        } else {
            break; // 因为已经排序, 后面更重的更不可能装入
        }
    }
    return selectedIndices.size();
}

int main() {
    // 示例: 载重量 20, 5 个集装箱重量 8,3,7,5,4
    vector<double> weights = {8, 3, 7, 5, 4};
    double C = 20;

    vector<int> selected;
    int maxNum = maxLoadGreedy(weights, C, selected);

    cout << " 最大装载数量: " << maxNum << endl;
}

```

```

cout << " 选中集装箱的原下标 (0-based): ";
for (int idx : selected) cout << idx << " ";
cout << endl;
cout << " 对应重量: ";
for (int idx : selected) cout << weights[idx] << " ";
cout << endl;

return 0;
}

```

运行输出:

最大装载数量: 4

选中集装箱的原下标 (0-based): 1 4 3 2

对应重量: 3 4 5 7

3.5.3.2 Python 实现

```

def max_load_greedy(weights, C):
    """
    贪心求解最大物品数装载问题
    :param weights: list[float], 集装箱重量列表
    :param C: float, 轮船载重量
    :return: (最大数量, 选中集装箱的原下标列表)
    """
    # 将重量与原始下标绑定, 并按重量升序排序
    items = sorted([(w, i) for i, w in enumerate(weights)], key=lambda x: x[0])

    total_weight = 0.0
    selected_indices = []

    for w, idx in items:
        if total_weight + w <= C:
            total_weight += w
            selected_indices.append(idx)
        else:
            break # 后面重量更大, 无需继续

    return len(selected_indices), selected_indices

if __name__ == "__main__":
    # 示例
    weights = [8, 3, 7, 5, 4]

```

```
C = 20
```

```
max_num, selected = max_load_greedy(weights, C)
```

```
print(f" 最大装载数量: {max_num}")
```

```
print(f" 选中集装箱的原下标 (0-based): {selected}")
```

```
print(f" 对应重量: {[weights[i] for i in selected]}")
```

运行输出:

最大装载数量: 4

选中集装箱的原下标 (0-based): [1, 4, 3, 2]

对应重量: [3, 4, 5, 7]

3.5.4 正确性证明

我们采用**贪心选择性质**和**最优子结构**来证明贪心算法得到最优解。

引理 5.1 (贪心选择性质): 存在一个最优装载方案, 它包含了重量最轻的集装箱。

证明: 设 k 为重量最轻的集装箱, 即 $w_k = \min\{w_1, w_2, \dots, w_n\}$ 。任取一个最优解 S^* , 若 $k \in S^*$, 则引理得证。若 $k \notin S^*$, 则 S^* 中所有集装箱的重量均不小于 w_k 。由于 S^* 是可行解, 总重量 $\sum_{i \in S^*} w_i \leq C$ 。将 S^* 中的任意一个集装箱 j 替换为 k , 得到新集合 $S' = (S^* \setminus \{j\}) \cup \{k\}$ 。因为 $w_k \leq w_j$, 所以 $\sum_{i \in S'} w_i \leq \sum_{i \in S^*} w_i \leq C$, 且 $|S'| = |S^*|$, 故 S' 也是最优解且包含 k 。因此, 总存在一个最优解包含最轻的集装箱。□

最优子结构: 在选择了最轻集装箱 k 之后, 原问题简化为子问题: 在剩余集装箱 (重量均不小于 w_k) 中, 在载重量剩余 $C - w_k$ 的条件下, 装载最多数量的集装箱。贪心算法接下来会在剩余集装箱中继续选择最轻的, 这恰好是子问题的贪心策略。由归纳法可知, 贪心算法得到全局最优解。

定理 5.1: 上述贪心算法能够求得最大物品数装载问题的最优解。

证明: 对集装箱个数 n 进行归纳。当 $n = 0$ 或 $C = 0$ 时结论平凡。假设对少于 n 个集装箱的问题贪心算法正确。考虑 n 个集装箱的问题。由引理 5.1, 存在最优解包含最轻集装箱 k 。将 k 加入装载, 问题变为在剩余 $n - 1$ 个集装箱中, 载重量为 $C - w_k$ 的子问题。由于剩余集装箱重量均不小于 w_k , 贪心算法下一步会继续选择剩余中最轻的, 这与子问题的最优解一致。由归纳假设, 贪心算法得到子问题的最优解, 加上 k 后即为原问题的最优解。□

也可以采用**交换论证 (exchange argument)**, 它直接比较贪心解与任意最优解, 并通过一系列“交换”将最优解逐步转化为贪心解而不损失数量。

3.5.4.1 交换论证证明

定理: 按重量升序依次选取集装箱 (直到无法装入) 得到的解 G 是最大数量装载问题的最优解。

证明:

设贪心算法选择的集装箱 (按排序后的顺序) 为 $g_1, g_2, \dots, g_k$, 重量分别为 $w_{g_1} \leq w_{g_2} \leq \dots \leq w_{g_k}$, 且满足 $\sum_{j=1}^k w_{g_j} \leq C$, 但再加入下一个 (第 $k + 1$ 个最轻的) 就会超重。

任取一个最优解 O , 其集装箱数量为 $|O|$ 。我们证明 $|O| \leq k$ 。

假设 $|O| > k$ 。将 O 中的集装箱按重量从小到大排序为 $o_1, o_2, \dots, o_{|O|}$, 满足 $w_{o_1} \leq w_{o_2} \leq \dots \leq w_{o_{|O|}}$ 。

由于贪心算法选择了前 k 个最轻的集装箱 (即 $g_1, \dots, g_k$ 是全局重量最小的 k 个集装箱), 因此对任意 $i = 1, \dots, k$, 有 $w_{g_i} \leq w_{o_i}$ 。

于是:

$$\sum_{i=1}^k w_{g_i} \leq \sum_{i=1}^k w_{o_i}.$$

因为 O 是可行解, 其任意前缀的重量和也 $\leq C$, 特别地 $\sum_{i=1}^k w_{o_i} \leq C$ 。但根据贪心算法的停止条件, 前 $k+1$ 个最轻的集装箱总重量超过 C , 即:

$$\sum_{i=1}^{k+1} w_{g_i} > C.$$

由于 $w_{g_{k+1}} \leq w_{o_{k+1}}$ (因 g_{k+1} 是第 $k+1$ 轻的, 而 o_{k+1} 至少是第 $k+1$ 轻的), 我们有:

$$\sum_{i=1}^{k+1} w_{o_i} \geq \sum_{i=1}^{k+1} w_{g_i} > C.$$

这与 O 是可行解矛盾 (因为 O 的前 $k+1$ 个物品已经超重)。因此假设 $|O| > k$ 不成立, 故 $|O| \leq k$ 。而贪心解本身可行, 所以 $|G| = k \geq |O|$, 即贪心解最优。

3.5.4.2 两种证明方法对比

方法	优点	缺点
归纳法	结构清晰, 适合递归算法; 自然引出最优子结构	需要归纳假设, 稍微繁琐
交换论证	直接、简洁; 不需要显式归纳; 常用于证明贪心算法	需要巧妙的排序和不等式推导

对于“最大物品数装载”这种简单问题, 交换论证更为明快, 且直接揭示了“重量最轻的 k 个物品是最优的”这一核心事实。您可以根据教材风格选择使用。

3.5.5 复杂度分析

- 排序: 使用快速排序或归并排序, 时间复杂度 $O(n \log n)$ 。
- 扫描: 线性扫描一遍, 时间复杂度 $O(n)$ 。
- 总体: $O(n \log n)$, 主要开销在排序。

若输入已按重量排序, 则时间复杂度可降至 $O(n)$ 。

3.5.6 示例

例 5.3: 设轮船载重量 $C = 20$, 5 个集装箱的重量分别为 $w = [8, 3, 7, 5, 4]$ 。

贪心算法执行过程: 1. 排序: $[3, 4, 5, 7, 8]$ 2. 装载 3 (总重 3), 装载 4 (总重 7), 装载 5 (总重 12), 装载 7 (总重 19), 下一个 8 无法装入 ($19+8=27>20$)。3. 得到装载数量 4, 对应原集装箱重量 3,4,5,7。

若采用其他策略 (例如先装重的), 最多只能装 3 个 (如 $8+7+5=20$, 或 $8+7+4=19$ 等), 因此贪心正确。

3.5.7 小结

最大物品数装载问题是贪心法的典型应用。通过按重量升序排序并依次装载，可以在线性时间内得到最优解。其正确性基于重量最轻的集装箱必被某个最优解包含这一事实，体现了贪心策略“轻者优先”的合理性。该问题与“背包问题”形成对比：当物品价值相同时，贪心可获最优；当价值不同时，则需动态规划。

思考题：如果将问题改为“装载的集装箱总重量尽可能大”（即经典的 0-1 背包问题），贪心策略（按重量升序）是否仍然最优？为什么？

3.6 哈夫曼编码 (Huffman Coding)

3.6.1 问题引入

假设我们需要将一个文件压缩成二进制串，文件中只出现 6 种字符：a、b、c、d、e、f，它们在文件中出现的次数（频率）分别为：

字符	频率
a	5
b	9
c	12
d	13
e	16
f	45

如果采用定长编码，因为 $2^3=8>6$ ，每个字符至少需要 3 位二进制，则文件总长度为 $(5+9+12+13+16+45)\times 3 = 100\times 3 = 300$ 位。能否设计一种**不等长编码**，使得高频字符用短编码、低频字符用长编码，从而减少总长度呢？哈夫曼编码正是解决这一问题的贪心算法。

3.6.2 前缀码与二叉树表示

不等长编码有一个关键要求：**任意一个字符的编码都不能是另一个字符编码的前缀**，否则解码时会产生歧义。这种编码称为**前缀码**。

让我们先看一个**非前缀码**的例子。假设我们为四个字符设计了如下编码：

- A: 10
- B: 01
- C: 010
- D: 001

现在收到一段二进制报文：**0101001**。由于 B 的编码“01”是 C 的编码“010”的前缀，这段报文既可以解释为“BBD” (01+01+001)，也可以解释为“CAB” (010+10+01)，导致解码歧义。这正是非前缀码的问题所在。

前缀码可以自然地用一棵**二叉树**表示：

- 每个叶子节点对应一个字符。
- 从根到叶子的路径：左分支代表 0，右分支代表 1，路径上的 0/1 串即为该字符的编码。

- 由于每个字符对应一个叶子，任何编码都不会是其他编码的前缀。

我们的目标就是构造一棵这样的二叉树，使得所有叶子节点的深度加权和（即总编码长度）最小，即最小化

$$\sum c \square \text{ 字符 } f c \square l c, c \square \text{ 字符 } \sum f c \square l c,$$

其中 f_c 为字符频率， l_c 为编码长度。

3.6.3 哈夫曼算法：贪心合并

哈夫曼算法的基本思想是：每次从集合中选出两个频率最小的节点，合并为一个新节点（频率为两者之和），反复进行，直到只剩一个节点。这个节点就是哈夫曼树的根。

算法步骤：

1. 为每个字符创建一个叶子节点，权值为其频率，放入优先队列（最小堆）。
2. 重复以下步骤，直到队列中只剩一个节点：
 - 从队列中取出两个权值最小的节点 x 和 y 。
 - 创建一个新节点 z ，权值为 x 和 y 的权值之和，左右孩子分别为 x 和 y （左右顺序任意，但通常规定左小右大）。
 - 将 z 放回队列。
3. 最后剩下的节点即为哈夫曼树的根。

3.6.4 手算演示：逐步合并

以我们的例子（频率：5,9,12,13,16,45）演示合并过程（如 @fig-hafuman）：

- **初始**：有 6 棵单节点树，频率为 [5,9,12,13,16,45]（按升序排列）。
- **第 1 步**：取出最小的 5 和 9，合并为 14。剩余节点：[12,13,14,16,45]（注意排序）。
- **第 2 步**：取出最小的 12 和 13，合并为 25。剩余节点：[14,16,25,45]。
- **第 3 步**：取出最小的 14 和 16，合并为 30。剩余节点：[25,30,45]。
- **第 4 步**：取出最小的 25 和 30，合并为 55。剩余节点：[45,55]。
- **第 5 步**：取出 45 和 55，合并为 100。得到根节点。

注意，在合并过程中，左右分支的位置可以交换，因此最终的编码可能不唯一，但总码长固定。

注意，在合并过程中，左右分支的位置可以交换，因此最终的编码可能不唯一，但总码长固定。

从根到叶子的路径（左 0 右 1）得到各字符编码：

- f (45): 0
- c (12): 100
- d (13): 101
- a (5): 1100
- b (9): 1101
- e (16): 111

总码长计算：

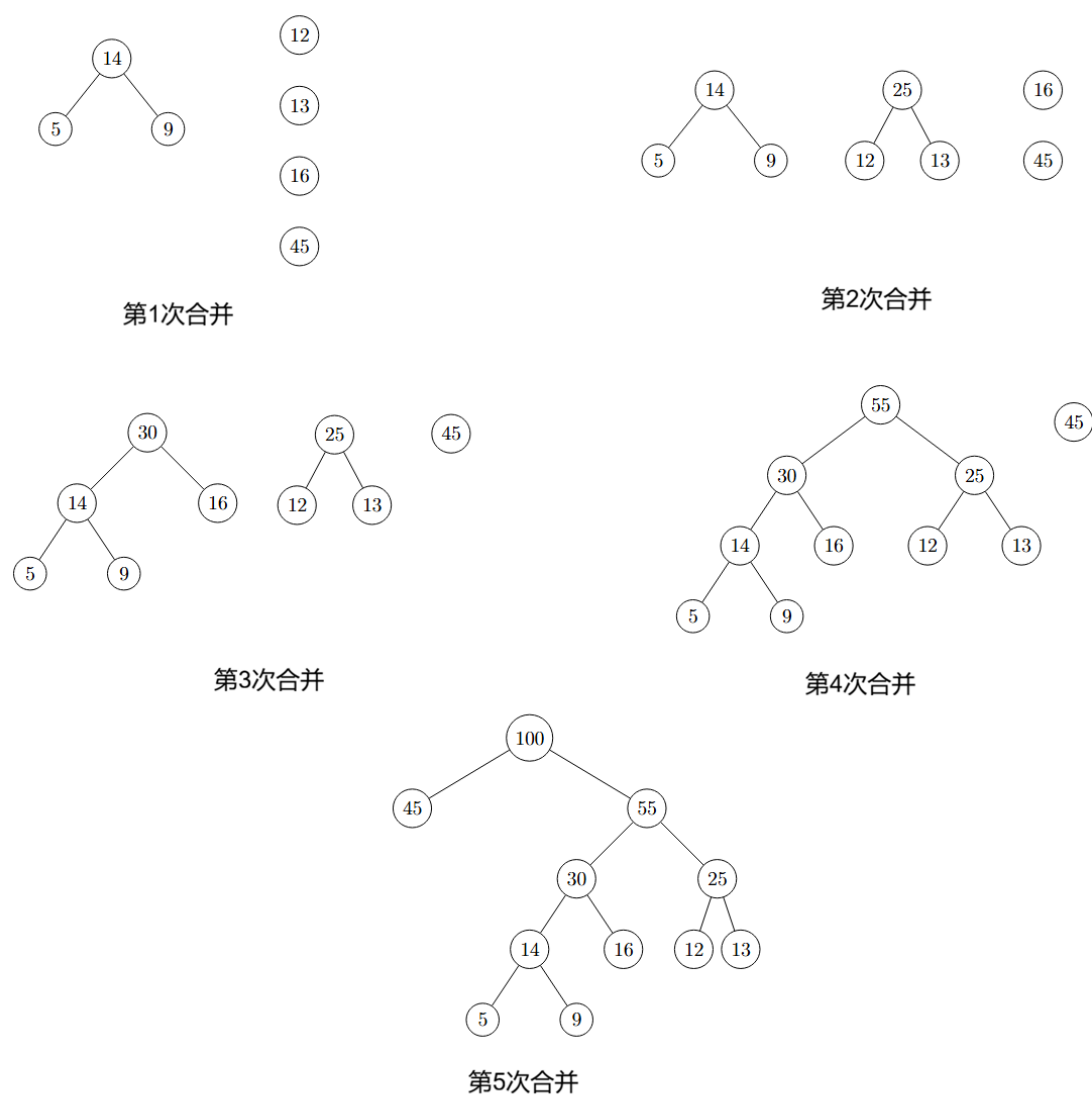


Figure 3.4: 哈夫曼编码树的构建过程

- f: $45 \times 1 = 45$
- c: $12 \times 3 = 36$
- d: $13 \times 3 = 39$
- a: $5 \times 4 = 20$
- b: $9 \times 4 = 36$
- e: $16 \times 3 = 48$

总和 = $45 + 36 + 39 + 20 + 36 + 48 = 224$ 位。

相比等长编码 (300 位), 节省约 25% 空间。可见哈夫曼编码的压缩效果。

3.6.5 算法实现

伪代码:

```
Algorithm huffman(C, freq)
    // C: 字符集, freq: 频率数组
    n ← |C|
    Q ← priority queue (min-heap) of nodes, each node has freq, left, right
    for i ← 1 to n
        create node z with freq[i]
        insert z into Q
    for i ← 1 to n-1
        x ← extract-min(Q)
        y ← extract-min(Q)
        create node z with freq = x.freq + y.freq, left = x, right = y
        insert z into Q
    return extract-min(Q) // root of Huffman tree
```

C++ 实现 (使用优先队列):

```
#include <iostream>
#include <queue>
#include <vector>
#include <string>
using namespace std;

struct Node {
    int freq;
    Node *left, *right;
    Node(int f) : freq(f), left(nullptr), right(nullptr) {}
};

struct Compare {
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq; // 最小堆
    }
};
```

```

    }
};

Node* huffman(vector<int>& freq) {
    priority_queue<Node*, vector<Node*>, Compare> pq;
    for (int f : freq) {
        pq.push(new Node(f));
    }
    while (pq.size() > 1) {
        Node* x = pq.top(); pq.pop();
        Node* y = pq.top(); pq.pop();
        Node* z = new Node(x->freq + y->freq);
        z->left = x;
        z->right = y;
        pq.push(z);
    }
    return pq.top();
}

// 辅助函数：生成编码
void generateCodes(Node* root, string code, vector<string>& codes) {
    if (!root) return;
    if (!root->left && !root->right) {
        codes.push_back(code);
        return;
    }
    generateCodes(root->left, code + "0", codes);
    generateCodes(root->right, code + "1", codes);
}

// 主函数示例
int main() {
    vector<int> freq = {5, 9, 12, 13, 16, 45}; // 示例频率
    Node* root = huffman(freq);
    vector<string> codes;
    generateCodes(root, "", codes);
    for (size_t i = 0; i < codes.size(); i++) {
        cout << " 字符" << i << " 编码: " << codes[i] << endl;
    }
    return 0;
}

```

Python 实现:

```

import heapq

class Node:
    def __init__(self, freq, left=None, right=None):
        self.freq = freq
        self.left = left
        self.right = right
    def __lt__(self, other):
        return self.freq < other.freq

def huffman(freq):
    heap = [Node(f) for f in freq]
    heapq.heapify(heap)
    while len(heap) > 1:
        x = heapq.heappop(heap)
        y = heapq.heappop(heap)
        z = Node(x.freq + y.freq, x, y)
        heapq.heappush(heap, z)
    return heap[0]

def generate_codes(root, code="", codes=None):
    if codes is None:
        codes = []
    if root:
        if not root.left and not root.right:
            codes.append(code)
        else:
            generate_codes(root.left, code + "0", codes)
            generate_codes(root.right, code + "1", codes)
    return codes

# 主函数示例
if __name__ == "__main__":
    freq = [5, 9, 12, 13, 16, 45]
    root = huffman(freq)
    codes = generate_codes(root)
    for i, code in enumerate(codes):
        print(f" 字符{i} 编码: {code}")

```

3.6.6 正确性证明

下面用数学归纳法严格证明哈夫曼算法的正确性。我们首先建立两个关键引理。

3.6.6.1 引理 1 (贪心选择性质)

设 x 和 y 是频率最小的两个字符, 则存在一个最优前缀码树, 使得 x 和 y 是兄弟节点 (即它们有相同的父节点, 且深度相同) 且深度最大。

证明: 设 T 是任意一个最优前缀码树。在 T 中, 深度最大的两个兄弟节点记为 a 和 b (它们位于最底层, 共享同一个父节点)。交换 x 与 a 、 y 与 b 的位置, 得到新树 T' 。我们证明 T' 也是最优树。

记 $d_a = d_b = D$ 为 a, b 的深度 (D 是树的最大深度)。设 d_x, d_y 为 x, y 在原树中的深度。交换后, x 和 y 的深度变为 D , a 和 b 的深度变为 d_x 和 d_y 。代价变化量 $\Delta = B(T') - B(T)$ 为:

$$\begin{aligned}\Delta &= [f(x)D + f(y)D + f(a)d_x + f(b)d_y] - [f(x)d_x + f(y)d_y + f(a)D + f(b)D] \\ &= f(x)(D - d_x) + f(y)(D - d_y) + f(a)(d_x - D) + f(b)(d_y - D) \\ &= (f(x) - f(a))(D - d_x) + (f(y) - f(b))(D - d_y).\end{aligned}$$

由于 x 和 y 是频率最小的字符, 有 $f(x) \leq f(a)$ 且 $f(y) \leq f(b)$ 。同时 $D \geq d_x$ 且 $D \geq d_y$ (因为 D 是最大深度)。因此 $(f(x) - f(a)) \leq 0$, $(D - d_x) \geq 0$, 所以第一项 ≤ 0 ; 同理第二项 ≤ 0 。于是 $\Delta \leq 0$, 即 $B(T') \leq B(T)$ 。又 T 是最优的, 故 $B(T') = B(T)$, T' 也是最优树。在 T' 中, x 和 y 成为深度最大的兄弟节点。(Figure 3.5)

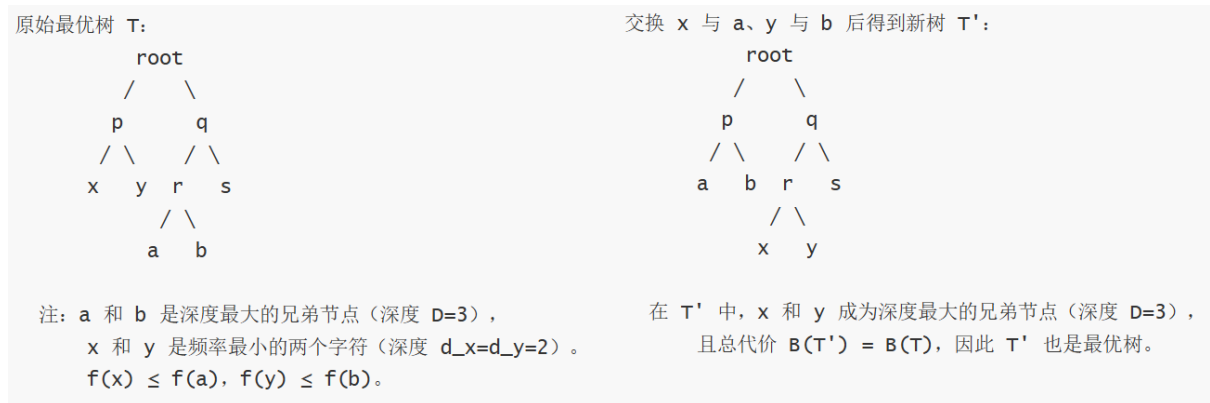


Figure 3.5: 频率最小的 x, y 结点分别与深度最大的结点 a, b 交换, 得到的仍然是最优树

3.6.6.2 引理 2 (最优子结构性质)

将原问题中的最小频率字符 x 和 y 合并为一个新字符 z , 其频率为 $f(z) = f(x) + f(y)$, 得到新字符集 $C' = (C \setminus \{x, y\}) \cup \{z\}$ 。则: - 如果 T' 是 C' 的一个最优前缀码树, 那么将 T' 中 z 的叶子节点替换为以 x 和 y 为左右孩子的内部节点, 得到的树 T 是原问题的一个最优前缀码树。- 反之, 如果 T 是原问题的一个最优前缀码树, 且 x 和 y 是兄弟 (由引理 1 知存在这样的最优树), 那么将 x 和 y 合并为 z , 得到的树 T' 是 C' 的一个最优前缀码树。

证明:

设 T 是原问题的一棵最优树, 且 x 和 y 是兄弟, 深度均为 d 。合并它们得到 T' , 则 T' 中 z 的深度为 $d - 1$ 。计算原树代价:

$$B(T) = \sum_{c \in C \setminus \{x, y\}} f(c) \cdot \text{depth}_T(c) + f(x)d + f(y)d.$$

新树 T' 的代价:

$$B(T') = \sum_{c \in C \setminus \{x, y\}} f(c) \cdot \text{depth}_{T'}(c) + f(z)(d-1).$$

注意对于 $c \neq x, y$, $\text{depth}_{T'}(c) = \text{depth}_T(c)$ (因为合并只影响 x, y 所在子树)。代入 $f(z) = f(x) + f(y)$, 得:

$$B(T') = \sum_{c \in C \setminus \{x, y\}} f(c) \cdot \text{depth}_T(c) + (f(x) + f(y))(d-1).$$

于是

$$B(T) = B(T') + f(x) + f(y).$$

这个关系式表明, $B(T)$ 与 $B(T')$ 相差一个常数 $f(x) + f(y)$ 。因此, $B(T)$ 最小当且仅当 $B(T')$ 最小。即: - 若 T' 是 C' 的最优树, 则 T 是原问题的最优树。- 若 T 是原问题的最优树 (且 x, y 为兄弟), 则 T' 是 C' 的最优树。

3.6.6.3 定理: 哈夫曼算法产生的编码是最优前缀

证明 (数学归纳法)

归纳基础:

- 当 $n = 1$ 时, 树只有一个节点, 代价为 0, 算法直接返回该节点, 显然最优。
- 当 $n = 2$ 时, 算法将两个节点合并为根, 唯一可能的树就是这两个字符作为兄弟, 显然最优 (因为只有一种结构)。

归纳假设: 假设对于所有字符数小于 n ($n \geq 3$) 的实例, 哈夫曼算法都能构造出最优前缀码树。

归纳步骤: 考虑字符数 n 的实例 C 。设 x 和 y 是 C 中频率最小的两个字符。算法第一步将它们合并为新字符 z , 得到新字符集 $C' = (C \setminus \{x, y\}) \cup \{z\}$, $|C'| = n - 1$ 。

由归纳假设, 算法对 C' 运行将得到一棵最优前缀码树 T' (即算法从合并后的状态继续执行, 最终得到的树是 C' 的最优树)。现在将 T' 中对应于 z 的叶子节点替换为以 x 和 y 为左右孩子的内部节点, 得到树 T 。我们需要证明 T 是原问题 C 的最优树。

根据引理 2 的第一部分, 如果 T' 是 C' 的最优树, 那么这样构造的 T 就是原问题的最优树。因此, 只要 T' 是 C' 的最优树, T 就是原问题的最优树。

而 T' 正是算法对 C' 运行的结果, 由归纳假设, 它是 C' 的最优树。所以 T 是原问题的最优树。这正是算法对原问题运行的结果 (第一步合并 x, y , 然后继续执行, 最后展开节点)。因此, 算法对 n 个字符的实例也得到最优树。

归纳完成, 哈夫曼算法正确。□

更简化的证明:

归纳基础: 当字符数 $n = 1$ 或 $n = 2$ 时, 算法构造的树显然是最优的 (唯一可能的结构)。

归纳步骤: 假设算法对所有字符数小于 n ($n \geq 3$) 的实例都能得到最优树。考虑字符数 n 的实例 C , 设 x 和 y 是频率最小的两个字符。算法第一步将它们合并为新字符 z , 得到新字符集 $C' = (C \setminus \{x, y\}) \cup \{z\}$, $|C'| = n - 1$ 。由归纳假设, 算法对 C' 运行得到的树 T' 是 C' 的最优树。根据引理 2 的第一部分, 将 T' 中 z 的叶子节点替换为以 x 和 y 为孩子的内部节点, 得到的树 T 是原问题 C 的最优树。而 T 正是算法对原实例运行的结果 (因为算法后续步骤与对 C' 相同, 即算法首先合并 x 和 y 得到 z , 然后对 C' 递归执行 (得到 T'))。因此算法对规模 n 的实例也正确。

由数学归纳法, 哈夫曼算法对所有字符集都能构造出最优前缀码树。□

3.6.7 思考题

1. 如果字符频率相同，哈夫曼编码是否唯一？
2. 哈夫曼编码的压缩率如何计算？
3. 在合并过程中，左右孩子的顺序会影响最终编码，但总码长是否改变？为什么？

3.7 最小生成树问题

3.7.1 问题描述

假设有若干个城市，需要铺设光缆使它们都能联网，且总光缆长度最短。每个城市作为一个顶点，可以铺设光缆的路线作为边，边的权值为路线长度。问题转化为：在连通无向带权图中，找出一棵包含所有顶点的树，使得所有边的权值之和最小。这棵树就是**最小生成树**（Minimum Spanning Tree, MST）。

给定一个连通无向图 $G = (V, E)$ ，每条边有非负权值 $w(u, v)$ 。求一棵生成树，包含所有顶点，且边权之和最小。这样的树称为**最小生成树**（MST）。

如图 @fig-mst，左图是一个连通无向图，右图是它的一个最小生成树。

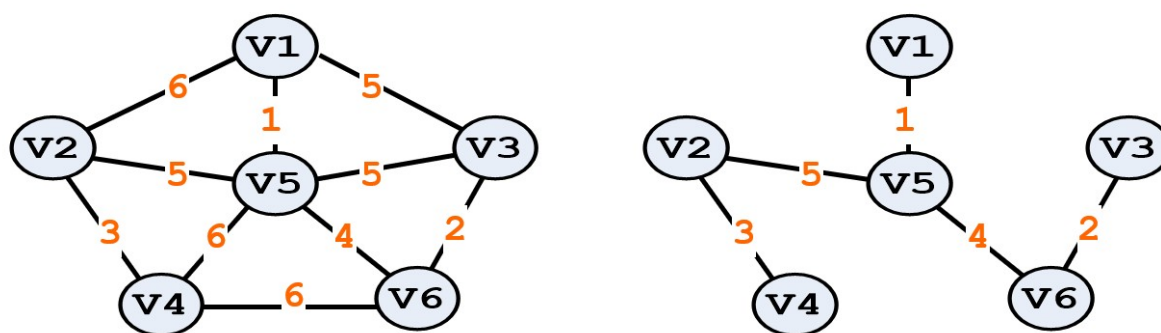


Figure 3.6: 连通无向图的最小生成树

最小生成树问题在现实中有广泛的应用，例如通信网络设计（铺设光缆）、电路布线、交通规划、聚类分析等领域，都是通过寻找最小成本连接所有节点的方案来优化资源。

下面介绍两个基于贪心法求解连通无向图最小生成树的经典算法：**Prim** 算法和 **Kruskal** 算法。

3.7.2 Prim 算法

Prim 算法是一种贪心算法，它从一个顶点开始，逐步扩张生成树，每次选择连接已在树中顶点与不在树中顶点的**最小权边**，将新顶点加入树中。

算法步骤：

1. 任选起点 u_0 ，初始化 $U = u_0$ ， T 为空。
2. 重复 $|V| - 1$ 次：
 - 从所有连接 U 与 $V - U$ 的边中，选择权值最小的边 (u, v) ，其中 $u \in U$ ， $v \notin U$ 。
 - 将 v 加入 U ，将边 (u, v) 加入 T 。

3.7.2.1 Prim 算法示例

考虑下图所示的带权无向图，有 4 个顶点 A、B、C、D，边及其权值如下：

- A—B: 2
- A—C: 3
- A—D: 6
- B—C: 4
- B—D: 5
- C—D: 1

图的结构可以表示为：

```

      A
     /\
    2/  \3
   /  |  \
  B 6|   C
   \  |  /
    5\  /1
      \|/
       D

```

下面用 Prim 算法从顶点 A 开始构造最小生成树：

- **初始：** $U = A$, $T = \emptyset$ 。候选边为与 A 相连的边：AB(2)、AC(3)、AD(6)。
- **第 1 步：**选择最短边 AB(2)，将 B 加入 U 。此时 $U = A, B$, $T = (A, B)$ 。新增候选边为与 B 相连且连接树外的边：BC(4)、BD(5)。
- **第 2 步：**当前连接 U 与 $V - U$ 的边有：AC(3)、AD(6)、BC(4)、BD(5)。最短的是 AC(3)，将 C 加入 U 。 $U = A, B, C$, $T = (A, B), (A, C)$ 。新增候选边为与 C 相连的 CD(1)（注意 BC 已考虑过）。
- **第 3 步：**现在连接 U 与 $V - U$ 的边有：AD(6)、BD(5)、CD(1)。最短的是 CD(1)，将 D 加入 U 。 $U = A, B, C, D$, $T = (A, B), (A, C), (C, D)$ 。

此时所有顶点已加入，算法结束。最小生成树的边为 AB、AC、CD，总权值 $2 + 3 + 1 = 6$ 。

伪代码：

```

Algorithm Prim(G, w)
    // 输入：连通无向图  $G=(V,E)$ ，边权函数  $w$ 
    // 输出：最小生成树的边集合  $T$ 
     $U \leftarrow \{\text{任意一个顶点}\}$ 
     $T \leftarrow \emptyset$ 
    while  $|U| < |V|$ 
        选择边  $(u,v)$  满足  $u \in U, v \notin U$  且  $w(u,v)$  最小
         $U \leftarrow U \cup \{v\}$ 
         $T \leftarrow T \cup \{(u,v)\}$ 
    return  $T$ 

```

时间复杂度

- 使用邻接矩阵时，每次查找最小边需要 $O(|V|)$ 时间，总时间复杂度 $O(|V|^2)$ ，适合稠密图。
- 使用二叉堆优化时，可达 $O((|V| + |E|) \log |V|)$ ，适合稀疏图。

C++ 实现 (朴素 $O(n^2)$):

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

vector<pair<int,int>> prim(vector<vector<pair<int,int>>>& adj) {
    int n = adj.size();
    vector<int> key(n, INT_MAX);      // 到 U 的最小边权
    vector<int> parent(n, -1);       // 记录最小边来自哪个顶点
    vector<bool> inMST(n, false);
    key[0] = 0;                      // 从顶点 0 开始

    for (int i = 0; i < n; i++) {
        // 选取不在 MST 中且 key 最小的顶点
        int u = -1;
        for (int j = 0; j < n; j++) {
            if (!inMST[j] && (u == -1 || key[j] < key[u]))
                u = j;
        }
        inMST[u] = true;

        // 更新邻接点的 key
        for (auto& [v, w] : adj[u]) {
            if (!inMST[v] && w < key[v]) {
                key[v] = w;
                parent[v] = u;
            }
        }
    }

    // 收集 MST 的边
    vector<pair<int,int>> mst;
    for (int v = 1; v < n; v++) {
        mst.emplace_back(parent[v], v);
    }
    return mst;
}
```



```
// 主函数示例
int main() {
    // 构造一个简单图，顶点数 4
    vector<vector<pair<int,int>>> adj(4);
    // 添加边 (u,v,w)
    adj[0].push_back({1,2}); adj[1].push_back({0,2});
    adj[0].push_back({2,3}); adj[2].push_back({0,3});
    adj[0].push_back({3,6}); adj[3].push_back({0,6});
    adj[1].push_back({2,4}); adj[2].push_back({1,4});
    adj[1].push_back({3,5}); adj[3].push_back({1,5});
    adj[2].push_back({3,1}); adj[3].push_back({2,1});

    auto mst = prim(adj);
    cout << " 最小生成树边: " << endl;
    for (auto& [u, v] : mst) {
        cout << u << " - " << v << endl;
    }
    return 0;
}
```

Python 实现 (朴素)

```
import sys

def prim(adj):
    n = len(adj)
    key = [sys.maxsize] * n
    parent = [-1] * n
    in_mst = [False] * n
    key[0] = 0

    for _ in range(n):
        # 选取不在 MST 中且 key 最小的顶点
        u = -1
        min_key = sys.maxsize
        for j in range(n):
            if not in_mst[j] and key[j] < min_key:
                min_key = key[j]
                u = j
        in_mst[u] = True

        # 更新邻接点的 key
        for v, w in adj[u]:
```

```

        if not in_mst[v] and w < key[v]:
            key[v] = w
            parent[v] = u

# 收集 MST 的边
mst = []
for v in range(1, n):
    mst.append((parent[v], v))
return mst

# 主函数示例
if __name__ == "__main__":
    # 构造一个简单图，顶点数 4
    adj = [[] for _ in range(4)]
    adj[0].append((1,2)); adj[1].append((0,2))
    adj[0].append((2,3)); adj[2].append((0,3))
    adj[0].append((3,6)); adj[3].append((0,6))
    adj[1].append((2,4)); adj[2].append((1,4))
    adj[1].append((3,5)); adj[3].append((1,5))
    adj[2].append((3,1)); adj[3].append((2,1))

    mst = prim(adj)
    print(" 最小生成树边: ")
    for u, v in mst:
        print(f"{u} - {v}")

```

3.7.3 正确性证明

证明 (归纳法): 设 T_k 为算法执行 k 步后得到的树 (包含 $k + 1$ 个顶点)。归纳假设: T_k 是某个最小生成树的子图。

- **基础:** $k = 0$, T_0 只含一个顶点, 显然属于任何最小生成树。
- **归纳步骤:** 假设 T_k 是某个最小生成树 M 的子图。算法在第 $k + 1$ 步选择了一条连接 U 与 $V - U$ 的最小权边 $e = (u, v)$, 其中 $u \in U$, $v \notin U$ 。若 e 已经在 M 中, 则 T_{k+1} 仍是 M 的子图。若 e 不在 M 中, 则将 e 加入 M 必然形成回路, 该回路中必有一条边 e' 连接 U 与 $V - U$ (因为回路中至少有一条边跨过割 $(U, V - U)$)。由 e 的选择, $w(e) \leq w(e')$ 。从 M 中删除 e' 加入 e 得到 M' , 则 M' 也是生成树且权和不大于 M , 故 M' 也是最小生成树。而 T_{k+1} 是 M' 的子图。归纳成立。

最终 T_{n-1} 是最小生成树。Figure 3.7

3.7.4 Kruskal 算法

Kruskal 算法是另一种求解最小生成树的贪心算法。与 Prim 算法从一个顶点开始逐步扩张不同, Kruskal 算法直接关注边, 它按照权值从小到大的顺序依次考虑每条边, 如果当前边加入后不会形成回路, 则将其加入生成树, 否则丢弃。这个过程持续到生成树包含 $|V| - 1$ 条边为止。

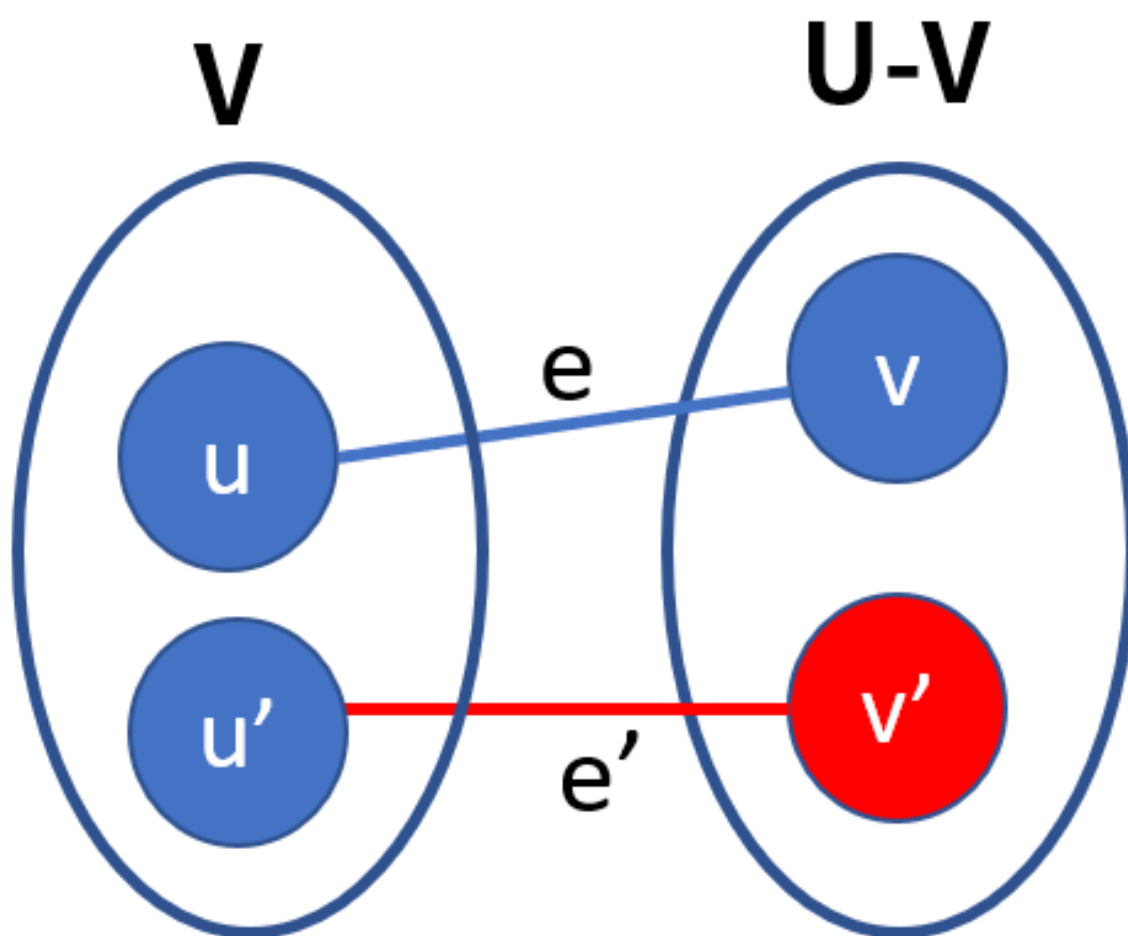


Figure 3.7: e 是连接 U 和 $V - U$ 的最小权边，必存在包含 e 的最小生成树

算法步骤：

1. 将边按权值从小到大排序。
2. 初始化：每个顶点单独构成一个连通分量。
3. 遍历每条边 (u, v, w) ：
 - 如果 u 和 v 属于不同的连通分量（即加入该边后不会形成回路），
 则将该边加入生成树，并合并 u 和 v 所在的分量。

示例

继续使用与 Prim 算法相同的图，顶点 A、B、C、D，边及权值如下：

- A—B: 2
- A—C: 3
- A—D: 6
- B—C: 4
- B—D: 5
- C—D: 1

步骤演示：

1. **排序边**：按权值升序排列：CD(1)、AB(2)、AC(3)、BC(4)、BD(5)、AD(6)。
2. **初始化**：每个顶点自成一个集合： $\{A\}$ 、 $\{B\}$ 、 $\{C\}$ 、 $\{D\}$ 。
3. **处理边 CD(1)**：C 和 D 属于不同集合，加入 CD，合并 C 和 D 所在集合。此时生成树边集： $\{CD\}$ ，集合： $\{A\}$ 、 $\{B\}$ 、 $\{C,D\}$ 。
4. **处理边 AB(2)**：A 和 B 属于不同集合，加入 AB，合并 A 和 B。生成树边集： $\{CD, AB\}$ ，集合： $\{A,B\}$ 、 $\{C,D\}$ 。
5. **处理边 AC(3)**：A 和 C 属于不同集合 ($\{A,B\}$ 与 $\{C,D\}$)，加入 AC，合并两个集合。生成树边集： $\{CD, AB, AC\}$ ，此时已有 3 条边，而顶点数为 4，恰好 $|V| - 1 = 3$ ，算法结束。
6. **后续边检查**：剩余边 BC(4)、BD(5)、AD(6) 若加入都会形成回路（因为所有顶点已连通），故跳过。

最终得到的最小生成树边为 CD、AB、AC，总权值 $1 + 2 + 3 = 6$ ，与 Prim 算法结果一致。

并查集的使用在实现上述算法时，需要高效地判断两个顶点是否属于同一连通分量，以及合并两个分量。**并查集 (Disjoint Set Union)** 是一种用于处理这类问题的数据结构，它支持近乎常数时间的查找 (FIND) 和合并 (UNION) 操作，非常适合用于 Kruskal 算法。下面伪代码中，我们用 MAKE-SET、FIND-SET 和 UNION 来表示并查集的基本操作。

算法步骤：

1. 将边按权值从小到大排序。
2. 初始化并查集，每个顶点单独一个集合。
3. 遍历每条边 (u, v, w) ：
 - 如果 u 和 v 不在同一集合（即加入后不形成回路），则将该边加入生成树，并合并 u 和 v 的集合。

伪代码：

```

Algorithm Kruskal(G, w)
    sort edges by w ascending
     $T \leftarrow \emptyset$ 
    for each vertex  $v$  in  $V$ :
        MAKE-SET( $v$ )
    for each edge  $(u,v,w)$  in sorted order:
        if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ):
             $T \leftarrow T \cup \{(u,v)\}$ 
            UNION( $u,v$ )
    return  $T$ 

```

C++ 实现 (使用并查集):

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Edge {
    int u, v, w;
    bool operator<(const Edge& other) const { return w < other.w; }
};

vector<Edge> kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end());
    vector<int> parent(n);
    for (int i = 0; i < n; i++) parent[i] = i;
    function<int(int)> find = [&](int x) {
        if (parent[x]  $\neq$  x) parent[x] = find(parent[x]);
        return parent[x];
    };
    vector<Edge> result;
    for (auto& e : edges) {
        int ru = find(e.u), rv = find(e.v);
        if (ru  $\neq$  rv) {
            result.push_back(e);
            parent[ru] = rv;
        }
    }
    return result;
}

// 主函数示例

```

```

int main() {
    vector<Edge> edges = {
        {0,1,2}, {0,2,3}, {0,3,6},
        {1,2,4}, {1,3,5},
        {2,3,1}
    };
    int n = 4; // 顶点数
    auto mst = kruskal(n, edges);
    cout << " 最小生成树边: " << endl;
    for (auto& e : mst) {
        cout << e.u << " - " << e.v << " 权值 " << e.w << endl;
    }
    return 0;
}

```

Python 实现

```

class DisjointSet:
    def __init__(self, n):
        self.parent = list(range(n))
    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx != ry:
            self.parent[rx] = ry
            return True
        return False

def kruskal(n, edges):
    edges.sort(key=lambda x: x[2]) # 按权值排序
    dsu = DisjointSet(n)
    mst = []
    for u, v, w in edges:
        if dsu.union(u, v):
            mst.append((u, v, w))
    return mst

# 主函数示例
if __name__ == "__main__":
    edges = [

```

```

    (0,1,2), (0,2,3), (0,3,6),
    (1,2,4), (1,3,5),
    (2,3,1)
]
n = 4
mst = kruskal(n, edges)
print(" 最小生成树边: ")
for u, v, w in mst:
    print(f"{u} - {v} 权值 {w}")

```

3.7.4.1 时间复杂度

Kruskal 算法的时间复杂度由以下两部分组成：

1. **排序边**：伪代码中首先执行 `sort edges by w ascending`，对 $|E|$ 条边按权值排序。时间复杂度为 $O(|E| \log |E|)$ 。
2. **遍历边并维护连通分量**：伪代码中随后遍历每条边，对每条边执行两次 FIND-SET 操作（判断两个端点是否在同一集合），以及可能的一次 UNION 操作（当边被加入生成树时）。使用并查集数据结构，并采用路径压缩和按秩合并优化时，单次 FIND-SET 或 UNION 操作的均摊时间复杂度为 $O(\alpha(|V|))$ ，其中 α 是阿克曼函数的反函数。这个函数增长极其缓慢：对于所有实际可能的输入规模（例如 $|V|$ 小于宇宙中原子数）， $\alpha(|V|) \leq 4$ ，因此可视为一个非常小的常数。从渐进意义上讲， $\alpha(|V|)$ 的增长速度远慢于 $\log |E|$ （事实上， $\alpha(n)$ 的增长速度比任何迭代对数函数都要慢），因此 $|E|\alpha(|V|)$ 的增长阶低于 $|E| \log |E|$ 。于是，遍历所有边并维护并查集的总时间可简化为 $O(|E|\alpha(|V|))$ ，但由于 $\alpha(|V|)$ 在渐进分析中可视为常数，该部分复杂度通常与排序部分合并，表达为 $O(|E| \log |E|)$ 。

综合两部分，Kruskal 算法的总时间复杂度为：

$$O(|E| \log |E| + |E|\alpha(|V|)) = O(|E| \log |E|)$$

该算法特别适合稀疏图（即 $|E|$ 远小于 $|V|^2$ 的情况）。

3.7.5 3.7.5 Kruskal 算法的正确性证明

证明：我们对算法已选边数进行归纳，证明一个不变式：每步之后，当前已选边集都是某个最小生成树的子集。

基础：初始时已选边集为空，显然是任何最小生成树的子集。

归纳步：假设前 $i-1$ 步后已选边集 F 是某个最小生成树 T 的子集，设 F 森林的连通分量集合为 $C_1, \dots, C_k$ ，如图 3-9 所示。现在算法选择第 i 条边 e （它是当前最小且不形成回路的边）。需要证明存在一个最小生成树包含 $F \cup \{e\}$ 。

若 $e \in T$ ，则 T 已经包含 $F \cup \{e\}$ ，得证。

若 $e \notin T$ ，则将 e 加入 T 会形成回路。设 e 连接的两个不同连通分量为 C_i 和 C_j （这些分量由 F 定义）。回路中必然存在另一条边 e' 也连接 C_i 和 C_j （否则回路无法跨越两个分量）。由于 e 是当前所有连接不同分量的边中权值最小的，而 e' 也是一条连接不同分量的边（且 $e' \notin F$ ，否则 C_i 和 C_j 早已连通），故 $w(e) \leq w(e')$ 。从 T 中删除 e' 并加入 e ，得到新树 T' ，其权值 $w(T') = w(T) + w(e) - w(e') \leq w(T)$ 。因 T 是最小生成树，所以 $w(T') = w(T)$ ，即 T' 也是最小生成树。显然 T' 包含 F （因为删掉的 e' 不在 F 中）且包含 e ，因此 T' 包含 $F \cup \{e\}$ 。如 @fig-Kruskal_proof 所示。

由归纳法，当算法选出 $n - 1$ 条边时，当前边集 F 是某个最小生成树的子集。此时 F 本身已有 $n - 1$ 条边且无回路，所以它是一棵生成树。一棵生成树是另一棵最小生成树的子集，则它们必然相等。故 F 就是最小生成树。

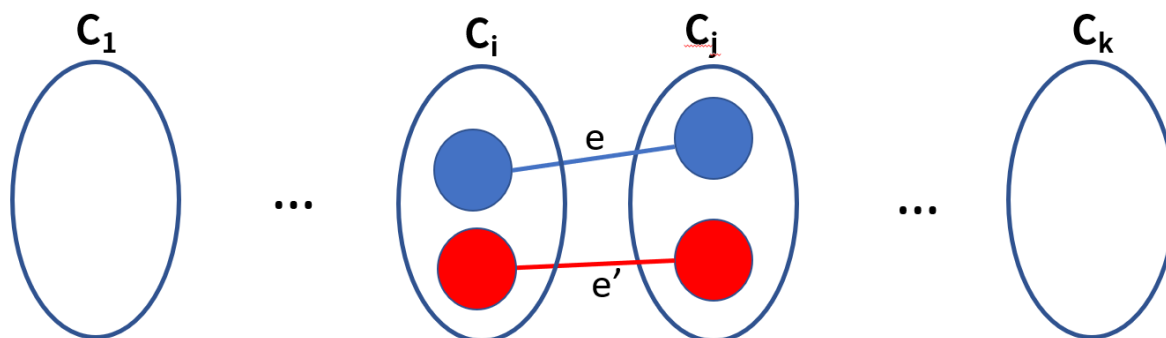


Figure 3.8: e 是连接 C_i, C_j 的最小权边且假如森林 F 不会产生回路，则存在一个最小生成树包含 e

3.7.5.1 两种算法对比

- **Prim 算法**：以顶点为中心，适合稠密图（邻接矩阵 $O(|V|^2)$ ）。
- **Kruskal 算法**：以边为中心，适合稀疏图（ $O(|E| \log |E|)$ ）。

无论哪种算法，得到的都是最小生成树。在实际应用中可根据图的稠密程度选择合适的算法。

3.7.6 思考题

1. 如果图中有负权边，Prim 和 Kruskal 算法还能正确工作吗？
2. 如何判断一个图的最小生成树是否唯一？

3.8 单源最短路径 Dijkstra 算法

3.8.1 问题描述

给定一个带权有向图（或无向图） $G = (V, E)$ ，每条边 (u, v) 的权值 $w(u, v)$ 为非负实数。给定一个源点 $s \in V$ ，求从 s 到图中所有其他顶点的最短路径长度（即路径上各边权值之和的最小值）。这个问题称为**单源最短路径问题**。

如 Figure 3.9 所示：

Dijkstra 算法是解决该问题的经典贪心算法，它要求所有边的权值非负。

应用场景：单源最短路径问题在现实世界中有着广泛的应用。例如，GPS 导航系统利用该算法计算从当前位置到目的地的最短行驶路线；计算机网络中的路由协议（如 OSPF）使用 Dijkstra 算法寻找数据包传输的最短路径；在社交网络中，可以用它来度量两个用户之间的最短关系链；机器人路径规划、城市规划中的交通流分析等领域也常依赖最短路径算法来优化决策。

3.8.2 Dijkstra 算法

Dijkstra 算法按路径长度递增的次序依次确定从源点到各个顶点的最短距离。它维护一个集合 S ，表示已经找到最短路径的顶点，以及一个距离数组 d ，表示当前从源点到每个顶点的最短路径估计。每次从 $V - S$ 中选出 d 值最小的顶

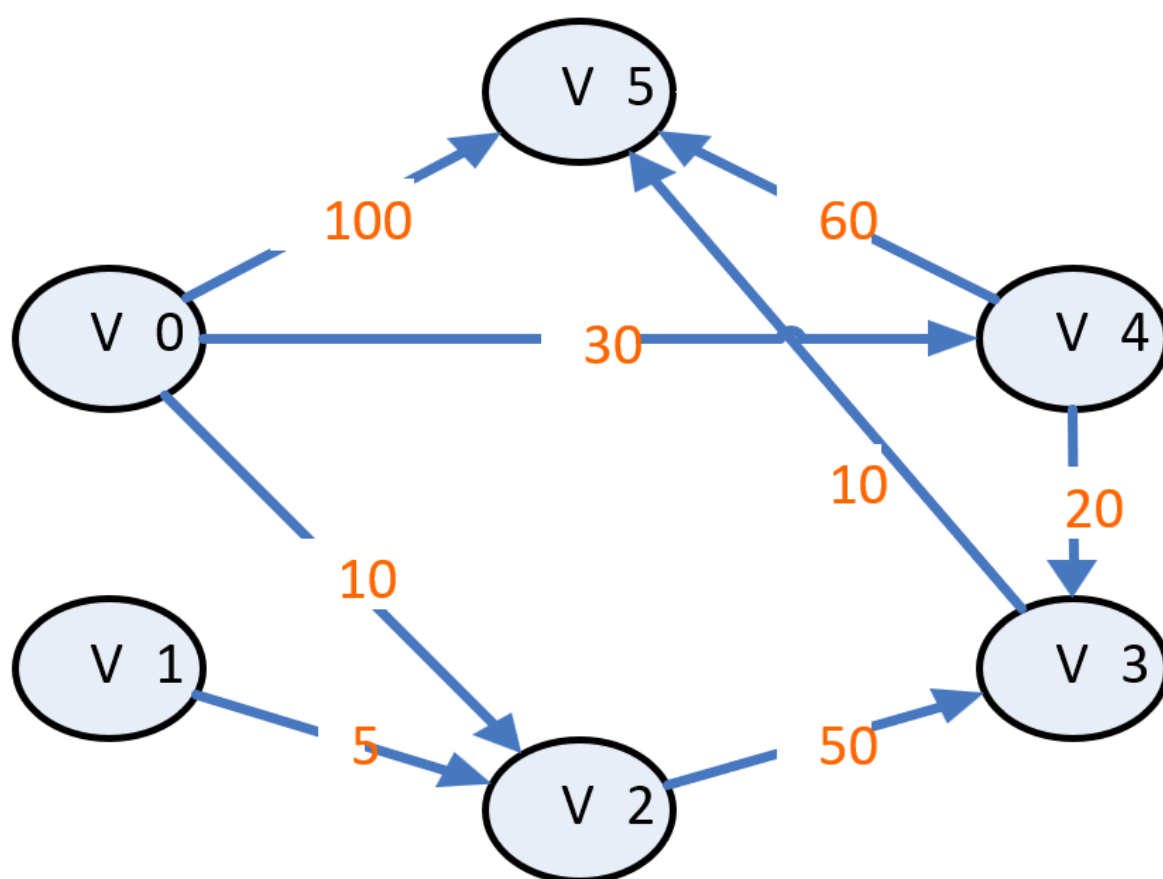


Figure 3.9: Dijkstra 算法：从起点 v_0 到其他所有顶点的最短路径

点 u 加入 S ，然后以 u 为中转站更新其邻接点的距离估计。

3.8.2.1 算法步骤

1. 初始化: $d[s] = 0$, 其余 $d[v] = \infty$, $S = \emptyset$ 。
2. 循环 $|V| - 1$ 次:
 - 从 $V - S$ 中选取 $d[u]$ 最小的顶点 u 。
 - 将 u 加入 S 。
 - 对于 u 的每个邻接点 v ,

如果 $d[u] + w(u, v) < d[v]$, 则更新 $d[v] = d[u] + w(u, v)$ 。

可以借助于优先队列选择权值最小的边:

算法步骤 (版本 2):

1. 初始化:
 - 对每个顶点 v , 令 $d[v] = \infty$, $parent[v] = \text{null}$ (用于记录路径)。
 - 令 $d[s] = 0$ 。
 - 集合 $S = \emptyset$ 。
 - 将所有顶点放入优先队列 Q , 键值为 $d[v]$ 。
2. 重复以下步骤直到 Q 为空:
 - 从 Q 中取出键值最小的顶点 u (即 $d[u]$ 最小)。
 - 将 u 加入 S 。
 - 对于 u 的每个邻接点 v , 如果 $d[u] + w(u, v) < d[v]$, 则更新 $d[v] = d[u] + w(u, v)$, 并更新 $parent[v] = u$, 同时更新 Q 中 v 的键值。
3. 算法结束时, $d[v]$ 即为从 s 到 v 的最短路径长度, 通过 $parent$ 可回溯路径。

3.8.2.2 示例

考虑下图所示的有向带权图, 顶点集为 A, B, C, D, E , 边及权值如下:

- $A \rightarrow B$: 10
- $A \rightarrow C$: 3
- $B \rightarrow C$: 1
- $B \rightarrow D$: 2
- $C \rightarrow B$: 4
- $C \rightarrow D$: 8
- $C \rightarrow E$: 2
- $D \rightarrow E$: 7
- $E \rightarrow D$: 9

(可用 CS Academy 绘制, 节点布局可自行调整)

下面用 Dijkstra 算法求从源点 A 到其他各顶点的最短路径。

初始化:

- $d[A] = 0, d[B] = d[C] = d[D] = d[E] = \infty$ 。
- $S = \emptyset$, 优先队列 Q 包含所有顶点, 键值为 d 值。

第 1 步: 从 Q 中取出键值最小的顶点, 即 A ($d = 0$)。将 A 加入 S 。检查 A 的邻接点: B 和 C 。

- 对于 B : $d[A] + w(A, B) = 0 + 10 = 10 < \infty$, 更新 $d[B] = 10, \text{parent}[B] = A$ 。
- 对于 C : $d[A] + w(A, C) = 0 + 3 = 3 < \infty$, 更新 $d[C] = 3, \text{parent}[C] = A$ 。

第 2 步: 当前 Q 中顶点及 d 值: $B(10), C(3), D(\infty), E(\infty)$ 。取出最小的 C ($d = 3$), 将 C 加入 S 。检查 C 的邻接点: B, D, E 。

- 对于 B : $d[C] + w(C, B) = 3 + 4 = 7 < 10$, 更新 $d[B] = 7, \text{parent}[B] = C$ 。
- 对于 D : $d[C] + w(C, D) = 3 + 8 = 11 < \infty$, 更新 $d[D] = 11, \text{parent}[D] = C$ 。
- 对于 E : $d[C] + w(C, E) = 3 + 2 = 5 < \infty$, 更新 $d[E] = 5, \text{parent}[E] = C$ 。

第 3 步: 当前 Q 中: $B(7), D(11), E(5)$ 。取出最小的 E ($d = 5$), 将 E 加入 S 。检查 E 的邻接点: D 。

- 对于 D : $d[E] + w(E, D) = 5 + 9 = 14 > 11$, 不更新。

第 4 步: 当前 Q 中: $B(7), D(11)$ 。取出最小的 B ($d = 7$), 将 B 加入 S 。检查 B 的邻接点: C 和 D 。注意 C 已在 S 中, 其最短路径已确定, 无需更新。对于 D :

- $d[B] + w(B, D) = 7 + 2 = 9 < 11$, 更新 $d[D] = 9, \text{parent}[D] = B$ 。

第 5 步: 最后取出 D ($d = 9$), 加入 S , 检查其邻接点 (E 已在 S 中), 无更新。

算法结束, 得到从 A 到各顶点的最短距离:

- $d[A] = 0$
- $d[B] = 7$, 路径 $A \rightarrow C \rightarrow B$
- $d[C] = 3$, 路径 $A \rightarrow C$
- $d[D] = 9$, 路径 $A \rightarrow C \rightarrow B \rightarrow D$
- $d[E] = 5$, 路径 $A \rightarrow C \rightarrow E$

伪代码

```
Algorithm Dijkstra( $G, w, s$ )
  for each  $v$  in  $V$ :
     $d[v] \leftarrow \infty$ 
     $\text{parent}[v] \leftarrow \text{null}$ 
   $d[s] \leftarrow 0$ 
   $S \leftarrow \emptyset$ 
   $Q \leftarrow$  priority queue of vertices keyed by  $d$ 
  while  $Q$  is not empty:
     $u \leftarrow \text{extract-min}(Q)$ 
     $S \leftarrow S \cup \{u\}$ 
    for each  $v$  in  $\text{adj}[u]$ :
      if  $d[u] + w(u, v) < d[v]$ :
         $d[v] \leftarrow d[u] + w(u, v)$ 
        decrease-key( $Q, v, d[v]$ )
```

```

        parent[v] ← u
    return d, parent

```

C++ 实现 (使用优先队列):

```

#include <iostream>
#include <vector>
#include <queue>
#include <limits>
using namespace std;

vector<int> dijkstra(vector<vector<pair<int,int>>>& adj, int s) {
    int n = adj.size();
    vector<int> dist(n, INT_MAX);
    dist[s] = 0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        int d = pq.top().first, u = pq.top().second;
        pq.pop();
        if (d > dist[u]) continue; // 过时的条目
        for (auto& [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

// 主函数示例
int main() {
    // 构造有向图, 顶点数 4
    vector<vector<pair<int,int>>> adj(4);
    adj[0].push_back({1,2});
    adj[0].push_back({2,5});
    adj[1].push_back({2,1});
    adj[1].push_back({3,4});
    adj[2].push_back({3,1});
    int source = 0;
    vector<int> dist = dijkstra(adj, source);
    cout << " 从顶点" << source << " 到各顶点的最短距离: " << endl;
}

```

```

    for (int i = 0; i < dist.size(); i++) {
        cout << " 到" << i << ": " << dist[i] << endl;
    }
    return 0;
}

```

Python 实现

```

import heapq

def dijkstra(adj, s):
    n = len(adj)
    dist = [float('inf')] * n
    dist[s] = 0
    pq = [(0, s)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist

# 主函数示例
if __name__ == "__main__":
    adj = [[] for _ in range(4)]
    adj[0].append((1, 2))
    adj[0].append((2, 5))
    adj[1].append((2, 1))
    adj[1].append((3, 4))
    adj[2].append((3, 1))
    source = 0
    dist = dijkstra(adj, source)
    print(f" 从顶点{source}到各顶点的最短距离: ")
    for i, d in enumerate(dist):
        print(f" 到{i}: {d}")

```

3.8.3 正确性证明

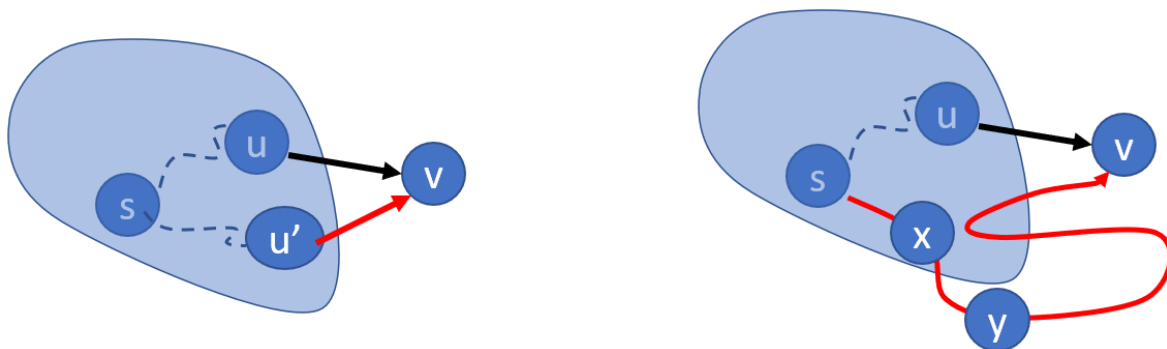
证明 (数学归纳法): 设 S 为已确定最短路径的顶点集合。归纳假设: 对于 S 中的每个顶点 v , $dist[u]$ 就是源点 s 到 v 的最短路径长度。

- 基础：\$S\$ 初始为空，成立。
- 归纳步骤：假设 \$|S| = k\$ 时成立，现在要从 \$V - S\$ 中选出 \$dist\$ 最小的顶点 \$v\$ 加入 \$S\$。需要证明 \$dist[v]\$ 就是 \$s\$ 到 \$v\$ 的最短路径长度。

用反证法：

假设存在一条从 \$s\$ 到 \$v\$ 的更短路径 \$P\$，其长度 \$< dist[v]\$。由于 \$s \in S\$ (\$s\$ 在 \$S\$ 中)，\$v \notin S\$，路径 \$P\$ 必然从 \$S\$ 出发后第一次离开 \$S\$，设该离开点为 \$y\$，即 \$P\$ 上存在一个顶点 \$y \in V - S\$，它是路径上第一个不在 \$S\$ 中的顶点，其前驱 \$x \in S\$。设从 \$s\$ 到 \$y\$ 的路径长度为 \$L\$，则 \$L \geq dist[y]\$ (因为 \$y\$ 未被加入 \$S\$，但 \$dist[y]\$ 是当前通过 \$S\$ 中顶点得到的估计值)。由于 \$P\$ 是从 \$s\$ 到 \$v\$ 的更短路径，且 \$y\$ 在 \$v\$ 之前，有 \$L \leq\$ 路径 \$P\$ 的长度 \$< dist[v]\$。又因为 \$dist[v]\$ 是 \$V - S\$ 中的最小值，所以 \$dist[v] \leq dist[y]\$。于是 \$dist[v] \leq dist[y] \leq L < dist[v]\$，矛盾。因此不存在更短路径，\$dist[v]\$ 即为最短。

该证明依赖于边权非负，否则归纳不成立。



{#fig-Dijkstra_proof width=100% fig-align="center"}

3.8.4 时间复杂度

Dijkstra 算法的时间复杂度取决于具体实现方式，下面结合两种常见版本的伪代码进行分析。

3.8.4.1 版本 1：朴素实现（使用邻接矩阵）

```
Algorithm Dijkstra_naive(G, w, s)
  for each v in V
    d[v] ← ∞
    parent[v] ← null
  d[s] ← 0
  S ← ∅
  while |S| < |V|
    u ← vertex in V \ S with smallest d[u]    // 需要扫描所有未确定顶点
    S ← S ∪ {u}
    for each v in adj[u]
      if d[u] + w(u, v) < d[v]
        d[v] ← d[u] + w(u, v)
  return d, parent
```

分析:

- 初始化部分: $O(|V|)$ 。
- 主循环执行 $|V| - 1$ 次。每次循环中:
 - 查找最小 $d[u]$: 需要扫描 $V \setminus S$ 中的所有顶点, 共 $O(|V|)$ 时间。
 - 更新邻接点: 对于每个顶点 u , 遍历其所有邻接点。所有顶点的邻接点总数为 $|E|$, 因此更新操作的总时间为 $O(|E|)$ (所有循环累加)。
- 总时间复杂度为 $O(|V|^2 + |E|)$ 。对于稠密图 ($|E| \approx |V|^2$), 该复杂度为 $O(|V|^2)$ 。

3.8.4.2 版本 2: 优先队列优化 (使用二叉堆)

```
Algorithm Dijkstra_pq(G, w, s)
  for each v in V
    d[v] ← ∞
    parent[v] ← null
  d[s] ← 0
  Q ← priority queue of vertices keyed by d
  while Q is not empty
    u ← extract-min(Q)
    for each v in adj[u]
      if d[u] + w(u,v) < d[v]
        d[v] ← d[u] + w(u,v)
        decrease-key(Q, v, d[v]) // 或直接插入新对
        parent[v] ← u
  return d, parent
```

分析:

- 初始化优先队列: 构建包含 $|V|$ 个顶点的优先队列。若采用逐个插入的方式, 每个插入操作需要 $O(\log |V|)$ 时间, 总时间为 $O(|V| \log |V|)$; 若采用堆化 (heapify) 操作, 则可在 $O(|V|)$ 时间内完成。通常假设使用堆化, 因此初始化部分视为 $O(|V|)$ 。
- 主循环执行 $|V|$ 次 extract-min 操作, 每次 $O(\log |V|)$, 总计 $O(|V| \log |V|)$ 。
- 每条边可能触发一次更新: 最坏情况下, 每次更新都需要执行 decrease-key 操作 (若使用支持该操作的堆)。使用二叉堆时, decrease-key 的时间复杂度为 $O(\log |V|)$, 因此所有边的更新总时间为 $O(|E| \log |V|)$ 。若采用插入新键值对的方式 (不执行 decrease-key, 允许堆中出现重复顶点), 则堆的大小可能增长至 $O(|E|)$, 每次插入操作 $O(\log |E|) = O(\log |V|)$, 总时间仍为 $O(|E| \log |V|)$ 。

因此总时间复杂度为 $O((|V| + |E|) \log |V|)$ 。对于稀疏图 ($|E|$ 远小于 $|V|^2$), 这比 $O(|V|^2)$ 更优。

3.8.4.3 版本 3: 斐波那契堆优化

若使用斐波那契堆实现优先队列, extract-min 的均摊时间为 $O(\log |V|)$, 而 decrease-key 的均摊时间为 $O(1)$, 因此总复杂度可降至 $O(|V| \log |V| + |E|)$ 。但在实际应用中, 斐波那契堆常数较大, 通常较少使用。

以上分析表明, Dijkstra 算法的效率与图的稠密程度及所选数据结构密切相关, 实际应用中需根据问题特点选择合适的实现方式。

3.8.5 思考题

- 1. 如果图中存在负权边，Dijkstra 算法为什么失效？请举例说明。
- 2. 如何用 Dijkstra 算法求两点间的最短路径（而不求所有点）？

3.9 贪心策略总结

3.9.1 本章实例的贪心策略

下表总结了本章各问题所采用的贪心准则：

问题	贪心准则	正确性条件
找零钱（美元体系）	每次选择不超过剩余金额的最大面值硬币	美元面值下成立
区间调度	每次选择结束时间最早且与已选区间不冲突的区间	总成立
分数背包	按性价比（价值/重量）降序，优先装性价比高的物品	总成立
哈夫曼编码	每次合并频率最小的两棵子树	总成立
最小生成树（Prim）	每次选择连接已选顶点集与未选顶点集的最小权边	总成立
最小生成树（Kruskal）	每次选择权值最小且不形成回路的边	总成立
单源最短路径（Dijkstra）	每次从未确定最短路径的顶点中选出当前距离最小的顶点	仅适用于非负权图

3.9.2 贪心法的特点与局限性

- **优点：**算法设计简单，通常具有较低的时间复杂度；易于理解和实现。
- **缺点：**并非普遍适用，必须满足**贪心选择性质**和**最优子结构**才能保证全局最优；正确性证明往往需要精巧的数学论证。
- **应用边界：**对于某些问题（如 0-1 背包、旅行商问题），贪心法只能得到近似解；即使问题具有最优子结构，也不一定满足贪心选择性质（例如带权区间调度）。

贪心法是一种简单而强大的算法设计思想，通过局部最优选择期望得到全局最优。它适用于具有贪心选择性质和最优子结构的问题。本章介绍了找零钱、区间调度、分数背包、哈夫曼编码、最小生成树和 Dijkstra 算法等经典例子，并给出了正确性证明和实现。通过本章学习，应掌握贪心策略的适用条件、证明方法以及实际应用。

3.10 习题

3.10.1 选择题（共 10 题）

- 1. 下列关于贪心算法的描述，哪一个是不正确的？
 - A. 贪心算法通常通过一系列局部最优选择来构造全局最优解
 - B. 贪心算法必须对所有输入实例都能得到最优解

- C. 贪心算法一般比动态规划算法效率更高
 - D. 贪心算法适用于具有“贪心选择性质”和“最优子结构”的问题
2. 在找零钱问题中, 如果硬币面值为 1、5、10、25 美分, 贪心算法总能得到最优解的根本原因是:
- A. 所有面值都是整数
 - B. 每种面值都是前面面值的倍数
 - C. 存在 1 美分硬币
 - D. 面值之间满足正则贪心性质
3. 有 n 个活动, 每个活动有开始时间 s_i 和结束时间 f_i , 采用贪心法选择最大相容活动子集, 通常按什么排序?
- A. 按开始时间升序
 - B. 按结束时间升序
 - C. 按持续时间升序
 - D. 按开始时间降序
4. 给定一组字符及其频率, 使用 Huffman 编码时, 贪心策略是:
- A. 每次选择频率最高的两个节点合并
 - B. 每次选择频率最低的两个节点合并
 - C. 每次选择深度最深的两个节点合并
 - D. 每次选择出现最早的两个字符合并
5. 关于最小生成树的 Prim 算法, 以下说法正确的是:
- A. 每次选择连接已选集合和未选集合的最小权边
 - B. 每次选择全局最小权边且不形成环
 - C. 必须从某个指定顶点开始, 且结果唯一
 - D. 时间复杂度总是优于 Kruskal 算法
6. 关于 Dijkstra 算法, 下列说法正确的是:
- A. 可以处理带有负权边的图
 - B. 每次选择当前距离源点最近的未确定顶点
 - C. 采用深度优先搜索策略
 - D. 不能用于有向图
7. 对于分数背包问题, 贪心策略是:
- A. 按重量从小到大选择
 - B. 按价值从大到小选择
 - C. 按单位重量价值从大到小选择
 - D. 随机选择
8. 在 Kruskal 算法中, 用于判断是否形成回路的数据结构通常是:
- A. 栈
 - B. 队列
 - C. 哈希表
 - D. 并查集
9. 下列哪个问题不能用贪心算法保证得到最优解?
- A. 活动安排问题

- B. 分数背包问题
- C. 0-1 背包问题
- D. 哈夫曼编码

10. 在哈夫曼编码中, 若所有字符频率相同, 则编码结果:

- A. 所有字符编码长度相同
 - B. 所有字符编码相同
 - C. 编码不唯一但总码长固定
 - D. 编码唯一
-

3.10.2 判断题 (共 10 题)

- 1. 贪心算法对所有最优化问题都能得到全局最优解。()
 - 2. 在活动安排问题中, 如果按结束时间最早贪心, 得到的一定是包含活动数量最多的子集。()
 - 3. Kruskal 算法和 Prim 算法都使用了贪心思想, 但 Kruskal 算法需要对边按权值排序。()
 - 4. 0-1 背包问题可以用贪心法按单位重量价值排序得到最优解。()
 - 5. 对于部分背包问题, 贪心法按价值密度排序可以保证得到最优解。()
 - 6. Dijkstra 算法中, 一旦一个顶点被加入集合 S, 其最短距离就再也不会被更新。()
 - 7. 哈夫曼树一定是满二叉树。()
 - 8. 在找零钱问题中, 只要存在 1 分硬币, 贪心法就一定得到最优解。()
 - 9. Prim 算法和 Kruskal 算法求出的最小生成树总权值一定相同。()
 - 10. 贪心算法比动态规划算法时间复杂度一定更低。()
-

3.10.3 填空题 (共 10 题)

说明: 请在以下各题的横线处填写正确答案。

- 1. 贪心算法求解问题的两个关键性质是 _____ 和 _____。
- 2. 在 Huffman 编码中, 给定频率 [5, 9, 12, 13, 16, 45], 第一次合并的两个频率是 _____ 和 _____。
- 3. 使用 Prim 算法求最小生成树, 从顶点 A 开始, 边权: AB=1, AC=3, AD=4, BC=2, BD=5, CD=6, 依次加入的边顺序是 _____ → _____ → _____。

4. 设有 n 个任务，每个任务有截止时间 d_i 和利润 p_i ，采用贪心法调度使利润最大，通常先按 _____ 降序处理任务，并尽量安排在 _____ 时间执行。
 5. 有硬币面值 $\{1, 3, 4\}$ ，要凑出 6 元，贪心法（每次取最大面值）会使用 _____ 枚硬币，而最优解需要 _____ 枚。
 6. Dijkstra 算法要求图中所有边的权值 _____，否则可能得到错误结果。
 7. 在 Kruskal 算法中，判断加入一条边是否会形成回路，通常使用 _____ 数据结构。
 8. 对于活动安排问题，按 _____ 最早进行贪心选择可以得到最优解。
 9. 哈夫曼编码是一种 _____ 编码，即没有任何字符的编码是另一个字符编码的前缀。
 10. 分数背包问题中，物品可以被 _____，这是它能用贪心法求解的关键。
-

3.10.4 简答题（共 5 题）

1. 简述贪心算法与动态规划的主要区别。
2. 在区间调度问题中，为什么按结束时间最早贪心能得到最优解？请用交换论证简要说明。
3. 为什么 0-1 背包问题不能用贪心法按价值密度求解，而分数背包可以？
4. 哈夫曼编码中，为什么合并频率最小的两个字符能得到最优前缀码？
5. Dijkstra 算法为什么要求边权非负？请举例说明负权边会导致错误。

3.10.5 证明题（共 6 题）

证明题 1：找零钱问题（美元硬币体系）

题目：

在美元硬币体系（面值为 1, 5, 10, 25, 50, 100 分）中，证明贪心算法（每次选择不超过剩余金额的最大面值硬币）总能得到最优解（即所用硬币数量最少）。

要求：

- (1) 说明贪心选择性质；
- (2) 说明最优子结构；
- (3) 给出完整证明（可用反证法或交换论证）。

参考证明框架：

设贪心解为 $G = (g_{100}, g_{50}, g_{25}, g_{10}, g_5, g_1)$ ，最优解为 $O = (o_{100}, o_{50}, o_{25}, o_{10}, o_5, o_1)$ 。

通过归纳法证明对每种面值 d ，有 $g_d = o_d$ 。

关键引理：对于 $d \in \{5, 10, 25, 50, 100\}$ ，小于 d 的所有硬币面值所能组成的最大金额小于 d ，因此任何最优解中必须包含足够多的大面值硬币。

证明题 2：活动安排问题

题目：

给定 n 个区间 (s_i, e_i) ，每个区间表示一个活动。证明：按结束时间最早贪心选择得到的活动集合是包含活动数量最多的可行解。

要求：

使用交换论证法证明。

参考证明框架：

设贪心解为 $G = (g_1, g_2, \dots, g_k)$ ，按结束时间递增排序；最优解为 $O = (o_1, o_2, \dots, o_m)$ ，同样按结束时间递增。

引理：对任意 $i \leq k$ ， g_i 的结束时间 $\leq o_i$ 的结束时间。

证明：归纳法。 $i = 1$ 时显然成立。假设对 $i - 1$ 成立，则 g_{i-1} 结束 $\leq o_{i-1}$ 结束 $\leq o_i$ 开始，故 o_i 在贪心第 i 步时仍是候选，因此 g_i 结束 $\leq o_i$ 结束。

结论：若 $m > k$ ，则 o_{k+1} 的开始 $\geq o_k$ 结束 $\geq g_k$ 结束，贪心会继续选，矛盾。故 $k = m$ 。

证明题 3：分数背包问题**题目：**

对于分数背包问题（物品可分割），证明按单位重量价值 (v_i/w_i) 降序贪心可以得到最大总价值。

要求：

使用交换论证法或反证法证明。

参考证明框架：

设物品按性价比降序为 $1, 2, \dots, n$ 。贪心解 G 取前 $t - 1$ 个物品全部，第 t 个物品取比例 α ($0 < \alpha \leq 1$)，后面物品不取。

假设最优解 O 与 G 不同，找到第一个性价比不同的位置 i ，则 O 中必然存在某个 $j > i$ 取了正比例而 i 未取满。交换 i 和 j 的部分重量，总价值增加（因为 $v_i/w_i > v_j/w_j$ ），与 O 最优矛盾。

证明题 4：哈夫曼编码最优性**题目：**

证明哈夫曼算法（每次合并频率最小的两个节点）构造的前缀码树具有最小加权外路径长度（即总编码长度最小）。

要求：

使用数学归纳法，并利用引理：存在一个最优前缀码树，其中频率最小的两个字符是兄弟节点且位于最深层。

参考证明框架：

归纳基础： $n = 2$ 时显然。

归纳步骤：设 x, y 为频率最小的两个字符，合并为 z ，频率 $f(z) = f(x) + f(y)$ 。由归纳假设，算法对 $n - 1$ 个字符（包含 z ）得到最优树 T' 。将 T' 中的叶子 z 替换为以 x, y 为孩子的内部节点，得到 T 。

由引理，存在最优树使 x, y 为兄弟，且 $B(T) = B(T') + f(x) + f(y)$ 。若 T 不是最优，则存在更优树 T^* ，将其中的 x, y 兄弟合并为 z 得到 $T^{*'}$ ，则 $B(T^{*'}) < B(T')$ ，与 T' 最优矛盾。

证明题 5：Prim 算法的正确性

题目：

对于连通无向带权图 $G = (V, E)$ ，证明 Prim 算法（每次从已选顶点集 U 到未选顶点集 $V - U$ 中选择最小权边加入）得到的生成树是最小生成树。

要求：

使用循环不变式或归纳法证明。

参考证明框架：

循环不变式：在算法每一步，已选边集 T 是某个最小生成树的子集。

初始化： $T = \emptyset$ ，显然成立。

保持：假设 T 是某最小生成树 M 的子集。算法选择边 $e = (u, v)$ ，其中 $u \in U$ ， $v \notin U$ ，且 $w(e)$ 最小。若 $e \in M$ ，不变式保持。若 $e \notin M$ ，则 $M + e$ 形成回路，回路中必有另一条边 e' 连接 U 与 $V - U$ ，且 $w(e') \geq w(e)$ 。用 e 替换 e' 得到 M' ，则 M' 也是最小生成树且包含 $T \cup \{e\}$ 。

终止： $|U| = |V|$ 时， T 包含 $|V| - 1$ 条边，且是某最小生成树的子集，故 T 本身就是最小生成树。

证明题 6: Kruskal 算法的正确性**题目：**

对于连通无向带权图 $G = (V, E)$ ，证明 Kruskal 算法（每次选择权值最小且不形成回路的边加入生成树）得到的生成树是最小生成树。

要求：

使用循环不变式或归纳法证明，并说明为什么“不形成回路”的条件是必要的。

参考证明框架：**方法一：循环不变式法**

循环不变式：在算法的每一步，当前已选边集 F 是某个最小生成树的子集。

初始化： $F = \emptyset$ ，显然是任何最小生成树的子集。

保持：假设 F 是某个最小生成树 T 的子集。算法选择下一条边 $e = (u, v)$ ，它是所有未考虑边中权值最小且加入 F 后不形成回路的边。分两种情况：

- 若 $e \in T$ ，则 $F \cup \{e\}$ 仍然是 T 的子集，不变式保持。
- 若 $e \notin T$ ，则 T 中必然存在一条从 u 到 v 的唯一路径 P （因为 T 是树）。由于 F 中无边连接 u 和 v （否则加入 e 会形成回路），路径 P 上至少有一条边 e' 不在 F 中。考虑将 e' 加入 F 的情况：由于 $F \subseteq T$ ，且 e 是目前权值最小的可行边，而 e' 在 e 之后才会被考虑（或者权值更大），因此 $w(e) \leq w(e')$ 。从 T 中删除 e' 并加入 e ，得到新树 $T' = T \setminus \{e'\} \cup \{e\}$ ，则 $w(T') = w(T) + w(e) - w(e') \leq w(T)$ 。由于 T 是最小生成树，所以 $w(T') = w(T)$ ，即 T' 也是最小生成树。而 $F \cup \{e\} \subseteq T'$ ，因此不变式保持。

终止：当 $|F| = |V| - 1$ 时， F 是一棵生成树，且是某个最小生成树的子集，因此 F 本身就是最小生成树。

方法二：数学归纳法（按边数归纳）

归纳基础：当 $|F| = 0$ 时， F 是任何最小生成树的子集，成立。

归纳假设：假设算法选定的前 k 条边 $e_1, e_2, \dots, e_k$ 是某个最小生成树 T_k 的子集。

归纳步骤：考虑第 $k+1$ 条边 $e_{k+1} = (u, v)$ 。若 $e_{k+1} \in T_k$ ，则结论显然。若 $e_{k+1} \notin T_k$ ，则在 T_k 中 u 和 v 之间有一条唯一路径 P 。由于 e_{k+1} 加入 F_k 不形成回路，路径 P 上至少有一条边 e' 不在 F_k 中（否则 F_k 已经连通了 u 和 v ）。由 Kruskal 算法的选择规则， $w(e_{k+1}) \leq w(e')$ 。构造 $T_{k+1} = T_k \setminus \{e'\} \cup \{e_{k+1}\}$ ，则 T_{k+1} 也是最小生成树且包含 F_{k+1} 。归纳成立。

终止：当 $k = |V| - 1$ 时， F 即为最小生成树。

补充说明：为什么“不形成回路”条件必要？

如果允许加入形成回路的边，则最终得到的子图会包含环，不再是树。包含环的子图必然有多余的边，删除环上任意一条边可以得到更小权值的生成树（因为边权非负）。因此，形成回路的边不可能出现在任何最小生成树中，Kruskal 算法通过并查集检查并跳过这些边是正确且必要的。

与 Prim 算法的对比

算法	贪心策略	数据结构	适用场景	正确性证明核心
Prim	每次选连接已选集合与未选集合的最小边	邻接矩阵/堆	稠密图	割的性质 + 交换论证
Kruskal	每次选全局最小且不形成回路的边	并查集	稀疏图	回路性质 + 交换论证

两种算法都基于贪心思想，但 Kruskal 的证明更依赖于“最小权边跨越某个割”这一性质，而 Prim 则直接利用割的性质逐步扩张。

证明题 7: Dijkstra 算法的正确性

题目：

对于边权非负的有向图 $G = (V, E)$ ，证明 Dijkstra 算法（每次从 $V - S$ 中选择 $dist$ 最小的顶点加入 S ）得到的 $dist[v]$ 是从源点 s 到 v 的最短路径长度。

要求：

使用归纳法证明，并说明非负权条件为何必要。

参考证明框架：

归纳假设：当顶点 u 被加入集合 S 时， $dist[u]$ 是 s 到 u 的最短路径长度。

归纳基础： s 首先被加入， $dist[s] = 0$ ，成立。

归纳步骤：假设 S 中所有顶点满足假设。设 v 是下一个被加入的顶点，即 $v = \arg \min_{x \notin S} dist[x]$ 。

反证：假设存在一条更短的 s 到 v 的路径 P ，其长度 $< dist[v]$ 。设 y 是 P 上第一个不在 S 中的顶点， x 是 y 的前驱 ($x \in S$)。则 $dist[y] \leq dist[x] + w(x, y) \leq \text{路径 } P \text{ 的长度} < dist[v]$ ，与 v 是 $V - S$ 中 $dist$ 最小的矛盾。

因此 $dist[v]$ 就是最短路径长度。

非负权必要性：若存在负权边，则 $dist[y]$ 可能大于实际经过负权边后的距离，导致归纳失效。

3.10.6 客观编程题

编程题 1：找零钱问题（贪心法验证）

题目描述

给定一个硬币面值系统（已按降序排列，包含 1 分硬币以保证任意金额都可找零）和一个需要找零的金额 M （单位：分），请使用贪心法（每次选择不超过剩余金额的最大面值硬币）计算所需的最少硬币数量，并输出每种面值硬币的使用数量。

注意：本题仅要求实现贪心法，不需要判断该面值系统下贪心法是否最优。

输入格式

第一行两个整数 n 和 M ， n 表示硬币面值种类数， M 表示需要找零的金额（ $1 \leq n \leq 10$ ， $1 \leq M \leq 10^6$ ）。

第二行 n 个整数 $coins[0..n-1]$ ，表示硬币面值，已按降序排列，且 $coins[n-1] = 1$ 。

输出格式

第一行输出一个整数，表示最少硬币数量（按贪心法计算）。

第二行输出 n 个整数，表示每种面值硬币的使用数量，按输入顺序输出。

样例输入

```
6 34
25 10 5 2 1
```

样例输出

```
6
1 0 1 0 4
```

编程题 2：活动安排问题

题目描述

设有 n 个活动，每个活动有一个开始时间 s_i 和结束时间 e_i （ $s_i < e_i$ ）。同一时间只能进行一个活动（端点相接视为不重叠，即一个结束时间等于另一个开始时间时可以都选）。请用贪心法（按结束时间最早）选择尽可能多的互不重叠的活动，输出最大可安排的活动数量以及按结束时间升序排列的选中活动。

输入格式

第一行一个整数 n （ $1 \leq n \leq 10^5$ ）。

接下来 n 行，每行两个整数 s_i, e_i （ $0 \leq s_i < e_i \leq 10^9$ ）。

输出格式

第一行输出一个整数 k ，表示最大可安排的活动数量。

接下来 k 行，每行两个整数，按结束时间升序输出选中的活动（若结束时间相同按开始时间升序），每个活动输出其开始时间和结束时间。

样例输入

```
11
1 4
3 5
```

```
0 6
5 7
3 8
5 9
6 10
8 11
8 12
2 13
12 14
```

样例输出

```
4
1 4
5 7
8 11
12 14
```

编程题 3：分数背包问题**题目描述**

有 n 种物品，每种物品有重量 w_i 和价值 v_i ，可以任意分割。背包容量为 C 。请使用贪心法（按单位重量价值 v_i/w_i 降序）计算能获得的最大总价值，结果四舍五入保留两位小数。

输入格式

第一行两个整数 n 和 C ($1 \leq n \leq 10^3$, $1 \leq C \leq 10^6$)。
接下来 n 行，每行两个整数 v_i 和 w_i ($1 \leq v_i, w_i \leq 10^5$)。

输出格式

输出一个浮点数，表示最大总价值，保留两位小数。

样例输入

```
3 50
60 10
100 20
120 30
```

样例输出

```
240.00
```

编程题 4：哈夫曼编码**题目描述**

给定 n 个字符及其出现频率，请使用哈夫曼算法构造最优前缀码，并输出每个字符的哈夫曼编码。如果两个节点频率相同，则优先合并**先出现**的节点（按输入顺序，索引小的优先）。编码规则：左分支为'0'，右分支为'1'。

输入格式

第一行一个整数 n ($2 \leq n \leq 100$)。

接下来 n 行，每行一个字符（小写字母）和一个整数频率 f ($1 \leq f \leq 10^5$)。

输出格式

输出 n 行，每行一个字符和对应的哈夫曼编码，格式为字符：编码，按输入顺序输出。

样例输入

```
6
a 5
b 9
c 12
d 13
e 16
f 45
```

样例输出

```
a:1100
b:1101
c:100
d:101
e:111
f:0
```

编程题 5：最小生成树（Kruskal 算法）

题目描述

给定一个 n 个顶点 m 条边的无向连通带权图，顶点编号从 1 到 n 。请使用 Kruskal 算法（贪心法）求最小生成树的边权之和，并输出按权值升序、权值相同时按顶点编号升序排列的选中边（每条边输出两个端点编号，小号在前）。

输入格式

第一行两个整数 n 和 m ($2 \leq n \leq 10^5$, $n - 1 \leq m \leq 2 \times 10^5$)。

接下来 m 行，每行三个整数 u, v, w ($1 \leq u < v \leq n$, $1 \leq w \leq 10^9$)，表示一条连接 u 和 v 的边，权值为 w 。输入保证图连通且没有重边。

输出格式

第一行输出一个整数，表示最小生成树的总权值。

第二行输出 $n - 1$ 条边的信息，每条边格式为 $u-v$ ，按权值升序输出，权值相同时按 u 升序， u 相同时按 v 升序。

样例输入

```
4 6
1 2 2
```

```
1 3 3
1 4 6
2 3 4
2 4 5
3 4 1
```

样例输出

```
6
3-4 1-2 1-3
```

编程题 6: 单源最短路径 (Dijkstra 算法)**题目描述**

给定一个 n 个顶点 m 条边的有向带权图，顶点编号从 1 到 n ，所有边权均为**非负整数**。求从源点 s 到所有其他顶点的最短路径长度。如果某个顶点不可达，输出-1。

输入格式

第一行三个整数 n, m, s ($2 \leq n \leq 10^5$, $1 \leq m \leq 2 \times 10^5$, $1 \leq s \leq n$)。

接下来 m 行，每行三个整数 u, v, w ($1 \leq u, v \leq n$, $u \neq v$, $0 \leq w \leq 10^9$)，表示从 u 到 v 有一条有向边，权值为 w 。输入可能包含重边，取最小权值。

输出格式

输出一行 n 个整数，表示从 s 到顶点 $1, 2, \dots, n$ 的最短距离，用空格分隔。若不可达，输出-1。 s 到自身的距离为 0。

样例输入

```
5 7 1
1 2 10
1 3 3
2 3 1
2 4 2
3 2 4
3 4 8
3 5 2
4 5 7
5 4 9
```

样例输出

```
0 7 3 9 5
```


4 分治法

4.1 分治法的基本思想

分治法 (Divide and Conquer) 是一种非常直观且强大的算法设计策略。它的核心思想可以概括为十六个字：**将问题分解，分别求解，然后合并**。具体来说，分治法将一个难以直接解决的大问题，划分成若干个规模较小、性质相同（或相似）的子问题，递归地求解这些子问题，然后再将子问题的解合并得到原问题的解。

分治法的典型步骤包含三个阶段：

1. **分 (Divide)**：将原问题分解成若干个规模较小的子问题，这些子问题与原问题形式相同。
2. **治 (Conquer)**：递归地求解各个子问题。当子问题规模足够小时（例如只有一个元素），直接求解。
3. **合 (Combine)**：将子问题的解合并为原问题的解。

分治法的通用算法模板可以表示如下：

```
Algorithm divideAndConquer(P)
    // 如果问题规模足够小，直接求解
    if |P| is small enough:
        return solveDirect(P)
    // 将问题分解为若干子问题
    subproblems = divide(P)
    // 递归求解每个子问题
    for each sub in subproblems:
        subResult = divideAndConquer(sub)
    // 合并子问题的解
    return combine(subResults)
```

分治法并不要求子问题之间相互独立。即使子问题有重叠，分治法仍然可以工作，只是可能会重复计算相同子问题，导致效率降低。例如，朴素的递归斐波那契算法就是一种分治法，但由于子问题大量重叠，其时间复杂度高达指数级。当子问题相互独立时，分治法往往能达到最优效率；当子问题不独立时，我们通常会考虑动态规划等方法来避免重复计算。

本章将通过多个经典问题展示分治法的应用，并学习如何分析递归算法的时间复杂度。

4.2 二分查找

4.2.1 问题描述

给定一个按**非递减**顺序排列的数组 $A[0..n-1]$ 和一个目标值 key ，找出 key 在数组中的位置（下标），若不存在则返回 -1。

4.2.2 分治思路

下面假设这个有序数组是**非递减**的数组。二分查找是分治法的典型应用，其核心思想可以清晰地对应分治法的三个步骤：

- **分**：将当前查找区间 $[L, R]$ 分成两半，取中间位置 $mid = (L+R)//2$ ，比较 $A[mid]$ 与 key 。根据比较结果，确定目标值可能存在的子区间：
 - 若 $A[mid] = key$ ，则查找成功，直接返回；
 - 若 $A[mid] > key$ ，则说明 key 只能在左半区间 $[L, mid-1]$ ；
 - 若 $A[mid] < key$ ，则说明 key 只能在右半区间 $[mid+1, R]$ 。
- **治**：递归地在选定的子区间中执行相同的查找过程，直到找到目标或区间为空（即 $L > R$ ）为止。
- **合**：由于每次递归直接返回查找结果（找到的位置或 -1 ），无需额外的合并操作，因此合并阶段极为简单。

4.2.3 例子

假设有数组 $A = [2, 5, 8, 12, 16, 23, 38, 45, 56]$ ，我们要查找 $key = 23$ 。

- 初始区间 $[0, 8]$ ， $mid = 4$ ， $A[4]=16 < 23$ ，进入右半区间 $[5, 8]$ 。
- 新区间 $[5, 8]$ ， $mid = 6$ ， $A[6]=38 > 23$ ，进入左半区间 $[5, 5]$ 。
- 区间 $[5, 5]$ ， $mid = 5$ ， $A[5]=23$ ，找到，返回下标 5。

若查找 $key = 10$ ，最后区间缩小到 $[3, 2]$ （即 $L > R$ ），返回 -1 。

4.2.4 伪代码

```
Algorithm binarySearch(A, L, R, key)
  if L > R then
    return -1
  mid = floor((L + R) / 2)
  if A[mid] == key then
    return mid
  else if A[mid] > key then
    return binarySearch(A, L, mid-1, key)
  else
    return binarySearch(A, mid+1, R, key)
```

4.2.5 完整代码

4.2.5.1 C++ 实现

```
#include <iostream>
#include <vector>
using namespace std;

int binarySearch(const vector<int>& A, int L, int R, int key) {
  if (L > R) return -1;
  int mid = L + (R - L) / 2; // 避免溢出
  if (A[mid] == key)
    return mid;
  else if (A[mid] > key)
```

```

        return binarySearch(A, L, mid - 1, key);
    else
        return binarySearch(A, mid + 1, R, key);
}

int main() {
    vector<int> A = {2, 5, 8, 12, 16, 23, 38, 45, 56};
    int key = 23;
    int pos = binarySearch(A, 0, A.size() - 1, key);
    if (pos != -1)
        cout << " 找到 " << key << " 在下标 " << pos << endl;
    else
        cout << key << " 不存在" << endl;
    return 0;
}

```

4.2.5.2 Python 实现

```

def binary_search(A, L, R, key):
    if L > R:
        return -1
    mid = (L + R) // 2
    if A[mid] == key:
        return mid
    elif A[mid] > key:
        return binary_search(A, L, mid - 1, key)
    else:
        return binary_search(A, mid + 1, R, key)

if __name__ == "__main__":
    A = [2, 5, 8, 12, 16, 23, 38, 45, 56]
    key = 23
    pos = binary_search(A, 0, len(A) - 1, key)
    if pos != -1:
        print(f" 找到 {key} 在下标 {pos}")
    else:
        print(f"{key} 不存在")

```

4.2.6 时间复杂度分析

每次递归调用将问题规模减半，且递归内部只做常数次比较，因此递推关系为：

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + O(1)$$

解此递推式可得 $T(n) = O(\log n)$ 。

思考：如果数组中有重复元素，如何找到第一个出现的位置？最后一个出现的位置？这需要对二分查找进行微调，你能想到修改方法吗？

4.3 汉诺塔问题

4.3.1 问题描述

有三根柱子 A、B、C，初始时 A 柱上有 n 个大小不等的圆盘，按从小到大的顺序叠放（最大的在底部）。要求将所有圆盘从 A 柱移动到 C 柱，每次只能移动一个圆盘，且任何时候都不能将大盘放在小盘上面。问如何移动，并给出移动序列。

4.3.2 分治思路

将 n 个盘子从 A 移到 C 可以分解为以下三个步骤：

1. 将上面 $n - 1$ 个盘子从 A 借助 C 移到 B；
2. 将最大的盘子直接从 A 移到 C；
3. 将 B 上的 $n - 1$ 个盘子借助 A 移到 C。

步骤 1 和 3 与原问题相同，只是规模减一，因此可以递归求解。这就是典型的“分-治-合”过程。

4.3.3 例子： $n = 3$ 的递归树

当 $n = 3$ 时，我们可以用一棵递归树来清晰地展示整个递归调用过程（如 Figure 4.1 所示）。树的根节点表示原问题 $\text{hanoi}(3, A, B, C)$ ，每个节点表示一次函数调用，叶子节点表示直接移动一个盘子。

```

hanoi(3, A, B, C)
├─ hanoi(2, A, C, B)
│  ├─ hanoi(1, A, B, C)
│  │  └─ move A→C
│  └─ move A→B
│     └─ hanoi(1, C, A, B)
│        └─ move C→B
├─ move A→C
└─ hanoi(2, B, A, C)
   └─ hanoi(1, B, C, A)
      └─ move B→A
         └─ move B→C
            └─ hanoi(1, A, B, C)
               └─ move A→C
  
```

按照递归树的前序遍历（或实际递归顺序），我们得到移动序列：

1. move A→C（将 1 号盘从 A 移到 C）

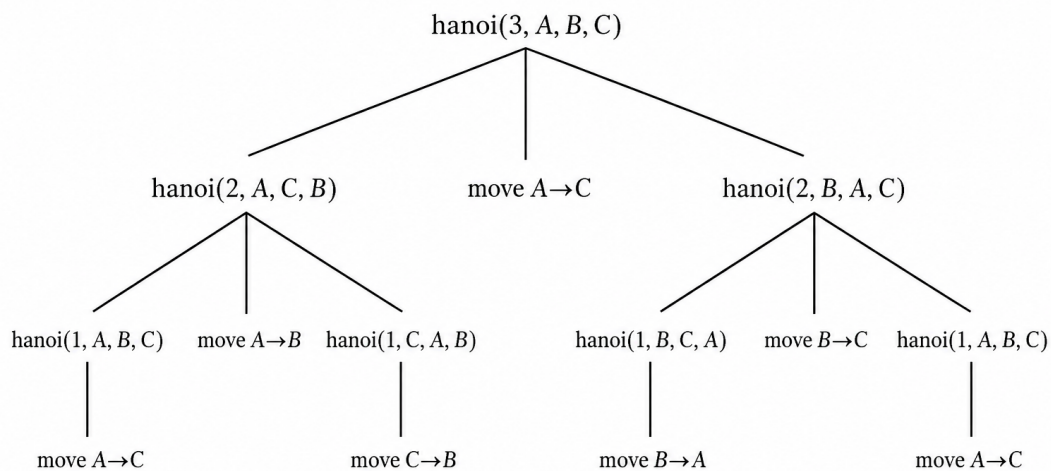


Figure 4.1: $n = 3$ 时汉诺塔递归调用过程的递归树

2. move A→B (将 2 号盘从 A 移到 B)
3. move C→B (将 1 号盘从 C 移到 B)
4. move A→C (将 3 号盘从 A 移到 C)
5. move B→A (将 1 号盘从 B 移到 A)
6. move B→C (将 2 号盘从 B 移到 C)
7. move A→C (将 1 号盘从 A 移到 C)

共 $2^3 - 1 = 7$ 步。

4.3.4 伪代码

```

Algorithm hanoi(n, A, B, C)
    // 将 n 个盘子从 A 借助 B 移到 C
    if n = 1 then
        move(A, C)          // 将唯一的盘子从 A 移到 C
    else
        hanoi(n-1, A, C, B) // 将上面 n-1 个盘子从 A 借助 C 移到 B
        move(A, C)          // 将最大的盘子从 A 移到 C
        hanoi(n-1, B, A, C) // 将 B 上的 n-1 个盘子从 B 借助 A 移到 C
    end if
  
```


4.3.5 完整代码

4.3.5.1 C++ 实现

```
#include <iostream>
using namespace std;

void move(char src, char dst) {
    cout << src << " -> " << dst << endl;
}

void hanoi(int n, char A, char B, char C) {
    if (n == 1) {
        move(A, C);
    } else {
        hanoi(n - 1, A, C, B);
        move(A, C);
        hanoi(n - 1, B, A, C);
    }
}

int main() {
    int n = 3;
    cout << " 移动 " << n << " 个盘子的步骤: " << endl;
    hanoi(n, 'A', 'B', 'C');
    return 0;
}
```

4.3.5.2 Python 实现

```
def move(src, dst):
    print(f"{src} -> {dst}")

def hanoi(n, A, B, C):
    if n == 1:
        move(A, C)
    else:
        hanoi(n - 1, A, C, B)
        move(A, C)
        hanoi(n - 1, B, A, C)

if __name__ == "__main__":
    n = 3
```

```
print(f" 移动 {n} 个盘子的步骤：")
hanoi(n, 'A', 'B', 'C')
```

4.3.6 时间复杂度分析

设 $T(n)$ 为移动 n 个盘子所需步数。根据算法，有：

$$T(n) = 2T(n-1) + 1, \quad T(1) = 1$$

解此递推式： $T(n) = 2^n - 1$ ，故时间复杂度为 $O(2^n)$ 。

思考：汉诺塔问题有没有非递归的解法？能否用栈模拟递归过程？

4.4 归并排序

4.4.1 问题描述

给定一个长度为 n 的数组 $A[0 \dots n-1]$ ，将其按非降序排序。

4.4.2 分治思路

归并排序是分治法的经典案例。

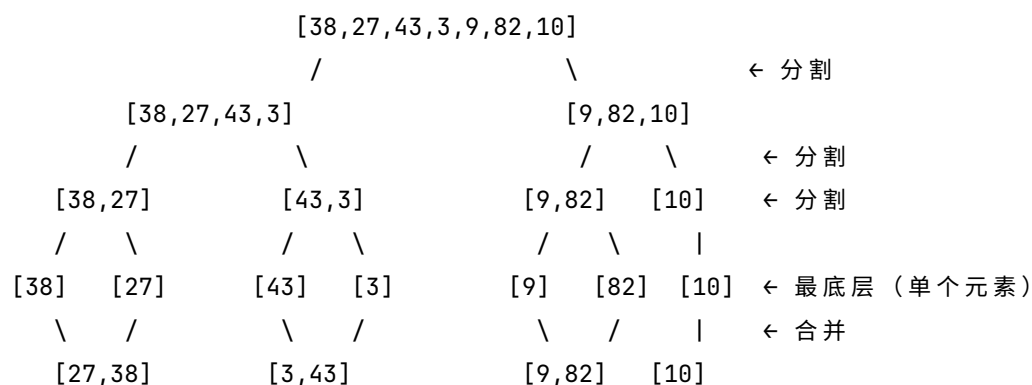
- **分：**将数组从中间位置分成两个规模大致相等的子数组。
- **治：**递归地对两个子数组进行归并排序。
- **合：**将两个已排序的子数组合并成一个有序数组。

合并过程需要额外的辅助数组，通过双指针依次比较两个子数组的元素，将较小者放入结果中，最后将剩余部分复制进去。

4.4.3 例子

对数组 $[38, 27, 43, 3, 9, 82, 10]$ 进行归并排序。

分治递归过程中的其分割和合并过程如图 4.2 所示：



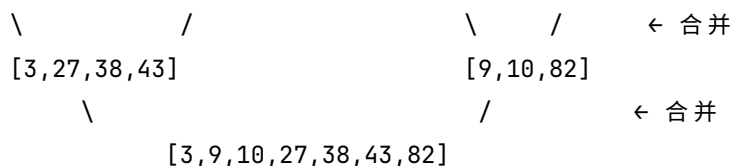


图 4.1 [38, 27, 43, 3, 9, 82, 10] 的归并排序过程

Algorithm mergeSort(A, L, R)

```

if L = R then
    return
mid = floor((L + R) / 2)
mergeSort(A, L, mid)
mergeSort(A, mid+1, R)
merge(A, L, mid, R)

```

Algorithm merge(A, L, M, R)

```

// 创建临时数组 B, 大小为 R-L+1
B = new array of size R-L+1
i = L, j = M+1, k = 0
while i ≤ M and j ≤ R do
    if A[i] ≤ A[j] then
        B[k] = A[i]; i++; k++
    else
        B[k] = A[j]; j++; k++
    end if
end while
while i ≤ M do
    B[k] = A[i]; i++; k++
end while
while j ≤ R do
    B[k] = A[j]; j++; k++
end while
// 将 B 复制回 A[L..R]
for t = 0 to k-1 do
    A[L + t] = B[t]
end for

```

4.4.5 完整代码

4.4.5.1 C++ 实现

```
#include <iostream>
#include <vector>
using namespace std;

void merge(vector<int>& A, int L, int M, int R) {
    vector<int> B(R - L + 1);
    int i = L, j = M + 1, k = 0;
    while (i <= M && j <= R) {
        if (A[i] <= A[j])
            B[k++] = A[i++];
        else
            B[k++] = A[j++];
    }
    while (i <= M) B[k++] = A[i++];
    while (j <= R) B[k++] = A[j++];
    for (int t = 0; t < k; t++)
        A[L + t] = B[t];
}

void mergeSort(vector<int>& A, int L, int R) {
    if (L >= R) return;
    int mid = L + (R - L) / 2;
    mergeSort(A, L, mid);
    mergeSort(A, mid + 1, R);
    merge(A, L, mid, R);
}

int main() {
    vector<int> A = {38, 27, 43, 3, 9, 82, 10};
    mergeSort(A, 0, A.size() - 1);
    for (int x : A) cout << x << " ";
    cout << endl;
    return 0;
}
```

4.4.5.2 Python 实现

```
def merge(A, L, M, R):
    B = []
```

```

i, j = L, M + 1
while i <= M and j <= R:
    if A[i] <= A[j]:
        B.append(A[i])
        i += 1
    else:
        B.append(A[j])
        j += 1
while i <= M:
    B.append(A[i])
    i += 1
while j <= R:
    B.append(A[j])
    j += 1
A[L:R+1] = B

def mergeSort(A, L, R):
    if L >= R:
        return
    mid = (L + R) // 2
    mergeSort(A, L, mid)
    mergeSort(A, mid + 1, R)
    merge(A, L, mid, R)

if __name__ == "__main__":
    A = [38, 27, 43, 3, 9, 82, 10]
    mergeSort(A, 0, len(A) - 1)
    print(A)

```

4.4.6 时间复杂度分析

设 $T(n)$ 为归并排序的时间。每次划分需要 $O(1)$ ，递归处理两个规模为 $n/2$ 的子问题，合并需要 $O(n)$ 。因此：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

解此递推式，可得 $T(n) = O(n \log n)$ 。

思考：归并排序需要额外的 $O(n)$ 空间，有没有原地归并的方法？代价如何？

4.5 逆序数

4.5.1 从音乐评分到逆序对

假设你是一个音乐爱好者，在一个音乐网站上给自己喜欢的歌曲按喜爱程度排序。你对 5 首歌曲的排序是 $[1, 2, 3, 4, 5]$ (1 表示最喜爱，5 表示最不喜爱)。另一位用户也对这 5 首歌曲给出了他的排序，但与你不同： $[1, 3, 4, 2, 5]$ 。这意味着，对于你心目中排名第 2 的歌曲，他认为应该排在第 3；而你排名第 4 的歌曲，他认为应该排在第 2，等等。那么，如何量化你们之间口味的差异呢？

一种直观的方法是计算这个用户排序序列中的“混乱”程度，即有多少对歌曲的相对顺序与你的排序不一致。在你的排序中，所有歌曲是按编号递增的，所以任何一对歌曲 (i, j) 如果 $i < j$ ，你应该更喜欢歌曲 i 而不是 j 。如果另一个用户对这对歌曲的排序与你相反（即他把 j 排在 i 前面），那么就产生了一个“逆序”。这种逆序对的总数就可以衡量他与你的口味差异。

形式化地说，给定一个数组 $A[0..n-1]$ ，如果 $i < j$ 且 $A[i] > A[j]$ ，则称 (i, j) 为一个**逆序对**。数组中所有逆序对的总数称为该数组的**逆序数**。逆序数越大，表示数组与标准顺序（递增）的差异越大。

例子：用户排序 $[1, 3, 4, 2, 5]$ 中，我们找出所有逆序对：

- 1 后面所有数都大于 1，无逆序。
- 3 后面：4>3（不是逆序），2<3（是逆序，因为 3>2 且位置靠前），5>3（不是）。所以得到一个逆序 (3,2)。
- 4 后面：2<4（逆序），5>4（不是）。得到 (4,2)。
- 2 后面：5>2（不是）。
- 总共 2 个逆序对。这个数 (2) 就反映了用户的排序与你（标准递增顺序）的差异程度。

因此，逆序数可以作为一种简单的相似性度量。在统计学中，这被称为 **Kendall tau** 距离。

4.5.2 逆序数的应用

逆序数在多个领域有重要应用：

- **排序算法分析：**插入排序的时间复杂度与逆序数成正比（对于基本有序的数组，插入排序很快）。
- **推荐系统：**通过计算用户评分之间的逆序数（Kendall tau 距离），可以衡量用户兴趣的相似度，用于协同过滤。
- **生物信息学：**比较基因序列的相似性，例如在基因重排分析中。
- **投票理论：**衡量投票结果的一致性，例如在选举中计算不同投票者之间的偏好差异。

4.5.3 用分治法计算逆序数

最直接的方法就是双重循环：对于每个 i ，遍历所有 $j > i$ ，检查 $A[i] > A[j]$ 是否成立。这种方法的时间复杂度是 $O(n^2)$ 。当数组长度达到几万甚至几百万时，这个速度就难以接受了。采用分治法可以将时间降至 $O(n \log n)$ 。

分治思路：

- **分：**将数组从中间分成左右两个子数组 L 和 R 。
- **治：**递归计算左子数组内部的逆序数 inv_L 和右子数组内部的逆序数 inv_R 。
- **合：**统计跨越左右两个子数组的逆序对，即左子数组中的元素大于右子数组中的元素的对数，记为 inv_{LR} 。总逆序数为 $inv_L + inv_R + inv_{LR}$ 。

例如，考虑数组 $[1, 5, 4, 8, 10, 2, 6, 9, 3, 7]$ ，我们可以将其从中间分成左右两部分：左边 $[1, 5, 4, 8, 10]$ ，右边 $[2, 6, 9, 3, 7]$ 。通过肉眼观察：

- 左边内部的逆序对只有 $(5, 4)$ ，所以 $inv_L = 1$ 。
- 右边内部的逆序对有 $(6, 3)$ 、 $(9, 3)$ 、 $(9, 7)$ ，所以 $inv_R = 3$ 。
- 跨越左右的逆序对：左边每个元素与右边每个元素比较，得到 $(5, 2)$ 、 $(5, 3)$ 、 $(4, 2)$ 、 $(4, 3)$ 、 $(8, 2)$ 、 $(8, 3)$ 、 $(8, 6)$ 、 $(8, 7)$ 、 $(10, 2)$ 、 $(10, 3)$ 、 $(10, 6)$ 、 $(10, 7)$ 、 $(10, 9)$ ，共 13 个。因此总逆序数 $= 1 + 3 + 13 = 17$ 。

这个例子中，计算跨越逆序对需要比较 $5 \times 5 = 25$ 次，即 $O(n^2)$ 量级。如果数组很大，这样的比较将非常耗时。那么，有没有办法更快地统计跨越逆序对呢？

4.5.4 如何高效计算逆序数？

观察发现，如果左右两个子数组是**有序**的，那么统计跨越逆序对就会变得非常简单。例如，将上面的左右子数组排序后得到：

- 左边有序： $[1, 4, 5, 8, 10]$
- 右边有序： $[2, 3, 6, 7, 9]$

可以用**双指针法**来统计跨越逆序对。如图 4-1 所示，指针 i 扫描左边，指针 j 扫描右边，当左边当前元素大于右边当前元素时，左边从 i 到末尾的所有元素都大于这个右边元素，因此可以一次性统计多个逆序对。具体过程如下：

1. 用指针 i 指向左边开头，指针 j 指向右边开头。
2. 比较 左边 $[i]$ 和 右边 $[j]$ ：
 - 如果 左边 $[i] \leq$ 右边 $[j]$ ：说明左边这个元素不大于右边当前元素，不构成逆序对， i 右移。
 - 如果 左边 $[i] >$ 右边 $[j]$ ：**此时左边从 i 到末尾的所有元素都大于右边 $[j]$** （因为左边有序递增），所以它们都和 右边 $[j]$ 构成逆序对。一次性加上左边剩余元素的个数，然后 j 右移。

通过双指针只需要把两个数组各扫描一遍（ $5+5=10$ 次比较）就能得到跨越两个有序数组的逆序对。能够避免 $O(n^2)$ 的比较（如 @fig-inversion_number 所示）。

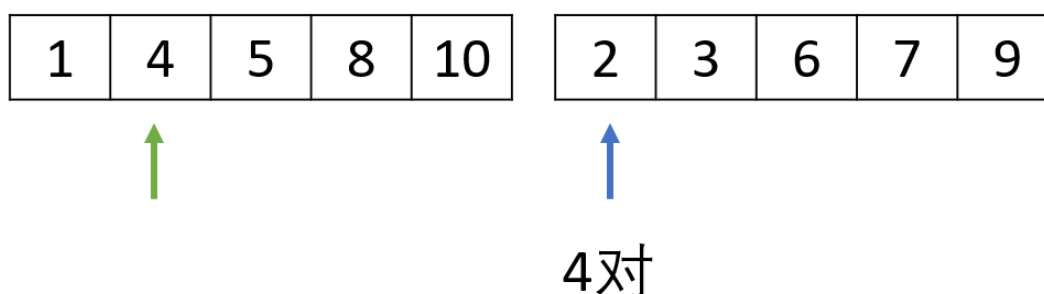


Figure 4.2: 两个指针 i, j 分别指向左右两个有序数组，如果 $a_i > b_j$ ，则 a_i 之后的 a_k 也都 $a_k > a_j$

逆序数的分治过程和归并排序的“一分为二”的分治过程非常类似：它们都是将数组一分为二，然后递归处理 2 个子数组，最后合并阶段过程也类似，都是用两个指针遍历有序数组，归并排序是合并有序数组，而逆序数是计算跨越有序数组的逆序数数目，但前提是两个子数组有序，但如何使得子数组有序呢？

如果将两者结合在一起，就可以利用两个子序列有序的特性，在合并的同时快速计算跨越这两个子序列的逆序数。

实际上，归并排序的合并过程恰好提供了这样的机会：当两个子数组已经分别有序时，我们可以在线性时间内合并它们，并在合并过程中顺便统计跨越逆序对。这正是我们所需要的。

因此，我们可以将逆序对计数与归并排序结合起来，在归并排序的递归过程中，除了排序外，额外统计逆序数。这样既完成了排序，又得到了逆序数，时间复杂度仍为 $O(n \log n)$ 。

4.5.5 伪代码

下面是结合归并排序过程的计算逆序数的分治递归算法：

```

算法 countInversions(A, low, high)
    // 输入：数组 A，以及需要计算的区间范围 [low, high]
    // 输出：区间 [low, high] 内的逆序对总数
    如果 low == high:
        返回 0
    mid = (low + high) / 2
    // 递归计算左半和右半的逆序数
    leftInv = countInversions(A, low, mid)
    rightInv = countInversions(A, mid+1, high)
    // 合并并统计跨越左右两半的逆序数
    crossInv = mergeAndCount(A, low, mid, high)
    返回 leftInv + rightInv + crossInv
  
```

mergeAndCount 算法计算跨越左右子序列的逆序数：

```

算法 mergeAndCount(A, low, mid, high)
    // 输入：数组 A，其中 A[low..mid] 和 A[mid+1..high] 已经分别按升序排序
    // 输出：将这两个有序子数组合并为一个有序数组（仍存回 A[low..high]），
    //       并返回跨越这两个子数组的逆序对总数
    创建临时数组 temp, 大小 = high - low + 1
    i = low           // 左子数组当前指针
    j = mid+1         // 右子数组当前指针
    k = 0             // temp 当前指针
    count = 0         // 跨越逆序计数

    while i <= mid 且 j <= high:
        if A[i] <= A[j]:
            temp[k] = A[i]
            i = i + 1
        else:
            // A[i] > A[j]，说明左子数组中从 i 到 mid 的所有元素都与 A[j] 构成逆序
            count = count + (mid - i + 1)
            temp[k] = A[j]
            j = j + 1
        k = k + 1

    // 复制剩余元素
    while i <= mid:
        temp[k] = A[i]; i = i + 1; k = k + 1
    while j <= high:
        temp[k] = A[j]; j = j + 1; k = k + 1
  
```



```

// 将 temp 复制回 A[low..high]
for p = 0 to k-1:
    A[low + p] = temp[p]

返回 count

```

4.5.6 代码实现

4.5.6.1 C++ 实现 (包含主函数)

```

#include <iostream>
#include <vector>
using namespace std;

// 合并两个有序子数组并统计跨越逆序对
long long mergeAndCount(vector<int>& A, int low, int mid, int high) {
    vector<int> temp(high - low + 1);
    int i = low, j = mid + 1, k = 0;
    long long count = 0;

    while (i <= mid && j <= high) {
        if (A[i] <= A[j]) {
            temp[k++] = A[i++];
        } else {
            count += (mid - i + 1);
            temp[k++] = A[j++];
        }
    }
    while (i <= mid) temp[k++] = A[i++];
    while (j <= high) temp[k++] = A[j++];

    for (int p = 0; p < k; ++p)
        A[low + p] = temp[p];

    return count;
}

// 分治递归计算逆序数
long long countInversions(vector<int>& A, int low, int high) {
    if (low >= high) return 0;
    int mid = low + (high - low) / 2;
    long long leftInv = countInversions(A, low, mid);

```

```

    long long rightInv = countInversions(A, mid + 1, high);
    long long crossInv = mergeAndCount(A, low, mid, high);
    return leftInv + rightInv + crossInv;
}

int main() {
    vector<int> arr = {5, 4, 6, 1, 3, 2};
    cout << " 原数组: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    long long invCount = countInversions(arr, 0, arr.size() - 1);

    cout << " 逆序对总数 = " << invCount << endl;
    cout << " 排序后的数组: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    return 0;
}

```

4.5.6.2 Python 实现 (包含主函数)

```

def merge_and_count(A, low, mid, high):
    """ 合并两个有序子数组并统计跨越逆序对 """
    temp = []
    i, j = low, mid + 1
    count = 0
    while i <= mid and j <= high:
        if A[i] <= A[j]:
            temp.append(A[i])
            i += 1
        else:
            count += (mid - i + 1)
            temp.append(A[j])
            j += 1
    temp.extend(A[i:mid+1])
    temp.extend(A[j:high+1])
    A[low:high+1] = temp
    return count

def count_inversions(A, low, high):

```

```

""" 分治递归计算逆序数"""
if low >= high:
    return 0
mid = (low + high) // 2
left_inv = count_inversions(A, low, mid)
right_inv = count_inversions(A, mid + 1, high)
cross_inv = merge_and_count(A, low, mid, high)
return left_inv + right_inv + cross_inv

if __name__ == "__main__":
    arr = [5, 4, 6, 1, 3, 2]
    print(" 原数组:", arr)
    inv_count = count_inversions(arr, 0, len(arr) - 1)
    print(" 逆序对总数 =", inv_count)
    print(" 排序后的数组:", arr)

```

4.5.7 时间复杂度分析

与归并排序相同，每次递归处理两个规模减半的子问题，合并及计数需要线性时间，因此递推关系为：

$$T(n) = 2T(n/2) + O(n),$$

所以 $T(n) = O(n \log n)$ 。

4.6 快速排序

4.6.1 问题描述

给定一个长度为 n 的数组，我们希望将它按从小到大（非降序）排序。

4.6.2 分治思路

快速排序是一种基于分治法的排序算法，它的核心思想可以用三个字概括：**分、治、合**。

- **分（划分）**：从数组中选择一个元素作为**基准**（pivot），然后重新排列数组，使得**所有小于基准的元素都放在它的左边，所有大于基准的元素都放在它的右边**。这一步称为**划分**（partition）。划分结束后，基准元素就已经处于它最终的正确位置上了（即排序后它应该在的位置）。
- **治（递归）**：递归地对基准左边和右边的子数组进行同样的快速排序。
- **合（合并）**：由于划分后左右两边已经分别有序，并且基准在中间，整个数组自然就有序了，不需要额外的合并操作。

快速排序之所以快，是因为每次划分都能将一个元素放到正确位置，而划分操作本身只需要线性时间 $O(n)$ 。

快速排序的核心是“**划分**”操作：选择一个基准元素（pivot），将数组重新排列，使得所有小于基准的元素都移到基准左边，大于基准的元素都移到右边。然后递归地对左右两部分进行快速排序。合并阶段无需额外工作，因为划分后数组

已局部有序。

4.6.3 划分过程详解 (Hoare 划分)

划分是快速排序的核心。我们采用一种经典的方法——**Hoare 划分**，它使用两个指针从数组的两端向中间移动，交换那些放错位置的元素。这种方法直观且高效。

思想：

- 选择最左边的元素作为基准（也可选其他位置，但为了方便讲解，我们选最左边）。
- 设置两个指针：i 从左端开始向右移动，j 从右端开始向左移动。
- 当 i 指向的元素小于基准时，说明它本来就应该在左边，于是 i 继续向右移动；当 i 指向的元素大于等于基准时，i 暂停。
- 同时，当 j 指向的元素大于基准时，说明它本来就应该在右边，于是 j 继续向左移动；当 j 指向的元素小于等于基准时，j 暂停。
- 一旦 i 和 j 都暂停，说明 A[i] 应该放在右边，而 A[j] 应该放在左边，于是交换它们。
- 然后继续移动，直到 i 和 j 相遇（即 $i \geq j$ ）。此时，将基准（最左边元素）与 A[j] 交换（因为 j 最终会停在小于等于基准的区域），基准就放到了正确位置。

4.6.3.1 例子：数组 [5, 2, 9, 1, 7, 6]，选第一个元素 5 为基准

我们用图示一步一步展示指针的移动和交换过程。两个指针 i 和 j 分别从左右两端向中间移动，交换错位的元素。

4.6.3.2 步骤 1

数组：[5, 2, 9, 1, 7, 6]

i=0 j=5

- 移动 i 向右，直到 $A[i] \geq 5$ ：i 停在 0（因为 $5 \geq 5$ ）。
- 移动 j 向左，直到 $A[j] \leq 5$ ：j 从 5 左移， $6 > 5$ ， $7 > 5$ ， $1 \leq 5$ ，j 停在 3。此时 $i=0 < j=3$ ，交换 A[0] 和 A[3]，得到：

数组：[1, 2, 9, 5, 7, 6]

i=0 j=3

4.6.3.3 步骤 2

数组：[1, 2, 9, 5, 7, 6]

i=2 j=3

- 继续移动 i 向右：从 0 开始， $1 < 5$ ， $2 < 5$ ， $9 \geq 5$ ，i 停在 2。
- 移动 j 向左：从 3 开始， $5 \leq 5$ ，j 停在 3（注意 $A[3]=5$ 满足条件）。此时 $i=2 < j=3$ ，交换 A[2] 和 A[3]，得到：

数组：[1, 2, 5, 9, 7, 6]

i=2 j=3

4.6.3.4 步骤 3

数组：[1, 2, 5, 9, 7, 6]

i=2 j=2

- 移动 i 向右：从 2 开始， $5 \geq 5$ ，i 停在 2。
- 移动 j 向左：从 3 开始， $9 > 5$ ，j 左移到 2， $5 \leq 5$ ，j 停在 2。此时 $i=2$ ， $j=2$ ， $i \geq j$ ，循环结束。返回分界点 $j=2$ 。

划分结果：左半部分 $[0..2] = [1, 2, 5]$ （全部 ≤ 5 ），右半部分 $[3..5] = [9, 7, 6]$ （全部 ≥ 5 ）。

Hoare 划分伪代码：

```
i = L-1
j = R+1
while True
    repeat i++ until A[i] >= pivot
    repeat j-- until A[j] <= pivot
    if i >= j then break
    swap A[i] and A[j]
swap A[L] and A[j]
return j
```

4.6.4 递归与完整算法

划分操作将数组分成两部分：左半部分 $A[L..j]$ 中的所有元素都 $\leq$ 基准，右半部分 $A[j+1..R]$ 中的所有元素都 $\geq$ 基准。此时，基准不一定在中间，但整个数组已经满足“左边小，右边大”的性质。接下来，我们只需要递归地对左半部分和右半部分分别进行快速排序，直到每个子数组的长度为 1 或 0（即 $L \geq R$ ），此时数组自然有序。

4.6.4.1 快速排序完整递归过程

以数组 [5, 2, 9, 1, 7, 6] 为例，展示每一次递归调用及其对数组的影响。我们用缩进表示递归深度，每个节点标明 `quickSort(L, R)` 以及该次调用处理的子数组内容和划分后的数组状态。注意数组是全局更新的，因此每个节点后数组的状态反映了所有已完成操作的结果。

初始数组

[5, 2, 9, 1, 7, 6]

递归树

```
quickSort(0,5) 处理 [5,2,9,1,7,6]
|   划分（基准5）→ j=2，数组变为 [1,2,5,9,7,6]
|
├─ quickSort(0,2) 处理左半 [1,2,5]
|   |   划分（基准1）→ j=0，数组保持 [1,2,5,9,7,6]
|   |
|   └─ quickSort(0,0) 处理 [1] → 直接返回
|   |
|   |
```

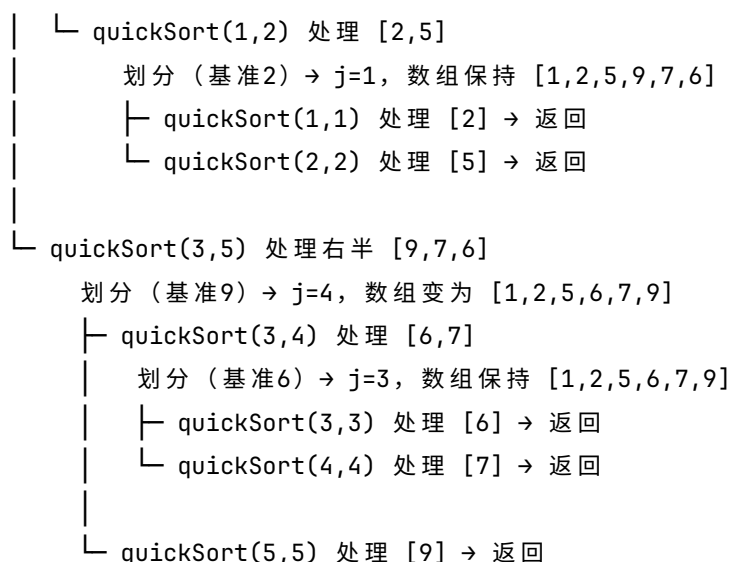


图 4.4 对数组 [5, 2, 9, 1, 7, 6] 快速排序的递归调用过程

递归过程说明

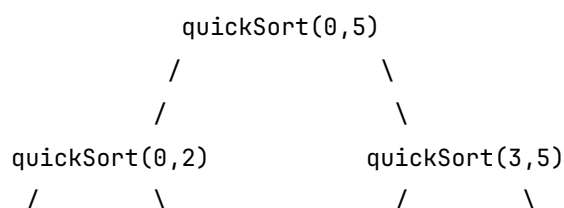
1. **顶层调用** quickSort(0,5) 对整个数组进行划分，基准为 5。划分后数组变为 [1,2,5,9,7,6]，分界点 j=2。接下来递归处理左半 [0,2] 和右半 [3,5]。
2. **左半递归** quickSort(0,2) 处理子数组 [1,2,5]（当前数组的这部分）。以 1 为基准划分后，分界点 j=0，数组不变。然后递归：
 - quickSort(0,0) 处理单元素 [1]，直接返回。
 - quickSort(1,2) 处理 [2,5]，以 2 为基准划分，分界点 j=1，数组不变。再递归处理两个单元素后返回。左半部分完全有序。
3. **右半递归** quickSort(3,5) 处理子数组 [9,7,6]。以 9 为基准划分后，数组变为 [1,2,5,6,7,9]，分界点 j=4。然后递归：
 - quickSort(3,4) 处理 [6,7]，以 6 为基准划分，分界点 j=3，数组不变。再递归处理两个单元素。
 - quickSort(5,5) 处理 [9]，直接返回。
4. 所有递归返回后，整个数组已有序：[1,2,5,6,7,9]。

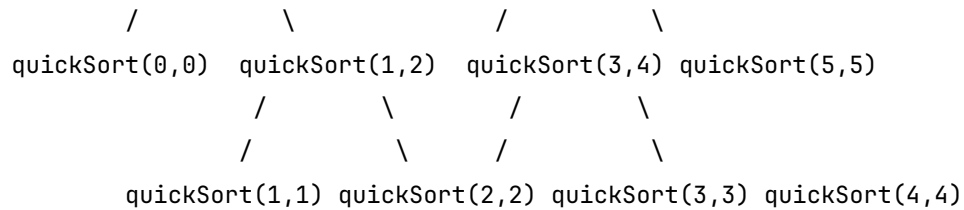
4.6.5 关键观察

- 每次划分将一个元素（基准）放到最终位置，但 Hoare 划分的基准不一定在分界点上，然而分界点保证了左右子数组的大小关系。
- 递归深度为数组长度的对数级（本例深度为 3），体现了分治法的效率。
- 数组的最终有序是通过逐层划分逐步实现的，每个子数组在递归返回时都已排好序。

递归过程示意图

可以用下面的递归树来直观展示整个流程（每个节点表示一次 quickSort 调用，括号内为区间）：





最终，所有叶子节点（长度为 1 的区间）返回，数组排序完成。

快速排序算法伪代码（完整版）

结合上述递归过程，完整的快速排序算法如下：

```

Algorithm quickSort(A, L, R)
  // 对数组 A 的区间 [L, R] 进行快速排序
  if L < R then
    // 划分，返回分界点 j
    j = partition(A, L, R)
    // 递归排序左半部分 [L, j]
    quickSort(A, L, j)
    // 递归排序右半部分 [j+1, R]
    quickSort(A, j+1, R)
  end if

Algorithm partition(A, L, R)
  // Hoare 划分，以 A[L] 为基准
  pivot = A[L]
  i = L - 1
  j = R + 1
  while true do
    repeat i = i + 1 until A[i] >= pivot
    repeat j = j - 1 until A[j] <= pivot
    if i >= j then return j
    swap A[i] and A[j]
  end while
  
```

4.6.6 关键点说明

- 递归的终止条件是 $L \geq R$ ，此时子数组已经有序（或为空）。
- 每次划分后，分界点 j 将数组分成两个非空的子数组（可能其中一个为空，但递归调用时会自然处理）。
- 快速排序是原地排序，不需要额外空间（除了递归栈）。

4.6.7 完整代码

4.6.7.1 C++ 实现

```
#include <iostream>
#include <vector>
using namespace std;

int partition(vector<int>& A, int L, int R) {
    int pivot = A[R];
    int i = L - 1;
    for (int j = L; j < R; j++) {
        if (A[j] <= pivot) {
            i++;
            swap(A[i], A[j]);
        }
    }
    swap(A[i + 1], A[R]);
    return i + 1;
}

void quickSort(vector<int>& A, int L, int R) {
    if (L < R) {
        int p = partition(A, L, R);
        quickSort(A, L, p - 1);
        quickSort(A, p + 1, R);
    }
}

int main() {
    vector<int> A = {70, 74, 60, 76, 83, 72, 55, 65, 79};
    quickSort(A, 0, A.size() - 1);
    for (int x : A) cout << x << " ";
    cout << endl;
    return 0;
}
```

4.6.7.2 Python 实现

```
def partition(A, L, R):
    pivot = A[R]
    i = L - 1
    for j in range(L, R):
```



```

        if A[j] <= pivot:
            i += 1
            A[i], A[j] = A[j], A[i]
    A[i + 1], A[R] = A[R], A[i + 1]
    return i + 1

def quickSort(A, L, R):
    if L < R:
        p = partition(A, L, R)
        quickSort(A, L, p - 1)
        quickSort(A, p + 1, R)

if __name__ == "__main__":
    A = [70, 74, 60, 76, 83, 72, 55, 65, 79]
    quickSort(A, 0, len(A) - 1)
    print(A)

```

4.6.8 4.6.8 时间复杂度分析

- **最好情况：**每次划分都均匀，递归深度 $\log n$ ，每层划分总时间 $O(n)$ ，故 $T(n) = 2T(n/2) + O(n) = O(n \log n)$ 。
- **最坏情况：**每次划分都只减少一个元素（如数组已经有序）， $T(n) = T(n - 1) + O(n) = O(n^2)$ 。
- **平均情况：**随机序列下，期望时间复杂度为 $O(n \log n)$ 。可以通过随机选择基准来避免最坏情况。

思考：如何实现随机化快速排序？只需在 `partition` 前随机选一个元素与 `A[R]` 交换即可。尝试修改代码。

4.7 大整数乘法（Karatsuba 算法）

4.7.1 问题的引出

在计算机中，两个 n 位整数的乘法是一项基本运算。我们从小就学会的竖式乘法需要逐位相乘再相加，其时间复杂度为 $O(n^2)$ 。例如，计算 1234×5678 需要 $4 \times 4 = 16$ 次位乘。当 n 很大时（比如上千位）， $O(n^2)$ 的算法就显得非常缓慢。那么，有没有更快的方法呢？

4.7.2 分治法的初步想法

我们可以尝试用分治法来加速。将两个 n 位整数 x 和 y 各分成两段：高位和低位。设 n 为偶数（如果不是，可以在前面补 0），令

$$x = 10^{n/2} \cdot a + b, \quad y = 10^{n/2} \cdot c + d$$

其中 a, b, c, d 都是 $n/2$ 位的整数。那么乘积可以写成：

$$x \cdot y = 10^n \cdot ac + 10^{n/2} \cdot (ad + bc) + bd$$

这个公式告诉我们：要计算 $x \cdot y$ ，我们需要计算 ac, ad, bc, bd 这四个乘积，再加上一些移位和加法操作。于是我们得到了一个分治算法：递归地计算这四个 $n/2$ 位乘法，然后合并。其时间复杂度的递推式为：

$$T(n) = 4T(n/2) + O(n)$$

解这个递推式，根据主定理， $a = 4, b = 2, \log_b a = 2$ ，而 $f(n) = n$ 是 $O(n^{2-1})$ ，因此 $T(n) = O(n^2)$ 。可见，这种直接分治并没有带来任何改进，仍然是 $O(n^2)$ 。

4.7.3 一个简单的例子

让我们用一个具体的两位数乘法来感受一下。假设 $x = 42, y = 37$ 。这里 $n = 2, n/2 = 1$ ，所以 $a = 4, b = 2, c = 3, d = 7$ 。按照公式：

- $ac = 4 \times 3 = 12$
 - $ad = 4 \times 7 = 28$
 - $bc = 2 \times 3 = 6$
 - $bd = 2 \times 7 = 14$ 那么 $x \cdot y = 10^2 \times 12 + 10^1 \times (28 + 6) + 14 = 1200 + 340 + 14 = 1554$ 。
- 确实，我们用了 4 次一位数乘法。这和平常的竖式乘法 ($4 \times 3, 4 \times 7, 2 \times 3, 2 \times 7$) 一样，没有节省。

4.7.4 Karatsuba 的巧妙观察

1960 年，年仅 23 岁的 Anatoly Karatsuba 发现了一个恒等式，可以将四次乘法减少到三次：

$$ad + bc = (a + b)(c + d) - ac - bd$$

这个等式为什么成立？展开右边：

$$(a + b)(c + d) = ac + ad + bc + bd$$

减去 ac 和 bd 后正好剩下 $ad + bc$ 。于是我们只需要计算三个乘积：

- $p = ac$
- $q = bd$
- $r = (a + b)(c + d)$ 然后 $ad + bc = r - p - q$ 。这样， $x \cdot y = 10^n \cdot p + 10^{n/2} \cdot (r - p - q) + q$ 。

仍然用刚才的例子 $x = 42, y = 37$ ：

- $a = 4, b = 2, c = 3, d = 7$
- $p = ac = 4 \times 3 = 12$
- $q = bd = 2 \times 7 = 14$
- $r = (a + b)(c + d) = (4 + 2) \times (3 + 7) = 6 \times 10 = 60$
- $ad + bc = r - p - q = 60 - 12 - 14 = 34$
- $x \cdot y = 10^2 \times 12 + 10^1 \times 34 + 14 = 1200 + 340 + 14 = 1554$ ，结果一致！

我们只用了 3 次乘法！对于两位数来说，节省一次乘法似乎微不足道，但当数字很大时，这种节省会被递归放大。

4.7.5 递归过程与复杂度分析

Karatsuba 算法递归地应用上述技巧。为了计算 ac , bd , 和 $(a+b)(c+d)$, 我们同样可以递归地调用 Karatsuba 算法 (因为 a, b, c, d 的位数大约是 $n/2$, 而 $a+b$ 和 $c+d$ 可能变成 $n/2+1$ 位, 但可以处理)。这样, 规模为 n 的问题被分解为 3 个规模约为 $n/2$ 的子问题, 加上一些加法和移位操作 ($O(n)$)。于是递推关系为:

$$T(n) = 3T(n/2) + O(n)$$

根据主定理, $a = 3, b = 2, \log_b a = \log_2 3 \approx 1.585$, 而 $f(n) = n$ 是 $O(n^{\log_2 3 - 1})$, 因此:

$$T(n) = O(n^{\log_2 3}) \approx O(n^{1.585})$$

这比 $O(n^2)$ 有了明显进步!

4.7.6 算法伪代码

下面给出 Karatsuba 算法的抽象描述。为了处理 n 为奇数的情况, 我们可以先将被乘数和乘数补零到偶数位, 或者直接递归处理。这里假设 n 是 2 的幂, 以便简化。

算法 karatsuba(x, y, n)

```
// 输入: 两个  $n$  位正整数  $x$  和  $y$  ( $n$  为 2 的幂)
// 输出:  $x * y$  的乘积
如果  $n == 1$ :
    返回  $x * y$ 
 $k = n / 2$ 
// 将  $x$  和  $y$  分成高位和低位
 $a = x$  的高  $k$  位 (即  $x // 10^k$ )
 $b = x$  的低  $k$  位 (即  $x \% 10^k$ )
 $c = y$  的高  $k$  位
 $d = y$  的低  $k$  位
// 递归计算三个乘积
 $p = \text{karatsuba}(a, c, k)$ 
 $q = \text{karatsuba}(b, d, k)$ 
 $r = \text{karatsuba}(a + b, c + d, k)$  // 注意  $a+b$  和  $c+d$  可能为  $k+1$  位
// 合并结果
返回  $p * 10^{2k} + (r - p - q) * 10^k + q$ 
```

注意: 实际实现中, $a+b$ 和 $c+d$ 可能产生进位, 导致位数增加, 但递归时仍可正常处理 (只需将位数参数调整为 $k+1$ 或采用动态位数)。在编程时, 通常使用字符串或数组表示大整数, 以处理任意长度。

4.7.7 代码实现

下面给出 C++ 和 Python 的简化实现。为了便于演示, 这里假设 n 是 2 的幂且两数位数相同, 用 long long 类型处理 (实际上 long long 只能处理有限位数, 但足以说明算法思想)。

C++ 实现

```

#include <iostream>
#include <cmath>
using namespace std;

// 返回 10^k
long long pow10(int k) {
    long long res = 1;
    while (k--) res *= 10;
    return res;
}

long long karatsuba(long long x, long long y, int n) {
    if (n == 1) return x * y;
    int k = n / 2;
    long long p = pow10(k);
    long long a = x / p;
    long long b = x % p;
    long long c = y / p;
    long long d = y % p;

    long long ac = karatsuba(a, c, k);
    long long bd = karatsuba(b, d, k);
    long long abcd = karatsuba(a + b, c + d, k);

    return ac * p * p + (abcd - ac - bd) * p + bd;
}

int main() {
    long long x = 4265, y = 5378;
    int n = 4; // 位数
    cout << x << " * " << y << " = " << karatsuba(x, y, n) << endl;
    return 0;
}

```

Python 实现

```

def karatsuba(x, y, n):
    if n == 1:
        return x * y
    k = n // 2
    p = 10 ** k
    a = x // p
    b = x % p

```

```

c = y // p
d = y % p
ac = karatsuba(a, c, k)
bd = karatsuba(b, d, k)
abcd = karatsuba(a + b, c + d, k)
return ac * p * p + (abcd - ac - bd) * p + bd

if __name__ == "__main__":
    x, y = 4265, 5378
    n = 4
    print(f"{x} * {y} = {karatsuba(x, y, n)}")

```

运行结果：

4265 * 5378 = 22932770

4.7.8 思考与拓展

1. 上述代码假设 n 是 2 的幂且两数位数相同。实际应用中，如何推广到任意位数的整数？提示：可以补零到最近 2 的幂，或者修改递归边界条件。
2. Karatsuba 算法虽然减少了乘法次数，但增加了加法和减法的次数，常数因子较大。对于小规模整数，普通乘法可能更快。在实际实现中，通常会设定一个阈值，当 n 小于该阈值时改用普通乘法。
3. 进一步的研究：Toom-Cook 算法将整数分成更多段，进一步降低复杂度；FFT（快速傅里叶变换）乘法可以达到 $O(n \log n)$ ，但常数更大，适合超大规模整数。

Karatsuba 算法的思想启发了后来许多分治算法，它的核心在于通过代数恒等式减少子问题个数，是算法设计中的经典技巧。

4.8 矩阵乘法（Strassen 算法）

4.8.1 问题的引出

矩阵乘法是科学计算和工程应用中最基本的操作之一。给定两个 $n \times n$ 矩阵 A 和 B ，计算它们的乘积 $C = A \times B$ ，其中

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

直接按照定义计算，每个 C_{ij} 需要 n 次乘法和 $n - 1$ 次加法，总共有 n^2 个元素，因此时间复杂度为 $O(n^3)$ 。当 n 很大时，例如 $n = 1000$ ，就需要十亿次运算，这显然很慢。那么，有没有更快的方法呢？

4.8.2 分治法的初步想法

我们可以尝试用分治法来加速。将矩阵按行和列分块，假设 n 是 2 的幂（如果不是，可以在外围补零），那么可以将 A 和 B 各分成四个 $\frac{n}{2} \times \frac{n}{2}$ 的子矩阵：

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

那么乘积 $C = A \times B$ 可以表示为：

$$C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$

这个公式告诉我们：要计算 C ，需要计算 8 个 $\frac{n}{2} \times \frac{n}{2}$ 矩阵的乘法 ($A_{11}B_{11}$ 、 $A_{12}B_{21}$ 等)，以及 4 次矩阵加法（加法的时间复杂度为 $O(n^2)$ ）。于是我们得到一个分治算法：递归地计算这 8 个子问题，然后合并。其时间复杂度的递推式为：

$$T(n) = 8T(n/2) + O(n^2)$$

解这个递推式，根据主定理， $a = 8, b = 2, \log_b a = 3$ ，而 $f(n) = n^2$ 是 $O(n^{3-1})$ ，因此 $T(n) = O(n^3)$ 。可见，这种直接分治并没有任何改进，仍然是 $O(n^3)$ 。

4.8.3 一个简单的例子

让我们用一个具体的 2×2 矩阵来感受一下。设

$$A = \begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix}, \quad B = \begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix}.$$

按照定义直接计算：

$$\begin{aligned} C &= \begin{pmatrix} 1 \cdot 6 + 3 \cdot 4 & 1 \cdot 8 + 3 \cdot 2 \\ 7 \cdot 6 + 5 \cdot 4 & 7 \cdot 8 + 5 \cdot 2 \end{pmatrix} \\ &= \begin{pmatrix} 6 + 12 & 8 + 6 \\ 42 + 20 & 56 + 10 \end{pmatrix} \\ &= \begin{pmatrix} 18 & 14 \\ 62 & 66 \end{pmatrix}. \end{aligned}$$

这里我们用了 8 次乘法和 4 次加法，这与递推式 $T(2) = 8T(1) + O(4)$ 一致，其中 $T(1)$ 表示一次乘法， $O(4)$ 对应加法次数。

4.8.4 Strassen 的巧妙观察

1969 年，德国数学家 Volker Strassen 发现了一种方法，可以将 8 次乘法减少到 7 次。他构造了 7 个中间矩阵：

$$\begin{aligned}
M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\
M_2 &= (A_{21} + A_{22})B_{11} \\
M_3 &= A_{11}(B_{12} - B_{22}) \\
M_4 &= A_{22}(B_{21} - B_{11}) \\
M_5 &= (A_{11} + A_{12})B_{22} \\
M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\
M_7 &= (A_{12} - A_{22})(B_{21} + B_{22})
\end{aligned}$$

然后通过加减运算得到 C 的四个子矩阵：

$$\begin{aligned}
C_{11} &= M_1 + M_4 - M_5 + M_7 \\
C_{12} &= M_3 + M_5 \\
C_{21} &= M_2 + M_4 \\
C_{22} &= M_1 - M_2 + M_3 + M_6
\end{aligned}$$

以 2×2 矩阵为例：- $A_{11} = 1, A_{12} = 3, A_{21} = 7, A_{22} = 5 - B_{11} = 6, B_{12} = 8, B_{21} = 4, B_{22} = 2$

计算七个中间矩阵：

$$\begin{aligned}
M_1 &= (1 + 5)(6 + 2) = 6 \times 8 = 48 \\
M_2 &= (7 + 5) \times 6 = 12 \times 6 = 72 \\
M_3 &= 1 \times (8 - 2) = 1 \times 6 = 6 \\
M_4 &= 5 \times (4 - 6) = 5 \times (-2) = -10 \\
M_5 &= (1 + 3) \times 2 = 4 \times 2 = 8 \\
M_6 &= (7 - 1) \times (6 + 8) = 6 \times 14 = 84 \\
M_7 &= (3 - 5) \times (4 + 2) = (-2) \times 6 = -12
\end{aligned}$$

然后组合：

- $C_{11} = 48 + (-10) - 8 + (-12) = 18$
- $C_{12} = 6 + 8 = 14$
- $C_{21} = 72 + (-10) = 62$
- $C_{22} = 48 - 72 + 6 + 84 = 66$

结果与直接计算完全一致！我们只用了 7 次乘法，而不是 8 次。虽然对于 2×2 矩阵，节省一次乘法似乎微不足道，但当矩阵规模很大时，这种节省会被递归放大。

4.8.5 递归过程与复杂度分析

Strassen 算法递归地应用上述技巧。为了计算 M_1 到 M_7 ，每个 M_i 都是两个 $\frac{n}{2} \times \frac{n}{2}$ 子矩阵的乘积，因此可以递归调用 Strassen 算法。此外，还需要进行一些矩阵加法和减法（总共约 18 次加减法），这些操作的时间复杂度为 $O(n^2)$ 。于是，规模为 n 的问题被分解为 7 个规模为 $n/2$ 的子问题，加上 $O(n^2)$ 的合并开销。递推关系为：

$$T(n) = 7T(n/2) + O(n^2)$$

根据主定理, $a = 7, b = 2, \log_b a = \log_2 7 \approx 2.807$, 而 $f(n) = n^2$ 是 $O(n^{2.807-1})$, 因此:

$$T(n) = O(n^{\log_2 7}) \approx O(n^{2.807})$$

这比 $O(n^3)$ 有了显著进步! 当 n 很大时, 例如 $n = 1000$, $n^3 = 10^9$, 而 $n^{2.807} \approx 10^{8.42} \approx 2.6 \times 10^8$, 速度快了约 4 倍。随着 n 增大, 优势更加明显。

4.8.5.1 算法伪代码

下面给出 Strassen 算法的抽象描述。为了简化, 假设矩阵大小 n 是 2 的幂。如果不是, 可以在矩阵外围补零, 使其变为 2 的幂。

```

算法 strassen(A, B, n)
    // 输入: 两个 n×n 矩阵 A 和 B (n 为 2 的幂)
    // 输出: A × B 的结果矩阵 C
    如果 n == 1:
        返回 [[A[0][0] * B[0][0]]]
    // 将 A 和 B 分成四个子矩阵
    k = n / 2
    令 A11, A12, A21, A22 分别为 A 的左上、右上、左下、右下 k×k 子矩阵
    令 B11, B12, B21, B22 分别为 B 的左上、右上、左下、右下 k×k 子矩阵
    // 递归计算 7 个中间矩阵
    M1 = strassen(A11 + A22, B11 + B22, k)
    M2 = strassen(A21 + A22, B11, k)
    M3 = strassen(A11, B12 - B22, k)
    M4 = strassen(A22, B21 - B11, k)
    M5 = strassen(A11 + A12, B22, k)
    M6 = strassen(A21 - A11, B11 + B12, k)
    M7 = strassen(A12 - A22, B21 + B22, k)
    // 计算 C 的四个子矩阵
    C11 = M1 + M4 - M5 + M7
    C12 = M3 + M5
    C21 = M2 + M4
    C22 = M1 - M2 + M3 + M6
    // 将 C11, C12, C21, C22 组合成 C 并返回
    C = 将 C11, C12, C21, C22 按照对应位置组合成 n×n 矩阵
    返回 C

```

注意: 算法中的矩阵加法和减法都是对应元素的加减, 时间复杂度 $O(k^2)$ 。

4.8.6 代码实现

下面给出 C++ 和 Python 的简化实现。为了便于演示，我们假设矩阵大小为 2 的幂，用二维 vector 或列表表示。由于 Strassen 算法涉及大量矩阵复制，代码会稍长，但核心逻辑清晰。

C++ 实现

```
#include <iostream>
#include <vector>
using namespace std;

typedef vector<vector<int>> Matrix;

// 矩阵加法
Matrix add(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<int>(n));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            C[i][j] = A[i][j] + B[i][j];
    return C;
}

// 矩阵减法
Matrix sub(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<int>(n));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            C[i][j] = A[i][j] - B[i][j];
    return C;
}

// Strassen 乘法
Matrix strassen(const Matrix& A, const Matrix& B, int n) {
    if (n == 1) {
        Matrix C(1, vector<int>(1));
        C[0][0] = A[0][0] * B[0][0];
        return C;
    }
    int k = n / 2;
    // 分割子矩阵
    Matrix A11(k, vector<int>(k)), A12(k, vector<int>(k)),
        A21(k, vector<int>(k)), A22(k, vector<int>(k));
```

```

Matrix B11(k, vector<int>(k)), B12(k, vector<int>(k)),
      B21(k, vector<int>(k)), B22(k, vector<int>(k));
for (int i = 0; i < k; ++i) {
    for (int j = 0; j < k; ++j) {
        A11[i][j] = A[i][j];
        A12[i][j] = A[i][j + k];
        A21[i][j] = A[i + k][j];
        A22[i][j] = A[i + k][j + k];
        B11[i][j] = B[i][j];
        B12[i][j] = B[i][j + k];
        B21[i][j] = B[i + k][j];
        B22[i][j] = B[i + k][j + k];
    }
}

// 计算 7 个中间矩阵
Matrix M1 = strassen(add(A11, A22), add(B11, B22), k);
Matrix M2 = strassen(add(A21, A22), B11, k);
Matrix M3 = strassen(A11, sub(B12, B22), k);
Matrix M4 = strassen(A22, sub(B21, B11), k);
Matrix M5 = strassen(add(A11, A12), B22, k);
Matrix M6 = strassen(sub(A21, A11), add(B11, B12), k);
Matrix M7 = strassen(sub(A12, A22), add(B21, B22), k);
// 计算 C 的子矩阵
Matrix C11 = add(sub(add(M1, M4), M5), M7);
Matrix C12 = add(M3, M5);
Matrix C21 = add(M2, M4);
Matrix C22 = add(sub(add(M1, M3), M2), M6);
// 组合成 C
Matrix C(n, vector<int>(n));
for (int i = 0; i < k; ++i) {
    for (int j = 0; j < k; ++j) {
        C[i][j] = C11[i][j];
        C[i][j + k] = C12[i][j];
        C[i + k][j] = C21[i][j];
        C[i + k][j + k] = C22[i][j];
    }
}

return C;
}

// 打印矩阵
void printMatrix(const Matrix& M) {

```

```

    for (const auto& row : M) {
        for (int x : row) cout << x << "\t";
        cout << endl;
    }
}

int main() {
    Matrix A = {{1, 3}, {7, 5}};
    Matrix B = {{6, 8}, {4, 2}};
    Matrix C = strassen(A, B, 2);
    cout << " 结果矩阵 C = " << endl;
    printMatrix(C);
    return 0;
}

```

Python 实现

```

import numpy as np  # 仅用于对比，这里我们手动实现

def add(A, B):
    n = len(A)
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]

def sub(A, B):
    n = len(A)
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]

def strassen(A, B, n):
    if n == 1:
        return [[A[0][0] * B[0][0]]]
    k = n // 2
    # 分割子矩阵
    A11 = [[A[i][j] for j in range(k)] for i in range(k)]
    A12 = [[A[i][j+k] for j in range(k)] for i in range(k)]
    A21 = [[A[i+k][j] for j in range(k)] for i in range(k)]
    A22 = [[A[i+k][j+k] for j in range(k)] for i in range(k)]
    B11 = [[B[i][j] for j in range(k)] for i in range(k)]
    B12 = [[B[i][j+k] for j in range(k)] for i in range(k)]
    B21 = [[B[i+k][j] for j in range(k)] for i in range(k)]
    B22 = [[B[i+k][j+k] for j in range(k)] for i in range(k)]

    # 计算 7 个中间矩阵
    M1 = strassen(add(A11, A22), add(B11, B22), k)

```

```

M2 = strassen(add(A21, A22), B11, k)
M3 = strassen(A11, sub(B12, B22), k)
M4 = strassen(A22, sub(B21, B11), k)
M5 = strassen(add(A11, A12), B22, k)
M6 = strassen(sub(A21, A11), add(B11, B12), k)
M7 = strassen(sub(A12, A22), add(B21, B22), k)

# 计算 C 的子矩阵
C11 = add(sub(add(M1, M4), M5), M7)
C12 = add(M3, M5)
C21 = add(M2, M4)
C22 = add(sub(add(M1, M3), M2), M6)

# 组合
C = [[0]*n for _ in range(n)]
for i in range(k):
    for j in range(k):
        C[i][j] = C11[i][j]
        C[i][j+k] = C12[i][j]
        C[i+k][j] = C21[i][j]
        C[i+k][j+k] = C22[i][j]
    return C

# 测试
A = [[1, 3], [7, 5]]
B = [[6, 8], [4, 2]]
C = strassen(A, B, 2)
print(" 结果矩阵 C = ")
for row in C:
    print(row)

```

运行结果与之前一致：

```

结果矩阵 C =
[18, 14]
[62, 66]

```

4.8.7 思考与拓展

1. **实际应用**：Strassen 算法虽然减少了乘法次数，但增加了大量的加法操作，而且递归过程中矩阵复制的开销也不小。因此，对于小规模矩阵（比如 $n < 100$ ），普通乘法可能更快。在实际实现中，通常会设定一个阈值，当 n 小于阈值时改用普通乘法。
2. **矩阵大小的处理**：如果矩阵大小不是 2 的幂，可以补零到最近的 2 的幂，或者采用更复杂的动态分块方法。
3. **后续改进**：Strassen 算法之后，人们发现了更优的矩阵乘法算法，如 Coppersmith–Winograd 算法，其时

间复杂度可低至 $O(n^{2.376})$ ，但常数极大，只适用于天文数字级的矩阵。目前实际中广泛使用的仍是优化后的 Strassen 或其变种。

Strassen 算法的思想——通过代数恒等式减少子问题个数——是算法设计中的经典范例，它启发了许多其他领域的优化方法。掌握这种思想，对理解分治法和算法优化大有裨益。

4.9 选择最小与第二小元素：从特例到一般

在正式学习一般选择问题（第 k 小元素）之前，我们先考察两个最简单的特例：**最小值**和**第二小值**。这些特例不仅可以帮助我们建立对选择问题的直觉，而且它们的最优算法——锦标赛算法——完美体现了“复用比较信息”的思想，为后续学习快速选择、堆排序等高级算法奠定基础。

4.9.1 问题引入

假设有一个包含 n 个可比较元素的无序数组 $A[1..n]$ 。我们可能遇到以下两类查询：

- **最小值**：找出数组中最小的元素。
- **第二小值**：找出数组中严格大于最小值的元素中的最小值（若所有元素相等则不存在）。

直觉上，最小值可以用一次线性扫描找到，而第二小值似乎只需要再扫描一次就能得到。但这是否是最优的？能否利用第一次扫描的信息来减少第二次扫描的工作量？本章将从这两个基础问题出发，引出一种优雅的算法——**锦标赛算法**，它达到了比较次数的理论下界。

4.9.2 最小值：线性扫描

4.9.2.1 算法描述

最小值问题有一种非常直接的算法：将第一个元素设为当前最小值，然后遍历剩余元素，每遇到更小的元素就更新它。

```
def find_min(A):
    min_val = A[0]
    for i in range(1, len(A)):
        if A[i] < min_val:
            min_val = A[i]
    return min_val
```

比较次数：循环执行 $n - 1$ 次，每次执行一次 $<$ 比较，故总比较次数为 $n - 1$ 。

4.9.2.2 最优性证明

定理 4.1：在比较模型下，任何确定性算法在最坏情况下至少需要 $n - 1$ 次比较才能找到 n 个元素中的最小值。

证明：将每次比较视为一场“比赛”，败者被淘汰。最终的最小值必须“战胜”其他 $n - 1$ 个元素（直接或间接）。每次比较最多产生一个败者，因此至少需要 $n - 1$ 次比较才能淘汰 $n - 1$ 个元素。线性扫描算法恰好进行了 $n - 1$ 次比较，故已达到下界。□

4.9.3 第二小：朴素两遍扫描

最简单直接的想法是：先找到最小值 m_1 ，然后在除 m_1 之外的其余元素中再找一次最小值，得到的就是第二小值 m_2 。

```
def find_second_naive(A):
    # 第一遍找最小值
    min1 = A[0]
    for i in range(1, len(A)):
        if A[i] < min1:
            min1 = A[i]

    # 第二遍找除 min1 之外的最小值
    min2 = float('inf')
    for i in range(len(A)):
        if A[i] != min1 and A[i] < min2:
            min2 = A[i]

    return min2
```

比较次数分析：- 第一遍： $n - 1$ 次比较。- 第二遍：跳过最小值位置，对剩下的 $n - 1$ 个元素扫描，需要 $n - 2$ 次比较（因为每次与当前 min2 比较）。- 总计： $(n - 1) + (n - 2) = 2n - 3$ 。

这是否已经足够好？我们能否做得更少？答案是可以，通过利用第一遍比较中隐藏的信息。

4.9.4 锦标赛算法：最优解法

4.9.4.1 核心思想

锦标赛算法（Tournament Method）模拟了体育淘汰赛的过程：

1. 将数组元素作为参赛者，两两比较，胜者（较小者）晋级下一轮。
2. 记录每个胜者所“击败”的对手（即与之直接比较过的败者）。
3. 最终决出全局最小值 m_1 。
4. **关键观察：**第二小的元素 m_2 一定出现在与 m_1 直接比较过的那些对手之中。
 - 理由：任何没有与 m_1 直接比较过的元素 x ，必然被某个非 m_1 的元素 y 所击败（或间接大于 y ），而 $y \geq m_2$ ，所以 x 不可能成为第二小。
5. 因此，我们只需在这些对手中找出最小值，即为 m_2 。

4.9.4.2 算法步骤

1. **构建胜者树：**
 - 将 n 个元素作为叶子节点。
 - 从底层开始两两比较，较小者进入父节点，同时记录该父节点“击败”了谁（即本轮比较的败者）。
 - 重复直到产生根节点，根节点即为全局最小值。此阶段需 $n - 1$ 次比较。
2. **收集候选者：**
 - 从根节点出发，沿最小值晋级路径向下，收集所有与最小值直接比较过的对手（即每个内部节点中记录的被击败者）。这些对手的数量等于最小值在树中的深度，即 $\lceil \log_2 n \rceil$ 。
3. **在候选者中找最小值：**
 - 对候选者进行线性扫描，找出最小值。需 $\lceil \log_2 n \rceil - 1$ 次比较。

总比较次数:

$$(n-1) + (\lceil \log_2 n \rceil - 1) = n + \lceil \log_2 n \rceil - 2.$$

4.9.4.3 实例演示

考虑数组 $[5, 2, 8, 3, 9, 1, 4, 7]$ ($n = 8$)。如 @fig-winner_tree_for_second 所示:

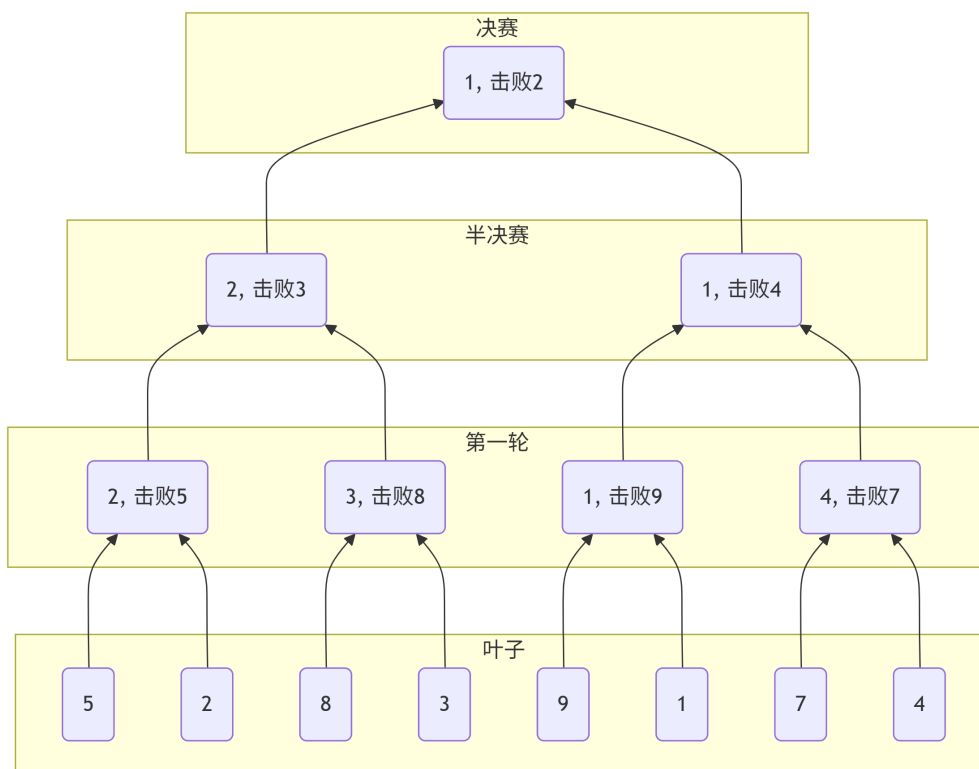


Figure 4.3: 一个 8 元素数组的胜者树: 全局最小值 1 及其直接比较过的对手 (2、4、9)

- **第一轮:** $5 \text{ vs } 2 \rightarrow$ 胜者 2 (击败 5); $8 \text{ vs } 3 \rightarrow$ 胜者 3 (击败 8); $9 \text{ vs } 1 \rightarrow$ 胜者 1 (击败 9); $4 \text{ vs } 7 \rightarrow$ 胜者 4 (击败 7)。
- **第二轮:** $2 \text{ vs } 3 \rightarrow$ 胜者 2 (击败 3); $1 \text{ vs } 4 \rightarrow$ 胜者 1 (击败 4)。
- **决赛:** $2 \text{ vs } 1 \rightarrow$ 胜者 1 (击败 2)。

全局最小值 = 1。与 1 直接比较过的对手有: 2 (决赛)、4 (半决赛)、9 (第一轮) $\rightarrow$ 候选集 $\{2, 4, 9\}$ 。在候选集中找最小值 $\rightarrow 2$, 即为第二小元素。

4.9.4.4 伪代码

为便于实现, 我们通常不需要显式构建树, 而是采用数组模拟。以下伪代码返回最小值和第二小值 (假设元素互异):

```
def find_min_and_second(A):
    n = len(A)
    # 每个元素包装成 (值, 被击败者列表)
    # 初始时每个叶子节点的被击败者列表为空
    # 为了方便, 直接构建一个代表胜者树的双层数组
```

```

# 更简单的实现：使用一个队列进行两两比较，记录每个胜者击败的对手

# 使用链表或列表记录每一轮的胜者及他们的“手下败将”
# 这里给出一种直观的队列实现
from collections import deque

# 初始：每个元素作为一个“节点”，包含值和一个空列表（记录击败的对手）
nodes = [(val, []) for val in A]
q = deque(nodes)

while len(q) > 1:
    # 取出两个节点进行比赛
    node1 = q.popleft()
    node2 = q.popleft()
    val1, losers1 = node1
    val2, losers2 = node2
    if val1 < val2:
        winner_val = val1
        # 胜者将败者加入自己的击败列表，并继承败者的击败列表？不，我们只需要直接击败的对手
        # 注意：锦标赛算法只要求记录直接击败的对手，不需要传递。
        winner_losers = losers1 + [val2]    # 新击败的对手
    else:
        winner_val = val2
        winner_losers = losers2 + [val1]
    q.append((winner_val, winner_losers))

# 最后剩下的就是全局最小值及其直接击败的对手列表
min_val, opponents = q[0]
# 在对手中找最小值
second_min = min(opponents)    # 对手列表非空，因为 n>=2
return min_val, second_min

```

注意：上述实现中，每个节点的 losers 列表只记录了该节点在**当前比赛**中直接击败的对手，而没有传递祖先的对手。实际上，当胜者晋级后，它之前的对手仍然保留在自己的列表中。在最终根节点，其 losers 列表包含了所有与它直接比赛过的对手（即它获胜的每一轮中的败者），这正是我们需要收集的候选集。此实现的空间复杂度为 $O(n \log n)$ （每个元素最多被击败一次且被记录一次）。实际可以采用更高效的数据结构（如用数组存储树），但上述代码清晰表达了思想。

4.9.4.5 复杂度分析

- **时间**：构建胜者树需要 $n - 1$ 次比较；在候选集中找最小值需要 $\lceil \log_2 n \rceil - 1$ 次比较。总比较次数为 $n + \lceil \log_2 n \rceil - 2$ 。
- **空间**：需要存储树结构，通常为 $O(n)$ （或 $O(n \log n)$ 若显式记录所有对手列表）。可以优化到 $O(n)$ 空间。

4.9.4.6 最优性证明

定理 4.2: 在比较模型下, 任何确定性算法在最坏情况下至少需要 $n + \lceil \log_2 n \rceil - 2$ 次比较才能找出 n 个元素中的最小值和第二小值。

证明: 任何算法必须首先确定最小值 m_1 , 这至少需要 $n - 1$ 次比较 (定理 4.1)。设与 m_1 直接比较过的元素集合为 S , $|S| = t$ 。为了确定第二小值, 必须找出 S 中的最小值, 这至少需要 $t - 1$ 次比较 (因为 S 中除 m_1 外还有 t 个元素? 注意 m_1 自己也在比较中, 但它是胜者, 所以 S 实际包含的是那些被 m_1 击败的对手, 数量为 t 。要找出这些对手的最小值, 需要 $t - 1$ 次比较)。因此总比较次数至少为 $(n - 1) + (t - 1) = n + t - 2$ 。

接下来证明 $t \geq \lceil \log_2 n \rceil$ 。考虑胜者树中从根到 m_1 叶子的路径。每个内部节点代表 m_1 参与的一次比较, 其对手即为该轮被 m_1 击败的元素。由于 m_1 每晋级一轮, 它必须击败一个对手, 而它的对手数量等于它获得冠军所需要的比赛场数。在一棵完全二叉树中, 有 n 个叶子, 树的高度为 $\lceil \log_2 n \rceil$ (从叶子到根经过的边数)。 m_1 从叶子上升到根需要经过 $\lceil \log_2 n \rceil$ 场比赛, 因此它击败了 $\lceil \log_2 n \rceil$ 个不同的对手, 故 $t = \lceil \log_2 n \rceil$ (在最坏情况下, m_1 需要打满所有轮次)。因此下界为 $n + \lceil \log_2 n \rceil - 2$ 。

锦标赛算法达到了此下界, 因而在比较次数意义下是最优的。□

4.9.5 分治法求解第二小

除了锦标赛算法, 也可以使用分治法递归地求解第二小元素:

- **分解:** 将数组分成左右两半, 分别递归求出各半的最小值和第二小值。
- **合并:** 设左半结果为 $(\min L, \text{sec} L)$, 右半结果为 $(\min R, \text{sec} R)$ 。全局最小值为 $\min(\min L, \min R)$ 。假设 $\min L < \min R$, 则全局第二小值候选为 $\{\text{sec} L, \min R\}$ 中的较小者。若 $\min R < \min L$ 同理。合并需要 3 次比较。
- **递归式:** $T(n) = 2T(n/2) + 3$, 解得 $T(n) = \frac{3n}{2} - 2$ (n 为 2 的幂)。比较次数比锦标赛算法多, 但仍是线性, 且思想简单, 适合教学。

4.9.6 小结

- 找最小值有简单的线性扫描算法, 且已经达到比较次数下界 $n - 1$ 。
- 找第二小元素的朴素两遍扫描需 $2n - 3$ 次比较, 但通过复用比较信息, 锦标赛算法将比较次数降低到 $n + \lceil \log_2 n \rceil - 2$, 并证明是最优的。
- 锦标赛算法的核心是**记录胜者击败过的对手**, 这使得第二小元素可以被高效定位。
- 锦标赛思想可以推广到同时找最大最小值 (配对法) 以及更一般的选择问题 (如败者树)。

4.9.7 习题

1. 实现锦标赛算法 (可选用数组模拟树结构), 测试在 $n = 10$ 和 $n = 100$ 的随机数组上, 统计实际比较次数, 并与理论值比较。
2. 证明: 如果数组所有元素相等, 则第二小元素不存在。此时锦标赛算法应如何处理? 修改算法使其返回特殊值 (如 None)。
3. 假设 n 不是 2 的幂, 如何构建胜者树? 这时候候选者个数会是多少?
4. 设计一个算法, 用最少的比较次数同时找出最小值和最大值。写出伪代码并分析比较次数。
5. **思考题:** 能否使用锦标赛算法的思想来找出第三小元素? 如果可以, 需要多少比较次数? 如果不能, 请说明理由。

4.10 选择问题（第 k 小元素）和快速选择算法

4.10.1 问题提出

想象一下，你是老师，批改完全班 50 人的试卷后，只想知道排名第 10 的分数是多少。最直接的办法是把所有分数从低到高排个序，然后看第 10 个——但排序需要把全班人的分数都排一遍，显然做了很多无用功。我们真正关心的只是一个特定位置的元素，并不需要完整的排序结果。

这就是经典的**选择问题**：给定一个包含 n 个元素的数组 A 和一个整数 k ($1 \leq k \leq n$)，要求找出数组中第 k 小的元素（即从小到大排在第 k 位的那个数）。例如，数组 $[9, 3, 7, 1, 6, 2, 8]$ 的第 3 小元素是 3。

有没有一种算法能**不用完全排序**，就能高效地找到第 k 小的元素呢？答案是肯定的，这就是本节要介绍的**快速选择算法**，它是分治策略的一个巧妙应用。

4.10.2 直观例子：分治递归思想的体现

快速选择的核心思想可以用一个生活中的例子来理解。

假设你有一堆身高不同的人站成一排，你想找出个子第二矮的人。你可以随便拉一个人出来当“标杆”，然后让所有比他矮的人站到他的左边，比他高的人站到他的右边。这样一来，你就知道了这个标杆在所有人中的排名——比如他左边有 3 个人，那么他就是第 4 矮的。

现在，你想找的是第 2 矮的人，而标杆是第 4 矮的，那么第 2 矮的人一定在**左边的那堆人里**（因为左边的人都比标杆矮）。于是，你只需要把注意力集中在左边的那堆人里，用同样的方法继续找第 2 矮的人。如果标杆恰好是第 2 矮的，那你就直接找到了答案；如果标杆比目标矮（比如他是第 1 矮），那么目标就在右边的那堆人里。

这个过程完美体现了**分治递归**的思想：

- **分解**：通过一次划分，将原问题分解成两个子问题（左边集合和右边集合），但**我们只需要处理其中一个子问题**，因为目标元素只可能存在于一侧。
- **解决**：递归地在选定的子问题中继续寻找第 k 小的元素。
- **合并**：由于我们只关心一个元素，不需要合并操作，直接返回找到的元素即可。

与快速排序不同，快速排序在划分后需要递归处理**两边**，而快速选择每次只处理一边，因此平均情况下效率更高。

4.10.3 核心思想（图示理解）

快速选择的核心操作是**划分**（Partition），和快速排序中的划分完全相同。划分的过程如下：

1. 从当前数组中选一个元素作为基准（pivot）。
2. 将数组重新排列，使得所有小于基准的元素放在它左边，所有大于基准的元素放在它右边，基准放在中间（相等元素可以放在任意一边，为简化，我们假设元素互异）。
3. 划分完成后，基准的位置就是它在最终有序数组中的位置，记作 p (p 从 1 开始计数，表示它是第 p 小的元素)。

例如，数组 $A = [9, 3, 7, 1, 6, 2, 8]$ ，选择基准为 6，划分后可能得到：

$$[3, 1, 2] \quad 6 \quad [9, 7, 8]$$

此时，基准 6 左边有 3 个数，所以它是第 4 小的元素，即 $p = 4$ 。

现在，假设我们要找第 k 小的元素。比较 k 与 p ：

- 如果 $k = p$ ，那么基准就是我们要找的元素，直接返回。
- 如果 $k < p$ ，说明目标比基准小，一定在左边的子数组中，我们只需递归地在左边寻找第 k 小的元素。
- 如果 $k > p$ ，说明目标比基准大，一定在右边的子数组中，我们只需递归地在右边寻找第 k 小的元素（注意，在右边子数组中，元素的全局排名需要减去左边元素的个数，但如果我们用下标从 0 开始的方式，递归时保持 k 不变，只需调整搜索范围即可）。

4.10.4 算法描述

快速选择算法的步骤如下（假设数组下标从 0 开始，且要找第 k 小的元素， k 也是从 0 开始计数的，即第 0 小是最小元素）：

1. 如果当前区间 $[\text{left}, \text{right}]$ 只有一个元素（即 $\text{left} = \text{right}$ ），则返回该元素。
2. 调用划分函数 `partition`，将区间 $[\text{left}, \text{right}]$ 重新排列，并返回基准元素最终的下标 p 。
3. 比较 k 与 p ：
 - 如果 $k = p$ ，则 $A[p]$ 就是第 k 小的元素，返回它。
 - 如果 $k < p$ ，则第 k 小的元素在左半部分，递归地在 $[\text{left}, p-1]$ 中寻找。
 - 如果 $k > p$ ，则第 k 小的元素在右半部分，递归地在 $[p+1, \text{right}]$ 中寻找（注意 k 保持不变，因为右半部分的元素在全局中的下标都大于 p ）。

4.10.5 伪代码

快速选择主算法：

```
function quickSelect(A, left, right, k)
    // left, right: 当前搜索范围的左右边界（包含）
    // k: 要找的第 k 小元素的下标（0-based）
    if left == right
        return A[left]

    pivotIndex = partition(A, left, right)    // 划分，返回基准的最终下标
    if k == pivotIndex
        return A[k]
    else if k < pivotIndex
        return quickSelect(A, left, pivotIndex - 1, k)
    else
        return quickSelect(A, pivotIndex + 1, right, k)
```

划分函数：

```
function partition(A, left, right)
    pivot = A[right]                // 选择最右边的元素作为基准
    i = left                        // i 指向小于基准的区域的末尾
    for j from left to right-1
        if A[j] <= pivot
            swap A[i] and A[j]
```

```

        i = i + 1
    swap A[i] and A[right]    // 将基准放到正确位置
    return i

```

注意：这里使用的划分是 Lomuto 方案，简单易懂，但效率稍低。也可以使用 Hoare 划分，效果类似。

4.10.6 Quickselect 正确性证明

命题 P(n): 对于长度为 n 的数组 $A[l..r]$ 且 $1 \leq k \leq n$, $\text{Quickselect}(A, l, r, k)$ 能够正确返回数组中第 k 小元素。

证明：

1. 基本情况 (Base Case)

当 $n = 1$ 时，数组只有一个元素 $A[l] = A[r]$ 。

- 由于 $1 \leq k \leq n = 1$ ，此时 $k = 1$ 。
- 算法直接返回 $A[l]$ ，显然返回正确。

因此，命题在 $n = 1$ 时成立。

2. 归纳假设 (Inductive Hypothesis)

假设命题在 **长度小于 n** 的数组上成立：

对任意长度 $m < n$ 的数组 B 且 $1 \leq k \leq m$, $\text{Quickselect}(B, l', r', k)$ 能正确返回第 k 小元素。

3. 归纳步骤 (Inductive Step)

考虑长度为 n 的数组 $A[l..r]$ ，执行一次 partition 操作：

1. 选择 pivot，记为 $A[p]$
2. 执行 partition，使得：

$$A[l..p-1] < A[p] < A[p+1..r]$$

其中 pivot 的最终位置为 p 。

设 pivot 的排名（在当前子数组中的序号）为：

$$\text{rank} = p - l + 1$$

情况 1: $k = \text{rank}$

- 第 k 小元素就是 pivot
- 算法直接返回 $A[p]$
- 正确

情况 2: $k < \text{rank}$

- 第 k 小元素在左侧子数组 $A[l..p-1]$
- 左侧子数组长度 $= p - l < n$
- 根据归纳假设，递归调用

$$\text{Quickselect}(A, l, p-1, k)$$

能够正确返回第 k 小元素。

因此算法正确。

情况 3: $k > \text{rank}$

- 第 k 小元素在右侧子数组 $A[p+1..r]$
- 右侧子数组长度 $= r - p < n$
- 递归调用

$$\text{Quickselect}(A, p+1, r, k - \text{rank})$$

能够正确返回右侧子数组中第 $k - \text{rank}$ 小元素，即整个数组的第 k 小元素。

因此算法正确。

4. 结论

- 三种情况覆盖了 $1 \leq k \leq n$
- 每次递归都调用长度小于 n 的子数组
- 基于归纳假设，递归返回正确结果

因此根据数学归纳法，命题 $P(n)$ 对所有 $n \geq 1$ 且 $1 \leq k \leq n$ 成立。

4.10.7 复杂度分析

快速选择的性能取决于划分的平衡程度。

- **最好情况:** 每次划分都将数组分成大小相等的两半。设 $T(n)$ 为处理规模为 n 的数组所需时间，则有递归式：

$$T(n) = T(n/2) + O(n)$$

解得 $T(n) = O(n)$ 。直观理解： $n + n/2 + n/4 + \dots < 2n$ 。

- **平均情况:** 如果基准随机选择，即从当前区间中均匀随机地选取一个元素作为基准，那么分区后左子区间的大小 i 可以是 0 到 $n-1$ 中的任何一个值，且每个值的概率均为 $1/n$ （假设所有元素互异）。由于我们只递归处理包含第 k 小元素的那一侧，递归的规模取决于 i 与 k 的关系。为简化分析，通常考虑 k 为中位数的情况（即 $k = \lfloor n/2 \rfloor$ ），其他 k 的分析类似。当基准选定后，若左子区间大小 i 小于中位数的位置，则第 k 小元素落在右子区间，规模为 $n-1-i$ ；否则落在左子区间，规模为 i 。因此，递归的平均规模为

$$\frac{1}{n} \sum_{i=0}^{n-1} \max(i, n-1-i) \approx \frac{3n}{4}.$$

由此得到期望时间递推关系：

$$T(n) \leq n - 1 + T\left(\frac{3n}{4}\right).$$

解此递推式可得 $T(n) = O(n)$ 。更严格的数学分析（如通过求解递归式或使用归纳法）可以证明，快速选择的期望比较次数确实是线性的，常数约为 $2n$ 。

这里的“期望”是对算法内部的随机选择取平均，与输入数据的分布无关。因此，无论输入如何，随机化快速选择都能保证期望时间复杂度为 $O(n)$ 。而最坏情况虽然存在，但其概率极低，在实际应用中几乎不会遇到。

- **最坏情况：**如果每次划分都选到最小或最大的元素作为基准，那么每次只能排除一个元素，递归式变为：

$$T(n) = T(n - 1) + O(n)$$

解得 $T(n) = O(n^2)$ 。例如，在已经有序的数组中，如果总是选第一个或最后一个元素作为基准，就会发生这种情况。

那么，有没有办法避免这种最坏情况，使得算法在任何输入下都能保证线性时间复杂度呢？关键在于如何选择基准。如果每次都能选到一个“好”的基准，使得划分后左右两边的规模都至少是原问题的一个固定比例（比如 $1/10$ 和 $9/10$ ），那么递归深度就是 $O(\log n)$ ，总时间 $O(n)$ 。但如何保证这一点呢？我们需要一个确定性的方法，而不是依赖随机性。这正是下一节要介绍的 median-of-medians 算法。

4.10.8 改进版本：随机化快速选择

为了避免最坏情况，通常采用**随机选择基准**的方法。在每次划分前，从当前区间中随机选择一个元素与最右边元素交换，然后调用标准的划分函数。这样，任何特定的输入都很难导致最坏情况，算法的期望时间复杂度保持 $O(n)$ ，且最坏情况发生的概率极低。

随机化快速选择伪代码：

```
function randomizedQuickSelect(A, left, right, k)
    if left == right
        return A[left]

    randIndex = random(left, right)    // 随机选择一个下标
    swap A[randIndex] and A[right]    // 将随机选中的元素放到最右边作为基准
    pivotIndex = partition(A, left, right)

    if k == pivotIndex
        return A[k]
    else if k < pivotIndex
        return randomizedQuickSelect(A, left, pivotIndex - 1, k)
    else
        return randomizedQuickSelect(A, pivotIndex + 1, right, k)
```

C++ 代码：

```

#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
using namespace std;

// 划分函数 (Lomuto), 以 A[right] 为基准, 返回基准最终下标
int partition(vector<int>& A, int left, int right) {
    int pivot = A[right];
    int i = left; // i 指向小于基准的区域的末尾
    for (int j = left; j < right; ++j) {
        if (A[j] <= pivot) {
            swap(A[i], A[j]);
            ++i;
        }
    }
    swap(A[i], A[right]); // 将基准放到正确位置
    return i;
}

// 随机化快速选择主函数
// 在 A[left..right] 中找第 k 小的元素 (0-based)
int randomizedQuickSelect(vector<int>& A, int left, int right, int k) {
    if (left == right) // 只有一个元素
        return A[left];

    // 随机选择一个下标作为基准, 并交换到最右边
    int randIndex = left + rand() % (right - left + 1);
    swap(A[randIndex], A[right]);

    int pivotIndex = partition(A, left, right);

    if (k == pivotIndex)
        return A[k];
    else if (k < pivotIndex)
        return randomizedQuickSelect(A, left, pivotIndex - 1, k);
    else
        return randomizedQuickSelect(A, pivotIndex + 1, right, k);
}

// 对外接口: 在数组 A 中找第 k 小的元素 (k 从 0 开始)
int quickSelect(vector<int>& A, int k) {

```

```

srand(time(nullptr)); // 初始化随机种子
return randomizedQuickSelect(A, 0, A.size() - 1, k);
}

// 测试
int main() {
    vector<int> arr = {9, 3, 7, 1, 6, 2, 8};
    int k = 3; // 找第 4 小元素 (0-based 第 3 个)
    cout << "The " << k << "-th smallest element is " << quickSelect(arr, k) << endl;
    return 0;
}

```

python 代码:

```

import random

def partition(A, left, right):
    """ 划分函数, 以 A[right] 为基准, 返回基准最终下标 """
    pivot = A[right]
    i = left # i 指向小于基准的区域的末尾
    for j in range(left, right):
        if A[j] <= pivot:
            A[i], A[j] = A[j], A[i]
            i += 1
    A[i], A[right] = A[right], A[i]
    return i

def randomized_quick_select(A, left, right, k):
    """ 在 A[left..right] 中找第 k 小的元素 (0-based) """
    if left == right:
        return A[left]

    # 随机选择一个下标作为基准, 并交换到最右边
    rand_index = random.randint(left, right)
    A[rand_index], A[right] = A[right], A[rand_index]

    pivot_index = partition(A, left, right)

    if k == pivot_index:
        return A[k]
    elif k < pivot_index:
        return randomized_quick_select(A, left, pivot_index - 1, k)
    else:

```



```

        return randomized_quick_select(A, pivot_index + 1, right, k)

def quick_select(A, k):
    """ 对外接口：在数组 A 中找第 k 小的元素 (k 从 0 开始) """
    return randomized_quick_select(A, 0, len(A) - 1, k)

# 测试
if __name__ == "__main__":
    arr = [9, 3, 7, 1, 6, 2, 8]
    k = 3 # 找第 4 小元素 (0-based 第 3 个)
    print(f"The {k}-th smallest element is {quick_select(arr, k)}")

```

4.10.9 实际应用

快速选择算法在实际中有着广泛的应用，因为它能在线性时间内解决选择问题，比排序后再取值的 $O(n \log n)$ 更快。

- **求中位数**：中位数是第 $n/2$ 小的元素，用快速选择可以在 $O(n)$ 时间内找到。
- **Top-K 问题**：例如找出前 K 个最大的元素。先用快速选择找到第 K 大的元素（即第 $n - K + 1$ 小的元素），然后扫描数组，收集所有大于等于该元素的数（注意处理重复值）。
- **数据流中的分位数**：在大量数据中快速估计分位数。
- **机器学习**：在构建决策树时，需要快速找到特征的中位数或某个分位数进行划分。

4.10.10 练习题

1. **手工模拟**：用快速选择算法在数组 $[12, 5, 7, 3, 19, 1, 9]$ 中寻找第 4 小的元素。请写出每一步的划分结果和递归过程（假设基准总是选当前区间的最后一个元素）。
2. **最坏情况分析**：给出一个具体的数组，使得快速选择（总是选最后一个元素作为基准）的时间复杂度达到 $O(n^2)$ ，并解释为什么。
3. **代码实现**：用你熟悉的编程语言实现随机化快速选择，并测试在随机数组上寻找中位数的运行时间，与直接排序取中位数的方法进行比较。
4. **找第 K 大元素**：如何修改快速选择算法，使其能够找出第 K 大的元素？写出思路和伪代码。
5. **思考题**：快速选择与快速排序都是基于分治递归的算法，它们的主要区别是什么？为什么快速选择的平均时间复杂度比快速排序更低？

4.11 Median-of-Medians 算法

快速选择算法虽然在平均情况下表现出色（期望 $O(n)$ ），但它的最坏情况时间复杂度是 $O(n^2)$ 。尽管随机化可以极大降低最坏情况发生的概率，但在某些对运行时间有严格上界要求的场景（如实时系统、安全关键系统），我们仍然需要一个确定性的线性时间算法。1973 年，Blum、Floyd、Pratt、Rivest 和 Tarjan 提出了 **Median-of-Medians 算法**（也称为 BFPRT 算法），它通过一种巧妙的 pivot 选择策略，保证了在最坏情况下也能达到 $O(n)$ 的时间复杂度。

4.11.1 核心思想：如何保证“好”的 pivot?

快速选择的最坏情况发生在每次选到的 pivot 都是最小或最大元素，导致划分极度不平衡。如果每次都能选到一个“近似中位数”的 pivot，使得划分后左右两边的规模都至少是原问题的某个固定比例（例如 $1/4$ 和 $3/4$ ），那么递归深度就是 $O(\log n)$ ，总时间 $O(n)$ 。问题是：如何在不对整个数组排序的情况下，快速找到一个近似中位数？

Median-of-Medians 算法的想法是：

1. 将数组分成若干组，每组固定大小（通常选 5）。
2. 找出每组的中位数（可以用插入排序等简单方法，因为每组大小是常数）。
3. 递归地在这些中位数中找出它们的中位数（即“中位数的中位数”），将这个值作为 pivot。

可以证明，这样选出的 pivot 至少保证了至少有 $\lfloor n/4 \rfloor$ 个元素小于它，也至少有 $\lfloor n/4 \rfloor$ 个元素大于它（具体证明见后）。因此，最坏情况下递归规模也能缩小至少 $1/4$ 。

4.11.2 算法步骤

假设我们要在数组 A 的区间 $[left, right]$ 中找第 k 小的元素 (k 从 1 开始计数)。

1. **分组**：将当前区间内的元素每 5 个一组（最后一组可能不足 5 个）。
2. **找每组的中位数**：对每组内的元素进行排序（因为每组大小 ≤ 5 ，排序时间可视为常数），找出每组的中位数。
3. **递归求中位数的中位数**：将这些中位数收集起来，组成一个新的数组 M ，递归调用 Median-of-Medians 算法找出 M 的中位数（即 M 中第 $\lfloor |M|/2 \rfloor$ 小的元素）。这个值就是我们要选的 pivot。
4. **划分**：用这个 pivot 对原数组进行划分（可以使用快速排序的划分算法，如 Lomuto 或 Hoare），得到 pivot 的最终位置 p 。
5. **递归**：比较 k 与 p ，决定在左半部分还是右半部分递归查找（与快速选择相同）。

算法概述（简洁伪代码）：

```
将 n 个元素分成  $\lceil n/5 \rceil$  组
确定每组的中位数 m_j
递归调用本算法找到所有中位数的中位数 m
用 m 将数组划分为三部分：AL（小于 m），p（等于 m），AR（大于 m）
if len(AL) == k-1: return p
else if len(AL) < k-1: return Select(AR, k - len(AL) - 1)
else: return Select(AL, k)
```

4.11.3 代码实现

下面分别给出 C++ 和 Python 的完整实现。为了简化代码，我们使用插入排序处理每组（每组最多 5 个元素），并使用 Lomuto 划分方案。

4.11.3.1 C++ 实现

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```

using namespace std;

// 插入排序，用于对每组最多 5 个元素排序
void insertionSort(vector<int>& arr, int left, int right) {
    for (int i = left + 1; i <= right; ++i) {
        int key = arr[i];
        int j = i - 1;
        while (j >= left && arr[j] > key) {
            arr[j + 1] = arr[j];
            --j;
        }
        arr[j + 1] = key;
    }
}

// 根据给定值 val 划分数组，返回 val 的最终下标
int partitionByValue(vector<int>& arr, int left, int right, int val) {
    // 找到 val 的位置并交换到最右边
    for (int i = left; i <= right; ++i) {
        if (arr[i] == val) {
            swap(arr[i], arr[right]);
            break;
        }
    }
    int pivot = arr[right];
    int i = left;
    for (int j = left; j < right; ++j) {
        if (arr[j] <= pivot) {
            swap(arr[i], arr[j]);
            ++i;
        }
    }
    swap(arr[i], arr[right]);
    return i;
}

// Median-of-Medians 主函数
int medianOfMedians(vector<int>& arr, int left, int right, int k) {
    if (left == right) return arr[left]; // 只有一个元素

    int n = right - left + 1;
    vector<int> medians;

```

```

// 1. 分组, 每组 5 个, 找出每组的中位数
for (int i = 0; i < (n + 4) / 5; ++i) {
    int groupLeft = left + i * 5;
    int groupRight = min(left + i * 5 + 4, right);
    insertionSort(arr, groupLeft, groupRight);
    int medianPos = groupLeft + (groupRight - groupLeft) / 2;
    medians.push_back(arr[medianPos]);
}

// 2. 递归求中位数的中位数
int pivot;
if (medians.size() == 1) {
    pivot = medians[0];
} else {
    pivot = medianOfMedians(medians, 0, medians.size() - 1, medians.size() / 2);
}

// 3. 划分
int pivotIndex = partitionByValue(arr, left, right, pivot);
int leftSize = pivotIndex - left;

// 4. 根据 k 决定递归方向
if (k == leftSize + 1) {
    return arr[pivotIndex];
} else if (k <= leftSize) {
    return medianOfMedians(arr, left, pivotIndex - 1, k);
} else {
    return medianOfMedians(arr, pivotIndex + 1, right, k - leftSize - 1);
}
}

// 对外接口
int findKthSmallest(vector<int>& arr, int k) {
    return medianOfMedians(arr, 0, arr.size() - 1, k);
}

// 测试代码 (示例)
int main() {
    vector<int> arr = {2, 6, 3, 8, 1, 5, 9, 4, 7, 0};
    int k = 5;
    int result = findKthSmallest(arr, k);
    cout << "The " << k << "-th smallest element is " << result << endl;
}

```

```

    return 0;
}

```

4.11.3.2 Python 实现

```

def insertion_sort(arr, left, right):
    """ 对 arr[left..right] 进行插入排序 """
    for i in range(left + 1, right + 1):
        key = arr[i]
        j = i - 1
        while j >= left and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

def partition_by_value(arr, left, right, val):
    """ 将 arr[left..right] 按值 val 划分, 返回 val 的最终下标 """
    # 找到 val 并交换到末尾
    for i in range(left, right + 1):
        if arr[i] == val:
            arr[i], arr[right] = arr[right], arr[i]
            break
    pivot = arr[right]
    i = left
    for j in range(left, right):
        if arr[j] <= pivot:
            arr[i], arr[j] = arr[j], arr[i]
            i += 1
    arr[i], arr[right] = arr[right], arr[i]
    return i

def median_of_medians(arr, left, right, k):
    """ 返回 arr[left..right] 中第 k 小的元素 (k 从 1 开始) """
    if left == right:
        return arr[left]

    n = right - left + 1
    medians = []

    # 1. 分组, 每组 5 个, 找每组中位数
    for i in range((n + 4) // 5):
        group_left = left + i * 5

```

```

        group_right = min(left + i * 5 + 4, right)
        insertion_sort(arr, group_left, group_right)
        median_pos = group_left + (group_right - group_left) // 2
        medians.append(arr[median_pos])

# 2. 递归求中位数的中位数
if len(medians) == 1:
    pivot = medians[0]
else:
    pivot = median_of_medians(medians, 0, len(medians) - 1, len(medians) // 2)

# 3. 划分
pivot_index = partition_by_value(arr, left, right, pivot)
left_size = pivot_index - left

# 4. 递归选择
if k == left_size + 1:
    return arr[pivot_index]
elif k <= left_size:
    return median_of_medians(arr, left, pivot_index - 1, k)
else:
    return median_of_medians(arr, pivot_index + 1, right, k - left_size - 1)

def find_kth_smallest(arr, k):
    """ 对外接口 """
    return median_of_medians(arr, 0, len(arr) - 1, k)

# 测试代码
if __name__ == "__main__":
    arr = [2, 6, 3, 8, 1, 5, 9, 4, 7, 0]
    k = 5
    result = find_kth_smallest(arr, k)
    print(f"The {k}-th smallest element is {result}")

```

4.11.4 时间复杂度分析：为什么能保证线性时间？

Median-of-Medians 算法的核心在于通过递归选取“中位数的中位数”作为枢轴 x ，从而保证每一轮递归能淘汰掉固定比例的元素，使最坏情况下的总时间仍为线性。

4.11.4.1 基本递归框架

将 n 个元素分为 $g = \lceil n/5 \rceil$ 组（每组最多 5 个）。算法的主要代价包括：

- 分组及组内找中位数: $O(n)$
- 递归寻找 g 个中位数的中位数: $T(\lceil n/5 \rceil)$
- 以 x 为枢轴进行划分: $O(n)$
- 在左侧或右侧子数组中递归选择: $T(\text{子数组大小})$

因此, 存在常数 $a, b > 0$ 使得:

$$T(n) \leq T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T(\text{子数组大小}) + an + b.$$

关键问题是确定 **子数组大小的上界**。

4.11.4.2 排除量 W 的下界

设 M 为各组中位数构成的数组, x 是 M 的中位数。我们分析 **不小于** x 的元素个数至少为多少。

- 在 g 个中位数中, 至少有 $\lceil g/2 \rceil$ 个中位数 **不小于** x 。
- 考虑这些中位数对应的组:
 - **完整组** (大小为 5): 其中位数不小于 x 时, 该组至少有 3 个元素不小于 x (中位数及其右侧两个元素)。
 - **不完整组** (最后一组, 大小 $r \leq 4$): 若它的中位数不小于 x , 则至少贡献 1 个元素 (即它自身的中位数)。
- x 本身所在的组 (若为完整组) 已经包含在上述完整组中, 其贡献已计入 3。

为了获得一个不管输入分布如何都成立的**下界**, 我们需要考虑最坏情况 (即不完整组可能不存在或其中位数小于 x , 从而不贡献)。此时, 所有 $\lceil g/2 \rceil$ 个中位数不小于 x 的组必须都是完整组, 每个贡献 3 个元素, 因此:

$$W \geq 3 \left\lceil \frac{g}{2} \right\rceil.$$

若存在不完整组且其中位数也不小于 x , 则完整组的数量至少为 $\lceil g/2 \rceil - 1$, 加上不完整组的 1 个, 得 $3(\lceil g/2 \rceil - 1) + 1 = 3\lceil g/2 \rceil - 2$ 。两种可能的数量中, **较小者**是 $3\lceil g/2 \rceil - 2$, 这才是在所有情况下都成立的下界 (因为我们总是可以取最坏情况)。因此:

$$W \geq 3 \left\lceil \frac{g}{2} \right\rceil - 2.$$

代入 $g = \lceil n/5 \rceil \geq n/5$, 有 $\lceil g/2 \rceil \geq \frac{n}{10}$, 因此:

$$W \geq 3 \cdot \frac{n}{10} - 2 = \frac{3n}{10} - 2.$$

4.11.4.3 对称性与子问题规模上界

由对称性, 同样可证 **不大于** x 的元素个数也至少为 $\frac{3n}{10} - 2$ 。

划分后, 若目标在左侧 (小于 x 的部分), 则右侧至少包含 $\frac{3n}{10} - 2$ 个元素 (这些元素不会进入下一轮递归)。因此左侧大小至多为:

$$n - \left(\frac{3n}{10} - 2 \right) = \frac{7n}{10} + 2.$$

同理, 若目标在右侧, 左侧也会排除至少相同数量的元素, 右侧大小同样不超过 $\frac{7n}{10} + 2$ 。

于是，递归子问题的规模上界为：

$$n' \leq \frac{7n}{10} + 2.$$

4.11.4.4 递归不等式

综合以上分析，得到递归式：

$$T(n) \leq T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\frac{7n}{10} + 2\right) + an + b,$$

其中 a, b 为正常数。对于足够大的 n ，可忽略取整和常数项，简化为：

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + cn.$$

4.11.4.5 数学归纳法证明

命题：存在常数 C 和正整数 N ，使得对任意 $n \geq 1$ ，有 $T(n) \leq Cn$ 。

证明：

1. 归纳基础

取足够小的 n_0 (例如 $n_0 = 100$)，对于所有 $n \leq n_0$ ，算法可直接使用暴力方法 (如排序)，其时间 $T(n)$ 有上界 $C_0 n$ ，其中 C_0 可取 $\max_{1 \leq n \leq n_0} \frac{T(n)}{n}$ 。我们将最终常数 C 选为不小于 C_0 且满足下面推导的值。

2. 归纳假设

假设对于所有 $m < n$ ($n > n_0$)，存在常数 C (待定) 使得 $T(m) \leq Cm$ 。

3. 归纳步骤

对于规模 n ，由递归式 (忽略取整与常数 +2，它们仅影响基础情况)：

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + cn.$$

应用归纳假设于 $\frac{n}{5} < n$ 和 $\frac{7n}{10} < n$ (当 $n > 0$)：

$$T(n) \leq C \cdot \frac{n}{5} + C \cdot \frac{7n}{10} + cn = \frac{9}{10}Cn + cn.$$

我们需要 $\frac{9}{10}Cn + cn \leq Cn$ ，即：

$$\frac{9}{10}C + c \leq C \implies c \leq \frac{C}{10} \implies C \geq 10c.$$

因此，取 $C = \max\{C_0, 10c\}$ ，则对于所有 $n > n_0$ 有 $T(n) \leq Cn$ 。

4. 结论

由基础情况和归纳步骤，对所有 $n \geq 1$ 有 $T(n) \leq Cn$ ，即 $T(n) = O(n)$ 。

补充说明：若考虑递归式中的常数项 +2 和取整，只需在归纳步骤中多处理一个常数， C 取更大值 (如 $C = 20c$) 并验证足够大的 n 即可，不影响线性结论。

注

- 每组取 5 是能保证线性时间的最小奇数。若每组取 3，则每组中位数只能保证至少 2 个元素不大于/不小于中位数，推导出的排除量下界约为 $2 \cdot \frac{n}{6} = \frac{n}{3}$ ，递归式变为 $T(n) \leq T(n/3) + T(2n/3) + O(n)$ ，其解为 $O(n \log n)$ 。因此选择 3 不能达到线性时间；而选择 7 或更大的奇数仍可保证线性，但常数因子会更大。
- 该分析证明了 Median-of-Medians 算法在最坏情况下具有线性时间复杂度，这是它区别于普通 QuickSelect 的关键特性。

4.11.5 小结

4.11.5.1 与快速选择的对比

特性	快速选择（随机化）	Median-of-Medians
平均时间复杂度	$O(n)$	$O(n)$
最坏时间复杂度	$O(n^2)$ （概率极低）	$O(n)$ （确定性）
常数因子	小（约 2-3）	较大（约 10-20）
实现复杂度	简单	复杂，需要递归求中位数数组
实际应用	广泛，因为最坏情况罕见且常数小	主要用于理论证明或对时间上界敏感的场景

4.11.5.2 实际应用中的注意事项

- **分组大小**：理论上选择 5 是能保证线性时间的最小奇数。若选择 3，算法将退化为 $O(n \log n)$ ；若选择 7 或更大，仍可保持线性，但常数因子会更大，且递归深度更浅但每层工作量略增。
- **排序每组**：每组大小固定（如 5），可以用插入排序或直接比较，因为常数时间。
- **内存开销**：需要存储中位数数组，但规模为 $O(n/5)$ ，可以接受。
- **性能**：由于常数因子较大，在实际问题中，随机化快速选择通常更快。Median-of-Medians 主要用于理论价值或在特殊需求下提供最坏情况保证。

Median-of-Medians 算法是选择问题的一个里程碑式成果，它证明了在最坏情况下也能在线性时间内解决选择问题。虽然它的实际运行效率不如随机化快速选择，但它为算法设计提供了重要的思想：通过精心选择 pivot 来控制递归规模。理解这个算法有助于加深对分治策略和时间复杂度分析的认识。

4.11.6 练习题

1. 手工模拟 Median-of-Medians 算法在数组 $[2, 6, 3, 8, 1, 5, 9, 4, 7, 0]$ 中找第 5 小的元素，写出每一步的分组、中位数选取和递归过程。
2. 如果每组大小改为 3，推导此时的递归式并证明算法的时间复杂度是 $O(n \log n)$ （而不是线性）。
3. 实现 Median-of-Medians 算法（可用插入排序排序每组），并与随机化快速选择在随机数据上比较运行时间。
4. 为什么通常选择 5 作为分组大小？如果选择 7，分析对递归式系数和常数因子的影响。

4.12 最大子数组问题

4.12.1 问题提出

最大子数组问题 (Maximum Subarray Problem) 是计算机科学中的一个经典问题, 其定义如下。

给定一个整数数组

$$A[1..n]$$

数组中的元素可以是正数、负数或零。需要找到一个**连续的非空子数组**

$$A[i..j] \quad (1 \leq i \leq j \leq n)$$

使得该子数组中所有元素的和最大。输出这个最大和, 在一些应用中还需要同时输出该子数组的起始下标和结束下标。

形式化地, 需要计算

$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j A[k]$$

并可能要求给出对应的 i 和 j 。

应用背景

最大子数组问题在多个领域都有实际应用, 例如:

- (1) **股票交易分析**: 如果将股票每天的价格变化表示为一个数组, 则最大子数组对应股票价格变化最大的连续区间, 可以理解为在该时间段内买入并卖出能够获得的最大收益。
- (2) **信号处理**: 在一维信号数据中, 可以通过最大子数组寻找能量最强的连续信号片段。
- (3) **生物信息学**: 在基因序列分析中, 可以用类似方法寻找相似性最高的连续片段。

4.12.2 直观示例

考虑数组 $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$ 。我们手动寻找最大子数组:

所有可能的连续子数组: 例如 $[-2]$ 和为 -2 , $[-2, 1]$ 和为 -1 , $[1]$ 和为 1 , $[1, -3]$ 和为 -2 , ... 直到整个数组。

通过观察, 子数组 $[4, -1, 2, 1]$ 的和为 $4 + (-1) + 2 + 1 = 6$ 。有没有更大的? 检查 $[4, -1, 2, 1, -5, 4]$ 和为 5 ; $[-1, 2, 1]$ 和为 2 ; $[2, 1]$ 和为 3 ; $[4]$ 和为 4 。因此最大和为 6 , 对应子数组从下标 4 到 7 (假设从 1 开始计数)。

这个示例说明, 最大子数组不一定从最左边开始, 也不一定到最右边结束, 它可能隐藏在数组中间。

4.12.3 算法思想

本节介绍求解最大子数组问题的一种经典算法——分治递归算法。

分治法的核心思想是将问题分解为规模更小的子问题, 递归求解, 然后合并结果。对于最大子数组问题, 可以将数组 $A[low..high]$ 分为两个规模大致相等的子数组: 左半部分 $A[low..mid]$ 和右半部分 $A[mid + 1..high]$, 其中 $mid = \lfloor (low + high)/2 \rfloor$ 。那么, 原数组的最大子数组必然出现在以下三种情况之一:

1. 完全位于左半部分 $A[low..mid]$ 中;
2. 完全位于右半部分 $A[mid + 1..high]$ 中;

3. 跨越中点，即左半部分包含一段以 mid 结尾的子数组，右半部分包含一段以 $mid + 1$ 开头的子数组，两者合并构成跨越中点的子数组。

对于情况 1 和 2，可以通过递归求解得到；对于情况 3，可以在线性时间内找到跨越中点的最大子数组（即从中点向左扫描找到以 mid 结尾的最大子数组和，再从中点向右扫描找到以 $mid + 1$ 开头的最大子数组和，然后相加）。最后，取这三种情况的最大值即为原问题的解。

4.12.4 算法描述

输入：数组 A ，下标范围 $low, high$ （初始调用为 $1, n$ ）**输出：**最大子数组的和（也可同时返回起止下标）

1. 如果 $low = high$ ，则只有一个元素，直接返回 $A[low]$ （以及该下标）。
2. 否则，计算中点 $mid = \lfloor (low + high) / 2 \rfloor$ 。
3. 递归求解左半部分的最大子数组，记为 $left_sum$ （若需下标，同时记录左半部分最大子数组的起止位置）。
4. 递归求解右半部分的最大子数组，记为 $right_sum$ （若需下标，同时记录右半部分最大子数组的起止位置）。
5. 求解跨越中点的最大子数组：
 - 从中点 mid 向左扫描，计算以 mid 结尾的最大连续和 $left_cross_sum$ ，并记录其起始位置。
 - 从中点 $mid + 1$ 向右扫描，计算以 $mid + 1$ 开头的最大连续和 $right_cross_sum$ ，并记录其结束位置。
 - 跨越中点的最大子数组和为 $cross_sum = left_cross_sum + right_cross_sum$ ，起止位置分别为向左扫描得到的起始下标和向右扫描得到的结束下标。
6. 取 $left_sum$ 、 $right_sum$ 、 $cross_sum$ 中的最大值作为结果返回，若需下标，同时返回对应子数组的起止位置。

4.12.5 伪代码

4.12.5.1 仅返回最大和

```
Algorithm FindMaxSubarray(A, low, high):
    if low == high then
        return A[low]
    else
        mid = floor((low + high) / 2)
        left_sum = FindMaxSubarray(A, low, mid)
        right_sum = FindMaxSubarray(A, mid+1, high)
        cross_sum = FindMaxCrossingSubarray(A, low, mid, high)
        return max(left_sum, right_sum, cross_sum)
    end if
```

```
Algorithm FindMaxCrossingSubarray(A, low, mid, high):
    left_sum = -∞
    sum = 0
    for i = mid downto low do
        sum = sum + A[i]
        if sum > left_sum then
```

```

        left_sum = sum
    end if
end for
right_sum = -∞
sum = 0
for i = mid+1 to high do
    sum = sum + A[i]
    if sum > right_sum then
        right_sum = sum
    end if
end for
return left_sum + right_sum

```

4.12.5.2 返回最大和及起止下标

为返回下标，需要在递归和跨越函数中记录相应位置。以下伪代码展示了如何返回一个包含（最大和，起始下标，结束下标）的三元组。

Algorithm FindMaxSubarrayWithIndex(A, low, high):

```

    if low == high then
        return (A[low], low, high)
    else
        mid = floor((low + high) / 2)
        (left_sum, left_start, left_end) = FindMaxSubarrayWithIndex(A, low, mid)
        (right_sum, right_start, right_end) = FindMaxSubarrayWithIndex(A, mid+1, high)
        (cross_sum, cross_start, cross_end) = FindMaxCrossingSubarrayWithIndex(A, low, mid, high)

        if left_sum >= right_sum and left_sum >= cross_sum then
            return (left_sum, left_start, left_end)
        else if right_sum >= left_sum and right_sum >= cross_sum then
            return (right_sum, right_start, right_end)
        else
            return (cross_sum, cross_start, cross_end)
        end if
    end if
end if

```

Algorithm FindMaxCrossingSubarrayWithIndex(A, low, mid, high):

```

    left_sum = -∞
    sum = 0
    max_left = mid
    for i = mid downto low do
        sum = sum + A[i]
        if sum > left_sum then

```

```

        left_sum = sum
        max_left = i
    end if
end for

right_sum = -∞
sum = 0
max_right = mid+1
for i = mid+1 to high do
    sum = sum + A[i]
    if sum > right_sum then
        right_sum = sum
        max_right = i
    end if
end for

return (left_sum + right_sum, max_left, max_right)

```

4.12.6 代码实现

4.12.6.1 C++ 实现

仅返回最大和

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
using namespace std;

// 寻找跨越中点的最大子数组和
int maxCrossingSum(const vector<int>& A, int low, int mid, int high) {
    int left_sum = INT_MIN;
    int sum = 0;
    for (int i = mid; i >= low; --i) {
        sum += A[i];
        if (sum > left_sum) left_sum = sum;
    }

    int right_sum = INT_MIN;
    sum = 0;
    for (int i = mid + 1; i <= high; ++i) {
        sum += A[i];
        if (sum > right_sum) right_sum = sum;
    }
}

```

```

    }

    return left_sum + right_sum;
}

// 分治递归求解最大子数组和
int maxSubarraySum(const vector<int>& A, int low, int high) {
    if (low == high) return A[low];    // 基础情况: 只有一个元素

    int mid = low + (high - low) / 2;
    int left_sum = maxSubarraySum(A, low, mid);
    int right_sum = maxSubarraySum(A, mid + 1, high);
    int cross_sum = maxCrossingSum(A, low, mid, high);

    return max({left_sum, right_sum, cross_sum});
}

// 包装函数
int maxSubarray(const vector<int>& A) {
    if (A.empty()) return 0;
    return maxSubarraySum(A, 0, A.size() - 1);
}

```

返回最大和及起止下标

```

#include <tuple>
using namespace std;

// 返回三元组 (最大和, 起始下标, 结束下标)
tuple<int, int, int> maxCrossingWithIndex(const vector<int>& A, int low, int mid, int high) {
    int left_sum = INT_MIN;
    int sum = 0;
    int max_left = mid;
    for (int i = mid; i >= low; --i) {
        sum += A[i];
        if (sum > left_sum) {
            left_sum = sum;
            max_left = i;
        }
    }

    int right_sum = INT_MIN;
    sum = 0;

```

```

    int max_right = mid + 1;
    for (int i = mid + 1; i <= high; ++i) {
        sum += A[i];
        if (sum > right_sum) {
            right_sum = sum;
            max_right = i;
        }
    }

    return {left_sum + right_sum, max_left, max_right};
}

tuple<int, int, int> maxSubarrayWithIndex(const vector<int>& A, int low, int high) {
    if (low == high) {
        return {A[low], low, high};
    }

    int mid = low + (high - low) / 2;
    auto [left_sum, left_start, left_end] = maxSubarrayWithIndex(A, low, mid);
    auto [right_sum, right_start, right_end] = maxSubarrayWithIndex(A, mid + 1, high);
    auto [cross_sum, cross_start, cross_end] = maxCrossingWithIndex(A, low, mid, high);

    if (left_sum >= right_sum && left_sum >= cross_sum)
        return {left_sum, left_start, left_end};
    else if (right_sum >= left_sum && right_sum >= cross_sum)
        return {right_sum, right_start, right_end};
    else
        return {cross_sum, cross_start, cross_end};
}

```

4.12.6.2 Python 实现

仅返回最大和

```

import sys

def max_crossing_sum(A, low, mid, high):
    left_sum = -sys.maxsize
    total = 0
    for i in range(mid, low - 1, -1):
        total += A[i]
        if total > left_sum:
            left_sum = total

```

```

    right_sum = -sys.maxsize
    total = 0
    for i in range(mid + 1, high + 1):
        total += A[i]
        if total > right_sum:
            right_sum = total

    return left_sum + right_sum

def max_subarray_sum(A, low, high):
    if low == high:
        return A[low]
    mid = (low + high) // 2
    left_sum = max_subarray_sum(A, low, mid)
    right_sum = max_subarray_sum(A, mid + 1, high)
    cross_sum = max_crossing_sum(A, low, mid, high)
    return max(left_sum, right_sum, cross_sum)

def max_subarray(A):
    if not A:
        return 0
    return max_subarray_sum(A, 0, len(A) - 1)

```

返回最大和及起止下标

```

def max_crossing_with_index(A, low, mid, high):
    left_sum = -10**9
    total = 0
    max_left = mid
    for i in range(mid, low - 1, -1):
        total += A[i]
        if total > left_sum:
            left_sum = total
            max_left = i

    right_sum = -10**9
    total = 0
    max_right = mid + 1
    for i in range(mid + 1, high + 1):
        total += A[i]
        if total > right_sum:
            right_sum = total
            max_right = i

```



```

    return (left_sum + right_sum, max_left, max_right)

def max_subarray_with_index(A, low, high):
    if low == high:
        return (A[low], low, high)

    mid = (low + high) // 2
    left_sum, left_start, left_end = max_subarray_with_index(A, low, mid)
    right_sum, right_start, right_end = max_subarray_with_index(A, mid + 1, high)
    cross_sum, cross_start, cross_end = max_crossing_with_index(A, low, mid, high)

    if left_sum >= right_sum and left_sum >= cross_sum:
        return (left_sum, left_start, left_end)
    elif right_sum >= left_sum and right_sum >= cross_sum:
        return (right_sum, right_start, right_end)
    else:
        return (cross_sum, cross_start, cross_end)

```

4.12.7 正确性证明

采用归纳法证明分治递归算法的正确性。

基础情况：当 $low = high$ 时，数组只有一个元素，最大子数组就是该元素本身，算法直接返回该元素，正确。

归纳步骤：假设对于任意长度小于 n 的数组，递归调用能正确返回其最大子数组和。考虑长度为 n 的数组 $A[low..high]$ ，将其划分为左半 $A[low..mid]$ 和右半 $A[mid + 1..high]$ ，长度均小于 n 。由归纳假设，递归调用 $FindMaxSubarray(A, low, mid)$ 和 $FindMaxSubarray(A, mid+1, high)$ 能正确得到左半和右半的最大子数组和。

原数组的最大子数组只有三种可能：

- 完全位于左半部分；
- 完全位于右半部分；
- 跨越中点。

对于跨越中点的情况，任何这样的子数组必然由左半部分中以 mid 结尾的一段和右半部分中以 $mid + 1$ 开头的一段拼接而成。算法通过从中点向左扫描，计算以 mid 结尾的最大连续和（即左半部分中所有以 mid 结尾的子数组和的最大值），再从中点向右扫描，计算以 $mid + 1$ 开头的最大连续和（即右半部分中所有以 $mid + 1$ 头的子数组和的最大值），两者之和即为跨越中点的最大子数组和。该过程遍历了所有跨越中点的可能，因此得到的是正确的。

最后，取三者中的最大值作为整个数组的最大子数组和。由于这三种情况覆盖了所有可能，算法返回的结果一定是正确的。由归纳法，算法对所有输入正确。

若算法同时记录下标，则在跨越中点扫描时记录了最大和对应的起止下标，在递归返回时也记录了子数组的下标，最终返回的下标对应最大值所在的子数组，同样正确。

4.12.8 复杂度分析

时间复杂度：设 $T(n)$ 为处理长度为 n 的数组所需时间。算法将问题分解为两个规模为 $n/2$ 的子问题（递归部分），合并时需要线性时间 $O(n)$ 来求解跨越中点的最大子数组（即两个方向的扫描）。因此有递推关系：

$$T(n) = 2T(n/2) + O(n)$$

根据主定理（Master Theorem），该递推式的解为 $T(n) = O(n \log n)$ 。

空间复杂度：递归调用栈的深度为 $O(\log n)$ ，每次递归调用使用常数额外空间，因此总体空间复杂度为 $O(\log n)$ （不计输入数组本身）。若考虑输入存储，则总空间为 $O(n)$ 。

总结：分治递归算法以 $O(n \log n)$ 的时间复杂度解决了最大子数组问题，是分治策略的典型应用。虽然它不是最优的（存在线性时间算法，将在“线性规划”章节介绍），但其思想清晰，易于理解和实现，并为学习更高级的算法奠定了基础。

4.13 最近点对问题

4.13.1 问题描述

在平面上有 n 个点，给定它们的坐标 (x_i, y_i) ，我们希望找到距离最近的两个点（即欧几里得距离最小的点对）。这个问题在许多领域都有应用，比如地理信息系统、计算机图形学、模式识别等。

最直接的方法是枚举所有点对，计算距离，然后取最小值。这种方法的时间复杂度为 $O(n^2)$ ，当 n 很大时（比如上百万甚至百万），显然不可行。那么，有没有更快的办法呢？答案是肯定的——我们可以用分治法将时间复杂度降到 $O(n \log n)$ 。

4.13.2 分治思路

分治法的核心思想是将问题分解成规模较小的子问题，递归求解，再合并结果。对于最近点对问题，我们可以按照以下步骤进行：

1. **分：**将所有点按照 x 坐标排序，然后取一条垂直的中位线（即 x 坐标的中位数），将点集分成左右两部分 S_1 和 S_2 ，每部分大约有 $n/2$ 个点。如 [@fig-Closest_Pair_of_Points\(a\)](#) 所示。
2. **治：**递归地求解 S_1 和 S_2 内部的最近点对距离，分别记为 d_1 和 d_2 。令 $d = \min(d_1, d_2)$ 。
3. **合：**需要考虑那些一个点在 S_1 、另一个点在 S_2 的“跨越”点对。如果存在这样的点对距离小于 d ，那么它们一定位于中位线附近的一个带状区域内，如 [Figure 4.4](#) 所示。我们需要高效地找出这个带状区域内的最近点对。

4.13.3 合并步骤的关键

接检查所有跨越左右两部分的点对仍然需要 $O(n^2)$ 时间。但我们可以利用以下观察来加速：

- 只有那些距离中位线水平距离小于 d 的点才可能构成距离小于 d 的跨越点对。因此，我们只需考虑一个宽度为 $2d$ 的垂直带状区域（中线左右各 d 宽）内的点。将这些点按 y 坐标排序后，对于每个点，只需要检查它与后面若干个（最多 6 个）的距离即可。

为什么只需要检查常数个点呢？下面我们通过几何直观和图示来证明这一点。

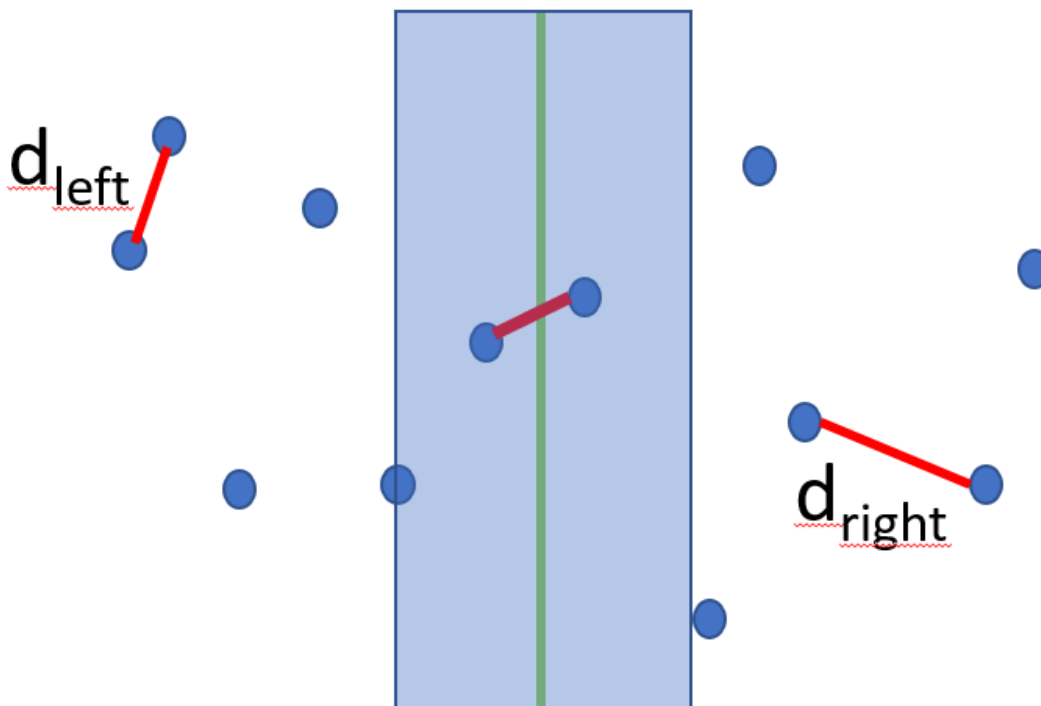


Figure 4.4: 最近点对问题的分治示意图：垂直分割线（中位线）与宽度为 d 的带状区域

4.13.4 为什么最多检查 6 个点？

设递归得到左右两半内部的最近距离为 d 。考虑跨越中线的点对，它们必然位于中线两侧宽度为 d 的带状区域内（即 $|x - x_{\text{mid}}| < d$ ）。将这些点按 y 坐标排序，设这些点的集合为 B_y 。对于其中任意一点 p （假设在左半），我们想找到所有可能与 p 构成距离小于 d 的右半部分点 q 。

考虑带状区域内的任意一点 p ，不妨假设 p 在左半部分。任何与 p 距离小于 d 的跨越点 q 必然位于右半部分，因此 q 一定在以 p 为圆心、半径为 d 的圆内，并且由于这些 q 都位于右半部分，这些 q 必须彼此相距至少 d 。

问题转化为：在一个半径为 d 的圆内，最多可以放置多少个点，使得任意两点距离 $\geq d$ ？下面证明这个数目不超过 6。

4.13.4.1 引理

设 p 是带状区域内中任意一点（不妨设 $p \in S_1$ ）， $Q = \{q \in S_2 \mid |pq| < d\}$ 是 S_2 中与 p 距离小于 d 的点集。则 $|Q| \leq 6$ 。

证明：如 @fig-closest_pair_max6_points 所示

- 由于 $|pq| < d$ ， q 必在以 p 为圆心、半径为 d 的圆内（包括边界）。同时，因为 q 在带状区域内，自然有 $|p.x - q.x| < d$ 和 $|p.y - q.y| < d$ ，但这是距离小于 d 的直接推论。
- 考虑 Q 中任意两点 q_i, q_j 。由于它们都属于 S_2 ，而 S_2 内部的最近点对距离至少为 d （由递归保证），因此 $|q_i q_j| \geq d$ 。
- 现在，以 p 为圆心，连接 p 与每个 q_i ，得到射线。这些射线将 360° 平面分割成 $|Q|$ 个角。假设 $|Q| \geq 7$ ，则至少有一个角 $\theta \leq 360^\circ/7 < 60^\circ$ 。设该角对应的两个点为 q_i 和 q_j ，则 $\angle q_i p q_j = \theta < 60^\circ$ 。
- 由余弦定理：

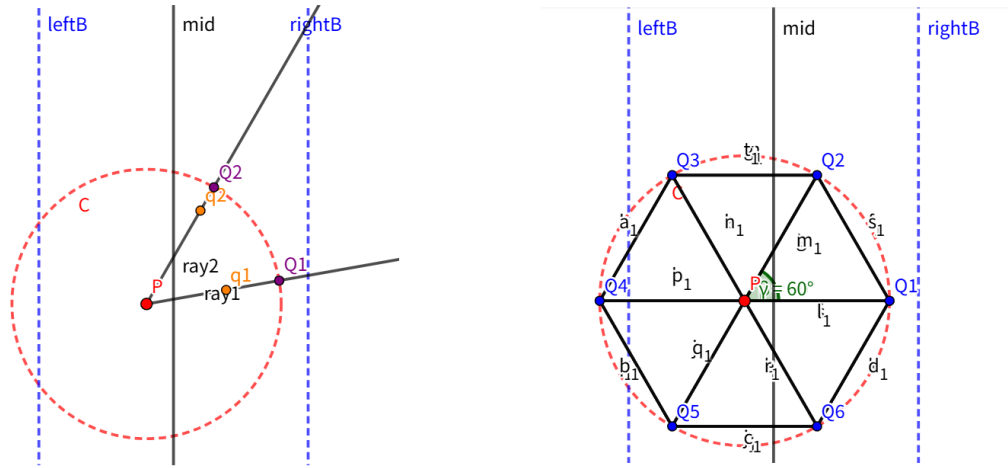


Figure 4.5: 对于带状区域中任意一点 p , 另一侧与 p 距离小于 d 的点不超过 6 个

$$|q_i q_j|^2 = |pq_i|^2 + |pq_j|^2 - 2|pq_i||pq_j| \cos \theta.$$

由于 $|pq_i| \leq d$, $|pq_j| \leq d$, 且 $\cos \theta > \cos 60^\circ = \frac{1}{2}$ (因为 $\theta < 60^\circ$), 可得:

$$|q_i q_j|^2 < d^2 + d^2 - 2 \cdot d \cdot d \cdot \frac{1}{2} = d^2.$$

因此 $|q_i q_j| < d$, 这与 $|q_i q_j| \geq d$ 矛盾。故假设不成立, $|Q| \leq 6$ 。

- 上述构造是可达的: 当 6 个点均匀分布在半径为 d 的圆周上 (正六边形顶点) 时, 每个相邻夹角为 60° , 且相邻顶点距离恰好为 d , 满足条件。因此上界 6 是紧的。

注: 在线绘图工具 <https://www.geogebra.org/classic> 用下列脚本绘制:

```
Delete[AllObjects]
d=1
P=(-0.2,0)
mid=Line((0,-2),(0,2))
leftB=Line((-1,-2),(-1,2))
rightB=Line((1,-2),(1,2))
C=Circle(P,d)
x1=x(P)+0.6*cos(20°)
y1=y(P)+0.6*sin(20°)
x2=x(P)+0.8*cos(80°)
y2=y(P)+0.8*sin(80°)
q1=Point({x1,y1})
q2=Point({x2,y2})
ray1=Ray(P,q1)
ray2=Ray(P,q2)
Q1=Intersect(ray1,C)
Q2=Intersect(ray2,C)
SetColor(P,"red")
```

```

SetColor(q1,"orange")
SetColor(q2,"orange")
SetColor(Q1,"purple")
SetColor(Q2,"purple")
SetColor(mid,"black")
SetLineStyle(mid,0)
SetColor(leftB,"blue")
SetLineStyle(leftB,2)
SetColor(rightB,"blue")
SetLineStyle(rightB,2)
SetColor(C,"red")
SetLineStyle(C,2)

```

4.13.5 分治算法框架

1. **预处理**: 将所有点按 x 坐标排序, 得到数组 P_x ; 同时按 y 坐标排序, 得到 P_y 。
2. **分治递归**:
 - 如果 $|P_x| \leq 3$, 直接暴力计算最近距离。
 - 否则, 取 x 坐标的中位数 x_{mid} , 将点集划分为左右两部分 S_1 和 S_2 , 并相应地将 P_y 也划分为 Q_y 和 R_y (保持 y 序)。
 - 递归求解 S_1 和 S_2 的最近距离, 分别记为 d_1 和 d_2 , 令 $d = \min(d_1, d_2)$ 。
 - **合并步骤**: 考虑跨越 S_1 和 S_2 的点对, 找出其中距离小于 d 的候选, 更新 d 。
3. **返回** d 。

对于每个点 p , 只需要在 B_y 中检查它后面至多 6 个点即可 (因为前面的点会在之前被检查过)。由于 B_y 已按 y 排序, 我们只需比较 p 与紧随其后的几个点, 直到 y 坐标差超过 d 为止 (此时后面的点距离必然更大)。这样, 每个点只进行常数比较, 整个合并步骤的时间复杂度为 $O(|B_y|) = O(n)$ 。

整体复杂度

递归关系 $T(n) = 2T(n/2) + O(n)$ 的解为 $T(n) = O(n \log n)$, 加上排序预处理 $O(n \log n)$, 总时间复杂度为 $O(n \log n)$ 。

4.13.6 一个具体的例子

让我们通过一个简单的例子来理解整个过程。假设有 6 个点:

$$P = \{(1,2), (2,3), (3,1), (4,5), (5,3), (6,4)\} \quad P = \{(1,2), (2,3), (3,1), (4,5), (5,3), (6,4)\}$$

首先, 我们将这些点按 x 坐标排序 (已经排好)。取中位线, 这里中间位置是第 3 个点 (下标从 0 开始), 即点 $(3,1)$ 和 $(4,5)$ 之间, 我们可以取 $x = 3.5$ 作为中位线, 将点集分成左半 $S_1 = (1,2), (2,3), (3,1)$ 和右半 $S_2 = (4,5), (5,3), (6,4)$ 。

递归求解左半 S_1 : 左半只有 3 个点, 可以直接暴力计算所有点对距离:

- 点对 $(1,2)$ 和 $(2,3)$: 距离 $\sqrt{(1-2)^2 + (2-3)^2} = \sqrt{1+1} \approx 1.414$
- 点对 $(1,2)$ 和 $(3,1)$: 距离 $\sqrt{(1-3)^2 + (2-1)^2} = \sqrt{4+1} \approx 2.236$
- 点对 $(2,3)$ 和 $(3,1)$: 距离 $\sqrt{(2-3)^2 + (3-1)^2} = \sqrt{1+4} \approx 2.236$ 所以左半最近距离 $d_1 = 1.414$ 。

递归求解右半 S_2 ：同样暴力计算：

- (4,5) 和 (5,3)：距离 $\sqrt{(4-5)^2 + (5-3)^2} = \sqrt{1+4} \approx 2.236$
- (4,5) 和 (6,4)：距离 $\sqrt{(4-6)^2 + (5-4)^2} = \sqrt{4+1} \approx 2.236$
- (5,3) 和 (6,4)：距离 $\sqrt{(5-6)^2 + (3-4)^2} = \sqrt{1+1} \approx 1.414$ 所以右半最近距离 $d_2 = 1.414$ 。

令 $d = \min(d_1, d_2) = 1.414$ 。

合并步骤：现在需要检查是否有跨越左右两部分的点对距离小于 d 。首先，找出所有 x 坐标与中位线 $x = 3.5$ 的距离小于 d 的点。中位线左右各 d 宽，即 x 在 $[3.5 - 1.414, 3.5 + 1.414] \approx [2.086, 4.914]$ 内的点。从排序后的点中筛选：

- 左半中的点：(1,2) 的 $x = 1$ 不在带内，(2,3) 的 $x = 2$ 不在，(3,1) 的 $x = 3$ 在带内。
- 右半中的点：(4,5) 的 $x = 4$ 在带内，(5,3) 的 $x = 5$ 不在 ($5 > 4.914$)，(6,4) 的 $x = 6$ 不在。因此带状区域内的点有：(3,1) 和 (4,5)，还有吗？注意 (2,3) 虽然 $x = 2$ 小于 2.086? $2 < 2.086$ ，所以不在。所以只有这两个点。计算它们之间的距离：(3,1) 与 (4,5) 距离 $\sqrt{(3-4)^2 + (1-5)^2} = \sqrt{1+16} \approx 4.123$ ，大于 d 。所以没有更小的跨越点对。

因此整个点集的最近距离就是 $d = 1.414$ ，对应点对 (1,2) 与 (2,3) 或 (5,3) 与 (6,4)。

这个例子中带状区域内点很少，但实际中可能会多些。下面我们演示一个更典型的情况，说明为什么需要按 y 排序并只检查常数个点。

4.13.7 算法伪代码（改进版）

我们维护两个数组：- P_x ：按 x 坐标升序排列的点列表。- P_y ：按 y 坐标升序排列的点列表。

递归过程要求同时将 P_y 分割为左右子集的 y 序列表，这可以通过一次扫描完成（因为 P_x 已按 x 分割，且已知中位线 x_m ）。

```
Algorithm closestPair( $P_x, P_y$ )
    //  $P_x, P_y$  为列表，长度  $n$ 
    if  $n \leq 3$  then
        return bruteForce( $P_x$ )           // 暴力求解
    mid =  $n // 2$ 
    midX =  $P_x[mid].x$                      // 中位线  $x$  坐标
     $Q_x = P_x[0:mid]$                        // 左半（按  $x$  排序）
     $R_x = P_x[mid:n]$                        // 右半（按  $x$  排序）

    // 将  $P_y$  分割为  $Q_y$  和  $R_y$ ，分别对应左半和右半，并保持各自  $y$  序
     $Q_y = []$ 
     $R_y = []$ 
    for each point  $p$  in  $P_y$ :
        if  $p.x < midX$ :
             $Q_y.append(p)$ 
        else:
             $R_y.append(p)$                  // 当  $x = midX$  时，可任意归入一侧，不影响正确性
```

```

dL = closestPair(Qx, Qy)
dR = closestPair(Rx, Ry)
d = min(dL, dR)

// 带状区域: 所有满足  $|p.x - \text{midX}| < d$  的点, 它们在  $P_y$  中原本有序, 现在仍在  $S_y$  中有序
Sy = [p for p in Py if  $|p.x - \text{midX}| < d$ ]

// 检查带状区域内相邻点对 (最多比较 7 个后继点)
for i = 0 to |Sy|-1:
    for j = i+1 to min(i+7, |Sy|-1):
        // 由于 y 已排序, 一旦 y 差  $\geq d$  即可跳出内层循环
        if Sy[j].y - Sy[i].y  $\geq d$ :
            break
        d = min(d, dist(Sy[i], Sy[j]))
return d

```

注: 常数 7 来源于几何证明: 在 $d \times 2d$ 的矩形内, 彼此距离 $\geq d$ 的点最多有 6 个; 因此对于当前点, 只需检查其后的 6 个点, 取 7 更安全。

4.13.8 代码实现

4.13.8.1 C++ 实现 (推荐教学用)

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include <limits>

using namespace std;

struct Point {
    double x, y;
    Point(double x_ = 0, double y_ = 0) : x(x_), y(y_) {}
};

double dist(const Point& a, const Point& b) {
    double dx = a.x - b.x, dy = a.y - b.y;
    return sqrt(dx*dx + dy*dy);
}

bool cmpX(const Point& a, const Point& b) { return a.x < b.x; }

```

```

bool cmpY(const Point& a, const Point& b) { return a.y < b.y; }

double bruteForce(const vector<Point>& pts, int l, int r) {
    double d = numeric_limits<double>::max();
    for (int i = l; i < r; ++i)
        for (int j = i+1; j < r; ++j)
            d = min(d, dist(pts[i], pts[j]));
    return d;
}

// 递归求解, 要求 ptsX 按 x 排序, ptsY 按 y 排序, 且 ptsY 内的点与 ptsX 相同
double closestPair(const vector<Point>& ptsX, const vector<Point>& ptsY) {
    int n = ptsX.size();
    if (n <= 3) return bruteForce(ptsX, 0, n);
    int mid = n / 2;
    double midX = ptsX[mid].x;

    // 分割 ptsY 为左半 Y 和右半 Y
    vector<Point> leftY, rightY;
    for (const Point& p : ptsY) {
        if (p.x < midX) leftY.push_back(p);
        else rightY.push_back(p);
    }

    // 分割 ptsX
    vector<Point> leftX(ptsX.begin(), ptsX.begin() + mid);
    vector<Point> rightX(ptsX.begin() + mid, ptsX.end());

    double dL = closestPair(leftX, leftY);
    double dR = closestPair(rightX, rightY);
    double d = min(dL, dR);

    // 带状区域 (按 y 已排序)
    vector<Point> strip;
    for (const Point& p : ptsY) {
        if (fabs(p.x - midX) < d) strip.push_back(p);
    }

    // 检查带状区域内的候选点
    int m = strip.size();
    for (int i = 0; i < m; ++i) {
        for (int j = i+1; j < m && (strip[j].y - strip[i].y) < d; ++j) {

```



```

        d = min(d, dist(strip[i], strip[j]));
    }
}
return d;
}

double closestPair(vector<Point>& pts) {
    if (pts.size() <= 1) return 0.0;
    vector<Point> ptsX = pts, ptsY = pts;
    sort(ptsX.begin(), ptsX.end(), cmpX);
    sort(ptsY.begin(), ptsY.end(), cmpY);
    return closestPair(ptsX, ptsY);
}

int main() {
    vector<Point> points = {{1,2},{2,3},{3,1},{4,5},{5,3},{6,4}};
    double ans = closestPair(points);
    cout << " 最近点对距离 = " << ans << endl;
    return 0;
}

```

4.13.8.2 Python 实现

```

import math

def dist(p1, p2):
    return math.hypot(p1[0]-p2[0], p1[1]-p2[1])

def brute_force(pts):
    n = len(pts)
    min_d = float('inf')
    for i in range(n):
        for j in range(i+1, n):
            min_d = min(min_d, dist(pts[i], pts[j]))
    return min_d

def closest_pair(px, py):
    n = len(px)
    if n <= 3:
        return brute_force(px)
    mid = n // 2
    mid_x = px[mid][0]

```

```

# 分割 px
qx = px[:mid]
rx = px[mid:]
# 分割 py (基于 x 坐标与 mid_x 的比较)
qy = []
ry = []
for p in py:
    if p[0] < mid_x:
        qy.append(p)
    else:
        ry.append(p)
dl = closest_pair(qx, qy)
dr = closest_pair(rx, ry)
d = min(dl, dr)
# 带状区域
strip = [p for p in py if abs(p[0] - mid_x) < d]
m = len(strip)
for i in range(m):
    for j in range(i+1, min(i+7, m)):
        if strip[j][1] - strip[i][1] >= d:
            break
        d = min(d, dist(strip[i], strip[j]))
return d

if __name__ == "__main__":
    points = [(1,2),(2,3),(3,1),(4,5),(5,3),(6,4)]
    px = sorted(points, key=lambda p: p[0])
    py = sorted(points, key=lambda p: p[1])
    print(" 最近点对距离 =", closest_pair(px, py))

```

4.13.9 时间复杂度分析

4.13.9.1 递归式

设 $T(n)$ 为处理 n 个点所需的时间。算法分为：

- **划分**：分割 P_y 为两个子列表需 $O(n)$ 时间。
- **递归**：两个子问题各 $T(n/2)$ 。
- **合并**：构造带状区域并检查候选点对。由于带状区域内的点已按 y 排序，内层循环对每个点最多检查常数 k 个后继点（几何证明 $k \leq 7$ ），因此合并代价为 $O(n)$ 。

故递归关系为：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

4.13.9.2 求解

由主定理 ($a = 2, b = 2, f(n) = O(n), \log_b a = 1$, 属于情况二), 得:

$$T(n) = O(n \log n)$$

加上初始排序 ($O(n \log n)$), 总时间复杂度为 $O(n \log n)$ 。该复杂度是渐近最优的 (最近点对问题有 $\Omega(n \log n)$ 代数决策树下界)。

4.13.10 小结

本节给出了最近点对问题的分治算法完整实现 (Python 和 C++)，并详细分析了时间复杂度。该算法是分治策略的经典应用，展示了如何通过预处理和几何限制将合并步骤控制在线性时间，从而达到最优的 $O(n \log n)$ 复杂度。读者应理解并掌握分治、排序与合并技巧，以及常数上界的几何证明思路。

4.14 凸包问题

4.14.1 问题提出

凸包问题 (Convex Hull Problem) 是计算几何中的经典问题，其定义如下：

给定平面上的一个点集 $P = \{p_1, p_2, \dots, p_n\}$ ，要求找到一个**最小的凸多边形**，使得点集 P 中的所有点要么在这个多边形的边界上，要么在其内部。这个凸多边形称为点集 P 的**凸包** (Convex Hull)。

形式化定义：

对于点集 P ，其凸包 $CH(P)$ 是包含 P 中所有点的最小凸集。在二维平面上，凸包是一个凸多边形，其顶点是 P 中的点。

几何直观：可以把点集中的每个点想象成钉在木板上的钉子，那么凸包就是用一个拉紧的橡皮筋将所有钉子围起来所形成的形状。

应用背景：凸包问题在多个领域有广泛应用：

计算机图形学：用于物体的碰撞检测、包围盒计算 **模式识别：**用于形状分析、分类问题 **地理信息系统：**用于区域范围划定、路径规划 **统计学：**用于异常点检测、数据可视化

4.14.2 直观示例

考虑平面上的的一组点： $P = \{(0, 0), (1, 1), (2, 0), (1, -1), (0.5, 0.5), (1.5, 0.5), (1, 0)\}$ 。我们通过观察找出这些点的凸包：

- 首先，最左边的点 $(0, 0)$ 和最右边的点 $(2, 0)$ 显然是凸包上的点。

- 最上方的点 $(1, 1)$ 也很可能位于凸包上。
- 最下方的点 $(1, -1)$ 同样可能位于凸包上。
- 内部的点如 $(0.5, 0.5)$ 、 $(1.5, 0.5)$ 和 $(1, 0)$ 都被包围在上述点构成的多边形内部。

通过连接 $(0, 0) \rightarrow (1, 1) \rightarrow (2, 0) \rightarrow (1, -1) \rightarrow (0, 0)$ 。我们得到一个四边形。检查所有点是否都在这个四边形内部：确实，所有点都位于这个四边形内或边界上。因此，这个四边形就是点集 P 的凸包。

这个示例说明，凸包上的点通常是点集中的“最外层”点，它们构成了一个包围所有点的凸多边形。

4.14.3 算法思想

求解凸包问题有多种算法，本节介绍分治递归算法 (Divide and Conquer Algorithm for Convex Hull)。

分治法的核心思想是将问题分解为规模更小的子问题，递归求解，然后合并结果。对于凸包问题，可以按照以下步骤进行：

1. **排序预处理**：将点集 P 中的所有点按照 x 坐标从小到大排序；如果 x 坐标相同，则按照 y 坐标从小到大排序。排序后，最左边的点 p_0 和最右边的点 p_{n-1} 一定是凸包上的点。
2. **问题分解**：用线段 p_0p_{n-1} 将点集分成两个子集：
 - 上凸包 (上包)：位于线段 p_0p_{n-1} 上方的点集 S_1 (包括可能在线段上的点)
 - 下凸包 (下包)：位于线段 p_0p_{n-1} 下方的点集 S_2 (包括可能在线段上的点)

整个点集的凸包由上包和下包拼接而成。
3. **递归构造上包**：对于上包点集 S_1 ，找到距离线段 p_0p_{n-1} 最远的点 p_{max} 。这个点一定是上包上的一个顶点。连接 p_0p_{max} 和 $p_{max}p_{n-1}$ ，形成两个新的线段。递归地对线段 p_0p_{max} 和 $p_{max}p_{n-1}$ 上方的点集继续构造上包，直到没有点位于线段上方为止。
4. **递归构造下包**：类似地，对于下包点集 S_2 ，找到距离线段 p_0p_{n-1} 最远的点 p_{min} ，递归地对线段 p_0p_{min} 和 $p_{min}p_{n-1}$ 下方的点集构造下包。
5. **合并结果**：将上包和下包的点合并 (注意避免重复端点)，得到完整的凸包。

这种算法也被称为 **QuickHull 算法**，类似于快速排序的分治思想。

4.14.4 算法描述

输入：平面点集 P ，包含 n 个点 **输出**：凸包上的顶点列表 (按逆时针或顺时针顺序排列)

算法步骤

1. **预处理排序**
 - 将点集 P 按照 x 坐标升序排序， x 坐标相同时按 y 坐标升序排序。
 - 得到有序点列表 $p_0, p_1, \dots, p_{n-1}$ ，其中 p_0 是最左下的点， p_{n-1} 是最右上的点。
2. **构造上凸包** (函数 `UpperHull(P, p_left, p_right)`)
 - 输入：点集 P ，左端点 p_{left} ，右端点 p_{right}
 - 输出：从 p_{left} 到 p_{right} 的上凸包顶点序列

具体步骤:

- a. 如果点集 P 为空, 则返回空列表。
- b. 在 P 中找出距离直线 $p_{left} - p_{right}$ 最远的点 p_{max} (即三角形面积最大的点)。
- c. 将 p_{max} 加入上凸包顶点序列。
- d. 找出位于直线 $p_{left} - p_{max}$ 左侧的点集 P_1 (即这些点位于线段 $p_{left} - p_{max}$ 的上方)。
- e. 找出位于直线 $p_{max} - p_{right}$ 左侧的点集 P_2 。
- f. 递归构造:

$$UpperHull(P_1, p_{left}, p_{max}) + [p_{max}] + UpperHull(P_2, p_{max}, p_{right})$$

3. 构造下凸包 (函数 LowerHull(P, p_left, p_right))

- 类似地构造下凸包, 但判断方向相反: 找位于直线右侧的点 (即下方)。

4. 合并凸包

- 上凸包结果 (不包括两端点) + 下凸包结果 (不包括两端点) = 完整凸包。

4.14.5 伪代码

4.14.5.1 主算法

Algorithm ConvexHull(P):

Input: 点集 P (列表, 每个点包含 x, y 坐标)

Output: 凸包顶点列表 (逆时针顺序)

// 1. 排序

sort P by x ascending, then by y ascending

if $|P| \leq 1$ **then return** P

$p_{left} = P[0]$ // 最左下的点

$p_{right} = P[|P|-1]$ // 最右上的点

// 2. 分割为上包点集和下包点集

$S_{upper} = []$ // 位于线段 $p_{left}-p_{right}$ 上方的点

$S_{lower} = []$ // 位于线段 $p_{left}-p_{right}$ 下方的点

for each point p in $P[1..|P|-2]$:

$d = \text{cross}(p_{left}, p_{right}, p)$ // 计算有向面积 (叉积)

if $d > 0$: // 点在直线左侧 (上方)

$S_{upper}.append(p)$

else if $d < 0$: // 点在直线右侧 (下方)

```

        S_lower.append(p)
    // d == 0 的点在线段上, 忽略 (凸包边界上的点会在端点处理)

    // 3. 递归构造上包和下包
    upper = UpperHull(S_upper, p_left, p_right)
    lower = LowerHull(S_lower, p_left, p_right)

    // 4. 合并结果 (避免重复端点)
    return [p_left] + upper + [p_right] + lower

```

4.14.5.2 构造上凸包

```

Algorithm UpperHull(P, p_left, p_right):
    if P is empty:
        return []

    // 找到距离直线 p_left-p_right 最远的点
    p_max = P[0]
    max_dist = 0
    for each point p in P:
        dist = abs(cross(p_left, p_right, p)) // 叉积的绝对值与距离成正比
        if dist > max_dist:
            max_dist = dist
            p_max = p

    // 分割点集
    P1 = [] // 位于直线 p_left-p_max 左侧的点
    P2 = [] // 位于直线 p_max-p_right 左侧的点

    for each point p in P:
        if p == p_max: continue
        if cross(p_left, p_max, p) > 0: // 在左侧
            P1.append(p)
        else if cross(p_max, p_right, p) > 0: // 在左侧
            P2.append(p)

    // 递归构造
    return UpperHull(P1, p_left, p_max) + [p_max] + UpperHull(P2, p_max, p_right)

```

4.14.5.3 构造下凸包

```

Algorithm LowerHull(P, p_left, p_right):
    if P is empty:
        return []

    // 找到距离直线 p_left-p_right 最远的点
    p_min = P[0]
    max_dist = 0
    for each point p in P:
        dist = abs(cross(p_left, p_right, p))
        if dist > max_dist:
            max_dist = dist
            p_min = p

    // 分割点集 (注意方向: 找右侧的点)
    P1 = [] // 位于直线 p_left-p_min 右侧的点
    P2 = [] // 位于直线 p_min-p_right 右侧的点

    for each point p in P:
        if p == p_min: continue
        if cross(p_left, p_min, p) < 0: // 在右侧
            P1.append(p)
        else if cross(p_min, p_right, p) < 0: // 在右侧
            P2.append(p)

    // 递归构造
    return LowerHull(P1, p_left, p_min) + [p_min] + LowerHull(P2, p_min, p_right)

```

4.14.5.4 辅助函数：计算叉积

```

// 计算向量 OA 和 OB 的叉积
// 返回值 > 0 表示 O->A 到 O->B 是逆时针转向 (点 B 在 OA 左侧)
// 返回值 < 0 表示顺时针转向 (点 B 在 OA 右侧)
// 返回值 = 0 表示三点共线
Function cross(O, A, B):
    return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x)

```

4.14.6 代码实现

4.14.6.1 C++ 实现

```

#include <iostream>
#include <vector>
#include <algorithm>

```

```

#include <cmath>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}

    bool operator==(const Point& other) const {
        return x == other.x && y == other.y;
    }
};

// 叉积计算
double cross(const Point& O, const Point& A, const Point& B) {
    return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
}

// 距离平方（避免开方）
double distSq(const Point& A, const Point& B) {
    return (A.x - B.x) * (A.x - B.x) + (A.y - B.y) * (A.y - B.y);
}

// 构造上凸包
vector<Point> upperHull(vector<Point>& P, const Point& left, const Point& right) {
    if (P.empty()) return {};

    // 找到距离直线 left-right 最远的点
    Point far = P[0];
    double maxDist = 0;
    for (const auto& p : P) {
        double d = abs(cross(left, right, p));
        if (d > maxDist) {
            maxDist = d;
            far = p;
        }
    }

    // 分割点集
    vector<Point> P1, P2;
    for (const auto& p : P) {
        if (p == far) continue;
        if (cross(left, far, p) > 0) {

```



```

        P1.push_back(p);
    } else if (cross(far, right, p) > 0) {
        P2.push_back(p);
    }
}

// 递归构造
vector<Point> result;
auto leftPart = upperHull(P1, left, far);
auto rightPart = upperHull(P2, far, right);

result.insert(result.end(), leftPart.begin(), leftPart.end());
result.push_back(far);
result.insert(result.end(), rightPart.begin(), rightPart.end());

return result;
}

// 构造下凸包
vector<Point> lowerHull(vector<Point>& P, const Point& left, const Point& right) {
    if (P.empty()) return {};

    // 找到距离直线 left-right 最远的点
    Point far = P[0];
    double maxDist = 0;
    for (const auto& p : P) {
        double d = abs(cross(left, right, p));
        if (d > maxDist) {
            maxDist = d;
            far = p;
        }
    }

    // 分割点集 (找右侧的点)
    vector<Point> P1, P2;
    for (const auto& p : P) {
        if (p == far) continue;
        if (cross(left, far, p) < 0) {
            P1.push_back(p);
        } else if (cross(far, right, p) < 0) {
            P2.push_back(p);
        }
    }
}

```

```

}

// 递归构造
vector<Point> result;
auto leftPart = lowerHull(P1, left, far);
auto rightPart = lowerHull(P2, far, right);

result.insert(result.end(), leftPart.begin(), leftPart.end());
result.push_back(far);
result.insert(result.end(), rightPart.begin(), rightPart.end());

return result;
}

// 主函数: 求解凸包
vector<Point> convexHull(vector<Point>& points) {
    if (points.size() <= 1) return points;

    // 按 x 坐标排序, x 相同按 y 排序
    sort(points.begin(), points.end(), [](const Point& a, const Point& b) {
        if (a.x != b.x) return a.x < b.x;
        return a.y < b.y;
    });

    Point left = points[0];
    Point right = points[points.size() - 1];

    // 分割为上包点集和下包点集
    vector<Point> upperPoints, lowerPoints;
    for (size_t i = 1; i < points.size() - 1; i++) {
        double d = cross(left, right, points[i]);
        if (d > 0) {
            upperPoints.push_back(points[i]);
        } else if (d < 0) {
            lowerPoints.push_back(points[i]);
        }
        // d == 0 的点在线段上, 忽略
    }

    // 递归构造
    vector<Point> upper = upperHull(upperPoints, left, right);
    vector<Point> lower = lowerHull(lowerPoints, left, right);

```

```

// 合并结果
vector<Point> hull;
hull.push_back(left);
hull.insert(hull.end(), upper.begin(), upper.end());
hull.push_back(right);
hull.insert(hull.end(), lower.begin(), lower.end());

return hull;
}

```

4.14.6.2 Python 实现

```

import math
from functools import cmp_to_key

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __repr__(self):
        return f"({self.x}, {self.y})"

def cross(O, A, B):
    """ 计算向量 OA 和 OB 的叉积 """
    return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x)

def upper_hull(P, left, right):
    """ 构造上凸包 """
    if not P:
        return []

    # 找到距离直线 left-right 最远的点
    far = max(P, key=lambda p: abs(cross(left, right, p)))

    # 分割点集
    P1 = [p for p in P if p != far and cross(left, far, p) > 0]
    P2 = [p for p in P if p != far and cross(far, right, p) > 0]

```

```

# 递归构造
return upper_hull(P1, left, far) + [far] + upper_hull(P2, far, right)

def lower_hull(P, left, right):
    """ 构造下凸包 """
    if not P:
        return []

    # 找到距离直线 left-right 最远的点
    far = max(P, key=lambda p: abs(cross(left, right, p)))

    # 分割点集 (找右侧的点)
    P1 = [p for p in P if p != far and cross(left, far, p) < 0]
    P2 = [p for p in P if p != far and cross(far, right, p) < 0]

    # 递归构造
    return lower_hull(P1, left, far) + [far] + lower_hull(P2, far, right)

def convex_hull(points):
    """ 求解凸包的主函数 """
    if len(points) <= 1:
        return points

    # 按 x 坐标排序, x 相同按 y 排序
    points.sort(key=lambda p: (p.x, p.y))

    left = points[0]
    right = points[-1]

    # 分割为上包点集和下包点集
    upper_points = []
    lower_points = []
    for p in points[1:-1]:
        d = cross(left, right, p)
        if d > 0:
            upper_points.append(p)
        elif d < 0:
            lower_points.append(p)
        # d == 0 的点在线段上, 忽略

    # 递归构造
    upper = upper_hull(upper_points, left, right)

```

```

lower = lower_hull(lower_points, left, right)

# 合并结果
return [left] + upper + [right] + lower

# 测试代码
if __name__ == "__main__":
    points = [
        Point(0, 0), Point(1, 1), Point(2, 0),
        Point(1, -1), Point(0.5, 0.5), Point(1.5, 0.5), Point(1, 0)
    ]

    hull = convex_hull(points)
    print(" 凸包顶点:", hull)
    # 预期输出: [(0, 0), (1, 1), (2, 0), (1, -1)]

```

4.14.7 正确性证明

采用归纳法证明分治凸包算法的正确性。

基础情况：当点集 P 为空时，返回空凸包，正确。当点集 P 只有一个点时，凸包就是该点本身，正确。

归纳假设：假设对于任意规模小于 n 的点集，递归调用能够正确构造出凸包。

归纳步骤：考虑规模为 n 的点集 P ，经过排序后得到最左点 p_0 和最右点 p_{n-1} 。用线段 p_0p_{n-1} 将点集分为上包点集 S_1 和下包点集 S_2 。

我们需要证明三点：

1. 点 p_0 和 p_{n-1} 一定是凸包上的点。
2. 上包中的点（位于线段上方的点）和下包中的点（位于线段下方的点）不可能出现在凸包的另一侧。
3. 递归构造的上包和下包能够正确合并为整个凸包。

证明：

1. p_0 是最左点，任何包含 p_0 的凸多边形必然以 p_0 为顶点；同理 p_{n-1} 是最右点，也必然是顶点。
2. 由凸包的定义，任何位于线段 p_0p_{n-1} 上方的点，如果出现在凸包中，只能出现在上包中；同理下方的点只能出现在下包中。线段 p_0p_{n-1} 本身是凸包的一条“分割线”。
3. 对于上包点集 S_1 ，我们找到距离线段 p_0p_{n-1} 最远的点 p_{max} 。这个点必然是上包的一个顶点，因为它是所有上包点中“最突出”的点。连接 p_0p_{max} 和 $p_{max}p_{n-1}$ ，将上包点集进一步分割。任何在三角形 $p_0p_{max}p_{n-1}$ 内部的点都不可能成为凸包顶点，因此可以忽略。递归地对分割后的点集构造上包，由归纳假设，递归调用正确。同理对下包也成立。

因此，合并后的结果就是整个点集的凸包。证毕。

4.14.8 复杂度分析

时间复杂度：

- 排序预处理需要 $O(n \log n)$ 时间。
- 在每一层递归中，需要遍历当前点集找出最远点并进行分割，这需要 $O(m)$ 时间 (m 为当前点集大小)。
- 递归关系与快速排序类似：最坏情况下，如果每次选择的点都不能很好地分割点集，时间复杂度可能达到 $O(n^2)$ 。但在平均情况下，递归深度为 $O(\log n)$ ，每层处理的总点数为 $O(n)$ ，因此平均时间复杂度为 $O(n \log n)$ 。
- 凸包问题的理论下界是 $\Omega(n \log n)$ ，因此分治算法在平均情况下是最优的。

空间复杂度：

- 递归调用栈的深度为 $O(\log n)$ (平均情况)。
- 每次递归调用需要存储当前点集的副本，因此总体空间复杂度为 $O(n \log n)$ (如果每次都创建新数组) 或通过原地操作优化到 $O(n)$ 。
- 排序需要额外的 $O(\log n)$ 栈空间。

总结：分治递归算法以 $O(n \log n)$ 的平均时间复杂度解决了凸包问题，是计算几何中的经典算法之一。

4.15 棋盘覆盖问题

4.15.1 问题提出

棋盘覆盖问题 (Chessboard Covering Problem) 是一个经典的分治算法问题，其定义如下：

给定一个 $2^k \times 2^k$ 的棋盘 (即棋盘由 2^k 行和 2^k 列组成，总方格数为 4^k)，其中恰好有一个特殊方格 (该方格与其他方格不同，可能是残缺的或标记为特殊)。要求使用 L 型骨牌 (也称为三格骨牌，由三个单位方格组成，形状像字母“L”，共有四种旋转形态) 覆盖棋盘上除特殊方格之外的所有方格。L 型骨牌可以任意旋转，但不能重叠，也不能覆盖到特殊方格上。

形式化定义：棋盘大小为 $2^k \times 2^k$ ，坐标范围从 $(0, 0)$ 到 $(2^k - 1, 2^k - 1)$ 。给定一个特殊方格的位置 (dr, dc) 。需要用若干个 L 型骨牌覆盖棋盘上所有其他方格，每个骨牌恰好覆盖三个方格，且骨牌之间不重叠。输出一种可行的覆盖方案 (通常用骨牌的编号或标记表示)。

应用背景：该问题在多个领域有应用，例如：

算法设计：作为分治策略的典型例题，展示如何将大问题分解为小问题。

图像处理：用于图像分割或压缩中的模式填充。

计算机图形学：在纹理映射或网格生成中可能用到类似的覆盖技术。

4.15.2 直观示例

考虑一个 $2^2 \times 2^2 = 4 \times 4$ 的棋盘，假设特殊方格位于左上角 $(0, 0)$ (即第一行第一列)。我们需要用 L 型骨牌覆盖其余 15 个方格。

L 型骨牌有四种方向：

向左上开口 (覆盖右、下、右下三个方向)

向左下开口

向右上开口

向右下开口

手动尝试覆盖时，我们可以将棋盘分为四个 2×2 的小棋盘。特殊方格位于左上角的子棋盘。在中心位置（即分界线交点）放置一个 L 型骨牌，使其覆盖其他三个子棋盘中靠近中心的一个方格。然后每个子棋盘就变成了一个规模减半的棋盘覆盖问题，且每个子棋盘都有一个特殊方格（对于原来没有特殊方格的子棋盘，我们人为将 L 型骨牌覆盖的那个方格视为特殊方格）。递归地解决每个子棋盘即可。

最终覆盖结果如下图所示（用数字表示骨牌编号）：

```

+---+---+---+---+
| S | 1 | 2 | 2 |
+---+---+---+---+
| 1 | 1 | 3 | 2 |
+---+---+---+---+
| 4 | 4 | 3 | 3 |
+---+---+---+---+
| 4 | 5 | 5 | 5 |
+---+---+---+---+

```

(S 表示特殊方格，相同数字表示同一块 L 型骨牌)

这个示例说明，通过分治法可以系统地构造出覆盖方案。

4.15.3 算法思想

棋盘覆盖问题采用分治递归算法求解。其核心思想是将大棋盘递归地划分为四个小棋盘，分别处理，并在划分时通过放置一块 L 型骨牌将三个没有特殊方格的小棋盘“创造”出新的特殊方格。

具体思路如下：

1. **划分**：将 $2^k \times 2^k$ 的棋盘从中间分成四个 $2^{k-1} \times 2^{k-1}$ 的子棋盘，划分线位于第 2^{k-1} 行和第 2^{k-1} 列。
2. **确定特殊方格所在子棋盘**：检查原始特殊方格位于四个子棋盘中的哪一个。
3. **放置中央骨牌**：在棋盘的中间位置（即划分线的交点附近）放置一块 L 型骨牌，使其覆盖除特殊方格所在子棋盘外的其他三个子棋盘中各一个方格。具体地，根据特殊方格的位置，选择适当的骨牌方向，使得骨牌的三个方格分别位于另外三个子棋盘的角落上（即每个子棋盘中靠近中心的那个方格）。这样，每个子棋盘现在都有一个“特殊方格”：原本的特殊方格在它自己的子棋盘中，而其他三个子棋盘则被中央骨牌覆盖的方格成为了它们新的特殊方格（尽管这些方格实际上被骨牌覆盖了，但在递归处理时，我们将它们视为需要保留的特殊方格，即它们不能被其他骨牌覆盖）。
4. **递归求解**：对四个子棋盘分别递归地执行棋盘覆盖算法，每个子棋盘的大小为 $2^{k-1} \times 2^{k-1}$ ，且每个子棋盘都有一个特殊方格（要么是原来的，要么是中央骨牌覆盖的方格）。
5. **合并**：递归完成后，所有方格都被覆盖，且骨牌之间不重叠。

这种分治策略确保问题规模每次减半，最终在 $k = 0$ （即 1×1 棋盘）时，只有一个方格，如果是特殊方格则无需覆盖，否则（不会出现）但递归基就是大小为 1 时无需操作。

4.15.4 算法描述

输入：

- 棋盘大小参数 k (棋盘为 $2^k \times 2^k$)
- 特殊方格坐标 (dr, dc) (通常行列从 0 开始)
- 当前要处理的子棋盘左上角坐标 (tr, tc) (表示该子棋盘在整个棋盘中的起始位置)

输出：一种覆盖方案，通常用一个二维数组 `board` 表示，其中每个方格记录它所属的骨牌编号 (特殊方格可标记为 0 或特殊值)。

算法步骤

1. **基础情况**：如果当前子棋盘大小为 1×1 (即 $k = 0$)，则无需覆盖，直接返回。
2. **划分**：计算当前子棋盘的中心点，将棋盘划分为四个子棋盘：
 - 左上子棋盘：行范围 $[tr, tr + size/2 - 1]$ ，列范围 $[tc, tc + size/2 - 1]$
 - 右上子棋盘：行范围 $[tr, tr + size/2 - 1]$ ，列范围 $[tc + size/2, tc + size - 1]$
 - 左下子棋盘：行范围 $[tr + size/2, tr + size - 1]$ ，列范围 $[tc, tc + size/2 - 1]$
 - 右下子棋盘：行范围 $[tr + size/2, tr + size - 1]$ ，列范围 $[tc + size/2, tc + size - 1]$ ，其中 $size = 2^k$ 。
3. **确定特殊方格所在子棋盘**：判断原始特殊方格 (dr, dc) 落在哪个子棋盘内。
4. **放置中央骨牌**：根据特殊方格的位置，在四个子棋盘的相邻角落放置一块 L 型骨牌 (骨牌编号记为 `tile`，每放置一块编号递增)。具体地：
 - 如果特殊方格在左上子棋盘，则中央骨牌覆盖以下三个位置：
 - 右上子棋盘的左下角： $(tr + size/2 - 1, tc + size/2)$
 - 左下子棋盘的右上角： $(tr + size/2, tc + size/2 - 1)$
 - 右下子棋盘的左上角： $(tr + size/2, tc + size/2)$
 - 如果特殊方格在右上子棋盘，则中央骨牌覆盖：
 - 左上子棋盘的右下角： $(tr + size/2 - 1, tc + size/2 - 1)$
 - 左下子棋盘的右上角： $(tr + size/2, tc + size/2 - 1)$
 - 右下子棋盘的左上角： $(tr + size/2, tc + size/2)$
 - 如果特殊方格在左下子棋盘，则中央骨牌覆盖：
 - 左上子棋盘的右下角： $(tr + size/2 - 1, tc + size/2 - 1)$
 - 右上子棋盘的左下角： $(tr + size/2 - 1, tc + size/2)$
 - 右下子棋盘的左上角： $(tr + size/2, tc + size/2)$
 - 如果特殊方格在右下子棋盘，则中央骨牌覆盖：
 - 左上子棋盘的右下角： $(tr + size/2 - 1, tc + size/2 - 1)$
 - 右上子棋盘的左下角： $(tr + size/2 - 1, tc + size/2)$
 - 左下子棋盘的右上角： $(tr + size/2, tc + size/2 - 1)$

覆盖时，将这些方格在 `board` 中标记为当前的骨牌编号 `tile`，然后 `tile++`。

5. **递归处理四个子棋盘**：

- 对于每个子棋盘，其新的特殊方格位置可能是原来的（如果该子棋盘包含原始特殊方格）或者是中央骨牌覆盖的方格（对于其他三个子棋盘）。分别调用递归函数：
 - 左上子棋盘：新特殊位置为 (dr, dc) 如果原始特殊在此，否则为 $(tr + size/2 - 1, tc + size/2 - 1)$ （中央骨牌覆盖的左上子棋盘的右下角）
 - 右上子棋盘：新特殊位置为 (dr, dc) 如果原始特殊在此，否则为 $(tr + size/2 - 1, tc + size/2)$ （中央骨牌覆盖的右上子棋盘的左下角）
 - 左下子棋盘：新特殊位置为 (dr, dc) 如果原始特殊在此，否则为 $(tr + size/2, tc + size/2 - 1)$ （中央骨牌覆盖的左下子棋盘的右上角）
 - 右下子棋盘：新特殊位置为 (dr, dc) 如果原始特殊在此，否则为 $(tr + size/2, tc + size/2)$ （中央骨牌覆盖的右下子棋盘的左上角）

6. **结束**：所有递归完成后，棋盘被完全覆盖。

4.15.5 伪代码

假设我们有一个全局二维数组 `board` 用于记录每个方格被哪块骨牌覆盖，初始时特殊方格位置标记为 0（或 -1 表示特殊），其他为 0 表示未覆盖。骨牌编号从 1 开始递增。

```
Algorithm ChessboardCover(tr, tc, dr, dc, size):
  Input:
    tr, tc: 当前子棋盘左上角坐标 (0-based)
    dr, dc: 特殊方格在全局棋盘中的坐标
    size: 当前子棋盘的边长 (2^k)
  Global: board[ ][ ], tile (骨牌编号)

  if size == 1 then
    return // 无需覆盖

  t = tile++ // 取当前骨牌编号并递增
  s = size / 2 // 子棋盘边长

  // 判断特殊方格在哪个子棋盘
  // 注意：这里的 dr, dc 是全局坐标，需要判断是否在当前子棋盘的范围内

  // 1. 左上子棋盘
  if dr < tr + s and dc < tc + s then
    // 特殊方格在左上，则中央骨牌覆盖其他三个子棋盘的角落
    board[tr + s - 1][tc + s] = t // 右上子棋盘的左下角
    board[tr + s][tc + s - 1] = t // 左下子棋盘的右上角
    board[tr + s][tc + s] = t // 右下子棋盘的左上角
    // 递归处理四个子棋盘
    ChessboardCover(tr, tc, dr, dc, s) // 左上（特殊不变）
    ChessboardCover(tr, tc + s, tr + s - 1, tc + s, s) // 右上（新特殊为覆盖点）
    ChessboardCover(tr + s, tc, tr + s, tc + s - 1, s) // 左下（新特殊）
```

```

        ChessboardCover(tr + s, tc + s, tr + s, tc + s, s)    // 右下 (新特殊)
// 2. 右上子棋盘
else if dr < tr + s and dc >= tc + s then
    board[tr + s - 1][tc + s - 1] = t    // 左上子棋盘的右下角
    board[tr + s][tc + s - 1] = t        // 左下子棋盘的右上角
    board[tr + s][tc + s] = t            // 右下子棋盘的左上角
    ChessboardCover(tr, tc, tr + s - 1, tc + s - 1, s)    // 左上 (新特殊)
    ChessboardCover(tr, tc + s, dr, dc, s)                // 右上 (特殊不变)
    ChessboardCover(tr + s, tc, tr + s, tc + s - 1, s)    // 左下 (新特殊)
    ChessboardCover(tr + s, tc + s, tr + s, tc + s, s)    // 右下 (新特殊)
// 3. 左下子棋盘
else if dr >= tr + s and dc < tc + s then
    board[tr + s - 1][tc + s - 1] = t    // 左上子棋盘的右下角
    board[tr + s - 1][tc + s] = t        // 右上子棋盘的左下角
    board[tr + s][tc + s] = t            // 右下子棋盘的左上角
    ChessboardCover(tr, tc, tr + s - 1, tc + s - 1, s)    // 左上 (新特殊)
    ChessboardCover(tr, tc + s, tr + s - 1, tc + s, s)    // 右上 (新特殊)
    ChessboardCover(tr + s, tc, dr, dc, s)                // 左下 (特殊不变)
    ChessboardCover(tr + s, tc + s, tr + s, tc + s, s)    // 右下 (新特殊)
// 4. 右下子棋盘
else
    board[tr + s - 1][tc + s - 1] = t    // 左上子棋盘的右下角
    board[tr + s - 1][tc + s] = t        // 右上子棋盘的左下角
    board[tr + s][tc + s - 1] = t        // 左下子棋盘的右上角
    ChessboardCover(tr, tc, tr + s - 1, tc + s - 1, s)    // 左上 (新特殊)
    ChessboardCover(tr, tc + s, tr + s - 1, tc + s, s)    // 右上 (新特殊)
    ChessboardCover(tr + s, tc, tr + s, tc + s - 1, s)    // 左下 (新特殊)
    ChessboardCover(tr + s, tc + s, dr, dc, s)            // 右下 (特殊不变)
end if

```

初始调用: ChessboardCover(0, 0, dr, dc, 2^k), 且 tile = 1。

4.15.6 代码实现

4.15.6.1 C++ 实现

```

#include <iostream>
#include <vector>
using namespace std;

vector<vector<int>> board; // 全局棋盘, 0 表示未覆盖, -1 表示特殊方格
int tile = 1; // 骨牌编号从 1 开始

void chessboardCover(int tr, int tc, int dr, int dc, int size) {

```

```

if (size == 1) return;

int t = tile++; // 当前骨牌编号
int s = size / 2;

// 左上子棋盘
if (dr < tr + s && dc < tc + s) {
    // 特殊方格在左上, 覆盖其他三个角落
    board[tr + s - 1][tc + s] = t;
    board[tr + s][tc + s - 1] = t;
    board[tr + s][tc + s] = t;

    chessboardCover(tr, tc, dr, dc, s);
    chessboardCover(tr, tc + s, tr + s - 1, tc + s, s);
    chessboardCover(tr + s, tc, tr + s, tc + s - 1, s);
    chessboardCover(tr + s, tc + s, tr + s, tc + s, s);
}

// 右上子棋盘
else if (dr < tr + s && dc >= tc + s) {
    board[tr + s - 1][tc + s - 1] = t;
    board[tr + s][tc + s - 1] = t;
    board[tr + s][tc + s] = t;

    chessboardCover(tr, tc, tr + s - 1, tc + s - 1, s);
    chessboardCover(tr, tc + s, dr, dc, s);
    chessboardCover(tr + s, tc, tr + s, tc + s - 1, s);
    chessboardCover(tr + s, tc + s, tr + s, tc + s, s);
}

// 左下子棋盘
else if (dr >= tr + s && dc < tc + s) {
    board[tr + s - 1][tc + s - 1] = t;
    board[tr + s - 1][tc + s] = t;
    board[tr + s][tc + s] = t;

    chessboardCover(tr, tc, tr + s - 1, tc + s - 1, s);
    chessboardCover(tr, tc + s, tr + s - 1, tc + s, s);
    chessboardCover(tr + s, tc, dr, dc, s);
    chessboardCover(tr + s, tc + s, tr + s, tc + s, s);
}

// 右下子棋盘
else {
    board[tr + s - 1][tc + s - 1] = t;

```

```

        board[tr + s - 1][tc + s] = t;
        board[tr + s][tc + s - 1] = t;

        chessboardCover(tr, tc, tr + s - 1, tc + s - 1, s);
        chessboardCover(tr, tc + s, tr + s - 1, tc + s, s);
        chessboardCover(tr + s, tc, tr + s, tc + s - 1, s);
        chessboardCover(tr + s, tc + s, dr, dc, s);
    }
}

int main() {
    int k = 3; // 8x8 棋盘
    int size = 1 << k; // 2^k
    int dr = 0, dc = 0; // 特殊方格位置, 例如左上角

    board.resize(size, vector<int>(size, 0));
    board[dr][dc] = -1; // 标记特殊方格

    chessboardCover(0, 0, dr, dc, size);

    // 输出结果
    for (int i = 0; i < size; ++i) {
        for (int j = 0; j < size; ++j) {
            cout << board[i][j] << "\t";
        }
        cout << endl;
    }
    return 0;
}

```

4.15.6.2 Python 实现

```

def chessboard_cover(tr, tc, dr, dc, size):
    global board, tile
    if size == 1:
        return

    t = tile
    tile += 1
    s = size // 2

    # 左上子棋盘

```

```

if dr < tr + s and dc < tc + s:
    board[tr + s - 1][tc + s] = t
    board[tr + s][tc + s - 1] = t
    board[tr + s][tc + s] = t

    chessboard_cover(tr, tc, dr, dc, s)
    chessboard_cover(tr, tc + s, tr + s - 1, tc + s, s)
    chessboard_cover(tr + s, tc, tr + s, tc + s - 1, s)
    chessboard_cover(tr + s, tc + s, tr + s, tc + s, s)

# 右上子棋盘
elif dr < tr + s and dc >= tc + s:
    board[tr + s - 1][tc + s - 1] = t
    board[tr + s][tc + s - 1] = t
    board[tr + s][tc + s] = t

    chessboard_cover(tr, tc, tr + s - 1, tc + s - 1, s)
    chessboard_cover(tr, tc + s, dr, dc, s)
    chessboard_cover(tr + s, tc, tr + s, tc + s - 1, s)
    chessboard_cover(tr + s, tc + s, tr + s, tc + s, s)

# 左下子棋盘
elif dr >= tr + s and dc < tc + s:
    board[tr + s - 1][tc + s - 1] = t
    board[tr + s - 1][tc + s] = t
    board[tr + s][tc + s] = t

    chessboard_cover(tr, tc, tr + s - 1, tc + s - 1, s)
    chessboard_cover(tr, tc + s, tr + s - 1, tc + s, s)
    chessboard_cover(tr + s, tc, dr, dc, s)
    chessboard_cover(tr + s, tc + s, tr + s, tc + s, s)

# 右下子棋盘
else:
    board[tr + s - 1][tc + s - 1] = t
    board[tr + s - 1][tc + s] = t
    board[tr + s][tc + s - 1] = t

    chessboard_cover(tr, tc, tr + s - 1, tc + s - 1, s)
    chessboard_cover(tr, tc + s, tr + s - 1, tc + s, s)
    chessboard_cover(tr + s, tc, tr + s, tc + s - 1, s)
    chessboard_cover(tr + s, tc + s, dr, dc, s)

```

测试

k = 3

```

size = 2 ** k
board = [[0] * size for _ in range(size)]
dr, dc = 0, 0 # 特殊方格位置
board[dr][dc] = -1
tile = 1

chessboard_cover(0, 0, dr, dc, size)

# 输出
for row in board:
    for val in row:
        print(f"{val:3d}", end=" ")
    print()

```

4.15.7 正确性证明

采用数学归纳法证明棋盘覆盖分治算法的正确性。

基础情况：当 $k = 0$ 时，棋盘大小为 1×1 ，只有一个方格。如果这个方格是特殊方格，则无需覆盖，算法直接返回，正确；如果不是特殊方格，则此情况不会出现（因为输入保证恰好有一个特殊方格）。所以基础情况成立。

归纳假设：假设对于所有规模小于 2^k （即 $k - 1$ 及以下）的棋盘，算法能够正确覆盖（即用 L 型骨牌覆盖除特殊方格外的所有方格，且不重叠）。

归纳步骤：考虑规模为 $2^k \times 2^k$ 的棋盘，特殊方格位于某处。算法首先将棋盘划分为四个 $2^{k-1} \times 2^{k-1}$ 的子棋盘，并判断特殊方格所在的子棋盘。然后，在中心位置放置一块 L 型骨牌，覆盖其他三个子棋盘中靠近中心的各一个方格。这样，每个子棋盘现在都有一个“特殊”方格：

- 包含原始特殊方格的子棋盘保持原来的特殊方格。
- 其他三个子棋盘中，被 L 型骨牌覆盖的方格成为该子棋盘的“新特殊方格”（尽管这些方格已被骨牌覆盖，但在递归处理时，我们视其为需要保留的空格，实际上递归会继续覆盖它们周围的方格，而它们本身已经被覆盖，因此不会冲突）。

由于 L 型骨牌恰好覆盖三个方格，且这三个方格位于三个不同的子棋盘，它们之间不会重叠。同时，这些方格原本都是空的（未被覆盖），因此放置骨牌是合法的。

接下来，对每个子棋盘递归调用算法。由归纳假设，每个规模为 $2^{k-1} \times 2^{k-1}$ 的子棋盘（且都有一个特殊方格）都能被正确覆盖。递归完成后，所有方格都被覆盖，且各子棋盘的覆盖区域互不相交（因为子棋盘不重叠），中央的 L 型骨牌也与递归覆盖的区域不重叠（因为中央骨牌覆盖的方格在递归中视为特殊方格，递归算法不会再次覆盖它们）。因此，整个棋盘被完全覆盖，且骨牌不重叠。

由归纳法，算法对所有 $k \geq 0$ 成立。

注意：算法中骨牌编号递增，保证了每块骨牌唯一标识，覆盖方案有效。

4.15.8 复杂度分析

时间复杂度：设 $T(n)$ 为覆盖一个边长为 $n = 2^k$ 的棋盘所需的时间。算法将问题分解为 4 个规模为 $n/2$ 的子问题，并在分解时进行常数操作（放置一块骨牌并判断位置）。因此有递推关系：

$$T(n) = 4T(n/2) + O(1)$$

根据主定理 (Master Theorem), $a = 4, b = 2, f(n) = O(1)$, 比较 $n^{\log_b a} = n^{\log_2 4} = n^2$ 与 $f(n) = 1$, 由于 $f(n) = O(n^{2-\epsilon})$ (取 $\epsilon = 1.5$), 故属于主定理的第一种情况, $T(n) = O(n^2)$ 。即算法的时间复杂度与棋盘上的方格数成正比, 为 $O(4^k)$, 这是最优的, 因为总共有 4^k 个方格需要覆盖, 每个方格至少被处理一次。

空间复杂度：递归调用栈的深度为 $k = \log n$, 因为每次递归规模减半。每次递归调用使用常数额外空间, 因此递归栈空间为 $O(\log n)$ 。此外, 需要存储棋盘覆盖结果, 即一个 $n \times n$ 的二维数组, 空间复杂度为 $O(n^2)$ 。因此总体空间复杂度为 $O(n^2) + O(\log n) = O(n^2)$, 即与棋盘大小成正比。

总结：分治递归算法以 $O(n^2)$ 时间（即与方格总数成正比）解决了棋盘覆盖问题, 是最优的算法, 因为必须至少访问每个方格一次。该算法展示了分治策略的典型应用, 通过巧妙地构造子问题, 实现了问题的完美分解与合并。

4.16 求最大最小值问题

4.16.1 问题描述

最大最小值问题 (Min-Max Problem) 定义为：给定一个含有 n 个可比较元素的集合（通常为数组 $A[0..n-1]$ ），要求同时找出其中的最大值和最小值。这是算法设计中一个基础且重要的问题，许多复杂算法（如图论中的剪枝、机器学习中的归一化）都会将其作为子过程调用。

本章讨论基于**比较**的算法，即只允许通过比较两个元素的大小来获取信息。所有算法的时间复杂度将以**比较次数**为主要度量指标。

4.16.2 朴素算法

最直观的方法是分两步进行：

1. 遍历数组，找出最大值，需要 $n - 1$ 次比较。
2. 在剩余 $n - 1$ 个元素中找出最小值，需要 $n - 2$ 次比较。

总比较次数为 $(n - 1) + (n - 2) = 2n - 3$ 。

另一种常见的朴素实现是在一次扫描中同时更新最大值和最小值（每个元素与当前 max 和 min 各比较一次），但这样会对每个元素（包括最终的最大值和最小值）都进行两次比较，导致 $2(n - 1)$ 次比较，比上述方法多 1 次。因此，**更优的朴素策略是先求最大值再求最小值（或反之）。**

算法伪代码（先求最大值，再求最小值）

```

算法 MinMaxNaive(A[0..n-1])
// 输入：数组 A, n ≥ 2
// 输出：最小值 min 和最大值 max
1. // 第一阶段：找出最大值及其下标

```

```

2.  $\text{max} \leftarrow A[0]; \text{maxIdx} \leftarrow 0$ 
3. for  $i \leftarrow 1$  to  $n-1$  do
4.     if  $A[i] > \text{max}$  then
5.          $\text{max} \leftarrow A[i]; \text{maxIdx} \leftarrow i$ 
6. // 第二阶段：在除  $\text{maxIdx}$  以外的元素中找最小值
7. if  $\text{maxIdx} = 0$  then  $\text{min} \leftarrow A[1]$  else  $\text{min} \leftarrow A[0]$ 
8. for  $i \leftarrow 1$  to  $n-1$  do
9.     if  $i \neq \text{maxIdx}$  then
10.        if  $A[i] < \text{min}$  then  $\text{min} \leftarrow A[i]$ 
11. return ( $\text{min}, \text{max}$ )

```

复杂度分析

- 比较次数： $2n - 3$ (当 $n \geq 2$)。当 $n = 1$ 时直接返回 $(A[0], A[0])$ ，无需比较。
- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(1)$ 。

该算法虽然简单，但比较次数仍偏多。当比较操作开销较大（如比较字符串、大整数或自定义对象）或数据规模巨大时，我们希望进一步减少比较次数。

4.16.3 分治法

分治法是一种经典算法设计策略，将原问题划分为规模相近的子问题，递归求解后再合并结果。

算法思想

1. **分解**：将数组从中间分成左右两半，规模分别为 $\lfloor n/2 \rfloor$ 和 $\lceil n/2 \rceil$ 。
2. **递归**：分别递归求出左半部分的最大最小值 $(\text{min}_L, \text{max}_L)$ 和右半部分的最大最小值 $(\text{min}_R, \text{max}_R)$ 。
3. **合并**：全局最小值为 $\min(\text{min}_L, \text{min}_R)$ ，全局最大值为 $\max(\text{max}_L, \text{max}_R)$ 。

递归基：当 $n = 1$ 时，最小值和最大值均为该元素；当 $n = 2$ 时，通过一次比较即可确定最大和最小值。

算法伪代码

```

算法 MinMaxDC(A, low, high)
// 输入：数组 A 的子区间 [low, high] ( $\text{low} \leq \text{high}$ )
// 输出：该区间的最小值和最大值
1. if  $\text{low} = \text{high}$  then
2.     return ( $A[\text{low}], A[\text{low}]$ )
3. if  $\text{low} + 1 = \text{high}$  then
4.     if  $A[\text{low}] < A[\text{high}]$  then return ( $A[\text{low}], A[\text{high}]$ )
5.     else return ( $A[\text{high}], A[\text{low}]$ )
6.  $\text{mid} \leftarrow \lfloor (\text{low} + \text{high})/2 \rfloor$ 
7.  $(\text{minL}, \text{maxL}) \leftarrow \text{MinMaxDC}(A, \text{low}, \text{mid})$ 
8.  $(\text{minR}, \text{maxR}) \leftarrow \text{MinMaxDC}(A, \text{mid}+1, \text{high})$ 
9.  $\text{min} \leftarrow \min(\text{minL}, \text{minR})$ 

```



```

10. max ← max(maxL, maxR)
11. return (min, max)

```

复杂度分析

设 $T(n)$ 为分治法对 n 个元素的比较次数。由算法可得：

$$T(1) = 0, \quad T(2) = 1, \quad T(n) = 2T\left(\frac{n}{2}\right) + 2$$

求解递推式（设 n 为 2 的幂）：

$$T(2^k) = 2T(2^{k-1}) + 2 = 2[2T(2^{k-2}) + 2] + 2 = 4T(2^{k-2}) + 4 + 2 = \dots = 2^{k-1}T(2) + 2(2^{k-2} + 2^{k-3} + \dots + 1)$$

代入 $T(2) = 1$ ：

$$T(2^k) = 2^{k-1} \cdot 1 + 2(2^{k-1} - 1) = 2^{k-1} + 2^k - 2 = 3 \cdot 2^{k-1} - 2 = \frac{3n}{2} - 2$$

当 n 不是 2 的幂时，比较次数仍约为 $\lfloor 3n/2 \rfloor - 2$ 。因此分治法的比较次数约为 $1.5n$ ，优于朴素算法的 $2n - 3$ 。

- 时间复杂度： $O(n)$ 。
- 空间复杂度：递归栈深度 $O(\log n)$ 。

4.16.3.1 C++ 代码实现

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// 返回一个 pair, first 为最大值, second 为最小值
pair<int, int> minMax(const vector<int>& A, int low, int high) {
    // 基础情况：只有一个元素
    if (low == high) {
        return {A[low], A[low]};
    }
    // 基础情况：有两个元素
    if (high == low + 1) {
        if (A[low] <= A[high]) {
            return {A[high], A[low]};
        } else {
            return {A[low], A[high]};
        }
    }
}

```

```

    int mid = low + (high - low) / 2;
    auto [lmax, lmin] = minMax(A, low, mid);
    auto [rmax, rmin] = minMax(A, mid + 1, high);

    int max_val = max(lmax, rmax);
    int min_val = min(lmin, rmin);

    return {max_val, min_val};
}

int main() {
    vector<int> A = {3, 5, 1, 8, 2, 9, 4, 7, 6};
    auto [max_val, min_val] = minMax(A, 0, A.size() - 1);
    cout << " 最大值: " << max_val << ", 最小值: " << min_val << endl;
    return 0;
}

```

4.16.3.2 Python 代码实现

```

def min_max(A, low, high):
    # 基础情况: 只有一个元素
    if low == high:
        return (A[low], A[low])
    # 基础情况: 有两个元素
    if high == low + 1:
        if A[low] <= A[high]:
            return (A[high], A[low])
        else:
            return (A[low], A[high])

    mid = (low + high) // 2
    lmax, lmin = min_max(A, low, mid)
    rmax, rmin = min_max(A, mid + 1, high)

    max_val = max(lmax, rmax)
    min_val = min(lmin, rmin)

    return (max_val, min_val)

# 测试
A = [3, 5, 1, 8, 2, 9, 4, 7, 6]
max_val, min_val = min_max(A, 0, len(A) - 1)

```

```
print(f" 最大值: {max_val}, 最小值: {min_val}")
```

4.16.4 成对比较法（最优迭代算法）

分治法虽然达到 $1.5n$ 量级的比较，但递归开销较大。我们可以通过迭代方式实现相同的比较次数，这就是**成对比较法**（也称分组法、两两比较法）。

算法思想

1. 将数组元素两两配对。如果 n 为奇数，则剩余一个单独元素。
2. 对每一对 (x, y) ，先进行一次比较，确定较大者 $maxPair$ 和较小者 $minPair$ 。
3. 将 $maxPair$ 与当前全局最大值比较，更新最大值；将 $minPair$ 与当前全局最小值比较，更新最小值。
4. 处理可能的奇数剩余元素。

关键步骤：初始值的选取

为减少比较次数，应避免将第一个元素与自身比较。通常先处理前两个元素（若 $n \geq 2$ ）确定初始的 min 和 max ，然后从第三和第四个元素开始成对处理。

算法伪代码

```

算法 MinMaxPairwise(A[0..n-1])
// 输入: 数组 A,  $n \geq 1$ 
// 输出: 最小值 min 和最大值 max
1. if  $n = 1$  then return (A[0], A[0])
2. // 初始化 min 和 max
3. if  $A[0] < A[1]$  then
4.      $min \leftarrow A[0]$ ;  $max \leftarrow A[1]$ 
5. else
6.      $min \leftarrow A[1]$ ;  $max \leftarrow A[0]$ 
7. // 从第 3 个元素开始 (索引 2), 每次处理两个元素
8. for  $i \leftarrow 2$  to  $n-1$  step 2 do
9.     if  $i+1 < n$  then // 有一对
10.        if  $A[i] < A[i+1]$  then
11.            if  $A[i] < min$  then  $min \leftarrow A[i]$ 
12.            if  $A[i+1] > max$  then  $max \leftarrow A[i+1]$ 
13.        else
14.            if  $A[i+1] < min$  then  $min \leftarrow A[i+1]$ 
15.            if  $A[i] > max$  then  $max \leftarrow A[i]$ 
16.        else // 奇数个元素, 最后一个落单
17.            if  $A[i] < min$  then  $min \leftarrow A[i]$ 
18.            if  $A[i] > max$  then  $max \leftarrow A[i]$ 
19. return (min, max)

```

比较次数分析

- 若 n 为偶数：

初始化需要 1 次比较（比较 $A[0]$ 与 $A[1]$ ）。

然后有 $\frac{n-2}{2}$ 对，每对内比较 1 次，然后每对内较大者与 $\max$ 比较，较小者与 $\min$ 比较，即每对 3 次比较。

总比较次数 $= 1 + \frac{n-2}{2} \times 3 = \frac{3n}{2} - 2$ 。

- 若 n 为奇数：

初始化仍处理前两个元素，消耗 1 次比较；然后有 $\frac{n-3}{2}$ 对，每对 3 次比较；最后剩余一个元素，需要与当前的 $\min$ 和 $\max$ 各比较一次，即 2 次额外比较。

总次数 $= 1 + \frac{n-3}{2} \times 3 + 2 = \frac{3n-3}{2}$ 。当 n 为奇数时， $\lceil 3n/2 \rceil - 2 = \frac{3n+1}{2} - 2 = \frac{3n-3}{2}$ ，两者一致。

因此，成对比较法达到比较次数下界 $\lceil 3n/2 \rceil - 2$ （对任意 $n \geq 2$ ）。该算法原地工作，无需额外空间，是工程上的首选。

4.16.5 锦标赛算法

锦标赛算法（Tournament Method）模拟体育淘汰赛制，通过构建一棵完全二叉树来逐步决出最大值，并利用比较历史高效地获得最小值。

算法思想

1. **构建淘汰赛树**：将数组元素作为叶子节点，相邻两两比较，胜者（较大者）进入下一轮，重复直到决出冠军（最大值）。
2. **寻找最小值**：在标准的单棵最大值树中，最小值不一定出现在与最大值直接比较过的元素中。为了同时获得最小值和最大值，通常采用**双向锦标赛**：同时构建两棵树，一棵决出最大值，一棵决出最小值；或者构建一棵树，但每对比较同时记录胜者和败者，分别送入两个不同的数组，然后递归处理。

双向锦标赛算法步骤：

1. 将数组元素两两配对（若 n 为奇数则最后一个元素轮空）。
2. 对每对 (x, y) 进行一次比较，将较大者放入数组 **Winners**，较小者放入数组 **Losers**。
3. 递归地在 **Winners** 中求最大值（这就是全局最大值）。
4. 递归地在 **Losers** 中求最小值（这就是全局最小值）。
5. 若 n 为奇数，将轮空元素与当前最大值和最小值分别比较，更新。

算法伪代码

```

算法 MinMaxTournament(A[0..n-1])
// 输入：数组 A
// 输出：最小值 min，最大值 max
1. if n == 1 then return (A[0], A[0])
2. if n == 2 then
3.     if A[0] < A[1] then return (A[0], A[1])
4.     else return (A[1], A[0])
5. // 配对阶段
6. 创建空数组 Winners, Losers
7. for i ← 0 to n-1 step 2 do
8.     if i+1 < n then
9.         if A[i] < A[i+1] then

```

```

10.         Winners.append(A[i+1]); Losers.append(A[i])
11.         else
12.             Winners.append(A[i]); Losers.append(A[i+1])
13.     else // 奇数个元素，最后一个轮空
14.         Winners.append(A[i]); Losers.append(A[i]) // 或单独处理
15. // 递归求最大值和最小值
16. (minL, maxW) ← MinMaxTournament(Winners) // 注意：Winners 中求 max 即可，但为统一接口
17. (minL, min) ← MinMaxTournament(Losers) // Losers 中求 min
18. // 处理轮空元素（简化逻辑，实际需结合奇数情况）
19. return (min, maxW)

```

(注：上述伪代码为示意，实际实现需仔细处理递归基和奇偶情况。)

比较次数分析：

- 第一轮配对比比较： $\lfloor n/2 \rfloor$ 次。
- 在胜者组（大小 $\lceil n/2 \rceil$ ）中递归求最大值需要 $\lceil n/2 \rceil - 1$ 次比较。
- 在败者组（大小 $\lfloor n/2 \rfloor$ ）中递归求最小值需要 $\lfloor n/2 \rfloor - 1$ 次比较。
- 总比较次数： $\lfloor n/2 \rfloor + (\lceil n/2 \rceil - 1) + (\lfloor n/2 \rfloor - 1)$ 。当 n 为偶数时， $\lfloor n/2 \rfloor = \lceil n/2 \rceil = n/2$ ，总次数 $= n/2 + (n/2 - 1) + (n/2 - 1) = 3n/2 - 2$ 。当 n 为奇数时， $\lceil n/2 \rceil = (n+1)/2$ ， $\lfloor n/2 \rfloor = (n-1)/2$ ，总次数 $= (n-1)/2 + ((n+1)/2 - 1) + ((n-1)/2 - 1) = (3n-3)/2 = \lceil 3n/2 \rceil - 2$ 。因此，锦标赛算法同样达到理论下界。

优缺点：

- 优点：
 - 天然适合**并行计算**：每一轮比较可在不同处理器上同时进行。
 - 便于扩展：可在锦标赛过程中同时记录次大值、次小值等额外信息。
 - 结构清晰，易于理解淘汰制思想。
- 缺点：
 - 需要额外的 $O(n)$ 空间存储树结构（或轮次数组），非原地算法。
 - 实现较成对比较法复杂，常数因子稍大（递归调用、数组拷贝等）。
 - 对于仅需求最大最小值的简单任务，不如成对比较法简洁高效。

与成对比较法的关系：

锦标赛算法本质上是**成对比较法的递归树形表达**。成对比较法采用迭代方式，每对元素处理后立即更新全局最值；而锦标赛算法先将所有配对结果分层存储，再递归处理子组。二者比较次数完全相同，但锦标赛算法的树形结构提供了更丰富的信息。

4.16.6 代码实现

4.16.6.1 C++ 实现

```

#include <iostream>
#include <vector>

```

```

#include <utility> // for pair
#include <climits> // for INT_MAX, INT_MIN (optional)

// 锦标赛算法：同时返回最小值和最大值
std::pair<int, int> minMaxTournament(const std::vector<int>& arr, int left, int right) {
    // 递归基
    if (left == right) {
        return {arr[left], arr[left]};
    }
    if (left + 1 == right) {
        if (arr[left] < arr[right])
            return {arr[left], arr[right]};
        else
            return {arr[right], arr[left]};
    }

    // 配对阶段：将区间内的元素两两比较，产生胜者组和败者组
    std::vector<int> winners, losers;
    for (int i = left; i ≤ right; i += 2) {
        if (i + 1 ≤ right) {
            if (arr[i] < arr[i + 1]) {
                winners.push_back(arr[i + 1]); // 较大者
                losers.push_back(arr[i]);       // 较小者
            } else {
                winners.push_back(arr[i]);
                losers.push_back(arr[i + 1]);
            }
        } else {
            // 奇数个元素，最后一个元素同时加入两组（或单独处理）
            // 简便处理：加入两组，不影响最终结果
            winners.push_back(arr[i]);
            losers.push_back(arr[i]);
        }
    }

    // 递归求胜者组的最大值（即全局最大值）和败者组的最小值（即全局最小值）
    auto [minFromLosers, maxFromWinners] = minMaxTournament(winners, 0, winners.size() - 1);

    // 注意：上面递归返回的 pair 中 first 是败者组的最小值，second 是胜者组的最大值
    // 但我们需要的是：对胜者组只取最大值（递归返回的是该组的 min 和 max，其实我们只需要 max）
    // 对败者组只取最小值。因此还需单独调用两次，或修改递归逻辑。
    // 更清晰的做法：分别对 winners 求最大值，对 losers 求最小值。
    // 为简化，这里改用两个辅助递归函数。

```

```

// 正确实现：分别递归
int globalMax = findMax(winners, 0, winners.size() - 1);
int globalMin = findMin(losers, 0, losers.size() - 1);

return {globalMin, globalMax};
}

// 辅助函数：递归求最大值
int findMax(const std::vector<int>& arr, int left, int right) {
    if (left == right) return arr[left];
    if (left + 1 == right) return std::max(arr[left], arr[right]);
    int mid = left + (right - left) / 2;
    int leftMax = findMax(arr, left, mid);
    int rightMax = findMax(arr, mid + 1, right);
    return std::max(leftMax, rightMax);
}

// 辅助函数：递归求最小值
int findMin(const std::vector<int>& arr, int left, int right) {
    if (left == right) return arr[left];
    if (left + 1 == right) return std::min(arr[left], arr[right]);
    int mid = left + (right - left) / 2;
    int leftMin = findMin(arr, left, mid);
    int rightMin = findMin(arr, mid + 1, right);
    return std::min(leftMin, rightMin);
}

// 对外接口
std::pair<int, int> minMaxTournament(const std::vector<int>& arr) {
    int n = arr.size();
    if (n == 0) return {0, 0}; // 空数组处理
    if (n == 1) return {arr[0], arr[0]};

    // 配对并构建胜者组和败者组
    std::vector<int> winners, losers;
    for (int i = 0; i < n; i += 2) {
        if (i + 1 < n) {
            if (arr[i] < arr[i + 1]) {
                winners.push_back(arr[i + 1]);
                losers.push_back(arr[i]);
            } else {
                winners.push_back(arr[i]);

```

```

        losers.push_back(arr[i + 1]);
    }
} else {
    // 奇数个元素，最后一个同时放入两组（不影响结果）
    winners.push_back(arr[i]);
    losers.push_back(arr[i]);
}
}

int globalMax = findMax(winners, 0, winners.size() - 1);
int globalMin = findMin(losers, 0, losers.size() - 1);
return {globalMin, globalMax};
}

// 示例
int main() {
    std::vector<int> arr = {5, 2, 9, 1, 7, 3, 4};
    auto [minVal, maxVal] = minMaxTournament(arr);
    std::cout << "Min: " << minVal << ", Max: " << maxVal << std::endl;
    return 0;
}

```

4.16.6.2 Python 实现

```

def min_max_tournament(arr):
    """
    锦标赛算法（双向锦标赛）同时求最小值和最大值。
    返回 (min_val, max_val)
    """
    n = len(arr)
    if n == 0:
        return (None, None)
    if n == 1:
        return (arr[0], arr[0])

    # 配对构建胜者组和败者组
    winners = []
    losers = []
    for i in range(0, n, 2):
        if i + 1 < n:
            if arr[i] < arr[i + 1]:
                winners.append(arr[i + 1]) # 较大者
                losers.append(arr[i])      # 较小者

```

```

        else:
            winners.append(arr[i])
            losers.append(arr[i + 1])
    else:
        # 奇数个元素，最后一个元素同时加入两组（不影响最终结果）
        winners.append(arr[i])
        losers.append(arr[i])

# 递归求胜者组的最大值和败者组的最小值
def find_max(lst, l, r):
    if l == r:
        return lst[l]
    if l + 1 == r:
        return max(lst[l], lst[r])
    mid = (l + r) // 2
    left_max = find_max(lst, l, mid)
    right_max = find_max(lst, mid + 1, r)
    return max(left_max, right_max)

def find_min(lst, l, r):
    if l == r:
        return lst[l]
    if l + 1 == r:
        return min(lst[l], lst[r])
    mid = (l + r) // 2
    left_min = find_min(lst, l, mid)
    right_min = find_min(lst, mid + 1, r)
    return min(left_min, right_min)

global_max = find_max(winners, 0, len(winners) - 1)
global_min = find_min(losers, 0, len(losers) - 1)
return (global_min, global_max)

# 更简洁的非递归版本（使用迭代代替递归求最值）
def min_max_tournament_iter(arr):
    n = len(arr)
    if n == 0:
        return (None, None)
    if n == 1:
        return (arr[0], arr[0])

    winners = []

```

```

losers = []
for i in range(0, n, 2):
    if i + 1 < n:
        if arr[i] < arr[i + 1]:
            winners.append(arr[i + 1])
            losers.append(arr[i])
        else:
            winners.append(arr[i])
            losers.append(arr[i + 1])
    else:
        winners.append(arr[i])
        losers.append(arr[i])

# 求最大值（直接使用内置函数，但为展示算法，手动实现）
max_val = winners[0]
for v in winners[1:]:
    if v > max_val:
        max_val = v

min_val = losers[0]
for v in losers[1:]:
    if v < min_val:
        min_val = v

return (min_val, max_val)

# 测试
if __name__ == "__main__":
    arr = [5, 2, 9, 1, 7, 3, 4]
    min_val, max_val = min_max_tournament(arr)
    print(f"Min: {min_val}, Max: {max_val}")
    min_val2, max_val2 = min_max_tournament_iter(arr)
    print(f"Iterative -> Min: {min_val2}, Max: {max_val2}")

```

4.16.7 比较次数的理论下界

定理 4.1: 任何基于比较的算法，在 n 个元素中同时找出最大值和最小值，至少需要 $\lceil 3n/2 \rceil - 2$ 次比较。

证明思路:

每个元素除了最大值和最小值外，都必须至少输掉一次比较（证明它不是最大值）和至少赢得一次比较（证明它不是最小值）。最大元素不输给任何人，最小元素不赢得任何人。因此，除了这两个特殊元素外，其余 $n - 2$ 个元素每个至少参与两次“有意义的比较”（一次作为较大者，一次作为较小者）。下界可由此导出。详细证明可参考算法理论教材。

成对比较法和锦标赛算法恰好达到这一下界，因此它们都是**最优**的（在比较次数意义上）。

4.16.8 算法对比与总结

算法	比较次数（最坏）	是否原地	是否递归	额外空间	适用场景
朴素法	$2n - 3$	是	否	$O(1)$	简单实现，对比较开销不敏感
分治法	$\approx 1.5n$	否	是	$O(\log n)$ （栈）	教学递归，易并行
成对比较法	$\lceil 3n/2 \rceil - 2$	是	否	$O(1)$	工程首选 ，最优且简洁
锦标赛法	$\lceil 3n/2 \rceil - 2$	否	是/可选代	$O(n)$	并行计算，需额外信息（次大/次小）

选择建议： - 若 n 很小（例如 $n \leq 10$ ），朴素法代码简洁且性能差异可忽略。 - 若比较操作昂贵（如比较长字符串、大整数、数据库记录），应首选成对比较法。 - 若需要理解递归思想或为后续并行算法打基础，可采用分治法。 - 若需要同时求解**最大、次大、最小、次小**，或进行硬件并行化设计，可选用锦标赛算法。

4.16.9 习题

- 假设数组 $A = [5, 2, 9, 1, 7, 3]$ ，手动模拟成对比较法的执行过程，统计比较次数。
- 证明：当 $n = 3$ 时，朴素法 ($2n - 3 = 3$ 次比较) 也能达到理论下界，但成对比较法仍为 3 次。请说明两种方法在 $n = 3$ 时的比较过程。
- 实现成对比较法的 C++ 或 Python 函数，并测试其与朴素法的性能差异（可生成随机大数组）。
- 分析如果数组元素是已排序的，上述三种算法的比较次数会变化吗？为什么？
- 修改成对比较法，使其同时返回最大值、最小值以及**次大值**和**次小值**，并分析增加的比较次数。
- 设计一个锦标赛算法的非递归实现，并分析其空间复杂度。

小结：求最大最小值问题看似简单，却蕴含了算法优化的重要思想——通过改变元素处理顺序（成对比较）或采用分治/锦标赛策略，可以在不改变渐进时间复杂度的情况下，将比较常数因子从 2（朴素法的渐近下界）降低到 1.5。成对比较法以其原地、迭代和最优比较次数的特点成为工程实践的首选；而锦标赛算法则因其树形结构在并行计算和扩展信息方面具有独特价值。这些思想在后续学习的锦标赛排序、选择问题等算法中均有体现。

4.17 总结

4.17.1 常见递推关系及解

递推关系	时间复杂度	示例
$T(n) = T(n/2) + O(1)$	$O(\log n)$	二分查找
$T(n) = 2T(n/2) + O(1)$	$O(n)$	求最大最小值
$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	归并排序、逆序对、最近点对、最大子数组
$T(n) = T(n-1) + O(n)$	$O(n^2)$	最坏情况快速排序
$T(n) = 3T(n/2) + O(n)$	$O(n^{\log_2 3})$	Karatsuba 乘法
$T(n) = 7T(n/2) + O(n^2)$	$O(n^{\log_2 7})$	Strassen 矩阵乘法
$T(n) = 4T(n/2) + O(1)$	$O(n^2)$	棋盘覆盖

4.17.2 分治法的适用条件

- 问题可以分解为规模较小的相同子问题；
- 子问题的解可以合并为原问题的解；
- 分治法不要求子问题相互独立，但若子问题大量重叠，则可能效率低下，此时应考虑动态规划。

4.17.3 思考题

1. 二分查找如果数组中有重复元素，如何找到第一个出现的位置？最后一个出现的位置？
2. 汉诺塔问题是否可以用非递归方法实现？尝试用栈模拟。
3. 归并排序的合并过程是否可以在原数组上进行（不使用额外空间）？代价如何？
4. 快速排序的最坏情况如何避免？证明随机化快速排序的期望复杂度为 $O(n \log n)$ 。
5. 实现 Karatsuba 算法，并比较与普通乘法在 n 较大时的性能。
6. 证明在最近点对问题中，每个点最多需要与带状区域中按 y 排序后的 6 个后续点比较。
7. 棋盘覆盖问题中，若 $2^k \times 2^k$ 棋盘有多个特殊方格，是否仍能用 L 型骨牌覆盖？
8. 设计一个分治算法求数组中的最大值和次大值，并分析时间复杂度。

本章小结：分治法通过将问题递归分解，解决了众多经典问题，从简单的二分查找到复杂的矩阵乘法、最近点对等。掌握分治法的关键在于正确划分问题、递归求解和高效合并。本章介绍的问题充分展示了分治法在不同领域的应用，读者应通过练习加深理解，并尝试将分治思想应用于实际问题中。

4.18 习题

4.18.1 选择题

1. 分治法通常包含的三个阶段是 () A. 分解、递归、合并 B. 分、治、合 C. 划分、求解、组合 D. 分割、处理、归并

答案：B

2. 以下算法中，不属于分治算法的是 () A. 归并排序 B. 二分查找 C. 动态规划求解斐波那契数列 D. 快速排序

答案：C（动态规划是另一种算法设计策略）

3. 二分查找算法的时间复杂度是 () A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

答案: B

4. 汉诺塔问题移动 n 个盘子所需的最少步数是 () A. n^2 B. $2n - 1$ C. $2^n - 1$ D. $n!$

答案: C

5. 归并排序在最坏情况下的时间复杂度是 () A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(\log n)$

答案: B

6. 在归并排序的合并过程中, 需要的额外空间复杂度是 () A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

答案: C

7. 快速排序在以下哪种情况下会达到最坏时间复杂度 $O(n^2)$? () A. 数组完全逆序 B. 数组元素全部相等 C. 每次选择的基准都是最小或最大元素 D. 以上都是

答案: D

8. Karatsuba 乘法算法的时间复杂度约为 () A. $O(n^2)$ B. $O(n^{1.585})$ C. $O(n \log n)$ D. $O(n)$

答案: B

9. Strassen 矩阵乘法算法的时间复杂度约为 () A. $O(n^2)$ B. $O(n^{2.807})$ C. $O(n^3)$ D. $O(n \log n)$

答案: B

10. 在快速选择算法中, 期望时间复杂度是 () A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

答案: B

4.18.2 判断题

1. 分治法要求子问题之间必须相互独立。(×)

解析: 分治法不要求子问题相互独立, 但若子问题大量重叠, 效率可能较低。

2. 二分查找算法要求待查找的数组必须是有序的。(√)

3. 汉诺塔问题的递归解法中，移动 n 个盘子需要调用递归函数 $2^n - 1$ 次。(×)

解析：移动步数为 $2^n - 1$ ，但递归调用次数更多。

4. 归并排序是一种稳定的排序算法。(√)

5. 快速排序在任何情况下时间复杂度都是 $O(n \log n)$ 。(×)

解析：最坏情况下为 $O(n^2)$ 。

6. 随机化快速选择算法的最坏情况时间复杂度是 $O(n^2)$ 。(√)

解析：虽然概率极低，但理论上仍存在最坏情况。

7. Median-of-Medians 算法的最坏情况时间复杂度是 $O(n)$ 。(√)

8. 在最近点对问题中，每个点最多只需要与带状区域中按 y 排序后的 7 个后续点比较。(√)

9. 棋盘覆盖问题中，L 型骨牌的总数为 $(4^k - 1)/3$ 。(×)

解析：总方格数为 4^k ，特殊方格 1 个，剩余 $4^k - 1$ 个方格，每块骨牌覆盖 3 个方格，故骨牌数为 $(4^k - 1)/3$ 。

10. 分治法求最大最小值的最坏比较次数优于直接遍历法。(√)

解析：分治法最坏比较次数约为 $3n/2$ ，直接遍历法为 $2n-2$ 。

4.18.3 填空题

1. 分治法的三个典型阶段是：_____、_____、_____。

答案：分 (Divide)、治 (Conquer)、合 (Combine)

2. 二分查找算法在查找失败时返回 _____。

答案：-1

3. 汉诺塔问题移动 n 个盘子所需的最少步数为 _____。

答案： $2^n - 1$

4. 归并排序的合并过程需要 _____ 的额外空间。

答案: $O(n)$

5. 逆序数的分治算法将归并排序与 _____ 相结合。

答案: 逆序对计数

6. 快速排序的核心操作是 _____。

答案: 划分 (Partition)

7. Karatsuba 乘法算法通过将 4 次乘法减少到 _____ 次乘法来提升效率。

答案: 3

8. Strassen 矩阵乘法算法的时间复杂度为 _____。

答案: $O(n^{\log_2 7})$ 或 $O(n^{2.807})$

9. Median-of-Medians 算法通过选择 _____ 作为基准来保证最坏情况线性时间复杂度。

答案: 中位数的中位数

10. 棋盘覆盖问题中, 将 $2^k \times 2^k$ 的棋盘划分为 _____ 个 $2^{(k-1)} \times 2^{(k-1)}$ 的子棋盘。

答案: 4

4.18.4 简答题

1. 简述分治法的基本思想及其三个主要步骤。

答案: 分治法的基本思想是将一个难以直接解决的大问题, 划分成若干个规模较小、性质相同 (或相似) 的子问题, 递归地求解这些子问题, 然后再将子问题的解合并得到原问题的解。三个主要步骤是: - 分 (Divide): 将原问题分解成若干个规模较小的子问题。- 治 (Conquer): 递归地求解各个子问题, 当子问题规模足够小时直接求解。- 合 (Combine): 将子问题的解合并为原问题的解。

2. 解释二分查找算法的原理, 并分析其时间复杂度。

答案: 二分查找的原理: 在有序数组中, 每次取中间位置的元素与目标值比较。若相等则查找成功; 若中间元素大于目标值, 则在左半部分继续查找; 若中间元素小于目标值, 则在右半部分继续查找。如此重复, 直到找到目标或区间为空。时间复杂度为 $O(\log n)$, 因为每次递归将问题规模减半。

3. 归并排序和快速排序都是基于分治法的排序算法，请比较它们的异同点。

答案：- **相同点：**都是分治算法，时间复杂度平均为 $O(n \log n)$ 。- **不同点：**- 归并排序先递归分解再合并，合并需要 $O(n)$ 额外空间；快速排序先划分再递归，是原地排序。- 归并排序最坏情况时间复杂度为 $O(n \log n)$ ；快速排序最坏情况为 $O(n^2)$ 。- 归并排序是稳定排序；快速排序通常不稳定。- 归并排序适合链表排序；快速排序适合数组排序。

4. 简述 Karatsuba 算法的核心思想，并说明它是如何减少乘法次数的。

答案：Karatsuba 算法的核心思想是利用代数恒等式将大整数乘法的 4 次乘法减少到 3 次。设 $x = 10^{(n/2)} \cdot a + b$, $y = 10^{(n/2)} \cdot c + d$, 普通计算需要 ac 、 ad 、 bc 、 bd 四次乘法。Karatsuba 观察到 $ad + bc = (a+b)(c+d) - ac - bd$, 因此只需计算 ac 、 bd 和 $(a+b)(c+d)$ 三次乘法，从而将时间复杂度从 $O(n^2)$ 降至 $O(n^{\log_2 3}) \approx O(n^{1.585})$ 。

5. 什么是逆序数？分治法如何高效计算逆序数？

答案：逆序数是指数组中满足 $i < j$ 且 $A[i] > A[j]$ 的配对 (i, j) 的总数。分治法计算逆序数结合了归并排序的过程：将数组分成左右两半，递归计算左半和右半内部的逆序数，然后在合并两个有序子数组时，若左半当前元素大于右半当前元素，则左半剩余元素都与该右半元素构成逆序对，一次性统计。总逆序数 = 左半逆序数 + 右半逆序数 + 跨越逆序数，时间复杂度为 $O(n \log n)$ 。

6. 解释 Median-of-Medians 算法如何保证最坏情况线性时间复杂度。

答案：Median-of-Medians 算法通过以下步骤选择基准：将数组每 5 个元素一组，找出每组的中位数，然后递归地找出这些中位数的中位数作为基准。可以证明，这样选出的基准至少能排除约 $1/4$ 的元素（即至少有 $n/4$ 个元素小于它， $n/4$ 个元素大于它），因此递归规模最多为 $3n/4$ 。递推式 $T(n) \leq T(n/5) + T(3n/4) + O(n)$ 的解为 $T(n) = O(n)$ 。

7. 简述棋盘覆盖问题的分治策略，并说明为什么该策略能够保证正确覆盖。

答案：棋盘覆盖问题的分治策略：将 $2^k \times 2^k$ 的棋盘划分为四个 $2^{(k-1)} \times 2^{(k-1)}$ 的子棋盘，在中心位置放置一块 L 型骨牌，使除特殊方格所在子棋盘外的三个子棋盘各获得一个新的“特殊方格”，然后递归覆盖四个子棋盘。该策略正确的原因是：每次放置的中央骨牌不重叠，且使每个子棋盘都转化为规模减半的相同问题，递归基础 (1×1 棋盘) 可直接处理，因此能完全覆盖所有非特殊方格。

8. 分治法求最大最小值与直接遍历法相比有何优劣？

答案：- **优势：**分治法的最坏比较次数约为 $3n/2$ ，直接遍历法为 $2n-2$ ，节省了约 25% 的比较次数。当元素比较代价较高时优势明显。- **劣势：**分治法需要递归调用，有额外的栈空间开销 ($O(\log n)$)，实现相对复杂。直接遍历法更简单直观，空间复杂度 $O(1)$ 。

4.18.5 客观编程题

编程题 1：二分查找（递归实现）

题目描述：给定一个按非递减顺序排列的整数数组 `nums` 和一个目标值 `target`，请使用递归的二分查找算法找出 `target` 在数组中的下标（从 0 开始计数）。如果 `target` 不存在于数组中，返回 -1。

输入格式：第一行一个整数 `n`，表示数组长度。第二行 `n` 个整数，表示数组元素（非递减顺序）。第三行一个整数 `target`，表示要查找的目标值。

输出格式：一个整数，表示 `target` 在数组中的下标，若不存在则输出 -1。

样例输入：

```
9
2 5 8 12 16 23 38 45 56
23
```

样例输出：

```
5
```

编程题 2：归并排序

题目描述：给定一个整数数组，请使用归并排序算法将其按非递减顺序排序，并输出排序后的结果。

输入格式：第一行一个整数 `n`，表示数组长度。第二行 `n` 个整数，表示待排序的数组元素。

输出格式：一行 `n` 个整数，表示排序后的数组，相邻元素之间用空格分隔。

样例输入：

```
7
38 27 43 3 9 82 10
```

样例输出：

```
3 9 10 27 38 43 82
```

编程题 3：逆序对计数

题目描述：给定一个整数数组，请计算该数组中的逆序对总数。逆序对定义为：若 $i < j$ 且 $A[i] > A[j]$ ，则 $(A[i], A[j])$ 为一个逆序对。要求使用分治法（基于归并排序）实现，时间复杂度 $O(n \log n)$ 。

输入格式：第一行一个整数 `n`，表示数组长度。第二行 `n` 个整数，表示数组元素。

输出格式：一个整数，表示逆序对的总数。

样例输入：

```
6
3 5 2 1 4 6
```

样例输出：

```
5
```

编程题 4：快速排序

题目描述：给定一个整数数组，请使用快速排序算法（选择最后一个元素作为基准）将其按非递减顺序排序，并输出排序后的结果。

输入格式：第一行一个整数 n ，表示数组长度。第二行 n 个整数，表示待排序的数组元素。

输出格式：一行 n 个整数，表示排序后的数组，相邻元素之间用空格分隔。

样例输入：

```
9
70 74 60 76 83 72 55 65 79
```

样例输出：

```
55 60 65 70 72 74 76 79 83
```

编程题 5：寻找第 k 小元素（快速选择）

题目描述：给定一个整数数组和一个整数 k ($1 \leq k \leq n$)，请使用快速选择算法找出数组中第 k 小的元素。要求平均时间复杂度 $O(n)$ 。

输入格式：第一行两个整数 n 和 k ， n 表示数组长度， k 表示要找第 k 小的元素（1-based）。第二行 n 个整数，表示数组元素。

输出格式：一个整数，表示第 k 小的元素。

样例输入：

```
7 4
9 3 7 1 6 2 8
```

样例输出：

```
6
```

(解释：排序后为 $[1,2,3,6,7,8,9]$ ，第 4 小的元素是 6)

编程题 6：最大子数组和（分治法）

题目描述：给定一个整数数组，请使用分治法找出一个具有最大和的连续子数组（子数组最少包含一个元素），并输出该最大和。

输入格式：第一行一个整数 n ，表示数组长度。第二行 n 个整数，表示数组元素。

输出格式：一个整数，表示最大子数组的和。

样例输入：

```
9
-2 1 -3 4 -1 2 1 -5 4
```

样例输出：

```
6
```

(解释：子数组 $[4,-1,2,1]$ 的和最大，为 6)

编程题 7：汉诺塔问题

题目描述：汉诺塔问题：有三根柱子 A、B、C，A 柱上有 n 个大小不等的圆盘，按从小到大的顺序叠放（最小在上，最大在下）。要求将所有圆盘从 A 柱移动到 C 柱，每次只能移动一个圆盘，且任何时候都不能将大盘放在小盘上面。请输出移动步骤。

输入格式：一个整数 n ，表示圆盘的数量。

输出格式：若干行，每行格式为 “X -> Y”，表示将一个圆盘从 X 柱移动到 Y 柱。

样例输入：

3

样例输出：

A -> C

A -> B

C -> B

A -> C

B -> A

B -> C

A -> C

编程题 8：Karatsuba 乘法

题目描述：实现 Karatsuba 大整数乘法算法。给定两个 n 位正整数（ n 为 2 的幂），计算它们的乘积。要求使用分治策略，时间复杂度 $O(n^{\log_2 3})$ 。

输入格式：第一行一个整数 n ，表示两个整数的位数（ n 为 2 的幂）。第二行两个整数 x 和 y ，中间用空格分隔。

输出格式：一个整数，表示 $x \times y$ 的结果。

样例输入：

4

4265 5378

样例输出：

22932770

编程题 9：最近点对问题

题目描述：给定平面上 n 个点的坐标，请找出距离最近的两个点，并输出它们之间的距离（保留两位小数）。要求使用分治法实现，时间复杂度 $O(n \log n)$ 。

输入格式：第一行一个整数 n ，表示点的个数。接下来 n 行，每行两个整数 x 和 y ，表示点的坐标。

输出格式：一个浮点数，表示最近点对的距离，保留两位小数。

样例输入：

6

1 2

2 3

3 1
4 5
5 3
6 4

样例输出：

1.41

编程题 10：棋盘覆盖问题

题目描述：给定一个 $2^k \times 2^k$ 的棋盘，其中有一个特殊方格（位置为 (dr, dc) ，行列从 0 开始计数）。请使用 L 型骨牌覆盖棋盘上除特殊方格外的所有方格。输出覆盖方案，用一个二维数组表示，特殊方格标记为 -1，其他方格标记为覆盖它的骨牌编号（骨牌编号从 1 开始递增）。

输入格式：第一行一个整数 k ，表示棋盘大小为 $2^k \times 2^k$ 。第二行两个整数 dr 和 dc ，表示特殊方格的行和列。

输出格式：输出 2^k 行，每行 2^k 个整数，表示每个方格被哪块骨牌覆盖（或 -1 表示特殊方格），整数之间用空格分隔。

样例输入：

2
0 0

样例输出：

-1 1 2 2
1 1 3 2
4 4 3 3
4 5 5 5

4.19 实验

4.19.1 基于分治与暴力法的逆序数计算比较

实验目的：1. 理解逆序数的定义及其在算法分析中的意义。2. 掌握用**暴力法**计算逆序数的基本思想与实现。3. 掌握用**归并排序（分治法）**高效计算逆序数的原理与实现。4. 比较两种方法在不同规模输入下的时间复杂度与运行效率。

实验内容：编写程序，对给定整数序列计算其中的逆序对数量。逆序对定义为：在数组 a 中，若 $i < j$ 且 $a[i] > a[j]$ ，则 $(a[i], a[j])$ 为一个逆序对。

要求实现两种方法：1. **暴力法**：双重循环，时间复杂度 $O(n^2)$ 。2. **分治法**：基于归并排序，时间复杂度 $O(n \log n)$ 。

程序需对同一组输入分别用两种方法计算并输出结果，并可选输出运行时间。

实验要求：1. 语言不限（推荐 C/C++/Java/Python）。2. 程序应能处理重复元素（重复元素不视为逆序）。3. 输入规模较大时（如 $n=10^5$ ），暴力法可不执行或自动跳过，仅输出分治法结果。4. 代码结构清晰，关键步骤注释。5. 提交实验报告，包含：- 算法原理简述 - 核心代码 - 测试结果及分析 - 时间复杂度分析

输入格式：第一行一个整数 n ，表示序列长度。

第二行 n 个整数，表示序列元素，以空格分隔。

输出格式：第一行输出暴力法的逆序数（若 $n > 10000$ 可输出“N/A（规模过大）”）。

第二行输出分治法的逆序数。

输入样例：

```
6
3 5 2 1 4 6
```

输出样例：

暴力法逆序数：5

分治法逆序数：5

5 个另外的测试样例：

样例 2（完全升序） 输入：

```
5
1 2 3 4 5
```

输出：

暴力法逆序数：0

分治法逆序数：0

样例 3（完全降序） 输入：

```
4
4 3 2 1
```

输出：

暴力法逆序数：6

分治法逆序数：6

样例 4（含重复元素） 输入：

```
6
2 3 2 1 3 2
```

输出：

暴力法逆序数：9

分治法逆序数：9

样例 5（单个元素） 输入：

```
1
100
```

输出：

暴力法逆序数：0

分治法逆序数：0

样例 6（中等规模） 输入：

10

5 8 2 6 9 1 3 7 4 0

输出：

暴力法逆序数：26

分治法逆序数：26

4.20 最大子段和（输出子数组下标）

4.20.1 题目描述

给定一个长度为 n 的整数序列 $a_1, a_2, \dots, a_n$ ，求该序列中 **连续非空子段** 的最大和，并输出 **该子段的起始下标和结束下标**（下标从 1 开始）。

如果存在多个不同的子段和均为最大值，则选择 **起始下标最小** 的那个；如果起始下标相同，则选择 **结束下标最小** 的那个。

4.20.2 输入格式

- 第一行一个整数 n ($1 \leq n \leq 10^5$)。
- 第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$)。

4.20.3 输出格式

输出一行三个整数，依次为：**最大子段和、起始下标、结束下标**，中间用空格隔开。

4.20.4 样例

4.20.4.1 样例 1

输入

5

1 2 3 4 5

输出

15 1 5

4.20.4.2 样例 2

输入

6

1 -2 3 5 -1 2

输出

9 3 6

4.20.4.3 样例 3

输入

4

-1 -2 -3 -4

输出

-1 1 1

4.20.4.4 样例 4

输入

9

-2 1 -3 4 -1 2 1 -5 4

输出

6 4 7

4.20.4.5 样例 5

输入

1

5

输出

5 1 1

4.20.4.6 样例 6

输入

3

0 0 0

输出

0 1 1

5 动态规划

动态规划是算法设计中最常用的方法之一，其应用遍布众多科学与工程领域：

- **生物信息学**：DNA 序列比对、基因测序中的最长公共子序列问题，本质上是动态规划中的编辑距离计算。
- **运筹学**：资源分配、库存管理、生产调度等优化问题，常通过动态规划获得最优策略。
- **控制科学**：动态规划是求解最优控制问题的基础方法，例如离散系统的贝尔曼最优化原理。
- **计算机科学**：图论中的最短路径（如 Floyd 算法）、自然语言处理中的拼写检查与文本相似度（编辑距离）、AI 强化学习中的价值迭代，以及机器人的路径规划等，都依赖动态规划思想。

例如，编辑距离（Edit Distance）可用于衡量两个字符串的相似程度：在 DNA 序列比较中找出基因变异，在论文查重中检测剽窃行为。这类问题如果用朴素的递归求解，时间复杂度是指数级的；而动态规划可将复杂度降为多项式级，使得实际应用成为可能。

本章将从最基础的斐波那契数列出发，逐步揭示动态规划的核心思想、实现方法及其在典型问题中的运用。

5.1 动态规划的基本思想

动态规划（Dynamic Programming，简称 DP）是一种算法设计方法，其核心思想可以概括为：

动态规划 = 分治递归 + 记忆化存储

分治递归将一个复杂问题分解为若干个规模较小的子问题，分别求解后再合并得到原问题的解。然而，如果这些子问题之间存在大量重叠，分治递归会反复计算相同的子问题，导致效率极低。动态规划通过**存储已经计算过的子问题的解**，当再次遇到同一个子问题时直接返回存储的结果，从而避免重复计算，将指数级的时间复杂度降低为多项式级。

5.1.1 从一个简单例子开始：斐波那契数列

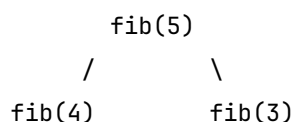
斐波那契数列的定义如下：

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad (n \geq 2)$$

5.1.1.1 分治递归的写法（低效）

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n-1) + fib(n-2);  
}
```

这段代码非常简洁，但它的效率却非常低。以计算 `fib(5)` 为例，其递归调用过程可以用树形结构表示（如 Figure 5.1 所示）。




```

      /      \      /      \
    fib(3)    fib(2) fib(2) fib(1)
    /      \    /      \    /      \
  fib(2) fib(1) ...    ...    ...

```

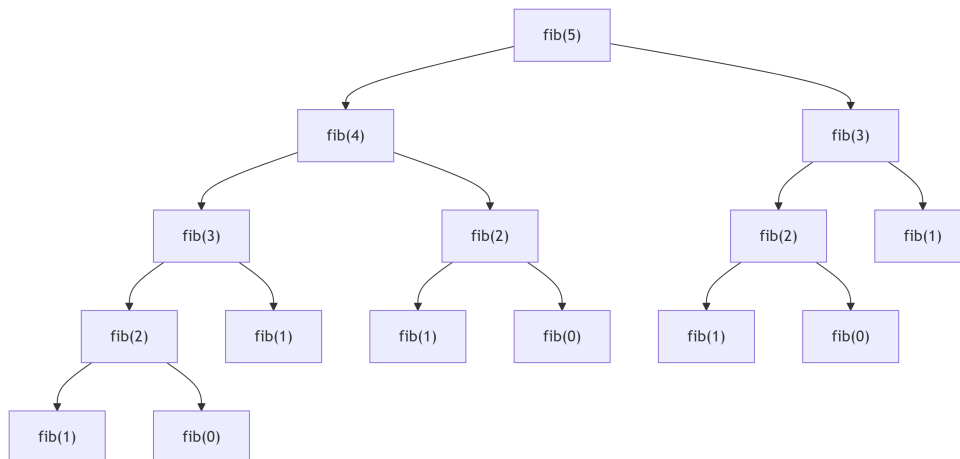


Figure 5.1: 传统递归计算 fib(5) 的调用树

从图中可以看到，fib(3) 被计算了两次，fib(2) 被计算了三次。当 n 更大时，重复计算的次数将急剧增加。可以证明，这种递归方法的时间复杂度为指数级（约 $O(2^n)$ ），当 $n = 50$ 时，计算机也无法在合理时间内完成。

5.1.1.2 问题是什么？——重叠子问题

上述现象被称为**重叠子问题**：在递归求解过程中，相同的子问题会多次出现。如果能够将每个子问题的结果保存起来，下次需要时直接使用，就可以节省大量计算。

5.1.1.3 如何做？——记忆化递归（自顶向下）

我们用一个数组 `f[]` 来记录已经计算过的斐波那契数值，初始时全部设为 -1 表示未计算。在递归函数中，先检查 `f[n]` 是否已经计算过，如果是则直接返回，否则计算并保存结果。

```

int fib(int n, int f[]) {
    if (n <= 1) return n;
    if (f[n] != -1) // 已经计算过
        return f[n];
    f[n] = fib(n-1, f) + fib(n-2, f);
    return f[n];
}

```

调用前需将数组 `f[0..n]` 初始化为 -1。这样一来，每个子问题只计算一次，时间复杂度降为 $O(n)$ ，空间复杂度 $O(n)$ 。

这种先递归、后存储的方式称为**自顶向下**的动态规划，也常被称为“递归加备忘录”。

记忆化递归的实际调用过程与朴素递归完全不同：当一个子问题已经计算并存储后，后续再遇到它时不会继续向下递归，而是直接返回结果。以 `fib(5)` 为例，实际发生的递归调用树如图 @fig-fib_memoized_recursive_tree 所示。

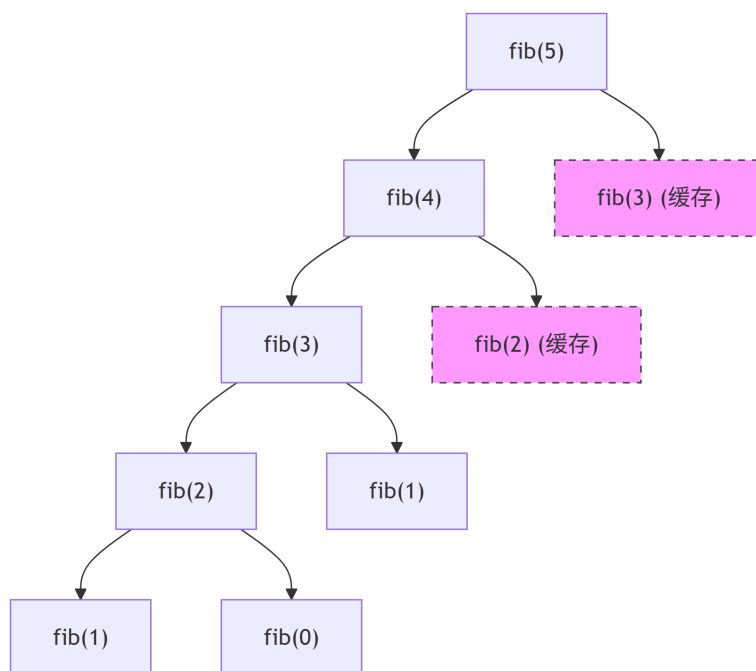


Figure 5.2: 记忆化递归实际调用树

图 5-2 (记忆化递归实际调用树)

5.1.1.4 另一种做法：递推（自底向上）

我们也可以从最小的子问题开始，逐步计算出所有子问题的解。例如，已知 F_0 和 F_1 ，就可以计算出 F_2 ，然后 F_3 ，以此类推，直到 F_n 。

```

int fib(int n) {
    int f[n+1];
    f[0] = 0;
    f[1] = 1;
    for (int i = 2; i <= n; i++)
        f[i] = f[i-1] + f[i-2];
    return f[n];
}
  
```

这种方式称为**自底向上**的动态规划，通常使用循环递推，效率更高，也更容易进行空间优化。

5.1.1.5 空间优化

注意到计算 $f[i]$ 时只需要用到前两个值 $f[i-1]$ 和 $f[i-2]$ ，因此可以用两个变量滚动更新，将空间复杂度降为 $O(1)$ ：

```
int fib(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

这个优化技巧在后续许多动态规划问题中都会用到。

5.1.2 动态规划的一般步骤

从斐波那契数列的例子中，我们可以总结出使用动态规划解决问题的基本步骤：

1. **定义目标函数**：明确问题的数学表示，例如 $f(n)$ 表示输入规模为 n 时问题的答案。
2. **建立递归方程**：将原问题分解为更小的子问题，写出 $f(n)$ 与较小规模 $f(\cdot)$ 之间的关系。
3. **确定基情形**：最小的子问题可以直接求解，无需递归。
4. **选择实现方式**：可以用自顶向下的递归加备忘录，也可以用自底向上的递推填表。
5. **计算并返回结果**：按照递归方程或递推顺序计算，得到原问题的解。

在后续各节中，我们将通过一系列经典问题，进一步体会这些步骤。

5.2 找零钱问题

5.2.1 问题描述

假设我们有若干种不同面额的硬币（每种硬币数量无限），需要凑出指定的金额 E 。问：最少需要多少枚硬币？如果无法凑出，返回 -1。

例如，硬币面额为 $\{11, 5, 1\}$ ，要凑出 15 元。直观地，我们可能会想尽量用大面额的硬币，即先用 11 元，剩下 4 元再用 4 个 1 元，总共 5 枚。但是，是否存在更优的方案？答案是肯定的：用三个 5 元只需要 3 枚硬币。可见，简单贪算法并不总是有效。

为什么贪算法会失效？ 因为局部最优（每次选最大面额）并不能保证全局最优。因此，我们必须系统地考察所有可能的分解方式。

5.2.2 定义目标函数与递归方程

动态规划的本质是带记忆的分治递归。

1. **定义目标函数**：设 $f(e)$ 表示凑出金额 e 所需的最少硬币数。我们最终要求的是 $f(E)$ 。
2. **建立递归方程**：

考虑凑出金额 e 的最后一枚硬币。假设这枚硬币的面额是 a_j ，那么凑出 e 所需的硬币数就等于凑出 $e - a_j$ 所需的最少硬币数再加 1。由于我们不知道最后一枚硬币是哪个，需要枚举所有可能的面额，并取最小值：

$$f(e) = \min\{f(e - a_j) + 1 \mid a_j \leq e\}$$

如果没有任何硬币面额小于等于 e ，则 $f(e)$ 无法定义（可以设为无穷大）。

3. 基情形 (Base Case):

$f(0) = 0$ ，因为凑齐 0 元不需要任何硬币。

这个递归方程的含义是：原问题的最优解可以通过子问题的最优解推导出来，这称为**最优子结构**性质。

5.2.3 自顶向下实现

我们用数组 $f[]$ 存储已经计算过的结果，初始化为 -1 表示未计算。递归函数如下：

C++ 代码实现：

```
#include <vector>
#include <algorithm>
using namespace std;

const int INF = 1e9;

int coinChange(int E, const vector<int>& a, vector<int>& f) {
    if (E == 0) return 0;
    if (f[E] != -1) return f[E];
    int best = INF;
    for (int coin : a) {
        if (E >= coin) {
            int sub = coinChange(E - coin, a, f);
            if (sub != INF && sub + 1 < best)
                best = sub + 1;
        }
    }
    f[E] = best;
    return best;
}

// 调用方式：
// vector<int> f(E+1, -1);
// int ans = coinChange(E, a, f);
// if (ans == INF) ans = -1;
```

Python 代码实现：

```

def coinChange(E, a):
    INF = 10**9
    f = [-1] * (E + 1)

    def dfs(e):
        if e == 0:
            return 0
        if f[e] != -1:
            return f[e]
        best = INF
        for coin in a:
            if e >= coin:
                sub = dfs(e - coin)
                if sub != INF and sub + 1 < best:
                    best = sub + 1
        f[e] = best
        return best

    ans = dfs(E)
    return -1 if ans == INF else ans

```

调用前需将 $f[0 \dots E]$ 初始化为 -1, INF 是一个足够大的数 (例如 10^9)。最终如果 $f[E]$ 仍是 INF, 说明无法凑出, 返回 -1。

5.2.4 自底向上实现

从 $e = 0$ 开始, 逐步计算到 $e = E$, 保证计算 $f[e]$ 时所有需要的 $f[e - a_j]$ 都已经算好。

C++ 代码实现:

```

#include <vector>
#include <algorithm>
using namespace std;

const int INF = 1e9;

int coinChange(int E, const vector<int>& a) {
    vector<int> f(E + 1, INF);
    f[0] = 0;
    for (int e = 1; e <= E; e++) {
        for (int coin : a) {
            if (e >= coin && f[e - coin] != INF) {
                f[e] = min(f[e], f[e - coin] + 1);
            }
        }
    }
}

```

```

    }
    return f[E] == INF ? -1 : f[E];
}

```

Python 代码实现:

```

def coinChange(E, a):
    INF = 10**9
    f = [INF] * (E + 1)
    f[0] = 0
    for e in range(1, E + 1):
        for coin in a:
            if e >= coin and f[e - coin] != INF:
                f[e] = min(f[e], f[e - coin] + 1)
    return -1 if f[E] == INF else f[E]

```

这个递推过程可以看作是一张表，从 $e = 1$ 到 E 逐行填写。例如，硬币面额 $\{1, 3, 4\}$ ，要凑 6 元，填表过程如下：

e	$f[e]$	说明
0	0	基情形
1	$\min(f[0] + 1) = 1$	用 1 元硬币
2	$\min(f[1] + 1) = 2$	用两个 1 元
3	$\min(f[2] + 1 = 3, f[0] + 1 = 1) = 1$	用一枚 3 元硬币
4	$\min(f[3] + 1 = 2, f[0] + 1 = 1) = 1$	用一枚 4 元硬币
5	$\min(f[4] + 1 = 2, f[2] + 1 = 3) = 2$	1+4 或 4+1
6	$\min(f[5] + 1 = 3, f[3] + 1 = 2, f[2] + 1 = 3) = 2$	用两个 3 元

最终 $f[6] = 2$ ，表示凑 6 元最少需要 2 枚硬币（两个 3 元）。

5.2.5 输出最优方案

除了知道最少硬币数，有时还需要知道具体用了哪些硬币。为此，可以在递推过程中增加一个数组 $p[e]$ ，记录在凑 e 时最后选用的硬币面额（或硬币编号）。然后在计算完 $f[E]$ 后，从 E 开始回溯。

C++ 代码实现:

```

#include <vector>
#include <algorithm>
#include <cstdio>
using namespace std;

const int INF = 1e9;

int coinChange(int E, const vector<int>& a) {
    vector<int> f(E + 1, INF);

```

```

vector<int> p(E + 1, -1);
f[0] = 0;
for (int e = 1; e <= E; e++) {
    for (int j = 0; j < (int)a.size(); j++) {
        int coin = a[j];
        if (e >= coin && f[e - coin] + 1 < f[e]) {
            f[e] = f[e - coin] + 1;
            p[e] = j;          // 记录最后选的硬币编号
        }
    }
}
if (f[E] == INF) return -1;
// 回溯输出方案
int e = E;
while (e > 0) {
    printf("%d ", a[p[e]]);
    e -= a[p[e]];
}
printf("\n");
return f[E];
}

```

Python 代码实现:

```

def coinChange(E, a):
    INF = 10**9
    f = [INF] * (E + 1)
    p = [-1] * (E + 1)
    f[0] = 0
    for e in range(1, E + 1):
        for j, coin in enumerate(a):
            if e >= coin and f[e - coin] + 1 < f[e]:
                f[e] = f[e - coin] + 1
                p[e] = j
    if f[E] == INF:
        return -1
    # 回溯输出方案
    e = E
    while e > 0:
        print(a[p[e]], end=' ')
        e -= a[p[e]]
    print()
    return f[E]

```

例如，对 $E = 6$ 和面额 $\{1, 3, 4\}$ ，回溯过程为：p[6]=1（面额 3）， e 变为 3；p[3]=1（面额 3）， e 变为 0。输出“3 3”。

5.2.6 小结

找零钱问题展示了动态规划如何通过递归方程将原问题与子问题联系起来，并通过存储子问题的解来避免重复计算。它也是一个典型的**最优子结构**问题：凑出 e 的最优解必然包含凑出 $e - a_j$ 的最优解。

5.3 机器人路径问题

5.3.1 问题描述

一个机器人位于一个 $m \times n$ 网格的左上角（起点），每次只能向右或向下移动一步，问到达右下角（终点）有多少条不同的路径？

这是 LeetCode 第 62 题，通常被称为“不同路径”问题。图 Figure 5.3 展示了一个 $m = 3, n = 7$ 的网格示例，机器人从起点 Start 出发，沿着向右或向下的方向移动，最终要到达终点 Finish。

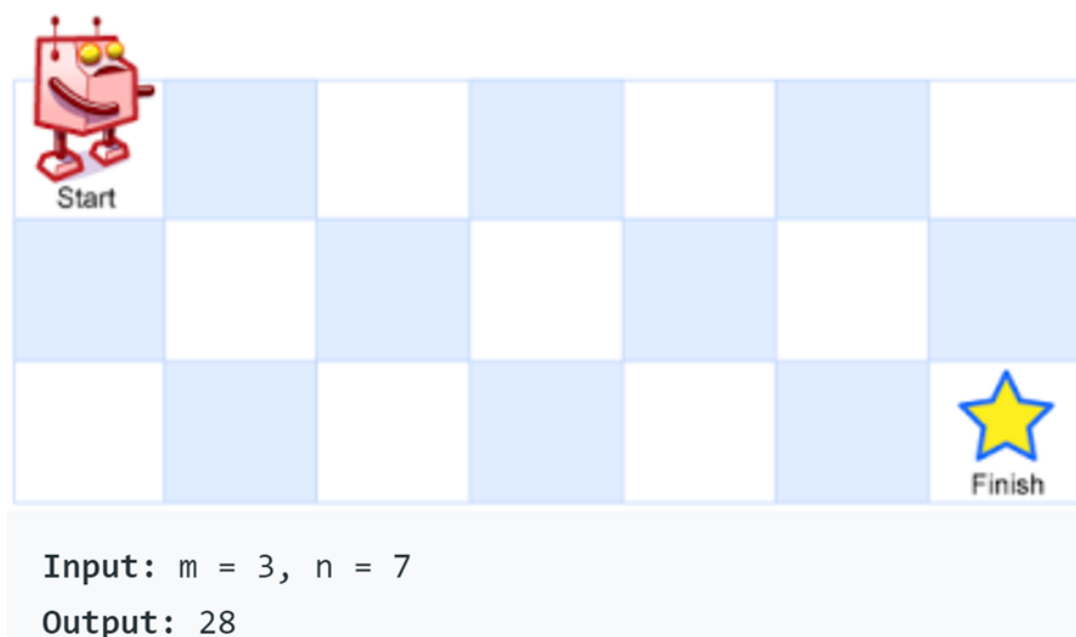


Figure 5.3: 一个 3×7 的网格，机器人从左上角 Start 出发，每次只能向右或向下移动一步，到达右下角 Finish

5.3.2 定义目标函数

设 $f(i, j)$ 表示从起点 $(1, 1)$ 走到格子 (i, j) 的不同路径数。由于机器人只能向右或向下移动，因此到达 (i, j) 只能从上方 $(i - 1, j)$ 或左方 $(i, j - 1)$ 过来。因此：

$$f(i, j) = f(i - 1, j) + f(i, j - 1)$$

基情形：第一行的所有格子只能从左边过来，所以 $f(1, j) = 1$ （只有一直向右一种走法）；第一列的所有格子只能从上边过来，所以 $f(i, 1) = 1$ （只有一直向下一种走法）。

5.3.3 自底向上实现（二维表）

我们可以用一个二维数组 f 来存储每个格子的路径数，按照行优先或列优先的顺序计算。

C++ 代码实现：

```
#include <vector>
using namespace std;

int uniquePaths(int m, int n) {
    // 创建 (m+1) x (n+1) 的二维表，下标从 1 开始使用
    vector<vector<int>> f(m + 1, vector<int>(n + 1, 0));
    for (int j = 1; j <= n; j++) f[1][j] = 1;
    for (int i = 1; i <= m; i++) f[i][1] = 1;
    for (int i = 2; i <= m; i++) {
        for (int j = 2; j <= n; j++) {
            f[i][j] = f[i-1][j] + f[i][j-1];
        }
    }
    return f[m][n];
}
```

Python 代码实现：

```
def uniquePaths(m: int, n: int) -> int:
    # 创建 (m+1) x (n+1) 的二维列表，下标从 1 开始使用
    f = [[0] * (n + 1) for _ in range(m + 1)]
    for j in range(1, n + 1):
        f[1][j] = 1
    for i in range(1, m + 1):
        f[i][1] = 1
    for i in range(2, m + 1):
        for j in range(2, n + 1):
            f[i][j] = f[i-1][j] + f[i][j-1]
    return f[m][n]
```

例如，当 $m = 3, n = 7$ 时，填表结果如表 5-1 所示（只展示部分行和列）。

表 5-1 机器人路径填表 ($m = 3, n = 7$)

$i \backslash j$	1	2	3	4	5	6	7
1	1	1	1	1	1	1	1
2	1	2	3	4	5	6	7

$i \backslash j$	1	2	3	4	5	6	7
3	1	3	6	10	15	21	28

最终 $f[3][7] = 28$ ，即答案。

5.3.4 空间优化

观察递推过程，计算第 i 行时，只需要用到第 $i - 1$ 行（上一行）和第 i 行左边已经计算好的值（当前行）。因此，我们可以只用一维数组 $f[j]$ 来同时存储两行的信息：**在从左到右扫描的过程中， $f[j]$ 左边的元素（下标小于 j ）已经是当前行的新值，而 $f[j]$ 及其右边的元素仍然保留着上一行的旧值。**初始时， $f[j]$ 代表第一行的值（全是 1）。然后从第二行开始，从左到右更新 $f[j]$ ，更新后的 $f[j]$ 就变成了当前行的值。

C++ 代码实现：

```
#include <vector>
using namespace std;

int uniquePaths(int m, int n) {
    vector<int> f(n, 1);          // 第一行全是 1
    for (int i = 1; i < m; i++) { // 从第二行开始
        for (int j = 1; j < n; j++) {
            f[j] = f[j] + f[j-1]; // 上一行的 f[j] 加上左边的 f[j-1]
        }
    }
    return f[n-1];
}
```

Python 代码实现：

```
def uniquePaths(m: int, n: int) -> int:
    f = [1] * n          # 第一行全是 1
    for i in range(1, m): # 从第二行开始
        for j in range(1, n):
            f[j] = f[j] + f[j-1] # 上一行的 f[j] 加上左边的 f[j-1]
    return f[n-1]
```

这个优化技巧将空间复杂度从 $O(mn)$ 降为 $O(n)$ 。理解这一过程的关键是： $f[j-1]$ 在本行已被更新，**是当前行左边格子的路径数**； $f[j]$ 尚未更新，**是上一行同列格子的路径数**。两者相加即得当前格子的总路径数，结果写回 $f[j]$ 后， $f[j]$ 就变成了当前行的新值。详细的变化过程见下一小节（5.3.5）。

5.3.5 理解优化过程

以 $m = 3, n = 7$ 为例，跟踪一维数组 f 的变化。数组下标从 0 到 6。**核心理解**：在每一行的计算中，当我们从左到右遍历 ' j ' 时，已更新部分（下标小于 j ）存储的是**当前行的新值**（图中用**黑体**表示），未更新部分（下标 $\geq j$ ）仍保留着上一行的旧值（普通字体）。随着 j 增大，黑体区域向右扩展。

约定：黑体数字表示当前行的值；普通数字表示上一行的值。

5.3.5.1 第一步：初始状态（第一行）

第一行所有格子只有一种走法，整个数组都是第一行的值。此时尚未开始计算第二行，因此全部视为上一行（即第一行）的值。但注意， $f[0]$ 在后续会被直接继承为第二行第一列的值（始终为 1），为了统一，我们仍将初始状态所有元素视为普通字体（上一行）：

$f = [1, 1, 1, 1, 1, 1, 1]$ （全为普通字体，表示上一行的值）

5.3.5.2 第二步：处理第二行（ $i = 1$ ）

在开始更新第二行之前，数组状态为： **$f[0]$** 已经代表当前行（第二行）第一列的值（始终为 1），而 $f[1]$ 到 $f[6]$ 仍是上一行（第一行）的值。因此，更新前的数组应表示为：

$f = [1, 1, 1, 1, 1, 1, 1]$

（ $f[0]$ 用黑体表示当前行值，其余普通字体表示上一行值）

依次更新 $j = 1$ 到 6，规则： $f[j] = f[j] + f[j-1]$ 。等号右边的 $f[j]$ 是上一行旧值（普通）， $f[j-1]$ 是已经更新过的当前行新值（黑体）。

- 更新 $j=1$: $f[1] = \text{上一行 } f[1] (1) + \text{左边的 } f[0] (1) = 2$
更新后: $f = [1, 2, 1, 1, 1, 1, 1]$

- $j=2$: $f[2] = \text{上一行 } f[2] (1) + \text{左边的 } f[1] (2) = 3 \rightarrow$
 $f = [1, 2, 3, 1, 1, 1, 1]$

- $j=3$: $f[3] = 1 + 3 = 4 \rightarrow [1, 2, 3, 4, 1, 1, 1]$

- $j=4$: $f[4] = 1 + 4 = 5 \rightarrow [1, 2, 3, 4, 5, 1, 1]$

- $j=5$: $f[5] = 1 + 5 = 6 \rightarrow [1, 2, 3, 4, 5, 6, 1]$

- $j=6$: $f[6] = 1 + 6 = 7 \rightarrow [1, 2, 3, 4, 5, 6, 7]$

此时所有元素均为黑体，表示整个数组已经存储了第二行的计算结果。

5.3.5.3 第三步：处理第三行（ $i = 2$ ）

在开始更新第三行之前，当前数组存储的是第二行的结果。此时， **$f[0]$** 已经自动成为第三行第一列的值（仍为 1），而 $f[1]$ 到 $f[6]$ 是上一行（第二行）的值。因此，更新前的数组表示为：

$f = [1, 2, 3, 4, 5, 6, 7]$

（ $f[0]$ （黑体）为当前行值， $f[1..6]$ （普通字体）为上一行值）

再次从左到右更新：

- 更新 $j=1$: $f[1] = \text{上一行 } f[1] (2) + \text{左边的 } f[0] (1) = 3 \rightarrow$
 $f = [1, 3, 3, 4, 5, 6, 7]$

- $j=2$: $f[2] = \text{上一行 } f[2] (3) + \text{左边的 } f[1] (3) = 6 \rightarrow$
 $f = [1, 3, 6, 4, 5, 6, 7]$

- $j=3$: $f[3] = \text{上一行 } f[3] (4) + \text{左边的 } f[2] (6) = 10 \rightarrow$
 $f = [1, 3, 6, 10, 5, 6, 7]$

- $j=4$: $f[4] = 5 + 10 = 15 \rightarrow [1, 3, 6, 10, 15, 6, 7]$
- $j=5$: $f[5] = 6 + 15 = 21 \rightarrow [1, 3, 6, 10, 15, 21, 7]$
- $j=6$: $f[6] = 7 + 21 = 28 \rightarrow [1, 3, 6, 10, 15, 21, 28]$

最终所有元素变为黑体，第三行计算完毕。 $f[6] = 28$ 即为 3×7 网格的不同路径总数。

5.4 动态规划的两个重要特征

通过前面的斐波那契数列、找零钱、机器人路径三个例子，我们已经感受到动态规划的魅力。现在归纳出动态规划适用的两个核心特征。

5.4.1 重叠子问题

定义：在递归求解过程中，相同的子问题会被多次重复计算。

- 在斐波那契数列的朴素递归中， $\text{fib}(3)$ 被计算了 2 次， $\text{fib}(2)$ 被计算了 3 次。当 n 增大时，重复次数呈指数级增长。
- 在找零钱问题中，凑金额 e 需要子问题 $e - a_j$ ，不同的 e 可能依赖同一个更小的子问题（例如凑 6 元和凑 5 元都可能依赖凑 3 元）。
- 在机器人路径问题中，计算 $f(3, 3)$ 需要 $f(2, 3)$ 和 $f(3, 2)$ ，而计算 $f(2, 4)$ 也需要 $f(2, 3)$ ，因此 $f(2, 3)$ 被多个父问题共享。

如何识别：画出递归树，如果相同的节点出现多次，说明存在重叠子问题。**注意：**并非所有递归都有重叠子问题。例如，归并排序每次将数组分成两个互不相交的子数组，左右两半没有任何重叠，因此不适用动态规划（分治递归已足够）。

5.4.2 最优子结构

定义：原问题的最优解可以由子问题的最优解组合而成。

- **找零钱问题：**凑金额 e 的最少硬币数，等于某个 $e - a_j$ 的最少硬币数再加 1。这里 $e - a_j$ 的“最少”是子问题的最优解，我们相信用它构造出的 e 的解也是最优的。这是典型的最优子结构。
- **机器人路径问题：**到达 (i, j) 的不同路径数，等于到达 $(i-1, j)$ 和 $(i, j-1)$ 的路径数之和。虽然这里不是“最优化”问题（是计数问题），但其子问题的解同时是最优的（只有一种方式能到达每个格子，且数量唯一），因此同样满足子问题最优叠加的性质。
- **反例：**求从 A 到 B 的最长简单路径（不允许重复顶点，路径长度以边数计）。如图 Figure 5.4 所示的无向图中，A 为起点，B 为终点。图中边为：A—C, A—D, C—D, C—B, D—B。
 - 从 A 到 B 的最长简单路径长度为 3，例如 A—C—D—B 或 A—D—C—B。
 - 考虑子问题：从 A 到 C 的最长简单路径是 A—D—C（长度为 2，经过 D），从 C 到 B 的最长简单路径是 C—D—B（长度为 2，也经过 D）。
 - 如果尝试将这两段最优子路径拼接，会得到 A—D—C—D—B，其中节点 D 重复，不是简单路径。
 - 而全局最优路径 A—C—D—B 中，从 A 到 C 的子路径是直接边 A—C（长度 1），并不是 A 到 C 的最长路径；从 C 到 B 的子路径是 C—D—B（长度 2），是 C 到 B 的最长路径。反之，另一条全局最优路径 A—D—C—B 中，从 A 到 C 的子路径是 A—D—C（长度 2，是最长），但从 C 到 B 的子路径是直接边 C—B（长度 1），不是最长。

因此，全局最优解并不要求每一段子路径都是子问题的最优解。该问题不具有最优子结构，不能用动态规划求解。

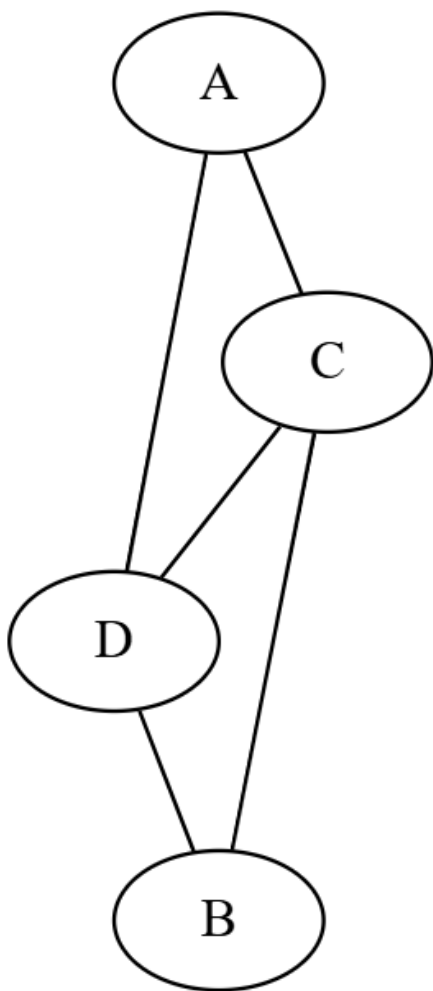


Figure 5.4: 最长简单路径反例：A 到 C 的最长路径（绕行 D、Y、X）与 C 到 B 的潜在路径冲突

如何判断最优子结构：考虑是否能够将原问题分解为若干个子问题，且原问题的最优解依赖于子问题的最优解。一个重要的前提是子问题的解具有“**无后效性**”——子问题一旦求解完毕，其结果不会因为后续决策而改变。在找零钱问题中，凑 $e - a_j$ 的最少硬币数是一个确定的值，与之后如何凑出 e 无关；在机器人路径问题中，到达某个格子 (i, j) 的路径数也是固定的，不受后续路径选择的影响。正因为这种无后效性，我们才可以用子问题的最优解直接构造原问题的最优解。反之，如果子问题的解会被后面决策影响（例如某些路径规划中，经过某个节点后禁止使用某些边），那么最优子结构可能不成立。

5.4.3 两个特征的关系

特征	作用	不满足的后果
重叠子问题	决定能否节省时间（需要储存）	若没有重叠，分治递归即可，DP 无性能优势
最优子结构	决定能否正确递推（子问题最优 $\rightarrow$ 全局最优）	若不满足，DP 给出的解可能是错误的

动态规划同时需要这两个特征。缺一不可：只有重叠子问题没有最优子结构，存储的结果无法正确组合；只有最优子结构没有重叠子问题，虽然可以用递归，但没必要用 DP（例如归并排序）。

5.4.4 小结

当你遇到一个新问题时，可以按以下思路判断是否适用动态规划：

1. 尝试写出递归解法，检查是否存在重复的子问题（重叠子问题）。
2. 验证最优子结构：全局最优解能否由子问题的最优解构造出来？
3. 如果两个条件都满足，通常就可以用动态规划来优化。

5.5 0-1 背包问题

5.5.1 问题描述

有 n 个物品，每个物品有重量 w_i 和价值 v_i 。现有一个背包，容量为 W 。问如何选择物品装入背包，使得总价值最大，且每个物品最多选一次（即 0-1 背包）。

这是一个经典的组合优化问题，直接枚举所有子集的时间复杂度为 $O(2^n)$ ，当 n 较大时不可行。动态规划可以高效求解。

5.5.2 定义目标函数

设 $F(i, w)$ 表示考虑前 i 个物品（即物品 1 到 i ），在背包容量为 w 时能获得的最大价值。最终答案是 $F(n, W)$ 。

对于第 i 个物品，有两种可能的选择：- **不选**：背包容量仍为 w ，最大价值等于考虑前 $i - 1$ 个物品时的价值，即 $F(i - 1, w)$ 。- **选**：前提是当前容量 w 至少为 w_i 。此时背包剩余容量为 $w - w_i$ ，价值为 $v_i + F(i - 1, w - w_i)$ 。

显然，我们应该取这两种情况中价值较大的一个。因此递归方程为：

$$F(i, w) = \begin{cases} F(i-1, w) & \text{if } w < w_i \\ \max\{F(i-1, w), v_i + F(i-1, w - w_i)\} & \text{if } w \geq w_i \end{cases}$$

基情形: $F(0, w) = 0$ (没有物品时价值为 0), $F(i, 0) = 0$ (容量为 0 时无法装入任何物品)。

5.5.3 自底向上实现 (二维表)

我们可以用二维数组 $F[i][w]$ 来存储所有子问题的解, 按 i 从 1 到 n 、 w 从 1 到 W 的顺序计算。

C++ 代码实现:

```
#include <vector>
#include <algorithm> // for max
using std::max;

int knapsack(int W, int w[], int v[], int n) {
    // 使用 vector 创建二维表, 自动初始化为 0
    vector<vector<int>> F(n+1, vector<int>(W+1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= W; j++) {
            if (j < w[i-1]) {
                F[i][j] = F[i-1][j];
            } else {
                F[i][j] = max(F[i-1][j], v[i-1] + F[i-1][j - w[i-1]]);
            }
        }
    }

    return F[n][W];
}
```

Python 代码实现:

```
def knapsack(W, w, v, n):
    # 创建 (n+1) x (W+1) 的二维表, 初始化为 0
    F = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for j in range(1, W + 1):
            if j < w[i-1]:
                F[i][j] = F[i-1][j]
            else:
                F[i][j] = max(F[i-1][j], v[i-1] + F[i-1][j - w[i-1]])

    return F[n][W]
```

时间复杂度 $O(nW)$ ，空间复杂度 $O(nW)$ 。

5.5.4 举例填表

假设有 4 个物品，重量和价值分别为：

物品	重量	价值
1	1	12
2	2	10
3	3	20
4	2	15

背包容量 $W = 5$ 。填表结果如表 5-2 所示（只列出部分行和列）。

表 5-2 背包问题填表

$i \setminus w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	12	12	12	12	12
2	0	12	12	22	22	22
3	0	12	12	22	22	32
4	0	12	15	27	27	37

解释几个关键格子：- $F[1][1]$ ：只有物品 1，重量 $1 \leq 1$ ，选它得 12，不选得 0，取 12。- $F[2][2]$ ：考虑物品 1 和 2。不选物品 2 得 $F[1][2] = 12$ ；选物品 2 则需容量 2，剩余 0，价值 $10 + F[1][0] = 10$ ，取 12。- $F[3][3]$ ：考虑物品 1、2、3。不选物品 3 得 $F[2][3] = 22$ ；选物品 3 需容量 3，剩余 0，价值 $20 + F[2][0] = 20$ ，取 22。- $F[3][5]$ ：不选物品 3 得 $F[2][5] = 22$ ；选物品 3 需容量 3，剩余 2，价值 $20 + F[2][2] = 20 + 12 = 32$ ，取 32。- $F[4][5]$ ：不选物品 4 得 $F[3][5] = 32$ ；选物品 4 需容量 2，剩余 3，价值 $15 + F[3][3] = 15 + 22 = 37$ ，取 37。

最终最大价值为 37。

5.5.5 输出最优方案

同样可以用一个二维数组 $p[i][w]$ 记录在计算 $F[i][w]$ 时是否选了第 i 个物品。然后在得到 $F[n][W]$ 后，从 (n, W) 开始回溯。

C++ 代码实现：

```
#include <vector>
#include <algorithm>
using std::max;
using std::vector;

int knapsack(int W, int w[], int v[], int n) {
```



```

// 创建二维表 F 和记录表 p, 自动初始化为 0
vector<vector<int>> F(n+1, vector<int>(W+1, 0));
vector<vector<int>> p(n+1, vector<int>(W+1, 0)); // p[i][w] = 1 表示选了物品 i

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= W; j++) {
        if (j < w[i-1] || F[i-1][j] >= v[i-1] + F[i-1][j - w[i-1]]) {
            F[i][j] = F[i-1][j];
            p[i][j] = 0; // 未选物品 i
        } else {
            F[i][j] = v[i-1] + F[i-1][j - w[i-1]];
            p[i][j] = 1; // 选了物品 i
        }
    }
}

// 回溯输出选中的物品
int i = n, j = W;
while (i > 0 && j > 0) {
    if (p[i][j] == 1) {
        printf(" 选物品 %d\n", i);
        j -= w[i-1];
    }
    i--;
}

return F[n][W];
}

```

Python 代码实现:

```

def knapsack_with_items(W, w, v, n):
    F = [[0] * (W + 1) for _ in range(n + 1)]
    p = [[0] * (W + 1) for _ in range(n + 1)] # 记录是否选中

    for i in range(1, n + 1):
        for j in range(1, W + 1):
            if j < w[i-1] or F[i-1][j] >= v[i-1] + F[i-1][j - w[i-1]]:
                F[i][j] = F[i-1][j]
                p[i][j] = 0
            else:
                F[i][j] = v[i-1] + F[i-1][j - w[i-1]]
                p[i][j] = 1

```

```

# 回溯
i, j = n, W
selected = []
while i > 0 and j > 0:
    if p[i][j] == 1:
        selected.append(i)
        j -= w[i-1]
    i -= 1

print(" 选中的物品:", selected[::-1])    # 逆序输出为正序
return F[n][W]

```

5.5.6 空间优化

观察递推式, $F[i][w]$ 只依赖于 $F[i-1][...]$, 与更早的行无关。因此我们可以用一维数组滚动更新, 但需要注意内层循环必须从大到小进行, 以避免覆盖掉仍然需要的上一行数据。

C++ 代码实现:

```

#include <vector>
#include <algorithm>
using std::max;
using std::vector;

int knapsack(int W, int w[], int v[], int n) {
    vector<int> F(W + 1, 0);    // 一维数组, 初始化为 0
    for (int i = 1; i <= n; i++) {
        for (int j = W; j >= w[i-1]; j--) {    // 逆序更新
            F[j] = max(F[j], v[i-1] + F[j - w[i-1]]);
        }
    }
    return F[W];
}

```

为什么需要逆序? 因为如果正序更新, $F[j-w[i-1]]$ 可能已经被当前行的更新改变了 (它本应是上一行的值)。逆序遍历保证了在计算 $F[j]$ 时, $F[j-w[i-1]]$ 仍然是上一行的旧值。

Python 代码实现:

```

def knapsack_optimized(W, w, v, n):
    F = [0] * (W + 1)
    for i in range(1, n + 1):
        # 逆序更新, 保证 F[j-w[i-1]] 是上一行的值
        for j in range(W, w[i-1] - 1, -1):
            F[j] = max(F[j], v[i-1] + F[j - w[i-1]])

```

```

    return F[W]

# 使用示例
if __name__ == "__main__":
    w = [1, 2, 3, 2]      # 重量
    v = [12, 10, 20, 15] # 价值
    W = 5
    n = 4

    print(knapsack(W, w, v, n))          # 输出 37
    print(knapsack_optimized(W, w, v, n)) # 输出 37
    knapsack_with_items(W, w, v, n)

```

5.6 最长公共子序列 (LCS)

5.6.1 问题描述

给定两个序列 $X[1..m]$ 和 $Y[1..n]$ ，求它们的最长公共子序列的长度。子序列是指原序列中删除若干元素（不一定连续）后得到的新序列，且保持原有顺序。

例如， $X = \text{"ABCB DAB"}$ ， $Y = \text{"BDCABA"}$ ，最长公共子序列可以是 "BCBA" ，长度为 4。

5.6.2 定义目标函数与递归方程

设 $dp[i][j]$ 表示 $X[1..i]$ 和 $Y[1..j]$ 的最长公共子序列长度。考虑两个序列的最后一个字符：

- 如果 $X[i] = Y[j]$ ，则这个字符一定属于某个最长公共子序列，所以

$$dp[i][j] = dp[i-1][j-1] + 1$$

- 如果 $X[i] \neq Y[j]$ ，则公共子序列要么来自 $X[1..i-1]$ 和 $Y[1..j]$ ，要么来自 $X[1..i]$ 和 $Y[1..j-1]$ ，取较大者：

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

基情形： $dp[0][j] = 0$ （空串与任何串的 LCS 长度为 0）， $dp[i][0] = 0$ 。

5.6.3 自底向上实现（二维表）

我们可以用一个二维表 dp 按行优先顺序计算。

C++ 代码实现：

```

#include <vector>
#include <algorithm>
#include <string>

```

```

using namespace std;

int LCS(const string& X, const string& Y) {
    int m = X.size(), n = Y.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (X[i-1] == Y[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m][n];
}

```

Python 代码实现:

```

def LCS(X: str, Y: str) -> int:
    m, n = len(X), len(Y)
    dp = [[0] * (n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

```

5.6.4 举例填表

以 $X = \text{"ABC"}$, $Y = \text{"AC"}$ 为例, 填表过程如下:

$i \ j$	0	1 (A)	2 (C)
0	0	0	0
1 (A)	0	1	1
2 (B)	0	1	1
3 (C)	0	1	2

- $dp[1][1]$: $X[1]=A$, $Y[1]=A \rightarrow dp[0][0] + 1 = 1$ 。
- $dp[1][2]$: $X[1]=A$, $Y[2]=C$, 不相等 $\rightarrow \max(dp[0][2] = 0, dp[1][1] = 1) = 1$ 。
- $dp[3][2]$: $X[3]=C$, $Y[2]=C \rightarrow dp[2][1] + 1 = 1 + 1 = 2$ 。

最终长度为 2, 对应子序列 "AC"。

5.6.5 输出一个最长公共子序列

为了输出具体的一个 LCS，我们可以在计算 dp 时额外记录方向，然后回溯。

C++ 代码实现：

```
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

string printLCS(const string& X, const string& Y) {
    int m = X.size(), n = Y.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    vector<vector<int>> dir(m+1, vector<int>(n+1, 0)); // 0: 无, 1: 左上, 2: 上, 3: 左
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (X[i-1] == Y[j-1]) {
                dp[i][j] = dp[i-1][j-1] + 1;
                dir[i][j] = 1; // 来自左上
            } else if (dp[i-1][j] >= dp[i][j-1]) {
                dp[i][j] = dp[i-1][j];
                dir[i][j] = 2; // 来自上
            } else {
                dp[i][j] = dp[i][j-1];
                dir[i][j] = 3; // 来自左
            }
        }
    }
    // 回溯构造子序列
    string lcs;
    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (dir[i][j] == 1) {
            lcs.push_back(X[i-1]);
            --i; --j;
        } else if (dir[i][j] == 2) {
            --i;
        } else {
            --j;
        }
    }
    reverse(lcs.begin(), lcs.end());
    return lcs;
}
```

```
}

```

Python 代码实现:

```
def printLCS(X: str, Y: str) -> str:
    m, n = len(X), len(Y)
    dp = [[0] * (n+1) for _ in range(m+1)]
    dir = [[0] * (n+1) for _ in range(m+1)] # 1: 左上, 2: 上, 3: 左
    for i in range(1, m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
                dir[i][j] = 1
            elif dp[i-1][j] >= dp[i][j-1]:
                dp[i][j] = dp[i-1][j]
                dir[i][j] = 2
            else:
                dp[i][j] = dp[i][j-1]
                dir[i][j] = 3

    # 回溯
    lcs = []
    i, j = m, n
    while i > 0 and j > 0:
        if dir[i][j] == 1:
            lcs.append(X[i-1])
            i -= 1
            j -= 1
        elif dir[i][j] == 2:
            i -= 1
        else:
            j -= 1
    return ''.join(reversed(lcs))

```

5.6.6 空间优化

观察递推式, 计算 $dp[i][j]$ 只用到当前行的上一行 ($i-1$) 和当前行左边 ($j-1$)。因此可以用两行一维数组滚动计算, 空间复杂度降为 $O(n)$ 。以下是优化后的长度计算 (不输出序列):

C++ 代码 (空间优化):

```
int LCS_opt(const string& X, const string& Y) {
    int m = X.size(), n = Y.size();
    vector<int> prev(n+1, 0), curr(n+1, 0);
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {

```

```

        if (X[i-1] == Y[j-1])
            curr[j] = prev[j-1] + 1;
        else
            curr[j] = max(prev[j], curr[j-1]);
    }
    swap(prev, curr);
}
return prev[n];
}

```

Python 代码 (空间优化):

```

def LCS_opt(X: str, Y: str) -> int:
    m, n = len(X), len(Y)
    prev = [0] * (n+1)
    curr = [0] * (n+1)
    for i in range(1, m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                curr[j] = prev[j-1] + 1
            else:
                curr[j] = max(prev[j], curr[j-1])
        prev, curr = curr, prev
    return prev[n]

```

5.6.7 小结

最长公共子序列问题是动态规划的经典应用,其核心是掌握递推式 $dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & X[i] = Y[j] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{otherwise} \end{cases}$ 。通过填表理解状态转移,并掌握空间优化技巧,可以解决许多序列比对问题(如生物信息学中的 DNA 序列相似度)。

5.7 编辑距离

5.7.1 问题描述

编辑距离 (Edit Distance) 是指将一个字符串转换为另一个字符串所需的最少编辑操作次数。通常允许的操作有:

- 插入一个字符
- 删除一个字符
- 替换一个字符

每种操作可以赋予不同的代价。这里我们假设**插入和删除的代价为 1**, **替换的代价为 2** (因为替换可以视为一次删除加一次插入,但有时替换成本较低,本例故意设为 2 以体现差异)。

例如，将 "horse" 转换为 "ros"，如果替换代价为 2，则最优方案为：

- 删除 'h' (代价 1)
- 删除 'o' (代价 1)
- 将 'r' 替换为 'r' (无需操作)
- 将 's' 替换为 's' (无需操作)
- 将 'e' 替换为 's'? 不对，实际上正确路径应计算。本例仅为说明。

更标准的例子见下文填表演示。

5.7.2 定义目标函数与递归方程

设 $dp[i][j]$ 表示将 $X[1..i]$ 转换为 $Y[1..j]$ 的最小总代价。考虑两个字符串的最后一个字符：

- 如果 $X[i] == Y[j]$ ，则最后一个字符不需要操作，直接匹配：

$$dp[i][j] = dp[i-1][j-1]$$

- 如果 $X[i] \neq Y[j]$ ，则有三种可能的操作，取最小代价：

- 删除 $X[i]$ ：代价 $1 + dp[i-1][j]$
- 插入 $Y[j]$ (在 X 末尾添加一个与 $Y[j]$ 相同的字符)：代价 $1 + dp[i][j-1]$
- 替换 $X[i]$ 为 $Y[j]$ ：代价 $2 + dp[i-1][j-1]$

综合起来：

$$dp[i][j] = \begin{cases} dp[i-1][j-1] & \text{if } X[i] = Y[j] \\ \min \begin{cases} dp[i-1][j] + 1, \\ dp[i][j-1] + 1, \\ dp[i-1][j-1] + 2 \end{cases} & \text{otherwise} \end{cases}$$

基情形：

- 将空串转换为 $Y[1..j]$ 需要插入 j 次，故 $dp[0][j] = j \times 1 = j$ 。
- 将 $X[1..i]$ 转换为空串需要删除 i 次，故 $dp[i][0] = i \times 1 = i$ 。

5.7.3 自底向上实现（二维表）

我们按 i 和 j 递增的顺序填表。

C++ 代码实现：

```
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

int editDistance(const string& X, const string& Y) {
    int m = X.size(), n = Y.size();
```



```

vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
// 初始化基情形
for (int i = 0; i <= m; ++i) dp[i][0] = i;
for (int j = 0; j <= n; ++j) dp[0][j] = j;

for (int i = 1; i <= m; ++i) {
    for (int j = 1; j <= n; ++j) {
        if (X[i-1] == Y[j-1]) {
            dp[i][j] = dp[i-1][j-1];
        } else {
            dp[i][j] = min({ dp[i-1][j] + 1,
                             dp[i][j-1] + 1,
                             dp[i-1][j-1] + 2 });
        }
    }
}
return dp[m][n];
}

```

Python 代码实现:

```

def editDistance(X: str, Y: str) -> int:
    m, n = len(X), len(Y)
    dp = [[0] * (n+1) for _ in range(m+1)]
    for i in range(m+1):
        dp[i][0] = i
    for j in range(n+1):
        dp[0][j] = j

    for i in range(1, m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = min(dp[i-1][j] + 1,
                               dp[i][j-1] + 1,
                               dp[i-1][j-1] + 2)

    return dp[m][n]

```

5.7.4 举例填表

设 $X = \text{"ab"}$, $Y = \text{"ba"}$, 替换代价为 2, 插入/删除代价为 1。

计算过程:

$i \ j$	0	1 (b)	2 (a)
0	0	1	2
1 (a)	1	?	?
2 (b)	2	?	?

- 初始化第一行、第一列如上。
- $i = 1, j = 1$: $X[1]='a', Y[1]='b'$ 不等
 删除: $dp[0][1] + 1 = 1 + 1 = 2$
 插入: $dp[1][0] + 1 = 1 + 1 = 2$
 替换: $dp[0][0] + 2 = 0 + 2 = 2$
 最小值 $= 2 \rightarrow dp[1][1] = 2$
- $i = 1, j = 2$: $X[1]='a', Y[2]='a'$ 相等 $\rightarrow dp[1][2] = dp[0][1] = 1$
- $i = 2, j = 1$: $X[2]='b', Y[1]='b'$ 相等 $\rightarrow dp[2][1] = dp[1][0] = 1$
- $i = 2, j = 2$: $X[2]='b', Y[2]='a'$ 不等
 删除: $dp[1][2] + 1 = 1 + 1 = 2$
 插入: $dp[2][1] + 1 = 1 + 1 = 2$
 替换: $dp[1][1] + 2 = 2 + 2 = 4$
 最小值 $= 2 \rightarrow dp[2][2] = 2$

最终编辑距离为 2 (例如: 将 “ab” 转换为 “ba” 可以先删除 ‘a’ 再插入 ‘a’ 得到 “ba”, 代价 $1+1=2$)。正确。

5.7.5 输出编辑方案 (可选)

为了得到具体的操作序列, 可以像 LCS 那样记录方向, 然后回溯。由于操作类型较多 (删除、插入、替换、匹配), 我们需要记录每一步的选择。

C++ 代码 (输出操作步骤):

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

void printEdits(const string& X, const string& Y) {
    int m = X.size(), n = Y.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    vector<vector<int>> op(m+1, vector<int>(n+1, 0)); // 0:match,1:delete,2:insert,3:replace
    for (int i = 0; i <= m; ++i) dp[i][0] = i;
    for (int j = 0; j <= n; ++j) dp[0][j] = j;
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (X[i-1] == Y[j-1]) {
                dp[i][j] = dp[i-1][j-1];
            }
        }
    }
}
```

```

        op[i][j] = 0; // 匹配, 不需操作
    } else {
        int del = dp[i-1][j] + 1;
        int ins = dp[i][j-1] + 1;
        int rep = dp[i-1][j-1] + 2;
        if (del <= ins && del <= rep) {
            dp[i][j] = del;
            op[i][j] = 1; // 删除
        } else if (ins <= del && ins <= rep) {
            dp[i][j] = ins;
            op[i][j] = 2; // 插入
        } else {
            dp[i][j] = rep;
            op[i][j] = 3; // 替换
        }
    }
}

// 回溯
int i = m, j = n;
while (i > 0 || j > 0) {
    if (op[i][j] == 0) { // 匹配
        // 字符相同, 无需操作
        --i; --j;
    } else if (op[i][j] == 1) { // 删除 X[i]
        cout << " 删除 " << X[i-1] << endl;
        --i;
    } else if (op[i][j] == 2) { // 插入 Y[j]
        cout << " 插入 " << Y[j-1] << endl;
        --j;
    } else { // 替换
        cout << " 替换 " << X[i-1] << " 为 " << Y[j-1] << endl;
        --i; --j;
    }
}
}
}

```

Python 代码：逻辑类似，不再赘述。可根据需要补充。

5.7.6 空间优化

观察递推式， $dp[i][j]$ 只依赖于当前行的上一行 ($i-1$) 和当前行左边 ($j-1$)。因此可以用**两行一维数组**滚动计算，空间复杂度降为 $O(n)$ 。

C++ 代码 (空间优化):

```

int editDistanceOpt(const string& X, const string& Y) {
    int m = X.size(), n = Y.size();
    vector<int> prev(n+1), curr(n+1);
    for (int j = 0; j <= n; ++j) prev[j] = j; // dp[0][j]
    for (int i = 1; i <= m; ++i) {
        curr[0] = i; // dp[i][0]
        for (int j = 1; j <= n; ++j) {
            if (X[i-1] == Y[j-1]) {
                curr[j] = prev[j-1];
            } else {
                curr[j] = min({ prev[j] + 1,      // 删除
                               curr[j-1] + 1,    // 插入
                               prev[j-1] + 2 }); // 替换
            }
        }
        swap(prev, curr);
    }
    return prev[n];
}

```

Python 代码:

```

def editDistanceOpt(X: str, Y: str) -> int:
    m, n = len(X), len(Y)
    prev = list(range(n+1)) # dp[0][j] = j
    curr = [0] * (n+1)
    for i in range(1, m+1):
        curr[0] = i
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                curr[j] = prev[j-1]
            else:
                curr[j] = min(prev[j] + 1,
                              curr[j-1] + 1,
                              prev[j-1] + 2)
        prev, curr = curr, prev
    return prev[n]

```

5.7.7 小结

编辑距离是动态规划在字符串处理中的经典应用。通过定义 $dp[i][j]$ 为子串 $X[1..i]$ 和 $Y[1..j]$ 的最小编辑代价，并利用递推式考虑最后字符的匹配、删除、插入、替换四种情况，可以高效求解。空间优化后只需 $O(n)$ 内存，非常适合处理长字符串。该算法广泛应用于拼写检查、DNA 序列比对、自然语言处理等领域。

5.8 最大子段和 (Kadane 算法)

5.8.1 问题描述

给定一个整数数组 $A[1..n]$ (可能包含负数)，求一个**连续子数组** (子段) 的最大和。例如，数组 $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ 的最大子段和为 **6** (对应子数组 $[4, -1, 2, 1]$)。

5.8.2 定义目标函数与递推公式

设 $S(i)$ 表示以 $A[i]$ 结尾的**最大子段和**。那么对于每个位置 i ，有两种可能：

- 该子段只包含 $A[i]$ 本身，即和为 $A[i]$ ；
- 该子段包含前面的子段 (以 $A[i-1]$ 结尾的最大子段) 再加上 $A[i]$ ，即和为 $S(i-1) + A[i]$ 。

我们取两者中的较大值：

$$S(i) = \max\{S(i-1) + A[i], A[i]\}$$

基情形： $S(1) = A[1]$ 。

整个数组的最大子段和就是所有 $S(i)$ 中的最大值：

$$\text{ans} = \max_{1 \leq i \leq n} S(i)$$

这个递推关系具有**最优子结构**：以 $A[i]$ 结尾的最优解必然包含以 $A[i-1]$ 结尾的最优解 (如果 $S(i-1)$ 为正) 或者从 $A[i]$ 重新开始。

5.8.3 自底向上实现

我们可以用一个数组 dp 存储每个位置的最优值，然后扫描一次即可。

C++ 代码实现：

```
#include <vector>
#include <algorithm>
using namespace std;

int maxSubArray(const vector<int>& A) {
    int n = A.size();
    if (n == 0) return 0;
    vector<int> S(n);
    S[0] = A[0];
    int ans = S[0];
    for (int i = 1; i < n; ++i) {
        S[i] = max(S[i-1] + A[i], A[i]);
    }
}
```

```

        ans = max(ans, S[i]);
    }
    return ans;
}

```

Python 代码实现:

```

def maxSubArray(A):
    if not A:
        return 0
    n = len(A)
    S = [0] * n
    S[0] = A[0]
    ans = S[0]
    for i in range(1, n):
        S[i] = max(S[i-1] + A[i], A[i])
        ans = max(ans, S[i])
    return ans

```

5.8.4 空间优化 (Kadane 算法)

观察递推式, $S(i)$ 只依赖于 $S(i-1)$, 因此完全不需要数组, 只需要两个变量滚动更新: 一个记录当前以 i 结尾的最大子段和 cur , 另一个记录全局最大值 ans 。

C++ 代码实现:

```

int maxSubArray(const vector<int>& A) {
    if (A.empty()) return 0;
    int cur = A[0];
    int ans = cur;
    for (size_t i = 1; i < A.size(); ++i) {
        cur = max(cur + A[i], A[i]);
        ans = max(ans, cur);
    }
    return ans;
}

```

Python 代码实现:

```

def maxSubArray(A):
    if not A:
        return 0
    cur = ans = A[0]
    for x in A[1:]:
        cur = max(cur + x, x)
        ans = max(ans, cur)

```

```
return ans
```

这就是著名的 **Kadane 算法**，时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

5.8.5 举例演示

以数组 $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ 为例：

i	A[i]	cur = max(cur+A[i], A[i])	ans
0	-2	-2	-2
1	1	max(-2+1, 1) = 1	1
2	-3	max(1-3, -3) = -2	1
3	4	max(-2+4, 4) = 4	4
4	-1	max(4-1, -1) = 3	4
5	2	max(3+2, 2) = 5	5
6	1	max(5+1, 1) = 6	6
7	-5	max(6-5, -5) = 1	6
8	4	max(1+4, 4) = 5	6

最终最大子段和为 6。

5.8.6 小结

最大子段和问题展示了动态规划中**一维状态转移**的典型应用。通过定义“以当前位置结尾的最优解”，我们得到了简洁的递推公式 $S(i) = \max(S(i-1) + A[i], A[i])$ 。Kadane 算法进一步将空间降为 $O(1)$ ，是动态规划中“原地滚动”思想的极佳示例。该算法也适用于股票买卖等类似问题。

5.9 最长递增子序列 (LIS)

5.9.1 问题描述

给定一个整数序列 $A[1..n]$ ，求**最长严格递增子序列**的长度。子序列不要求连续，但必须保持原有顺序。

例如，序列 $[5, 2, 8, 6, 3, 6, 9, 7]$ 的最长递增子序列为 $[2, 3, 6, 9]$ 或 $[2, 3, 6, 7]$ ，长度为 4。

5.9.2 定义目标函数与递推公式

设 $dp[i]$ 表示以 **$A[i]$ 结尾的最长递增子序列的长度**。那么对于每个 i ，我们考虑所有位于 i 之前的元素 $A[j]$ ($j < i$)。如果 $A[j] < A[i]$ ，则可以将 $A[i]$ 接在以 $A[j]$ 结尾的递增子序列后面，形成长度为 $dp[j] + 1$ 的新序列。我们取所有可能中的最大值。此外， $A[i]$ 也可以单独成为一个子序列（长度为 1）。

因此递推公式为：

$$dp[i] = \max(1, \max\{dp[j] + 1 \mid j < i \text{ 且 } A[j] < A[i]\})$$

最终答案为所有 $dp[i]$ 中的最大值：

$$ans = \max_{1 \leq i \leq n} dp[i]$$

5.9.3 自底向上实现（基础版， $O(n^2)$ ）

我们可以用一个数组 dp 记录每个位置的最优值，并通过两层循环计算。

C++ 代码实现：

```
#include <vector>
#include <algorithm>
using namespace std;

int LIS(const vector<int>& A) {
    int n = A.size();
    if (n == 0) return 0;
    vector<int> dp(n, 1);
    int ans = 1;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (A[j] < A[i]) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}
```

Python 代码实现：

```
def LIS(A):
    n = len(A)
    if n == 0:
        return 0
    dp = [1] * n
    ans = 1
    for i in range(n):
        for j in range(i):
            if A[j] < A[i]:
                dp[i] = max(dp[i], dp[j] + 1)
        ans = max(ans, dp[i])
    return ans
```

对于大规模数据，存在时间复杂度 $O(n \log n)$ 的解法（基于贪心与二分查找），但已超出动态规划范畴，本书不再展开。

5.9.4 举例演示

以序列 $A = [5, 2, 8, 6, 3, 6, 9, 7]$ 为例, 按照递推公式 $dp[i] = \max(1, \{dp[j] + 1 \mid j < i \text{ 且 } A[j] < A[i]\})$ 计算:

- $i=0$ ($A[0]=5$): 前面无元素 $\rightarrow dp[0] = 1$
- $i=1$ ($A[1]=2$): 考虑 $j=0$: $A[0]=5 \not< 2$, 无候选。 $dp[1] = \max(1) = 1$
- $i=2$ ($A[2]=8$): 候选 $j=0$ ($5 < 8$): $dp[0]+1=2$; $j=1$ ($2 < 8$): $dp[1]+1=2$ 。 $dp[2] = \max(1, 2, 2) = 2$
- $i=3$ ($A[3]=6$): 候选 $j=0$ ($5 < 6$): $dp[0]+1=2$; $j=1$ ($2 < 6$): $dp[1]+1=2$; $j=2$ ($8 \not< 6$) 跳过。 $dp[3] = \max(1, 2, 2) = 2$
- $i=4$ ($A[4]=3$): 候选 $j=0$ ($5 \not< 3$) 跳过; $j=1$ ($2 < 3$): $dp[1]+1=2$; $j=2$ ($8 \not< 3$); $j=3$ ($6 \not< 3$)。 $dp[4] = \max(1, 2) = 2$
- $i=5$ ($A[5]=6$): 候选 $j=0$ ($5 < 6$): $dp[0]+1=2$; $j=1$ ($2 < 6$): $dp[1]+1=2$; $j=2$ ($8 \not< 6$); $j=3$ ($6 \not< 6$, 严格递增); $j=4$ ($3 < 6$): $dp[4]+1=3$ 。 $dp[5] = \max(1, 2, 2, 3) = 3$
- $i=6$ ($A[6]=9$): 所有 $j < 6$ 且 $A[j] < 9$ 均有效, 其中最大 $dp[j]$ 是 $dp[5]=3$ (也可取 $dp[2]=2$ 等)。 $dp[6] = \max(1, dp[5] + 1 = 4, \text{其余} + 1 \leq 4) = 4$
- $i=7$ ($A[7]=7$): 候选 j 满足 $A[j] < 7$: $j=1$ (2): $dp[1]+1=2$; $j=3$ (6): $dp[3]+1=3$; $j=4$ (3): $dp[4]+1=3$; $j=5$ (6): $dp[5]+1=4$ 。 $dp[7] = \max(1, 2, 3, 3, 4) = 4$

最终 $\text{ans} = \max(dp) = 4$ 。最长递增子序列长度为 4, 例如 $[2, 3, 6, 9]$ 或 $[2, 3, 6, 7]$ 。

5.9.5 小结

最长递增子序列问题展示了动态规划中**一维递推**的另一类经典模式: $dp[i]$ 依赖于所有之前的状态 ($j < i$)。基础 $O(n^2)$ 算法易于理解, 而 $O(n \log n)$ 的优化技巧利用了贪心和二分, 是算法设计中“降低复杂度”的典型范例。该问题广泛应用于基因序列比对、文本相似度分析等领域。

5.10 矩阵链乘法

5.10.1 问题描述

给定一系列矩阵 $A_1, A_2, \dots, A_n$, 其中矩阵 A_i 的维度为 $p_{i-1} \times p_i$ 。计算乘积 $A_1 \times A_2 \times \dots \times A_n$ 时, 由于矩阵乘法满足结合律, 我们可以通过加括号来改变计算顺序。不同的括号化方式会导致**标量乘法次数**差异很大。矩阵链乘法问题的目标是找到一种加括号的方式, 使得总乘法次数最少。

示例: 四个矩阵的维度分别为:

- A_1 : 50×20
- A_2 : 20×1
- A_3 : 1×10
- A_4 : 10×100

不同括号化方案的乘法次数:

- $((A_1 A_2) A_3) A_4$: $50 \cdot 20 \cdot 1 + 50 \cdot 1 \cdot 10 + 50 \cdot 10 \cdot 100 = 1000 + 500 + 50000 = 51500$
- $(A_1 (A_2 A_3)) A_4$: $20 \cdot 1 \cdot 10 + 50 \cdot 20 \cdot 10 + 50 \cdot 10 \cdot 100 = 200 + 10000 + 50000 = 60200$
- $(A_1 A_2) (A_3 A_4)$: $50 \cdot 20 \cdot 1 + 1 \cdot 10 \cdot 100 + 50 \cdot 1 \cdot 100 = 1000 + 1000 + 5000 = 7000$ (最优)

由此可见，合理选择计算顺序可极大减少运算量。

5.10.2 定义目标函数与递归方程

设 $C(i, j)$ 表示计算矩阵乘积 $A_i \times A_{i+1} \times \cdots \times A_j$ 所需的最小乘法次数。最终答案为 $C(1, n)$ 。

考虑在某个位置 k ($i \leq k < j$) 将矩阵链分成两部分: $A_i \dots A_k$ 和 $A_{k+1} \dots A_j$ 。先分别计算左右部分，然后相乘。左右部分的最小代价分别为 $C(i, k)$ 和 $C(k+1, j)$ ；最后合并两个结果矩阵的乘法次数为 $p_{i-1} \cdot p_k \cdot p_j$ (左矩阵维度 $p_{i-1} \times p_k$ ，右矩阵 $p_k \times p_j$)。因此：

$$C(i, j) = \min_{i \leq k < j} \{C(i, k) + C(k+1, j) + p_{i-1} \cdot p_k \cdot p_j\}$$

基情形：当 $i = j$ 时，只有一个矩阵，不需要乘法，所以 $C(i, i) = 0$ 。

该问题具有**最优子结构**：全局最优解包含子问题的最优解。

5.10.3 自底向上实现（按反对角线填表）

矩阵链乘法的递推式

$$C(i, j) = \min_{i \leq k < j} \{C(i, k) + C(k+1, j) + p_{i-1} p_k p_j\}$$

表明：计算 $C(i, j)$ 时，需要知道所有长度更短的子链 $C(i, k)$ 和 $C(k+1, j)$ 。因此，不能按简单的行优先或列优先顺序填表，而应按照**区间长度**（即 $j - i + 1$ ）从小到大的顺序计算。在表格中，这等价于**按反对角线**（从主对角线开始，逐渐向右上方远离）依次填写。

5.10.3.1 填表过程详细演示

以 4 个矩阵为例，维度数组 $p = [50, 20, 1, 10, 100]$ 。下表展示了计算顺序，数字 $\square$ 、 $\square$ 、 $\square$ 表示不同区间长度的计算次序：

ij	1	2	3	4
1	0	$\square$ 1000	$\square$ 1500	$\square$ 7000
2	—	0	$\square$ 200	$\square$ 3000
3	—	—	0	$\square$ 1000
4	—	—	—	0

计算步骤（按区间长度递增顺序）：

1. **初始化**：所有 $C(i, i) = 0$ （主对角线）。

2. 区间长度 = 1 (计算 $\square$ 标记的格子):

- $C(1, 2)$: 只有 $k = 1$, 代价 $= C(1, 1) + C(2, 2) + p_0 p_1 p_2 = 0 + 0 + 50 \cdot 20 \cdot 1 = 1000$ 。
 $\rightarrow C(1, 2) = 1000$ 。
- $C(2, 3)$: $k = 2$, 代价 $= 0 + 0 + 20 \cdot 1 \cdot 10 = 200$ 。
 $\rightarrow C(2, 3) = 200$ 。
- $C(3, 4)$: $k = 3$, 代价 $= 0 + 0 + 1 \cdot 10 \cdot 100 = 1000$ 。
 $\rightarrow C(3, 4) = 1000$ 。

3. 区间长度 = 2 (计算 $\square$ 标记的格子):

- $C(1, 3)$:
 - $k = 1$: $C(1, 1) + C(2, 3) + p_0 p_1 p_3 = 0 + 200 + 50 \cdot 20 \cdot 10 = 200 + 10000 = 10200$
 - $k = 2$: $C(1, 2) + C(3, 3) + p_0 p_2 p_3 = 1000 + 0 + 50 \cdot 1 \cdot 10 = 1000 + 500 = 1500$
 取最小值 $\rightarrow C(1, 3) = 1500$ 。
- $C(2, 4)$:
 - $k = 2$: $C(2, 2) + C(3, 4) + p_1 p_2 p_4 = 0 + 1000 + 20 \cdot 1 \cdot 100 = 1000 + 2000 = 3000$
 - $k = 3$: $C(2, 3) + C(4, 4) + p_1 p_3 p_4 = 200 + 0 + 20 \cdot 10 \cdot 100 = 200 + 20000 = 20200$
 取最小值 $\rightarrow C(2, 4) = 3000$ 。

4. 区间长度 = 3 (计算 $\square$ 标记的格子):

- $C(1, 4)$:
 - $k = 1$: $C(1, 1) + C(2, 4) + p_0 p_1 p_4 = 0 + 3000 + 50 \cdot 20 \cdot 100 = 3000 + 100000 = 103000$
 - $k = 2$: $C(1, 2) + C(3, 4) + p_0 p_2 p_4 = 1000 + 1000 + 50 \cdot 1 \cdot 100 = 2000 + 5000 = 7000$
 - $k = 3$: $C(1, 3) + C(4, 4) + p_0 p_3 p_4 = 1500 + 0 + 50 \cdot 10 \cdot 100 = 1500 + 50000 = 51500$
 取最小值 $\rightarrow C(1, 4) = 7000$ 。

最终最小乘法次数为 7000, 对应分割点 $k = 2$, 即括号化方案 $(A_1 A_2)(A_3 A_4)$ 。

5.10.3.2 代码实现

按照上述反对角线顺序, 外层循环按区间长度 len 递增, 内层循环枚举起始位置 i , 再枚举分割点 k 。

C++ 代码实现 (使用 `vector`):

```
#include <vector>
#include <climits>
#include <cstdio>
using namespace std;

void printOptimal(const vector<vector<int>>& s, int i, int j) {
    if (i == j) {
```

```

        printf("A%d", i);
    } else {
        printf("(");
        printOptimal(s, i, s[i][j]);
        printOptimal(s, s[i][j] + 1, j);
        printf(")");
    }
}

int matrixChain(const vector<int>& p) {
    int n = p.size() - 1;
    vector<vector<int>> C(n+1, vector<int>(n+1, 0));
    vector<vector<int>> s(n+1, vector<int>(n+1, 0));

    for (int len = 2; len <= n; ++len) {
        for (int i = 1; i <= n - len + 1; ++i) {
            int j = i + len - 1;
            C[i][j] = INT_MAX;
            for (int k = i; k < j; ++k) {
                int cost = C[i][k] + C[k+1][j] + p[i-1] * p[k] * p[j];
                if (cost < C[i][j]) {
                    C[i][j] = cost;
                    s[i][j] = k;
                }
            }
        }
    }

    printf(" 最小乘法次数: %d\n", C[1][n]);
    printf(" 最优加括号方式: ");
    printOptimal(s, 1, n);
    printf("\n");
    return C[1][n];
}

```

Python 代码实现:

```

def matrix_chain(p):
    n = len(p) - 1
    C = [[0] * (n+1) for _ in range(n+1)]
    s = [[0] * (n+1) for _ in range(n+1)]

    for length in range(2, n+1):

```

```

    for i in range(1, n - length + 2):
        j = i + length - 1
        C[i][j] = float('inf')
        for k in range(i, j):
            cost = C[i][k] + C[k+1][j] + p[i-1] * p[k] * p[j]
            if cost < C[i][j]:
                C[i][j] = cost
                s[i][j] = k

def print_opt(i, j):
    if i == j:
        print(f"A{i}", end='')
    else:
        print("(", end='')
        print_opt(i, s[i][j])
        print_opt(s[i][j]+1, j)
        print(")", end='')

print(f" 最小乘法次数: {C[1][n]}")
print(" 最优加括号方式: ", end='')
print_opt(1, n)
print()
return C[1][n]

```

以上代码中的外层循环 `for len in range(2, n+1)` 直接对应“按区间长度递增”的填表顺序，与表格中的反对角线计算次序完全一致。通过详细的手工计算步骤和代码对照，读者可以深入理解矩阵链乘法动态规划的核心思想。

5.10.4 复杂度分析

- **时间复杂度**：三层循环，分别为区间长度 $O(n)$ 、起始位置 $O(n)$ 、分割点 $O(n)$ ，总 $O(n^3)$ 。
- **空间复杂度**：存储 C 和 s 表，各为 $O(n^2)$ 。

5.10.5 小结

矩阵链乘法是动态规划中**区间 DP**的典型应用。通过按区间长度递增的顺序填表，保证了每个子问题求解时依赖的更短子链已经计算完毕。该问题强调了括号化对计算效率的重要影响，并展示了动态规划相较于穷举的巨大优势。

5.11 小结

本章我们学习了动态规划的基本思想：通过定义目标函数和递归方程，将问题分解为重叠的子问题，并利用记忆化存储避免重复计算，从而高效求解。我们通过斐波那契数列、找零钱、机器人路径、0-1 背包、最长公共子序列、编辑距离、最大子段和、最长递增子序列、矩阵链乘法等一系列经典问题，展示了动态规划的应用和实现技巧。

动态规划的两个关键特征是**重叠子问题**和**最优子结构**。掌握动态规划的关键在于：- 能够正确地将原问题分解为子问题，并定义合适的目标函数；- 能够写出正确的递归方程；- 能够选择合适的实现方式（自顶向下或自底向上）；- 能够根据具体问题优化时间和空间复杂度。

动态规划是算法设计中最重要、最灵活的方法之一，需要大量练习才能真正掌握。本章的每个问题都可以作为独立练习，建议读者亲手实现并尝试修改条件（如改变硬币面额、增加物品限制等），以加深理解。

5.12 习题

5.12.1 选择题（单选）

1. 动态规划与分治算法的主要区别是：

- A. 动态规划使用递归，分治不使用递归
- B. 动态规划适用于子问题重叠的情况，分治通常要求子问题独立
- C. 动态规划只能求解最优化问题
- D. 动态规划的时间复杂度总是低于分治

答案：B

2. 对于斐波那契数列 $F_n = F_{n-1} + F_{n-2}$ ，使用记忆化递归的时间复杂度是：

- A. $O(2^n)$
- B. $O(n^2)$
- C. $O(n)$
- D. $O(\log n)$

答案：C

3. 关于 0-1 背包问题的动态规划解法，下列说法正确的是：

- A. 状态转移方程是 $dp[i][w] = \max(dp[i-1][w], dp[i][w-w_i] + v_i)$
- B. 空间优化为 1 维数组时内层循环应从小到大遍历
- C. 时间复杂度为 $O(nW)$ ，其中 W 是背包容量
- D. 该问题不具有最优子结构

答案：C

4. 最长公共子序列 (LCS) 问题中，若 $X = "ABC"$, $Y = "ACD"$ ，则 LCS 长度为：

- A. 1
- B. 2
- C. 3
- D. 4

答案：B (“AC”)

5. 编辑距离问题中，若允许插入、删除、替换代价均为 1，将字符串 “cat” 转换为 “car” 的最小编辑距离是：

- A. 0
- B. 1
- C. 2
- D. 3

答案：B (替换 ‘t’ 为 ‘r’)

6. 对于最大子段和问题 (Kadane 算法)，以下哪个说法是错误的？

- A. 状态 $dp[i]$ 表示以第 i 个元素结尾的最大子段和
- B. 递推公式为 $dp[i] = \max(dp[i-1] + A[i], A[i])$
- C. 算法时间复杂度为 $O(n \log n)$
- D. 空间复杂度可以优化到 $O(1)$

答案：C

7. 最长递增子序列 (LIS) 问题，对于序列 $[3, 1, 4, 1, 5, 9, 2, 6]$ ，LIS 长度是：

- A. 3
- B. 4
- C. 5
- D. 6

答案: B (例如 1,4,5,9 或 1,4,5,6 长度为 4)

8. 矩阵链乘法问题中, 计算 $C(i, j)$ 时依赖的子问题不包括:

A. $C(i+1, j)$ B. $C(i, k)$ C. $C(k+1, j)$ D. $C(i+1, j-1)$

答案: A

5.12.2 判断题 (正确打 $\checkmark$, 错误打 $\times$)

1. 任何具有重叠子问题的递归算法都可以用动态规划优化。($\times$)
2. 0-1 背包问题的动态规划解法是伪多项式时间算法。($\checkmark$)
3. 最长公共子序列问题只能使用动态规划求解, 不能用其他方法。($\times$)
4. 对于找零钱问题, 如果硬币面额为 1 元、3 元、4 元, 则贪心算法 (每次用最大面额) 一定得到最优解。($\times$)
5. 最大子段和问题中, 如果所有元素均为负数, 则最大子段和为最大的那个负数。($\checkmark$)
6. 编辑距离问题中, 替换操作可以看作一次删除加一次插入, 因此替换代价不能小于删除 + 插入的代价。($\times$)
7. 机器人路径问题 (仅向右向下) 的路径数可以用组合数学公式直接计算, 不一定需要动态规划。($\checkmark$)
8. 矩阵链乘法的最优括号化方案一定是对称的。($\times$)

5.12.3 填空题

1. 动态规划的两个核心特征是 _____ 和 _____。

答案: 重叠子问题、最优子结构

2. 使用记忆化递归实现斐波那契数列, 时间复杂度是 _____, 空间复杂度是 _____。

答案: $O(n)$ 、 $O(n)$

3. 0-1 背包问题的一维空间优化代码中, 内层循环需要按 _____ 方向遍历, 目的是避免覆盖上一行的值。

答案: 逆序 (从大到小)

4. 最长公共子序列的递推式: 当 $X[i] = Y[j]$ 时, $dp[i][j] =$ _____; 否则 $dp[i][j] =$ _____。

答案: $dp[i-1][j-1] + 1, \max(dp[i-1][j], dp[i][j-1])$

5. 编辑距离中, 若插入、删除代价为 1, 替换代价为 2, 则从空串转换成长度为 L 的字符串代价为 _____。

答案: L

6. Kadane 算法中, cur 表示以当前元素结尾的最大子段和, 更新式为 $cur = \max(\text{_____, } \text{_____})$ 。

答案: $cur + A[i]$ 、 $A[i]$

7. 最长递增子序列的 $O(n^2)$ 动态规划算法中, $dp[i]$ 表示 _____。

答案: 以 $A[i]$ 结尾的最长递增子序列的长度

8. 矩阵链乘法问题的填表顺序是按照 _____ 递增的顺序计算。

答案: 区间长度 ($j - i + 1$)

5.12.4 客观编程题（带参考解法）

编程题 2：斐波那契数列（记忆化递归）

题目描述：输入 n ($0 \leq n \leq 40$)，输出第 n 项斐波那契数 F_n ($F_0 = 0, F_1 = 1$)。要求使用记忆化递归实现，禁止使用循环递推。

输入格式：一个整数 n 。

输出格式：一个整数 F_n 。

样例输入 1：5

样例输出 1：5

样例输入 2：0

样例输出 2：0

编程题 2：找零钱问题（输出最少硬币数及硬币组合）

题目描述：给定 n 种硬币面额（每种无限），求凑出金额 $amount$ 所需的最少硬币数，并输出任意一种使用硬币的组成方案（按面额从小到大输出，可重复）。如果无法凑出，输出 -1。

输入格式：

第一行两个整数 n 和 $amount$ 。

第二行 n 个整数表示硬币面额。

输出格式：

- 若能凑出：第一行输出最少硬币数，第二行按面额非递减顺序输出所用硬币（空格分隔）。
- 若不能凑出：输出 -1。

样例输入：

3 15

11 5 1

样例输出：

3

5 5 5

编程题 3：机器人路径问题（空间优化）

题目描述：一个 $m \times n$ 网格，机器人从左上角走到右下角，每次只能向右或向下，求不同路径数。要求使用一维数组优化，空间复杂度 $O(n)$ 。

输入格式：一行两个整数 m 和 n ($1 \leq m, n \leq 100$)。

输出格式：一个整数，路径数。

样例输入：3 7

样例输出：28

编程题 4：0-1 背包问题（输出最大价值及物品编号）

题目描述：有 n 个物品，每个物品有重量 w_i 和价值 v_i ，背包容量为 W 。求能装入的最大总价值，并输出任意一种达到该价值的物品组合（按物品编号升序输出）。

输入格式：

第一行两个整数 n 和 W 。

接下来 n 行，每行两个整数 w_i 和 v_i 。

输出格式：

第一行输出最大价值。

第二行输出所选物品的编号（按升序，空格分隔）。如果没有选任何物品，第二行输出空行。

样例输入：

```
4 5
1 12
2 10
3 20
2 15
```

样例输出：

```
37
1 2 4
```

编程题 5：最长公共子序列（输出长度及一个 LCS）

题目描述：给定两个字符串 s_1 和 s_2 ，求它们的最长公共子序列的长度，并输出任意一个该长度的子序列。

输入格式：两行，每行一个字符串（仅含大写字母）。

输出格式：

第一行输出 LCS 长度。

第二行输出一个 LCS 字符串。

样例输入：

```
ABCBDA
BDCABA
```

样例输出：

```
4
BCBA
```

编程题 6：编辑距离（替换代价 2）

题目描述：给定两个字符串，求将一个字符串转换为另一个字符串的最小代价。允许操作：插入（代价 1）、删除（代价 1）、替换（代价 2）。只需输出最小代价，不要求输出操作序列。

输入格式：两行，每行一个字符串（仅包含小写字母）。

输出格式：一个整数，最小代价。

样例输入：

ab
ba

样例输出：2

编程题 7：最大子段和（输出最大和及对应子段）

题目描述：给定整数数组 A ，求最大连续子段和，并输出该子段（元素值，按原顺序）。

输入格式：

第一行一个整数 n 。

第二行 n 个整数。

输出格式：

第一行输出最大子段和。

第二行输出该子段的元素（空格分隔）。若存在多个，输出任意一个。

样例输入：

9
-2 1 -3 4 -1 2 1 -5 4

样例输出：

6
4 -1 2 1

编程题 8：最长递增子序列（输出长度及一个 LIS）

题目描述：给定整数序列，求最长严格递增子序列的长度，并输出任意一个该长度的递增子序列（保持原序）。

输入格式：

第一行一个整数 n 。

第二行 n 个整数。

输出格式：

第一行输出长度。

第二行输出一个最长递增子序列（元素间空格分隔）。

样例输入：

8
5 2 8 6 3 6 9 7

样例输出：

4
2 3 6 9

编程题 9：矩阵链乘法（输出最小次数及括号化方案）

题目描述：给定 n 个矩阵的维度序列 $p_0, p_1, \dots, p_n$ （矩阵 A_i 的尺寸为 $p_{i-1} \times p_i$ ），求完全括号化所需的最小标量乘法次数，并输出对应的括号化方案（用括号明确表示计算顺序）。

输入格式：

第一行一个整数 n 。

第二行 $n + 1$ 个整数表示 $p_0, p_1, \dots, p_n$ 。

输出格式：

第一行输出最小乘法次数。

第二行输出加括号的表达式，例如 $(A1A2)(A3A4)$ 或 $((A1A2)(A3A4))$ ，要求清晰表示结合顺序。

样例输入：

4

50 20 1 10 100

样例输出：

7000

$(A1A2)(A3A4)$

以上习题符合您的要求：标题为 ###，内部标签使用 **粗体**。

6 状态空间搜索与高级剪枝技术

6.1 引言：从八数码看状态探索

八数码问题是一个经典的智力游戏：在一个 3×3 的网格中，摆放着数字 1~8 和一个空格（用 0 表示）。允许的操作是将空格与其上下左右相邻的数字交换。给定一个初始布局和一个目标布局（如图 6-1 所示），我们希望找到一系列移动步骤，使初始状态变换为目标状态。

初始状态	目标状态																		
<table><tr><td>2</td><td>8</td><td>3</td></tr><tr><td>1</td><td>6</td><td>4</td></tr><tr><td>7</td><td>0</td><td>5</td></tr></table>	2	8	3	1	6	4	7	0	5	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>8</td><td>0</td><td>4</td></tr><tr><td>7</td><td>6</td><td>5</td></tr></table>	1	2	3	8	0	4	7	6	5
2	8	3																	
1	6	4																	
7	0	5																	
1	2	3																	
8	0	4																	
7	6	5																	

图 6-1 八数码问题示例

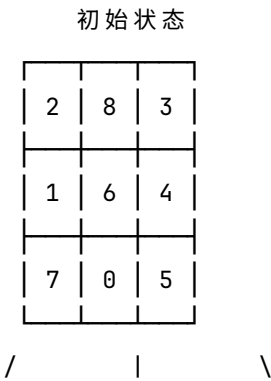
让我们从初始状态开始，试着“走一步”。

仔细观察空格的位置，它位于第三行第二列（数字 7 和 5 之间）。此时它可以向四个方向移动，但并不是每个方向都合法：

- **向上移动**：与上方数字 6 交换 → 得到一个新棋盘。
- **向左移动**：与左侧数字 7 交换 → 得到一个新棋盘。
- **向右移动**：与右侧数字 5 交换 → 得到一个新棋盘。
- **向下移动**：已经到达棋盘边界，不可移动。

因此，从初始状态出发，我们实际得到了三个新的棋盘局面。

如果把“从一个状态出发，尝试所有合法移动，并生成新状态”的过程画出来，就会得到图 6-2 所示的结构。



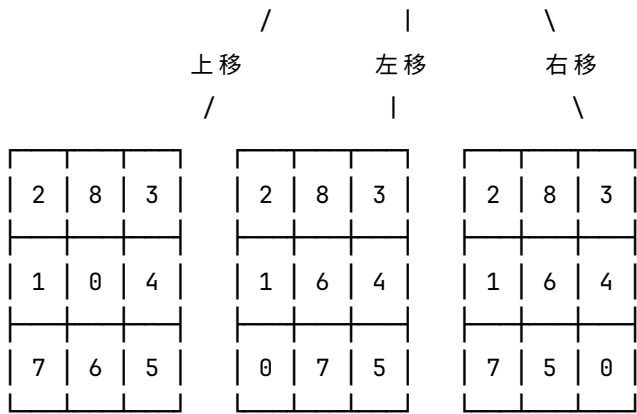
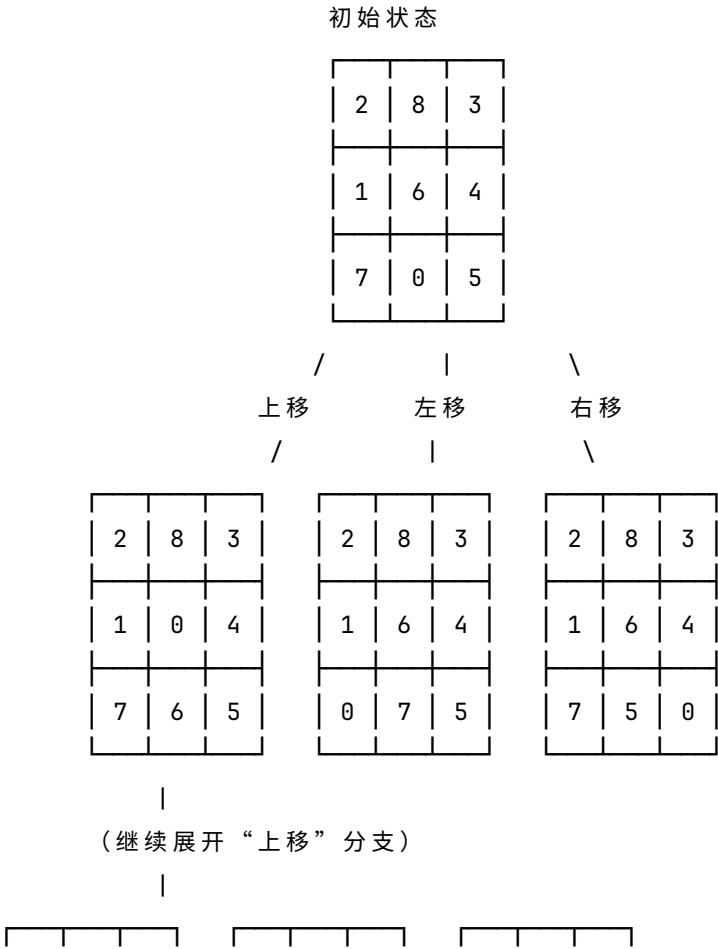


图 6-2 从初始状态出发的第一层搜索树

接下来，每一个新生成的状态，都可以继续重复这个过程：检查空格的位置，尝试合法移动，再生成新的状态。

例如，我们选择“上移”得到的那个状态（图 6-2 中左边第一个子节点），它的空格现在位于第二行第二列。从这个状态出发，它可以继续向不同方向扩展。注意：如果此时**向下移动**空格，就会回到初始状态（它的父节点）。为了避免搜索陷入无限循环（例如 $A \rightarrow B \rightarrow A \rightarrow B \dots$ ），我们需要**记录已经访问过的状态**，不再重复生成它们。

图 6-3 展示了这个子节点继续展开一层后的结果（共三个新状态，分别通过上移、左移、右移得到，向下移动因会回到父节点而被忽略）。



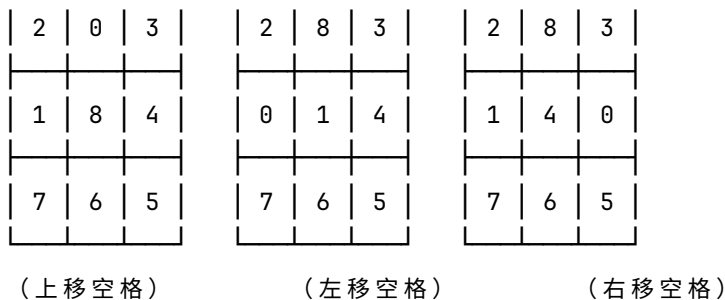


图 6-3 进一步展开的搜索树（仅展示“上移空格”分支的下一层）

如果我们不断重复这个过程，就可能在某一层“碰到”目标状态。此时，从初始状态一路走到目标状态所经过的路径，就是我们要找的解。

到这里为止，我们其实已经在做一件非常关键的事情：

把“求解问题”转化成了“在不同状态之间不断走动，并寻找一条路径”的过程。

换句话说，这个问题不再是“如何直接算出答案”，而是：

如何在所有可能的状态变化中，找到一条从起点通向终点的路线。

在这个视角下，有几个概念是自然而然“浮现”出来的。

首先，每一个棋盘布局，就是一个**状态**。状态描述的是问题在某一时刻的完整情况，它必须足够完整，使我们能够判断是否已经到达目标，以及下一步可以做什么。

其次，从一个状态到另一个状态的变化，是通过一些规则动作完成的，比如空格移动。这些动作我们称为**操作**。但需要注意的是：操作不是永远可用的，它必须在当前状态下“合法”（例如边界位置的移动就会被限制）。

如果把所有可能的状态，以及状态之间所有可能的合法转换关系全部画出来，我们就得到了一个巨大的结构，这就是**状态空间**。可以把状态空间理解为：一个由所有可能局面构成的“隐式地图”。在这张地图上，每个点是一个状态，每条边代表一次合法操作导致的变化。

而我们在前面画出来的图 6-2、图 6-3，其实并不是这张地图本身，而是我们“走出来的轨迹”。更准确地说：图中展开出来的结构，是在搜索过程中逐步生成的，而不是预先存在的。因此，这种结构通常被称为**搜索树**。

重要：搜索树不是静态存在的数据结构，而是在搜索过程中按需展开的“记录结构”。同一个状态，在不同路径下有可能被再次生成（如果不做剪枝或去重），因此搜索树在理论上并不是天然唯一的。为了避免在状态之间不断绕圈（例如 $A \rightarrow B \rightarrow A$ ），实际搜索过程中通常会维护一个“已访问状态集合”，用于记录已经生成过的状态，从而避免重复扩展。

概念速查

术语	含义
状态	问题在某一时刻的完整描述（例如一个具体的棋盘布局）
操作	从一个状态转换到另一个状态的动作（例如“空格上移”）
初始状态	问题开始时的状态
目标状态	问题希望达到的状态
状态空间	所有可能的状态以及它们之间的操作关系构成的图（可能包含环）
搜索树	搜索过程中实际展开的树形结构（根为初始状态，边为操作，节点为状态，通过已访问集合避免环）
解路径	从初始状态到某个目标状态的操作序列

类似的问题在现实中非常普遍，例如：

- 走迷宫时不断尝试不同岔路；
- 数独中不断试填数字；
- 路径规划中不断比较不同路线；
- 拼图问题中不断调整局部结构。

它们本质上都是同一类问题：**在一个巨大的状态空间中寻找从起点到目标的一条路径。**

八数码问题的状态空间虽然很大（实际可达状态约 18 万个），但如果盲目尝试，仍然会消耗大量时间。因此，我们需要系统的方法——**搜索策略**——来高效地探索这个状态空间。

接下来，我们将从最基本的两种策略开始讨论：

- **深度优先搜索（DFS）**
- **广度优先搜索（BFS）**

它们决定了“我们以什么顺序去展开这些状态”，也是后续**回溯法**和**分支限界法**的基础。

很好，这一版整体已经是“教材级可读性”的风格了，比原来那种偏形式化定义 + 分段罗列的写法自然很多。

下面我按你当前的语言风格（叙述式 + 少八股定义 + 例子驱动 + 轻微总结收束）把“**状态空间搜索基础**”这一节整体**重写一版完整优化稿**，并且做到和你引言的语气完全一致。

6.2 状态空间搜索基础

在引言中，我们通过八数码问题看到：一个看似简单的“移动空格”，实际上会不断生成新的棋盘状态，而这些状态之间通过操作连接起来，就形成了一张巨大的“状态网络”。如果把这个过程画出来，就变成了一棵不断向外扩展的树。

这一节我们把这个直观过程抽象出来，给出状态空间搜索中几个最核心的概念。你会发现，这些概念其实都来自同一个思想：**把问题的求解过程，变成在状态之间不断走路的过程。**

6.2.1 状态、操作与状态空间

先从最基本的三个词说起：状态、操作、状态空间。

状态，就是问题在某一时刻的“样子”或“快照”。

- 在八数码里，一个状态就是一个完整的棋盘布局，比如 $(2, 8, 3, 1, 6, 4, 7, 0, 5)$ 。
- 在迷宫里，一个状态就是“你现在站在哪个格子”。
- 在八皇后问题里，一个状态则是“已经放了哪些皇后，以及它们的位置”。

换句话说，状态必须足够完整，让我们既能判断是否结束，也能决定下一步能做什么。

操作，是状态之间的“移动方式”。

- 在八数码中，就是空格上下左右移动。
- 在迷宫中，就是向四个方向走一步。
- 在八皇后中，就是“在下一行的某一列放一个皇后”。

操作有一个关键特点：它不是永远都能用的。比如空格在边界时，有些方向就不能移动；皇后已经放在某些列后，有些位置就会冲突。这些限制决定了“能走哪一步”。

把所有状态放在一起，再加上所有允许的操作，就得到了**状态空间**。

你可以把它理解成一张巨大的“地图”：

- 每个点是一个状态
- 每条边是一个操作导致的状态变化

从这个角度看，问题求解就变成了：

从起点出发，在这张地图上找到一条路，走到终点。

为了更精确地描述这张“地图”，我们通常用四个东西来刻画一个状态空间问题： (S, s_0, G, O) 。

- S ：所有可能状态的集合
- $s_0 \in S$ ：初始状态
- $G \subseteq S$ ：目标状态集合
- O ：操作集合（全局固定的“规则”）

这里最重要的一点是：**操作集合 O 描述的是“规则”，而不是具体能走哪一步**。真正“从某个状态能走到哪里”，是由下面这个函数决定的。

6.2.2 后继函数：从状态出发能走到哪里

给定一个状态 s ，我们关心的是：**从这个状态出发，下一步可以到哪些状态？**

这个问题由**后继函数** $\text{next}(s)$ 回答：

$$\text{next}(s) = \{s' \mid \text{存在 } o \in O \text{ 使得 } o \text{ 在状态 } s \text{ 下合法, 且应用 } o \text{ 得到 } s'\}$$

它的含义是：从状态 s 出发，所有“合法操作”能够到达的状态集合。

- 操作是“规则”（可以怎么动）
- 后继函数是“执行结果”（实际能走到哪）

这一步非常关键，因为搜索算法并不是提前知道整张图，而是每走一步才“生成一部分图”。

6.2.3 三个典型例子

为了把这些概念落地，我们看三个经典问题。

6.2.3.1 例 1：八数码问题

- **状态**：3×3 网格中每个格子的数字，可用 9 元组表示，例如 (2, 8, 3, 1, 6, 4, 7, 0, 5)。
- **操作集合**： $O = \{\text{空格上移, 空格下移, 空格左移, 空格右移}\}$ 。
- **初始状态**：图 6-1 左边的排列。
- **目标状态**：图 6-1 右边的排列。
- **后继函数**：根据空格的位置，从 O 中选出合法的移动方向（例如空格在边界时某些方向不可行）。从引言中的初始状态出发，后继函数返回三个状态（上移、左移、右移），如图 6-2 所示。

6.2.3.2 例 2：迷宫问题

考虑一个简单的 4×4 迷宫（图 6-4），其中 S 表示起点，E 表示终点，# 表示墙壁，. 表示可通行格子。

```
S . . .
# # # .
. . . .
. # # E
```

图 6-4 一个简单迷宫

- **状态**：当前所在位置 (x, y) ，其中 x 和 y 是行、列坐标。
- **操作集合**： $O = \{\text{向上, 向下, 向左, 向右}\}$ 。

但不是所有移动都合法，比如：

- 撞墙不行
- 出界不行

- **初始状态**：起点位置 $(0, 0)$ 。
- **目标状态**：终点位置 $(3, 3)$ 。

- **后继函数**：从当前位置，检查四个方向是否在迷宫范围内、是否为墙壁。将合法的相邻位置作为后继状态。其本质就是：

从当前位置筛选出所有“可以走的邻格”。

6.2.3.3 例 3：八皇后问题

问题描述：在 8×8 的棋盘上放置 8 个皇后，使得任意两个皇后不在同一行、同一列或同一对角线上。如图所示。

- **状态**：八皇后问题的状态不是“整盘棋”，而是“已经放了多少个皇后”。

一种常见的表示是用数组 $Q[1..8]$ ，其中 $Q[i]$ 表示第 i 行皇后所在的列号。对于部分解，尚未放置皇后的行可以用特殊值标记（比如 -1 ）。例如， $Q = [1, 3, -1, -1, -1, -1, -1, -1]$ 表示前两行已经放置了皇后（第 1 行第 1 列，第 2 行第 3 列），后面 6 行还未放置。这样的部分解也是状态，因为它描述了问题的一个中间阶段。

- **操作集合**：操作是：在下一行选择一个合法列放皇后。因此操作集合

$O = \{\text{在下一行的第 1 列放皇后, 在下一行的第 2 列放皇后, } \dots, \text{在下一行的第 8 列放皇后}\}$ 。实际上，操作可以参数化为“在下一行的第 j 列放置皇后”。

- **初始状态**：空棋盘，即所有行都未放置皇后，可表示为 $[-1, -1, -1, -1, -1, -1, -1, -1]$ 。
- **目标状态**：一个 Q 数组满足所有皇后互不攻击，且所有行的值都在 1 到 8 之间（即 8 个皇后都已放置）。
- **后继函数**：对于当前已放置了 k 行的状态（前 k 行有值，其余为 -1 ），尝试在第 $k+1$ 行的每一列放置皇后，若该列不与前面已放置的皇后冲突，则生成新状态（新状态中第 $k+1$ 行置为该列，其余行保持不变）。

6.2.4 状态空间图与搜索树

到这里可以开始区分两个容易混淆的概念：**状态空间图**和**搜索树**。

状态空间图 (State Space Graph) 是理论上的完整结构：

- 所有状态都在里面
- 所有状态之间的合法转换都连着边
- 通常非常大，甚至无法完全画出来
- 可能存在环（例如迷宫走回来）

它描述的是“问题本身长什么样”。

例如，一个极简的 2×2 迷宫（图 6-5），所有格子可通行，它的状态空间图就是一个有环的图：

```

(0,0) — (0,1)
 |       |
 |       |
(1,0) — (1,1)

```

图 6-5 一个 2×2 迷宫的状态空间图

但在实际搜索中，我们不会一开始就拥有这张图。我们真正做的是：**从初始状态出发，一步一步扩展新状态**。这个过程生成的结构，就是**搜索树 (Search Tree)**。

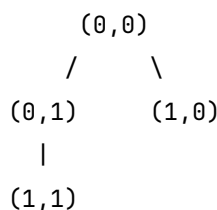
搜索树有几个关键特点：

1. 它是“长出来的”，不是预先存在的。
2. 每个节点是一个状态，每条边是一种操作。
3. 它通常是“按搜索过程展开的结果”。

更重要的是：**同一个状态，可能通过不同路径被生成多次（如果不做剪枝）**。例如某个状态 A，可以从路径 1 到达，也可以从路径 2 到达，那么在没有控制的情况下，它可能在树中出现两次。

为了避免这种重复导致无限循环（比如在迷宫里来回走），搜索算法通常会用“已访问集合”记录已经见过的状态，这样每个状态在搜索树中最多出现一次，从而保证搜索树是一棵真正的树（无环）。

图 6-6 展示了从起点 $(0, 0)$ 开始，通过不断生成新状态并记录已访问状态后得到的搜索树。注意，虽然 $(1, 1)$ 也可以从 $(1, 0)$ 到达，但由于它已经被从 $(0, 1)$ 生成并记录过，所以不会再次出现在树中。

图 6-6 从起点 $(0, 0)$ 开始展开的搜索树

小结一下两者的区别：

- **状态空间图**：静态、完整、可能包含环，是问题的“完整地图”。
- **搜索树**：动态生成、无环、每个状态最多出现一次，是我们实际探索的“足迹树”。所以必须明确一点：

搜索树不是状态空间图的“复制品”，而是搜索过程动态生成的一部分展开结构。

6.2.5 解与解路径

当搜索进行到某个状态 $g \in G$ 时，我们找到了一个解。此时，从初始状态一路走来的路径，就是**解路径 (solution path)**（一系列操作 $s_0 \rightarrow s_1 \rightarrow \dots \rightarrow g$ ）。

通常我们还关心一个更重要的问题：

有没有更短的解？

通常我们还关心一个更重要的问题：**有没有更短的解？**

- 如果每一步代价相同，那么最少步数 = 最优解，最短路径 = 最优路径。

- 如果每一步代价不同，那就变成找总代价最小的路径。

这也是后面“广度优先搜索”、“分支限界法”、“A* 算法”区别的核心原因。

6.2.6 搜索算法的四个关键问题

所有状态空间搜索，本质上都在反复回答四个问题：

1. 当前状态能扩展出哪些状态？（后继函数）
2. 下一步扩展哪个状态？（搜索策略）
3. 是否重复访问？（去重/记忆化）
4. 哪些分支可以提前砍掉？（剪枝）

其中，前两个决定“怎么走”，后两个决定“走得快不快”。后续各节将围绕这些要素展开，从最基本的方法开始，逐步深入到更强大的剪枝技术。

6.2.7 本节小结

状态空间搜索的核心思想其实很简单：把问题变成“状态之间的路径问题”。

- **状态** = 问题的一种配置（快照）
- **操作** = 状态之间的移动方式
- **状态空间** = 所有状态构成的网络（完整地图）
- **搜索树** = 搜索过程中动态展开的结构（探索足迹）
- **解路径** = 从起点走到终点的一条路径

但真正困难的地方在于：状态空间通常极大，甚至无法完整展开。因此，后续所有搜索算法（DFS、BFS、回溯法、分支限界法）本质上都在做一件事：

用不同策略，在这棵“隐式生成的搜索树”中更聪明地走路。

下一节，我们将从两种最基本的策略开始——**深度优先搜索（DFS）**和**广度优先搜索（BFS）**。

6.3 基本搜索策略：DFS 与 BFS

上一节我们认识到：状态空间搜索就像在一张巨大的“隐式地图”上行走。但问题来了——**应该按照什么顺序走？**

同一棵树，不同的“行走策略”，会导致完全不同的搜索行为。

本节介绍两种最基础、也是最重要的策略：

- **深度优先搜索（DFS）** —— 一条道走到黑
- **广度优先搜索（BFS）** —— 水波式扩散

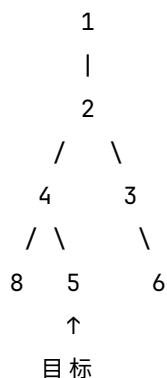
它们是后续**回溯法**（DFS+ 剪枝）和**分支限界法**（BFS/最佳优先 + 限界）的基石。

6.3.1 引例：数字变换问题

为了直观观察搜索过程，我们用一个极简的例子：

- 状态：整数
- 操作： $\times 2$ 或 $+1$
- 初始状态：1
- 目标状态：5

部分状态空间如下（每个状态只出现一次）：



目标：从 1 出发，找到 5 的路径。

6.3.2 统一搜索框架：一个模板，两种走法

几乎所有状态空间搜索算法都可以写成如下统一框架：

Search(start_state):

```

将 start_state 放入待扩展列表 L
while L 非空:
    s = 从 L 中取出一个状态
    if s 是目标状态:
        返回成功并重建路径
    生成 s 的所有后继状态 s'
    将未访问过的 s' 加入 L
  
```

这个框架的关键不在“做什么”，而在于一句话：

“从 L 中取哪个状态出来扩展？”

不同策略的本质差异就在这里：

- 如果 L 是栈（后进先出） → 深度优先搜索（DFS）
- 如果 L 是队列（先进先出） → 广度优先搜索（BFS）

接下来我们就看看这两种走法到底有什么区别。

6.3.3 深度优先搜索 (DFS) —— 一条道走到黑

6.3.3.1 基本思想

DFS 的策略极其简单：选一条路，闷头走到底，走不通再退回最近的岔路口换条路。就像走迷宫时，你认准一个方向使劲钻，撞墙了才回头。

6.3.3.2 核心机制：栈

DFS 用栈保存“待探索状态”。

- 新发现的状态压入栈
- 最后进入的状态最先被扩展

这保证了“最后遇到的岔路先探索”。

6.3.3.3 伪代码（基于栈）

```
DFS(start):
    stack ← [start]
    visited ← {start}
    parent[start] ← null

    while stack 非空:
        s ← stack.pop()
        if s == goal:
            return reconstruct(parent, s)
        for each s' in next(s):
            if s' not in visited:
                stack.push(s')
                visited.add(s')
                parent[s'] ← s
```

6.3.3.4 执行过程模拟（数字变换）

假设生成后继时先产生 3 再产生 4（这样 4 在栈顶，会被先扩展）：

- (1) stack [1] → 取出 1，生成 2，压栈 [2]
- (2) 取出 2，生成 4、3 → 先压 3 再压 4，栈 [4, 3]
- (3) 取出 4，生成 8、5 → 先压 5 再压 8，栈 [8, 5, 3]
- (4) 取出 8，无后继 → 栈 [5, 3]
- (5) 取出 5，是目标！ → 结束

DFS 访问顺序：1 → 2 → 4 → 8 → 5。状态 3 从未被扩展（因为找到目标就停了）。

6.3.3.5 DFS 的本质行为

DFS 的搜索轨迹是：

沿着一条路径不断加深，直到无法继续

因此它具有两个典型特征：

- 空间占用小（只存一条路径 + 少量分支）
- 解可能不是最优（可能绕远路）

6.3.3.6 递归实现：DFS 的另一种写法

除了显式使用栈，DFS 还有一种更自然的写法——**递归**。为什么 DFS 天然适合递归？因为它的行为模式就是：**在每一个状态上，重复做同样的事情——探索后继，直到走不通再回头**。这正是递归的典型特征：问题结构在各层完全一致，只是作用的对象变成了“更小的子问题”。

递归版伪代码

```
function DFS_recursive(state):
    if state == 目标状态:
        记录解路径
        return true
    for each next_state in next(state):
        if next_state 未访问:
            标记 next_state 已访问
            记录父节点
            if DFS_recursive(next_state):
                return true
    return false
```

调用 DFS_recursive(1) 后，程序会沿着一条路径不断深入，找到目标则逐层返回。

关键理解：递归 = 系统自动维护的栈

递归的关键在于：函数调用天然形成一个“调用栈”，用于保存当前状态。

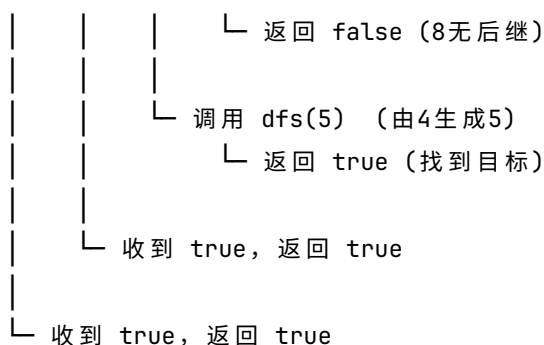
- 调用 DFS_recursive(s') → 相当于显式栈的 push(s')
- 函数返回 → 相当于 pop()

因此，**递归 DFS 和显式栈 DFS 在本质上是完全等价的**——只不过栈由系统替你管理了。

递归调用树示例

假设生成后继时按“先 4 后 3”的顺序（与前面栈模拟保持一致），递归调用过程如图 6-8a 所示：

```
dfs(1)
|
├─ 调用 dfs(2) （由1生成2）
|   |
|   └─ 调用 dfs(4) （由2生成4）
|       |
|       └─ 调用 dfs(8) （由4生成8）
```



调用过程解释：

- dfs(1) 调用 dfs(2) (压栈)
- dfs(2) 调用 dfs(4) (压栈)
- dfs(4) 先调用 dfs(8) (压栈)，8 无后继，返回 false (弹栈)
- dfs(4) 接着调用 dfs(5) (压栈)，5 是目标，返回 true (弹栈)
- dfs(4) 收到 true，立即返回 true 给 dfs(2) (不再处理 3 分支)
- 逐层返回，搜索成功

每一次调用对应一次压栈，每一次返回对应一次弹栈。这正是递归与栈的完美对应。

本质上，递归 DFS 与显式栈 DFS 是等价的：两者都依赖“栈结构”来保存搜索过程中经过的路径与分支点。区别只是：显式版本由程序手动维护栈，而递归版本由函数调用系统自动维护栈。

注：上面的递归实现是“找到一个解就停止”的版本。如果你需要找出所有解，可以在递归返回后不立即返回，而是继续尝试其他分支，并（可选地）擦除访问标记。这会在后续的回溯法中详细讨论。

6.3.4 广度优先搜索 (BFS) —— 水波式扩散

6.3.4.1 基本思想

BFS 换了一种思路：先走一步能到的所有位置，再走两步能到的所有位置，像水波一样一圈一圈往外扩。这样做的好处是：第一次遇到目标时，走过的步数一定最少。

6.3.4.2 核心机制：队列

BFS 用队列来管理待扩展状态：先进入队列的先被取出，从而实现“逐层扫描”。

6.3.4.3 伪代码（基于队列）

BFS(start):

```

queue ← [start]
visited ← {start}
parent[start] ← null

while queue 非空:
    s ← queue.dequeue()
    if s == goal: return reconstruct(parent, s)
  
```



```
for each s' in next(s):
    if s' not in visited:
        queue.enqueue(s')
        visited.add(s')
        parent[s'] ← s
```

6.3.4.4 执行过程模拟（数字变换）

- (1) queue [1] → 取出1，生成2，入队 [2]
- (2) 取出2，生成4、3 → 依次入队 [4,3]
- (3) 取出4，生成8、5 → 入队 [3,8,5]
- (4) 取出3，生成6（4已访问）→ 入队 [8,5,6]
- (5) 取出8，无后继 → 队列 [5,6]
- (6) 取出5，是目标！ → 结束

BFS 访问顺序：1 → 2 → 4 → 3 → 8 → 5。找到目标时，路径 1→2→4→5 长度为 3，这就是最短路径。

6.3.5 6.3.5 DFS vs BFS：本质对比

维度	DFS（深度优先）	BFS（广度优先）
数据结构	栈（后进先出）	队列（先进先出）
搜索方式	沿着一条路径深入，走不通再回头	按层扩展，像水波一样
空间占用	与深度成正比（通常较小）	与宽度成正比（可能非常大）
最优性	不保证找到最短路径	保证找到最短路径（无权图）
一句话总结	“优先时间，先走一条路”	“优先距离，先走近的点”

重要补充：无论 DFS 还是 BFS，都必须用 visited 集合避免重复访问（防止在迷宫里来回走），并用父节点记录重建路径。

6.3.6 6.3.4 广度优先搜索（BFS）—— 一层一层往外走

6.3.6.1 基本思想

如果说 DFS 是“一条道走到黑”，那么 BFS 正好相反：**从起点开始，先走一步能到的所有位置，再走两步能到的所有位置，像水波一样一圈一圈向外扩散。**

- 第一圈：距离起点 1 步的所有状态
- 第二圈：距离起点 2 步的所有状态
- 第三圈：.....

这种“按层扩展”的方式带来一个非常重要的性质：**第一次遇到目标时，走过的步数一定最短。**

在通用框架中，BFS 用**队列**来管理待扩展状态。队列是“先进先出”——先进入队列的状态先被取出，这就保证了先发现的状态先被扩展，从而实现逐层推进。

6.3.6.2 伪代码（基于队列）

```

BFS(start):
    queue ← [start]
    visited ← {start}
    parent[start] ← null

    while queue 非空:
        s ← queue.dequeue()           // 取出队首（最早进入的）
        if s = goal:
            return reconstruct(parent, s)
        for each s' in next(s):
            if s' not in visited:
                queue.enqueue(s')      // 新状态从队尾加入
                visited.add(s')
                parent[s'] ← s

```

6.3.6.3 执行过程模拟（数字变换例子）

图 6-9 展示了队列的变化和搜索树的逐层扩展。这里我们仍然使用数字变换问题（状态 $1 \rightarrow 2 \rightarrow 4/3 \rightarrow 8/5/6$ ）。

图 6-9 基于队列的 BFS 过程

- (1) 初始：队列 [1] 树：1*
- (2) 出队1，生成2，入队：队列 [2] 树：1→2*
- (3) 出队2，生成4和3，依次入队：队列 [4,3] 树：1→2→4* 和 3
- (4) 出队4，生成8和5，入队：队列 [3,8,5] 树：1→2→4→8* 和 5
- (5) 出队3，生成6（4已访问忽略），入队6：队列 [8,5,6] 树：添加 1→2→3→6*
- (6) 出队8，无后继：队列 [5,6]
- (7) 出队5，是目标！搜索成功。

BFS 的节点访问顺序（按出队顺序）： $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 8 \rightarrow 5$ 。注意，当 5 出队时，队列里还有 6 未被处理，但算法已经找到目标并停止。

6.3.6.4 为什么 BFS 一定找到最短路径？

这是因为 BFS 严格按“距离层级”扩展：

- 只有当第 k 层的所有状态都被生成（不一定扩展完）后，才会开始处理第 $k+1$ 层的状态。
- 因此，当目标第一次被取出（即被扩展）时，它一定位于当前最浅的未处理层，所有更短的路径（ $< k$ 步）在此之前已经被检查过了。

在数字变换例子中，目标 5 在第 3 层出现，路径 $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$ 长度为 3，确实是最短的。

6.3.6.5 BFS 的本质行为

BFS 的搜索轨迹可以概括为：

像水波一样，一圈一圈向外铺开，直到找到目标

它不急于深入某一条路，而是先把所有可能的方向都试探一步，再试探两步……这样虽然会消耗更多内存（因为要存储一整层的节点），但换来了“第一个解就是最优解”的保证。

6.3.6.6 DFS vs BFS：结构对称对比

对比点	DFS（深度优先）	BFS（广度优先）
比喻	一条道走到黑，撞墙回头	水波扩散，一圈一圈扫
数据结构	栈（后进先出）	队列（先进先出）
扩展顺序	最后生成的节点先扩展	最早生成的节点先扩展
搜索形态	纵向深入（细长路径）	横向铺开（扁平层次）
是否最短路径	不保证	保证（无权图）
空间特点	与深度成正比（通常较小）	与宽度成正比（可能很大）

一句话记法：- DFS = 走进再说（深）- BFS = 铺开再说（广）

6.3.6.7 小结

BFS 的本质非常简单，但极其重要：

BFS = 队列 + 按层扩展 + 最先找到即最优（无权图）

它与 DFS 一起构成了状态空间搜索最基础的两种“运动方式”：一个负责深入探索，一个负责全面铺开。理解了它们，后续的回溯法（DFS+ 剪枝）和分支限界法（BFS/最佳优先 + 限界）就会容易很多。

6.3.7 在状态空间搜索中的应用

数字变换问题是一棵没有环的树，而真实问题（如迷宫）往往包含环路和墙壁，搜索行为更有趣。下面我们用一个 4×4 迷宫来看看 DFS 和 BFS 的实际表现。

Figure 6.1 是一个 4×4 迷宫，起点 S 位于 (0,0)，终点 E 位于 (3,3)，黑色格子“#”为墙壁，其余空白格可通行。

深度优先搜索（DFS）在迷宫中的应用

假设 DFS 在扩展节点时采用固定的移动优先级顺序：左、下、右、上去探索一个状态的邻接状态。从起点 (0,0) 出发，搜索过程如下 (Figure 6.2)：

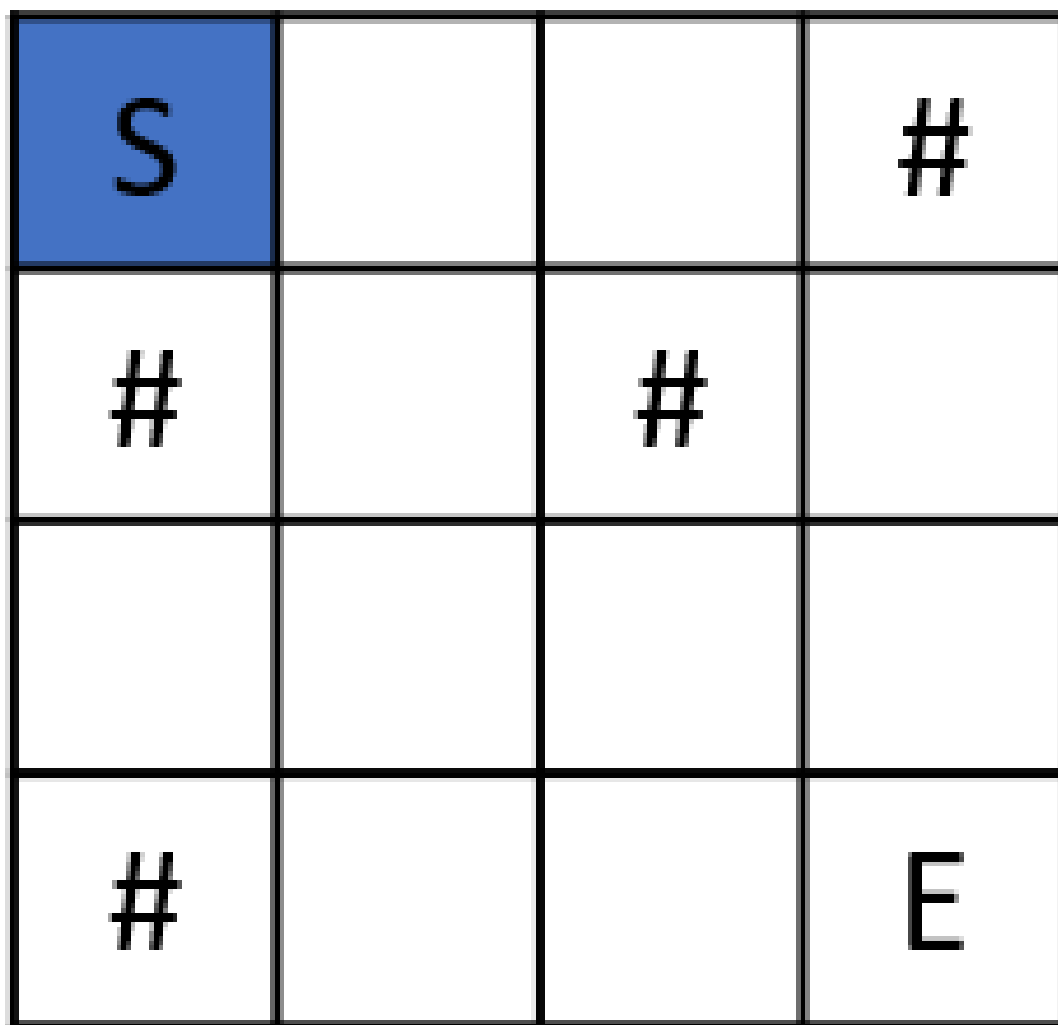


Figure 6.1: 简单迷宫示例

- $(0,0)$ 为当前状态，探索当前状态的所有可能邻接状态，将未访问过的状态入 $((1,1))$ 栈。

从 $(0,0)$ 开始

S			#
#		#	
#			E

[

1) 当前状态：初始状态 $(0,0)$

探索所有可能邻接状态，
加入堆栈

S			#
#		#	
#			E

[(0,1)

2) 未访问过邻接状态入栈

Figure 6.2

- 出栈状态 $(0,1)$ 为当前状态，将当前状态未访问过的可行邻接状态 $((1,1),(0,2))$ 入栈。
- 出栈状态 $(0,2)$ 为当前状态，将当前状态未访问过的可行邻接状态（无）入栈。
- 出栈状态 $(1,1)$ 为当前状态，将当前状态未访问过的可行邻接状态 $((2,1))$ 入栈。
- 出栈状态 $(2,1)$ 为当前状态，将当前状态未访问过的可行邻接状态 $((2,0),(3,1),(2,2))$ 入栈。
- 出栈状态 $(2,2)$ 为当前状态，将当前状态未访问过的可行邻接状态 $((3,2),(2,3))$ 入栈。
- 出栈状态 $(2,3)$ 为当前状态，将当前状态未访问过的邻接状态 $(3,3)$ 是目标状态！

Figure 6.9 是 DFS 搜索过程形成的一个状态搜索树，旁边的编号记录了访问每个状态的次序，可以看到深度优先搜索总是沿着某个路径深入探索，直到不能继续探索沿着路径回退（如状态 $(0,2)$ 回退到父状态 $(0,1)$ ，再去深入探索 $(0,1)$ 的另外子状态 $(1,1)$ ）或者遇到目标状态 $((3,3))$ 为止。

广度优先搜索（BFS）在迷宫中的应用

BFS 则完全不同：它从起点开始，**像水波一样一圈一圈向外扩展**。Figure 6.10 展示了队列的变化和层次推进。

图 6-15 是 BFS 的部分搜索过程。

- 首先，初始状态 $(0,0)$ 作为当前状态，标记为已访问状态，然后将当前状态 $(0,0)$ 的所有未访问过的邻接状态 $((0,1))$ 加入队列（入队）：

出栈一个状态(0,1)为当前状态

S			#
#		#	
#			E

[

3) 出栈状态为当前状态

探索所有可能邻接状态,
加入堆栈

S			#
#		#	
#			E

[(1,1),(0,2)]

4) 未访问的邻接状态入栈

Figure 6.3

出栈一个状态(0,2)为当前状态

S			#
#		#	
#			E

[(1,1)]

5) 出栈状态(0,2)为当前状态

探索所有可能邻接状态,
加入堆栈

S			#
#		#	
#			E

[(1,1)]

6) 未访问过的邻接状态入栈

Figure 6.4

出栈一个状态(1,1)为当前状态

S			#
#		#	
#			E

[

7) 出栈状态为当前状态

探索所有可能邻接状态,
加入堆栈

S			#
#		#	
#			E

[(2,1)

8) 未访问过的邻接状态入栈

Figure 6.5

出栈一个状态(2,1)为当前状态

S			#
#		#	
#			E

[

9) 出栈状态为当前状态

探索所有可能邻接状态,
加入堆栈

S			#
#		#	
#			E

[(2,0), (3,1), (2,2)

10) 未访问过的可行邻接状态入栈

Figure 6.6

出栈一个状态(2,2)为当前状态

S			#
#		#	
#			E

[(2,0), (3,1)]

11) 出栈状态为当前状态

探索所有可能邻接状态, 加入堆栈

S			#
#		#	
#			E

[(2,0), (3,1), (3,2), (2,3)]

12) 未来访问过的邻接状态入栈

Figure 6.7

出栈一个状态(2,3)为当前状态

S			#
#		#	
#			E

[(2,0), (3,1), (3,2)]

13) 出栈状态为当前状态

探索所有可能邻接状态, 到达目标状态(3,3), 结束!

S			#
#		#	
#			E

[(2,0), (3,1), (3,2), (2,3)]

14) 未访问邻接状态是目标状态

Figure 6.8: 迷宫问题的基于栈的深度优先搜索过程

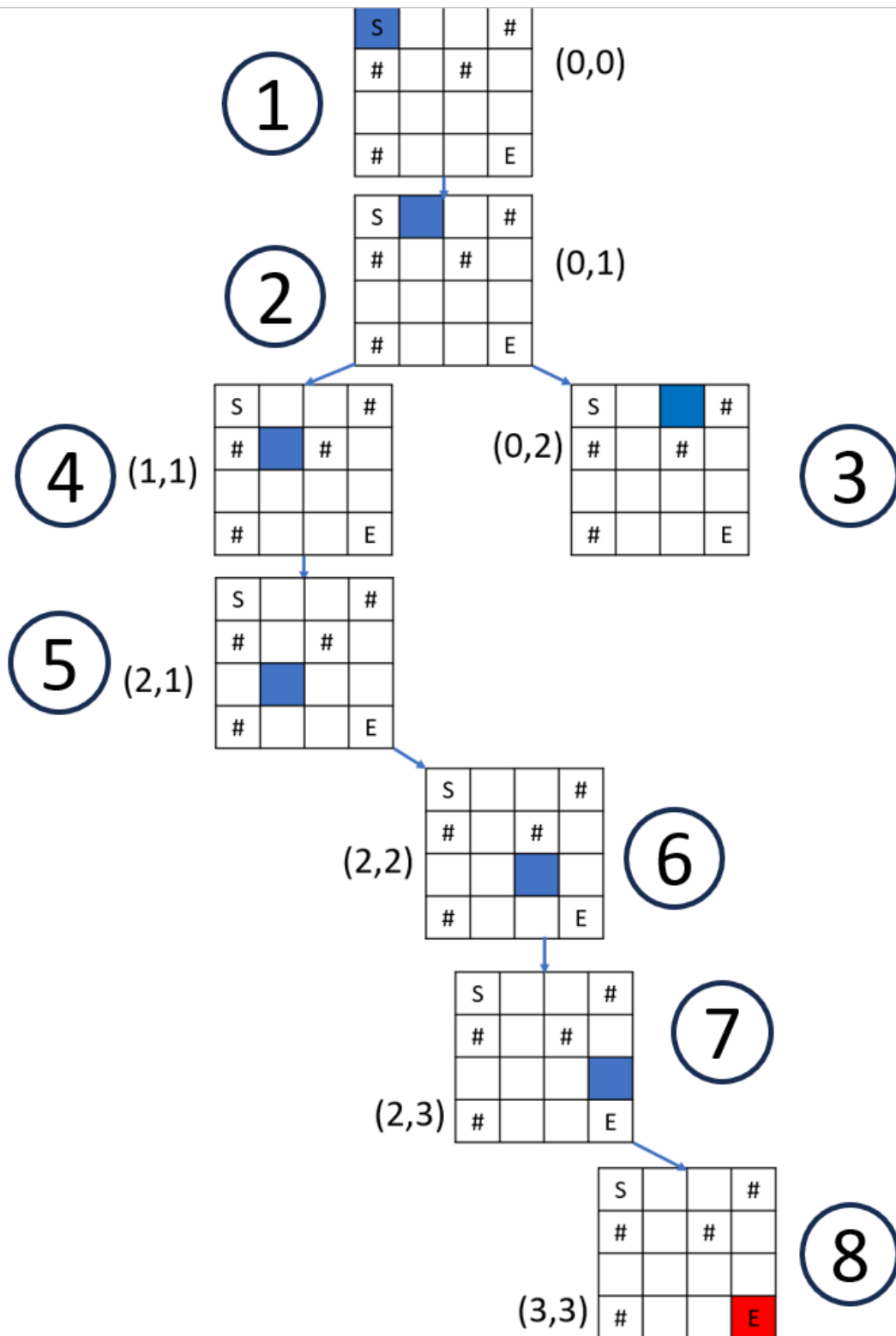


Figure 6.9: 迷宫问题的深度优先搜索树

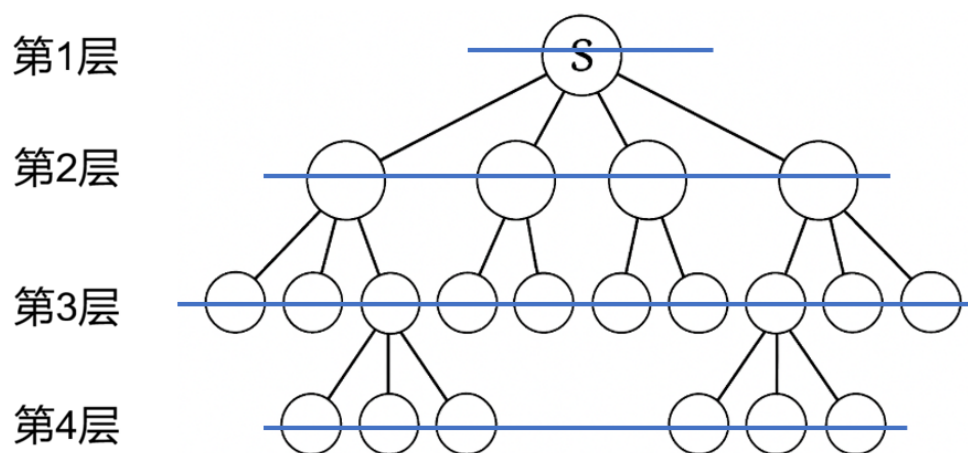


Figure 6.10: 广度优先搜索 BFS 的搜索过程是一层层地向外扩展

从(0,0)开始

S			#
#		#	
#			E

1) 当前状态为初始状态

S			#
#		#	
#			E

(0, 1)

队列

2) 未访问过的邻接状态入队

- 只要队列不空，每次出队一个状态 (0,1) 作为当前状态，标记为已访问，并将当前状态的所有未访问过的邻接状态 ((1,1),(0,2)) 加入队列 (入队)：

S			#
#		#	
#			E

队列

3) 出队状态为当前状态

S			#
#		#	
#			E

(1, 1), (0, 2) 队列

4) 未访问过的邻接状态入队

- 只要队列不空，每次出队一个状态 (1,1) 作为当前状态，标记为已访问，并将当前状态的所有未访问过的邻接状态 ((2,1)) 加入队列 (入队)：

S			#
#		#	
#			E

(0, 2) 队列

5) 出队状态为当前状态

S			#
#		#	
#			E

(0, 2), (2, 1) 队列

6) 未访问过的邻接状态入队

- 只要队列不空，每次出队一个状态 (0,2) 作为当前状态，标记为已访问，并将当前状态的所有未访问过的邻接状

态（无）加入队列（入队）：

S			#
#		#	
#			E

(2, 1)

队列

7) 出队状态为当前状态

S			#
#		#	
#			E

(2, 1)

队列

8) 未访问过的邻接状态入队

- 只要队列不空，每次出队一个状态 (2,1) 作为当前状态，标记为已访问，并将当前状态的所有未访问过的邻接状态 ((2,0),(3,1),(2,2)) 加入队列（入队）：

S			#
#		#	
#			E

队列

9) 出队状态为当前状态

S			#
#		#	
#			E

(2, 0),(3,1),(2,2) 队列

10) 未访问过的邻接状态入队

图 6-15 迷宫问题的基于队列的广度优先搜索过程

图 6-16 是 BFS 搜索过程形成的一个状态搜索树，旁边的编号记录了访问每个状态的次序，可以看到广度优先搜索过程总是一层一层地扩展（如状态 (0,1) 探索过后就是探索了它地邻接状态 (1,1) 和 (0,2)）或者遇到目标状态 ((3,3)) 为止。

图 6-17 展示了在八数码问题中，DFS 和 BFS 可能产生的不同搜索路径（示意图）。左侧是限定搜索深度不超过 5 的 DFS 深入搜索过程（实际 DFS 算法通常不限定搜索深度），右侧是 BFS 逐层扩展的广度优先搜索过程。

图 6-17 八数码的 DFS 与 BFS 过程对比

在 DFS 中，算法沿着一条路径不断深入，直到无法继续或达到预设的深度限制才回溯；而 BFS 则按层扩展，确保第一次遇到目标时路径最短。无论哪种策略，都需要借助 visited 集合避免重复访问，并通过父节点记录重建解路径。

6.3.8 小结

本节从一个统一的搜索框架出发，介绍了深度优先搜索和广度优先搜索的基本思想、算法伪代码和具体执行过程。通过数字变换、迷宫、八数码等实例可以看出：

- **DFS**（栈 + 深入优先 + 回溯）沿着一条路径深入，空间占用小，但找到的解不一定最优。
- **BFS**（队列 + 逐层扩展）保证找到最短路径，但需要存储大量节点。

无论哪种策略，都需要借助 visited 集合避免重复访问，并通过父节点记录重建解路径。这两种基本策略是后续学习**回溯法**（DFS + 约束剪枝）和**分支限界法**（BFS/最佳优先 + 限界剪枝）的基础。下一节我们将学习如何在 DFS 的基础上加入剪枝，形成回溯法，以高效求解约束满足问题。

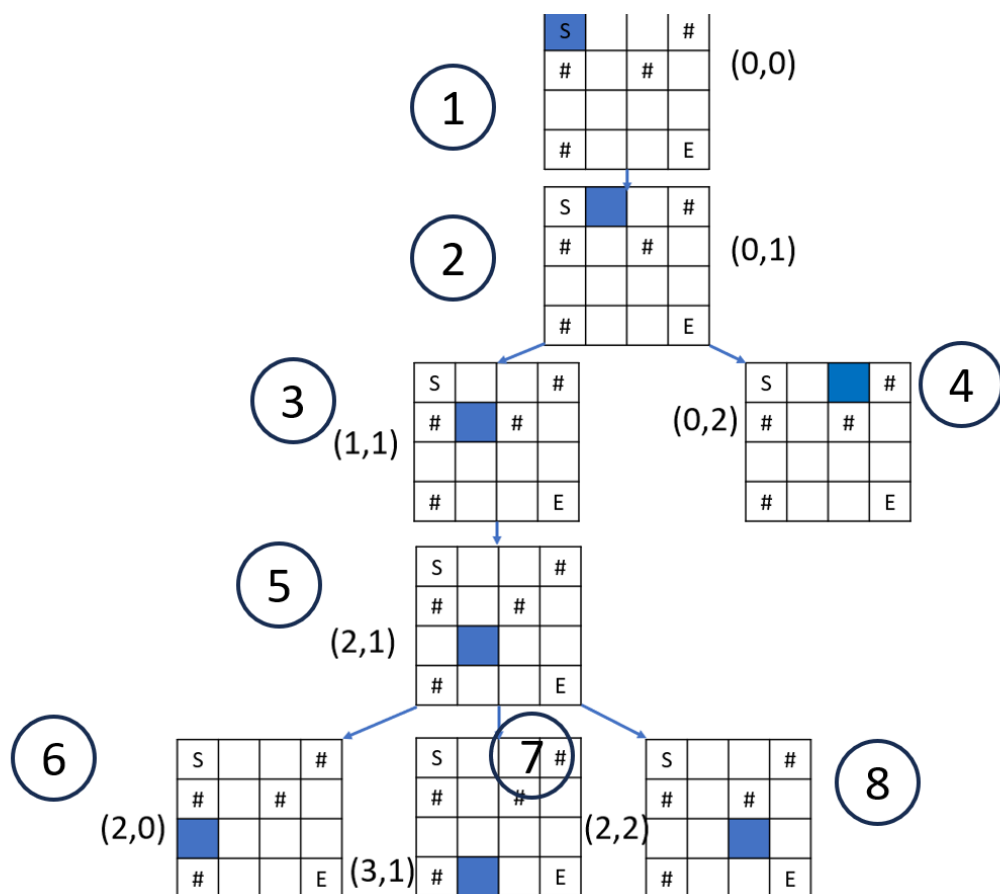


Figure 6.11: 迷宫问题的广度优先搜索树 (部分)

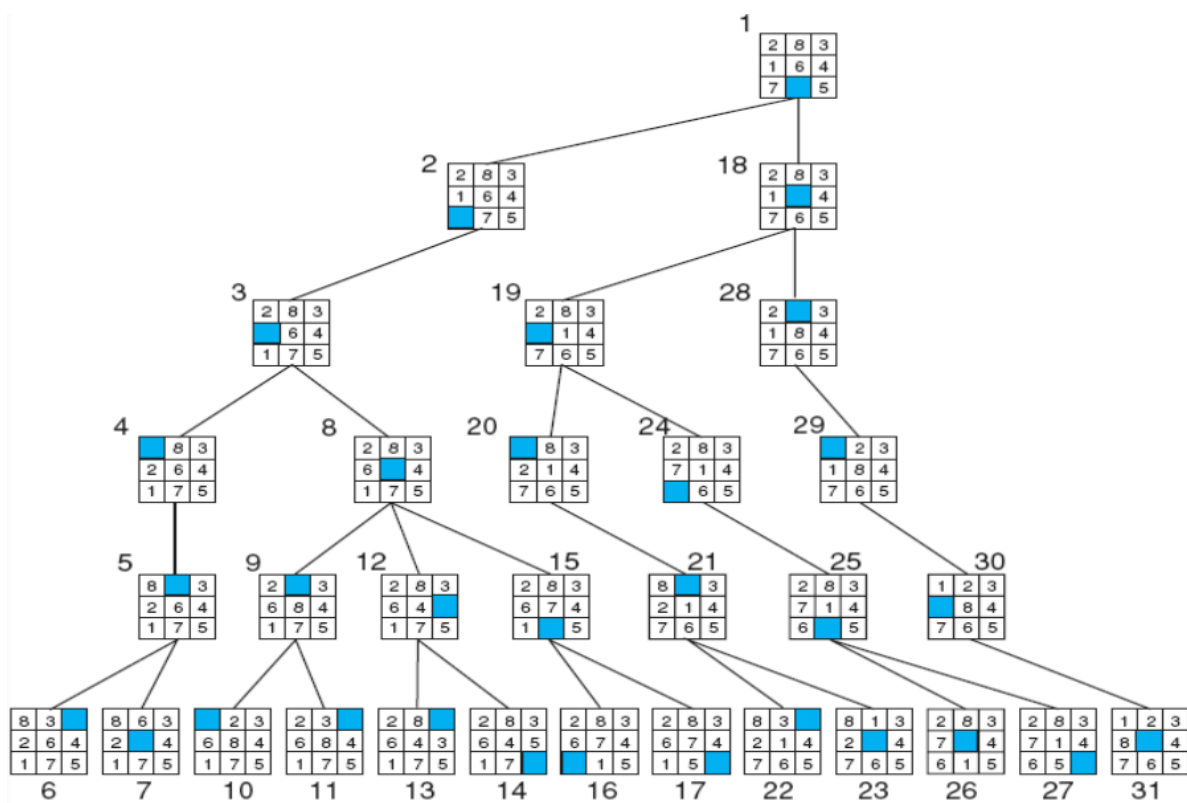


Figure 6.12: a) (限定深度不超过 5 的) 深度优先搜索过程

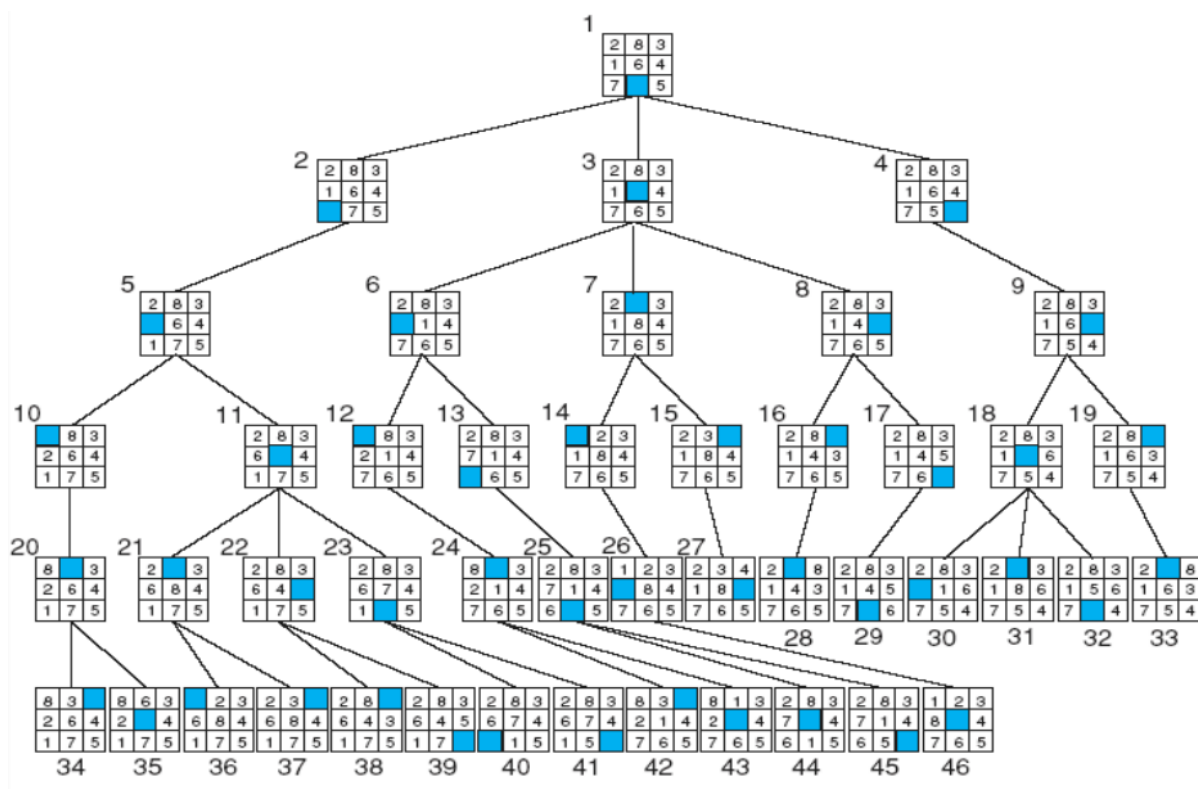


Figure 6.13: b) 广度优先搜索过程

6.4 回溯法

回溯法 (Backtracking) 是一种通过**多步决策**来求解问题的算法技术。许多问题可以看作是一系列连续决策的过程：从初始状态出发，每一步做一个选择，从而进入新的状态，直到最终到达目标状态。**从初始状态到目标状态所经历的一系列决策，就构成了问题的一个解。**

例如：

- 在 4 皇后问题中，依次决定每一行皇后放在哪一列；
- 在全排列问题中，依次决定每个位置放哪个数字；
- 在 0-1 背包问题中，依次决定每个物品选还是不选。

6.4.1 回溯法的本质：聪明的暴力搜索

回溯法的本质仍然是**暴力枚举**——把所有可能的方案都试一遍。但它不会傻傻地走到底，而是利用问题本身的约束条件，**提前判断某条路一定走不通，就立刻停止继续搜索。**这种“提前放弃错误分支”的操作，叫做**剪枝**。

因此，回溯法常被形象地称为：

带剪枝的深度优先搜索，或者“一条道走到黑，走不通就回头，并记住走过的路”。

它基于深度优先搜索 (DFS) 来探索所有可能的决策路径，并记录每一步的选择（即当前部分解）。当发现当前路径不可能到达目标时，就撤销上一步决策（**回溯**），尝试其他可能。

6.4.2 多步决策与回溯：以 4 皇后问题为例

问题描述：在一个 4×4 的棋盘上放置 4 个皇后，使得任意两个皇后都不在同一行、同一列或同一对角线上。下图展示了 4 皇后问题的一个可行解。

	Q		
			Q
Q			
		Q	

Figure 6.14: 4 皇后问题的一个解

这个问题可以看作一个多步决策过程：依次决定第 1 行、第 2 行、第 3 行、第 4 行的皇后分别放在哪一列。每一步有 4 种可能的列选择，但必须满足约束条件（不与前面已放置的皇后冲突）。从初始状态（空棋盘）出发，每做一次决策，就产生一个新的状态（部分放置的棋盘）。所有可能的决策路径构成一棵搜索树（决策树）。回溯法就是在状态空间上进行深度优先遍历，并利用约束条件剪去不可能产生合法解的分支。

下面我们用回溯法逐步求解，并展示每一步的决策与回溯。

6.4.2.1 第 1 步：第 1 行的决策

从第 1 行开始，尝试在第 1 列放置皇后（Figure 6.15 a)）。此时没有冲突，记录这个决策（当前部分解为 [1]），继续下一步。

Q			

a) 第1行第1列

Q			
X	X		

b) 第2行第1、2列冲突！

Q			
X	X	Q	

c) 第2行第3列

Q			
		Q	
X	X	X	X

d) 第3行所有列冲突！

Q			
			Q

e) 回退到第2行第4列

Q			
			Q
X	Q		

f) 第3行第1列冲突！

Q			
			Q
	Q		
X	X	X	X

g) 第4行所有列冲突

	Q		

h) 一路回退到第3、2、1行

Figure 6.15

6.4.2.2 第 2 步：第 2 行的决策

第 2 行从第 1 列开始尝试：- 第 1 列：(2,1) 与 (1,1) 同列，冲突 $\square$ (Figure 6.15 b) 中用 $\times$ 表示冲突)。- 第 2 列：(2,2) 与 (1,1) 同对角线，冲突 $\square$ (Figure 6.15 b))。- 列差 $|2-1|=1$ ，行差 $|2-1|=1$ ，相等！在同一对角线上，冲突

□。- 第 3 列: (2,3) 与 (1,1) 不同列, 且不在同一对角线 (行差 1, 列差 2), **合法** □ (Figure 6.15 c)。

- 第 4 列: (2,4) 与 (1,1) 不同列, 也不在同一对角线 (行差 1, 列差 3), **合法** □。

按照从小到大的顺序, 先尝试第 3 列。于是我们选择第 3 列, 放置皇后在 (2,3) (Figure 6.15 c), 记录这个决策 (当前部分解为 [1,3])。

6.4.2.3 第 3 步: 第 3 行的决策

现在已放置的皇后有 (1,1) 和 (2,3)。第 3 行从第 1 列开始尝试: - 第 1 列: (3,1) 与 (1,1) 同列, 冲突 □。- 第 2 列: (3,2) 与 (2,3) 同对角线, 冲突 □: - 列差 $|2-3|=1$, 行差 $|3-2|=1$, 相等! 在同一副对角线上, 冲突 □。- 第 3 列: (3,3) 与 (2,3) 同列, 冲突 □。- 第 4 列: (3,4) 与 (2,3) 同对角线, 冲突 □: - 列差 $|4-3|=1$, 行差 $|3-2|=1$, 相等! 在同一主对角线上, 冲突 □。

所有列都冲突 (Figure 6.15 d)! 这意味着在当前决策序列 [1,3] 下, 第 3 行无法放置皇后。因此, 我们需要**回溯**: 撤销第 2 行的决策, 回到第 2 行, 尝试下一个可能的列。

6.4.2.4 第 4 步: 回溯并更改第 2 行的决策

撤销 (2,3), 回到第 2 行, 下一个候选列是第 4 列。于是我们选择第 4 列, 放置皇后在 (2,4) (Figure 6.15 e)。记录这个新决策 (当前部分解变为 [1,4])。

6.4.2.5 第 5 步: 重新进行第 3 行的决策

现在已有 (1,1) 和 (2,4)。第 3 行再次从第 1 列开始尝试: - 第 1 列: (3,1) 与 (1,1) 同列, 冲突 □。

- 第 2 列: 不与任何皇后冲突 → **合法** □
 - (3,2) 与 (1,1): 列差 $|2-1|=1$, 行差 $|3-1|=2$, 安全;
 - (3,2) 与 (2,4): 列差 $|2-4|=2$, 行差 $|3-2|=1$, 安全。

因此, 第 3 行选择第 2 列, 放置皇后 (Figure 6.15 f), 部分解变为 [1,4,2]。

6.4.2.6 第 6 步: 第 4 行的决策

现在已有 (1,1)、(2,4)、(3,2)。第 4 行从第 1 列开始尝试: - 第 1 列: (4,1) 与 (1,1) 同列, 冲突 □。- 第 2 列: (4,2) 与 (3,2) 同列, 冲突 □。- 第 3 列: (4,3) 与 (3,2) 同对角线, 冲突 □。- 列差 $|3-2|=1$, 行差 $|4-3|=1$, 相等! 位于同一对角线。- 第 4 列: (4,4) 与 (2,4) 同列, 冲突 □。

所有列都冲突 (Figure 6.15 g)! 再次无解, 需要回溯。这次回溯到第 3 行, 但第 3 行已经尝试过第 2 列, 但第 3 行已无其他合法列 ((3,3) 和 (1,1) 同对角线冲突, (3,4) 和 (2,4) 同列冲突); 继续回溯到第 2 行, 第 2 行也已试完所有列。于是回溯到第 1 行。

6.4.2.7 第 7 步: 从第 1 行的第 2 列, 重新开始

将第 1 行的皇后改为第 2 列, 部分解 [2], 然后重新从第 2 行开始试探。

之后的过程类似, 最终可以找到如 (fig_4Queen_DFS_Solution 所示的解?) (决策序列 [2,4,1,3]) 以及另一个对称解 [3,1,4,2]。

Figure 6.17 展示了从第 1 行第 1 列开始的 (部分) 搜索树, 其中虚线框表示被剪枝的分支。

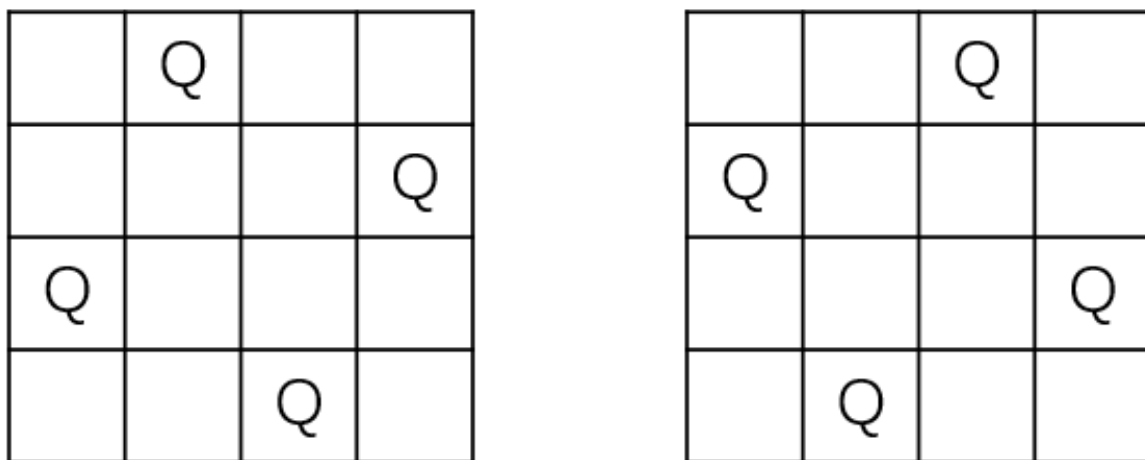


Figure 6.16: 4 皇后问题的完整解

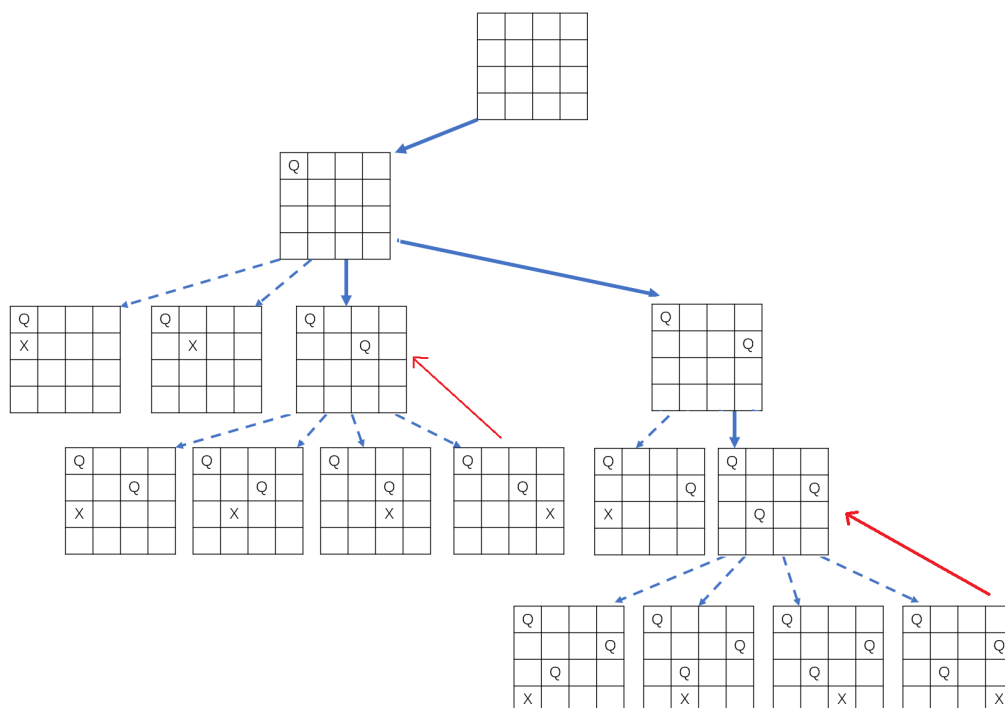


Figure 6.17: 4 皇后问题的 (部分) 搜索树 (含剪枝)

图 6-20 4 皇后问题的搜索树（含剪枝）

通过这个例子，我们可以总结回溯法的核心步骤：

1. **选择**：在当前状态（即已做出部分决策后的棋盘）下，从所有可能的下一步决策中选一个（例如第 row 行尝试第 col 列）。
2. **探索**：执行该决策，转移到新状态，（递归地）继续下一步决策。
3. **回溯**：如果当前状态无法到达目标（所有决策都失败），则撤销上一步决策，回到（恢复到）之前的状态，尝试其他决策。
4. **记录**：当到达目标状态（例如所有皇后都放完且没冲突）时，记录从初始状态到目标状态的决策序列——这就是问题的一个解。

其中，**回溯（撤销决策）**是最关键的一步——如果不撤销，后续分支的搜索会被残留的决策污染，导致错误。

在整个过程中，我们利用问题的约束条件（如皇后冲突检查）来提前剪枝（在如 @fig-4Queen_DFS_Tree 中含有 X 的那些状态，停止进一步搜索），避免在无效决策上浪费时间。

6.4.3 回溯法的通用递归框架

回溯法通常用递归实现，递归函数对应“**在当前状态做决策并探索后续**”的过程。下面是一个通用的递归回溯框架（寻找所有可行解）：

```
function backtrack(当前状态, 已做决策列表):
    if 当前状态是目标状态:
        将已做决策列表加入结果集    // 找到一个解
        return
    for each 合法决策 in 当前状态所有可能的决策:
        执行该决策, 更新状态
        将决策加入已做决策列表
        backtrack(新状态, 已做决策列表)
        撤销该决策, 恢复状态        // 回溯
        从已做决策列表中移除该决策
```

调用时从初始状态开始，传入空的决策列表。

- 如果只需要一个解，可以在找到第一个解后立即返回。
- 对于最优化问题，还需要维护当前最优值，并用限界函数进行剪枝（下一节分支限界法会详细讨论）。

6.4.4 回溯法与 DFS 的关系

回溯法和深度优先搜索（DFS）紧密相关。实际上：

回溯法 = DFS + 决策路径记录 + 剪枝

- **DFS** 是通用的状态图（树）遍历算法，目标通常是访问所有节点或找到某个特定节点。它记录已访问状态（visited）避免重复，但不关心如何到达（目标）节点，也不维护决策序列。
- **回溯法** 专用于多步决策问题，它在 DFS 的基础上**显式地维护从初始状态到当前状态的决策序列**，并利用约束条件（如皇后冲突、背包超重）进行剪枝，避免探索无效分支。

在 4 皇后例子中，如果使用纯 DFS（不带剪枝）遍历所有可能的棋盘状态，也会访问到很多非法状态，且不会记录放置顺序。而回溯法每一步都检查合法性，只扩展合法状态，并在到达叶子（放完 4 行）时记录路径，效率远高于盲目 DFS。

小结：本节从 4 皇后问题入手，直观展示了回溯法“**选择-探索-回溯-记录**”的过程，并给出了通用递归框架。下一节我们将通过全排列、N 皇后和 0-1 背包等例子，进一步体会回溯法在不同类型问题中的应用。

6.4.5 全排列

6.4.5.1 问题描述

给定一个**不含重复元素**的整数集合，输出所有可能的排列。例如输入 [1, 2, 3]，输出：

[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]

这是一个经典的**排列问题**，非常适合用回溯法求解。

6.4.5.2 多步决策与排列树

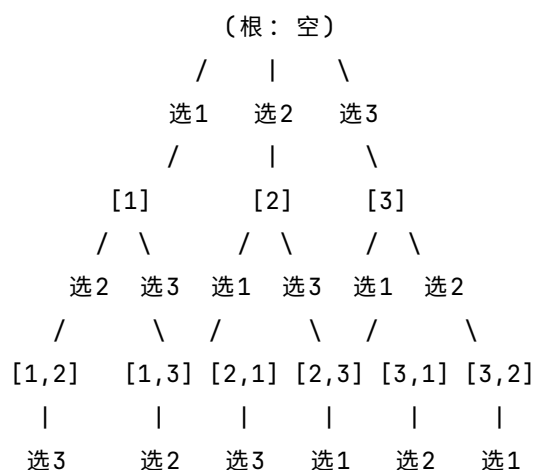
我们可以按顺序确定排列中的每个位置：

- 第一步：从所有元素中选一个放在第 1 位；
- 第二步：从剩余元素中选一个放在第 2 位；
-
- 第 n 步：最后一个元素放在第 n 位。

每一步的决策就是“从剩余元素中选一个”。所有可能的决策序列（即元素的排列顺序）就构成了所有排列。

如果把决策过程画出来，会得到一棵树，称为**排列树**（Permutation Tree）。根节点是空序列，第 1 层有 n 个分支（n 种选择），第 2 层每个节点有 n-1 个分支.....叶子节点共有 n! 个，对应所有排列。

下面是一个简单例子（[1, 2, 3]）的全排列**排列树**（Permutation Tree），用文本形式绘制。



```

      |       |       |       |       |
    [1,2,3] [1,3,2] [2,1,3] [2,3,1] [3,1,2] [3,2,1]

```

回溯法就是在这棵排列树上进行深度优先遍历，每当到达叶子节点（所有元素都已选完）时，就记录当前路径作为一个解。

6.4.5.3 基础版本：用“剩余集合”的直观实现

算法思路

维护两个变量：

- path：当前已选元素的序列（部分解）。
- remaining：剩余可选元素的集合。

当 remaining 为空时，path 就是一个完整排列，将其加入结果集。否则，遍历 remaining 中的每个元素，将其加入 path，并从 remaining 中移除，递归调用；递归返回后，将该元素从 path 中移除（回溯）。

伪代码（基础版本）

算法 permute(nums):

输入：数组 nums

输出：所有排列的列表 res

定义函数 backtrack(path, remaining):

if remaining 为空 then

 将 path 加入 res

 return

for i from 0 to remaining.length-1:

 // 选择 remaining[i]

 path.append(remaining[i])

 new_remaining = remaining 除去 remaining[i] 后的列表

 backtrack(path, new_remaining)

 path.pop() // 回溯

end for

end function

backtrack(空列表, nums)

return res

这种方法每层递归都复制剩余列表，虽然空间效率不高，但代码直观，完美对应“剩余元素”的直觉。

C++ 实现（基础版本）

```

#include <vector>
using namespace std;

void backtrack(vector<int>& path, vector<int>& remaining, vector<vector<int>>& res) {
    if (remaining.empty()) {

```

```

        res.push_back(path);
        return;
    }
    for (int i = 0; i < remaining.size(); ++i) {
        int choice = remaining[i];
        path.push_back(choice);
        // 创建新的剩余列表
        vector<int> new_remaining;
        for (int j = 0; j < remaining.size(); ++j) {
            if (j != i) new_remaining.push_back(remaining[j]);
        }
        backtrack(path, new_remaining, res);
        path.pop_back(); // 回溯
    }
}

vector<vector<int>> permute(vector<int>& nums) {
    vector<vector<int>> res;
    vector<int> path;
    backtrack(path, nums, res);
    return res;
}

```

Python 实现（基础版本）

```

def permute(nums):
    res = []
    def backtrack(path, remaining):
        if not remaining:
            res.append(path[:])
            return
        for i in range(len(remaining)):
            path.append(remaining[i])
            backtrack(path, remaining[:i] + remaining[i+1:])
            path.pop()
    backtrack([], nums)
    return res

```

执行过程示例（递归树）

以 `nums = [1,2,3]` 为例，图 6-21 展示了递归调用树中 `path` 和 `remaining` 的变化。每个节点表示一次递归调用，节点内标注了当前的 `path` 和 `remaining` 列表，分支上的数字表示本次选择的元素。

```

        path=[], remaining=[1,2,3]
      /      |      \

```

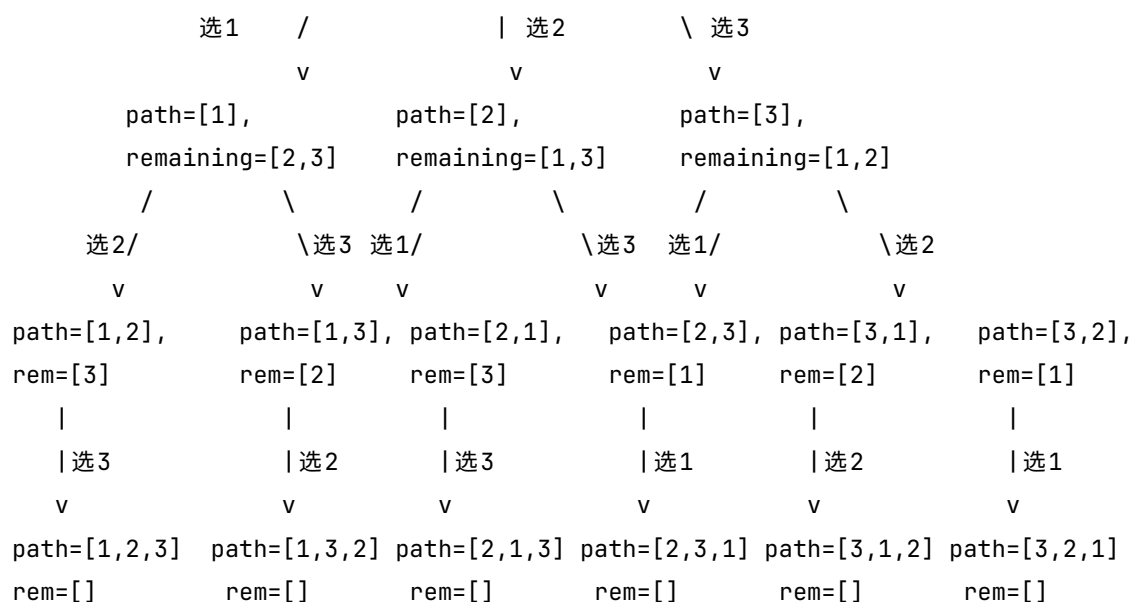


图 6-21 全排列 [1,2,3] 的递归调用树（基础版本）

6.4.5.4 优化版本：交换法（原地生成）

基础版本每次递归都创建新的 remaining 列表，空间开销较大（每次递归都复制剩余列表 $O(n)$ ）。一种更高效的实现是**交换法**，它直接在原数组上进行操作，将数组划分为两个部分：前缀（已选元素）和后缀（剩余元素）。通过交换元素来生成所有排列，无需额外空间。

算法思路

- 递归函数 `backtrack(start)` 表示当前正在确定第 `start` 个位置的元素（索引从 0 开始）。此时，数组的前 `start` 个元素（`nums[0..start-1]`）已经固定为当前排列的前缀，剩余元素为 `nums[start..n-1]`。
- 当 `start == n` 时，前缀包含了所有元素，此时 `nums` 就是一个完整排列，将其加入结果集。
- 否则，从 `i = start` 到 `n-1` 遍历剩余元素：
 - 交换 `nums[start]` 和 `nums[i]`，将 `nums[i]` 放到当前位置。
 - 递归调用 `backtrack(start+1)`，确定下一个位置。
 - 交换回来（回溯），恢复数组原状，以便尝试下一个元素。

伪代码

算法 `permute(nums)`:

结果集 `res = []`

`n = nums.length`

函数 `backtrack(start)`:

if `start == n`:

`res.add(nums 的副本)` // 注意要复制当前数组

return

for `i = start` to `n-1`:

`swap(nums[start], nums[i])` // 选择 `nums[i]` 放到当前位置


```

        backtrack(start+1)
        swap(nums[start], nums[i])    // 回溯，恢复原状
    end function

    backtrack(0)
    return res

```

C++ 实现（交换法）

```

void backtrack(vector<int>& nums, int start, vector<vector<int>>& res) {
    if (start == nums.size()) {
        res.push_back(nums);
        return;
    }
    for (int i = start; i < nums.size(); ++i) {
        swap(nums[start], nums[i]);
        backtrack(nums, start + 1, res);
        swap(nums[start], nums[i]); // 回溯
    }
}

vector<vector<int>> permute(vector<int>& nums) {
    vector<vector<int>> res;
    backtrack(nums, 0, res);
    return res;
}

```

Python 实现（交换法）

```

def permute(nums):
    res = []
    n = len(nums)
    def backtrack(start):
        if start == n:
            res.append(nums[:])
            return
        for i in range(start, n):
            nums[start], nums[i] = nums[i], nums[start]
            backtrack(start + 1)
            nums[start], nums[i] = nums[i], nums[start]
    backtrack(0)
    return res

```

执行过程示例（交换法）

以 `nums = [1, 2, 3]` 为例，交换法的递归调用过程可以用递归树清晰地表示：

```
backtrack(0)
```

```
├ i=0: swap(0,0) → [1,2,3] → backtrack(1)
│   ├── i=1: swap(1,1) → [1,2,3] → backtrack(2)
│   │   └ i=2: swap(2,2) → [1,2,3] → backtrack(3) → 记录 [1,2,3]
│   └ i=2: swap(1,2) → [1,3,2] → backtrack(2)
│       └ i=2: swap(2,2) → [1,3,2] → backtrack(3) → 记录 [1,3,2]
├ i=1: swap(0,1) → [2,1,3] → backtrack(1)
│   ├── i=1: swap(1,1) → [2,1,3] → backtrack(2) → 记录 [2,1,3]
│   └ i=2: swap(1,2) → [2,3,1] → backtrack(2) → 记录 [2,3,1]
└ i=2: swap(0,2) → [3,2,1] → backtrack(1)
    ├── i=1: swap(1,1) → [3,2,1] → backtrack(2) → 记录 [3,2,1]
    └ i=2: swap(1,2) → [3,1,2] → backtrack(2) → 记录 [3,1,2]
```

在这个递归树中，每个分支对应一次递归调用。从根节点 `backtrack(0)` 开始，每次选择不同的 `i` 进行交换，然后进入下一层递归。当 `start == n` 时，记录当前排列。通过交换和回溯，算法高效地遍历了所有排列。

注意：交换法直接修改原数组，因此必须在递归前后恢复状态，这是回溯法的典型操作。基础版本由于每次传入新列表的副本，无需显式恢复，但代价是额外的内存开销。两种方法各有优劣，实际应用中可根据问题特点选择。

6.4.5.5 两种方法的对比与复杂度分析

方法	时间开销	额外空间	优点	缺点
基础版	$O(n \times n!)$ (复制列表常数大)	$O(n^2)$ (递归栈 + 列表副本)	代码直观，易于理解	空间效率低， n 大时内存大
交换法	$O(n \times n!)$ (交换常数小)	$O(n)$ (递归栈)	空间高效，常数小	稍抽象，需要理解“交换回溯”

- **时间复杂度：**两种方法都需要遍历整棵排列树，叶子节点 $n!$ 个，内部节点总数约 $e \cdot n!$ ，每个节点操作（复制或交换）平均 $O(1) \sim O(n)$ ，总时间复杂度均为 $O(n \times n!)$ 。但交换法的常数因子远小于基础版。
- **空间复杂度：**基础版每层递归都创建新的剩余列表，这些列表在递归栈中同时存在，总大小为 $n + (n-1) + \dots + 1 = O(n^2)$ 。交换法仅需递归栈 $O(n)$ ，无额外列表。

6.4.5.6 小结

全排列问题完美展示了回溯法的核心思想：**按顺序做决策** → **递归深入** → **到达叶子记录解** → **回溯撤销选择**。我们介绍了两种实现：

- **基础版（复制剩余列表）：**代码直观，适合初学者理解搜索树结构，但空间复杂度 $O(n^2)$ 。
- **交换法（原地交换）：**空间复杂度 $O(n)$ ，常数小，是工程实践中常用的写法。

两种方法的时间复杂度相同，但空间效率差异显著。掌握了全排列，下一节我们将学习 N 皇后问题，它在回溯的基础上加入了**约束剪枝**，能大幅减少搜索空间。

6.4.6 N 皇后问题（约束剪枝）

6.4.6.1 问题描述

N 皇后问题要求在一个 $(n \times n)$ 的棋盘上放置 (n) 个皇后，使得任意两个皇后都不能互相攻击。也就是说，必须满足：

- 不在同一行
- 不在同一列
- 不在同一条对角线

以 4 皇后为例，一个可行解如下：

```
.Q..  
...Q  
Q...  
..Q.
```

这类问题的特点是：**解必须满足一组全局约束**，而不仅仅是排列或组合。

6.4.6.2 多步决策：逐行放置皇后

我们把问题转化为一个逐步决策过程：

- 第 0 行：选择一个列放皇后
- 第 1 行：选择一个列放皇后
-
- 第 $(n-1)$ 行：完成最后一个皇后

每一步的决策是：**在当前行中，选择一个合法的列**。与全排列不同的是，这里不是“任意选”，而是“在约束下选”。这正是回溯法的典型应用场景：**搜索 + 剪枝**。

6.4.6.3 状态表示（核心抽象）

我们用一个一维数组来表示棋盘状态：

`board[i]` = 第 i 行皇后所在的列号（下标从 0 开始）

例如，`board = [1, 3, 0, 2]` 表示：

- 第 0 行在第 1 列
- 第 1 行在第 3 列
- 第 2 行在第 0 列
- 第 3 行在第 2 列

这种表示方式的关键优势是：**天然避免了行冲突**（每行只放一个皇后）。

6.4.6.4 冲突判断（剪枝的核心）

当我们尝试在 (row, col) 放置皇后时，只需要检查之前的所有行 ($r = 0 \dots \text{row}-1$):

- 列冲突: $\text{board}[r] = \text{col}$
- 对角线冲突: $|\text{board}[r] - \text{col}| = \text{row} - r$

对角线冲突的判断依据是：如果两个皇后的行差等于列差，则它们在同一个 45° 或 135° 对角线上。公式中的绝对值保证了两个方向的对角线都会被检测到。

6.4.6.5 回溯算法框架

每一行的处理逻辑非常清晰：

1. 依次尝试每一列
2. 判断是否与已放置的皇后冲突
3. 如果不冲突，则放置皇后（记录列号）
4. 递归处理下一行
5. 递归返回后，撤销当前选择（回溯）

当 $\text{row} = n$ 时，说明所有行都已成功放置，得到一个合法解，将其转换成棋盘字符串形式并保存。

6.4.6.6 伪代码

算法 `solveNQueens(n)`:

输入：整数 n

输出：所有可行解的列表 `res`

`board` = 长度为 n 的数组，初始化为 `-1`

`res` = []

函数 `isSafe(row, col)`:

for $r = 0$ to $\text{row}-1$:

if $\text{board}[r] = \text{col}$ or $|\text{board}[r] - \text{col}| = \text{row} - r$:

return false

return true

函数 `backtrack(row)`:

if $\text{row} = n$:

// 将 `board` 转换为棋盘字符串表示

`solution` = 构造 n 个字符串，每行 `.'` 中 `board[i]` 位置改为 `'Q'`

`res.append(solution)`

return

for $\text{col} = 0$ to $n-1$:

if `isSafe(row, col)`:

```

        board[row] = col
        backtrack(row + 1)
        board[row] = -1    // 回溯，撤销选择

    backtrack(0)
    return res

```

C++ 实现

```

#include <iostream>
#include <vector>
#include <string>
#include <cmath>
using namespace std;

// 辅助函数：检查在 (row, col) 放置皇后是否安全
bool isSafe(const vector<int>& board, int row, int col) {
    for (int r = 0; r < row; ++r) {
        if (board[r] == col || abs(board[r] - col) == row - r)
            return false;
    }
    return true;
}

// 回溯函数
void backtrack(vector<int>& board, int row, vector<vector<string>>& res) {
    int n = board.size();
    if (row == n) {
        // 将 board 转换为字符串棋盘
        vector<string> solution;
        for (int i = 0; i < n; ++i) {
            string line(n, '.');
            line[board[i]] = 'Q';
            solution.push_back(line);
        }
        res.push_back(solution);
        return;
    }
    for (int col = 0; col < n; ++col) {
        if (isSafe(board, row, col)) {
            board[row] = col;
            backtrack(board, row + 1, res);
            board[row] = -1; // 回溯
        }
    }
}

```

```

    }
}

// 主求解函数
vector<vector<string>> solveNQueens(int n) {
    vector<vector<string>> res;
    vector<int> board(n, -1);
    backtrack(board, 0, res);
    return res;
}

// 测试代码
int main() {
    int n = 4; // 测试 4 皇后
    vector<vector<string>> solutions = solveNQueens(n);

    cout << " 找到 " << solutions.size() << " 个解: " << endl;
    for (const auto& sol : solutions) {
        for (const auto& line : sol) {
            cout << line << endl;
        }
        cout << endl;
    }
    return 0;
}

```

Python 实现

```

def solveNQueens(n):
    res = []
    board = [-1] * n

    def is_safe(row, col):
        for r in range(row):
            if board[r] == col or abs(board[r] - col) == row - r:
                return False
        return True

    def backtrack(row):
        if row == n:
            # 构造棋盘输出
            solution = []
            for r in range(n):

```

```

        line = ['.'] * n
        line[board[r]] = 'Q'
        solution.append(''.join(line))
        res.append(solution)
        return
    for col in range(n):
        if is_safe(row, col):
            board[row] = col
            backtrack(row + 1)
            board[row] = -1

    backtrack(0)
    return res

# 测试代码
if __name__ == '__main__':
    n = 4 # 测试 4 皇后
    solutions = solveNQueens(n)
    print(f" 找到 {len(solutions)} 个解: ")
    for sol in solutions:
        for line in sol:
            print(line)
        print()

```

执行结果示例

运行上述代码 (n=4) 将输出:

找到 2 个解:

```

.Q..
...Q
Q...
..Q.

```

```

..Q.
Q...
...Q
.Q..

```

6.4.6.7 剪枝效果: 从 256 到 20

如果没有剪枝, 仅按照“每行任选一列”暴力枚举, 总共有 ($4^4 = 256$) 种放置方案。而回溯法通过**约束剪枝**——在每步放置前检查冲突, 一旦发现冲突就立即放弃该分支——实际探索的节点数只有约 20 个 (参见前文 4 皇后的搜索树图)。随着 (n) 增大, 剪枝效果更加显著: 例如 8 皇后, 暴力枚举 ($8^8 \approx 1677$) 万种, 回溯法只需探索约 1.5 万个节点。

6.4.6.8 为什么这个问题是“回溯进阶的关键点”？

相比全排列，N 皇后问题在三个方面实现了升级：

1. 从“生成排列”到“搜索合法解”

全排列中所有叶子节点都是解；而 N 皇后中只有极少数解，必须依赖剪枝。

2. 从“无约束选择”到“约束检查”

每一步都要问：“这个选择会不会导致未来无解？”只有通过 isSafe 检查的分支才会继续。

3. 搜索空间爆炸但被剪枝压缩

理论状态空间是 (n^n) ，但实际搜索的节点数远远小于这个值——这正是回溯法的威力所在。

6.4.6.9 小结

N 皇后问题完美展示了回溯法的核心模式：**选择** → **约束检查** → **递归** → **回溯**。它与全排列的区别在于，每一步决策前都会调用冲突检测函数，不合法的选择直接跳过（剪枝）。这种“边试探边剪枝”的思想，是回溯法能够高效求解组合优化问题的根本原因。掌握了 N 皇后，你就掌握了约束满足问题的典型解法。下一节，我们将学习回溯法在 0-1 背包问题中的应用，并由此引出更强大的**分支限界法**。

6.4.7 0-1 背包问题：在指数空间中寻找最优解

6.4.7.1 问题描述

给定 (n) 个物品，每个物品有重量 (w_i) 和价值 (v_i) ，以及一个容量为 (C) 的背包。我们需要选择若干物品装入背包，使得总重量不超过 (C) ，且总价值最大。

这是一个典型的**组合优化问题**。与全排列（所有排列都是解）或 N 皇后（解必须满足全部约束）不同，这里我们关心的是：**在所有可行子集中，哪一个价值最大。**

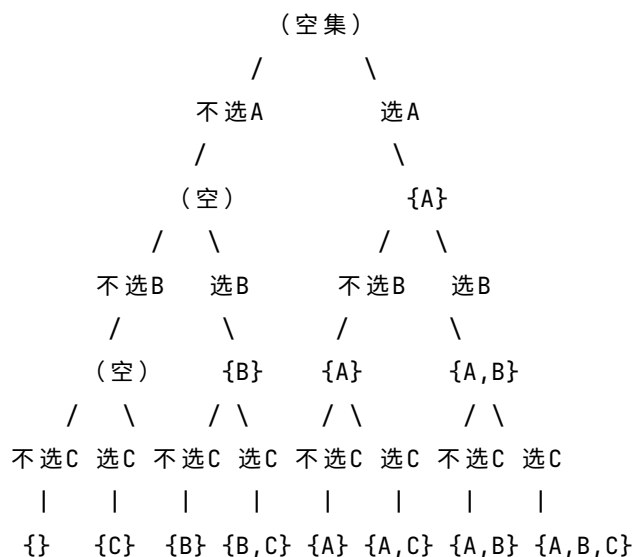
6.4.7.2 多步决策：子集树模型

对于每一个物品，我们只有两个选择：**选 (1) 或不选 (0)**。因此，整个问题的决策过程可以看作一棵**二叉决策树**，称为**子集树**：

- 第 0 层：空集（未考虑任何物品）
- 第 1 层：对物品 0 做选择 → 选 / 不选
- 第 2 层：对物品 1 做选择 → 选 / 不选
-
- 第 (n) 层：得到一个完整子集（叶子节点）

这棵树的叶子节点共有 (2^n) 个，对应所有可能的物品组合。我们的任务就是在这棵树上**找出满足重量约束且价值最大的叶子节点**。

下面以 3 个物品 $\{A, B, C\}$ 为例，画出完整的子集树：



6.4.7.3 回溯算法思路（基础版）

我们用深度优先搜索遍历这棵子集树。在遍历过程中，维护以下信息：

- i ：当前考虑的物品索引（从 0 开始）
- cur_w ：当前已选物品的总重量
- cur_v ：当前已选物品的总价值
- $choice$ ：当前已选物品的决策序列（0/1 列表）

当 $i = n$ 时，说明已经对所有物品做出了决策，得到了一个完整的子集。此时如果 cur_v 大于当前记录的最优值 $best_val$ ，就更新最优解。

在递归过程中，我们通过**重量约束**进行剪枝：如果选择当前物品后总重量超过容量 (C)，则直接跳过该分支（不递归）。这就是**可行性剪枝**。

注意：这里只用到了重量剪枝，没有使用价值上界剪枝（那是下一节分支限界法的内容）。因此该算法在最坏情况下仍然会遍历所有 (2^n) 个子集，时间复杂度 ($O(2^n)$)，仅适用于 (n) 较小的情况（通常 ($n \leq 20$))。

伪代码

算法 `knapsack(items, C)`:

输入：物品列表 `items[0..n-1]`，每个物品有重量 w 和价值 v ；背包容量 C

输出：最优总价值 `best_val` 和对应的选择序列 `best_choice`

```
n = items.length
```

```
best_val = 0
```

```
best_choice = []
```

函数 `backtrack(i, cur_w, cur_v, choice)`:

```

    if i == n:
        if cur_v > best_val:
            best_val = cur_v
            best_choice = choice 的副本
        return

    // 分支1: 不选当前物品
    choice.append(0)
    backtrack(i+1, cur_w, cur_v, choice)
    choice.pop()

    // 分支2: 选当前物品 (如果不超过容量)
    if cur_w + items[i].w ≤ C:
        choice.append(1)
        backtrack(i+1, cur_w + items[i].w, cur_v + items[i].v, choice)
        choice.pop()
end function

backtrack(0, 0, 0, [])
return best_val, best_choice

```

C++ 实现

```

#include <iostream>
#include <vector>
using namespace std;

struct Item {
    int w, v;
};

int best_val;
vector<int> best_choice;

void backtrack(const vector<Item>& items, int C, int i, int cur_w, int cur_v, vector<int>& choice)
{
    if (i == items.size()) {
        if (cur_v > best_val) {
            best_val = cur_v;
            best_choice = choice;
        }
        return;
    }

    // 不选当前物品
    choice.push_back(0);

```

```

    backtrack(items, C, i+1, cur_w, cur_v, choice);
    choice.pop_back();
    // 选当前物品（如果可行）
    if (cur_w + items[i].w <= C) {
        choice.push_back(1);
        backtrack(items, C, i+1, cur_w + items[i].w, cur_v + items[i].v, choice);
        choice.pop_back();
    }
}

pair<int, vector<int>> knapsack(const vector<Item>& items, int C) {
    best_val = 0;
    best_choice.clear();
    vector<int> choice;
    backtrack(items, C, 0, 0, 0, choice);
    return {best_val, best_choice};
}

int main() {
    vector<Item> items = {{2, 3}, {3, 4}, {4, 5}, {5, 6}};
    int C = 8;
    auto [val, choice] = knapsack(items, C);
    cout << " 最大价值: " << val << endl;
    cout << " 选择方案: ";
    for (int x : choice) cout << x << " ";
    cout << endl;
    return 0;
}

```

Python 实现

```

def knapsack(items, C):
    n = len(items)
    best_val = 0
    best_choice = []
    choice = []

    def backtrack(i, cur_w, cur_v):
        nonlocal best_val, best_choice
        if i == n:
            if cur_v > best_val:
                best_val = cur_v
                best_choice = choice[:]

```

```

        return

    # 不选当前物品
    choice.append(0)
    backtrack(i+1, cur_w, cur_v)
    choice.pop()

    # 选当前物品
    if cur_w + items[i][0] <= C:
        choice.append(1)
        backtrack(i+1, cur_w + items[i][0], cur_v + items[i][1])
        choice.pop()

    backtrack(0, 0, 0)
    return best_val, best_choice

# 测试
items = [(2,3), (3,4), (4,5), (5,6)]
C = 8
val, choice = knapsack(items, C)
print(" 最大价值:", val)
print(" 选择方案:", choice)

```

执行结果示例

对于测试数据（物品重量和价值分别为 (2,3)、(3,4)、(4,5)、(5,6)，容量为 8），程序输出：

最大价值：9

选择方案：[0, 1, 1, 0]

即选择物品 1 和 2（重量 3+4=7，价值 4+5=9），不选物品 0 和 3。

6.4.7.4 回溯法在这里的局限：为什么需要更强的剪枝？

当前算法只使用了**重量约束剪枝**（可行性剪枝），它只能排除那些**已经超重**的分支。但即便不超重，一个分支也可能**永远无法超过当前最优解**。例如，假设我们当前已经找到了一个价值为 9 的解，而某个部分解即使把后面所有物品都选上，其价值上限也小于 9，那么这个分支完全没有必要继续探索。

这种“估计剩余物品最大可能价值”的剪枝被称为**限界剪枝**，它需要结合一个**上界函数**。例如，可以按单位价值降序对剩余物品进行分数背包估计，得到从当前状态出发能获得的最大价值上界。如果这个上界 $\leq$ 当前最优值，就剪枝。

限界剪枝是分支限界法的核心，我们将在下一节详细讲解。将限界剪枝与回溯结合，就得到了求解最优化问题的强大工具——**分支限界法**。

6.4.7.5 本节小结

0-1 背包问题展示了回溯法的另一种应用场景：**在子集树上搜索最优解**。与全排列（枚举所有排列）和 N 皇后（约束满足）不同，背包问题需要在指数级的候选解中找出价值最大的可行解。

- **决策模型**：每个物品“选或不选”，形成子集树，叶子节点 (2^n) 个。
- **剪枝方式**：目前仅使用重量可行性剪枝（超重则停止）。
- **时间复杂度**：最坏情况仍为 ($O(2^n)$)，仅适合小规模 ($n \leq 20$)。
- **下一步**：引入价值上界限剪枝，将回溯升级为分支限界法，从而高效求解更大规模的问题。

通过本节，你应当理解：回溯法不仅能求解“是否存在解”的问题，也能求解“最优解”问题，但需要配合恰当的剪枝策略。下一节，我们将学习专门用于最优化问题的**分支限界法**。

6.5 分支限界法

分支限界法 (Branch and Bound) 是一种用于求解**组合优化问题**的系统性搜索方法。它在解空间树（或状态空间图）上进行搜索，并通过**限界函数**判断某个分支是否仍然“有希望”产生更优解，从而提前剪去无效的搜索路径。

可以这样理解：

分支限界法 = 状态空间搜索 + 最优性估计（限界） + 剪枝策略

与回溯法相比，两者的核心区别不在于“是否剪枝”，而在于**剪枝的依据**：

- **回溯法**主要使用**约束剪枝**（可行性）：如果当前部分解已经违反约束（如背包超重），就停止扩展。
- **分支限界法**使用**最优性剪枝**：如果当前部分解即使再乐观地估计，也不可能超过已知的最优解，就停止扩展。

因此，分支限界法专门用于求解**最优化问题**（求最大值或最小值），而不仅仅是“是否存在解”。

6.5.1 从一个例子出发：0-1 背包问题

让我们从一个具体例子开始，理解分支限界法的核心思想。

例：有 4 个物品，重量和价值分别为：

- 物品 0：重量 4，价值 40
- 物品 1：重量 7，价值 42
- 物品 2：重量 5，价值 25
- 物品 3：重量 3，价值 12

背包容量 ($C = 10$)。求能装入背包的最大价值。

这是一个典型的组合优化问题。解空间是一棵**子集树**（每个物品选或不选），共有 ($2^4 = 16$) 个叶子节点。如果采用普通的回溯法（深度优先搜索 + 重量剪枝），虽然可以避免超重分支，但仍然可能遍历许多节点。

分支限界法换一种思路：在搜索过程中，**不仅检查当前部分解是否可行，还估计从这个部分解继续扩展下去可能达到的最大价值**。如果这个估计值（**上界**）比当前已经找到的**最优解**还要小，那么该分支就没有继续探索的必要了。

例如，假设当前已选择了物品 0（重量 4，价值 40），剩余容量为 6。剩下的物品中，物品 1 的单位价值最高（ $\frac{42}{7} = 6$ ），物品 2（ $\frac{25}{5} = 5$ ），物品 3（ $\frac{12}{3} = 4$ ）。如果允许**分数装入**（即可以只装一部分物品），那么剩余容量 6 最多可以再获得（ $6 \times 6 = 36$ ）的价值（即装入 $\frac{6}{7}$ 个物品 1），加上当前的 40，总价值最多可达 76。这个 76 就是**从当前部分解出发可能达到的最大价值的乐观估计**，称为**上界**。

如果当前已经找到的最优解是 70，那么 $76 > 70$ ，这个分支还有希望，需要继续探索；如果当前最优解已经是 80，那么 $76 < 80$ ，这个分支就可以直接剪掉。

关键：对于最大值问题，上界必须是**乐观估计**——它只能大于或等于实际能达到的最大值，否则可能误剪掉正确答案。

6.5.2 基本概念

6.5.2.1 上界与下界

- **上界 (Upper Bound)：**对于最大化问题，从当前部分解出发，可能达到的目标函数值的**最大值的一个乐观估计**。实际能达到的值一定不会超过这个上界。
- **下界 (Lower Bound)：**对于最小化问题，从当前部分解出发，可能达到的目标函数值的**最小值的一个乐观估计**。实际能达到的值一定不会低于这个下界。

6.5.2.2 限界函数

计算上界或下界的函数称为**限界函数**。它的设计是关键：既要容易计算，又要尽可能紧（接近真实值），以便更有效地剪枝。对于 0-1 背包问题，常用的限界函数是：在当前已选物品的基础上，假设剩余物品可以按**单位价值降序分数装入**，从而得到上界。

6.5.2.3 为什么要乐观估计？

限界函数必须满足**乐观性**（也叫**可采纳性**，Admissible）。对于**最大化问题**，上界不能低估真实最大值。只有这样，当“上界 $\leq$ 当前最优解”时，我们才能确信该分支不可能产生更优解，从而安全剪枝。如果上界过于悲观（低估），就可能剪掉包含最优解的分支，导致算法错误。对于**最小值问题**也是类似的：下界不能高估真实最小值，当下界 $\geq$ 当前最优解时即可剪枝。

6.5.3 两种分支限界方式

根据搜索过程中选择下一个扩展节点的方式，分支限界法主要分为两类：

6.5.3.1 FIFO 分支限界（队列式）

使用普通队列，按**先进先出**的顺序扩展节点。这种方式类似于广度优先搜索（BFS），但增加了限界剪枝。优点是实现简单，缺点是可能扩展大量低质量的节点后才找到较好的上界。

算法框架（以最大化问题为例）：

```

算法 FIFO_BranchBound(initial_state):
    创建队列 queue，将 initial_state 入队
    初始化当前最优值 best =  $-\infty$ 
  
```

```

while queue 非空:
    node = queue.dequeue()
    if node 是可行解:
        if node.value > best:
            best = node.value
            记录最优解
    else:
        for each child in generate_children(node):
            计算 child 的上界 bound
            if bound > best:    // 有希望
                queue.enqueue(child)
return best 及对应的解

```

6.5.3.2 LC 分支限界（优先队列式）

LC 是 **Least Cost** 或 **Largest Bound** 的缩写，意思是每次扩展**最有希望**的节点。它使用**优先队列**（堆），按节点的上界（或下界）排序：对于最大化问题，上界大的节点优先扩展；对于最小化问题，下界小的节点优先扩展。这样可以更快地得到高质量的解，从而提高剪枝效率。

算法框架（最大化问题）：

```

算法 LC_BranchBound(initial_state):
    创建优先队列 pq，按上界降序排列
    将 initial_state 加入 pq
    初始化当前最优值 best =  $-\infty$ 
    while pq 非空:
        node = pq.pop()
        if node.bound ≤ best:    // 无法优于当前最优，剪枝
            continue
        if node 是可行解:
            if node.value > best:
                best = node.value
                记录最优解
        else:
            for each child in generate_children(node):
                计算 child 的上界 bound
                if bound > best:
                    pq.push(child)
    return best 及对应的解

```

6.5.4 0-1 背包问题的 LC 分支限界求解

回到开头的例子，我们详细演示 LC 分支限界的执行过程。

6.5.4.1 准备工作

首先，将物品按单位价值（价值/重量）从大到小排序，以便于计算上界。排序后：

- 物品 0：重量 4，价值 40，单位价值 10
- 物品 1：重量 7，价值 42，单位价值 6
- 物品 2：重量 5，价值 25，单位价值 5
- 物品 3：重量 3，价值 12，单位价值 4

物品列表：items = [(4,40), (7,42), (5,25), (3,12)], 索引 0~3。

6.5.4.2 上界函数

对于当前节点（下一个待决策物品索引为 k，当前重量 w，当前价值 v），上界计算如下：

- 如果 ($w > C$)，则此节点不可行，上界设为 $(-\infty)$ 。
- 否则，剩余容量 $\text{remain} = C - w$ 。从物品 k 开始，依次尝试整件装入，直到装不下最后一个物品，再用分数装入剩余容量。这样得到的总价值（当前价值 + 后续物品的乐观价值）就是上界。

具体函数：

```
bound(k, w, v):
    if w > C: return -∞
    remain = C - w
    total = v
    i = k
    while i < n and items[i].weight ≤ remain:
        total += items[i].value
        remain -= items[i].weight
        i++
    if i < n:
        total += (remain / items[i].weight) * items[i].value
    return total
```

6.5.4.3 搜索过程

我们用优先队列（最大堆）存储待扩展的节点，每个节点包含 (k, w, v, choice, bound)：

- **k**：下一个待决策的物品索引（取值范围 0~n）。当 $k = n$ 时，表示所有物品都已决策完毕。
- **w**：当前已选物品的总重量。
- **v**：当前已选物品的总价值。
- **choice**：已做的决策序列（长度为 k，0 表示不选，1 表示选）。此信息在记录最优解时需要，但在搜索过程中可以省略。
- **bound**：从当前状态出发的上界。

初始节点为根节点：k=0, w=0, v=0, choice=[]。计算根节点的上界：

- 从物品 0 开始贪心：剩余容量 10，整装物品 0（重 4），价值 40，剩余 6；物品 1 重量 7>6，只能分数装入 $(6/7) \times 42 \approx 36$ ，总上界 = $40 + 36 = 76$ 。

根节点的 $\text{bound}=76$ ，将其加入优先队列，当前最优值 $\text{best} = -\infty$ 。

第一步：从优先队列中取出 bound 最大的节点（根节点， $\text{bound}=76$ ）。因为 $\text{bound} > \text{best}$ ，扩展它，生成两个子节点：

- **不选物品 0：**生成新节点， $k=1$ （下一个决策物品 1）， $w=0$ ， $v=0$ ， $\text{choice}=[0]$ 。计算 bound ：
 - 从物品 1 开始贪心：剩余 10，整装物品 1（重 7），价值 42，剩余 3；物品 2 重量 $5 > 3$ ，分数装入 $(3/5)*25=15$ ，总上界 $= 42+15=57$ 。
 - 新节点 $\text{bound}=57$ 。
- **选物品 0：**生成新节点， $k=1$ ， $w=4$ ， $v=40$ ， $\text{choice}=[1]$ 。计算 bound ：
 - 从物品 1 开始贪心：剩余 6，物品 1 重量 $7 > 6$ ，分数装入 $(6/7)*42 \approx 36$ ，总上界 $= 40+36=76$ 。
 - 新节点 $\text{bound}=76$ 。

这两个子节点的 bound 分别为 57 和 76，均大于当前 $\text{best} (-\infty)$ ，因此都加入优先队列。此时队列中有节点 A: ($k=1, w=0, v=0, \text{choice}=[0]$, $\text{bound}=57$) 和节点 B: ($k=1, w=4, v=40, \text{choice}=[1]$, $\text{bound}=76$)。按 bound 降序，B 在前。

第二步：取出节点 B ($\text{bound}=76$)，此时 $k=1$ ，下一个要决策的是物品 1。- **不选物品 1：**生成节点， $k=2$ ， $w=4$ ， $v=40$ ， $\text{choice}=[1,0]$ 。计算 bound ：- 从物品 2 开始贪心：剩余 6，整装物品 2（重 5），价值 25，剩余 1；物品 3 重量 $3 > 1$ ，分数装入 $(1/3)*12=4$ ，总上界 $= 40+25+4=69$ 。新节点 $\text{bound}=69$ 。- **选物品 1：**检查重量 $4+7=11 > 10$ ，不可行，因此无此分支。

将新节点 ($\text{bound}=69$) 加入优先队列。此时队列中有：节点 A (57)、新节点 C (69)。优先取 C。

第三步：取出节点 C ($\text{bound}=69$)，此时 $k=2$ ，下一个要决策的是物品 2。- **不选物品 2：**生成节点， $k=3$ ， $w=4$ ， $v=40$ ， $\text{choice}=[1,0,0]$ 。计算 bound ：- 从物品 3 开始贪心：剩余 6，整装物品 3（重 3），价值 12，剩余 3（无后续物品），总上界 $= 40+12=52$ 。新节点 $\text{bound}=52$ 。- **选物品 2：**检查重量 $4+5=9 \leq 10$ ，生成节点， $k=3$ ， $w=9$ ， $v=65$ ， $\text{choice}=[1,0,1]$ 。计算 bound ：- 从物品 3 开始贪心：剩余 1，物品 3 重量 $3 > 1$ ，分数装入 $(1/3)*12=4$ ，总上界 $= 65+4=69$ 。新节点 $\text{bound}=69$ 。

两个新节点 bound 分别为 52 和 69，均大于 best ，加入队列。此时队列中有：A(57), D(52), E(69)。优先取 E。

第四步：取出节点 E ($\text{bound}=69$)，此时 $k=3$ ，下一个要决策的是物品 3。- **不选物品 3：**生成叶子节点， $k=4$ （表示所有物品都已决策）， $w=9$ ， $v=65$ ， $\text{choice}=[1,0,1,0]$ 。这是一个可行解，其价值 65。更新 $\text{best} = 65$ 。- **选物品 3：**检查重量 $9+3=12 > 10$ ，不可行，跳过。

第五步：此时队列中剩余节点 A (57) 和 D (52)。它们的 bound 均小于等于当前 best (65)，因此当它们出队时， $\text{bound} \leq \text{best}$ 条件触发，直接剪枝。队列空，搜索结束。

最终最优解为选择物品 0 和物品 2，总价值 65，重量 9，选择方案 $[1,0,1,0]$ 。

通过这个例子可以看到，LC 分支限界通过优先扩展上界高的节点，很快找到了一个较好的可行解 (65)，并用它剪掉了上界低于 65 的分支，大大减少了搜索节点数。整个过程中实际扩展的节点只有根节点、B、C、E 等少数几个，远少于子集树的 16 个叶子。

6.5.4.4 0-1 背包问题的 LC 分支限界实现

下面给出 LC 分支限界法求解 0-1 背包问题的伪代码，以及 C++ 和 Python 的具体实现。

伪代码

算法 Knapsack_BB(items, C)

输入：物品列表 `items[0..n-1]`，每个物品包含重量 `w` 和价值 `v`；背包容量 `C`

输出：最优总价值 `bestVal` 和对应的选择序列 `bestChoice`

// 按单位价值降序排序

`sort items by v[i]/w[i] descending`

`n = items.length`

`bestVal = 0`

`bestChoice = []`

// 定义节点结构：包含 `k`(下一个待决策物品索引), `w`(当前重量), `v`(当前价值), `choice`(决策序列), `bound`(

// 优先队列按 `bound` 降序排列 (最大堆)

创建优先队列 `pq`，按 `bound` 降序

// 计算根节点的上界

`rootBound = bound(0, 0, 0)`

`root = (k=0, w=0, v=0, choice=[], bound=rootBound)`

`pq.push(root)`

while `pq` 非空 do

`node = pq.pop()`

 if `node.bound ≤ bestVal` then

 continue // 剪枝

 if `node.k == n` then

 // 叶子节点，更新最优解

 if `node.v > bestVal` then

`bestVal = node.v`

`bestChoice = node.choice`

 continue

 // 处理不选当前物品的分支

`child1.k = node.k + 1`

`child1.w = node.w`

`child1.v = node.v`

`child1.choice = node.choice + [0]`

`child1.bound = bound(child1.k, child1.w, child1.v)`

 if `child1.bound > bestVal` then

`pq.push(child1)`

 // 处理选当前物品的分支 (如果可行)

 if `node.w + items[node.k].w ≤ C` then

`child2.k = node.k + 1`

`child2.w = node.w + items[node.k].w`

```

        child2.v = node.v + items[node.k].v
        child2.choice = node.choice + [1]
        child2.bound = bound(child2.k, child2.w, child2.v)
        if child2.bound > bestVal then
            pq.push(child2)

    return bestVal, bestChoice

// 上界函数
function bound(k, w, v):
    if w > C then return -∞
    remain = C - w
    total = v
    i = k
    while i < n and items[i].w ≤ remain do
        total = total + items[i].v
        remain = remain - items[i].w
        i = i + 1
    if i < n then
        total = total + (remain / items[i].w) * items[i].v
    return total

```

C++ 实现

```

#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;

struct Item {
    int w, v;
    double unit; // 单位价值
};

// 按单位价值降序排序的比较函数
bool cmp(Item a, Item b) {
    return a.unit > b.unit;
}

struct Node {
    int k;           // 下一个待决策的物品索引
    int w;           // 当前总重量
    int v;           // 当前总价值

```

```

vector<int> choice; // 已做的决策序列
double bound; // 上界

// 优先队列需要按 bound 降序, 因此重载 < 运算符为比较 bound (注意优先队列默认是最大堆, 但我们需要自定义比较)
bool operator<(const Node& other) const {
    return bound < other.bound; // 使得 bound 大的节点优先级高
}

};

// 上界函数
double bound(int k, int w, int v, const vector<Item>& items, int C) {
    if (w > C) return -1e9;
    int remain = C - w;
    double total = v;
    int i = k;
    int n = items.size();
    while (i < n && items[i].w <= remain) {
        total += items[i].v;
        remain -= items[i].w;
        i++;
    }
    if (i < n) {
        total += (double)remain / items[i].w * items[i].v;
    }
    return total;
}

pair<int, vector<int>> knapsackBB(vector<Item>& items, int C) {
    // 按单位价值降序排序
    for (auto& item : items) item.unit = (double)item.v / item.w;
    sort(items.begin(), items.end(), cmp);

    int n = items.size();
    int bestVal = 0;
    vector<int> bestChoice;

    priority_queue<Node> pq; // 最大堆

    // 根节点
    Node root;
    root.k = 0;
    root.w = 0;

```

```
root.v = 0;
root.choice = {};
root.bound = bound(0, 0, 0, items, C);
pq.push(root);

while (!pq.empty()) {
    Node node = pq.top();
    pq.pop();

    if (node.bound <= bestVal) continue; // 剪枝

    if (node.k == n) {
        // 叶子节点
        if (node.v > bestVal) {
            bestVal = node.v;
            bestChoice = node.choice;
        }
        continue;
    }

    // 不选当前物品
    Node child1;
    child1.k = node.k + 1;
    child1.w = node.w;
    child1.v = node.v;
    child1.choice = node.choice;
    child1.choice.push_back(0);
    child1.bound = bound(child1.k, child1.w, child1.v, items, C);
    if (child1.bound > bestVal) pq.push(child1);

    // 选当前物品
    if (node.w + items[node.k].w <= C) {
        Node child2;
        child2.k = node.k + 1;
        child2.w = node.w + items[node.k].w;
        child2.v = node.v + items[node.k].v;
        child2.choice = node.choice;
        child2.choice.push_back(1);
        child2.bound = bound(child2.k, child2.w, child2.v, items, C);
        if (child2.bound > bestVal) pq.push(child2);
    }
}
```

```

    return {bestVal, bestChoice};
}

int main() {
    vector<Item> items = {
        {4, 40, 0},
        {7, 42, 0},
        {5, 25, 0},
        {3, 12, 0}
    };
    int C = 10;
    auto [val, choice] = knapsackBB(items, C);
    cout << " 最大价值: " << val << endl;
    cout << " 选择方案: ";
    for (int x : choice) cout << x << " ";
    cout << endl;
    // 输出应为: 最大价值: 65, 选择方案: 1 0 1 0 (对应物品 0 选, 物品 1 不选, 物品 2 选, 物品 3 不选)
    return 0;
}

```

Python 实现

```

import heapq

class Node:
    def __init__(self, k, w, v, choice, bound):
        self.k = k          # 下一个待决策的物品索引
        self.w = w          # 当前总重量
        self.v = v          # 当前总价值
        self.choice = choice # 已做的决策序列
        self.bound = bound  # 上界

    # 为了在优先队列中按 bound 降序, Python 的 heapq 是最小堆, 因此我们自定义比较, 存储 (-bound)
    def __lt__(self, other):
        return self.bound > other.bound  # 使得 bound 大的节点优先级高

def knapsackBB(items, C):
    # 按单位价值降序排序
    items.sort(key=lambda x: x[1]/x[0], reverse=True)
    n = len(items)
    best_val = 0
    best_choice = []

```

```

# 上界函数
def bound(k, w, v):
    if w > C:
        return -float('inf')
    remain = C - w
    total = v
    i = k
    while i < n and items[i][0] <= remain:
        total += items[i][1]
        remain -= items[i][0]
        i += 1
    if i < n:
        total += (remain / items[i][0]) * items[i][1]
    return total

# 根节点
root_bound = bound(0, 0, 0)
root = Node(0, 0, 0, [], root_bound)
pq = [(-root_bound, root)] # 用负值模拟最大堆
heapq.heapify(pq)

while pq:
    neg_bound, node = heapq.heappop(pq)
    if node.bound <= best_val:
        continue
    if node.k == n:
        if node.v > best_val:
            best_val = node.v
            best_choice = node.choice
        continue

    # 不选当前物品
    child1 = Node(node.k+1, node.w, node.v, node.choice + [0], 0)
    child1.bound = bound(child1.k, child1.w, child1.v)
    if child1.bound > best_val:
        heapq.heappush(pq, (-child1.bound, child1))

    # 选当前物品
    if node.w + items[node.k][0] <= C:
        child2 = Node(node.k+1, node.w + items[node.k][0],
                      node.v + items[node.k][1], node.choice + [1], 0)
        child2.bound = bound(child2.k, child2.w, child2.v)

```

```

        if child2.bound > best_val:
            heapq.heappush(pq, (-child2.bound, child2))

    return best_val, best_choice

if __name__ == '__main__':
    items = [(4,40), (7,42), (5,25), (3,12)]
    C = 10
    val, choice = knapsackBB(items, C)
    print(" 最大价值:", val)
    print(" 选择方案:", choice)
    # 输出应为: 最大价值: 65, 选择方案: [1,0,1,0]

```

代码说明

- **排序**: 在算法开始前, 物品按单位价值降序排列, 这是计算上界的前提。
- **节点结构**: 每个节点记录了下一个待决策的物品索引 k , 当前总重量 w , 当前总价值 v , 已做决策序列 $choice$, 以及上界 $bound$ 。
- **优先队列**: 使用最大堆 (C++ 的 `priority_queue` 默认是最大堆, Python 用 `heapq` 模拟最大堆), 保证每次取出上界最大的节点扩展。
- **剪枝**: 当节点的上界小于等于当前已知最优值时, 该节点被丢弃。
- **叶子节点**: 当 $k = n$ 时, 所有物品决策完毕, 更新最优解。
- **分支生成**: 对每个节点, 考虑“不选”和“选”(如果可行) 两个子节点, 计算其上界后加入优先队列。

运行上述代码, 将得到与之前手工推导一致的结果: 最大价值为 65, 选择方案为 $[1, 0, 1, 0]$ (即选物品 0 和 2, 不选物品 1 和 3)。

6.5.5 分支限界法与回溯法的对比

特性	回溯法	分支限界法
搜索方式	深度优先	广度优先 (FIFO) 或最佳优先 (LC)
主要用途	约束满足问题、组合优化 (可求所有解)	最优化问题 (求最优解)
节点扩展	一次扩展一个子节点, 递归深入	一次扩展所有子节点, 加入待扩展表
剪枝依据	约束函数 (排除不可行)、可选限界函数	限界函数 (排除不可能更优)
数据结构	栈 (递归隐含栈)	队列或优先队列
空间复杂度	与树高成正比	与待扩展节点数成正比, 通常更大
最优性	不保证最优 (除非遍历所有可行解)	保证找到最优解 (若限界函数正确)

选择指南: - 如果问题只需要任意可行解, 或约束条件很强 (剪枝效果好), 优先考虑回溯法。- 如果问题要求最优解, 且能够设计有效的限界函数, 优先考虑分支限界法 (尤其是 LC 分支限界)。- 如果状态空间极大, 分支限界法可能因内存不足而失效, 此时可考虑启发式搜索或动态规划。

6.5.6 本节小结

本节从 0-1 背包问题出发，引出了分支限界法的核心思想：**通过上界估计来剪枝，优先扩展有希望的分支**。我们介绍了限界函数的概念及其乐观性要求，并给出了 FIFO 分支限界和 LC 分支限界两种实现方式。通过 LC 分支限界求解 0-1 背包的详细步骤，展示了该方法如何利用上界快速收敛到最优解。分支限界法是求解组合优化问题的强有力工具，与回溯法互补，适用于不同的问题场景。

6.5.7 本章小结

本章围绕“状态空间搜索”这一核心思想，系统介绍了如何将实际问题转化为状态图中的路径搜索，并学习了从基础到高级的搜索策略与剪枝技术。

状态空间建模是搜索的起点。我们将问题的每个配置抽象为**状态**，将状态之间的转换定义为**操作**，所有状态及其转换关系构成**状态空间图**。搜索过程中动态生成的部分称为**搜索树**，解则是从初始状态到目标状态的一条路径。

深度优先搜索 (DFS) 和 **广度优先搜索 (BFS)** 是最基本的两种搜索策略。DFS 使用栈，沿着一条路径深入到底再回溯，空间占用小但不保证最短路径；BFS 使用队列，逐层向外扩展，保证第一次遇到目标时路径最短，但空间可能较大。两者构成了后续高级算法的基础。

回溯法是 DFS 的增强版，专门解决多步决策问题。它在 DFS 的基础上显式维护决策路径，并利用**约束剪枝**（如皇后冲突、背包超重）提前终止不可能产生合法解的分支。全排列、N 皇后、0-1 背包等经典问题展示了回溯法的灵活应用。

分支限界法则面向最优化问题，采用广度优先或最佳优先搜索，并引入**限界函数**进行最优性剪枝。通过对当前部分解的未来潜力进行乐观估计（上界或下界），可以剪去那些即使再努力也无法超越当前最优解的分支。LC 分支限界（优先队列）因其快速收敛而最为常用。

从 visited 去重，到约束剪枝，再到限界剪枝，剪枝技术的层层递进正是搜索算法效率提升的关键。实际应用中，应根据问题需求选择策略：只需可行解且约束强时用回溯法；求最优解且有有效限界函数时用分支限界法；状态空间极大时可考虑启发式搜索或动态规划。

掌握本章内容，你将具备分析和设计搜索算法的基本能力，为解决更复杂的组合优化问题打下坚实基础。

6.6 习题

6.6.1 概念理解题

1. 什么是状态空间？请用你自己的话解释状态、操作、初始状态、目标状态的含义，并以八数码问题为例说明。
 2. 状态空间图与搜索树有什么区别？为什么在搜索过程中通常需要记录已访问状态？
 3. 什么是解路径？什么是最优解？在什么情况下 BFS 能保证找到最优解？
 4. 解释回溯法中的“剪枝”概念。约束函数和限界函数分别起什么作用？请举例说明。
 5. 分支限界法中的“限界函数”为什么必须满足“乐观性”？如果限界函数不满足乐观性会有什么后果？
-

6.6.2 BFS 与 DFS 算法应用题

6. 考虑一个简单的数字变换问题：从初始数字 1 出发，允许两种操作：乘以 2，或加 1。目标数字是 5。请画出从 1 到 5 的部分搜索树（只画出必要的节点）。然后分别按照深度优先搜索（DFS）和广度优先搜索（BFS）的顺序写出节点的访问序列（假设在 DFS 中按照操作顺序“ $\times 2$ 优先于 $+1$ ”进行扩展）。并说明 DFS 找到的路径是否一定是最短的。
7. 对于下图所示的迷宫，S 为起点，E 为终点，黑色格子为墙。请画出从 S 到 E 的完整状态空间图（只考虑可通行格子）。然后分别用 DFS 和 BFS 在图上搜索一条从 S 到 E 的路径，写出搜索过程中每个状态的访问顺序（假设在 DFS 中按“上、下、左、右”的优先级移动，在 BFS 中按同样优先级生成邻居）。并指出 BFS 找到的路径长度。

```

0 1 2 3
0 S . . #
1 # . # .
2 . . . .
3 # . . E

```

6.6.3 回溯法应用题

8. 使用回溯法求解 4 皇后问题。请画出完整的搜索树（包括被剪枝的分支），并标出剪枝点。最后写出所有解。
9. 给定一个集合 $\{2, 3, 5, 7\}$ ，目标和 $T = 10$ 。请用回溯法找出所有和为 10 的子集（每个元素最多用一次）。要求画出部分搜索树，并说明剪枝条件。
10. 0-1 背包问题：有 3 个物品，重量和价值分别为 $(2, 3), (3, 4), (4, 5)$ ，背包容量 $C = 6$ 。请用回溯法求解最优值，并画出搜索树（包括剪枝）。如果采用限界函数（如按单位价值贪心上界）进行剪枝，请说明哪些分支可以被提前剪掉。

6.6.4 分支限界法应用题

11. 考虑 0-1 背包问题：物品重量和价值分别为 $(5, 10), (4, 8), (3, 5), (2, 3)$ ，背包容量 $C = 10$ 。请按单位价值降序排序后，用 LC 分支限界法求解最优解。要求：
 - 计算每个扩展节点的上界。
 - 画出优先队列的变化过程（包括每次出队和入队的节点）。
 - 最终给出最优值和选择方案。
12. 对于旅行商问题（TSP），简述如何设计一个下界函数。如果当前已走过的路径为 $0 \rightarrow 2 \rightarrow 1$ ，剩余城市为 $\{3, 4\}$ ，距离矩阵如下，请计算当前节点的下界（可采用基于归约矩阵的方法，或简单的最小边权法）。

	0	1	2	3	4
0	∞	3	4	2	7
1	3	∞	4	6	3
2	4	4	∞	5	8
3	2	6	5	∞	6

0	1	2	3	4
4	7	3	8	6
				∞

6.6.5 综合设计题

13. 设计一个简单的“华容道”问题（例如 2×3 棋盘，有不同大小的方块），请定义其状态、操作、初始状态和目标状态。分析其状态空间大小，并讨论适合用哪种搜索策略（BFS、DFS、回溯法、分支限界）求解，说明理由。
14. 某公司需要从 n 个项目中挑选若干项目进行投资，每个项目有投资额 c_i 和预期收益 p_i ，总资金上限为 M 。要求选择一组项目使得总收益最大，且总资金不超过 M 。请问该问题属于哪种类型？你会选择哪种算法求解？请给出算法的主要步骤，并分析其时间复杂度和空间复杂度。

6.6.6 编程实践题

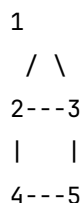
15. 编程实现回溯法求解 N 皇后问题 ($N \leq 10$)，要求输出所有解。并统计 $N=8$ 时解的个数以及搜索过程中生成的节点总数（包括被剪枝的节点）。可与纯 DFS（不剪枝）的节点数对比。
16. 编程实现 LC 分支限界法求解 0-1 背包问题，并随机生成 10 个物品（重量和价值均为 1~10 之间的整数），背包容量取总重量的一半左右。与动态规划法进行效率对比（运行时间或扩展节点数）。

注：带 * 的题目为选做题，可供学有余力的同学深入思考。

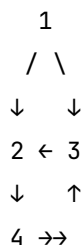
A 附录 A 图的基本概念

本附录介绍图论中的基本术语和概念，为理解第 3 章中的最小生成树、最短路径等算法提供必要的背景知识。为了便于说明，我们定义两个示例图贯穿全文：

- **无向图** G_u ：顶点集 $V = \{1, 2, 3, 4, 5\}$ ，边集 $E = \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 5), (4, 5)\}$ 。



- **有向图** G_d ：顶点集 $V = \{1, 2, 3, 4\}$ ，边集 $E = \{\langle 1, 2 \rangle, \langle 1, 3 \rangle, \langle 2, 4 \rangle, \langle 3, 2 \rangle, \langle 4, 3 \rangle\}$ 。其结构大致如下：

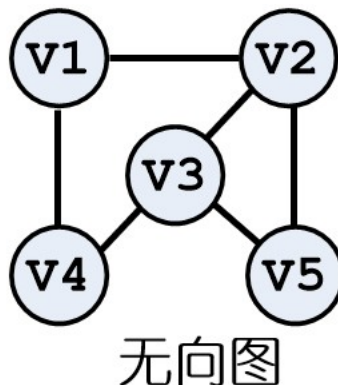
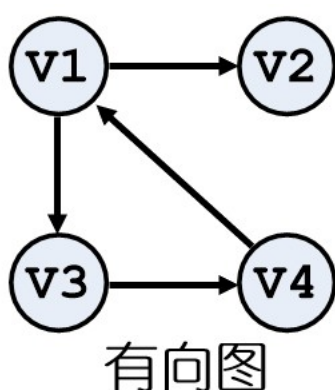


A.0.1 A.1 图的定义与分类

图 (Graph) 是由一个顶点集合和一个边集合组成的二元组，记为 $G = (V, E)$ ，其中：

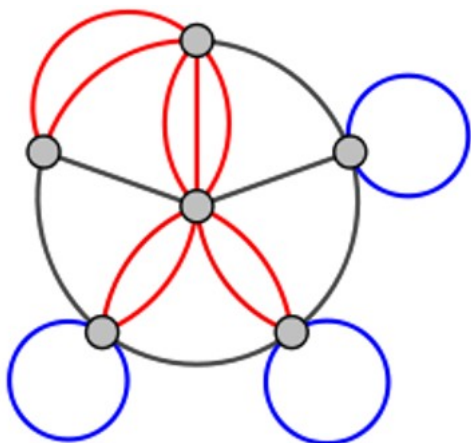
- V 是顶点的非空有限集合。
- E 是边的集合，每条边连接两个顶点。

根据边的方向性，图可分为两类：- **无向图 (Undirected Graph)**：边是无序的顶点对，记作 (u, v) 或 $\{u, v\}$ ，表示 u 和 v 之间有一条双向的连接。例如在 G_u 中，边 $(1, 2)$ 连接顶点 1 和 2。- **有向图 (Directed Graph)**：边是有序的顶点对，记作 $\langle u, v \rangle$ ，表示从 u 指向 v 的弧， u 称为弧尾， v 称为弧头。例如在 G_d 中，边 $\langle 1, 2 \rangle$ 表示从 1 到 2 的一条有向边。



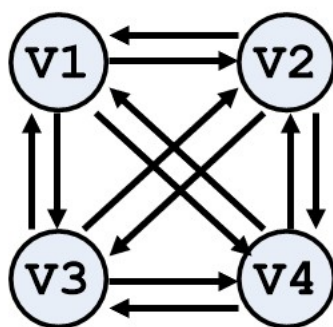
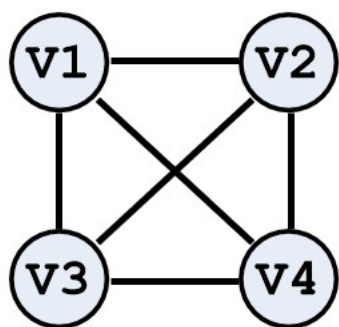
此外, 还有两种特殊情况:

- **多重图 (Multigraph)**: 允许两个顶点之间存在多条相同的边。
- **自环 (Self-loop)**: 一条边的两个端点是同一个顶点。
- **简单图 (Simple Graph)**: 既没有自环, 也没有多重边的图。本书中如无特别说明, 所讨论的图均为简单图。 G_u 和 G_d 都是简单图。



A.0.2 A.2 完全图与子图

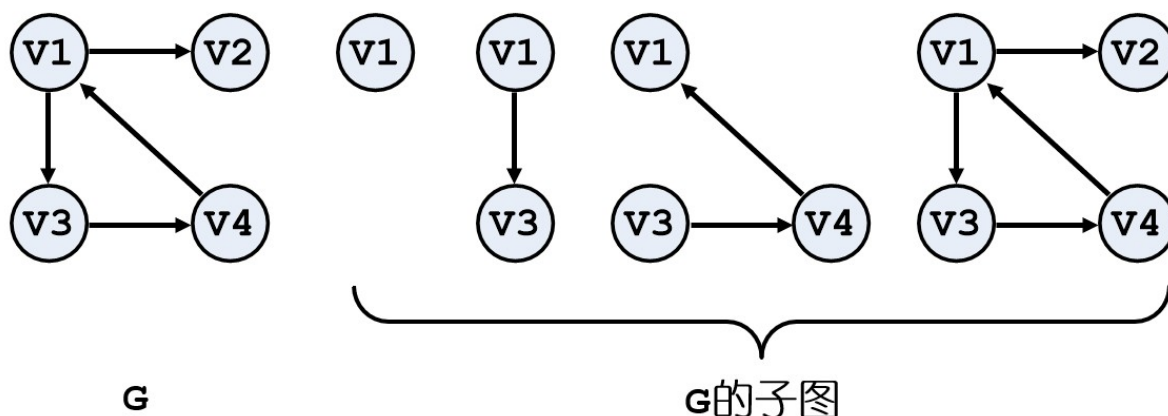
完全图 (Complete Graph): 若一个无向图中任意两个不同的顶点之间都有一条边, 则称该图为完全无向图, 记为 K_n , 其中 $n = |V|$ 。完全无向图的边数为 $\frac{n(n-1)}{2}$ 。例如, K_4 有 4 个顶点, 任意两点相连, 共有 6 条边。



类似地, 若有向图中任意两个不同的顶点 u, v 之间都存在两条方向相反的弧 $\langle u, v \rangle$ 和 $\langle v, u \rangle$, 则称该图为完全有向图, 其边数为 $n(n-1)$ 。

子图 (Subgraph): 给定两个图 $G = (V, E)$ 和 $G' = (V', E')$, 如果 $V' \subseteq V$ 且 $E' \subseteq E$, 则称 G' 是 G 的子图。若进一步有 $V' = V$, 则称 G' 为 G 的生成子图。

例如, 取 G_u 的顶点子集 $\{1, 2, 3\}$ 和边子集 $\{(1, 2), (2, 3)\}$, 得到的图是 G_u 的一个子图 (也是路径图)。



A.0.3 A.3 顶点的度

邻接点 (Adjacent Vertex): 在无向图中, 若存在边 (u, v) , 则称 u 和 v 互为邻接点。例如在 G_u 中, 顶点 1 的邻接点有 2 和 3。

在有向图中, 若存在弧 $\langle u, v \rangle$, 则称 v 是 u 的邻接点 (或 u 邻接到 v)。在 G_d 中, 顶点 1 邻接到 2 和 3, 但顶点 2 不邻接到 1。

度 (Degree): - 在无向图中, 顶点 v 的度是与其关联的边的数目, 记为 $\deg(v)$ 。在 G_u 中, $\deg(1) = 2$ (边 $(1,2)$ 和 $(1,3)$), $\deg(2) = 3$ (边 $(1,2), (2,3), (2,4)$), $\deg(3) = 3$, $\deg(4) = 2$, $\deg(5) = 2$ 。 - 在有向图中, 顶点 v 的度分为**入度 (In-degree)**和**出度 (Out-degree)**。入度是以 v 为终点的弧的数目, 记为 $\text{in}(v)$; 出度是以 v 为起点的弧的数目, 记为 $\text{out}(v)$ 。顶点的度等于入度与出度之和: $\deg(v) = \text{in}(v) + \text{out}(v)$ 。在 G_d 中: - 顶点 1: $\text{out}(1) = 2$ (到 2 和 3), $\text{in}(1) = 0$, $\deg(1) = 2$ 。 - 顶点 2: $\text{out}(2) = 1$ (到 4), $\text{in}(2) = 2$ (来自 1 和 3), $\deg(2) = 3$ 。 - 顶点 3: $\text{out}(3) = 1$ (到 2), $\text{in}(3) = 2$ (来自 1 和 4), $\deg(3) = 3$ 。 - 顶点 4: $\text{out}(4) = 1$ (到 3), $\text{in}(4) = 1$ (来自 2), $\deg(4) = 2$ 。

握手定理: 对于任意无向图 $G = (V, E)$, 所有顶点的度数之和等于边数的两倍:

$$\sum_{v \in V} \deg(v) = 2|E|.$$

验证 G_u : 度数之和 $2 + 3 + 3 + 2 + 2 = 12$, 边数 6, $2 \times 6 = 12$, 成立。该结论同样适用于有向图 (此时度数为入度与出度之和)。

A.0.4 A.4 路径与回路

路径 (Walk): 从顶点 u 到顶点 v 的一个顶点和边的交替序列, 其中每条边的两个端点恰好是序列中前后相邻的顶点。序列中允许重复顶点和重复边。

例如在 G_u 中, 路径 $1 - 2 - 3 - 2 - 4$ 是一个 walk (顶点 2 重复, 边 $(2,3)$ 和 $(3,2)$ 虽然是无向边但实际是同一无向边? 注意在无向图中边无方向, 所以边 $(2,3)$ 只出现一次, 但这里写法可能引起歧义。更合适的例子: 在有向图 G_d 中, walk $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 2$ 是合法的, 因为边 $\langle 1, 2 \rangle, \langle 2, 4 \rangle, \langle 4, 3 \rangle, \langle 3, 2 \rangle$ 都存在。

迹 (Trail): 边不重复的路径 (顶点可以重复)。例如在 G_u 中, $1 - 2 - 4 - 5 - 3$ 所有边均不重复 (边依次为 $(1,2), (2,4), (4,5), (5,3)$), 是一个 trail。

简单路径 (Path): 顶点和边均不重复的路径。通常简称为“路径”。例如在 G_u 中, $1-2-4-5$ 是一个简单路径, 因为顶点序列 $1,2,4,5$ 没有重复。

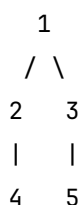
回路 (Circuit): 起点和终点相同的迹 (边不重复, 顶点可重复)。例如在 G_d 中, $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 2$ 的终点是 2, 不是回路。需要起点终点相同, 如 $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$ 检查边是否重复? 有重复边? 实际上这个序列中边 $\langle 2, 1 \rangle$ 不存在。更好的例子: 在 G_u 中, $1-2-3-1$ 是一个回路 (边 $(1,2), (2,3), (3,1)$ 均存在且不重复)。

简单回路 (Cycle): 起点和终点相同且没有重复顶点 (除起点与终点重合外) 的回路。简单回路也称为**环**。例如 G_u 中的 $1-2-3-1$ 是一个简单回路 (环), 因为顶点 $1,2,3$ 互异, 边也是三条。

A.0.5 A.5 树

树 (Tree): 一个连通且无简单回路的无向图称为树。

例: 考虑图 $T = (V, E)$, 其中 $V = \{1, 2, 3, 4, 5\}$, $E = \{(1, 2), (1, 3), (2, 4), (3, 5)\}$ 。其结构如下:



这是一个连通无回路的图, 因此是一棵树。它有 5 个顶点, 4 条边, 符合 $|E| = |V| - 1$ 。

树具有许多重要性质, 以下是几个常用的等价刻画 (定理):

1. **唯一路径性**: 树中任意两个顶点之间有且仅有一条简单路径。

证明思路: 连通性保证存在路径; 若存在两条不同路径, 则它们会构成一个回路, 矛盾。在示例树中, 顶点 4 到 5 的唯一路径是 $4-2-1-3-5$ 。

2. **边数与顶点数的关系**: 设树有 n 个顶点, 则其边数为 $n - 1$ 。

证明: 可用数学归纳法对顶点数归纳。当 $n = 1$ 时边数为 0; 假设对 $n - 1$ 成立, 考虑一棵 n 个顶点的树, 删除一条边将树分成两个连通分支, 每个分支是树, 由归纳假设可得总边数为 $(n_1 - 1) + (n_2 - 1) + 1 = n - 1$ 。

3. **叶子节点个数**: 任何一棵非平凡树 ($n \geq 2$) 至少有两个叶子节点 (度为 1 的顶点)。

证明: 利用握手定理 $2(n-1) = \sum \deg(v_i)$, 假设叶子节点数为 n_1 , 则 $\sum \deg(v_i) \geq n_1 + 2(n - n_1) = 2n - n_1$, 从而 $2n - 2 \geq 2n - n_1$, 即 $n_1 \geq 2$ 。示例树中, 叶子节点为 4 和 5, 恰好两个。

4. **定理 (树的等价刻画)**: 对于 n 个顶点的无向图 G , 以下三个条件中的任意两个成立时, G 必为树, 且第三个也成立:

- (1) G 是连通的;
- (2) G 无回路;
- (3) $|E| = |V| - 1$ 。

证明:

- **若 (1) 和 (2) 成立**: 由树的定义, G 本身就是一棵树。根据树的边数性质 (定理 2), 必有 $|E| = |V| - 1$, 即 (3) 成立。

- **若 (1) 和 (3) 成立:** 已知 G 连通且边数为 $|V| - 1$ 。连通图至少包含一棵生成树, 生成树的边数恰好为 $|V| - 1$ 。由于 G 的边数等于这个最小值, 因此 G 本身必定是一棵生成树, 从而无回路, 即 (2) 成立。
- **若 (2) 和 (3) 成立:** 已知 G 无回路且边数为 $|V| - 1$ 。假设 G 不连通, 则它可分解为 k ($k \geq 2$) 个连通分量。每个连通分量是无回路的连通图, 因此都是树。设第 i 个分量的顶点数为 n_i , 边数为 $n_i - 1$ 。则总顶点数 $n = \sum_{i=1}^k n_i$, 总边数 $e = \sum_{i=1}^k (n_i - 1) = n - k$ 。由已知 $e = n - 1$, 得 $k = 1$, 与假设矛盾。故 G 连通, 即 (1) 成立。

综上, 三个条件中任意两个成立均可推出第三个, 且 G 为树。□

生成树 (Spanning Tree): 若图 G 是连通的, 则 G 的生成子图如果是一棵树, 就称为 G 的一棵生成树。生成树保留了原图的所有顶点, 但边数恰好为 $|V| - 1$ 。第 3 章讨论的**最小生成树**就是所有生成树中边权总和最小的那棵。例如, 对于 G_u , 可以删除一条边得到一棵生成树, 比如删除边 (2,3) 得到树 T (仍连通且无回路)。

